资源环境绩效评估方法：DEA模型与Python应用

目录

资源环境绩效评估方法：DEA模型与Python应用 **1**

1 绪论 **3**

1.1 全球资源环境危机与评估需求 . . . 3
1.2 可持续发展的核心要义与多维度挑战 . . . 4
1.3 资源环境绩效评估的必要性与复杂性 . . . 4
1.4 传统评估方法的局限性与数据包络分析 . . . 5
1.4.1 传统评估方法的不足 . . . 5
1.4.2 数据包络分析（DEA）的核心原理与优势 . . . 5
1.4.3 DEA应用的考量与挑战 . . . 6
1.5 Python在资源环境绩效评估中的应用优势 . . . 7
1.5.1 Python的易用性与广泛适用性 . . . 7
1.5.2 强大的科学计算与数据处理生态系统 . . . 7
1.5.3 Python助力DEA模型实现与调试 . . . 8
1.6 一个贯穿全书的问题：投入多是否意味着效率低 . . . 8
1.7 本书结构与内容概览 . . . 9
1.7.1 章节安排与知识体系 . . . 9
1.7.2 理论与实践的融合 . . . 10
1.8 本书的读者对象与价值 . . . 10
1.8.1 目标读者 . . . 10
1.8.2 本书的独特价值 . . . 11
参考文献 . . . 11

2 Python基础知识 **15**

2.1 Python的安装与环境配置 . . . 15
2.1.1 Anaconda下载与安装 . . . 15
2.1.2 安装DEA相关Python包 . . . 17
2.2 开发环境选择与Jupyter Notebook最佳实践 . . . 19
2.2.1 Jupyter Notebook基础 . . . 19
2.2.2 Jupyter Notebook最佳实践（DEA编程场景） . . . 20
2.2.3 其他IDE选择：VS Code与PyCharm . . . 21

2.3 Python语法精粹 . . . 21
2.3.1 缩进与注释 . . . 21
2.3.2 代码换行 . . . 22
2.3.3 基本数据类型 . . . 23
2.3.4 列表与字典（DEA场景） . . . 23
2.3.5 控制结构 . . . 24
2.3.6 函数定义与调用 . . . 25
2.3.7 模块导入 . . . 26
2.4 面向DEA的Python编程范式 . . . 26
2.4.1 向量化思维：告别Python循环 . . . 26
2.4.2 函数封装：从脚本到可复用工具 . . . 27
2.4.3 矩阵构建：用NumPy构造DEA数据矩阵 . . . 30
2.5 错误处理与调试入门 . . . 31
2.5.1 异常处理：try-except-finally . . . 31
2.5.2 DEA编程常见错误排查表 . . . 33
2.5.3 防御性编程：输入数据检查 . . . 34
2.6 NumPy模块及其在DEA中的应用 . . . 35
2.6.1 NumPy模块的导入 . . . 35
2.6.2 ndarray数组的创建 . . . 36
2.6.3 ndarray数组的索引与切片（DEA场景） . . . 36
2.6.4 ndarray数组的属性与矩阵运算 . . . 37
2.7 pandas模块及其在DEA中的应用 . . . 37
2.7.1 pandas模块的导入 . . . 38
2.7.2 Series数据结构 . . . 38
2.7.3 DataFrame数据结构 . . . 38
2.7.4 DEA面板数据处理 . . . 39
2.7.5 使用pandas读写数据 . . . 40
2.7.6 loc与iloc索引（DEA关键操作） . . . 40
2.8 deabook 模块组成 . . . 41
2.8.1 模块组成一览表 . . . 41
2.8.2 在书中如何导入 . . . 43
2.9 从 Python 到 deabook：DEA 程序标准工作流（预览） . . . 44
2.10 案例应用：读取并检查美国电厂数据 . . . 45
2.11 本章小结 . . . 46

3 数据包络分析的经典模型 47
3.1 生产技术的设定 . . . 48
3.1.1 技术集 . . . 48
3.1.2 产出集 . . . 48
3.1.3 投入集 . . . 49

3.2 效率的测量 . . . 49
3.2.1 产出距离函数 . . . 49
3.2.2 投入距离函数 . . . 50
3.2.3 同时缩小投入和扩张产出的 Hyperbolic 距离函数 . . . 52
3.2.4 方向距离函数 . . . 52
3.3 DEA 模型的估计 . . . 54
3.3.1 基于产出距离函数的技术效率估计 . . . 54
3.3.2 基于投入距离函数的技术效率估计 . . . 55
3.3.3 基于双曲距离函数的技术效率估计 . . . 56
3.3.4 基于方向距离函数的技术效率估计 . . . 57
3.4 面板数据的技术参照集构造 . . . 58
3.4.1 为什么需要构造技术参照集 . . . 59
3.4.2 参照集与参照技术的一般形式 . . . 59
3.4.3 当期 DEA 模型 . . . 59
3.4.4 Window DEA 模型 . . . 60
3.4.5 时序 DEA 模型 . . . 60
3.4.6 全局 DEA 模型 . . . 61
3.5 经典DEA模型的Python实现 . . . 62
3.5.1 模块的导入与依赖 . . . 63
3.5.2 五类效率测度的统一接口 . . . 63
3.5.3 参数语义与规模报酬设定 . . . 64
3.5.4 返回结构与效率列的经济含义 . . . 65
3.5.5 调用示例与实证研究的衔接 . . . 66
3.6 案例应用：美国燃煤电厂的技术效率 . . . 68
3.7 本章小结 . . . 69
参考文献 . . . 69

4 全要素生产率测度方法 73
4.1 经典DEA模型的生产率指数 . . . 74
4.1.1 谢泼德距离函数对应的Malmquist指数 . . . 74
4.1.2 基于方向距离函数的Malmquist指数 . . . 78
4.2 生产率测度的Python实现 . . . 80
4.2.1 接口导入与模块组织 . . . 80
4.2.2 模型参数与参考集设定 . . . 81
4.2.3 返回值的经济含义 . . . 82
4.2.4 调用示例与约定 . . . 85
4.2.5 生产率分解：`DEA.mqfgnz`、`DEA.mqrd`、`DEA.gmqdecom` 及其 DDF 变体 . . . 88
4.2.6 三要素分解接口的参数、返回值与调用示例 . . . 88
4.2.7 与 `DEA.mq` 的等价关系 . . . 92
4.3 案例应用：中国火电行业的发电生产率 . . . 93

4.4 本章小结 . . . 94
参考文献 . . . 94

5 能源效率与能源生产率测度方法 97
5.1 投入距离函数 . . . 97
5.1.1 谢泼德投入距离函数模型 . . . 97
5.1.2 短板效应 . . . 98
5.2 谢泼德能源距离函数模型 . . . 99
5.3 方向距离函数模型 . . . 100
5.3.1 只调整能源一个方向 . . . 100
5.3.2 能源缩减与产出扩张同时进行 . . . 100
5.4 能源生产率 . . . 101
5.4.1 基于能源专项谢泼德距离函数的能源生产率 . . . 102
5.4.2 基于能源专项方向距离函数的能源生产率 . . . 103
5.5 能源效率与能源生产率模型的Python实现 . . . 103
5.5.1 数据接口约定 . . . 104
5.5.2 测度口径与方向向量设定 . . . 105
5.5.3 返回结果与指标解读 . . . 106
5.5.4 调用示例与理论公式的对应 . . . 106
5.6 案例应用：中国省级能源效率与生产率 . . . 107
5.7 本章小结 . . . 109
参考文献 . . . 109

6 环境绩效测度方法 111
6.1 生产技术 . . . 111
6.1.1 规模报酬性质 . . . 112
6.1.2 生产技术的参数刻画 . . . 113
6.2 绩效测度 . . . 115
6.2.1 谢泼德距离函数 . . . 116
6.2.2 双曲距离函数 . . . 122
6.2.3 方向距离函数 . . . 127
6.2.4 非径向方向距离函数 . . . 133
6.3 环境绩效测度模型的Python实现 . . . 140
6.3.1 非期望产出的声明与参数框架 . . . 140
6.3.2 方向设定与规模报酬 . . . 142
6.3.3 返回指标及其效率换算 . . . 142
6.3.4 四类距离函数的可运行示例 . . . 144
6.4 案例应用：OECD国家的碳排放效率 . . . 146
6.5 本章小结 . . . 148
参考文献 . . . 148

7 包含非期望产出的生产率 149
7.1 Malmquist 生产率指数 . . . 149
7.1.1 期望产出方向 . . . 149
7.1.2 投入方向 . . . 151
7.2 基于方向距离函数的全要素生产率指数 . . . 152
7.2.1 期望产出和非期望产出方向同时调整 . . . 152
7.2.2 投入方向 . . . 154
7.2.3 投入、期望产出、非期望产出同时调整 . . . 155
7.3 基于非径向方向距离函数的全要素生产率指标 . . . 156
7.3.1 期望产出和非期望产出方向同时调整 . . . 156
7.3.2 投入方向 . . . 157
7.3.3 投入、期望产出、非期望产出同时调整 . . . 157
7.4 动态环境绩效 . . . 158
7.4.1 基于非期望产出导向谢泼德距离函数的动态环境绩效 . . . 158
7.4.2 基于非期望产出导向方向距离函数的动态环境绩效 . . . 159
7.5 包含非期望产出生产率模型的Python实现 . . . 160
7.5.1 面板数据的样本结构与变量设定 . . . 161
7.5.2 方向向量、技术参照集与环境绩效分解口径 . . . 162
7.5.3 理论指标与测算返回列的口径对应 . . . 163
7.5.4 测算接口的调用与结果获取 . . . 165
7.6 案例应用：OECD动态环境绩效 . . . 167
7.7 本章小结 . . . 168
参考文献 . . . 169

8 基于松弛测度的效率模型 171
8.1 径向效率测度中的松弛问题 . . . 172
8.2 生产技术与符号设定 . . . 172
8.3 不含非期望产出的 SBM 模型 . . . 173
8.3.1 投入导向 SBM . . . 173
8.3.2 产出导向 SBM . . . 174
8.3.3 非导向 SBM . . . 174
8.3.4 非导向 SBM 的线性化 . . . 175
8.4 含非期望产出的 SBM 模型 . . . 175
8.4.1 含非期望产出的综合非导向 SBM . . . 176
8.4.2 含非期望产出的线性化 . . . 176
8.4.3 投入导向环境 SBM . . . 177
8.4.4 期望产出导向环境 SBM . . . 177
8.4.5 非期望产出导向 SBM . . . 178
8.5 基于方向距离函数的松弛测度（SD） . . . 179
8.5.1 不含非期望产出的 SD：Färe 与Grosskopf（2010） . . . 179

8.5.2 含非期望产出的 SD：Pham 与Zelenyuk（2018） . . . 180
8.6 SBM 模型的Python实现 . . . 181
8.6.1 接口模块与变量结构 . . . 181
8.6.2 方向向量对效率口径的编码 . . . 182
8.6.3 求解结果的解释口径 . . . 184
8.6.4 与理论模型的调用对应 . . . 185
8.6.5 SD 模型的Python实现 . . . 187
8.7 案例应用：美国电厂的投入冗余与发电不足 . . . 188
8.8 本章小结 . . . 189
参考文献 . . . 189

9 基于物质平衡的环境绩效测度方法 191
9.1 物质平衡原则 . . . 191
9.2 物质平衡条件 . . . 192
9.2.1 距离函数与物质平衡条件 . . . 192
9.2.2 符合物质平衡的生产技术的公理 . . . 193
9.3 物质平衡生产技术的非参数刻画 . . . 194
9.4 绩效测度 . . . 195
9.4.1 非期望产出方向上的技术效率 . . . 195
9.4.2 投入方向上的技术效率 . . . 200
9.4.3 期望产出方向上的技术效率 . . . 201
9.4.4 混合方向上的技术效率 . . . 201
9.5 物质平衡条件的发展：含末端减排活动的扩展 . . . 202
9.6 物质平衡模型的Python实现 . . . 203
9.6.1 物质流声明与模型接口 . . . 203
9.6.2 分解层级及其效率含义 . . . 204
9.6.3 返回结果与后处理指标 . . . 206
9.6.4 多层级求解的依次调用 . . . 207
9.7 案例应用：中国火电行业的物质平衡分解 . . . 208
9.8 本章小结 . . . 209
参考文献 . . . 209

10 随机非参数数据包络分析 211
10.1 单产出模型设定 . . . 212
10.1.1 凸非参数最小二乘法 . . . 212
10.1.2 确定性前沿 . . . 213
10.1.3 随机非参数(StoNED)模型 . . . 215
10.1.4 乘法模型 . . . 218
10.1.5 背景变量 . . . 219
10.1.6 凸非参数分位数回归 . . . 220

10.2 多产出StoNED模型的设定 . . . 221
10.3 有非期望产出的StoNED模型的设定 . . . 223
10.3.1 有背景变量和非期望产出的StoNED模型 . . . 226
10.3.2 有非期望产出的凸非参数分位数回归 . . . 227
10.3.3 基于 Shephard 距离函数的弱可处置 StoNED 模型 . . . 228
10.4 计算问题 . . . 229
10.5 随机非参数前沿模型的Python实现 . . . 231
10.5.1 调用前的数据与模型设定 . . . 231
10.5.2 从 CNLS 前沿到 StoNED 效率 . . . 233
10.5.3 两类前沿的调用结构 . . . 236
10.6 案例应用：芬兰配电企业的随机成本前沿 . . . 237
10.7 本章小结 . . . 239
参考文献 . . . 239

11 DEA模型编程的基本思路 241
11.1 DEA模型求解的程序化思路 . . . 241
11.2 单个决策单元规划问题的建模映射 . . . 242
11.2.1 决策变量的定义 . . . 243
11.2.2 目标函数的构造 . . . 244
11.2.3 约束条件的定义 . . . 244
11.2.4 模型求解 . . . 246
11.2.5 优化结果的提取 . . . 246
11.3 效率评价的循环迭代 . . . 248
11.4 封装为可复用函数 . . . 250
11.4.1 dea1：基础函数 . . . 250
11.4.2 dea2：支持pandas DataFrame . . . 252
11.4.3 dea3：支持查询条件指定评价单元 . . . 254
11.4.4 dea4：支持分别指定评价单元和参照单元 . . . 257
11.4.5 dea5：保留原始索引 . . . 259
11.4.6 dea6：支持公式字符串 . . . 261
11.4.7 dea7：支持CRS/VRS切换 . . . 264
11.4.8 dea8：支持投入导向/产出导向 . . . 267
11.5 代码重构：用类封装DEA模型 . . . 271
11.5.1 异常处理：让代码更健壮 . . . 271
11.5.2 DEAModel类 . . . 272
11.5.3 使用DEAModel类 . . . 277
11.6 常见错误与调试技巧 . . . 278
11.7 案例应用：把单个电厂求解扩展为循环 . . . 279
11.8 本章小结 . . . 280
11.8.1 从自定义函数到deabook包 . . . 281

11.8.2 变量命名对照表 281

12 DEA程序编写的实战演练 283

12.1 SBM模型的程序编写 283
12.2 包含非期望产出的SBM模型的程序编写 289
12.3 公共工具模块的建立 294
12.3.1 从代码重复到模块化的动机 294
12.3.2 公共导入约定 295
12.3.3 公式字符串的解析 296
12.3.4 数据验证与准备 297
12.3.5 求解与结果格式化 299
12.3.6 求解器的选择 302
12.4 DDF模型的程序编写 302
12.4.1 从SBM到DDF：模型逻辑的转变 303
12.5 NDDF模型的程序编写 308
12.5.1 从DDF到NDDF：非径向化的设计哲学 308
12.5.2 结果解读与使用建议 313
12.6 Malmquist生产率指数的程序编写 313
12.6.1 Malmquist指数的基本原理 313
12.6.2 Malmquist分解 314
12.6.3 从距离函数到代码实现：Malmquist计算的整体架构 315
12.6.4 基础DEA函数 315
12.6.5 dea8函数的设计解析 318
12.6.6 产出导向的Malmquist指数 318
12.6.7 mpi函数的深度解析：从理论公式到代码实现 322
12.6.8 Malmquist-Luenberger生产率指数 325
12.6.9 从Malmquist到ML：DDF替换的设计考量 325
12.6.10 ML与MPI的代码对比总结 328
12.7 结果验证与敏感性分析 329
12.7.1 为什么需要专门的结果验证？ 329
12.7.2 效率值合理性检查 330
12.7.3 合理性检查的设计依据 331
12.7.4 与deabook包的交叉验证 331
12.7.5 交叉验证策略的选择 332
12.7.6 敏感性分析示例 333
12.7.7 敏感性分析的方法论解读 335
12.8 从脚本到模块：代码组织进阶 336
12.8.1 为什么需要从脚本走向模块？ 336
12.8.2 推荐项目结构 336
12.8.3 项目结构的设计哲学 337

12.8.4 完整模块代码 . . . 337
12.8.5 使用示例 . . . 341
12.8.6 函数式设计与面向对象设计的权衡 . . . 341
12.8.7 单元测试示例 . . . 343
12.8.8 常见错误排查速查表 . . . 344
12.9 案例应用：同一省级样本的程序核验 . . . 345
12.10本章小结 . . . 345
参考文献 . . . 346

13 基于DEA对偶问题的影子价格估计 349
13.1 线性规划的对偶问题 . . . 350
13.1.1 从经济学直觉出发：资源定价视角 . . . 350
13.1.2 DEA的两种形式：包络形式与乘数形式 . . . 351
13.1.3 线性规划对偶的数学推导 . . . 351
13.1.4 对偶转换的一般规则 . . . 353
13.1.5 强对偶定理与互补松弛条件 . . . 354
13.2 SBM模型的对偶问题 . . . 354
13.3 基于SBM模型对偶问题的影子价格计算程序 . . . 356
13.3.1 数值注意事项与数据预处理 . . . 356
13.3.2 单个DMU的影子价格计算 . . . 356
13.3.3 迭代求解所有DMU . . . 360
13.3.4 面向复用的函数封装 . . . 362
13.3.5 支持样本和参照集限定 . . . 366
13.3.6 支持公式字符串调用 . . . 370
13.4 影子价格的合理解释范围 . . . 373
13.4.1 影子价格的经济含义 . . . 373
13.4.2 异常值的识别与处理 . . . 374
13.4.3 退化问题与处理建议 . . . 375
13.4.4 在同一省级样本中检查退化 . . . 375
13.5 NDDF模型的对偶问题 . . . 376
13.5.1 NDDF模型的原始形式 . . . 376
13.6 基于NDDF模型对偶问题的影子价格计算程序 . . . 377
13.6.1 单个DMU的影子价格计算 . . . 377
13.6.2 迭代求解所有DMU . . . 380
13.6.3 面向复用的函数封装 . . . 382
13.6.4 高级版本：支持方向向量、权重和查询条件 . . . 385
13.7 不同模型设定下的影子价格差异 . . . 390
13.7.1 CRS与VRS的影响 . . . 390
13.7.2 投入导向与产出导向的影响 . . . 391
13.7.3 模型选择决策树 . . . 391

13.8 案例应用：中国省级排放约束的边际价值 . . . 392
13.9 影子价格的重抽样诊断与统计推断 . . . 394
13.9.1 为什么需要置信区间？ . . . 394
13.9.2 重抽样敏感性诊断及其边界 . . . 394
13.9.3 Python实现：样本中位数的重抽样诊断 . . . 394
13.9.4 Bootstrap在DEA中的特殊性 . . . 395
13.10 数值稳定性与求解器选择 . . . 395
13.10.1 约束缩放 . . . 395
13.10.2 求解器选择与容差设置 . . . 397
13.10.3 退化检测与处理 . . . 397
13.11 进一步阅读与前沿方法 . . . 398
13.11.1 超效率DEA的影子价格 . . . 398
13.11.2 网络DEA的影子价格 . . . 399
13.11.3 DEA与SFA影子价格的对比 . . . 399
13.11.4 StoNED的影子价格 . . . 399
13.12 本章小结 . . . 399
参考文献 . . . 400

Appendices **401**

A 附录. Python编写的DEA程序的调试 **401**
A.1 A.1 开发环境配置指南 . . . 402
A.1.1 A.1.1 Python环境管理 . . . 402
A.1.2 A.1.2 核心包安装 . . . 402
A.1.3 A.1.3 求解器安装详细指南 . . . 403
A.1.4 A.1.4 验证安装 . . . 406
A.1.5 A.1.5 跨平台注意事项 . . . 408
A.2 A.2 Python基础错误速查 . . . 408
A.2.1 A.2.1 常见错误速查表 . . . 408
A.2.2 A.2.2 包导入错误的DEA场景 . . . 409
A.2.3 A.2.3 使用pathlib处理文件路径 . . . 410
A.3 A.3 Pyomo与DEA特有错误排查 . . . 410
A.3.1 A.3.1 DEA程序调试决策树 . . . 410
A.3.2 A.3.2 最小可复现示例（MRE）指南 . . . 411
A.3.3 A.3.3 模型构建阶段错误 . . . 413
A.3.4 A.3.4 数据相关错误 . . . 422
A.3.5 A.3.5 求解阶段错误 . . . 423
A.3.6 A.3.6 结果相关错误 . . . 425
A.4 A.4 Jupyter Notebook特有问题 . . . 426
A.4.1 A.4.1 单元执行顺序混乱 . . . 426

A.4.2 A.4.2 内核重启后变量丢失 . . . 426
A.4.3 A.4.3 大数据集的内存问题 . . . 427
A.4.4 A.4.4 图形不显示 . . . 428
A.5 A.5 性能优化与大规模问题处理 . . . 428
A.5.1 A.5.1 DEA计算复杂度分析 . . . 428
A.5.2 A.5.2 提升求解速度的实用技巧 . . . 429
A.5.3 A.5.3 内存优化建议 . . . 434
A.5.4 A.5.4 性能优化决策流程 . . . 434
A.6 A.6 防御式编程与单元测试 . . . 435
A.6.1 A.6.1 输入验证函数 . . . 435
A.6.2 A.6.2 断言（Assertion）的使用 . . . 438
A.6.3 A.6.3 单元测试入门 . . . 439
A.7 A.7 环境可复现性与版本管理 . . . 445
A.7.1 A.7.1 为什么需要版本管理 . . . 445
A.7.2 A.7.2 导出环境配置 . . . 445
A.7.3 A.7.3 本书推荐的environment.yml . . . 446
A.7.4 A.7.4 在代码中记录版本信息 . . . 447
A.8 A.8 获取帮助的渠道 . . . 447
A.8.1 A.8.1 在线社区与资源 . . . 447
A.8.2 A.8.2 提问的最佳实践 . . . 447
A.8.3 A.8.3 学术资源 . . . 448
A.9 附录末：运行DEA程序前的检查清单 . . . 449
A.10 术语表 . . . 449

资源环境绩效评估方法：DEA模型与Python应用

本书系统介绍如何运用数据包络分析（Data Envelopment Analysis，DEA）与Python 编程语言，构建资源环境绩效评估的理论框架与实操工具，为研究人员、政策制定者和从业者提供一套科学、可复现的评估方法。

全书共分为三个部分：

- **第一部分（第 2 章）**：Python 编程基础。
- **第二部分（第 3—10 章）**：DEA 理论与方法。
- **第三部分（第 11—13 章及附录）**：DEA 程序实现与应用拓展。

第 1 章

绪论

当前，全球正面临严峻的资源环境约束：全球变暖、生物多样性丧失、水资源短缺、关键矿产资源耗竭与各类污染相互叠加。从极端气候事件到生态系统功能退化，从能源资源的紧张约束到污染排放的持续累积，这些问题彼此交织，对经济社会的长期发展构成实质性压力。如何在有限的资源储量与环境容量约束下维持经济增长、增进社会福利，已成为各国政府、产业界与学术界共同面对的重要问题。回答这一问题的前提，是科学地评估“资源利用是否高效、环境代价是否可控、发展方式是否可持续”——这正是资源环境绩效评估所要处理的核心议题，也是本书的出发点。

1.1 全球资源环境危机与评估需求

在气候变化领域，政府间气候变化专门委员会（IPCC）第六次评估报告综合报告指出：2011—2020年全球地表温度较1850—1900年约升高1.1°C；大气、海洋、冰冻圈和生物圈均已发生广泛而迅速的变化，气候变化正在加剧全球各地区的极端天气气候事件，给自然生态系统与人类社会造成广泛的不利影响、损失与损害（IPCC, 2023）。世界气象组织《2024年全球气候状况》进一步显示，2024年全球平均海平面（卫星观测记录）与海洋热含量均再创新高（WMO, 2025）。

气候系统还存在临界风险。一旦某些子系统越过临界点，其状态可能发生难以迅速逆转的转变（IPCC, 2023）。Lenton 等（2008）在政策相关尺度上给出了9个气候临界要素的短名单，并另列若干未进入短名单的候选系统。Lenton 等（2019）的评论指出，这类临界过程的发生概率、影响及其相互关联可能高于以往的通行判断，并以亚马孙雨林、北极海冰、格陵兰冰原等为例发出警告，但并未对全部临界要素作出“已经触发”的系统判定。后续研究发现，地球系统中若干临界要素之间存在遥相关，某一要素的变化可能提高其他要素越过临界点的风险，从而形成级联（Liu 等, 2023）。

气候危机并非孤立发生。Ceballos 等（2015）指出，当代物种灭绝速度已远超自然背景速率，地球可能正在经历第六次物种大灭绝。Ceballos 和Ehrlich （2023）进一步估计，在栖息地破坏、过度捕捞和狩猎等人类活动影响下，自公元1500年以来已有73个陆生脊椎动物属（不含鱼类）灭绝；若按过去百万年的背景灭绝速率，这一规模的属丧失大约需要18,000年才会发生。

生态系统退化与气候变化相互叠加，也反映在资源消耗与污染上。全球城市固体废物规模已达数十亿吨量级，并仍呈增长趋势（Kaza 等, 2018）；空气污染造成的过早死亡会带来以万亿美元计的福利损失（World Bank, 2016）。上述问题并非彼此孤立：人口、资源、环境与经济子系统之间存在复杂相互作用，任一

子系统的持续失衡都可能外溢至其他子系统。

在此背景下，对资源环境绩效进行科学、系统且可比的评估，有助于识别风险、追踪进展并讨论资源配置。评估本身并不唯一对应于某一种方法。本书选择数据包络分析（Data Envelopment Analysis，DEA）与Python，是因为前者能够在缺乏价格信息的条件下处理多投入、多产出（含环境结果）的相对效率问题，后者则便于把模型写成可复现的计算流程。二者构成全书的方法主线，而非对其他评估工具的替代性否定。

1.2 可持续发展的核心要义与多维度挑战

可持续发展的经典表述来自世界环境与发展委员会：既满足当代人的需要，又不对后代人满足其需要的能力构成危害（WCED, 1987）。经济、环境与社会维度需要统筹，但“最大化代际整体福利”并不是上述定义的原文，也不应写成已经形成共识的操作性公式。自联合国2030年议程及可持续发展目标（SDGs）获193个成员国通过以来，全球在监测进展方面投入了大量努力，实际进展却并不均衡。Fuso Nerini 等（2024）指出，全球 SDGs 进展已明显偏离2030年轨道，大量指标停滞甚至倒退。《自然》杂志社论在议程过半时写道：17项目标中可能无一能如期完全实现，在当时可评估的靶标中仅约12%走在正轨（Nature Editorial, 2023）。联合国《2024年可持续发展目标报告》更新后的判断是：在可评估靶标中，仅约17%进展足以在2030年达标，近半数偏离轨迹，超过三分之一停滞或相对2015年基线倒退（United Nations, 2024）。Xing 等（2025）基于多国指标时间序列的预测则表明，若按既有轨迹外推，全球 SDG 综合得分到2030年大约只能达到63%左右；若要在综合得分意义上接近总体目标，各国SDG 得分需达到约4%的年均增速。后一组数字来自学术预测，用来补充而非代替联合国官方进度评估。

可持续发展评估还面临方法论困难。常见做法可大致对应三种理论视角：三支柱视角强调经济、环境与社会维度的协调（United Nations, 2005），覆盖面宽，但容易遇到量纲不一、权重外生和可比性不足的问题；生态视角更关注人类活动对资源和环境系统的压力，接近强可持续性立场（Daly, 1990），直观但口径较窄，数据约束也更强；资本视角以人造资本、自然资本等存量变化衡量可持续性（Pearce 和 Atkinson, 1993），便于加总比较，却高度依赖货币化与替代性假设。若把这些维度拆开、用单一比率或主观加权指数分别打分，结论容易片面。

在需要同时处理多投入、多产出且往往缺乏可靠价格信息时，数据包络分析（DEA）作为一种基于线性规划的非参数效率前沿方法，因此在环境与可持续性评估中得到持续使用。DEA 不必预先设定生产函数形式，也不必外生给定全部权重，能够识别样本内的相对最佳实践。Song 等（2012）综述了 DEA 在环境效率评价中的应用，覆盖径向与非径向、单期与多期等方向；Patel 等（2025）则从文献计量角度梳理了 DEA 用于企业可持续性测量的研究脉络。需要强调的是：这些优势成立的前提是投入产出变量已经给定；变量选择本身仍是研究设定，不能由DEA 自动证明为“正确”。

1.3 资源环境绩效评估的必要性与复杂性

第一，没有可用的度量，管理就缺少可核对的依据。生态环境政策绩效评估是政策周期中的重要环节，用来反映实施效果，为调整、优化或退出提供证据，而不是代替政治决策本身。

第二，评估有助于识别相对较好的实践、追踪改善并讨论资源配置。面对资源约束、环境污染和生态系统退化，只有对不同地区、行业或企业的资源利用效率与环境绩效作口径清楚的量化比较，才可能指出效率较低的环节，并讨论把有限资源投向何处。这里的“最佳实践”是相对于所设技术与样本而言的，不宜直接等同于因果意义上的政策成功。

第三，评估可以服务于可持续发展目标的监测，但二者并不等同。DEA 已被用于生态效率评价及相关指标体系研究。现有文献显示，网络 DEA、动态DEA、 DEA-Malmquist 以及与机器学习结合的混合方法，常被用来讨论能源系统与碳减排中的相对效率问题（Song 等, 2012; Patel 等, 2025）。这些工作提供的是样本内相对效率框架，不能自动回答某一国家或地区是否“实现了SDG”。

资源环境绩效评估的复杂性，来自人口、资源、环境与经济子系统之间的耦合。当前SDG 全球指标框架的设计也存在明显张力：偏重数据供给而相对忽视实施问题，指标内容存在缺口，政治目标与测量内容可能脱节，部分指标甚至会因其测量方式改写目标本身，从而削弱2030议程的实施效果（Koerber, 2024）。SDG 指标还普遍面临数据不足、指标冗余、覆盖不完整以及与地方情境和优先事项错位等挑战（Sato, 2025）。如果评估工具不能说明低效率相对于何种技术、何种样本、何种导向而言，政策讨论就容易偏离可检验的对象。本书关注的，正是一套口径可陈述、步骤可复现的绩效评估工具。

1.4 传统评估方法的局限性与数据包络分析

在资源环境绩效评估中，单一比率和主观加权指数往往难以同时刻画多投入与多产出。DEA 最初是运筹学中的相对效率测量方法，由Farrell（1957）的效率概念发展而来，经 Charnes 等（1978）形式化为可计算的线性规划模型，其后才被广泛引入环境与能源绩效研究。它并非为应对当代全球环境危机而“应运而生”，但其多投入多产出结构与无需求价格的特点，使它适合作为该领域的一种评估工具。本节先说明传统方法的不足，再概述DEA 的原理、优势与应用中的限制。

1.4.1 传统评估方法的不足

传统绩效评估常依赖单一比率（如单位投入的产出）或简单加总指标。这类指标直观，但把多种投入、多种产出压缩到一个比例上，无法描述要素之间的替代与联合生产。即便列出产值、利润、资金、设备等多个可量化项目，若仍停留在分项比较，也得不到一个口径一致的综合效率测度。至于自然条件、制度环境等难以观测或难以进入生产账户的因素，传统比率处理不了，标准DEA 同样不能把未观测因素自动纳入前沿；正确的做法是在模型外讨论它们对结果解释的影响，而不是宣称某一种方法已经克服了不可测问题。

许多多准则方法还需要事先为各指标指定权重。权重若来自经验或专家打分，就可能偏离真实的运营约束，也难以反映非线性关系。例如，若在地区能源效率评价中把经济产出权重设得过高、环境代价过低，排序就会被设定本身主导。Krivonozhko 等（2011）从数学上指出，比率分析相当于把多维观测反复投影到不同子空间，可能显著扭曲绩效比较。这一结果为改用前沿方法提供了形式依据，而不是证明DEA 没有设定问题。

此外，单一综合得分往往缺少对低效率来源的分解：它通常不能指出改进应主要来自哪一类投入或产出，也难以在异质样本中给出可核对的参照单元。在资源环境问题中，生产同时涉及能源、资本、劳动等投入以及经济产出与环境结果，单一比率或主观加权指数很难同时兼顾这些维度。

1.4.2 数据包络分析（DEA）的核心原理与优势

DEA 是一种非参数相对效率测量技术，用于评估决策单元（decision making unit, DMU）相对于样本前沿的表现。它根据观测数据构造分段线性的效率前沿。思想渊源可追溯至Farrell（1957）；作为可计算方法论的确立，则以 Charnes 等（1978）的CCR（Charnes–Cooper–Rhodes）模型为标志。该模型通过线性规划，在约束下最大化加权产出与加权投入之比，为每个决策单元计算效率得分：落在前沿上的单元为相对有效，

前沿内部的单元为相对无效，得分表示与样本内最佳实践的差距。Banker 等（1984）将模型拓展到规模报酬可变（Variable Returns to Scale，VRS），得到BCC（Banker–Charnes–Cooper）模型。规模报酬不变（Constant Returns to Scale，CRS）设定更常用于讨论长期潜在效率或跨规模比较，VRS 则更常用于短期运营效率与规模效率识别。二者都是相对于样本与变量设定的相对效率，而不是绝对的技术真理。

DEA 的一个主要优点，是能够同时处理多个投入和多个产出，并且不必事先外生指定全部权重。它也不要求预先设定投入与产出之间的参数化函数形式。线性规划会为每个决策单元寻找一组对该单元最有利的权重，从而避免把一套专家权重强加给所有单元（Charnes 等, 1978）。但“最有利” 并不等于“没有研究者判断”：投入产出清单、规模报酬假设、导向（投入、产出或方向距离）以及是否加入权重约束，仍然由研究设定决定。若不对权重加以限制，某些变量可能获得极端权重，解释时需要谨慎。

在资源环境绩效评估中，这一结构可以直接容纳资源消耗（投入）、经济产出（期望产出）与污染排放（非期望产出）。例如，评价地区碳排放效率时，可以把能源、资本、劳动作为投入，把GDP 作为期望产出、CO2 作为非期望产出，在给定技术假设下识别样本前沿。非期望产出如何进入模型（弱可处置、方向距离、物质平衡等），将在后续章节专门讨论。

DEA 还具有基准化含义：它可以指出参照单元，以及低效率单元在既定方向上的改进幅度。这使分析从单一得分扩展到可核对的差距来源。实践中可用于比较地区或企业相对于样本前沿的位置，也可以作为政策讨论的描述性证据；若要评估规制的因果效应，仍需额外的识别策略。方法层面，Chen 和Wang（2024）讨论了在比较生态效率与生产效率时，因模型维度不同而可能出现的得分膨胀，并给出校正思路，而不是一劳永逸地取消维数问题。Emrouznejad 等（2023）则从文献计量角度梳理了生态效率 DEA 与净零相关议题的研究脉络。

1.4.3 DEA应用的考量与挑战

DEA 应用中的一个关键限制是维数问题。当投入产出个数相对于决策单元数目过高时，模型容易把过多单元判为有效，区分度下降。例如，样本很小却放入数十个指标时，几乎每个单元都可能在某一维度上显得“有效”。Chen 和 Wang（2024）也指出，高维设定会削弱判别力。因此，尽管 DEA 在形式上能容纳多指标，变量仍须按研究问题筛选，并报告稳健性。

传统 DEA 把前沿以外的偏差全部记为无效率，因而对测量误差和随机扰动敏感。Kuosmanen 和 Kortelainen（2012）提出的随机非参数数据包络（Stochastic Nonparametric Envelopment of Data，StoNED），在形状约束下引入由无效率项与噪声项构成的复合误差，试图在保留非参数结构的同时分离噪声。Lin 和 Lu（2024）比较了机会约束 DEA、StoNED 与 Bootstrap 在城市文化再生绩效中的表现，说明三者在不确定性处理、函数假设和计算负担上并不相同，方法选择应随问题而定。

结果还依赖于样本与数据质量。Simar 和Wilson（1998）表明，非参数前沿上的效率得分会随抽样变异而显著波动，可用自助法评估其敏感性。应用于资源环境问题时应交代数据来源、异常值处理和样本边界，避免把一次点估计写成稳健事实。

模型设定同样关键。DEA 本身不能检验“所选投入产出是否正确”；不同变量组合可以改变排名。是否把环境规制强度当作投入、是否用物质平衡来约束非期望产出，都会改变结论的含义。需要结合理论、数据与研究问题选择变量，并做对照设定。

DEA 作为确定性前沿，不能区分无效率与噪声；随机前沿分析（Stochastic Frontier Analysis，SFA）则用复合误差做这种分解，但依赖函数形式与分布假设。Kang 和Kim（2018）对中国产业层面数据的比较发现，DEA 与 SFA 的效率得分正相关，全要素生产率趋势相近，而 SFA 的分解路径更平滑。该研究样本与期

刊覆盖有限，宜视为一项对照证据，而不是“SFA 一律优于 DEA” 或相反。研究者应按目标与数据在二者之间选择，或并行报告。

随着维度增加，结果解释也更依赖对行业与制度背景的理解。使用 DEA 的关键不在于“跑通模型”，而在于说清楚技术假设、样本和得分的含义。

总体而言， DEA 适合处理多投入多产出、避免外生赋权的相对效率问题，同时也受数据质量、变量选择和统计推断的约束。把它与稳健性检验、必要时与SFA 或其他方法对照，才能更好支撑资源环境绩效评估。

1.5 Python在资源环境绩效评估中的应用优势

要把 DEA 写成可核对的计算过程，需要能处理表格数据、求解线性规划并绘图的工具。Python 语法相对直观，科学计算库完整，在数据科学和科研计算中使用广泛，因此本书用它实现模型。它是该领域常用工具之一，而不是物理、生物或天文研究中唯一的“主流语言”。以下从易用性、科学计算生态和DEA 实现三个方面说明。

1.5.1 Python的易用性与广泛适用性

Python 语法可读性较高，有利于把公式转成可检查的代码，也便于没有长期编程训练的研究者上手。与Java 或 C++ 相比，同样的数据处理流程通常更短，这有助于降低本书读者的入门成本。易用并不等于可以省略对模型假设的理解。

Python 的另一个条件是社区与文档：开源库、教程和问答平台使常见计算问题较容易查到已有实现。这对数据科学中快速变化的工具链是实际便利，而不是质量保证。

其应用范围覆盖统计、经济、工程和自然科学中的数据分析任务。研究人员常用它完成收集、清洗、建模与可视化（Castro 等, 2024）。掌握 Python 既服务于本书的 DEA 实现，也可以迁移到其他数据分析工作。

1.5.2 强大的科学计算与数据处理生态系统

Python 的库生态为表格处理、数值计算、优化和绘图提供了现成组件。在资源环境绩效评估中，它们可以覆盖从读入数据到报告图形的流程。Pandas、NumPy、SciPy 和 Matplotlib 是其中最常用的基础组合。

Pandas 提供 DataFrame 和 Series，面向统计分析中常见的结构化表（McKinney, 2010）。投入产出数据多以表格存储，清洗、缺失值、合并与分组都可以在同一对象上完成，并做初步的描述统计。

NumPy 提供多维数组 ndarray 及相应的向量化运算，是科学计算的底层依赖； Harris 等（2020）指出，它出现在包括引力波探测和黑洞成像在内的科研软件栈中。与Python 列表相比， ndarray 在大规模数值运算上更有效，线性代数等函数可直接作用于数组。

SciPy 在 NumPy 之上提供优化、插值、积分、稀疏矩阵等算法。Virtanen 等（2020）将其视为 Python 科学算法的事实标准之一。其中 `linprog` 等例程可用于中小规模线性规划，从而支撑入门级 DEA 求解；更大规模或带整数约束的问题则通常交给专门建模层。SciPy 与Pandas、NumPy 可以在同一脚本中衔接。

Matplotlib 用于折线、散点、柱状等常规图形，接口接近MATLAB，便于控制出版所需的坐标轴与标注（Hunter, 2007）。Seaborn 在其之上提供更省默认样式的统计绘图接口，并与 DataFrame 配合较好（Waskom, 2021）。

四者分工大致是：NumPy 做数组运算，Pandas 做表，SciPy 做算法，Matplotlib 做图。统一环境减少了在多种商业软件之间反复导数据的成本，也便于把代码和结果一并存档。

1.5.3 Python助力DEA模型实现与调试

DEA 需要整理投入产出表、求解线性规划并报告效率与权重。Python 可以把这几步写在同一流程里。

数据准备阶段，Pandas 适合从 Excel、CSV 或数据库读入 DMU 表，做筛选、缺失值与异常值处理，再整理成求解器所需的矩阵。

求解阶段，SciPy 的 `linprog` 适合中小规模问题；规模更大或约束更复杂时，可用 Pyomo 等建模库书写线性、整数或非线性规划。专门面向本领域的包也在增加，例如pyStoNED 提供凸回归、StoNED 及部分 DEA 相关例程。

亦有应用研究在 Python 中调用 PyDEA 等接口。Radovilsky 等（2021）在建筑能源效率比较中使用了包括 PyDEA 在内的工具，并报告 CCR 设定下结果较为一致。这类研究说明相关接口可以完成计算，但样本与问题都较特定，不足以把某一软件包写成已经过普遍验证的“标准可靠工具”。本书后续章节以可阅读的线性规划与 `deabook` 流程为主，读者应能不依赖黑箱包核对模型。

从预处理、求解到作图，Python 可以减少在不同软件间转换数据的步骤。Jupyter Notebook 把代码与中间图放在同一文档中，便于逐步检查约束、目标值和效率得分。典型步骤是：用Pandas 组织数据，用 SciPy、PuLP 或 Pyomo 写规划，调用求解器，再提取效率与权重。Python 中的 DEA 实现正从零散脚本转向可复用库，但理解模型仍优先于调用封装。

因此，选择 Python 是为了把 DEA 的数学形式落成可检查、可修改的代码，而不是因为某种语言具有方法上的特权。NumPy、Pandas、SciPy、Pyomo 与 Matplotlib 构成全书示例的主要工具链。

1.6 一个贯穿全书的问题：投入多是否意味着效率低

美国19家燃煤电厂中，燃料消耗与发电量共同上升，但这条关系本身不能识别效率。两家电厂可能有相似的燃料生产率，却使用不同数量的劳动和装机容量。图中的点对应真实电厂，表中仅汇总发电量与燃料的比值；这个单要素比值用于提出问题，不能替代第3章的多投入DEA。数据来自Färe、Grosskopf和Kokkelenberg（1989），公开版本见deaR的Electric_plants。

```
from deabook.example.book_cases.run import FUN
FUN[1]()  # 重新计算本章，输出完整CSV、摘要表与PNG/PDF图
```

表 1.1: 一个贯穿全书的问题：投入多是否意味着效率低：结果摘要

电厂	劳动	燃料	装机	发电量	发电燃料比
Coffeen	179.000	3969.100	1005.500	3531.400	0.890
Grant Tower	96.000	896.320	194.600	841.300	0.939
Gudsonville	94.000	803.170	200.000	690.500	0.860
Meredosia	141.000	1629.900	564.100	1498.100	0.919
Newton	147.000	3264.400	617.400	3347.700	1.026
Fisk	261.000	1427.500	347.100	1263.200	0.885

电厂	劳动	燃料	装机	发电量	发电燃料比
Joliet	448.000	6365.300	1680.400	5734.000	0.901
Kinkaid	223.000	4911.000	1319.400	4107.700	0.836

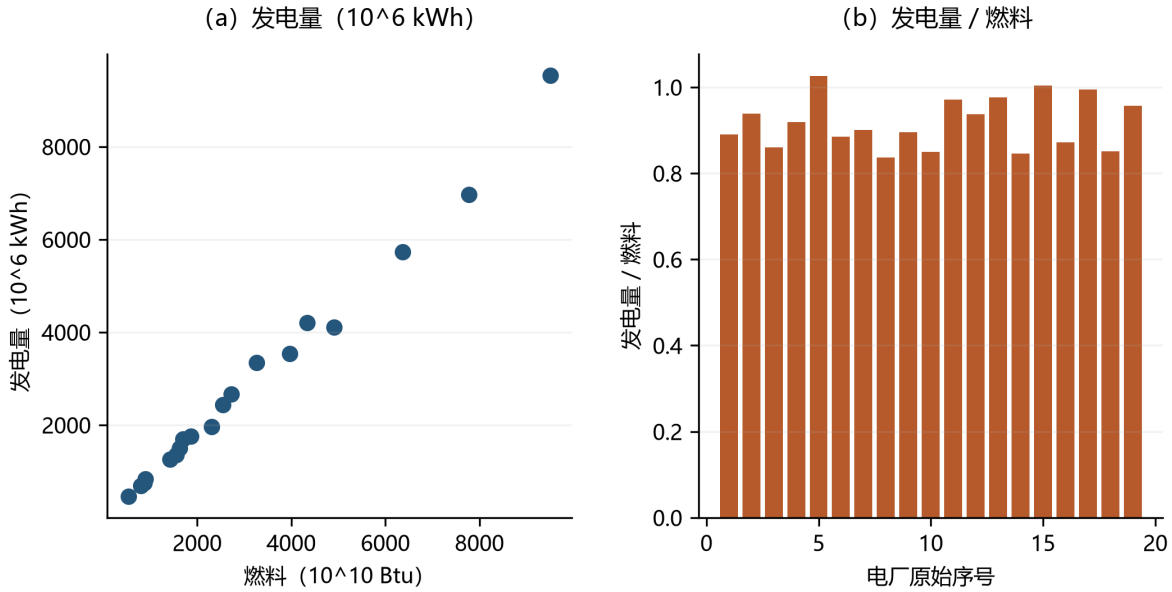


图 1.1: 美国电厂的燃料与发电量：为何需要多投入比较

表中按原文件顺序列出前8家电厂，供读者核对劳动、燃料、装机与发电量的量纲，并比较发电燃料比。同样的单项比率可能对应不同投入组合，不能直接替代多投入效率。图左显示投入产出的规模关系，图右展示全部19家电厂的单项比率。

燃料生产率高的电厂未必在考虑劳动和装机后仍然有效。后续章节用同一生产技术中的多投入约束回答这个问题；绪论不据单个比值给电厂排序。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch01_full.csv`。表中P10和P90为第10和第90百分位数（如适用），展示分布而不占用逐个决策单元的行数。

1.7 本书结构与内容概览

1.7.1 章节安排与知识体系

全书按“计算基础—DEA 理论—程序实现” 递进，分为三个部分：第一部分（第 2 章）为Python 基础；第二部分（第 3 章至第 10 章）为 DEA 理论与方法；第三部分（第 11 章至第 13 章及附录）为程序实现与应用拓展。各章安排如下。

第 2 章 Python基础知识 介绍数据分析与 DEA 实现所需的语法、数据结构、控制流程以及 NumPy、Pandas 等库的基本用法，使不同编程背景的读者能够阅读后续代码。

第 3 章 数据包络分析的经典模型讨论生产技术设定、产出/投入/方向距离函数，以及 CCR、BCC 模型

的数学形式与估计，为后面各章提供共同语言。

第 4 章 传统生产率测度方法 介绍基于 DEA 的 Malmquist 生产率指数及其分解，包括产出导向、投入导向以及方向距离函数对应的指数，并用OECD 国家与中国省份案例说明动态效率的计算与读图口径。

第 5 章 能源效率与能源生产率测度方法讨论同时缩减能源与其他投入、仅缩减能源、同时调整投入与产出等测度方式，并引入Malmquist 指数做动态能源生产率分析。

第 6 章 环境绩效测度方法（非期望产出）在含非期望产出的生产技术下，比较谢泼德距离函数、双曲距离函数、方向距离函数与非径向方向距离函数，并用OECD 国家和中国省份案例做实证对照。

第 7 章 包含非期望产出的生产率把静态环境效率扩展到跨期变化，引入 Malmquist 指数、Malmquist-Luenberger 指数、Luenberger 指标与非径向方向距离函数，并将生产率变化分解为效率变化与技术变化。

第 8 章 基于松弛测度的效率模型针对径向模型忽略松弛导致的高估，介绍不含与含非期望产出的 SBM（投入导向、产出导向与非导向），并结合 deabook 说明方向向量、规模报酬与结果口径。

第 9 章 基于物质平衡的环境绩效测度方法把物质平衡原则写入效率评估，说明其与距离函数的关系，给出符合物质守恒的非参数模型，并用中国火电行业案例说明分解指标的读法。

第 10 章 随机非参数数据包络分析 介绍凸非参数最小二乘（CNLS）与StoNED 的统一框架，说明如何把 DEA 的公理化结构与噪声分解结合起来，并讨论多产出及非期望产出情形。

第 11 章 DEA模型编程的基本思路 说明如何把 DEA 写成求解器可识别的线性规划、如何在数据集上循环，以及如何封装为 Python 函数。

第 12 章 DEA程序编写的实战演练 给出 SBM、含非期望产出的SBM、方向距离函数（DDF）、非径向方向距离函数（NDDF）、Malmquist 指数与Malmquist-Luenberger 指数等模型的编程示例。

第 13 章 基于DEA对偶问题的影子价格估计由对偶问题讨论影子价格，包括线性规划对偶、SBM 与 NDDF 的对偶及其 Python 实现，用于讨论资源和污染的隐含估值。

附录 Python编写的DEA程序的调试 介绍阅读报错、打印变量以及 Pyomo 模型检查等调试方法。

1.7.2 理论与实践的融合

各章以数学表达和图形说明概念，并配以可运行的 Python 示例，便于对照公式与代码。融合大致分三层。方法层：介绍一种模型时给出线性规划形式和求解框架。案例层：第 3 章至第 10 章中的多数理论章使用真实或公开整理的数据（如OECD 国家、中国省份或行业样本）演示读入、求解与作图；第 2 章以语言与库的基础练习为主，第 11 章以程序结构为主，并不各配一套完整实证。实战层：第12、13章提供可改写的函数代码，附录处理调试。

这样安排是为了同时回答方法“是什么、为什么” 以及计算“如何做”。案例选自资源环境场景，用于说明口径和步骤，而不是对所涉国家或行业作政策鉴定。

1.8 本书的读者对象与价值

1.8.1 目标读者

本书面向需要把资源环境绩效写成可计算问题的读者，主要包括三类。

一是学术与研究界。环境科学、环境经济学、运筹学、工业工程、能源研究、可持续发展以及管理科学的研究生、高年级本科生和科研人员，若需要一套从模型到代码的相对效率工具，可以把本书作为教材或手

册。

二是从业者与政策分析人员。生态环境、能源、发展改革等部门，以及咨询机构中从事基准比较、减排测算或报告编制的人员，可以使用书中的模型和代码做描述性评估。结论应连同样本、变量和技术假设一起报告。

三是数据科学与分析背景的读者。若希望把 Python 与优化用于效率和环境问题，可以把各章案例当作领域模板。

1.8.2 本书的独特价值

本书把 DEA 的模型表述与 Python 实现放在同一条线索里，区别于只讲定理或只给软件操作的读物。

第一，它处理多投入（资本、劳动、能源）与多产出（经济产出、环境结果）的相对效率，并给出可改写的代码，使读者能把问题写成规划而不是停留在单一比率。

第二，它强调诊断：Malmquist 分解、影子价格、非径向方向距离函数等，用来讨论差距更可能来自哪一类变化。这些结果仍是相对效率分解，需要额外设计才能支持因果评价。

第三，代码以开源 Python 编写，便于复现和改用到读者自己的数据，也避免对昂贵商业许可的依赖。公开可获取不等于免除对数据授权与引用的责任。

第四，内容从语法与经典 DEA 递进到 StoNED 与影子价格，附录补充调试，以便不同程度的读者选择入口。

本书的目标，是让读者能够完成口径清楚、步骤可核对的数据驱动评估，为资源管理和环境政策讨论提供可检验的描述证据。

参考文献

Banker R D, Charnes A, Cooper W W, 1984. Some Models for Estimating Technical and Scale Inefficiencies in Data Envelopment Analysis[J]. Management Science, 30(9): 1078-1092.

Castro O, Bruneau P, Sottet J S, et al., 2024. Landscape of High-Performance Python to Develop Data Science and Machine Learning Applications[J]. ACM Computing Surveys, 56(3): 1-30.

Ceballos G, Ehrlich P R, Barnosky A D, et al., 2015. Accelerated modern human-induced species losses: Entering the sixth mass extinction[J]. Science Advances, 1(5): e1400253.

Ceballos G, Ehrlich P R, 2023. Mutilation of the tree of life via mass extinction of animal genera[J]. Proceedings of the National Academy of Sciences, 120(39): e2306987120.

Charnes A, Cooper W W, Rhodes E, 1978. Measuring the efficiency of decision making units[J]. European Journal of Operational Research, 2(6): 429-444.

Chen C M, Wang H, 2024. Comparing eco-efficiency with productive efficiency: Addressing the dimensionality issue[J]. European Journal of Operational Research, 313(3): 1170-1179.

Daly H E, 1990. Toward some operational principles of sustainable development[J]. Ecological Economics, 2(1): 1-6.

Emrouznejad A, Marra M, Yang G L, et al., 2023. Eco-efficiency considering NetZero and data envelopment analysis: a critical literature review[J]. IMA Journal of Management Mathematics, 34(4): 599-632.

Farrell M J, 1957. The Measurement of Productive Efficiency[J]. Journal of the Royal Statistical Society.

Series A (General), 120(3): 253-281.

Fuso Nerini F, Mazzucato M, Rockström J, et al., 2024. Extending the Sustainable Development Goals to 2050 — a road map[J]. Nature, 630(8017): 555-558.

Harris C R, Millman K J, Van Der Walt S J, et al., 2020. Array programming with NumPy[J]. Nature, 585(7825): 357-362.

Hunter J D, 2007. Matplotlib: A 2D Graphics Environment[J]. Computing in Science & Engineering, 9(3): 90-95.

IPCC, 2023. Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change[R]. Geneva: Intergovernmental Panel on Climate Change.

Kang Z N, Kim T H, 2018. A Comparative Study of DEA-SFA for Industry-level in China[J]. The Journal of International Trade & Commerce, 14(3): 17-31.

Kaza S, Yao L, Bhada-Tata P, et al., 2018. What a Waste 2.0: A Global Snapshot of Solid Waste Management to 2050[R]. Washington, DC: World Bank.

Koerber W, 2024. Eyes on the data—How the methodology of the 2030 Agenda contributes to its probable failure[J]. World Development Sustainability, 5: 100188.

Krivonozhko V E, Piskunov A A, Lychev A V, 2011. On comparison of ratio analysis and data envelopment analysis as performance assessment tools[J]. IMA Journal of Management Mathematics, 22(4): 357-370.

Kuosmanen T, Kortelainen M, 2012. Stochastic non-smooth envelopment of data: semi-parametric frontier estimation subject to shape constraints[J]. Journal of Productivity Analysis, 38(1): 11-28.

Lenton T M, Held H, Kriegler E, et al., 2008. Tipping elements in the Earth's climate system[J]. Proceedings of the National Academy of Sciences, 105(6): 1786-1793.

Lenton T M, Rockström J, Gaffney O, et al., 2019. Climate tipping points — too risky to bet against[J]. Nature, 575(7784): 592-595.

Lin S W, Lu W M, 2024. A comparison of chance-constrained data envelopment analysis, stochastic nonparametric envelopment of data and bootstrap method: A case study of cultural regeneration performance of cities[J]. European Journal of Operational Research, 316(3): 1179-1191.

Liu T, Chen D, Yang L, et al., 2023. Teleconnections among tipping elements in the Earth system[J]. Nature Climate Change, 13(1): 67-74.

McKinney W, 2010. Data Structures for Statistical Computing in Python[C]//Proceedings of the 9th Python in Science Conference. Austin, Texas: 56-61.

Nature Editorial, 2023. The world's plan to make humanity sustainable is failing. Science can do more to save it[J]. Nature, 618(7966): 647.

Patel G, Cvetkoska V, Kitanovikj B, 2025. Measuring Corporate Sustainability with Data Envelopment Analysis: A Multi-Technique Bibliometric Review and Future Research Agenda[M]//Emrouznejad A, Ray S C, Patel G. Advances in the Theory and Practice of Data Envelopment Analysis. Cham: Springer Nature Switzerland: 77-94.

Pearce D W, Atkinson G D, 1993. Capital theory and the measurement of sustainable development: an indicator of "weak" sustainability[J]. Ecological Economics, 8(2): 103-108.

Radovilsky Z, Taneja P, Sahay P, 2021. Data Envelopment Analysis and Analytics Software for Optimizing

Building Energy Efficiency[J]. International Journal of Business Analytics, 9(1): 1-17.

Sato I, 2025. An indicator framework for post-2030 international development goals[J]. Communications Earth & Environment, 6: 816.

Simar L, Wilson P W, 1998. Sensitivity analysis of efficiency scores: How to bootstrap in nonparametric frontier models[J]. Management Science, 44(1): 49-61.

Song M, An Q, Zhang W, et al., 2012. Environmental efficiency evaluation based on data envelopment analysis: A review[J]. Renewable and Sustainable Energy Reviews, 16(7): 4465-4469.

United Nations, 2005. 2005 World Summit Outcome[R]. A/RES/60/1. New York: United Nations.

United Nations, 2024. The Sustainable Development Goals Report 2024[R]. New York: United Nations.

Virtanen P, Gommers R, Oliphant T E, et al., 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python[J]. Nature Methods, 17(3): 261-272.

Waskom M, 2021. seaborn: statistical data visualization[J]. Journal of Open Source Software, 6(60): 3021.

World Bank, 2016. The Cost of Air Pollution: Strengthening the Economic Case for Action[R]. Washington, DC: World Bank.

World Commission on Environment and Development, 1987. Our Common Future[M]. Oxford: Oxford University Press.

World Meteorological Organization, 2025. State of the Global Climate 2024[R]. WMO-No. 1368. Geneva: World Meteorological Organization.

Xing Q, Lu L, Wang L, et al., 2025. Country-specific progress toward the Sustainable Development Goals: Past, present, and prospects[J]. Proceedings of the National Academy of Sciences, 122(52): e2524299122.

第 2 章

Python基础知识

Python是一种通用编程语言，功能与扩展性均十分强大。本章不打算系统介绍Python编程，而是聚焦于编写DEA程序所需的Python基础知识，为后续的DEA程序开发做必要的知识准备[1]。与一般的Python入门教程不同，本章始终围绕DEA编程的实际需求展开，每个知识点均说明其在DEA编程中的具体用法。需要强调的是，本章并非独立的编程教程，而是方法论正文的” 工具箱”：本书自第3 章起即进入 DEA 模型的实证实现，所有案例都建立在 NumPy 数组、pandas 数据框与 Pyomo 线性规划建模这三项能力之上，而本章结尾的小节 2.8 与 小节 2.9 将进一步把这些能力收束为 `deabook` 工具包的统一调用约定，作为进入后续章节的方法论钥匙。

2.1 Python的安装与环境配置

DEA 实证研究的起步往往不在建模，而在把数据、求解器与运行环境理顺——一旦环境不一致，同一段 `deabook` 代码在不同机器上就可能产生不可复现的结果。本节先确立一套可复现、可移植的Python 环境，作为后续各章案例运行的共同前提。

推荐使用 Anaconda（或体积更小的 Miniconda）发行版，它自带 Python 解释器、包管理器 `conda` 以及常用的科学计算库，可一次性配齐 DEA 编程所需的基础环境，后续只需按需安装本书用到的少量扩展包即可。下面分Windows 与 macOS 两种系统，介绍下载、安装、环境变量配置、虚拟环境管理与包安装的全过程；Linux 用户的步骤与 macOS 基本一致（仅安装包格式不同，可用 `bash Anaconda-*.sh` 脚本安装）。

2.1.1 Anaconda下载与安装

（1）下载安装包

打开 Anaconda 官网 https://www.anaconda.com/download；页面会自动识别操作系统并给出对应安装包，也可在“Downloads” 区域手动选择：

- **Windows**：选择 **Windows → 64-Bit (x86) Installer**，得到 `.exe` 文件。
- **macOS**：在 “macOS” 下先确认芯片类型——Apple 芯片（M1/M2/M3 等）选 **Apple Silicon (arm64)**，Intel 芯片选 **x86_64**，得到 `.pkg` 文件。若磁盘空间紧张，可改用 **Miniconda**（仅含 Python 与 `conda`，约

[1]本章面向已具备任意一门编程语言基础（如R、MATLAB、C或Java）的读者。如果读者完全没有编程基础，建议先阅读以下资源之一：Python官方Tutorial（https://docs.python.org/3/tutorial/），完成第1—5章；Jake VanderPlas 著《Python数据科学手册》第1—2章。

几十MB，下载与安装步骤相同）。

（2）安装

- **Windows**：双击 `.exe` 启动安装向导，依次点击 **Next → I Agree**；“Install For” 一步建议选 **Just Me**（仅当前用户，可避免权限问题）；“Choose Install Location” 一步**安装路径不要包含中文或空格**（例如 `C:\Users\<你的用户名>\Anaconda3`）。关键一步是 **Advanced Options**，其中两项建议都勾选：
  - **Add Anaconda3 to my PATH environment variable**：勾选后可在任意命令行直接使用 `conda`/`python`（若不勾选，则需通过开始菜单的“Anaconda Prompt” 使用）。
  - **Register Anaconda3 as my default Python 3.x**：将其设为系统默认 Python。然后点击 **Install**，等待进度条完成，最后 **Next → Finish**。
- **macOS**：双击 `.pkg` 启动安装向导，依次 **Continue** → 阅读并 **Agree** 许可协议 → 选择安装位置（默认即可）→ **Install**，按提示输入系统登录密码，完成后 **Close**。安装程序会自动把 `conda` 初始化到当前用户的shell（Apple 芯片默认 zsh，对应`~/.zshrc`；Intel 芯片多为`~/.bash_profile`），使新开终端即可识别 `conda`。

（3）验证安装

安装完成后，打开一个**全新**的终端窗口验证（Windows 可用 “Anaconda Prompt”、PowerShell 或 cmd；macOS 用 “终端” Terminal）：

```
conda --version      # 应输出类似 conda 24.x.x
python --version     # 应输出 Anaconda 自带的 Python 版本
```

若能正常输出版本号，说明 Anaconda 已安装成功并可被调用。

（4）配置系统环境变量（将 conda 加入 PATH）

若安装时未勾选 “Add to PATH”，或在新终端中输入 `conda` 提示” 不是内部或外部命令 / command not found”，需手动把 Anaconda 加入系统 PATH。

- **Windows（图形界面）**：右键” 此电脑”→ **属性** → **高级系统设置** → **环境变量**；在” 用户变量”（仅当前用户）或” 系统变量”（所有用户）中找到 **Path**，点击 **编辑** → **新建**，依次添加以下三项（用户名与安装目录按实际情况修改）：

  `C:\Users\<你的用户名>\Anaconda3`

  `C:\Users\<你的用户名>\Anaconda3\Scripts`

  `C:\Users\<你的用户名>\Anaconda3\Library\bin`

  确认后**重新打开**终端，再执行 `conda --version` 验证。
- **macOS / Linux（修改 shell 配置）**：在终端中用文本编辑器打开shell 配置文件（`~/.zshrc` 或 `~/.bash_profile`），在末尾追加一行（路径按实际安装目录修改）：

  ```
  export PATH="$HOME/opt/anaconda3/bin:$PATH"
  ```

  保存后执行 `source ~/.zshrc`（或 `source ~/.bash_profile`）使配置生效，再运行 `conda --version` 验证。也可直接执行 `conda init zsh`（或 `conda init bash`）让 conda 自动写入初始化代码。

（5）用 conda 创建与管理虚拟环境

DEA 项目往往依赖特定版本的 NumPy、pandas、Pyomo 与求解器；为避免不同项目间的版本冲突、保证结果可复现，推荐为每个项目（或本书）单独创建一个虚拟环境。

```
# 创建名为 dea、Python 版本为 3.11 的虚拟环境
conda create -n dea python=3.11

# 激活环境(激活后命令行前缀会出现 (dea))
conda activate dea   # 若在普通 cmd 中报错,先运行 conda init cmd.exe,或改用 Anaconda Prompt

# 查看本机所有环境(带 * 的为当前激活环境)
conda env list

# 退出当前环境
conda deactivate

# 删除环境(谨慎)
conda env remove -n dea
```

激活环境后安装的所有包都只存在于该环境内,互不干扰。**强烈建议**本书全部案例都在同一个 dea 环境中运行:先 conda activate dea,再安装下一节的 DEA 相关包。

(6)安装 Python 包(conda 与 pip)

conda 与 pip 都可安装 Python 包,但分工不同:conda 能管理二进制依赖(含 glpk、cbc 等需要系统可执行文件的求解器),pip 则面向 PyPI 上的纯 Python 包。务必在**已激活的 dea 环境**中安装:

```
# conda 安装(必要时加 -c conda-forge 指定社区源)
conda install numpy pandas scipy

# pip 安装
pip install highspy
```

为完整复现本书环境,可直接导出/导入环境清单:

```
conda env export > environment.yml     # 导出当前环境
conda env create -f environment.yml    # 在另一台机器上据此重建
```

本书专用包清单:上面只是通用方法。本书第 3–10 章所需的核心包(deabook、pyomo、求解器 highs/glpk/cbc、amplpy 等)及验证命令,见下一节 2.1.2。

2.1.2 安装DEA相关Python包

本书后续章节需要以下核心包,建议在开始之前统一安装:

```
# 本书配套的 deabook 工具包(第 3 – 10 章全部案例均依赖此包)
pip install deabook        # 也可: conda install -c conda-forge deabook

# 基础科学计算
conda install numpy pandas scipy
```

```
# 线性规划建模(Pyomo)
conda install -c conda-forge pyomo

# 求解器(按用途区分):
#   • 经典 / 能源 / 动态 DEA(第 3 - 9 章)—— 开源 LP 求解器即可
pip install highspy                        # HiGHS: Pyomo 的 appsi_highs 纯 Python 接口
conda install -c conda-forge coincbc      # cbc(COIN-OR 分支定界)
conda install -c conda-forge glpk         # glpk(GNU 线性规划工具包)
#   • 随机非参数前沿 CNLS(第 10 章)—— 需商业求解器 mosek 或 knitro(学术许可,见附录 A.1)
#   • 凸非参数分位数回归 CQR(第 10 章)—— 需 amplpy
pip install amplpy

# 验证安装
python -c "import numpy, pandas, pyomo, deabook; print('所有包安装成功')"
```

三个开源求解器均可免费使用。其中 highs（即 Pyomo 的 appsi_highs）依赖纯 Python 包 highspy、安装最省事，也是本书第 3–9 章的默认调用名（solver="highs"）；glpk 与 cbc 则需系统可执行文件，通常通过 conda 安装。三者可按 highs → glpk → cbc 的顺序回退（ 表 2.1）：

> **求解器的分工：**第 3–9 章的全部 DEA 模型（含 SBM、Malmquist 动态指数、含非期望产出的环境效率）都属于线性/凸规划，用上述任一开源求解器即可。第10 章的随机非参数前沿（CNLS）需要商业求解器 **mosek** 或 **knitro**，凸非参数分位数回归（CQR）需要 **amplpy**；这两者需单独的许可证，安装方式见附录 A.1，第 10 章也会再次说明。

表 2.1: 开源求解器对照

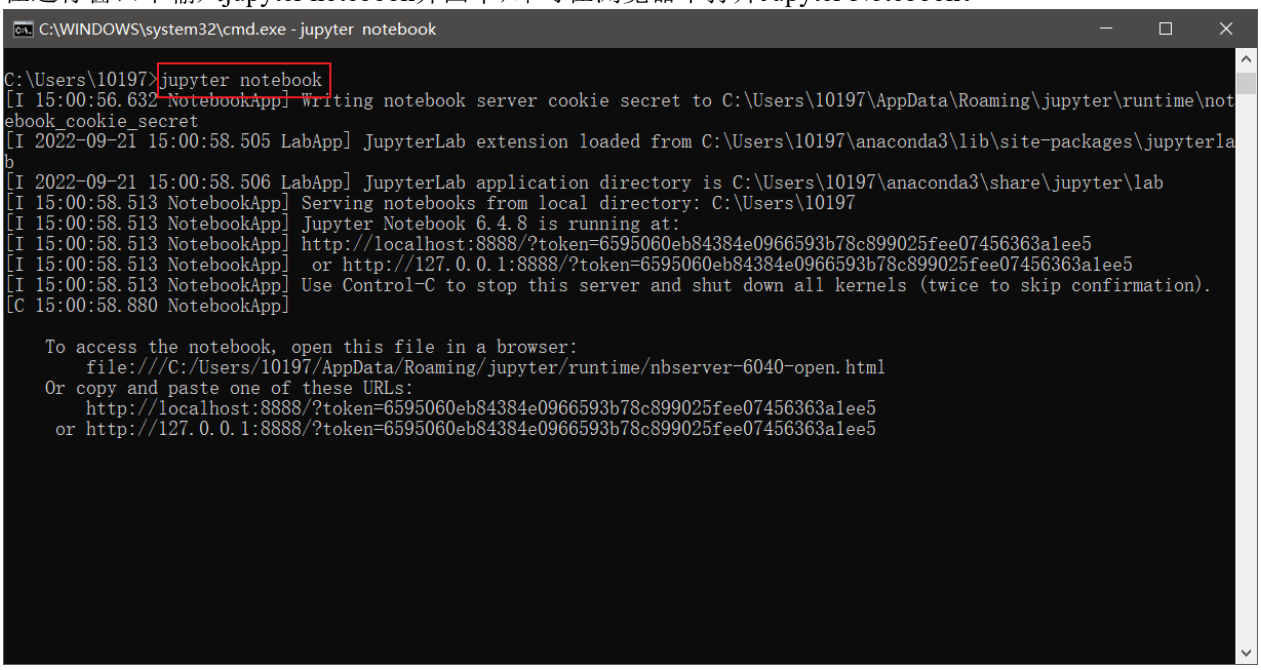
求解器	类型	安装方式	说明
highs（即 appsi_highs）	开源 LP/MIP	pip install highspy	HiGHS 求解器；本书第 3–9 章统一以 solver="highs" 调用
glpk	开源 LP/MIP	conda install -c conda-forge glpk	GNU 线性规划工具包，Pyomo 经典备选求解器
appsi_highs	开源 LP/MIP	pip install highspy	HiGHS 求解器的 Pyomo appsi 接口，纯 Python 安装、无需外部可执行文件
cbc	开源 LP/MIP	conda install -c conda-forge coincbc	COIN-OR 分支定界求解器

2.2 开发环境选择与Jupyter Notebook最佳实践

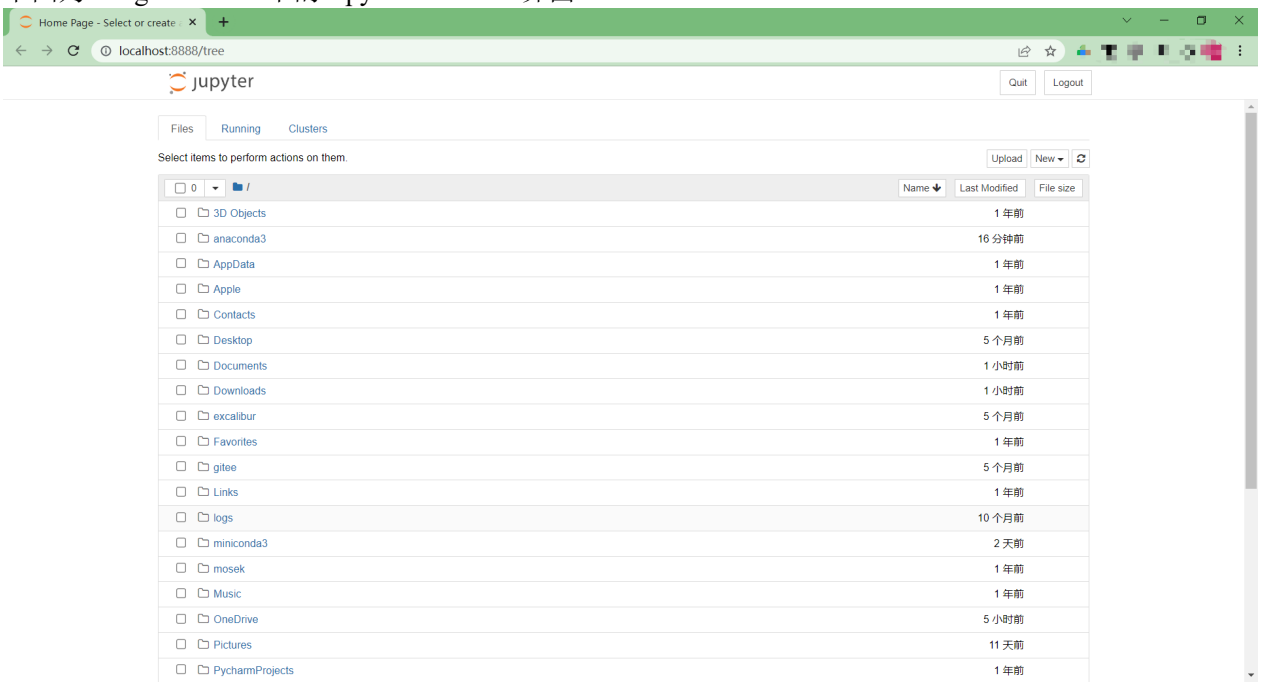
DEA 研究兼具” 探索” 与” 封装” 两类活动：前者是在真实数据上快速试算模型、观察效率分布，后者是把验证过的模型沉淀为可复用的分析流程。开发环境的选择，本质上服务于这两种活动之间的切换——交互式探索适合Jupyter，而生产级的函数与包封装更适合成熟的 IDE。

2.2.1 Jupyter Notebook基础

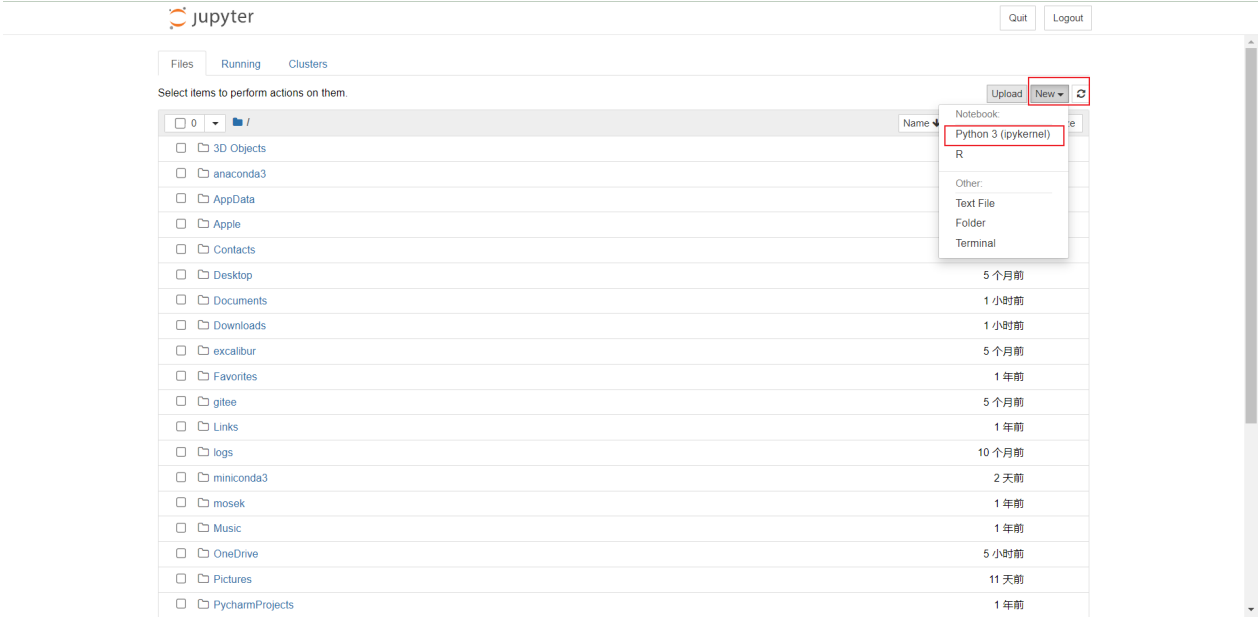
在运行窗口中输入jupyter notebook并回车，即可在浏览器中打开Jupyter Notebook。



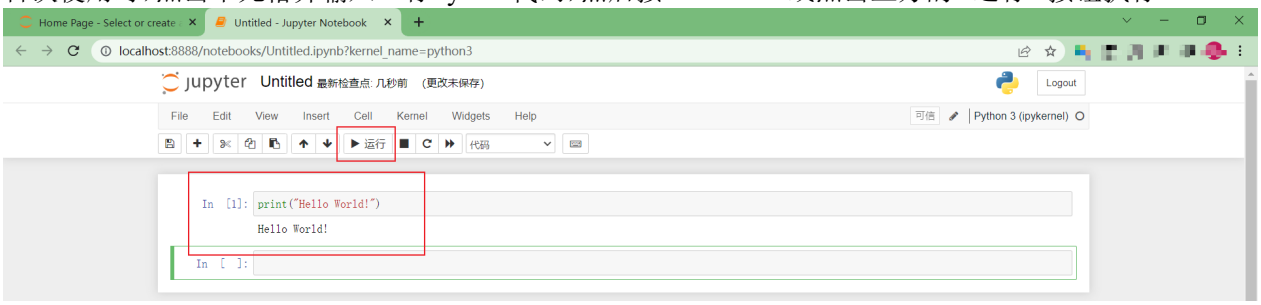
下图为Google Chrome中的Jupyter Notebook界面。



要新建一个Notebook，点击按钮New，选择”Python3”。



首次使用时，点击单元格并输入一行Python代码，然后按Shift-Enter或点击上方的” 运行” 按钮执行。



2.2.2 Jupyter Notebook最佳实践（DEA编程场景）

Jupyter Notebook是交互式数据分析与DEA模型原型开发的首选工具，但在实际使用中需注意以下几个问题。

（1）单元格执行顺序管理

Jupyter的一个常见陷阱是**单元格的乱序执行**：可以先执行单元格10，再回头执行单元格5，这会导致变量状态混乱。DEA编程中的一个典型场景是：修改了投入产出数据后，忘记重新执行数据加载单元格，便直接运行模型求解单元格，结果使用的仍是旧数据。

建议： - 使用菜单 **Kernel → Restart & Run All** 定期从头运行整个Notebook - 在关键步骤之间使用 `print()` 确认变量状态 - 使用魔法命令 `%whos` 查看当前所有变量：

```
from deabook import data_path, PANEL
from deabook.constant import CONTEMPORARY
# 查看当前命名空间中的所有变量
%whos
```

（2）重启Kernel的时机

以下情况建议重启Kernel（Kernel → Restart）：- 安装了新的Python包 - 变量状态混乱、结果不可复现 -

遇到难以解释的 `NameError`（可能是之前的变量已被删除）- 运行大样本DEA计算后需要释放内存

（3）Notebook版本控制

Jupyter的`.ipynb`文件为JSON格式，在Git中会产生大量无意义的diff。建议：- 将Notebook转换为`.py`脚本进行版本控制：**File → Download as → Python (.py)** - 或使用 `jupytext`工具实现Notebook与Python脚本的双向同步

2.2.3 其他IDE选择：VS Code与PyCharm

对于大型DEA项目或Python包的封装，以下IDE比Jupyter Notebook更为适合：

特性	Jupyter Notebook	VS Code + Python插件	PyCharm
交互式探索	★★★ 优秀	★★☆ 良好（支持Notebook）	★☆☆ 一般
代码调试	★☆☆ 仅打印	★★★ 断点调试	★★★ 断点调试
代码重构	★☆☆ 无	★★★ 重命名、提取函数	★★★ 智能重构
项目管理	★☆☆ 单个文件	★★★ 工作区	★★★ 项目
DEA原型开发	★★★ 推荐	★★☆ 可用	★★☆ 可用
生产级开发	★☆☆ 不推荐	★★★ 推荐	★★★ 推荐

建议：学习阶段使用Jupyter Notebook，项目封装阶段切换到VS Code或PyCharm。

2.3 Python语法精粹

本节简明介绍DEA编程中最常用的Python语法要点。鉴于读者已有编程基础，解释从简。需要留意的是，DEA 代码中若干易错点正源于 Python 语法本身——例如线性规划里惯用的强度变量符号 λ 在 Python 中是保留字、必须改用 `intensity`，又如本书全程使用的 `sent` 公式字符串依赖对字符串与切片的准确理解。因此本节在交代语法的同时，也直接以DEA 场景说明每一处约定的来由，而非泛泛罗列。

2.3.1 缩进与注释

Python使用空白字符（4个空格）组织代码，而不像R、C++和Java那样使用括号。

```
# Python使用缩进标识代码块（推荐4个空格）
for x in data_array:
    if x < threshold:
        less.append(x)
    else:
        greater.append(x)
```

Python使用#进行单行注释，使用''' 或"""进行多行注释。本书约定：**函数使用docstring（三重引号）注释**，代码行使用#注释。

```
# 单行注释
de = 0.85  # 决策单元A的效率值(注释在代码后面)

'''
多行注释:下面是DEA数据的说明
- x1, x2: 投入指标
- y1: 产出指标
'''
```

函数的 docstring 用于说明函数用途、参数与返回值，可通过 `help(函数名)` 或 `函数名.__doc__` 查看：

```
def calc_efficiency(x, y):
    """计算单个 DMU 的产出导向技术效率,返回 (0, 1] 之间的效率值。"""
    # 函数体略
    ...
```

注释的良好实践是**解释" 为什么" 而非" 是什么"**：代码本身已说明" 做了什么"，注释应补充设计意图、口径约定或易错点。在DEA 编程中，尤其应注释投入产出变量的含义、方向向量的口径以及规模报酬假设，以免后续读者误解。

2.3.2 代码换行

Python 一行代码过长时需换行以保持可读性（本书约定单行不超过 88 个字符）。Python 提供两种换行方式。

方式一：隐式换行（推荐）。在圆括号 `()`、方括号 `[]`、花括号 `{}` 内部可直接换行，无需任何续行符号，Python 会自动将其视为同一语句。DEA 编程中最常见的是超长函数调用与链式方法调用：

```
from deabook.example.book_cases.loaders import plants
from deabook import DEA
data = (
    plants()
    .sort_values("Plant")
    .reset_index(drop=True)
)
model = DEA.ccr(
    data, sent="Labor Fuel Capital=Output"
).optimize(solver="highs")
```

方式二：显式换行（反斜杠\）。在行尾加反斜杠 \ 表示续行。注意反斜杠后**不能有任何字符（包括空格）**，否则会报语法错误，因此本书不推荐使用。

```
# 显式续行(不推荐:反斜杠后若误留空格会报错)
total = x1 + x2 + x3 + x4 + \
        x5 + x6 + x7
```

字符串的换行有两种常用做法：用三重引号直接跨行，或在括号内利用相邻字符串字面量自动拼接：

```
# 三重引号跨行
note = """这是一段跨越多行的说明文字，
可以自然地换行书写。"""

# 相邻字符串字面量自动拼接（常用于较长的提示信息）
message = (
    "DMU 的效率值为 "
    f"{efficiency:.4f}"
)
```

与本书的关系：第 6–13 章的大量 DEA 模型调用（如 `SBM(...)`、`DEAweak(...)`、`MQDDFweak(...)`）均采用”方式一”的圆括号内换行。阅读或复现这些代码时，请把括号内的多行内容视为一条完整的调用语句。

2.3.3 基本数据类型

类型	说明	DEA编程示例
`int`	整数（无限精度）	`num_dmus = 20`
`float`	浮点数（可能有精度误差）	`efficiency = 0.8537`
`bool`	布尔值（`True`/`False`）	`is_efficient = True`
`str`	字符串	`formula = "Y=K L"`

```
# 浮点数精度问题在DEA中尤为重要
efficiency = 1.0 / 3.0
print(f"原始值: {efficiency}")
print(f"四舍五入到4位: {efficiency:.4f}")  # DEA结果通常保留4位小数
```

2.3.4 列表与字典（DEA场景）

列表用于存储有序数据，在DEA中最常见的用途是存储决策单元名称或效率值列表。

```
# 决策单元名称列表
dmu_names = ['DMU_A', 'DMU_B', 'DMU_C', 'DMU_D']
print(dmu_names[0])       # 索引从0开始 → 'DMU_A'
print(dmu_names[1:3])     # 切片 → ['DMU_B', 'DMU_C']

# 效率值列表
efficiencies = [0.85, 1.00, 0.72, 0.93]
```

命名说明:变量名使用 snake_case（小写下划线分隔），如 dmu_names而非 dmuNames。这是Python的PEP 8标准命名规范。

字典用于存储键值对数据，在DEA中常用于存储DMU的效率结果。

```
# 用字典存储各DMU的效率值
dea_results = {
    "DMU_A": 0.85,
    "DMU_B": 1.00,
    "DMU_C": 0.72,
    "DMU_D": 0.93
}
print(dea_results["DMU_A"])   # 通过键访问值 → 0.85

# 遍历效率结果
for dmu, eff in dea_results.items():
    print(f"{dmu}: {eff:.4f}")
```

2.3.5 控制结构

条件判断：DEA中常用于根据效率值判断DMU是否有效。

```
efficiency = 0.85
if efficiency >= 1.0:          # 径向模型下,效率值等于1为样本内有效
    status = "DEA有效"
else:
    status = "DEA无效"
print(status)
```

口径提醒：CCR/BCC 等径向模型只有” 有效（效率值等于1）“与” 无效（效率值小于1）“两档，不存在”0.8以上即相对有效” 这样的中间等级。

for循环：DEA中用于遍历所有决策单元进行效率评价。

```
dmu_list = ['A', 'B', 'C', 'D', 'E']
for dmu in dmu_list:
    print(f"正在评价决策单元: {dmu}")

# range()的用法
for i in range(5):        # 0, 1, 2, 3, 4
    print(i)
```

列表推导式（DEA批量计算利器）：

```
# 传统循环方式
efficiency_list = []
```

```
for i in range(5):
    efficiency_list.append(1.0 / (i + 1))

# 列表推导式(更简洁,推荐)
efficiency_list = [1.0 / (i + 1) for i in range(5)]

# 带条件的列表推导式
dmu_names = [f"DMU_{i}" for i in range(1, 6)]
efficient_dmus = [name for name, eff in zip(dmu_names, [0.85, 1.0, 0.72, 1.0,
    0.93])
                  if eff >= 1.0]
```

2.3.6 函数定义与调用

函数是DEA编程的核心工具。第 11 章将把DEA模型封装为可复用函数。

本章的导向与符号约定：本章与第 11 章一致，默认采用**产出导向**。此时线性规划中的 θ 是产出的**扩张因子**（$\theta \geqslant 1$），技术效率为 $\mathrm{TE} = 1/\theta$。若改用投入导向，则 θ 是投入的**收缩比**（$0 < \theta \leqslant 1$），此时 θ 本身即技术效率，不要再取一次倒数。下文所有示例均为产出导向。

```
def calculate_efficiency_score(theta):
    """
    根据产出导向模型的theta值计算技术效率得分

    Parameters
    ----------
    theta : float
        产出导向模型中的theta值,即产出的扩张因子(theta >= 1)

    Returns
    -------
    float
        技术效率TE（technical efficiency),等于1/theta,取值在(0, 1]
    """
    return 1.0 / theta

# 调用函数
te = calculate_efficiency_score(1.176)
print(f"技术效率: {te:.4f}")
```

命名说明：函数命名使用 `snake_case`（小写+下划线），如 `calculate_efficiency_score`。函数名应清晰表达其功能，参数名应使用描述性名称而非缩写。函数必须具有完整的docstring文档

（包含Parameters和Returns说明），这是本书的强制规范。

2.3.7 模块导入

Python程序可以调用一组基本的内建函数，也可以导入外部模块。本书约定使用 `import`语句在文件顶部统一导入所有模块。

```
# 推荐：文件顶部统一导入
import numpy as np          # 科学计算
import pandas as pd         # 数据处理
from pyomo.environ import *  # 线性规划建模

# 不推荐：函数内部导入（除非解决循环导入问题）
```

2.4 面向DEA的Python编程范式

本节是本章的核心——建立从”Python基础” 到”DEA编程” 的直接桥梁。以下三个编程范式是后续所有DEA代码的基础。

2.4.1 向量化思维：告别Python循环

DEA模型涉及大量矩阵运算。Python原生的 `for`循环速度较慢，而NumPy的向量化运算（底层由C实现）可达到数十倍甚至上百倍的加速。

示例场景：计算5个DMU的投入加权平均值（DEA中构造参考集的常见操作）。

```
import numpy as np
import time

# DEA数据集：5个DMU，2个投入，1个产出
# 列：[x1, x2, y1]
data = np.array([
    [3.0, 2.0, 5.0],   # DMU_A
    [4.0, 3.0, 7.0],   # DMU_B
    [5.0, 4.0, 9.0],   # DMU_C
    [6.0, 5.0, 11.0],  # DMU_D
    [7.0, 6.0, 13.0]   # DMU_E
])

# 强度变量（lambda值）
intensity = np.array([0.2, 0.3, 0.1, 0.2, 0.2])

# 方法1：Python原生循环（慢）
```

```python
def weighted_sum_loop(data, intensity):
    """使用Python循环计算加权投入"""
    n_inputs = 2
    result = []
    for k in range(n_inputs):           # 对每个投入指标
        weighted_sum = 0.0
        for i in range(len(data)):      # 对每个DMU
            weighted_sum += intensity[i] * data[i, k]
        result.append(weighted_sum)
    return result

# 方法2: NumPy向量化运算(快)
def weighted_sum_vectorized(data, intensity):
    """使用NumPy矩阵乘法计算加权投入"""
    x = data[:, :2]         # 投入矩阵 (5×2)
    return intensity @ x    # 矩阵乘法 (1×5) @ (5×2) = (1×2)

# 验证结果一致
result_loop = weighted_sum_loop(data, intensity)
result_vec = weighted_sum_vectorized(data, intensity)
print(f"循环方法结果: {result_loop}")
print(f"向量化方法结果: {result_vec}")

# 性能对比
%timeit weighted_sum_loop(data, intensity)
%timeit weighted_sum_vectorized(data, intensity)
```

性能对比结果（典型值;绝对数值随硬件与NumPy版本而异,量级供参考）:

方法	5个DMU	100个DMU	1000个DMU
Python循环	~3μs	~54μs	~0.57ms
NumPy向量化	~2μs	~2μs	~3μs
加速比	**2x**	**30x**	**200x**

DEA编程原则:能用矩阵运算的地方,绝不用Python循环。DEA模型中的加权求和 $\sum_i \lambda_i X_{ki}$ 应写成 `intensity @ x_matrix`。注意向量化在小规模下的优势有限(函数调用与临时数组开销占比高),其收益随DMU数量增大而显著扩大。

2.4.2 函数封装:从脚本到可复用工具

第 11 章将从单个DMU的求解脚本出发,逐步封装为通用DEA函数。本节先展示函数封装的基本思路。

示例:将" 求解单个DMU的CCR模型" 封装为函数。本示例采用与第 11 章一致的**产出导向**口径:目标为最大化产出扩张因子 θ,约束为 $\lambda X \leqslant x^*$、$\lambda Y \geqslant \theta y^*$,技术效率为 $\mathrm{TE} = 1/\theta$。

```
import numpy as np
from pyomo.environ import *

def solve_ccr_for_one_dmu(x_eval, y_eval, x_ref, y_ref):
    """
    求解单个DMU的CCR产出导向模型(CRS)

    这是DEA函数封装的最小示例。第11章将在此基础上逐步扩展功能。
    模型为: max theta, s.t. lambda • X <= x_eval, lambda • Y >= theta • y_eval,
    lambda >= 0。theta为产出扩张因子,技术效率TE = 1/theta。

    Parameters
    ----------
    x_eval : np.ndarray
        被评价DMU的投入向量 (n_inputs,)
    y_eval : np.ndarray
        被评价DMU的产出向量 (n_outputs,)
    x_ref : np.ndarray
        参照DMU的投入矩阵 (n_dmus, n_inputs)
    y_ref : np.ndarray
        参照DMU的产出矩阵 (n_dmus, n_outputs)

    Returns
    -------
    float
        被评价DMU的技术效率TE,等于1/theta,取值在(0, 1]
    """
    n_ref = x_ref.shape[0]       # 参照DMU数量
    n_inputs = x_ref.shape[1]    # 投入指标数量
    n_outputs = y_ref.shape[1]   # 产出指标数量

    model = ConcreteModel()
    model.I = Set(initialize=range(n_ref))
    model.K = Set(initialize=range(n_inputs))
    model.L = Set(initialize=range(n_outputs))

    # 决策变量: theta (产出扩张因子)和强度变量
```

```
    model.theta = Var(bounds=(0.0, None), doc='产出扩张因子')
    model.intensity = Var(model.I, bounds=(0.0, None), doc='强度变量')

    # 目标函数:最大化theta（产出导向）
    def obj_rule(m):
        return m.theta
    model.obj = Objective(rule=obj_rule, sense=maximize)

    # 投入约束:参照单元的加权投入不超过被评价单元投入
    def input_rule(m, k):
        return sum(m.intensity[i] * x_ref[i, k] for i in m.I) <= x_eval[k]
    model.input_con = Constraint(model.K, rule=input_rule)

    # 产出约束:参照单元的加权产出不低于theta倍被评价单元产出
    def output_rule(m, l):
        return sum(m.intensity[i] * y_ref[i,
            l] for i in m.I) >= m.theta * y_eval[l]
    model.output_con = Constraint(model.L, rule=output_rule)

    # 求解
    opt = SolverFactory('highs')   # 开源 LP 求解器;第 10 章商业前沿模型改用 mosek / knitro
    solution = opt.solve(model)

    return 1.0 / value(model.theta)     # 技术效率TE = 1/theta

# 调用示例
data = np.array([
    [3.0, 2.0, 5.0],
    [4.0, 3.0, 7.0],
    [5.0, 4.0, 9.0],
    [6.0, 5.0, 11.0],
    [7.0, 6.0, 13.0]
])
x_ref = data[:, :2]    # 投入矩阵 (5×2)
y_ref = data[:, 2:]    # 产出矩阵 (5×1)

# 评价第1个DMU (DMU_A)
te = solve_ccr_for_one_dmu(x_ref[0], y_ref[0], x_ref, y_ref)
print(f"DMU_A的技术效率: {te:.4f}")
```

若要改为投入导向：把目标函数改为 `sense=minimize`，把 θ 从产出约束右端移到投入约束右端（即`<= m.theta * x_eval[k]` 与`>= y_eval[l]`），并直接返回 `value(model.theta)`——投入导向下 θ 本身就是技术效率，不要再取倒数。第 11 章 `dea8` 用一个 `orient` 参数统一了这两种写法。

函数封装的价值：- 可复用：一次编写，可评价任意DMU - **可测试：**在小数据集上验证正确性后，即可批量使用 - **可扩展：**后续章节在此基础上增加VRS、方向选择等功能

2.4.3 矩阵构建：用NumPy构造DEA数据矩阵

DEA模型本质上是一系列矩阵运算，正确构造投入矩阵 `X`和产出矩阵 `Y`是DEA编程的第一步。

示例： 3个DMU、2投入1产出的小型DEA数据集

```
import numpy as np

# === DEA数据集构建 ===
# 3个DMU (A, B, C), 2个投入 (x1, x2), 1个产出 (y1)

# 方法1:从Python列表构建
x_list = [[3.0, 2.0],    # DMU_A的投入
          [4.0, 3.0],    # DMU_B的投入
          [5.0, 4.0]]    # DMU_C的投入
y_list = [[5.0],         # DMU_A的产出
          [7.0],         # DMU_B的产出
          [9.0]]         # DMU_C的产出

x_matrix = np.array(x_list)   # 投入矩阵 X (3×2)
y_matrix = np.array(y_list)   # 产出矩阵 Y (3×1)

print("投入矩阵 X:")
print(x_matrix)
print(f"\nX的维度: {x_matrix.shape}")  # (n_dmus, n_inputs)

print("\n 产出矩阵 Y:")
print(y_matrix)
print(f"\nY的维度: {y_matrix.shape}")  # (n_dmus, n_outputs)

# 方法2:从合并的数据矩阵中提取
data = np.array([
    [3.0, 2.0, 5.0],   # DMU_A: [x1, x2, y1]
    [4.0, 3.0, 7.0],   # DMU_B
    [5.0, 4.0, 9.0]    # DMU_C
```

```
])

num_inputs = 2
x_matrix = data[:, :num_inputs]   # 所有行,前2列 → 投入矩阵
y_matrix = data[:, num_inputs:]   # 所有行,第2列后 → 产出矩阵

# DEA约束的矩阵表示(产出导向 CRS: max θ)
# 投入约束: λ @ X ≤ X_eval
# 产出约束: λ @ Y ≥ θ * Y_eval
#(投入导向则为 λ @ X ≤ θ * X_eval、λ @ Y ≥ Y_eval)
lambda_vec = np.array([0.5, 0.3, 0.2])  # 强度变量

weighted_input = lambda_vec @ x_matrix   # 加权投入 (1×3 @ 3×2 = 1×2)
weighted_output = lambda_vec @ y_matrix  # 加权产出 (1×3 @ 3×1 = 1×1)

print(f"\n加权投入: {weighted_input}")
print(f"加权产出: {weighted_output}")
```

命名说明：x_matrix（或简称 x）表示投入矩阵，维度为 (n_dmus, n_inputs)；y_matrix（或简称 y）表示产出矩阵，维度为 (n_dmus, n_outputs)。x_eval表示被评价单元的投入向量，xref表示参照单元的投入矩阵。

2.5 错误处理与调试入门

DEA 程序的脆弱点通常不在语法，而在运行期——缺失值、零值、求解器未就绪、矩阵维度错位都会让一次批量评价中途崩溃。对面向真实面板数据的分析而言，稳健的错误处理与输入校验不是锦上添花，而是结果可信度的前提。

2.5.1 异常处理：try-except-finally

在DEA编程中，求解器未安装、数据维度不匹配、线性规划无可行解等情况均会导致程序崩溃。使用try-except块可以优雅地处理这些异常。

```
import numpy as np
from pyomo.environ import *

def safe_solve_ccr(x_eval, y_eval, x_ref, y_ref):
    """
    带异常处理的CCR产出导向模型求解

    Parameters
```

```
    ----------
    x_eval, y_eval, x_ref, y_ref : np.ndarray
        DEA投入产出数据

    Returns
    -------
    float or None
        技术效率TE（=1/theta),求解失败返回None
    """
    try:
        n_ref = x_ref.shape[0]
        n_inputs = x_ref.shape[1]
        n_outputs = y_ref.shape[1]

        model = ConcreteModel()
        model.I = Set(initialize=range(n_ref))
        model.K = Set(initialize=range(n_inputs))
        model.L = Set(initialize=range(n_outputs))

        model.theta = Var(bounds=(0.0, None))
        model.intensity = Var(model.I, bounds=(0.0, None))

        def obj_rule(m):
            return m.theta
        model.obj = Objective(rule=obj_rule, sense=maximize)

        def input_rule(m, k):
            return sum(m.intensity[i] * x_ref[i, k] for i in m.I) <= x_eval[k]
        model.input_con = Constraint(model.K, rule=input_rule)

        def output_rule(m, l):
            return sum(m.intensity[i] * y_ref[i,
                l] for i in m.I) >= m.theta * y_eval[l]
        model.output_con = Constraint(model.L, rule=output_rule)

        # 依次尝试可用求解器:开源优先,商业求解器(mosek)兜底
        solver = None
        for name in ('highs', 'glpk', 'cbc', 'mosek'):
            opt = SolverFactory(name)
```

```
            if opt.available():
                solver = name
                print(f"使用求解器: {name}")
                break
        if solver is None:
            raise RuntimeError("未找到可用的求解器(highs/glpk/cbc/mosek) ")

        solution = opt.solve(model)
        return 1.0 / value(model.theta)

    except ValueError as e:
        print(f"数据错误: {e}")
        print("请检查投入产出矩阵维度是否匹配")
        return None
    except Exception as e:
        print(f"求解失败: {e}")
        return None

# 测试:正常数据
x_ref = np.array([[3.0, 2.0], [4.0, 3.0], [5.0, 4.0]])
y_ref = np.array([[5.0], [7.0], [9.0]])
result = safe_solve_ccr(x_ref[0], y_ref[0], x_ref, y_ref)
print(f"技术效率: {result}")

# 测试:维度不匹配的错误数据
try:
    wrong_y = np.array([[5.0, 6.0], [7.0, 8.0]])  # 2列而非1列
    result = safe_solve_ccr(x_ref[0], wrong_y[0], x_ref, wrong_y)
except Exception as e:
    print(f"捕获异常: {e}")
```

2.5.2 DEA编程常见错误排查表

表 2.5: DEA编程常见错误与解决方案

错误现象	可能原因	解决方案
`SolverFactory('highs')/'mosek'` 报错	求解器未安装	开源 LP 模型改用`'highs'/'glpk'/'cbc'`（需 `pip install highspy` 或 conda 安装）；mosek/knitro 需申请学术许可（见附录 A.1）
`ValueError: shapes not aligned`	矩阵维度不匹配	检查 `x_ref.shape[1]`与 `x_eval.shape[0]`是否一致
`KeyError: 'Y'`	DataFrame列名不存在	检查列名拼写，使用 `df.columns` 查看所有列名
`TypeError: 'NoneType'`	函数返回None被误用	检查输入数据是否有缺失值（NaN）
效率值全为1.0	指标过多而DMU过少（维数灾难），或各单元均具独特投入产出组合，样本内无法区分	增加样本量、合并或删减指标；检查参照集是否过小（是否误把每个DMU当作自己的唯一参照）
`IndexError: list index out of range`	索引超出范围	Python索引从0开始，检查循环范围
模型无可行解或求解失败	数据含0、负值，或产出全为0	检查并处理非正值；确认模型对零值的假设
变量名 `lambda`报错	`lambda`是Python关键字	使用 `intensity`代替

2.5.3 防御性编程：输入数据检查

```
def validate_dea_data(x, y):
    """
    验证DEA投入产出数据的合法性

    Parameters
    ----------
    x : np.ndarray
        投入矩阵 (n_dmus, n_inputs)
    y : np.ndarray
        产出矩阵 (n_dmus, n_outputs)

    Returns
    -------
    bool
```

```
        数据合法返回True,否则抛出异常
    """
    # 检查是否为numpy数组
    if not isinstance(x, np.ndarray) or not isinstance(y, np.ndarray):
        raise TypeError("投入和产出数据必须是numpy.ndarray类型")

    # 检查维度
    if x.ndim != 2 or y.ndim != 2:
        raise ValueError("投入和产出数据必须是二维矩阵")

    # 检查DMU数量是否一致
    if x.shape[0] != y.shape[0]:
        raise ValueError(f"投入矩阵行数({x.shape[0]})与产出矩阵行数({y.shape[0]})不一致")

    # 检查是否有缺失值
    if np.isnan(x).any() or np.isnan(y).any():
        raise ValueError("数据中存在缺失值(NaN),请先清洗数据")

    # 检查是否有非正值(本章径向模型的约定:投入产出为正)
    if (x <= 0).any() or (y <= 0).any():
        raise ValueError("本章的径向模型要求投入产出为正数,数据中包含0或负值")

    print(f"数据验证通过: {x.shape[0]}个DMU, {x.shape[1]}个投入, {y.shape[1]}个产出")
    return True

# 使用示例
x = np.array([[3.0, 2.0], [4.0, 3.0], [5.0, 4.0]])
y = np.array([[5.0], [7.0], [9.0]])
validate_dea_data(x, y)
```

2.6 NumPy模块及其在DEA中的应用

NumPy（Numerical Python）是Python数值计算最重要的基础包，大多数科学计算包均以NumPy数组作为构建基础。在DEA编程中，NumPy用于构造投入产出矩阵、执行矩阵运算、存储效率结果。

2.6.1 NumPy模块的导入

```
import numpy as np
```

习惯上在导入NumPy包后将其重命名为 `np`。因此，代码中出现 `np.`即表示NumPy。

2.6.2 ndarray数组的创建

NumPy最重要的特点是其N维数组对象(即ndarray),它是一个快速而灵活的大数据集容器。

```
# 从列表创建数组(DEA常用)
data_list = [6, 7.5, 8, 0, 1]
arr1 = np.array(data_list)

# 创建二维数组(DEA投入产出矩阵)
data2 = [[1, 2, 3, 4],
         [5, 6, 7, 8]]
arr2 = np.array(data2)
print(arr2)

# 常用数组创建方法
arr3 = np.arange(0, 5, 1)       # [0, 1, 2, 3, 4]
arr4 = np.ones((3, 2))          # 3×2的全1矩阵
arr5 = np.zeros((3, 2))         # 3×2的全0矩阵
arr6 = np.empty((3, 2))         # 3×2的空矩阵(未初始化)
arr7 = np.random.rand(4, 2)     # 4×2的随机数矩阵
arr8 = np.linspace(0, 1, 5)     # [0, 0.25, 0.5, 0.75, 1.0]
```

2.6.3 ndarray数组的索引与切片(DEA场景)

数组索引与切片是DEA编程中最频繁的操作——提取投入矩阵、产出矩阵、被评价单元数据等。

```
data = np.array([
    [3.0, 2.0, 5.0, 10.0],   # DMU_A: [x1, x2, y1, y2]
    [4.0, 3.0, 7.0, 12.0],   # DMU_B
    [5.0, 4.0, 9.0, 14.0],   # DMU_C
    [6.0, 5.0, 11.0, 16.0],  # DMU_D
    [7.0, 6.0, 13.0, 18.0]   # DMU_E
])

# DEA关键切片操作
n_inputs = 2
x_matrix = data[:, :n_inputs]        # 所有行,前2列 → 投入矩阵 (5×2)
y_matrix = data[:, n_inputs:]        # 所有行,第2列后 → 产出矩阵 (5×2)

x_eval_a = data[0, :n_inputs]        # 第1行(DMU_A)的投入向量
y_eval_a = data[0, n_inputs:]        # 第1行(DMU_A)的产出向量
```

```
print("投入矩阵 X:")
print(x_matrix)
print("\n 产出矩阵 Y:")
print(y_matrix)
print(f"\nDMU_A的投入: {x_eval_a}")
print(f"DMU_A的产出: {y_eval_a}")
```

2.6.4 ndarray数组的属性与矩阵运算

```
arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

# 常用属性
print(f"维度: {arr.shape}")        # (3, 4) —— DEA中用于确认(n_dmus, n_vars)
print(f"数据类型: {arr.dtype}")    # int64
print(f"元素个数: {arr.size}")     # 12

# 矩阵运算(DEA核心)
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

print(a + b)           # 逐元素相加
print(a * 2)           # 标量乘法
print(a @ b)           # 矩阵乘法(DEA加权求和用)
print(a.T)             # 转置
print(np.dot(a, b))    # 矩阵乘法(等价于a @ b)

# DEA常用:矩阵乘法表示加权求和
intensity = np.array([0.2, 0.3, 0.1, 0.2, 0.2])  # 5个DMU的强度
x_matrix = np.array([[3, 2], [4, 3], [5, 4], [6, 5], [7, 6]])  # 5×2
weighted = intensity @ x_matrix  # (1×5) @ (5×2) = (1×2)
print(f"加权投入向量: {weighted}")
```

2.7 pandas模块及其在DEA中的应用

pandas提供了使数据清洗与分析更快、更简便的数据结构和操作工具。与NumPy不同，pandas专为处理表格和混杂数据而设计，而NumPy更适合处理统一的数值数组。在DEA编程中，pandas用于读取数据、组织数据集、处理面板数据、输出结果。

2.7.1 pandas模块的导入

```
import pandas as pd
```

习惯上将pandas重命名为 pd。因此，代码中出现 pd.即表示pandas。

2.7.2 Series数据结构

Series是一种类似于一维数组的对象，由一组数据（可为各种NumPy数据类型）以及一组与之对应的数据标签（即索引）组成。

```
# 从字典创建Series（DEA效率结果常用）
efficiency_data = {'DMU_A': 0.85, 'DMU_B': 1.00, 'DMU_C': 0.72,
                   'DMU_D': 0.93, 'DMU_E': 1.00}
eff_series = pd.Series(efficiency_data)
print(eff_series)

# 按索引查询
print(eff_series['DMU_A'])       # 0.85
print(eff_series[['DMU_A', 'DMU_C']])  # 批量查询
```

2.7.3 DataFrame数据结构

DataFrame是一种表格型数据结构，也是DEA数据组织的主要形式。标准DEA数据集格式为：**每行一个DMU，列分为投入与产出。**

```
# === 标准DEA数据集格式 ===
# 创建标准DEA数据集（每行一个DMU，列分投入/产出）
de_data = {
    'dmu': ['A', 'B', 'C', 'D', 'E'],
    'x1': [3.0, 4.0, 5.0, 6.0, 7.0],   # 投入1
    'x2': [2.0, 3.0, 4.0, 5.0, 6.0],   # 投入2
    'y1': [5.0, 7.0, 9.0, 11.0, 13.0]  # 产出1
}
df = pd.DataFrame(de_data)
print("标准DEA数据集:")
print(df)

# 提取投入矩阵和产出矩阵（NumPy格式，用于DEA计算）
x = df[['x1', 'x2']].values    # 投入矩阵 (5×2)
y = df[['y1']].values           # 产出矩阵 (5×1)
print(f"\n 投入矩阵维度: {x.shape}")
```

```
print(f"产出矩阵维度: {y.shape}")
```

2.7.4 DEA面板数据处理

面板数据（多个时期、多个DMU）是DEA Malmquist指数分析的基础，pandas提供了便捷的面板数据处理方法。

```
# === DEA面板数据示例 ===
# 5个DMU, 3个时期的面板数据
panel_data = {
    'dmu': ['A', 'A', 'A', 'B', 'B', 'B', 'C', 'C', 'C'],
    'year': [2020, 2021, 2022, 2020, 2021, 2022, 2020, 2021, 2022],
    'x1': [3.0, 3.2, 3.5, 4.0, 4.2, 4.5, 5.0, 5.5, 6.0],
    'x2': [2.0, 2.1, 2.3, 3.0, 3.2, 3.4, 4.0, 4.3, 4.6],
    'y1': [5.0, 5.5, 6.0, 7.0, 7.8, 8.5, 9.0, 10.0, 11.0]
}
df_panel = pd.DataFrame(panel_data)
print("面板数据:")
print(df_panel)

# 按年份分组计算各年效率
grouped = df_panel.groupby('year')
for year, group in grouped:
    x_year = group[['x1', 'x2']].values
    y_year = group[['y1']].values
    print(f"\n{year}年: {x_year.shape[0]}个DMU, "
          f"{x_year.shape[1]}个投入, {y_year.shape[1]}个产出")

# 选取特定年份的数据
df_2022 = df_panel[df_panel['year'] == 2022]
print("\n2022年数据:")
print(df_2022)

# 使用query方法筛选数据（第11章大量使用）
df_selected = df_panel.query("year==2022 and dmu in ['A', 'B']")
print("\n 筛选结果 (2022年, DMU A和B):")
print(df_selected)
```

2.7.5 使用pandas读写数据

pandas提供了一系列用于将表格型数据读取为DataFrame对象的函数。

```
# 使用本章同一电厂样本演示三种格式
from deabook import data_path
from pathlib import Path
from tempfile import TemporaryDirectory

df_csv = pd.read_csv(data_path("us_electric_plants.csv"))
with TemporaryDirectory() as temp_dir:
    temp_dir = Path(temp_dir)
    df_csv.to_excel(temp_dir / "plants.xlsx", index=False)
    df_csv.to_stata(temp_dir / "plants.dta", write_index=False, version=118)
    df_excel = pd.read_excel(temp_dir / "plants.xlsx")
    df_stata = pd.read_stata(temp_dir / "plants.dta")

# 写入数据
df.to_csv("dea_results.csv", index=False)       # 不保存行索引
df.to_excel("dea_results.xlsx", index=False)    # Excel格式

# 将DEA结果合并回原始数据
eff_df = pd.DataFrame({
    'dmu': ['A', 'B', 'C', 'D', 'E'],
    'efficiency': [0.85, 1.00, 0.72, 0.93, 1.00]
})
df_merged = df.merge(eff_df, on='dmu')  # 按dmu列合并
print("合并结果:")
print(df_merged)
```

2.7.6 loc与iloc索引（DEA关键操作）

loc使用标签索引，iloc使用位置索引，DEA编程中需要频繁在两种索引方式之间切换。

```
de_data = {
    'dmu': ['A', 'B', 'C', 'D', 'E'],
    'x1': [3.0, 4.0, 5.0, 6.0, 7.0],
    'x2': [2.0, 3.0, 4.0, 5.0, 6.0],
    'y1': [5.0, 7.0, 9.0, 11.0, 13.0]
}
df = pd.DataFrame(de_data)
```

```
# loc: 使用标签索引
df_indexed = df.set_index('dmu')        # 将dmu列设为索引
print(df_indexed.loc['A', ['x1', 'x2']])    # DMU_A的投入
print(df_indexed.loc['B':'D', 'y1'])        # DMU_B到D的产出

# iloc: 使用位置索引(从0开始)
print(df.iloc[0, 1:3])      # 第1行,第2-3列(DMU_A的投入)
print(df.iloc[1:4, 1:])     # 第2-4行,第2列以后

# DEA中的典型用法:提取被评价单元的数据
dmu_id = 'C'  # 要评价的DMU
x_eval = df_indexed.loc[dmu_id, ['x1', 'x2']].values
y_eval = df_indexed.loc[dmu_id, ['y1']].values
print(f"\n{dmu_id}的投入: {x_eval}")
print(f"{dmu_id}的产出: {y_eval}")
```

2.8 deabook 模块组成

至此，本章已备齐 DEA 编程的全部底层能力；本节起，视线从” 通用Python” 转向” 本书专属的工具包”，为随后各章逐一定位接口。

deabook 是本书配套的数据包络分析（DEA）Python 工具包，第 3–10 章的全部案例均通过它实现。它采用” 一个模型族对应一个模块” 的组织方式：每个模块对外暴露一批**函数式接口**（如 DEA.ccr、DEAweak.ddf）和对应的**面向对象类**（如 DEA.CCR、DEAweak.DDFweak）；函数式接口在后台构造并返回模型对象，调用 .optimize() 即可得到结果 DataFrame。下面按” 模型族 → 模块 → 主要接口 → 对应章节” 给出包内各模块的构成。

2.8.1 模块组成一览表

模块	主要函数（函数式接口）	主要类（面向对象）	功能定位	对应章节
deabook.DEA	ccr、bcc、hyperbolic、ite、ddf、sbm、mq、mqddf、mqfgnz、mqrd、gmqdecom、ep、epddf	CCR、BCC、Hyperbolic、ITE、DDF、_SBM	经典径向/方向距离 DEA、SBM，以及能源专项与 Malmquist 系列动态指数	第3、4、5、8 章

模块	主要函数（函数式接口）	主要类（面向对象）	功能定位	对应章节
deabook.DEAweak	ushephard、hyper、ddf、nddf、dep、depddf、lddf、sbmu	DEAweak、DEAweakRho、DDFweak、DDFweakRho、NDDFweak、NDDFweakRho	含非期望产出的环境效率（弱可处置）与动态环境生产率指数	第 6、7、8 章
deabook.DEAmb	—	MB、MBx、MBx2、MBxy、MBxy2、MB2	基于物质平衡约束的环境绩效测度	第 9 章
deabook.CNLSSDFDDFweak		CNLSSDweak、CNLSSDweakG、CNLSDDFweak、CNLSDDFweakG	含非期望产出的随机非参数前沿（Shephard / 方向距离函数）	第 10 章
deabook.CQERDFDDFweak		CQRweak、CERweak、CQRDDFweak、CERDDFweak（及其 G 变体 CQRweakG、CERweakG、CQRDDFweakG、CERDDFweakG）	含非期望产出的凸非参数分位数/期望回归	第 10 章
deabook.StoNED	—	StoNED、StoNEDContext	CNLS 前沿上的随机非参数效率恢复（JLMS 条件无效率）	第 10 章
deabook.PANEL	tech（面板求解器构造器 PANEL.tech(...)，等价于面向对象类 PANEL.tech）	tech（PANEL.tech，Panel 子类，作为构造器）、Panel（PANEL.Panel，基础面板类）、StoNEDPanel（PANEL.StoNEDPanel）	面板数据估计器：把任意静态效率工厂（如 DEA.ccr、DEAweak.ddf）注入后，按所选参照技术（tech=CONTEMPORARY / GLOBAL / SEQUENTIAL / WINDOW）在各 DMU×各期上循环求解	第 3–10 章

模块	主要函数（函数式接口）	主要类（面向对象）	功能定位	对应章节
deabook.constant	—	—	全局常量：规模报酬 RTS_CRS/RTS_VRS/RTS_VRS1/RTS_VRS2、复合误差 CET_ADDI/CET_MULT、前沿类型 FUN_PROD/FUN_COST、求解方式 OPT_LOCAL/OPT_DEFAULT、参照技术（前沿技术） CONTEMPORARY/SEQUENTIAL/WINDOW/GLOBAL、导数方向 LEFT/RIGHT	全书

说明：除上表所列”对外”模块外，deabook 还包含若干**内部支撑模块**，它们不直接面向读者调用，而是被上述模块在底层复用——core（数据解析 parse_sent/prepare_dea_data 与 CNLS 求解）、mqcore（Malmquist 引擎 MalmquistModel）、utils.tools（数据校验 assert_* 与物质平衡拆分 split_MB）。

2.8.2 在书中如何导入

各章开头的导入语句与上述模块一一对应，例如：

```
from deabook import DEA, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook import DEAweak, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook import DEAmb, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook import CNLSSDFDDFweak, StoNED, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook.CQERDFDDFweak import CQRweak, CQRDDFweak  # 第 10 章：凸分位数回归
from deabook.constant import RTS_CRS, RTS_VRS1, CET_ADDI, FUN_PROD, OPT_LOCAL
```

掌握这张”模块—章节”地图后，读者即可按章节定位到对应的 deabook 接口，跟随正文逐章展开。

2.9 从 Python 到 deabook：DEA 程序标准工作流（预览）

上一节的”模块—章节”地图告诉读者每个模型族落在何处；本节则把分散的接口收束为一条贯穿全书的统一调用流程，作为连接本章基础与后续方法论章节的最后一座桥。

第 3 章起，本书所有案例都通过 deabook 工具包完成，而不再手写Pyomo 模型（那种”从零搭建”的写法留待第 11–13 章讲解）。理解下面这条**标准四步工作流**，就能读懂后续每一章的代码骨架：

1. **载入数据**：投入、产出（以及含非期望产出模型中的非期望产出）放进同一个 `pandas.DataFrame`，每行一个决策单元（DMU）。
2. **用 sent 声明变量角色**：`sent` 是一个字符串，用`=` 分隔”投入”与”产出”，多个变量以空格分隔；当模型含非期望产出时，用`:` 引出非期望产出部分，例如 `"K L E=Y:CO2"` 表示投入为 `K`，`L`，`E`、期望产出为 `Y`、非期望产出为 `CO2`。
3. **调用函数式接口构造模型**：如 `DEA.ccr(...)`、`DEAweak.ddf(...)`，后台返回模型对象（与之等价的面向对象类为 `DEA.CCR`、`DEAweak.DDFweak`，二者可互换）。
4. **`.optimize()` 求解并取结果**：调用 `model.optimize(solver="highs")`，返回一个 `DataFrame`，效率值通常位于 `rho` 列。

```
import pandas as pd
from deabook import DEA, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook.constant import RTS_CRS

# 1) 美国19家电厂：同一数据，仅为函数调用重命名列
from deabook.example.book_cases.loaders import plants
data = plants().rename(columns={"Plant":"firm", "Output":"output", "Capital":"capital", "Labor":"labour

# 2) sent 声明变量角色："投入=产出"（多变量以空格分隔）
sent = "capital labour=output"

# 3) 函数式接口构造模型（后台返回模型对象）
model = DEA.ccr(data, sent=sent, orient="oo", rts=RTS_CRS)

# 4) .optimize() 求解（开源 LP 求解器 highs / cbc / glpk），返回结果 DataFrame
res = model.optimize(solver="highs")
print(res[["rho"]])
```

不同模型族的细微差异（后续章节会逐一展开）：- **含非期望产出**（第 6–8 章）的 `sent` 用 `:` 引出非期望产出，例如 `DEAweak.ddf(data, sent="K L E=Y:CO2", rts=RTS_CRS)`。- **动态 / 面板模型**（`mq`、`dep`、`lddf` 等Malmquist 类指数）的数据还需 `id` 与 `year` 两列，调用时额外传入 `id="firm"`、`year="year"`，`.optimize()` 返回的是跨期指数分解。- **物质平衡模型**（第 9 章 `DEAmb.MB`）在 `sent` 中用 `+` 区分产生污染的投入变量，例如 `"K L+F=E:CO2"`。- 第 10 章的 CNLS 与 CQR 属于随机 / 分位数前沿，需要 mosek/knitro 或 amplpy 等求解器，调用方式略有不同，详

见对应章节。

2.10 案例应用:读取并检查美国电厂数据

本章只用美国19家电厂练习数据读取、选列、数组转换和检查。Labor、Fuel、Capital分别为年平均就业人数、燃料消耗(10^10 Btu)和装机容量(MW),Output为发电量(10^6 kWh)。四列均严格为正且无缺失。表中按变量列出范围,图中各列除以自身中位数仅为比较离散程度;DEA计算仍使用原值或按列正比例换单位后的值,不做中心化。

```
from deabook.example.book_cases.run import FUN
FUN[2]()  # 重新计算本章,输出完整CSV、摘要表与PNG/PDF图
```

表 2.7: 读取并检查美国电厂数据:结果摘要

统计量	劳动	燃料	装机	发电量
有效样本数	19	19	19	19
均值	218.947	3106.814	785.453	2884.411
标准差	118.550	2502.175	555.111	2392.710
最小值	94.000	540.610	182.300	457.200
P10	95.800	865.562	198.920	738.340
中位数	182.000	2307.500	650.100	1961.000
P90	383.200	6648.100	1701.440	5980.040
最大值	500.000	9509.200	1892.100	9539.900

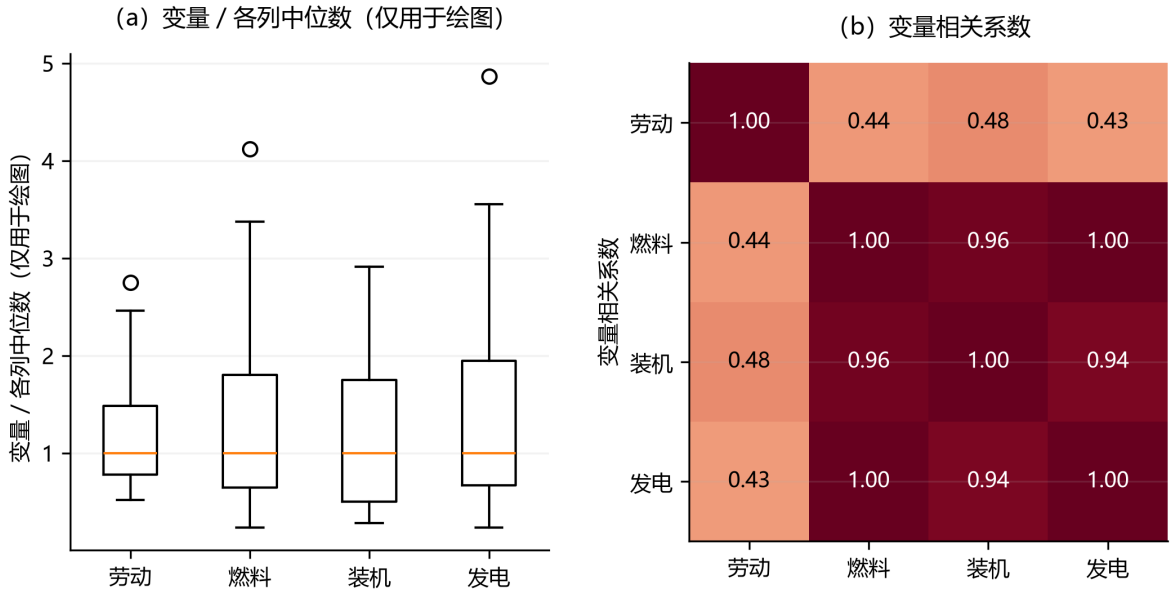


图 2.1: 读取数据后比较四个变量的相对离散程度

表中报告四个原始变量的有效样本数、均值、标准差及分位数，供读者检查数据尺度和极端值。图左比较各列除以自身中位数后的离散程度，保留离群点；图右显示四个变量的样本相关系数。相关性高不代表变量在生产技术中可以互相替代，绘图缩放也不改变原始数据。

运行时先核对行数、列名和正值，再把三个投入列与一个产出列转为NumPy数组。图中的缩放不写回原始数据；变量量纲不能通过中心化处理。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch02_full.csv`。表中P10和P90为第10和第90百分位数（如适用），展示分布而不占用逐个决策单元的行数。

2.11 本章小结

本章从” 面向 DEA 的 Python 准备” 视角出发，逐层铺垫了后续建模所需的底层能力：先是 Anaconda与Jupyter 的开发环境、 Python 语法精粹（列表、字典、函数与控制结构），再到面向 DEA 的编程范式——向量化思维、函数封装与矩阵构建这三块基石，以及错误处理、防御性编程与常见错误排查；随后用NumPy与 pandas 把上述能力落到 DEA 数据上，覆盖数据矩阵构造、标准数据集格式与面板数据处理。前面的小节 2.8 给出了 `deabook` 工具包的” 模块—章节” 地图，小节 2.9 则提炼出贯穿全书的” 数据 → `sent` → 接口 → `.optimize()`“四步工作流。

第 3 章

数据包络分析的经典模型

数据包络分析（Data Envelopment Analysis）是一种基于线性规划的效率评价方法，用于评估生产或服务系统的效率和生产率水平。自Charnes, Cooper and Rhodes (1978) 提出以来，DEA 在经济效率估计中得到广泛应。例如，Färe, Grosskopf and Lee (1990) 分析了加州 82 个水稻农场，同年 Ferrier and Lovell (1990) 将其用于银行业，随后 Sueyoshi (1991)、Chavas and Aliber (1993)、Ray and Kim (1995)、Yunos and Hawdon (1997) 分别考察了美国AT&T、美国农场、美国钢铁行业与马来西亚国家电力局。

1998 年后，该方法的覆盖面进一步扩展。Ray, Seiford and Zhu (1998)、Goto and Tsutsui (1998)、Linna (1998) 分别将其应用于中国钢铁、日美电力与芬兰卫生部门。Camanho, Silva and Piran (2024) 的综述显示，1999–2020 年间相关论文数量持续上升，应用涵盖制造业、食品、农业、卫生、能源、交通、电信、教育等领域。

与传统参数方法相比，DEA 模型具有以下优点：

（1）不需要对生产函数进行假设：传统参数方法需预设生产函数形式，若函数形式与实际不符，估计可能产生偏误；DEA 则不作此类假设，可直接基于实际投入产出数据评价。

（2）能够评价多种输入和输出因素：DEA 模型可同时处理多种投入与多种产出，适用于联合生产与竞争性生产情形。

（3）不依赖需求曲线和价格信息：DEA 模型无需需求曲线与价格信息，可在缺乏这些数据时评价效率，对新兴或市场经验不足的企业尤其适用。

在实际应用中，DEA 模型广泛用于评估企业、行业、地区和国家等各类经济单位的效率水平。例如，可用于评估单一企业的生产效率与要素利用情况，识别效率提升空间；也可对比不同企业的最优生产方式以指导决策；此外还可评价地区或国家的经济发展水平，以及行业的竞争力与发展趋势。

本章围绕经典 DEA 模型展开。3.1 节用技术集、产出集与投入集三种等价形式刻画生产技术；3.2 节定义产出距离、投入距离、hyperbolic 距离与方向距离四类效率测度；3.3 节介绍对应的 DEA 线性规划与效率指数；3.4 节讨论面板数据下技术参照集的构造，给出当期、Window、时序与全局四类参照技术的数学表达式；3.5 节介绍 Python 程序包 `deabook` 的统一调用接口；3.6 节以OECD 国家与中国省份的技术效率为例进行应用演示。

3.1 生产技术的设定

效率测度可以分为两步：第一步刻画生产技术并确定前沿形状，第二步选定距离函数度量实际生产与前沿的偏离。不同的距离函数对应不同的评估标准，因而得到不同的效率衡量。

对于生产过程中投入与产出之间的关系，我们可以从三个角度描述生产技术：技术集、产出集和投入集。这三种表征方式在数学意义上等价，只是观测视角不同。

3.1.1 技术集

考虑一个使用 K 种投入要素 $x=(x_1,x_2,\cdots,x_K)^{\mathrm{T}}\in\mathfrak{R}_+^K$ 生产 L 种产出 $y=(y_1,y_2,\cdots,y_L)^{\mathrm{T}}\in\mathfrak{R}_+^L$ 的生产过程。生产技术可用集合 T 表示为：

$$T=\{(x,y):x\text{ 能生产 }y\} \tag{3.1}$$

技术集包含了生产技术的前沿和生产技术内部的所有投入产出组合。也就是说，所有观察值都满足 $(x,y)\in T$。该生产技术满足如下标准假设：

(1) 对于任意 x，$(x,0)\in T$；

(2) 若 $(x,y)\in T$，且 $x=0$，则 $y=0$；

(3) 若 $(x,y)\in T$，且 $y'\leqslant y$，则 $(x,y')\in T$；

(4) 若 $(x,y)\in T$，且 $x^{'}\geqslant x$，则 $\left(x^{'},y\right)\in T$；

(5) 封闭性；

(6) 凸性；

其中，(1) 表明在有投入的情况下可以没有产出；(2) 表明没有投入便没有产出；(3) 和 (4) 分别为产出和投入的强可处置性：一定投入所能生产的产出组合，也一定能生产其任意更小的产出（产出强可处置性）；能生产一定产出的投入组合，使用更多投入同样可行（投入强可处置性）。

(5) 表明技术集合是封闭的，即包含自身的边界。需要注意的是，标准公理只要求 T **封闭**，而**有界性**是针对给定投入下的产出集 $P(x)$ 而言的（稀缺性：有限的投入不能生产无限的产出，见 3.1.2 节性质 (5)）。T 本身一般不有界：在 crs 假设下 T 是一个锥，沿射线方向无界；投入强可处置时投入也可任意加大。(6) 的凸性起到扩大技术集的作用，在只有少数观测数据可用时尤为重要：若两个生产计划均可行，通常也假定二者的加权平均可行。

在前面6个假设的基础上，文献还通常加上规模报酬的技术假设。例如，规模报酬不变（crs）指 $(x,y)\in T\Rightarrow\theta(x,y)\in T$ 对任意 $\theta>0$ 成立，即投入产出可按同一比例任意放大或缩小；规模报酬可变（vrs）则由观测点的凸组合生成技术，不允许对观测点任意缩放，对应 3.3 节 LP 中的约束 $\sum_{i=1}^{N}\lambda_i=1$。技术集 T 是定义同时调整投入与产出类测度（hyperbolic 距离函数、方向距离函数）的自然基础，将在 3.2.3、3.2.4 节用到。

3.1.2 产出集

生产技术还可以使用产出集合来表示。产出集表示给定投入向量 x 所能生产的全部产出向量 y 的集合，数学上可定义为：

$$P(x)=\{y:x\text{ 能生产 }y\}=\{y:(x,y)\in T\} \tag{3.2}$$

产出集的性质满足以下标准假设：

(1) 可以在有投入的情况下而没有产出：对于任意 x，$0 \in P(x)$；

(2) 没有投入就没有产出：若 $y \in P(x)$，且 $x = 0$，则 $y = 0$；

(3) 产出的强可处置性：若 $y \in P(x)$，且 $y' \leqslant y$，则 $y^{'} \in P(x)$；

(4) 投入的强可处置性：若 $y \in P(x)$，且 $x' \geqslant x$，则 $y \in P(x^{'})$；

(5) $P(x)$ 是封闭有界的（有界性即稀缺性：有限的投入不能生产无限的产出）；

(6) $P(x)$ 是凸的。

其中，（6）要求：若两种产出水平均可被生产出来，则其任意加权平均也可以被生产出来。产出集 $P(x)$ 把视角聚焦于" 给定投入下的产出可能集"，是 3.2.1 节产出距离函数的定义基础。

3.1.3 投入集

生产技术也可以使用投入集合来表示。投入集表示生产给定产出向量 y 所需要的所有投入向量 x 的集合，数学上可定义为：

$$L(y) = \{x : x \text{ 能生产 } y\} = \{x : (x, y) \in T\} \tag{3.3}$$

投入集的性质满足以下标准假设：

(1) 可以在有投入的情况下而没有产出：$0 \in L(0)$；

(2) 没有投入就没有产出：若 $x \in L(y)$，且 $x = 0$，则 $y = 0$；

(3) 产出的强可处置性：若 $x \in L(y)$，且 $y' \leqslant y$，则 $x \in L(y^{'})$；

(4) 投入的强可处置性：若 $x \in L(y)$，且 $x' \geqslant x$，则 $x' \in L(y)$；

(5) $L(y)$ 是封闭的（注意：投入集一般不有界——投入强可处置时可行投入可任意加大，“有限投入不能生产无限产出" 是对产出集 $P(x)$ 的要求）；

(6) $L(y)$ 是凸的。

其中，（6）要求：若两种投入组合均可生产给定产出，则其任意加权平均也可用于生产该产出。投入集 $L(y)$ 对应" 给定产出下的投入需求集"，是 3.2.2 节投入距离函数的定义基础。

3.2 效率的测量

技术效率的概念可以追溯到Debreu (1951) 和 Farrell (1957)。他们最先提出效率的定义方法：在固定投入下测量产出的最大扩张比例，或在固定产出下测量投入的最大收缩比例，借助距离函数估计效率。随后，Färe, Grosskopf and Lovell (1985) 提出同时缩小投入和扩大产出的 hyperbolic 效率测度。在此基础上，研究者发展了距离函数的一般形式——方向距离函数，以实现对效率更为广义的衡量。下面依次定义产出距离、投入距离、hyperbolic 距离与方向距离四类测度。

3.2.1 产出距离函数

产出距离函数（Shephard, 1970）度量给定投入下产出的最大扩张程度，其通过产出集定义：

$$D_O(x, y) = \inf\{\theta : y/\theta \in P(x)\} \tag{3.4}$$

式中，θ 是产出距离函数的值。当 $\theta = 1$ 时，表示被评价的决策单元位于生产前沿上；当 $\theta < 1$ 时，表示被评价的决策单元位于生产前沿内部，产出有扩张的空间。

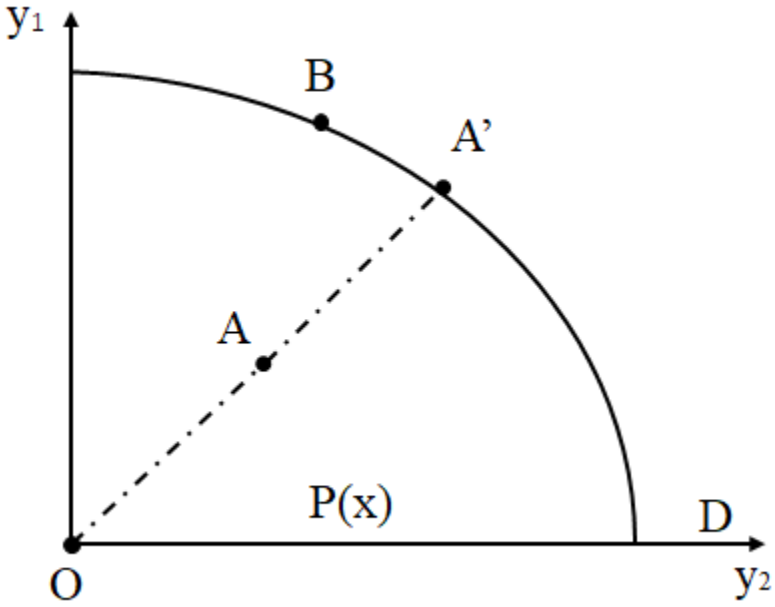


图 3.1: 产出距离函数示意图

如 图 3.1 所示，曲线为生产前沿，给定投入水平下，被评价 DMU 的实际产出 y 位于前沿内部。沿产出方向将 y 径向扩张至前沿上的 y/θ（θ 为产出距离函数值），扩张比例 $1/\theta$ 即度量了产出的无效程度。$\theta = 1$ 表示 DMU 已在前沿上。

易知，产出距离函数具有以下性质：

(1) 对所有非负投入 x，$D_O(x, 0) = 0$；
(2) $D_O(x, y)$ 在 y 上非递减，在 x 上非递增；
(3) $D_O(x, y)$ 在 y 上是线性齐次的；
(4) $D_O(x, y)$ 在 x 上是拟凸的，在 y 上是凸的；
(5) 生产集上的决策单元的产出距离函数的取值范围为 $[0, 1]$；且生产前沿上的决策单元的产出距离函数为 1。

3.2.2 投入距离函数

与产出距离函数对偶，投入距离函数刻画给定产出下投入的最大可缩减比例，从投入缩减的视角衡量效率。

（一）所有投入同时变化：投入距离函数通过投入集定义：

$$D_I(x, y) = \sup\{\phi : x/\phi \in L(y)\} \tag{3.5}$$

式中，ϕ 是投入距离函数的值。当 $\phi = 1$ 时，表示被评价的决策单元位于生产前沿上；当 $\phi > 1$ 时，表示被评价的决策单元位于生产前沿内部，投入有缩小的空间。

如 图 3.2 所示，在固定产出水平下，被评价 DMU 的实际投入 x 位于前沿外。沿投入方向将 x 径向缩减至前沿上的 x/ϕ（ϕ 为投入距离函数值），缩减比例 ϕ 即度量了投入的无效程度。$\phi = 1$ 表示 DMU 已

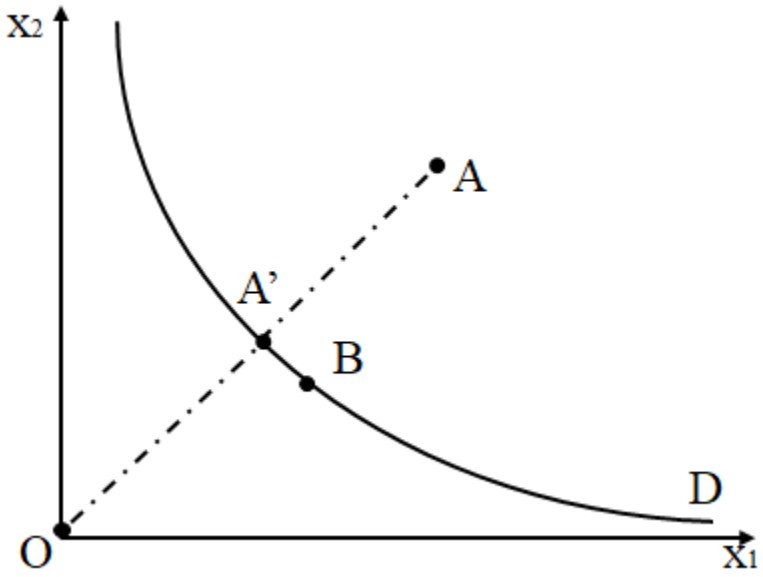


图 3.2: 投入距离函数示意图

在前沿上。

投入距离函数具有以下性质：

(1) 对所有非负产出 y，$D_I(0,y)=0$；

(2) $D_I(x,y)$ 在 y 上非递增，在 x 上非递减；

(3) $D_I(x,y)$ 在 x 上是线性齐次的；

(4) $D_I(x,y)$ 在 y 上是拟凹的，在 x 上是凹的；

(5) 生产集上的决策单元的投入距离函数的取值范围为 $[1,\infty)$；且生产前沿上的决策单元的投入距离函数为 1。

（二）单个投入变化：投入距离函数也可只对单个投入定义，用于考察特定投入的利用效率。以资本为例，若只关心资本这一项投入的利用状况，可把投入向量拆分为资本 k 与其他投入 x，只让资本沿投入方向径向缩减，其余投入与产出保持不变。

具体而言，谢泼德资本距离函数度量给定产出与其他投入下资本投入的最大缩减程度，其通过投入集定义：

$$D_I(x,k,y)=\sup\{\phi:(x,k/\phi)\in L(y)\} \tag{3.6}$$

式中，x 代表其他投入，k 代表资本投入，y 代表产出，ϕ 是谢泼德资本距离函数的值。当 $\phi=1$ 时，表示被评价的决策单元位于生产前沿上；当 $\phi>1$ 时，表示被评价的决策单元位于生产前沿内部，资本投入有缩小的空间。

与 (3.5) 让全部投入等比例缩减不同，(3.6) 只让资本 k 按比例缩减、其余投入固定不动，因此 ϕ 衡量的是**资本利用的无效程度：**在维持现有产出与其他投入不变的前提下，资本最多可压缩至当前的 $1/\phi$。图 3.3 给出了对应的几何示意：被评价单元位于 A 点（资本 k_A、其他投入 x_A），生产前沿（产出 y 对应的等产量面）是一条过原点单调递减的曲线；保持其他投入 x_A 不变，将资本沿水平方向向左投影到前沿上的 A' 点（资本 $k_{A'}$），则 $\phi=k_A/k_{A'}$，相应的资本利用效率为 $1/\phi=k_{A'}/k_A$。当 $A=A'$（即 $k_A=k_{A'}$）时 $\phi=1$，

资本利用是有效的。

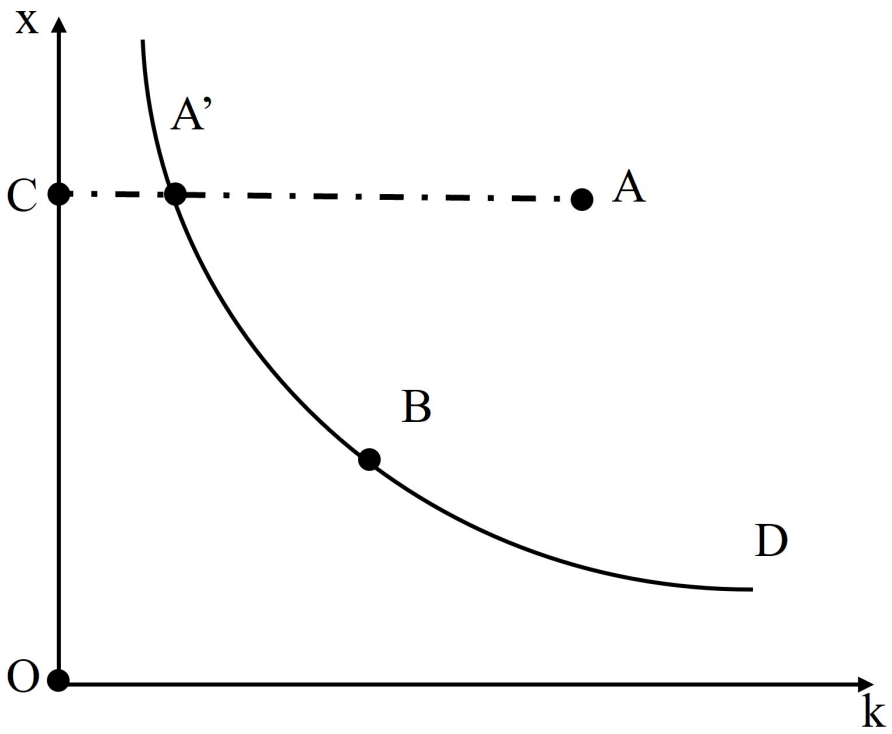


图 3.3: 资本方向谢泼德距离函数示意图

3.2.3 同时缩小投入和扩张产出的 Hyperbolic 距离函数

传统的效率衡量方法往往单独关注扩张产出或缩小投入，而实际生产中通常需要同时考虑二者，hyperbolic 距离函数即基于技术集对此作综合度量。

$$G(x,y) = \inf\{G : (Gx, y/G) \in T\} \tag{3.7}$$

式中，G 是 hyperbolic 距离函数的值。当 $G = 1$ 时，表示被评价的决策单元位于生产前沿上；当 $G < 1$ 时，表示被评价的决策单元位于生产前沿内部，有缩小投入和扩大产出的空间。

如 图 3.4 所示，hyperbolic 距离函数同时沿投入缩减与产出扩张两个方向移动：投入乘以 G、产出除以 G，沿一条双曲路径逼近生产前沿。$G = 1$ 表示 DMU 已在前沿上，$G < 1$ 表示存在同时缩减投入与扩张产出的空间。这一测度自然引出更一般的方向距离函数（见 3.2.4 节）。

3.2.4 方向距离函数

Hyperbolic 距离函数要求投入与产出按同一比例 G 反向调整，方向固定。方向距离函数（Directional Distance Function, DDF）则允许通过方向向量 (g_x, g_y) 自由设定改进方向，把谢泼德投入距离与产出距离作为精确特例包含在内。根据Chambers, Chung and Färe (1998) 的定义，同时缩小投入和扩张产出的方向距

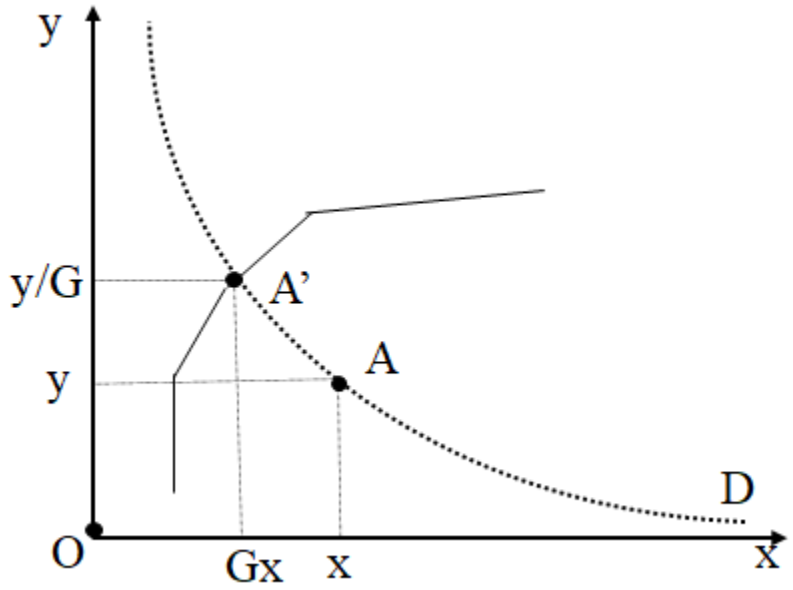


图 3.4: hyperbolic 距离函数示意图

离函数可表示为：

$$\vec{D}\left(x,y;g_x,g_y\right)=\sup\left\{\beta:\left(x-\beta g_x,y+\beta g_y\right)\in T\right\} \tag{3.8}$$

式中，T 为技术集；$g_x=\left(g_{x_1},g_{x_2},\cdots,g_{x_K}\right)^{\mathrm{T}}\in\mathfrak{R}_+^K$，$g_y=\left(g_{y_1},g_{y_2},\cdots,g_{y_L}\right)^{\mathrm{T}}\in\mathfrak{R}_+^L$ 用于设定被评价决策单元的生产改进方向；β 是方向距离函数的取值，表示扩张期望产出同时缩减投入的最大比例。

方向距离函数满足以下性质：

(1) 非负性：若 $(x,y)\in T$, 则 $\vec{D}\left(x,y;g_x,g_y\right)\geqslant 0$；

(2) 平移性：$\vec{D}\left(x-\alpha g_x,y+\alpha g_y;g_x,g_y\right)=\vec{D}\left(x,y;g_x,g_y\right)-\alpha$；

(3) 在方向向量上的负一次齐次性：$\vec{D}\left(x,y;\lambda g_x,\lambda g_y\right)=(1/\lambda)\vec{D}\left(x,y;g_x,g_y\right),\lambda>0$；

(4) 如果 $y'\geqslant y$, 则 $\vec{D}\left(x,y;g_x,g_y\right)\geqslant\vec{D}\left(x,y';g_x,g_y\right)\geqslant 0$；

(5) 如果 $x'\geqslant x$, 则 $\vec{D}\left(x,y;g_x,g_y\right)\leqslant\vec{D}\left(x',y;g_x,g_y\right)\geqslant 0$；

(6) 如果 T 是凸的, 那么 $\vec{D}\left(x,y;g_x,g_y\right)$ 对于 (x,y) 是凹的。

其中，性质 (4) 与 (5) 表明：期望产出增大时，方向距离函数值减小；投入增大时，方向距离函数值增加。二者均反映方向距离函数值随无效程度的上升而增大。性质(6) 表明，当技术集为凸时，方向距离函数是凹函数。

以 图 3.5 所示的单投入单产出生产函数为例。当取方向向量为 $g=(1,1)$ 时，A 点将沿 $(-1,1)$ 的方向投影至 A' 点。当方向向量取特定值时，方向距离函数可以精确地表为谢泼德距离函数。当取方向向量为 $g=(x,0)$ 时，投入方向距离函数与方向距离函数之间的关系为 $\vec{D}(x,y;x,0)=1-1/D_I(x,y)$。当方向向量为 $g=(0,y)$ 时，产出方向距离函数与方向距离函数之间的关系为 $\vec{D}(x,y;0,y)=1/D_O(x,y)-1$。这两条关系都是恒等式。

hyperbolic 不是 DDF 的特例：hyperbolic 测度的投影路径是双曲线 $(Gx,y/G)$，而 DDF 的投影路径是直线 $(x-\beta g_x,y+\beta g_y)$，二者不是同一条路径，因此hyperbolic 测度不能写成 DDF 的特例。

取 $g=(x,y)$ 时，只有在 β 很小时才近似有 $G(x,y)\approx 1/(1+\vec{D}(x,y;x,y))\approx 1-\vec{D}(x,y;x,y)$，这是一阶近似而非恒等式。Färe, Margaritis, Rouse and Roshdi (2016) 正是利用双曲距离与 DDF 的联系，给出用至多两个线性规划**精确**计算双曲距离函数的算法（尤其解决vrs 下双曲问题非线性、难以直接线性化的困难）。

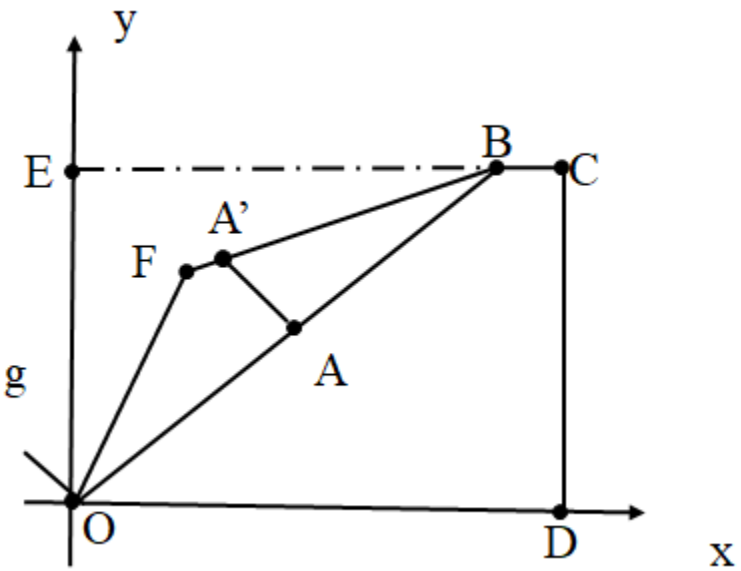


图 3.5: 方向距离函数示意图

3.3 DEA 模型的估计

在 3.1、3.2 节刻画生产技术并定义各类距离函数之后，本节将其转化为可求解的 DEA 线性规划。DEA 模型无需对生产前沿的函数形式做出参数假设，可直接基于实际投入产出数据估计各决策单元相对于前沿的效率。

3.3.1 基于产出距离函数的技术效率估计

产出导向在固定投入下径向扩张产出。需注意，本节 LP 中的决策变量 ϕ 是产出扩张倍数（$\phi\geqslant 1$），与 3.2.1 中产出距离函数 $\theta\in[0,1]$ 互为倒数：$\phi=1/\theta=1/D_O$。本节统一记号：下标 0 标识被评价 DMU，$i=1,\dots,N$ 标识参考集中的 DMU，λ_i 为强度变量。可用下面的 DEA 程序估计 DMU_0 的效率：

$$
\begin{aligned}
&\max_{\phi,\lambda_1,\dots,\lambda_N}\ \phi\\
&\text{s.t.}\ \sum_{i=1}^{N}\lambda_i x_{ki}\leq x_{k0},\quad k=1,\dots,K\\
&\sum_{i=1}^{N}\lambda_i y_{li}\geq \phi y_{l0},\quad l=1,\dots,L\\
&\lambda\in\Lambda^N(\gamma).
\end{aligned}
\tag{3.9}
$$

式中，γ 表示不同的规模报酬假设，$\gamma \in \{crs, vrs\}$，$\Lambda^N(\gamma)$ 隐含 $\lambda_i \geqslant 0$。当 $\gamma = crs$ 时，对应规模报酬不变的生产技术，即著名的 CCR 模型（Charnes, Cooper and Rhodes, 1978 ）。相应的 DEA 程序为：

$$\begin{aligned} &\max_{\phi,\lambda_1,\ldots,\lambda_N} \phi \\ &\text{s.t.} \sum_{i=1}^{N} \lambda_i x_{ki} \leq x_{k0}, \quad k = 1, \ldots, K \\ &\sum_{i=1}^{N} \lambda_i y_{li} \geq \phi y_{l0}, \quad l = 1, \ldots, L \\ &\lambda_i \geq 0. \end{aligned} \tag{3.10}$$

当 $\gamma = vrs$ 时，对应规模报酬可变的生产技术，即著名的 BCC 模型 (Banker, Charnes and Cooper, 1984)。相应的 DEA 程序为：

$$\begin{aligned} &\max_{\phi,\lambda_1,\ldots,\lambda_N} \phi \\ &\text{s.t.} \sum_{i=1}^{N} \lambda_i x_{ki} \leq x_{k0}, \quad k = 1, \ldots, K \\ &\sum_{i=1}^{N} \lambda_i y_{li} \geq \phi y_{l0}, \quad l = 1, \ldots, L \\ &\sum_{i=1}^{N} \lambda_i = 1, \\ &\lambda_i \geq 0. \end{aligned} \tag{3.11}$$

以上模型在保持投入不变的情况下，径向扩张所有产出。若目标函数值为 ϕ^*，则最佳期望产出量为 $\phi^* y$，纯产出效率（PEO）为：

$$PEO = \frac{y}{\phi^* y} = \frac{1}{\phi^*}.$$

仅当 $\phi^* = 1$ 时，$PEO = 1$，代表产出位于生产前沿上；否则，当 $\phi^* > 1$ 时，$PEO < 1$，表明产出效率存在提升空间。

3.3.2 基于投入距离函数的技术效率估计

与产出导向对偶，投入导向在固定产出下径向缩减投入。此处 LP 的决策变量 θ 是投入收缩比（$\theta \leqslant 1$），与 3.2.2 中投入距离函数 $\phi \in [1, \infty)$ 互为倒数：$\theta = 1/\phi = 1/D_I$。注意勿将本式中的 θ 与(3.4) 中的产出距离 θ 混淆。可用下面的 DEA 程序估计 DMU_0 的效率：

$$\begin{aligned} &\min_{\theta,\lambda_1,\ldots,\lambda_N} \theta \\ &\text{s.t.} \sum_{i=1}^{N} \lambda_i x_{ki} \leq \theta x_{k0}, \quad k = 1, \ldots, K \\ &\sum_{i=1}^{N} \lambda_i y_{li} \geq y_{l0}, \quad l = 1, \ldots, L \\ &\lambda \in \Lambda^N(\gamma). \end{aligned} \tag{3.12}$$

式中，γ 表示不同的规模报酬假设，$\gamma \in \{crs, vrs\}$。以上模型在保持产出不变的情况下，径向缩减所有投入。若目标函数值为 θ^*，则最佳投入量为 $\theta^* x$，纯投入效率（PEI）为

$$PEI = \frac{\theta^* x}{x} = \theta^*.$$

当且仅当 $\theta^* = 1$ 时，$PEI = 1$，表示投入利用是有效的；否则，当 $\theta^* < 1$ 时，$PEI < 1$，表明投入效率存在提升空间。

若只关心某一项投入（如资本）的利用状况，则可沿用 (3.12) 的框架，但**只让该项投入按比例收缩、其余投入与产出保持不变**。设被考察的投入为 x_1，其 DEA 程序为：

$$\begin{aligned}
&\min_{\theta,\lambda_1,\dots,\lambda_N} \theta \\
&\text{s.t.} \sum_{i=1}^{N} \lambda_i x_{1i} \le \theta x_{10} \\
&\sum_{i=1}^{N} \lambda_i x_{ki} \le x_{k0}, \quad k = 2, \dots, K \\
&\sum_{i=1}^{N} \lambda_i y_{li} \ge y_{l0}, \quad l = 1, \dots, L \\
&\lambda \in \Lambda^N(\gamma).
\end{aligned} \tag{3.13}$$

与 (3.12) 让全部投入等比例缩减不同，(3.13) 只沿**单个投入**方向收缩，对应 3.2.2.2 节定义的单投入谢泼德距离函数(3.6)。若最优值为 θ^*，则该项投入的利用效率为

$$PEI_1 = \theta^*,$$

对应的谢泼德单投入距离函数为 $D_I = 1/\theta^*$。当且仅当 $\theta^* = 1$ 时，$PEI_1 = 1$，表示该项投入已无压缩空间；否则 $\theta^* < 1$ 表明该项投入存在节约潜力。由于其余投入被固定，(3.13) 的效率值不会低于全投入情形 (3.12) 的 θ^*——约束更少，可行域更大。

3.3.3 基于双曲距离函数的技术效率估计

产出导向与投入导向分别只在单侧（产出或投入）做径向调整。若希望 DMU 沿双曲路径同时缩减投入并扩张产出，则需引入 Hyperbolic 距离函数。在 crs 技术中，参考 Färe, Grosskopf and Lovell (1985)，可用下面的 DEA 程序估计同时调整投入和产出的 DMU_0 的效率：

$$\begin{aligned}
&\min_{\eta,\lambda_1,\dots,\lambda_N} \eta \\
&\text{s.t.} \sum_{i=1}^{N} \lambda_i x_{ki} \le \eta x_{k0}, \quad k = 1, \dots, K \\
&\sum_{i=1}^{N} \lambda_i y_{li} \ge \frac{y_{l0}}{\eta}, \quad l = 1, \dots, L \\
&\lambda_i \ge 0.
\end{aligned} \tag{3.14}$$

该非线性问题可按下述方法线性化：在每个约束中乘 η，并令 $\eta^2 = \eta'$，$\lambda_i \eta = \lambda_i'$（由 $\eta > 0$ 保证代换等价），得到下面的 LP 模型：

$$\begin{aligned}
&\min_{\eta',\lambda'_1,\ldots,\lambda'_N} \eta' \\
&\text{s.t.} \sum_{i=1}^{N} \lambda'_i x_{ki} \le \eta' x_{k0}, \quad k=1,\ldots,K \\
&\sum_{i=1}^{N} \lambda'_i y_{li} \ge y_{l0}, \quad l=1,\ldots,L \\
&\lambda'_i \ge 0.
\end{aligned} \tag{3.15}$$

求解上述线性规划模型，若目标函数值为 η'^*，由代换 $\eta^2=\eta'$ 得 $\eta^*=\sqrt{\eta'^*}$，则混合投入效率（MEI）或混合产出效率（MEO）为：

$$MEI = MEO = \eta^* = \sqrt{\eta'^*}.$$

当且仅当 $\eta^*=1$ 时，$MEI=MEO=1$，表示产出或投入利用有效；否则，当 $\eta^*<1$ 时，$MEI=MEO<1$，表明存在提升空间。

对于 vrs 技术，(3.14) 的上述代换不再成立（$\sum_i \lambda_i=1$ 在代换后变为 $\sum_i \lambda'_i=\eta$），因而无法用同样的方法线性化。一种常见的折中做法是用一阶（泰勒）近似，把双曲路径近似为方向向量 $g=(x,y)$ 的方向距离函数，从而求解下面的 LP：

$$\begin{aligned}
&\max_{\lambda_1,\ldots,\lambda_N,\beta} \ \beta \\
\text{s.t.} \quad &\textstyle\sum_{i=1}^{N} \lambda_i y_{li} \geqslant y_{l0}+\beta g_{ly}, l=1,\cdots,L \\
&\textstyle\sum_{i=1}^{N} \lambda_i x_{ki} \leqslant x_{k0}-\beta g_{kx}, k=1,\cdots,K \\
&\textstyle\sum_{i=1}^{N} \lambda_i = 1, \\
&\lambda_i \geqslant 0, i=1,\cdots,N
\end{aligned} \tag{3.16}$$

求解上述线性规划模型，若目标函数值为 β^*，则混合投入效率（MEI）或混合产出效率（MEO）的**一阶近似**分别为：

$$MEI \approx \exp\{-\beta^*\} \approx 1-\beta^*, \quad MEO \approx \frac{1}{\exp\{\beta^*\}} \approx \frac{1}{1+\beta^*}.$$

当且仅当 $\beta^*=0$ 时，$MEI=MEO=1$，表示决策单元位于生产前沿；否则，当 $\beta^*>0$ 时，$MEI=MEO<1$，表明存在提升空间。

注意：(3.16) 求出的是方向向量 $g=(x,y)$ 的**方向距离函数**，$1-\beta^*$ 与 $1/(1+\beta^*)$ 只是双曲效率的一阶近似，β^* 较大时误差不可忽略；它与 3.3.4 节 $g=(x,y)$ 的DDF 是同一个线性规划，因此二者不构成相互独立的稳健性对照。若需要 vrs 下**精确**的双曲距离，应采用 Färe, Margaritis, Rouse and Roshdi (2016) 的算法：借助双曲距离与 DDF 的联系，用至多两个线性规划精确求解，而不是用 $1-\beta^*$ 代替。

3.3.4 基于方向距离函数的技术效率估计

3.3.3 的 VRS 双曲距离已借助方向距离函数 (3.16) 作泰勒近似。事实上，方向距离函数本身是一类更灵活的测度——它允许沿任意预设方向 g 同时调整投入与产出，是上述径向、双曲测度的推广。3.2.4 节已给出 $g=(x,y)$、$(x,0)$、$(0,y)$ 三种方向设定，本节将它们转化为可求解的 LP (3.17)–(3.19)，并给出对应的绩效指数。

当设定 $g=(x,y)$ 时，对应同时缩小投入和扩张产出的方向距离函数，其 DEA 程序为：

$$
\begin{aligned}
&\vec{D}\left(x, y; g_x, g_y\right) = \max\ \beta \\
\text{s.t.}\quad &\sum_{i=1}^{N} \lambda_i y_{li} \geq y_{l0} + \beta g_{ly}, \quad l = 1, \cdots, L \\
&\sum_{i=1}^{N} \lambda_i x_{ki} \leq x_{k0} - \beta g_{kx}, \quad k = 1, \cdots, K \\
&\lambda \in \Lambda^N(\gamma).
\end{aligned}
\tag{3.17}
$$

式中，$\gamma \in \{crs, vrs\}$ 表示不同的规模报酬假设；下标 0 标识被评价 DMU，$i = 1, \ldots, N$ 标识参考集中的 DMU。

当设定 $g = (x, 0)$ 时，表示通过缩小投入达到生产前沿，对应的 DEA 程序为：

$$
\begin{aligned}
&\vec{D}\left(x, y; g_x, 0\right) = \max\ \beta \\
\text{s.t.}\quad &\sum_{i=1}^{N} \lambda_i y_{li} \geq y_{l0}, \quad l = 1, \cdots, L \\
&\sum_{i=1}^{N} \lambda_i x_{ki} \leq x_{k0} - \beta g_{kx}, \quad k = 1, \cdots, K \\
&\lambda \in \Lambda^N(\gamma).
\end{aligned}
\tag{3.18}
$$

同理，对方向 $g = (0, g_y)$，对应的 DEA 程序为：

$$
\begin{aligned}
&\vec{D}\left(x, y; 0, g_y\right) = \max\ \beta \\
\text{s.t.}\quad &\sum_{i=1}^{N} \lambda_i y_{li} \geq y_{l0} + \beta g_{ly}, \quad l = 1, \cdots, L \\
&\sum_{i=1}^{N} \lambda_i x_{ki} \leq x_{k0}, \quad k = 1, \cdots, K \\
&\lambda \in \Lambda^N(\gamma).
\end{aligned}
\tag{3.19}
$$

在同时调整投入与产出的方向 $g = (x, y)$（(3.17)）下，假定最优解为 β^*，可同时定义投入端与产出端的绩效指数：

$$
EI = 1 - \beta^*, \qquad EO = \frac{1}{1 + \beta^*}.
$$

二者刻画了同一 β^* 在投入与产出两端的对应效率——EI 是投入经投影后实际保留的比例，EO 是产出经扩张后实际达到的倍数的倒数。把两端综合，即得到同时反映投入缩减与产出扩张的环境绩效指数

$$
EEI = EI \cdot EO = \frac{1 - \beta^*}{1 + \beta^*}.
$$

三个指数均落在 $(0, 1]$，越接近 1 表示越有效。

若仅作单侧调整——投入方向 $g = (x, 0)$（(3.18)）或产出方向 $g = (0, y)$（(3.19)）——相应的 β 与上面的 β^* 一般不同（三种方向的前沿互不相同），投入效率仍为 $1 - \beta$、产出效率仍为 $1/(1 + \beta)$，只是 β 取自各自方向的LP。

3.4 面板数据的技术参照集构造

通常而言，数据包络分析（DEA）模型在评估决策单元的效率时，主要集中在截面维度的分析。然而，在处理面板数据时，加入了时间这一重要维度，从而丰富了信息含量。当前文献中提出了多种不同的处理方法，大致可以归类为四种：当期（contemporaneous）DEA 模型、Window DEA 模型、时序（sequential）DEA 模型与全局（global）DEA 模型。本节先说明**为什么必须先构造技术参照集**，再依次给出四类参照集的**数学表达式**、各自对应的技术演变假设，以及它们所回答的经济问题。

3.4.1 为什么需要构造技术参照集

DEA 不预设生产函数的参数形式，前沿直接由样本观测” 包络” 而成。因此，在用DEA 度量效率之前，必须先回答一个前置问题：**用哪些观测点来构造被比较的前沿？** 这些观测点的集合称为**技术参照集**（reference set），由它包络出的技术称为**参照技术**。

在截面数据中，这个问题没有歧义：参照集就是当期全部 N 个决策单元，3.3 节的线性规划 (3.9)–(3.19) 正是按此写出的（求和下标均为 $i = 1, \dots, N$）。一旦引入时间维度，全部观测增加到 $N \times T$ 个，参照集的选取就成为必须显式说明的建模决定。其重要性体现在三方面：

（1）**效率总是相对于特定技术而言的**。“某决策单元是否有效” 这一问法本身并不完整，必须补上” 相对于哪一套技术”。参照集不同，前沿位置不同，同一观测点的效率值也随之改变；由下文(3.21) 的单调性可知，**不同参照集下得到的效率值不可直接比较**。

（2）**参照集体现了对技术随时间变化方式的假设**。允许每期各有前沿（当期）、假定窗口内技术不变（Window）、假定技术只进不退（时序）、假定全样本技术不变（全局），对应四种不同的技术演变假设，回答的是四个不同的经济问题，不可混用。

（3）**参照集规模决定估计的自由度**。DEA 存在” 维度灾难”：观测数相对投入产出维数 $K + L$ 过少时，大量决策单元会被判为有效，前沿估计也容易受异常值影响。扩大参照集是缓解该问题的主要手段，但代价是更强的技术不变假设——这构成参照集选择的核心权衡。

3.4.2 参照集与参照技术的一般形式

设面板包含 N 个决策单元、T 个时期，第 i 个决策单元在第 t 期的投入、产出向量分别为 $x_i^t = (x_{1i}^t, \dots, x_{Ki}^t)^{\mathrm{T}} \in \mathfrak{R}_+^K$ 与 $y_i^t = (y_{1i}^t, \dots, y_{Li}^t)^{\mathrm{T}} \in \mathfrak{R}_+^L$。记参照集为观测下标对 (j, s) 的集合 $\mathcal{R} \subseteq \{1, \dots, N\} \times \{1, \dots, T\}$，其元素个数记为 $|\mathcal{R}|$。由 $\mathcal{R}$ 包络出的参照技术定义为：

$$T(\mathcal{R}) = \left\{ (x, y) \in \mathfrak{R}_+^{K+L} : \sum_{(j,s)\in\mathcal{R}} \lambda_{js}\, x_{kj}^s \leqslant x_k,\ k = 1, \dots, K, \quad \sum_{(j,s)\in\mathcal{R}} \lambda_{js}\, y_{lj}^s \geqslant y_l,\ l = 1, \dots, L, \quad \lambda \in \Lambda^{|\mathcal{R}|}(\gamma) \right\} \tag{3.20}$$

式中 λ_{js} 为赋予第 j 个决策单元第 s 期观测的强度变量；$\Lambda^n(\gamma)$ 的设定与 3.3 节一致——$\gamma = crs$ 时仅要求强度变量非负，$\gamma = vrs$ 时另加 $\sum_{(j,s)\in\mathcal{R}} \lambda_{js} = 1$。(3.20) 是 3.1 节技术集 T 的样本估计版本：它以观测点的非负（及 vrs 下的凸）组合外推可行生产计划，并继承 3.1.1 节 (1)–(7) 的全部公理。当 $\mathcal{R}$ 只含单一时期 t 的观测时，(3.20) 即退化为 3.3 节各线性规划所隐含的技术。

参照集的一个基本性质是**单调性**：若 $\mathcal{R}_A \subseteq \mathcal{R}_B$，则把 $\mathcal{R}_B \backslash \mathcal{R}_A$ 的强度变量取零即可得到 $\mathcal{R}_A$ 的任一可行组合，故

$$T(\mathcal{R}_A) \subseteq T(\mathcal{R}_B), \qquad \text{进而} \qquad PE(\mathcal{R}_B) \leqslant PE(\mathcal{R}_A). \tag{3.21}$$

即扩大参照集只会为包络提供更多观测，前沿**不会内移**，被评价点相对前沿的偏离不会变小，因而测得的径向效率（3.3 节的 PEO 与 PEI）**单调不增**。这既是扩大参照集能提高区分度的原因，也是不同参照集下效率值不可直接比较的根源。

3.4.3 当期 DEA 模型

当期参照集只使用与被评价观测**同一时期**的截面数据，即

$$\mathcal{R}^t = \{(i, t) :\ i = 1, \dots, N\}, \qquad |\mathcal{R}^t| = N.$$

相应的当期技术为：

$$T^t = \left\{ (x,y) \in \mathfrak{R}_+^{K+L} : \sum_{i=1}^{N} \lambda_i^t x_{ki}^t \leqslant x_k,\ \sum_{i=1}^{N} \lambda_i^t y_{li}^t \geqslant y_l,\ \lambda^t \in \Lambda^N(\gamma) \right\} \tag{3.22}$$

它回答的问题是：**该决策单元相对于同期最佳实践是否有效？**由于每期各有一条前沿，当期参照集不施加任何跨期的技术不变假设，前沿可以自由地前进或后退，因而最贴合" 同期可比" 的效率概念，也是 3.3 节各线性规划的默认设定。

其局限在于两点。其一，每期只有 N 个观测用于包络，样本量最小，维度灾难与异常值敏感问题最为突出。其二，由于允许前沿后退，某决策单元效率的" 改善" 可能仅仅源于**前沿自身的内移**（例如经济下行年份的最佳实践退化），而非该单元的真实进步；这正是下述时序参照集的出发点。

3.4.4 Window DEA 模型

Window DEA（Charnes, Clark, Cooper and Golany, 1985）把**连续若干期**的观测合并为一个参照集。令窗口宽度为 w，以向后窗口为例，第 t 期的窗口期集合为

$$W(t) = \{ s \in \{1,\dots,T\} :\ t-w+1 \leqslant s \leqslant t \}, \tag{3.23}$$

实践中也常用中心窗口 $W(t) = \{t - \lfloor w/2 \rfloor, \dots, t + \lfloor w/2 \rfloor\} \cap \{1,\dots,T\}$，两种设定的差别仅在于窗口的对齐方式。相应的窗口参照集与窗口技术为

$$\mathcal{R}^{t,w} = \{ (i,s) :\ i = 1,\dots,N,\ s \in W(t) \}, \qquad |\mathcal{R}^{t,w}| = N \cdot |W(t)|,$$

$$T^{t,w} = \left\{ (x,y) \in \mathfrak{R}_+^{K+L} : \sum_{s\in W(t)} \sum_{i=1}^{N} \lambda_i^s x_{ki}^s \leqslant x_k,\ \sum_{s\in W(t)} \sum_{i=1}^{N} \lambda_i^s y_{li}^s \geqslant y_l,\ \lambda \in \Lambda^{N|W(t)|}(\gamma) \right\}. \tag{3.24}$$

Window DEA 的关键做法是**把同一决策单元在不同时期的观测视为不同观测**：窗口内每个决策单元贡献 w 个观测点，参照集规模由 N 扩大到 $N \cdot w$，从而在较弱的跨期假设下提高估计自由度、缓解维度灾难。其假设是**窗口内技术不变**，而不同窗口之间技术可以不同。

由此产生的权衡十分直接：窗口越宽，观测越多、区分度越高，但" 窗口内技术不变" 的假设越强；$w = 1$ 时 (3.24) 退化为当期技术(3.22)。此外，由于相邻窗口相互重叠，同一决策单元—年份组合会出现在多个窗口中并得到多个效率值，实践中通常取其均值作为该组合的效率。

3.4.5 时序 DEA 模型

时序参照集使用**从第一期到当期**的全部观测，即

$$\mathcal{R}^{t,S} = \{ (i,s) :\ i = 1,\dots,N,\ s = 1,\dots,t \}, \qquad \left|\mathcal{R}^{t,S}\right| = N \cdot t,$$

相应的时序技术为

$$T^{t,S} = \left\{ (x,y) \in \mathfrak{R}_+^{K+L} : \sum_{s=1}^{t} \sum_{i=1}^{N} \lambda_i^s x_{ki}^s \leqslant x_k,\ \sum_{s=1}^{t} \sum_{i=1}^{N} \lambda_i^s y_{li}^s \geqslant y_l,\ \lambda \in \Lambda^{Nt}(\gamma) \right\}. \tag{3.25}$$

时序参照集的核心性质是**技术嵌套**：由 $\mathcal{R}^{t,S} \subseteq \mathcal{R}^{t+1,S}$ 及 (3.21) 的单调性，立即得到

$$T^{1,S} \subseteq T^{2,S} \subseteq \cdots \subseteq T^{t,S} \subseteq T^{t+1,S} \subseteq \cdots \subseteq T^{T,S}. \tag{3.26}$$

即技术**只进不退**：历史上出现过的最佳实践在后续各期始终可行。这正对应于Tulkens and Vanden Eeckaut (1995) 为排除" 技术退步" 而提出的时序参照技术。

这一设定解决的是当期参照集的第二个局限：在当期参照集下，经济下行年份的前沿可能位于往年前沿之内，从而被解释为技术退步；时序参照集则把这种" 退步" 排除在模型之外，将其重新解释为效率变化。代价是时序参照集对技术进步的识别较为保守——本会被判定为技术退步的情形都被吸收为效率恶化。

3.4.6 全局 DEA 模型

全局参照集使用**全样本、全时期**的全部观测，即

$$\mathcal{R}^G = \{(i,s):\ i=1,\dots,N,\ s=1,\dots,T\}, \qquad \left|\mathcal{R}^G\right| = N\cdot T,$$

相应的全局技术为

$$T^G = \left\{(x,y)\in\mathfrak{R}_+^{K+L}:\ \sum_{s=1}^{T}\sum_{i=1}^{N}\lambda_i^s x_{ki}^s \leqslant x_k,\ \sum_{s=1}^{T}\sum_{i=1}^{N}\lambda_i^s y_{li}^s \geqslant y_l,\ \lambda\in\Lambda^{NT}(\gamma)\right\}. \tag{3.27}$$

全局参照集为整个样本期设定**唯一一条共同前沿**（Pastor and Lovell, 2005）。所有决策单元—年份组合都相对同一套技术评价，因而跨期距离值可直接比较，由此构造的跨期比值即第 4 章的全局 Malmquist 指数 $mqpi = D^G(x^t,y^t)/D^G(x^{t-1},y^{t-1})$（对应 `tech=GLOBAL`）。它有三方面便利：其一，指数满足循环性（circularity），不受基期选择影响；其二，观测数最多（$N\cdot T$），区分度最高；其三，各期观测均属同一参照集，被评价观测自身即可构成可行解，因此不会出现当期参照集下跨期距离函数可能无可行解的问题。

其代价是假设最强：全局参照集要求**全样本期内技术不变**，真实的跨期技术进步被吸收进唯一的前沿之中，因而在该口径下直接得到的是相对固定前沿的效率变化，而不再单独给出" 前沿移动" 这一分量。

在向后窗口下，四类参照集满足包含关系 $\mathcal{R}^t \subseteq \mathcal{R}^{t,w} \subseteq \mathcal{R}^{t,S} \subseteq \mathcal{R}^G$，由(3.21) 即得 $T^t \subseteq T^{t,w} \subseteq T^{t,S} \subseteq T^G$，而效率值则按相反方向排序；若改用中心窗口，$\mathcal{R}^{t,w}$ 因含 t 之后的时期而不再被 $\mathcal{R}^{t,S}$ 包含，上述链式包含关系不再成立。下表汇总四者的差异：

参照集	观测数	技术演变假设	回答的问题	主要局限
当期 T^t (3.22)	N	每期各有前沿，可进可退	相对同期最佳实践是否有效	样本量最小；前沿可能内移
Window $T^{t,w}$ (3.24)	$N\cdot w$	窗口内不变，窗口间可变	相对近期（窗口内）最佳实践是否有效	窗口宽度需主观设定
时序 $T^{t,S}$ (3.25)	$N\cdot t$	技术只进不退	相对截至当期的最好实践是否有效	对技术进步的识别偏保守
全局 T^G (3.27)	$N\cdot T$	全样本期技术不变	相对全样本共同前沿是否有效	吸收跨期技术进步

综上，参照集的选择不是技术细节，而是**对技术演变方式的声明**：它决定效率值相对于哪一套前沿而言、能否跨期比较，以及动态分析中技术变化与效率变化如何分离。

上述参照集的构造最终都要落实为对求解样本的限定，3.5 节将说明 `deabook` 如何用 `evalindex`/`refindex` 两个参数把这一选择参数化，并统一调用 3.3 节的各类线性规划。

3.5 经典DEA模型的Python实现

3.3 节将四类效率测度统一表述为可在给定参照技术上求解的线性规划(3.9)–(3.19)，并补充了单投入情形(3.13)。这些 LP 在实证研究中呈现三点共性，构成了统一软件实现的基础：其一，它们共享同一套数据接口——投入产出数据框、规模报酬假设与技术参照集设定；其二，尽管不同距离函数对应不同的调整方向与目标，底层求解框架相同；其三，导向、方向向量与规模报酬等" 建模开关" 可以参数化，而无须为每种模型单独编写求解代码。

基于这一结构，deabook 在 DEA 模块下提供五个并列接口，分别对应于 3.3 节的各类测度：DEA.ccr() 实现规模报酬不变（CRS）下的径向DEA，DEA.bcc() 实现规模报酬可变（VRS）下的径向 DEA，二者通过 orient 区分投入与产出导向；DEA.hyperbolic() 同时缩减投入并扩张产出，刻画双曲距离函数；DEA.ite() 仅缩减 sent 左侧第一个投入，度量单投入技术效率；DEA.ddf() 沿用户指定的方向向量 gx/gy 移动，实现方向距离函数。这种分组方式的经济含义在于：径向 DEA 回答" 在产出（或投入）固定时，投入（或产出）能等比例压缩（或扩张）到何种程度"；而hyperbolic 与 DDF 放松了单侧约束，允许投入缩减与产出扩张同步进行，从而更贴近" 既节约要素又扩大产出" 的现实改进路径。

为将 3.3 节的数学符号与代码参数一一对应，表 表 3.2 给出关键映射。

表 3.2: 数学符号与代码参数映射

数学符号	代码参数	含义
$x = (K, L, E)$	sent 左侧	投入向量
$y = (Y)$	sent 右侧	产出向量
g_x	gx（仅 DEA.ddf()；如 [1,1,1] 表全部投入等比例方向）	投入方向向量
g_y	gy（仅 DEA.ddf()；如 [0] 表不调整产出）	产出方向向量
$\gamma \in \{crs, vrs\}$	rts（RTS_CRS/RTS_VRS）	规模报酬假设
$\phi^*, \theta^*, \eta'^*, \beta^*$	rho（求解器返回的原始最优值）	各 LP 的目标最优值（双曲 CRS 为线性化目标 η'^*，由 te= $\sqrt{\rho}$ 还原 η^*）
$PEO, PEI, MEI, MEO, EI, EO$	te、rho	经转换后的效率指数（径向与单投入接口直接给 te；hyperbolic/ddf 给 rho，按表 3.6 换算）

下文围绕这五个接口的设计逻辑展开：先说明模块的导入与基本依赖，再分析各接口的建模语义与共享参数的经济含义，进而阐释返回值中各效率列对应的效率口径，最后以与 3.6 节案例一致的调用方式收束。

3.5.1 模块的导入与依赖

在 deabook 中估计 DEA 模型之前，需为生产数据的读写与数值计算建立基础的 Python 环境。除 DEA 主类外，规模报酬、求解方式等建模常数亦需从 constant 子模块显式导入，以便在调用时以 RTS_CRS/RTS_VRS 等符号声明技术假设——这种显式声明并非语法细节，而是把 3.1 节关于规模报酬的抽象假设落到具体估计的必要环节：

```
from deabook import data_path
from deabook.constant import GLOBAL
from deabook import DEA
from deabook.constant import RTS_VRS, RTS_CRS, OPT_LOCAL
import pandas as pd
import numpy as np
```

3.5.2 五类效率测度的统一接口

五个接口的分工由其建模语义决定，表 表 3.3 汇总了各自的规模报酬、导向与调整方式。径向 DEA（ccr/bcc）在固定产出的前提下等比例缩减投入（或反之），因而最适于回答单一维度的效率缺口；hyperbolic 放松方向约束，使投入缩减与产出扩张同时发生；ite 将改进限定于单一投入，刻画要素结构内部的配置效率；ddf 则通过方向向量 gx/gy 把改进路径完全交由研究者指定。这种" 同一求解内核、多组建模开关" 的安排，使不同效率口径可在同一数据上直接对照：

表 3.3: 五个接口与各自的调整方式

接口	规模报酬	导向	调整方式
DEA.ccr()	CRS（固定）	orient，默认"oo"（产出导向）	产出导向时扩张产出，投入导向时缩减投入
DEA.bcc()	VRS（固定）	orient，默认"oo"	同上
DEA.hyperbolic()	rts（默认 CRS）	无 orient 选项	同时缩减投入与扩张产出
DEA.ite()	rts（默认 CRS）	投入导向（固定）	只缩减 sent 左侧第一个投入，其余投入与产出不变
DEA.ddf()	rts（默认 CRS）	由 gx/gy 决定	沿给定的方向向量 gx/gy 移动

五者共享参数 data、sent、rts、evalindex、refindex；ccr/bcc 另取 orient 控制导向（也可写作"output"/"input"），hyperbolic 无 orient 选项，ite 按 sent 中投入的书写顺序确定被缩减的那一项——把要考察的投入写在最左侧第一位即可，只有 ddf 还需显式给定方向参数 gy、gx。

```
# CCR（CRS，投入导向）
model = DEA.ccr(data, sent="K L E=Y", orient="io")
```

```
res = model.optimize(solver="glpk")

# BCC（VRS,投入导向）
model = DEA.bcc(data, sent="K L E=Y", orient="io")
res = model.optimize(solver="glpk")

# hyperbolic（同时缩减投入、扩张产出，CRS）
model = DEA.hyperbolic(data, sent="K L E=Y", rts=RTS_CRS)
res = model.optimize(solver="glpk")

# ite（单投入技术效率:只缩减资本 K）
model = DEA.ite(data, sent="K L E=Y", rts=RTS_CRS)
res = model.optimize(solver="glpk")

# ddf（方向距离函数:沿给定方向 gx/gy 移动）
model = DEA.ddf(data, sent="K L E=Y", gy=[0], gx=[1,1,1], rts=RTS_CRS)
res = model.optimize(solver="glpk")
```

ccr/bcc/ite 直接返回效率列 te，值域 $(0, 1]$，越接近 1 越有效；hyperbolic 与 ddf 只返回线性规划值 rho，需按 表 3.6 换算为相应口径的效率。其中 ite 度量的是**单个投入**的利用效率：它对应 3.2.2.2 节的单投入谢泼德距离函数 (3.6) 及其线性规划(3.13)，只沿该投入方向收缩，其余投入与产出固定；ddf 的调用细节见 3.5.5。

3.5.3 参数语义与规模报酬设定

五个接口虽建模语义不同，却共享同一组参数（表 3.4），这正体现了它们源自同一求解内核。data 与 sent 描述数据与变量映射：sent 以 = 分隔投入与产出，其书写顺序本身也携带信息——ite 即按此顺序确定被缩减的投入。rts 声明规模报酬假设，是连接 3.1 节凸性公理与具体估计的开关。evalindex 与 refindex 则把 3.4 节技术参照集的选择参数化：前者限定被评价单元，后者限定参考前沿，二者共同决定效率值相对于哪一套技术而言。方向向量 gy、gx 仅 DEA.ddf() 需显式给定，用于设定方向距离函数的调整方向；ccr/bcc/hyperbolic 不提供方向向量参数，其调整方式见表 3.3。各参数的类型与可选值如下：

表 3.4: ccr/bcc/hyperbolic/ite 与 ddf 的参数说明（gy/gx 仅 ddf 使用）

参数	类型	可选值	含义
data	DataFrame	—	包含 sent 中指定列的数据，如K, L, E, Y
sent	str	"投入1 投入2 ...=产出1 产出2 ..."	= 左边为投入变量，= 右边为产出变量。多个变量用空格分隔

参数	类型	可选值	含义
gy	list[int]	[0]	不沿该产出方向调整
	list[int]	[1]	沿该产出方向扩大产出；与 gx≥1 同时使用时（双曲模式），表示沿产出方向同步扩大
gx	list[int]	[0]	不沿该投入方向调整
	list[int]	[1]	沿该投入方向缩减
rts	str	RTS_CRS	规模报酬不变
	str	RTS_VRS	规模报酬可变
email	str	OPT_LOCAL	本地求解
	str	邮箱地址	远程求解完成后发送通知
evalindex	str	None	被评价 DMU 筛选条件，如"t==2019"，None 表示全部评价
	str	"t=[2017,2018,2019]"	仅评价指定时期
refindex	str	None	参考技术筛选条件，如"t==2019"，None 表示使用全部 DMU 构建前沿
	str	"t=[2017,2018,2019]"	仅用指定时期构建参考前沿
solver	str	"mosek"	商业求解器，速度快、精度高
	str	"glpk"	开源求解器
	str	"cbc"	开源求解器

3.5.4 返回结构与效率列的经济含义

optimize() 的返回结构统一为 DataFrame，通用列包括 optimization_status（求解状态）、rho（求解器返回的原始最优值）与 objective_value（目标函数值）。rho 是各 LP 目标函数的直接取值，其经济含义随模型而异；理解返回值的关键，在于区分" 径向与单投入接口直接给出效率列" 和"hyperbolic 与 ddf 仅给出 rho、需二次换算" 两类情形。

径向与单投入接口（ccr/bcc/ite）的口径唯一，故除 rho 外直接给出效率列 te（ 表 3.5）：

表 3.5: ccr/bcc/ite 的效率列

接口与导向	RTS	te 与 rho 的关系	对应 LP
ccr/bcc 投入导向	CRS / VRS	$te = \rho$（$= \theta^*$）	(3.12)
ccr/bcc 产出导向	CRS / VRS	$te = 1/\rho$（$\rho = \phi^*$）	(3.9)–(3.11)
ite 单投入	CRS / VRS	$te = \rho$（该项投入的收缩比 θ^*）	(3.13)

hyperbolic 与 ddf 则只返回 rho，不直接给出效率列：二者的效率取值取决于规模报酬设定或所给定的方向向量，需按表 3.6 由 rho 换算。这样处理避免了用固定列名表达” 投入端/产出端” 两种口径。

表 3.6: hyperbolic 与 ddf 由 rho 换算效率

接口	设定	rho 的含义	效率换算
hyperbolic	CRS	$\rho = \eta'^*$（线性化目标）	$te = \sqrt{\rho}$，即 (3.15) 的 η^*
hyperbolic	VRS	$\rho = \beta^*$	投入端 $tei = 1 - \rho$；产出端 $teo = 1/(1+\rho)$
ddf	投入方向（$g_y = 0$）	$\rho = \beta^*$	$te = 1 - \rho$
ddf	产出方向（$g_x = 0$）	$\rho = \beta^*$	$te = 1/(1+\rho)$
ddf	双向（hyperbolic 方向）	$\rho = \beta^*$	$tei = 1 - \rho$ 与 $teo = 1/(1+\rho)$

所有效率指数均落在 $(0, 1]$，越接近 1 表示越有效。其中 hyperbolic 在 CRS 下的 $te = \sqrt{\rho}$ 要求全部投入沿双曲路径缩减，而 DEA.hyperbolic() 恒对全部投入做双曲缩减（不支持只缩减部分投入），故该式始终成立；VRS 下 tei、teo 只是一阶泰勒近似，β^* 较大时误差不可忽略（见(3.16) 处的说明）。

3.5.5 调用示例与实证研究的衔接

下面以 sent="K L E=Y" 为例，演示 3.6 节案例将用到的几类调用，其变量设定与后文 OECD 及中国省份数据完全一致：投入导向径向 DEA（资本 K、劳动 L、能源 E 三类投入等比例缩减，对应模型 (3.12) 的径向投入形式——即 3.2.2 节谢泼德投入距离函数 (3.5)，得到技术效率PEI）、双曲（hyperbolic） DEA（对应模型(3.15)/(3.16)，得到混合技术效率 TEI/TEO）与投入导向DDF（对应模型 (3.18)，得到技术方向的 DDF 效率 DI）。

投入导向径向 DEA（技术效率）：用 DEA.ccr()（CRS）与 DEA.bcc()（VRS）两个接口。orient="io" 表示投入导向（三类投入等比例缩减），对应模型 (3.12)。调用如下：

```
# CCR（CRS，投入导向）
model = DEA.ccr(data, sent="K L E=Y", orient="io")
res = model.optimize(solver="glpk")
```

```
# BCC（VRS,投入导向）
model = DEA.bcc(data, sent="K L E=Y", orient="io")
res = model.optimize(solver="glpk")
```

双曲 DEA（hyperbolic）：用 `DEA.hyperbolic()` 接口，该接口同时缩减投入与扩张产出，对应模型(3.15)/(3.16)。调用如下：

```
# hyperbolic（CRS）
model = DEA.hyperbolic(data, sent="K L E=Y", rts=RTS_CRS)
res = model.optimize(solver="glpk")

# hyperbolic（VRS）
model = DEA.hyperbolic(data, sent="K L E=Y", rts=RTS_VRS)
res = model.optimize(solver="glpk")
```

方向距离函数（投入导向）：用 `DEA.ddf()` 接口，需显式给定方向向量 `gx=[1,1,1]`、`gy=[0]`（投入等比例缩减、产出固定）：

```
# CRS
model = DEA.ddf(data, sent="K L E=Y", gy=[0], gx=[1,1,1], rts=RTS_CRS)
res = model.optimize(solver="glpk")

# VRS
model = DEA.ddf(data, sent="K L E=Y", gy=[0], gx=[1,1,1], rts=RTS_VRS)
res = model.optimize(solver="glpk")
```

单投入技术效率（ite）：用 `DEA.ite()` 接口估计模型(3.13)——只缩减被考察的投入，其余投入与产出固定。把要考察的投入写在 `sent` 左侧**第一位**即可；下例把资本 K 置于首位，故只缩减资本，而劳动 L 与能源 E 保持不变：

```
# 只缩减资本 K（sent 左侧第一位为被缩减的投入）
model = DEA.ite(data, sent="K L E=Y", rts=RTS_CRS)
res = model.optimize(solver="glpk")

# VRS 情形
model = DEA.ite(data, sent="K L E=Y", rts=RTS_VRS)
res = model.optimize(solver="glpk")
```

返回的 `te` 即 (3.13) 的 θ^*，对应 3.2.2.2 节单投入谢泼德距离函数 (3.6) 的倒数（$D_I = 1/\theta^*$）。

以上示例展示了 `ccr`/`bcc`/`hyperbolic`/`ite` 与 `ddf` 五个接口的最小调用方式。为说明这些模型在实证研究中的实际表现，3.6 节以 2016–2019 年 33 个 OECD 国家为样本，依次评估其技术效率、混合技术效率与技术方向的 DDF 效率，并通过中国省级数据进一步演示区域比较。

3.6 案例应用：美国燃煤电厂的技术效率

Färe、Grosskopf和Kokkelenberg（1989）研究了美国伊利诺伊州燃煤电厂；Simar和Wilson（1998）也使用了这一数据来源。公开文件包含1978年19家电厂。本节以劳动、燃料和装机容量为投入，以发电量为产出，比较投入和产出导向下的CRS、VRS效率。这里允许全部投入参与调整，是使用公开数据开展的教材重估，不是原论文固定资本产能模型的逐表复现。

```
from deabook.example.book_cases.run import FUN
FUN[3]()  # 重新计算本章，输出完整CSV、摘要表与PNG/PDF图
```

表 3.7: 美国燃煤电厂的技术效率：结果摘要

统计量	CRS投入	VRS投入	CRS产出	VRS产出	规模效率
有效样本数	19	19	19	19	19
均值	0.897	0.942	0.897	0.941	0.953
标准差	0.062	0.061	0.062	0.061	0.060
最小值	0.816	0.821	0.816	0.825	0.825
P10	0.828	0.865	0.828	0.866	0.837
中位数	0.882	0.953	0.882	0.954	0.986
P90	1.000	1.000	1.000	1.000	1.000
最大值	1.000	1.000	1.000	1.000	1.000

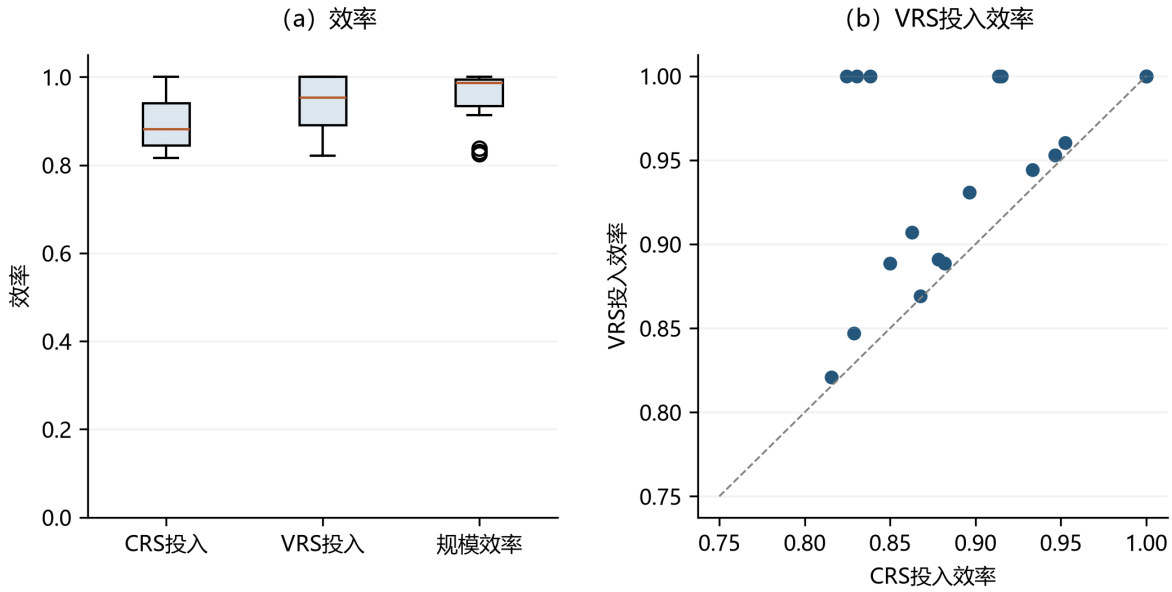


图 3.6: 美国19家电厂的效率分布与规模报酬比较

表中同时给出两种导向、两种规模报酬设定及规模效率的样本数与分布统计。比较均值和中位数可判断差异是否集中于少数低效率电厂，结合P10、P90和极值观察分布范围。规模效率采用投入导向CRS与

VRS效率之比。

CRS投入效率均值为0.897，VRS为0.942；规模效率均值为0.953。规模假设影响部分电厂的可达前沿。CRS下投入与产出效率相同，VRS下两种导向一般不同；表中不逐家排列，分布和对应关系由图展示。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch03_full.csv`。表中P10和P90为第10和第90百分位数（如适用），展示分布而不占用逐个决策单元的行数。

3.7 本章小结

CRS投入效率均值为0.897，VRS为0.942；规模效率均值为0.953。规模假设影响部分电厂的可达前沿。CRS下投入与产出效率相同，VRS下两种导向一般不同；表中不逐家排列，分布和对应关系由图展示。

参考文献

BANKER R D, CHARNES A, COOPER W W. Some models for estimating technical and scale inefficiencies in data envelopment analysis [J]. Management Science, 1984, 30(9): 1078-1092. DOI: 10.1287/mnsc.30.9.1078.

CAMANHO A S, SILVA M C, PIRAN F S. A literature review of economic efficiency assessments using data envelopment analysis [J]. European Journal of Operational Research, 2024, 315(1): 1-18. DOI: 10.1016/j.ejor.2024.01.040.

CHAMBERS R G, CHUNG Y, FÄRE R. Profit, directional distance functions, and Nerlovian efficiency [J]. Journal of Optimization Theory and Applications, 1998, 98(2): 351-364. DOI: 10.1023/A:1022637501082.

CHARNES A, CLARK C T, COOPER W W, GOLANY B. A developmental study of data envelopment analysis in measuring the efficiency of maintenance units in the U.S. Air Forces [J]. Annals of Operations Research, 1985, 2(1): 95-112. DOI: 10.1007/BF01874734.

CHARNES A, COOPER W W, RHODES E. Measuring the efficiency of decision making units [J]. European Journal of Operational Research, 1978, 2(6): 429-444. DOI: 10.1016/0377-2217(78)90138-8.

CHAVAS J-P, ALIBER M. An analysis of economic efficiency in agriculture: a nonparametric approach [J]. Journal of Agricultural and Resource Economics, 1993, 18(1): 1-16.

DEBREU G. The coefficient of resource utilization [J]. Econometrica, 1951, 19(3): 273-292. DOI: 10.2307/1906835.

DU K, LI J. Towards a green world: how do green technology innovations affect total-factor carbon productivity [J]. Energy Policy, 2019, 131: 240-250. DOI: 10.1016/j.enpol.2019.04.013.

FARRELL M J. The measurement of productive efficiency [J]. Journal of the Royal Statistical Society Series A (General), 1957, 120(3): 253-290. DOI: 10.2307/2343100.

FEENSTRA R C, INKLAAR R, TIMMER M P. The next generation of the Penn World Table [J]. American Economic Review, 2015, 105(10): 3150-3182. DOI: 10.1257/aer.20130954.

FERRIER G D, LOVELL C A K. Measuring cost efficiency in banking: econometric and linear programming evidence [J]. Journal of Econometrics, 1990, 46(1): 229-245. DOI: 10.1016/0304-4076(90)90052-Z.

FÄRE R, GROSSKOPF S, LEE H. A nonparametric approach to expenditure-constrained profit maximization [J]. American Journal of Agricultural Economics, 1990, 72(3): 574-581. DOI: 10.2307/1242588.

FÄRE R, GROSSKOPF S. Nonparametric productivity analysis with undesirable outputs: comment [J]. American Journal of Agricultural Economics, 2003, 85(4): 1070-1074. DOI: 10.1111/1467-8276.00510.

FÄRE R, GROSSKOPF S, LOVELL C A K. Hyperbolic graph efficiency measures [M]//Färe R, Grosskopf S, Lovell C A K. The measurement of efficiency of production. Dordrecht: Springer Netherlands, 1985: 107-130. DOI: 10.1007/978-94-015-7681-6.

FÄRE R, MARGARITIS D, ROUSE P, ROSHDI I. Estimating the hyperbolic distance function: a directional distance function approach [J]. European Journal of Operational Research, 2016, 254(1): 312-319. DOI: 10.1016/j.ejor.2016.03.039.

GLOBAL CARBON ATLAS [DB/OL]. Global Carbon Project. http://www.globalcarbonatlas.org (访问日期：2026-07-03).

GOTO M, TSUTSUI M. Comparison of productive and cost efficiencies among Japanese and US electric utilities [J]. Omega, 1998, 26(2): 177-194. DOI: 10.1016/S0305-0483(97)00035-1.

HAILU A, VEEMAN T S. Non-parametric productivity analysis with undesirable outputs: an application to the Canadian pulp and paper industry [J]. American Journal of Agricultural Economics, 2001, 83(3): 605-616. DOI: 10.1111/0002-9092.00181.

LIN B, DU K. Modeling the dynamics of carbon emission performance in China: a parametric Malmquist index approach [J]. Energy Economics, 2015, 49: 550-557. DOI: 10.1016/j.eneco.2015.02.005.

LIN B, DU K. Technology gap and China's regional energy efficiency: a parametric metafrontier approach [J]. Energy Economics, 2013, 40: 529-536. DOI: 10.1016/j.eneco.2013.07.015.

LINNA M. Measuring hospital cost efficiency with panel data models [J]. Health Economics, 1998, 7(5): 415-427. DOI: 10.1002/(SICI)1099-1050(199808)7:5<415::AID-HEC349>3.0.CO;2-O.

PASTOR J T, LOVELL C A K. A global Malmquist productivity index [J]. Economics Letters, 2005, 88(2): 266-271. DOI: 10.1016/j.econlet.2005.02.013.

RAY S C, KIM H J. Cost efficiency in the US steel industry: a nonparametric analysis using data envelopment analysis [J]. European Journal of Operational Research, 1995, 80(3): 654-671. DOI: 10.1016/0377-2217(95)90011-X.

RAY S C, SEIFORD L M, ZHU J. Market entity behavior of Chinese state-owned enterprises [J]. Omega, 1998, 26(2): 263-278. DOI: 10.1016/S0305-0483(97)00038-7.

SHEPHARD R W. Theory of cost and production functions [M]. Princeton: Princeton University Press, 1970.

SUEYOSHI T. Estimation of stochastic frontier cost function using data envelopment analysis: an application to the AT&T divestiture [J]. Journal of the Operational Research Society, 1991, 42(6): 463-477. DOI: 10.1057/jors.1991.84.

TULKENS H, VANDEN EECKAUT P. Non-frontier measures of efficiency, progress and regress for time series data [J]. International Journal of Production Economics, 1995, 39(1-2): 83-97. DOI: 10.1016/0925-5273(95)00002-6.

U.S. ENERGY INFORMATION ADMINISTRATION. International energy statistics [DB/OL]. Washington, D.C.: EIA. https://www.eia.gov/international/data/world (访问日期：2026-07-03).

YUNOS J M, HAWDON D. The efficiency of the National Electricity Board in Malaysia: an intercountry comparison using DEA [J]. Energy Economics, 1997, 19(2): 255-269. DOI: 10.1016/S0140-9883(96)01014-5.

ZHOU P, ANG B W, HAN J Y. Total factor carbon emission performance: A Malmquist index analysis [J].

Energy Economics, 2010, 32(1): 194-201. DOI: 10.1016/j.eneco.2009.10.003.

ZHOU P, ANG B W, ZHOU D Q. Measuring economy-wide energy efficiency performance: a parametric frontier approach [J]. Applied Energy, 2012, 90(1): 196-200. DOI: 10.1016/j.apenergy.2011.01.024.

第 4 章

全要素生产率测度方法

第 3 章介绍了经典DEA模型的基本构成、生产技术的设定以及静态效率测度方法，分析了投入、产出距离函数及其在评价企业运行中的应用。然而，随着时间推移，企业的行为与业绩会发生变化：既包括一般技术进步带来的外部效应，也包括企业相对于现有技术采取特定改进措施所产生的差异。为刻画这种动态变化，需构建能够反映企业生产率随时间变动的评价指标。Malmquist 指数是基准评估文献中最流行的动态效率测度方法，可在不依赖价格信息的条件下对多种投入与产出加以汇总，是度量企业生产率变化的基本工具。

Färe, Grosskopf, Lindgren and Roos (1992) 和 Färe, Grosskopf, Norris and Zhang (1994) 提出使用 Malmquist 指数衡量生产率变化，奠定了后续生产率测度文献的基础。该指数由 Caves, Christensen and Diewert (1982) 根据 Malmquist (1953) 的观点提出，用于衡量生产率随时间的变化。Färe, Grosskopf, Lindgren and Roos (1992) 研究了 42 家瑞典药店的生产率变化，并展示了如何将 Malmquist 指数衡量的生产率变化分解为技术效率变化和技术变化的指标；Färe, Grosskopf, Norris and Zhang (1994) 研究了 1979 年至 1988 年间经济合作与发展组织（OECD）17 个成员国的生产率变化，并将 Malmquist 指数分解为技术效率、规模效率和技术变化的指标。Malmquist 指数可乘法分解为各个分量（Färe, Grosskopf, Norris and Zhang，1994）。相关应用可参见 Kevork, Pange, Tzeremes and Tzeremes (2017)、Pastor, Lovell and Aparicio (2020)、Simar and Wilson (2023)。

作为与 Malmquist 指数并列的另一条路线，Chambers, Färe and Grosskopf (1996) 基于方向距离函数（DDF）定义了 Luenberger 指标。与 Malmquist 指数采用乘法分解不同，Luenberger 指标在加法形式下同时刻画产出扩张和投入收缩（其精确性与超等性质的研究见Chambers (2002)），其最新应用包括 Aparicio, Pastor and Vidal (2016)、Aparicio, Garcia-Nové, Kapelko and Pastor (2017)。本章聚焦于Malmquist 指数（乘法分解）及其在 DDF 形式下的等价表达，Luenberger 指标的加法分解留待后续章节展开。

本章按以下顺序展开：4.1 节在谢泼德距离函数与方向距离函数两条路径下推导 Malmquist 指数及其效率变化—技术变化分解，并建立两类距离函数的等价关系；4.2 节说明 `deabook` 包中 `DEA.mq` 与 `DEA.mqddf` 接口的参数、返回值与调用方式；4.3 节将上述模型应用于 33 个OECD 国家与中国 30 个省份 2016–2019 年的面板数据，数值上印证等价性并刻画动态产出效率的贡献结构；4.4 节给出总结。

4.1 经典DEA模型的生产率指数

为解释 Malmquist 指数的概念，令 $D^i(s,t)$ 为衡量第 i 个决策单元（decision making unit, DMU）在 s 时期相对于 t 时期技术表现的指标。例如，它可以是投入方向的谢泼德距离函数，即

$$D^i(s,t)=\sup\left\{\lambda>0 \mid \left(x^{is}/\lambda, y^{is}\right)\in T^t\right\},$$

该距离函数取值范围为 $[1,+\infty)$，在前沿上取值为1，越远离前沿取值越大，其倒数即 Farrell 投入效率。它也可以是其他衡量标准——产出方向的距离函数 $D_o\in(0,1]$ 本身即为效率值（越接近 1 越有效）；只要所选指标能反映" 越接近前沿绩效越好" 即可。需要说明的是，这里开始按技术与生产数据所处的时期对二者加以区分；后文统一采用上标表示参考技术所在时期、括号内自变量表示生产计划所在时期的记号（如 $D_o^t(x^s,y^s)$），不再使用 $D^i(s,t)$ 这一过渡记号。

4.1.1 谢泼德距离函数对应的Malmquist指数

（一）产出方向的谢泼德距离函数为更好地捕捉决策单元在产出扩张方面的改进，首先从产出角度构建谢泼德距离函数，其定义为：

$$D_o(x,y)=\inf\{\theta:(x,y/\theta)\in T\} \tag{4.1}$$

式中，T 为生产技术。设 t 时期和 $s\,(s>t)$ 时期决策单元观测值分别为 (x^t,y^t) 和 (x^s,y^s)，则基于 t 时期生产技术的 Malmquist 生产率指数定义为 s 时期距离函数与 t 时期距离函数之比，即：

$$M_o^t=\frac{D_o^t\left(x^s,y^s\right)}{D_o^t\left(x^t,y^t\right)} \tag{4.2}$$

类似地，基于 s 时期生产技术的 Malmquist 生产率指数定义为：

$$M_o^s=\frac{D_o^s\left(x^s,y^s\right)}{D_o^s\left(x^t,y^t\right)} \tag{4.3}$$

若企业从 t 期到 s 期（$s>t$）靠近前沿，则 $D_o^s(x^t,y^t)\le D_o^s(x^s,y^s)$，故 $M_o^s>1$；若远离前沿，则 $M_o^s<1$。例如，某 DMU 在 s 期技术下的产出效率由 t 期的 40% 升至 80%（即 s 期计划比 t 期计划更接近前沿），则 $M_o^s=0.8/0.4=2$，表示其相对于 s 期前沿的接近程度翻倍。当投入产出组合同时变化时，该比值仍刻画两期计划相对于同一参考前沿的接近程度之比，解释逻辑不变。

M^s 衡量的是相对于技术 s 的改进，也可以选择以 t 期技术作为固定参照。为消除时期选择对生产率指数的影响，产出导向的Malmquist 生产率指数最终定义为两个时期生产率指数的几何平均值，即

$$M_o=\left[\frac{D_o^t\left(x^s,y^s\right)}{D_o^t\left(x^t,y^t\right)}\times\frac{D_o^s\left(x^s,y^s\right)}{D_o^s\left(x^t,y^t\right)}\right]^{1/2} \tag{4.4}$$

上式包含 4 个产出方向谢泼德距离函数：其中两个是**同期距离**（技术与被评价生产计划属同一时期）$D_o^t(x^t,y^t)$ 与 $D_o^s(x^s,y^s)$，另两个是**跨期距离** $D_o^t(x^s,y^s)$ 与 $D_o^s(x^t,y^t)$；同期与跨期在乘号两侧交叉出现，而非乘号前一对同期、乘号后一对跨期。取几何平均的目的是不单独依赖 t 期或 s 期技术作为参照。Malmquist 指数衡量决策单元从时期 t 到时期 s 的业绩改进程度。然而，业绩变化源于两个可能相互强化或抵消的因素：一是所有企业共同面临的一般性技术进步（或退步），二是决策单元特有的管理方式，后者使

其业绩相对于其他DMU 发生变化。将两个同期距离函数提取至中括号外，上式即可分解为两个部分，即

$$\begin{aligned} M_o &= \frac{D_o^s\left(x^s, y^s\right)}{D_o^t\left(x^t, y^t\right)} \times\left[\frac{D_o^t\left(x^t, y^t\right)}{D_o^s\left(x^t, y^t\right)} \times \frac{D_o^t\left(x^s, y^s\right)}{D_o^s\left(x^s, y^s\right)}\right]^{1/2} \\ &= \text{MEFFCH} \times \text{MTECHCH} \end{aligned} \tag{4.5}$$

式中，效率指数 $MEFFCH$ 以当期技术为基准，衡量决策单元相对于产出技术前沿的追赶效应，即企业是否在逐步靠近现有的最佳技术状态。若企业的同期距离满足 $D_o^t(x^t, y^t) < D_o^s(x^s, y^s)$，则该比率大于1，表明企业正在有效追赶前沿水平。

另一方面，技术变化指数 $MTECHCH$ 通过两个比率的几何平均值衡量生产技术边界的变动：两个比率分别将决策单元在 t 时期和 s 时期的生产计划固定为参照点，据以观察技术的进步情况。具体而言，若技术进步（前沿上移），则在固定 t 时期生产计划 (x^t, y^t) 时，会出现 $D_o^t(x^t, y^t) > D_o^s(x^t, y^t)$ 的情形（上标为参考技术所在时期，括号内为被评价生产计划所在时期），即同一计划在更先进的 s 期技术下显得离前沿更远、效率得分更低，这恰好反映 s 期前沿相对于 t 期外移（此时几何平均值中的第一个比率大于1）；第二个比率的逻辑类似，只是固定的是 s 期的生产计划。

综上，$MTECHCH$ 衡量的是技术变化，其数值大于 1 时代表技术进步，即以更少的资源生产出更多的产品。

图 4.1 给出含有一个投入和一个产出的 Malmquist 指数的图形示意。设区域 $OABCD$ 和 $OA'B'C'D'$ 分别代表 t 和 s 时期的 DEA 技术；进一步假设某 DMU 在 t 和 s 时期使用相同数量的投入生产不同数量的产出，分别由 M 点和 N 点表示。效率变化部分为 $\frac{DN/DF}{CM/CB}$，即 (4.5) 式中的MEFFCH，衡量 DMU 从 t 期到 s 期相对于各自当期技术的追赶效应。技术变化部分为 $\left[\frac{DN/DE}{DN/DF} \cdot \frac{CM/CB}{CM/CA}\right]^{1/2} = \left[\frac{GE}{GF'} \cdot \frac{G'F}{G'E'}\right]^{1/2}$，即(4.5)式中的 MTECHCH，衡量在投入 x 水平上生产前沿从 OC 到 OD 的上移。

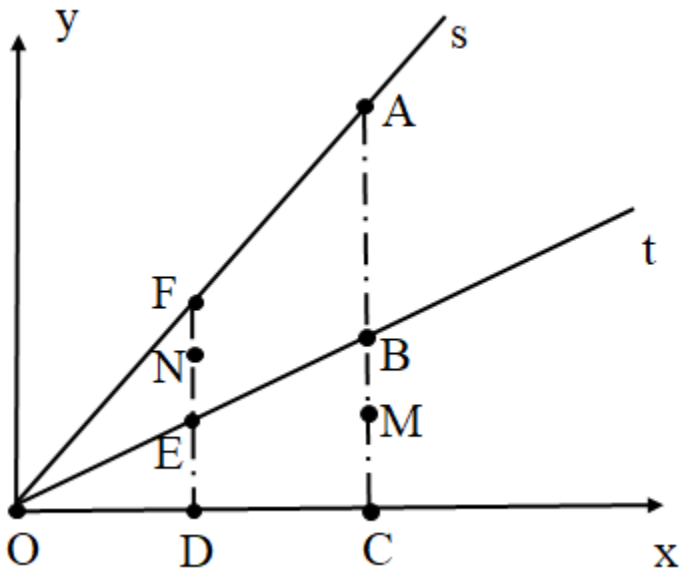


图 4.1: Malmquist 指数示意图

（二）投入方向的谢泼德距离函数以下从投入角度构建谢泼德距离函数，以捕捉决策单元在资源配置

与利用上的调整效果。投入方向的谢泼德距离函数定义为：

$$D_i(x,y)=\sup\{\theta:(x/\theta,y)\in T\} \tag{4.6}$$

式中，T 为生产技术。设 t 时期和 $s\,(s>t)$ 时期决策单元观测值分别为 (x^t,y^t) 和 (x^s,y^s)。由于投入方向的谢泼德距离函数取值范围为 $[1,+\infty)$，其倒数为效率值，故基于 t 时期生产技术的 Malmquist 生产率指数定义为 t 时期距离函数与 s 时期距离函数之比，即：

$$M_i^t=\frac{D_i^t\left(x^t,y^t\right)}{D_i^t\left(x^s,y^s\right)} \tag{4.7}$$

类似地，基于 s 时期生产技术的 Malmquist 生产率指数定义为：

$$M_i^s=\frac{D_i^s\left(x^t,y^t\right)}{D_i^s\left(x^s,y^s\right)} \tag{4.8}$$

为消除时期选择对生产率指数的影响，投入导向的 Malmquist 生产率指数最终定义为两个时期生产率指数的几何平均值，即

$$M_i=\left[\frac{D_i^t\left(x^t,y^t\right)}{D_i^t\left(x^s,y^s\right)}\times\frac{D_i^s\left(x^t,y^t\right)}{D_i^s\left(x^s,y^s\right)}\right]^{1/2} \tag{4.9}$$

上式包含 4 个投入方向的谢泼德距离函数：其中两个是**同期距离** $D_i^t(x^t,y^t)$ 与 $D_i^s(x^s,y^s)$，另两个是**跨期距离** $D_i^t(x^s,y^s)$ 与 $D_i^s(x^t,y^t)$，同期与跨期同样在乘号两侧交叉出现。将两个同期距离函数提取至中括号外，上式即可分解为两个部分，即

$$\begin{aligned}M_i&=\frac{D_i^t\left(x^t,y^t\right)}{D_i^s\left(x^s,y^s\right)}\times\left[\frac{D_i^s\left(x^s,y^s\right)}{D_i^t\left(x^s,y^s\right)}\times\frac{D_i^s\left(x^t,y^t\right)}{D_i^t\left(x^t,y^t\right)}\right]^{1/2}\\&=\text{MEFFCH}\times\text{MTECHCH}\end{aligned} \tag{4.10}$$

其中，$MEFFCH$ 衡量观测值相对于投入技术前沿的追赶效应，$MTECHCH$ 衡量生产技术边界的变动。

图 4.2 给出含有一个投入和一个产出的 Malmquist 指数在投入方向上的图形示意。设射线 t 和射线 s 分别代表 t 和 s 时期的 DEA 技术；进一步假设某 DMU 在 t 和 s 时期使用不同数量的投入生产相同数量的产出，分别由 M 点和 N 点表示。其中 A、E 分别是 t 期前沿与 s 期前沿在各自产出水平上的最小投入（即当期参照点），B、F 是跨期参照点（B 在 s 期前沿上、F 在 t 期前沿上）。于是两期的投入 Farrell 效率为 $\text{TE}_t=CA/CM$、$\text{TE}_s=DE/DN$，效率变化部分为 $\frac{DE/DN}{CA/CM}$，对应 (4.10) 式中的 MEFFCH（$=D_i^t(x^t,y^t)/D_i^s(x^s,y^s)=\text{TE}_s/\text{TE}_t$），衡量DMU 在投入缩减意义上的追赶效应。技术变化部分为 $\left[\frac{DN/DE}{DN/DF}\cdot\frac{CM/CB}{CM/CA}\right]^{1/2}=\left[\frac{DF}{DE}\cdot\frac{CA}{CB}\right]^{1/2}$，对应(4.10) 式中的 MTECHCH，衡量了在产出 y 水平上生产前沿的左移（前沿左移时该值大于 1）。

（三）Malmquist指数的分解 4.1.1 节（一）与（二）给出的两要素分解 $M_o=\text{MEFFCH}\times\text{MTECHCH}$（式 (4.5)）与 $M_i=\text{MEFFCH}\times\text{MTECHCH}$（式(4.10)）将生产率变化归因于" 效率变化" 与" 技术变化" 两部分。然而该分解存在两点不足：其一，效率变化 MEFFCH 同时混杂了管理层面的纯效率改进与规模层面的效率变化，无法区分二者；其二，"技术变化" 究竟是相对于规模报酬不变（CRS）前沿还是规模报酬可变（VRS）前沿度量，并未明确。Färe、Grosskopf、Norris 与 Zhang（1994，以下简称 FGNZ）提出的三要素分解，通过同时引入 CRS 与 VRS 两类谢泼德距离函数，将效率变化进一步拆解为纯效率变化与规模效率变化，并将技术变化明确置于CRS 前沿上度量。Ray 与 Desli（1997）则指出该三要素分解并不唯一，并提出

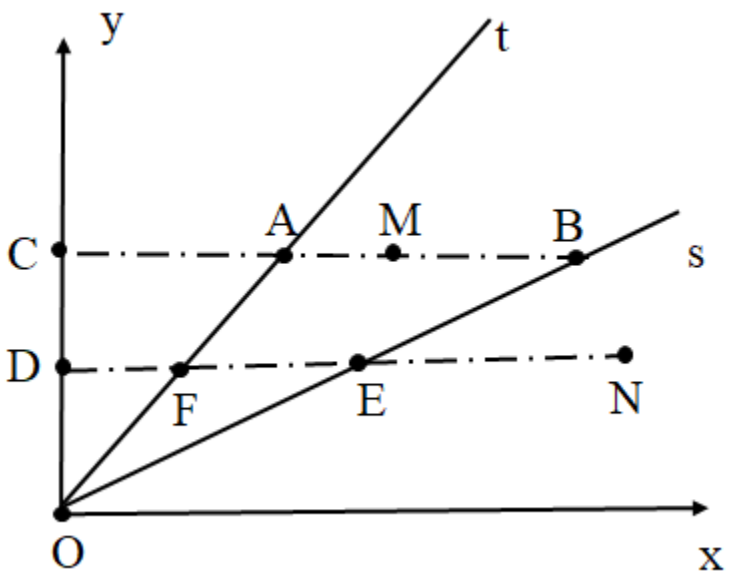


图 4.2: 投入方向的 Malmquist 指数示意图

以VRS 前沿度量技术变化的替代方案。Pastor 与Lovell（2005）从另一思路出发，构造跨期不变的全局技术，给出满足循环性的全局Malmquist 指数。以下分别介绍。

记 $D_o^{t,c}(\cdot)$ 与 $D_o^{t,v}(\cdot)$ 分别为以 t 期技术为参考、在 CRS 与 VRS 假设下的产出方向谢泼德距离函数（输入方向同理，将下标 o 换为 i）；上标 c、v 分别表示 CRS、VRS。

1. FGNZ（1994）三要素分解

FGNZ 将产出导向 Malmquist 指数分解为纯效率变化、规模效率变化与技术变化三个乘积项：

$$\text{PEFFCH} = \frac{D_o^{t,v}(x^t, y^t)}{D_o^{s,v}(x^s, y^s)} \tag{4.11}$$

$$\text{SEFFCH} = \frac{D_o^{t,c}(x^t, y^t) \Big/ D_o^{t,v}(x^t, y^t)}{D_o^{s,c}(x^s, y^s) \Big/ D_o^{s,v}(x^s, y^s)} \tag{4.12}$$

$$\text{TECHCH} = \left[\frac{D_o^{t,c}(x^t, y^t)}{D_o^{s,c}(x^t, y^t)} \cdot \frac{D_o^{t,c}(x^s, y^s)}{D_o^{s,c}(x^s, y^s)} \right]^{1/2} \tag{4.13}$$

三项相乘即还原为 Malmquist 指数：

$$M_o = \text{PEFFCH} \times \text{SEFFCH} \times \text{TECHCH} \tag{4.14}$$

其中，PEFFCH（纯效率变化，pure efficiency change）以 VRS 前沿为基准，刻画 DMU 相对于”最佳实践边界”的管理追赶效应，剔除了规模因素；SEFFCH（规模效率变化，scale efficiency change）为两期 CRS 效率与 VRS 效率之比的比值，衡量 DMU 是否向最优规模移动；TECHCH（技术变化）在 CRS 前沿上度量生产边界外移，其形式与式 (4.5) 中的 MTECHCH 一致。易见，两要素分解满足 MEFFCH = PEFFCH × SEFFCH 且 MTECHCH = TECHCH，故三要素分解是 (4.5) 在同时拥有 CRS、VRS 距离函数时的细化。各分量大于 1 分别表示纯效率改善、趋近最优规模、技术进步。

2. Ray 与 Desli（1997）的替代分解

Ray 与 Desli（1997）指出，FGNZ 的三要素分解并非唯一：若将 TECHCH 中的一部分规模相关因子转移进 SEFFCH，可得到形式上不同的分解，且不同分解给出的规模效率变化与技术变化解读并不一致。他们主张技术变化应相对于VRS 前沿（而非 CRS 前沿）度量，因为 VRS 前沿才是与决策单元实际所处规模相对应的生产边界。其分解将效率变化直接取为FGNZ 的纯效率变化：

$$\text{EFFCH} = \frac{D_o^{t,v}(x^t, y^t)}{D_o^{s,v}(x^s, y^s)} = \text{PEFFCH} \tag{4.15}$$

技术变化在 VRS 前沿上使用同期与跨期距离计算。以下用 t 表示基期、$t+1$ 表示报告期：

$$\text{TECHCH}^{\text{RD}} = \left[\frac{D_o^{t,v}(x^{t+1}, y^{t+1})}{D_o^{t+1,v}(x^{t+1}, y^{t+1})} \frac{D_o^{t,v}(x^t, y^t)}{D_o^{t+1,v}(x^t, y^t)} \right]^{1/2} \tag{4.16}$$

保留 CRS Malmquist 总指数时，还需单列规模变化项 $\text{SCALECH}^{\text{RD}} = M_o/(\text{EFFCH}\,\text{TECHCH}^{\text{RD}})$：

$$M_o = \text{EFFCH} \times \text{TECHCH}^{\text{RD}} \times \text{SCALECH}^{\text{RD}} \tag{4.17}$$

不能将 M_o/EFFCH 直接称为 VRS 技术变化，它还包含规模变化。代码分别返回 `EFFCH`、`TECHCH_RD`、`SCALECH_RD` 和 `MQ`；VRS跨期距离不可行时保留对应分解项缺失，不能仅凭总指数反推技术变化。

3. Pastor 与 Lovell（2005）全局 Malmquist 指数

前述分解均依赖相邻两期技术的几何平均；当考察多期序列时，式(4.4) 中跨期距离函数的参考技术随期而变，可能导致指数不满足循环性（即 $M(t,s) \times M(s,r) \neq M(t,r)$）。Pastor 与Lovell（2005）提出以跨期不变的全局技术 T^G 为统一参考：

$$T^G = \left\{ (x,y) : \sum_{t=1}^{T} \sum_{j=1}^{N_t} \lambda_j^t y_j^t \geq y,\ \sum_{t=1}^{T} \sum_{j=1}^{N_t} \lambda_j^t x_j^t \leq x,\ \lambda_j^t \geq 0 \right\} \tag{4.18}$$

即把各期所有观测值汇集到同一参考集下构造前沿。基于全局技术的 Malmquist 指数定义为相邻两期到全局前沿距离之比：

$$\text{mqpi} = \frac{D^G(x^t, y^t)}{D^G(x^{t-1}, y^{t-1})} \tag{4.19}$$

该指数天然满足循环性与传递性，且因参考技术固定，只需对每期计算一次全局距离即可，无需跨期距离函数。在 `deabook` 包中，将 `DEA.mq`/`DEA.mqddf` 的 `tech` 参数设为 `GLOBAL` 即对应此全局技术（返回列 `mqpi`，见 4.2 节全局技术组）；而前述 FGNZ 与Ray–Desli 分解所需的 CRS、VRS 距离函数，可分别调用 `rts=RTS_CRS` 与 `rts=RTS_VRS` 两次运行 `DEA.mq`，再按式 (4.11)–式 (4.17) 组合得到。在 `deabook` 中，这一组合已由独立接口 `DEA.mqfgnz`（FGNZ 三要素分解）、`DEA.mqrd`（Ray–Desli 分解）与 `DEA.gmqdecom`（跨期不变全局前沿分解）自动完成，见 4.2.5 节；三者的输入、返回列与 4.2 节的 `DEA.mq` 完全对应，区别仅在于将 CRS 与 VRS 两个径向距离函数组合成三要素分解。

4.1.2 基于方向距离函数的Malmquist指数

4.1.1 节构建的谢泼德距离函数属于径向测度——产出方向只能沿 y 等比例扩张、投入方向只能沿 x 等比例收缩，调整方向单一。当决策单元希望同时扩张产出、缩减投入，或仅沿某一子向量调整时，径向测

度不再适用。方向距离函数（DDF）通过引入方向向量 $g=(g_x,g_y)$，允许沿任意方向同时调整产出与投入，弥补了这一局限。本节在DDF 框架下重新表述 Malmquist 指数，并建立其与 4.1.1 节谢泼德形式的对应关系。

为简化记号，本节将沿方向 g、参考技术 T^t、生产计划 (x^s,y^s) 的方向距离函数简记为 $\vec{D}^t(s)$，即 $\vec{D}^t(s)\equiv\vec{D}(x^s,y^s;g)$ 以 T^t 为参考技术；上标为参考技术所在时期，括号内为待估生产计划所在时期。下文涉及两个方向：投入方向 $g=(x,0)$（对应(4.20)、(4.22)、(4.23)）与产出方向 $g=(0,y)$（对应(4.21)、(4.24)、(4.25)）。

特定方向的方向距离函数与谢泼德距离函数之间存在确定的关系（Chambers, Chung and Färe, 1998；产出方向的对应形式见 Chung, Färe and Grosskopf, 1997），据此可以使用投入或产出方向的方向距离函数构建 Malmquist 指数。具体做法是依据公式

$$\vec{D}(x,y;x,0)=1-1/D_i(x,y) \tag{4.20}$$

将 Malmquist 指数计算公式中的投入距离函数替换为沿 $g=(x,0)$ 方向的方向距离函数；或者，依据公式

$$\vec{D}(x,y;0,y)=\frac{1}{D_o(x,y)}-1 \tag{4.21}$$

将公式中的产出距离函数替换为沿 $g=(0,y)$ 方向的方向距离函数。当采用 $g=(x,0)$ 方向的方向距离函数衡量效率时，Malmquist 指数可计算为：

$$M(s,t)=\sqrt{M^sM^t}=\sqrt{\frac{1-\vec{D}^t(s)}{1-\vec{D}^t(t)}\frac{1-\vec{D}^s(s)}{1-\vec{D}^s(t)}} \tag{4.22}$$

式中，上标 t,s 表示参考技术所在时期，括号内 t,s 表示待估计的 DMU 所在时期。该指数可分解为：

$$\begin{aligned}M(s,t)&=\frac{1-\vec{D}^s(s)}{1-\vec{D}^t(t)}\cdot\sqrt{\frac{1-\vec{D}^t(s)}{1-\vec{D}^s(s)}\cdot\frac{1-\vec{D}^t(t)}{1-\vec{D}^s(t)}}\\&=MEFFCH(s,t)\cdot MTECHCH(s,t)\end{aligned} \tag{4.23}$$

同理，采用 $g=(0,y)$ 方向的方向距离函数计算效率时，Malmquist 指数为：

$$M(s,t)=\sqrt{M^sM^t}=\sqrt{\frac{1+\vec{D}^t(t)}{1+\vec{D}^t(s)}\frac{1+\vec{D}^s(t)}{1+\vec{D}^s(s)}} \tag{4.24}$$

同样可分解为：

$$\begin{aligned}M(s,t)&=\frac{1+\vec{D}^t(t)}{1+\vec{D}^s(s)}\cdot\sqrt{\frac{1+\vec{D}^s(s)}{1+\vec{D}^t(s)}\cdot\frac{1+\vec{D}^s(t)}{1+\vec{D}^t(t)}}\\&=MEFFCH(s,t)\cdot MTECHCH(s,t)\end{aligned} \tag{4.25}$$

至此，本节已在 DDF 框架下重新表述了 Malmquist 指数。当方向向量取 $g=(0,y)$ 或 $g=(x,0)$ 时，由 (4.21) 与 (4.20) 可知，DDF 形式的 Malmquist 指数与 4.1.1 节基于谢泼德距离函数的Malmquist 指数在数值上等价，差异仅在于表述形式——前者以加性冗余 $\vec{D}$ 度量，后者以径向比值 D_o、D_i 度量。这一等价性将在 4.3 节的实证中通过数值对照加以印证（其代数依据见(4.3)）。下一节介绍如何用 `deabook` 包实现上述全部模型。

4.2 生产率测度的Python实现

4.1 节已分别从谢泼德距离函数(4.1.1 节)与方向距离函数(4.1.2 节)两条路径，将 Malmquist 指数及其效率变化—技术变化分解归结为若干组线性规划。这些距离函数本质上是第 3.5 章 CCR、BCC 与方向距离函数(DDF)模型的特例：产出（或投入）导向的谢泼德距离对应 `DEA.ccr`/`DEA.bcc` 的径向距离，DDF 形式则直接对应 `DEA.ddf`。因此，生产率的 Python 实现并非引入新的求解引擎，而是以既有的 CCR/BCC/DDF 模型为计算内核，按式(4.4)–式 (4.10) 与式 (4.22)–式(4.25) 的代数结构，对相邻期与跨期的一组距离值加以组合与分解。循此逻辑，`deabook` 以 `DEA.mq()` 封装径向谢泼德形式、`DEA.mqddf()` 封装 DDF 形式；二者通过 `orient` 区分产出与投入导向，通过 `rts` 在CCR 与 BCC 间切换，并通过 `tech` 参数与第 3.4 章的四种技术参照集——当期 `CONTEMPORARY`（默认）、窗口 `WINDOW`、时序 `SEQUENTIAL`、全局 `GLOBAL`——一一对应（其数学定义分别见 3.4.3 节当期、3.4.4 节 Window、3.4.5 节时序、3.4.6 节全局，及式(3.22)–(3.27)）。这一设计使理论部分的参考集口径与实现部分完全一致，第三章建立的LP 框架可直接复用于生产率测度，下文仅在组合方式上做相应调整。

与第 3.5 章的 `DEA.ccr(...).optimize(...)` 一致，本节接口采用**惰性求解**：`DEA.mq(...)` 等工厂只构造模型、不立即计算，需调用 `.optimize(email, solver)` 才真正求解并返回结果；`email`、`solver` 也随之从构造参数移到了 `.optimize(...)` 的参数上。五个接口全部只基于 3.5 节的 `DEA.ccr`、`DEA.bcc` 与 `DEA.ddf` 实现，不再使用任何旧接口；`DEA.mqddf()` 的效率换算遵循 3.6 的规则（投入方向 $te = 1 - \rho$、产出方向 $te = 1/(1+\rho)$）。下文围绕各接口的经济含义与组合逻辑展开，而非以说明书式顺序罗列；读者可由此理解每一组距离值所对应的生产率分量，再辅以可运行示例把握调用约定。

4.2.1 接口导入与模块组织

`deabook` 的全部 Malmquist 计算由 `DEA` 模块下的五个并列接口完成，其导入与依赖如以下代码所示；这些接口共享同一套CCR/BCC/DDF 求解后端，差异仅在于将所得距离值按不同公式组合。

```
from deabook import data_path, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook import DEA, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook.constant import RTS_VRS, RTS_CRS, OPT_LOCAL
import pandas as pd
```

其中，`DEA.mq()`（径向谢泼德形式）与 `DEA.mqddf()`（DDF 形式）给出两要素分解（效率变化×技术变化）；`DEA.mqfgnz()`（FGNZ 1994）、`DEA.mqrd()`（Ray–Desli 1997）与 `DEA.gmqdecom()`（Pastor–Lovell 全局前沿）进一步给出三要素分解。导入的常量按功能分三组：`RTS_VRS`/`RTS_CRS` 控制规模报酬假设，`CONTEMPORARY`/`WINDOW`/`SEQUENTIAL`/`GLOBAL` 控制技术参照集（对应第 3.4 章四种参照技术），`OPT_LOCAL` 与 `solver` 控制求解方式。后三个三要素接口内部各自以 `RTS_CRS` 与 `RTS_VRS` 两次调用 `DEA.mq` 的径向距离，再按式(4.11)–式 (4.17) 组合，故用户无需手动拼接；它们与 `DEA.mq`、`DEA.mqddf` 共用同一套距离函数与求解后端，返回列亦在同一 `data` 上追加。

4.2.2 模型参数与参考集设定

五个接口共享同一组构造参数，其含义可从”测什么”与”以何参照”两个维度理解。`data`、`id`、`year`、`sent` 四项界定面板数据本身：`sent` 以等号分隔投入与产出变量，刻画第 4.1 节生产技术 T 的要素构成；`id` 与 `year` 则用于在跨期参考集中识别同一决策单元、排序相邻时期，从而构造式(4.4) 所需的同期与跨期距离。`orient` 决定生产率的度量方向——产出导向 `oo` 刻画在投入不变下扩张全部产出（对应谢泼德产出距离 D_o），投入导向 `io` 刻画在产出不变下缩减全部投入（对应谢泼德投入距离 D_i）。`rts` 在规模报酬不变（`RTS_CRS`，底层调用 `DEA.ccr`）与可变（`RTS_VRS`，底层调用 `DEA.bcc`）之间切换，直接决定距离函数所参照的前沿形状。`tech` 则决定参考集的跨期构成，与第 3.4 章的四种参照技术一一对应，其具体取值与含义汇总于表 表 4.1。

表 4.1: `DEA.mq` / `DEA.mqddf` 模型的主要参数

参数	类型	可选值	含义
`data`	DataFrame	—	面板数据，需包含 `sent` 中指定的列、`id` 列和 `year` 列
`id`	str	列名（如 `dmuname`）	DMU 标识列名，用于分组计算跨期效率
`year`	str	列名（如 `"year"`）	时间列名，需可排序
`sent`	str	`"投入1 投入2 ...=产出1 产出2 ..."`	`=` 左边为投入变量，右边为产出变量
`orient`	str	`"oo"`	产出导向：扩大全部产出，投入不变（对应谢泼德产出距离 D_o）
	str	`"io"`	投入导向：缩减全部投入，产出不变（对应谢泼德投入距离 D_i）
`rts`	str	`RTS_CRS`	规模报酬不变（底层调用 `DEA.ccr`）
	str	`RTS_VRS`	规模报酬可变（底层调用 `DEA.bcc`）
`tech`	str	`CONTEMPORARY`（默认）	当期技术：参考集仅为第 t 期观测（§3.4-3，式(3.22)）；计算D11、D12、D21

参数	类型	可选值	含义
	str	`WINDOW`	窗口技术：参考集为以 t 为中心、半宽 $\lfloor w/2 \rfloor$ 的局部窗（§3.4-4，式 (3.23)–(3.24)）
	str	`SEQUENTIAL`	时序技术：参考集为所有 $s \le t$ 期观测（§3.4-5，式 (3.25)），技术只进不退
	str	`GLOBAL`	全局技术：所有期合并为同一前沿（§3.4-6，式 (3.27)）；只计算 D11，额外返回 `mqpi`
`window`	int	奇数，默认 3	仅 `tech=WINDOW` 生效：窗口**全宽**，实际窗口为 $2\lfloor w/2 \rfloor + 1$ 期；`w=3` 即 $[t-1, t, t+1]$
`email`	str	`OPT_LOCAL`	本地求解（传给 `.optimize(...)`）
	str	邮箱地址	远程求解完成后发送通知
`solver`	str	`"mosek"`	商业求解器，速度快、精度高（传给 `.optimize(...)`）
	str	`"glpk"`	开源求解器
	str	`"cbc"`	开源求解器

表 表 4.1 中，`window` 仅在 `tech=WINDOW` 时生效，表示窗口的全宽（实际跨期数为 $2\lfloor w/2 \rfloor + 1$，`w=3` 即 $[t-1, t, t+1]$）。值得注意的是，`email` 与 `solver` 并不随工厂传入，而是在调用 `.optimize(email, solver)` 时给出，这与第 3.5 章 `DEA.ccr(...).optimize(...)` 的用法一致，也呼应了前述惰性求解的设计。`tech` 与 `orient` 二者共同决定 `.optimize()` 返回的列结构，下文即沿这两个维度展开对返回值的说明。

4.2.3 返回值的经济含义

`optimize()` 返回的 DataFrame，其列结构由 `tech` 参数与 `orient` 导向共同决定，因此也是在把第 4.1 节的理论分量映射为可解释的变量。本章涉及的 `DEA.mq` 与 `DEA.mqddf` 两个接口返回列名完全一致，差

异仅在底层距离值的计算方式（径向距离 vs DDF 冗余），故下文以编号表（ 表 4.2–4.6）覆盖两类接口在 `tech`（当期/窗口/时序/全局）与导向（`orient`）维度上的返回值，而不重复制表。读者可先把握两套列名模板：当期/窗口/时序技术的 `D11/D12/D21/MQ/MEFFCH/MTECHCH`、全局技术的 `D11/mqpi/MEFFCH/MTECHCH`（即 Pastor–Lovell 分解，$mqpi = \text{MEFFCH} \times \text{MTECHCH}$）。其中 `D11`、`D12`、`D21` 为构造指数所依赖的各期距离值，而 `MQ`、`MEFFCH`、`MTECHCH` 才是具有经济含义的生产率分量——分别度量总生产率变化、效率追赶与技术前沿移动；`mqpi` 则是全局前沿下的总指数。理解这些列名的经济含义，是解读 4.3 节实证结果的前提。

（一）通用列

表 4.2: 通用返回列

模型	通用列
`DEA.mq`	`optimization_status`
`DEA.mqddf`	`optimization_status`

（二）`DEA.mq`（径向 Malmquist）— 当期/窗口/时序技术

表 4.3: `DEA.mq` 当期/窗口/时序技术

返回列	含义
`D11`	当期技术下，t 期 DMU 对 t 期参考集的效率距离值
`D12`	当期技术下，t 期 DMU 对 $t-1$ 期参考集的效率距离值（首期为 NaN）
`D21`	当期技术下，t 期 DMU 对 $t+1$ 期参考集的效率距离值（末期为 NaN）
`MQ`	Malmquist 指数（> 1 表示生产率提升）
`MEFFCH`	效率变化（追赶效应），$= D11_t / D11_{t-1}$
`MTECHCH`	技术变化（前沿移动效应）

分解关系：$MQ = MEFFCH \times MTECHCH$。第一年无 MQ 值（需要相邻两期对比，首年 `MQ/MEFFCH/MTECHCH` 均为 NaN）。

以上是当期技术（`CONTEMPORARY`）下的 `DEA.mq` 返回表（表 4.3），核心列是 `MQ/MEFFCH/MTECHCH`，第一年因无相邻期对比而为NaN。窗口（`WINDOW`）与时序（`SEQUENTIAL`）技术沿用同一套列名模板，只是参考集跨期构成不同（见表 4.1）。下面转入全局技术（`GLOBAL`）组：由于参考集跨期不变，只需一个全局距离即可构造指数，故D12、D21 不再单独列出，仅保留全局距离值 `D11` 与相邻期比值 `mqpi`（即全局 Malmquist 指数，遵循 Pastor and Lovell, 2005 的全局前沿框架）。

（三）`DEA.mq`（径向 Malmquist）— GLOBAL

表 4.4: DEA.mq 全局技术

返回列	含义
D11	全局技术下，t 期 DMU 对全时期统一参考集的效率距离值 $D^G(x^t, y^t)$
mqpi	全局 Malmquist 生产率指数，$= D11_t / D11_{t-1}$
MEFFCH	效率变化：相邻两期**当期**距离之比，$= D^t(x^t, y^t) / D^{t-1}(x^{t-1}, y^{t-1})$
MTECHCH	技术变化：最佳实践缺口（BPG）变化，$= \left[D_t^G / D_t^t\right] / \left[D_{t-1}^G / D_{t-1}^{t-1}\right]$

全局技术下，mqpi 是两期观测相对**同一全局前沿**的距离之比，即 $mqpi = D^G(x^t, y^t)/D^G(x^{t-1}, y^{t-1})$（产出导向下距离取谢泼德 D_o，绩效提高时该比值大于 1）。按 Pastor and Lovell (2005)，该指数可分解为 $mqpi =$ MEFFCH $\times$ MTECHCH：其中 MEFFCH 是**当期技术**下相邻两期距离之比，刻画效率追赶；MTECHCH 是各期当期技术相对全局技术的**最佳实践缺口**（$BPG_t = D_t^G / D_t^t$）的变化，刻画生产边界的移动。这一分解同时计算全局与当期两类距离，故 mqpi 恒等于 MEFFCH 与 MTECHCH 的乘积。

至此 DEA.mq（径向）的两张返回表（表 4.3、4.4）已罗列完毕。下面为 DEA.mqddf（DDF）的对应配置，**列名与 DEA.mq 完全一致**，差异仅在底层距离值的计算方式（汇总见表 4.6）：径向投入导向存 $te = \rho$、产出导向存 $te = 1/\rho$；DDF 投入导向存 $tei = 1 - \rho$、产出导向存 $teo = 1/(1+\rho)$。该换算关系已在 4.1 节(4.20)/(4.21) 给出，与 3.6 一致。

（四）DEA.mqddf（DDF Malmquist）— 当期/窗口/时序

表 4.5: DEA.mqddf 当期/窗口/时序技术

返回列	含义
D11	当期技术下，t 期 DMU 对 t 期参考集的效率距离值
D12	当期技术下，t 期 DMU 对 $t-1$ 期参考集的效率距离值（首期为 NaN）
D21	当期技术下，t 期 DMU 对 $t+1$ 期参考集的效率距离值（末期为 NaN）
MQ	Malmquist 指数（> 1 表示生产率提升）
MEFFCH	效率变化（追赶效应），$= D11_t / D11_{t-1}$
MTECHCH	技术变化（前沿移动效应）

与 表 4.3 列名完全相同。差异仅在底层距离值计算方式：投入导向下底层 $tei = 1 - \rho$ 存入 D11；产出导向下底层 $teo = 1/(1+\rho)$ 存入 D11。全局技术（GLOBAL）下沿用全局列名模板 D11/mqpi。

（五）DEA.mq vs DEA.mqddf 底层差异汇总

表 4.6: DEA.mq 与 DEA.mqddf 底层差异汇总

特征	DEA.mq	DEA.mqddf
底层模型	DEA.ccr/DEA.bcc（径向）	DEA.ddf（方向距离函数）
投入导向距离值	谢泼德投入距离 $D_i = 1/te$，存入 D11	底层 $tei = 1 - \rho$，存入 D11
产出导向距离值	谢泼德产出距离 $D_o = te$，存入 D11	底层 $teo = 1/(1+\rho)$，存入 D11
分解公式	$MQ = MEFFCH \times MTECHCH$	相同
mqpi 公式	$mqpi = D11_t/D11_{t-1}$	相同

为便于对照，下表将 tech 的四个取值与第 3.4 章的参照集定义一一对应（式 (3.22)–(3.27)）：

表 4.7: tech 取值与第3章技术参照集的对应

tech	参考集（第 t 期）	技术演变假设	对应第3章
CONTEMPORARY（默认）	仅第 t 期观测	每期各有前沿，可进可退	§3.4-3，式 (3.22)
WINDOW	以 t 为中心、半宽 $\lfloor w/2 \rfloor$ 的窗	窗口内不变，窗口间可变	§3.4-4，式 (3.23)–(3.24)
SEQUENTIAL	所有 $s \le t$ 期观测	技术只进不退	§3.4-5，式(3.25)
GLOBAL	全部时期观测	全样本期技术不变	§3.4-6，式(3.27)

以上差异对 tech=CONTEMPORARY 和 tech=GLOBAL 均成立。

需要说明的是，由 (4.21) 可知，产出导向下 $teo = 1/(1+\rho) = D_o$，即 DEA.mqddf 存入 D11 的底层值与 DEA.mq 存入的谢泼德产出距离 D_o 完全相同，两者因而共用同一套 MQ/MEFFCH/MTECHCH 分解公式（见表 4.6“分解公式” 行）。换言之，在相同方向设定（如 $g = (0, y)$）下，DEA.mqddf 与 DEA.mq 给出完全一致的结果。该一致性由(4.20)/(4.21) 的代数关系构造性地保证，4.3 节表 4.11 与 表 4.11 的逐元吻合即为此关系的数值体现，而非独立的假设检验。

返回值结构至此梳理完毕。下面以七个最小可运行示例展示 DEA.mq 与 DEA.mqddf 在产出/投入导向、当期/窗口/时序/全局技术下的调用方式，分别对应 4.1 节的 (4.4)/(4.9)/(4.24) 三组公式。

4.2.4 调用示例与约定

为说明接口的调用约定，以下七个示例覆盖 4.1 节的常用组合：示例1–2 在产出导向下对照谢泼德与 DDF 两种形式，示例 3–4 给出对应的投入导向版本，示例 5 切换到全局技术，示例 6–7 演示时序与窗口技术。需特别留意，email、solver 现在传给.optimize(...) 而非工厂；不显式写 tech 时即默认当期技术。每一示例后所附说明，均指出其对应的理论公式，以便读者将代码与第 4.1 节的推导一一对应。

示例 1：DEA.mq 产出导向 — CRS + 当期技术（对应公式(4.4)–(4.5)，产出方向谢泼德距离函数。这是 4.3 节 OECD 实证的基准设定，给出标准 FGNZ (1994) 式 CRS 全要素生产率指数）：

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
model = DEA.mq(
    data, id=dmuname, year='year',
    sent="K L E=Y", orient="oo",
    rts=RTS_CRS, tech=CONTEMPORARY,
)
res = model.optimize(email=OPT_LOCAL, solver="mosek")
```

示例 2：`DEA.mqddf` 产出导向 — CRS + 当期技术（对应公式 (4.24)–(4.25)，以 $g = (0, y)$（即 `gx=[0,0,0]`, `gy=[1]`）替代径向产出测度，与示例 1 在该方向下数值等价，见 (4.21)；底层 $teo = 1/(1+\rho)$，换算见 3.6）：

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
model = DEA.mqddf(
    data, id=dmuname, year='year',
    sent="K L E=Y", gx=[0, 0, 0], gy=[1],
    rts=RTS_CRS, tech=CONTEMPORARY,
)
res = model.optimize(email=OPT_LOCAL, solver="mosek")
```

示例 3：`DEA.mq` 投入导向 — CRS + 当期技术（对应公式(4.9)–(4.10)，投入方向谢泼德距离函数；`orient="io"` 表示缩减全部投入，产出保持不变。底层存储的是 Farrell 投入效率 $te = 1/D_i$，故 `MQ>1` 仍表示生产率提升——旧 `MQDEA` 在 `orient="io"` 下返回其倒数，已在 `DEA.mq` 中修正）：

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
model = DEA.mq(
    data, id=dmuname, year='year',
    sent="K L E=Y", orient="io",
    rts=RTS_CRS, tech=CONTEMPORARY,
)
res = model.optimize(email=OPT_LOCAL, solver="mosek")
```

示例 4：`DEA.mqddf` 投入导向 — CRS + 当期技术（对应公式 (4.22)–(4.23)，以 $g = (x, 0)$（即 `gx=[1,0,0]`, `gy=[0]`）方向的 DDF 替代径向投入测度，与示例 3 数值等价，见 (4.20)；底层 $tei = 1 - \rho$，换算见 3.6）：

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
model = DEA.mqddf(
    data, id=dmuname, year='year',
    sent="K L E=Y", gx=[1, 0, 0], gy=[0],
    rts=RTS_CRS, tech=CONTEMPORARY,
)
```

```
res = model.optimize(email=OPT_LOCAL, solver="mosek")
```

示例 5：DEA.mq 产出导向 — CRS + 全局技术（采用 `tech=GLOBAL`，参考集跨期不变，返回 `D11` 与 `mqpi`，并按 Pastor–Lovell 给出 `MEFFCH/MTECHCH` 分解；可避免当期技术下相邻期参考集不可行的问题）：

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
model = DEA.mq(
    data, id=dmuname, year='year',
    sent="K L E=Y", orient="oo",
    rts=RTS_CRS, tech=GLOBAL,
)
res = model.optimize(email=OPT_LOCAL, solver="mosek")
```

示例 6：DEA.mq — 时序技术（`tech=SEQUENTIAL`，参考集为所有 $s \le t$ 期观测，技术只进不退，对应式 (3.25)）：

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
model = DEA.mq(
    data, id=dmuname, year='year',
    sent="K L E=Y", orient="oo",
    rts=RTS_CRS, tech=SEQUENTIAL,
)
res = model.optimize(email=OPT_LOCAL, solver="mosek")
```

示例 7：DEA.mq — 窗口技术（`tech=WINDOW` 并配 `window=3`，参考集为以 t 为中心、半宽 1 的局部窗 $[t-1, t, t+1]$，对应式 (3.23)）：

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
model = DEA.mq(
    data, id=dmuname, year='year',
    sent="K L E=Y", orient="oo",
    rts=RTS_CRS, tech=WINDOW, window=3,
)
res = model.optimize(email=OPT_LOCAL, solver="mosek")
```

上述七个示例仅演示了 API 语法，未涉及真实数据。下一节将这些接口应用于 33 个 OECD 国家与中国 30 个省份 2016–2019 年的面板数据，完成两项任务：一是以数值对照呈现 4.1 节(4.20)/(4.21) 隐含的等价性命题（其紧凑表达见 4.3 节 (4.3) 式）——产出方向谢泼德形式与 DDF 形式在 $g = (0, y)$ 下应给出完全一致的生产率指数；二是刻画两类样本动态产出效率的贡献结构（技术进步vs 效率追赶）。

4.2.5 生产率分解：DEA.mqfgnz、DEA.mqrd、DEA.gmqdecom 及其 DDF 变体

第 4.1.1 节（三）指出，FGNZ（1994）三要素分解与Ray–Desli（1997）分解的经济意义，在于把生产率变化进一步拆解为纯效率变化、规模效率变化与技术变化三项，从而区分管理层面的追赶与前沿本身的外移。但在计算上，二者都需要同时取得CRS 与 VRS 两类谢泼德距离函数，亦即各运行一次 DEA.mq 后，再按式(4.11)–式 (4.17) 手工组合。为避免这种" 两次运行＋手工拼接" 的重复劳动，deabook 将上述组合逻辑封装为以下并列接口，由求解后端自动完成：

- DEA.mqfgnz(...) —— FGNZ（1994）三要素分解，返回 PEFFCH、SEFFCH、TECHCH、MQ；
- DEA.mqrd(...) —— Ray–Desli（1997）分解，返回 EFFCH、TECHCH_RD、SCALECH_RD、MQ；
- DEA.gmqdecom(...) —— 跨期不变的全局前沿（Pastor and Lovell, 2005）分解，tech=GLOBAL 下按 Pastor–Lovell 返回 PEFFCH、SEFFCH、TECHCH、MQ、mqpi；
- DEA.mqfgnz.ddf(...) / DEA.mqrd.ddf(...) / DEA.gmqdecom.ddf(...) —— 以上三者的 **DDF 变体**：距离值改由 DEA.ddf 求得（按 3.6 换算），方向由 gy/gx 给定（仅支持单侧方向），分解结构与返回列和径向版完全一致，等价方向下数值亦完全一致。

四组接口与 DEA.mq、DEA.mqddf 共用同一个 tech 参数，支持第 3.4 章的全部四种技术参照集（当期 CONTEMPORARY、窗口 WINDOW、时序 SEQUENTIAL、全局 GLOBAL），mqfgnz（及其 DDF 变体）与 mqrd（及其 DDF 变体）默认当期技术，gmqdecom（及其 DDF 变体）默认全局技术（即Pastor–Lovell 前沿）。窗口带宽 window 默认 3（全宽，即 $[t-1, t, t+1]$）。

4.2.6 三要素分解接口的参数、返回值与调用示例

DEA.mqfgnz / DEA.mqrd / DEA.gmqdecom 的参数与 DEA.mq 完全一致（见 表 4.1），只是 tech 的默认值不同：mqfgnz/mqrd 默认当期（CONTEMPORARY），gmqdecom 全局（GLOBAL）。各接口内部自动以 RTS_CRS 与 RTS_VRS 各运行一次距离，故不需要 rts 参数。三个 .ddf 变体（mqfgnz.ddf/mqrd.ddf/gmqdecom.ddf）的参数与 DEA.mqddf 对应：以 gy/gx（单侧方向）替代 orient，其余相同，默认 tech 与各自的径向版一致。

> 时序技术（SEQUENTIAL）下，第 t 期 DMU 的参考集为所有年份 $\leq t$ 的 DMU（累积前沿）；窗口技术（WINDOW）下为以 t 为中心、带宽 window 的局部窗口（如 $[t-1, t, t+1]$）。二者都比当期技术拥有更大的参考集，因而相邻期跨期距离（式(4.11)–式 (4.17) 中的 $D_o^s(x^t, y^t)$、$D_o^t(x^s, y^s)$）更不易不可行——这正是第 4.3 节 VRS 当期技术大量出现 NaN 时的常用替代方案。全局技术（GLOBAL）参考集为全部时期的全部DMU，技术参照跨期不变。

4.2.6.1 返回值

四个接口均在输入的 data 上**追加**分解列后返回，行索引与输入一致，且不输出中间距离列 D*_v/D*_c；首期（及窗口/时序边界）因缺少相邻期对比而为 NaN。

（一）DEA.mqfgnz 返回列

表 4.8: DEA.mqfgnz 返回列

返回列	含义
PEFFCH	纯效率变化，$= D_o^{t,v}(x^t, y^t)/D_o^{s,v}(x^s, y^s)$，VRS 基准（式(4.11)）
SEFFCH	规模效率变化，$= (D_o^{t,c}/D_o^{t,v})/(D_o^{s,c}/D_o^{s,v})$，CRS/VRS 之比（式(4.12)）
TECHCH	技术变化，CRS 前沿上度量（式(4.13)；全局技术下恒为 1）
MQ	Malmquist 指数，= PEFFCH × SEFFCH × TECHCH（式 (4.14)）
MEFFCH	两要素效率变化别名，= PEFFCH × SEFFCH（与 DEA.mq 的 MEFFCH 对应）
MTECHCH	两要素技术变化别名，= TECHCH（与 DEA.mq 的 MTECHCH 对应）

（二）DEA.mqrd 返回列

表 4.9: DEA.mqrd 返回列

返回列	含义
EFFCH	效率变化，$= D_o^{t,v}(x^t, y^t)/D_o^{s,v}(x^s, y^s)$，即 FGNZ 的 PEFFCH（式 (4.15)）
TECHCH_RD	技术变化，改在 VRS 前沿上度量，使用 VRS 同期和跨期距离（式 (4.16)）
SCALECH_RD	规模变化，= MQ / (EFFCH × TECHCH_RD)
MQ	Malmquist 指数，= EFFCH × TECHCH_RD × SCALECH_RD（式 (4.17)）

（三）DEA.gmqdecom 返回列（tech=GLOBAL，即Pastor–Lovell 分解）

表 4.10: DEA.gmqdecom 返回列

返回列	含义
PEFFCH	纯效率变化：**当期技术**下相邻两期 VRS 距离之比，$D^{t,v}(x^t)/D^{t-1,v}(x^{t-1})$
SEFFCH	规模效率变化：**当期技术**下 CRS/VRS 之比的比值

返回列	含义
TECHCH	技术变化：最佳实践缺口（BPG）变化，$[D_t^G/D_t^t]/[D_{t-1}^G/D_{t-1}^{t-1}]$
MQ	全局 Malmquist 指数，= PEFFCH × SEFFCH × TECHCH
MEFFCH	两要素效率变化别名，= PEFFCH × SEFFCH（对应 DEA.mq 全局技术的 MEFFCH）
MTECHCH	两要素技术变化别名，= TECHCH（对应 DEA.mq 全局技术的 MTECHCH）
mqpi	全局生产率指数（CRS 全局距离比），$= D11_t/D11_{t-1}$，= MQ

在 tech=GLOBAL 下，gmqdecom 同时求解三组距离——当期VRS、当期 CRS 与全局 CRS——并按 Pastor–Lovell 把全局指数分解为" 当期纯效率变化 × 当期规模效率变化 × 最佳实践缺口变化"。若改用其他 tech 值，则退回该参照集下的 FGNZ 式三要素分解。

（四）DEA.gmqdecom.ddf 返回列

DEA.gmqdecom.ddf 的返回列与 表 4.10 完全一致，区别仅在于底层距离值改由 DEA.ddf 求得（按 3.6 换算，投入方向 $tei = 1 - \rho$、产出方向 $teo = 1/(1+\rho)$），方向由 gy/gx 给定。在等价方向（如 $g = (0, y)$ 对应 orient="oo"）下，其各列与 DEA.gmqdecom 数值完全一致。

4.2.6.2 调用示例

以下示例展示 mqfgnz 在各参照集、mqrd 在当期技术，以及 gmqdecom 的调用方式（仅演示 API，未附结果，读者可自行运行对照）；它们与 4.2.4 节的 DEA.mq 示例共用同一份 data、同一组 sent/orient/tech/solver 参数，区别仅在于分解方式由两要素转为三要素。

示例 6：DEA.mqfgnz — 当期技术（三要素分解）

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
res_fgnz = DEA.mqfgnz(
    data, id=dmuname, year='year',
    sent="K L E=Y", orient="oo",
    tech=CONTEMPORARY,          # 当期技术参照集（默认）
).optimize(email=OPT_LOCAL, solver="mosek")
# 返回列含 PEFFCH / SEFFCH / TECHCH / MQ
```

示例 7：DEA.mqfgnz — 时序技术与窗口技术

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
```

```
# 时序技术(累积前沿 year<=t)
res_seq = DEA.mqfgnz(
    data, id=dmuname, year='year',
    sent="K L E=Y", orient="oo",
    tech=SEQUENTIAL,
).optimize(email=OPT_LOCAL, solver="mosek")

# 窗口技术(带宽 3,参考集 [t-1, t, t+1])
res_win = DEA.mqfgnz(
    data, id=dmuname, year='year',
    sent="K L E=Y", orient="oo",
    tech=WINDOW, window=3,
).optimize(email=OPT_LOCAL, solver="mosek")
```

示例 8：**DEA.mqrd — Ray–Desli 分解(当期技术)**

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
res_rd = DEA.mqrd(
    data, id=dmuname, year='year',
    sent="K L E=Y", orient="oo",
    tech=CONTEMPORARY,
).optimize(email=OPT_LOCAL, solver="mosek")
# 返回列含 EFFCH / TECHCH_RD / SCALECH_RD / MQ
```

示例 9：**DEA.gmqdecom — 全局前沿分解**

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
res_global = DEA.gmqdecom(
    data, id=dmuname, year='year',
    sent="K L E=Y", orient="oo",
    tech=GLOBAL,
).optimize(email=OPT_LOCAL, solver="mosek")
# 返回列含 PEFFCH / SEFFCH / TECHCH(BPG 变化) / MQ / mqpi
```

示例 10：**DEA.mqfgnz.ddf / DEA.mqrd.ddf — 基于 DDF 的三要素分解**

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
res_fgnz_ddf = DEA.mqfgnz.ddf(
    data, id=dmuname, year='year',
    sent="K L E=Y", gy=[1], gx=[0, 0, 0],   # 产出方向 g=(0,y),仅支持单侧方向
```

```
    tech=CONTEMPORARY,                          # 默认即当期技术
).optimize(email=OPT_LOCAL, solver="mosek")
# 返回列含 PEFFCH / SEFFCH / TECHCH / MQ,与示例 6（mqfgnz）数值一致

res_rd_ddf = DEA.mqrd.ddf(
    data, id=dmuname, year='year',
    sent="K L E=Y", gy=[1], gx=[0, 0, 0],
    tech=CONTEMPORARY,
).optimize(email=OPT_LOCAL, solver="mosek")
# 返回列含 EFFCH / TECHCH_RD / SCALECH_RD / MQ,与示例 8（mqrd）数值一致
```

示例 11：DEA.gmqdecom.ddf — 基于 DDF 的全局前沿分解

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
res_global_ddf = DEA.gmqdecom.ddf(
    data, id=dmuname, year='year',
    sent="K L E=Y", gy=[1], gx=[0, 0, 0],   # 产出方向 g=(0,y),仅支持单侧方向
    tech=GLOBAL,                              # 默认即全局技术
).optimize(email=OPT_LOCAL, solver="mosek")
# 返回列含 PEFFCH / SEFFCH / TECHCH(BPG 变化) / MQ / mqpi,与示例 9 数值一致
```

上述三要素分解接口与 4.2.4 节的 DEA.mq 示例共用同一份 data、同一组 sent/orient/tech/solver 参数，区别仅在分解方式：示例 1–7 得到两要素分解 MQ/MEFFCH/MTECHCH，而本节的 mqfgnz/mqrd/gmqdecom 及其 DDF 变体得到三要素分解，二者可通过 4.2.7 的等价关系相互印证。

4.2.7 与 DEA.mq 的等价关系

由于 mqfgnz/mqrd/gmqdecom 内部使用的径向距离与 DEA.mq 完全相同，二者在重叠范围内满足以下核对关系，可作为代码正确性的自检：

- mqfgnz 的 MEFFCH（= PEFFCH×SEFFCH，即 CRS 综合效率变化 $D_o^{t,c}/D_o^{s,c}$）等于以 rts=RTS_CRS 运行 DEA.mq 的 MEFFCH；
- mqfgnz 的 MQ（CRS 基准三要素乘积）等于以 rts=RTS_CRS 运行 DEA.mq 的 MQ——这正是 4.3 节案例所报告的 CRS 基准 Malmquist 指数；
- mqrd 与 mqfgnz 保留同一个 CRS 总指数；mqrd 使用 VRS 跨期距离计算技术变化，并以 SCALECH_RD 单列规模变化。三项有效时乘积等于 MQ。
- gmqdecom（tech=GLOBAL）与 DEA.mq（tech=GLOBAL）的 Pastor–Lovell 分解完全对应：其 MEFFCH（= PEFFCH×SEFFCH，当期 CRS 综合效率变化）等于 DEA.mq 的 MEFFCH；其 TECHCH（最佳实践缺口变化）等于 DEA.mq 的 MTECHCH；其 MQ 与 mqpi 等于 DEA.mq 的 mqpi（CRS）。

4.3 案例应用:中国火电行业的发电生产率

本例使用power plant data.xlsx中2015—2017年30个省级地区的90个观测。投入为装机容量K、相关行业劳动L、标准煤口径燃料F,产出为发电量Y。数据来源沿用第9章的统计年鉴整理口径。这是能源生产行业的生产率测度;本章不把CO2放入传统Malmquist模型,污染约束的作用留给环境生产率章节。

```
from deabook.example.book_cases.run import FUN
FUN[4]()  # 输出完整CSV、摘要表与PNG/PDF图
```

表 4.11: 中国火电行业的发电生产率:结果摘要

期末年份	指标	有效数	几何均值	中位数	P10	P90
2016	生产率指数	30	0.992	0.998	0.944	1.031
2016	效率变化	30	0.975	0.985	0.928	1.014
2016	技术变化	30	1.017	1.011	0.999	1.036
2017	生产率指数	30	1.000	1.002	0.956	1.037
2017	效率变化	30	1.028	1.021	0.987	1.069
2017	技术变化	30	0.973	0.976	0.944	1.000

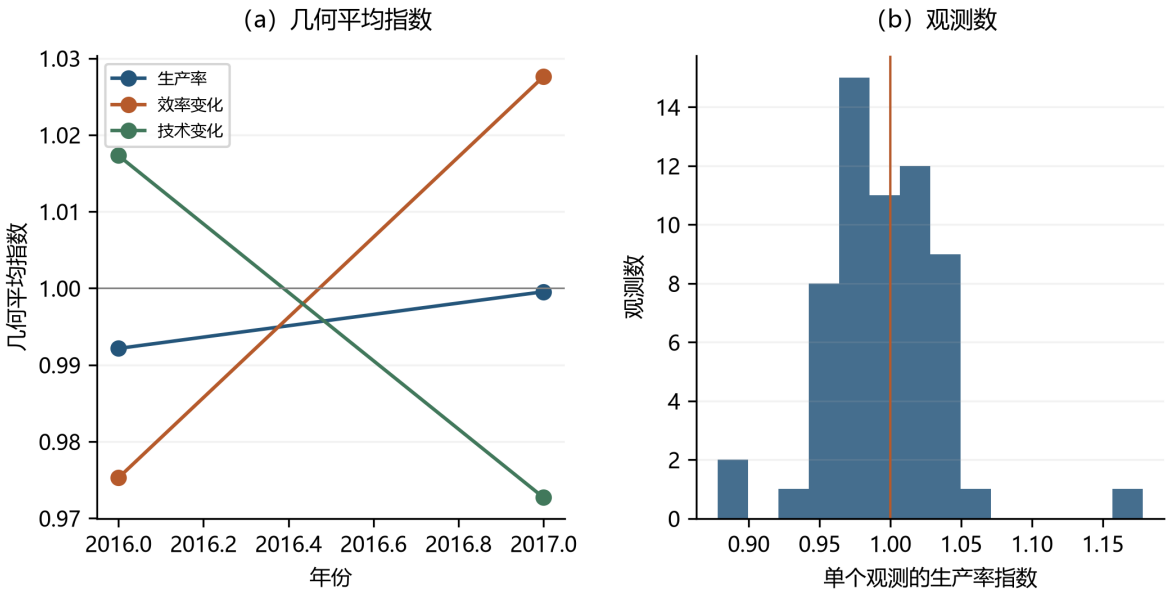


图 4.3: 中国火电行业发电生产率的年度分解与观测分布

表按相邻时期的期末年份展开生产率、效率变化和技术变化,分别报告有效样本数与分布。几何均值用于概括乘法指数;中位数和P10、P90用于观察个体差异。指数大于1表示改善,小于1表示下降。

采用当期CRS产出距离，60个相邻期组合均可计算。2016年发电生产率几何平均指数为0.992,效率变化0.975而技术变化1.017；2017年生产率约为1.000,效率追赶与技术变化方向相反。各指数在相同有效

样本上取几何平均，保持MQ=EC×TC。第4章和第9章虽复用同一火电数据，但分别研究跨期发电生产率与物质守恒约束，不重复列出省份排名。

完整结果保存在配套程序的 `results/ch04_full.csv`。正文只保留模型或年度摘要，省份、电厂明细不占用书内表格行数。

4.4 本章小结

火电发电生产率的变化需要区分效率追赶和技术前沿移动。完整面板用于跨期匹配，摘要表与趋势图承担不同展示任务。

参考文献

ANDERSEN P, PETERSEN N C. A procedure for ranking efficient units in data envelopment analysis [J]. Management Science, 1993, 39(10): 1261-1264. DOI: 10.1287/mnsc.39.10.1261.

APARICIO J, GARCIA-NOVÉ E M, KAPELKO M, PASTOR J T. Graph productivity change measure using the least distance to the Pareto-efficient frontier in data envelopment analysis [J]. Omega, 2017, 72: 1-14. DOI: 10.1016/j.omega.2016.10.005.

APARICIO J, PASTOR J T, VIDAL F. The weighted additive distance function [J]. European Journal of Operational Research, 2016, 254(1): 338-346. DOI: 10.1016/j.ejor.2016.04.006.

CAVES D W, CHRISTENSEN L R, DIEWERT W E. The economic theory of index numbers and the measurement of input, output, and productivity [J]. Econometrica, 1982, 50(6): 1393-1414. DOI: 10.2307/1913388.

CHAMBERS R G, CHUNG Y, FÄRE R. Profit, directional distance functions, and Nerlovian efficiency [J]. Journal of Optimization Theory and Applications, 1998, 98(2): 351-364. DOI: 10.1023/A:1022637501082.

CHAMBERS R G, FÄRE R, GROSSKOPF S. Productivity growth in APEC countries [J]. Pacific Economic Review, 1996, 1(3): 181-190. DOI: 10.1111/j.1468-0106.1996.tb00184.x.

CHAMBERS R G. Exact nonradial input, output, and productivity measurement [J]. Economic Theory, 2002, 20(4): 751-765. DOI: 10.1007/s001990100231.

CHUNG Y H, FÄRE R, GROSSKOPF S. Productivity and undesirable outputs: A directional distance function approach [J]. Journal of Environmental Management, 1997, 51(3): 229-240. DOI: 10.1006/jema.1997.0146.

FEENSTRA R C, INKLAAR R, TIMMER M P. The next generation of the Penn World Table [J]. American Economic Review, 2015, 105(10): 3150-3182. DOI: 10.1257/aer.20130954.

FÄRE R, GROSSKOPF S, LINDGREN B, ROOS P. Productivity changes in Swedish pharmacies 1980–1989: A non-parametric Malmquist approach [J]. Journal of Productivity Analysis, 1992, 3(1): 85-101. DOI: 10.1007/BF00158770.

FÄRE R, GROSSKOPF S, NORRIS M, ZHANG Z. Productivity growth, technical progress, and efficiency change in industrialized countries [J]. American Economic Review, 1994, 84(1): 66-83.

KEVORK I S, PANGE J, TZEREMES P, TZEREMES N G. Estimating Malmquist productivity indexes using probabilistic directional distances: An application to the European banking sector [J]. European Journal of Operational Research, 2017, 261(3): 1125-1140. DOI: 10.1016/j.ejor.2017.03.012.

MALMQUIST S. Index numbers and indifference surfaces [J]. Trabajos de Estadística, 1953, 4(2): 209-242. DOI: 10.1007/BF03006863.

PASTOR J T, LOVELL C A K, APARICIO J. Defining a new graph inefficiency measure for the proportional directional distance function and introducing a new Malmquist productivity index [J]. European Journal of Operational Research, 2020, 281(1): 222-230. DOI: 10.1016/j.ejor.2019.08.021.

PASTOR J T, LOVELL C A K. A global Malmquist productivity index [J]. Economics Letters, 2005, 88(2): 266-271. DOI: 10.1016/j.econlet.2005.02.013.

RAY S, DESLI E. Productivity growth, technical progress, and efficiency change in industrialized countries: Comment [J]. American Economic Review, 1997, 87(5): 1033-1039.

SIMAR L, WILSON P W. Another look at productivity growth in industrialized countries [J]. Journal of Productivity Analysis, 2023, 60(3): 257-272. DOI: 10.1007/s11123-023-00689-w.

第 5 章

能源效率与能源生产率测度方法

在非参数（DEA）框架下，能源效率测算的经典起点是 Hu and Wang (2006) 提出的全要素能源效率（total-factor energy efficiency, TFEE），即目标能源投入与实际能源投入之比。能源效率的数值内涵取决于其口径设定：同一决策单元在不同技术集、调整方向及规模报酬假设下所得效率水平并不具备直接可比性。能源效率与能源生产率测度的核心难点在于：如何在多投入、多产出的生产结构中，分离能源投入节约、其他要素配置与技术前沿移动三者之间的关系。

本章沿一条递进的线索构造能源效率测度。**第一步**从最一般的谢泼德投入距离函数出发：所有投入（能源与非能源）按同一比例径向缩减。该口径简单、与经典DEA 直接对接，但存在**短板效应**——径向缩减比例由节约潜力最小的投入决定，从而系统性低估能源专项节约潜力（Lin and Du, 2013）。**第二步**因此把能源从一般投入中剥离出来，固定其他投入与产出、只沿能源方向径向缩减，得到谢泼德能源距离函数（Zhou, Ang and Zhou, 2012）；Lin and Du (2013) 正是在这一口径下评估了中国各省份的能源效率。**第三步**再引入方向距离函数（DDF），把” 调整什么、调整多少” 完全交给研究者设定：既可以只沿能源一个方向调整，也可以让能源缩减与产出扩张同时发生，从而刻画” 节能” 与” 增产” 的联合改进空间。设非能源投入向量为 $x=(x_1,x_2,\cdots,x_K)^{\mathrm{T}}\in\mathfrak{R}_+^K$，能源投入向量为 $e=(e_1,e_2,\cdots,e_M)^{\mathrm{T}}\in\mathfrak{R}_+^M$，产出向量为 $y=(y_1,y_2,\cdots,y_L)^{\mathrm{T}}\in\mathfrak{R}_+^L$。下文先按上述三步给出静态效率模型，再在同一技术设定下构造跨期能源生产率指数。静态能源效率刻画某一时期观测点相对于当期前沿的位置，但无法区分效率变化来自决策单元自身追赶还是技术前沿移动；能源生产率指数正是为分离这两种来源而构造，本章 5.4 节将予以展开。

5.1 投入距离函数

5.1.1 谢泼德投入距离函数模型

投入距离函数是能源效率测算的自然起点。谢泼德投入距离函数（Shephard, 1970；Färe, Grosskopf and Lovell, 1985）采用径向收缩方式，将能源投入和非能源投入按同一比例压缩，适用于在产出水平保持不变的前提下考察全部投入共同节约空间的情形。

对被评价决策单元 o，在给定产出 y^o 不下降的条件下，模型通过最小化比例因子 θ 寻找可行的投入组合 $(\theta x^o,\theta e^o)$。若 $\theta=1$，说明该决策单元位于投入节约前沿；若 $\theta<1$，则存在按相同比例削减能源和非能

源投入的空间。规模报酬不变（CRS）情形的线性规划为：

$$\begin{aligned}
&\min_{\theta,\lambda^1,\ldots,\lambda^N} \theta \\
&\text{s.t.} \sum_{j=1}^{N} \lambda^j x_k^j \le \theta x_k^o, \quad k = 1, \ldots, K \\
&\sum_{j=1}^{N} \lambda^j e_m^j \le \theta e_m^o, \quad m = 1, \ldots, M \\
&\sum_{j=1}^{N} \lambda^j y_l^j \ge y_l^o, \quad l = 1, \ldots, L \\
&\lambda^j \ge 0
\end{aligned} \tag{5.1}$$

规模报酬可变（VRS）情形在式（5.1）的基础上额外加入权重和为 1 的凸性约束 $\sum_{j=1}^{N} \lambda^j = 1$，其规划为：

$$\begin{aligned}
&\min_{\theta,\lambda^1,\ldots,\lambda^N} \theta \\
&\text{s.t.} \sum_{j=1}^{N} \lambda^j x_k^j \le \theta x_k^o, \quad k = 1, \ldots, K \\
&\sum_{j=1}^{N} \lambda^j e_m^j \le \theta e_m^o, \quad m = 1, \ldots, M \\
&\sum_{j=1}^{N} \lambda^j y_l^j \ge y_l^o, \quad l = 1, \ldots, L \\
&\sum_{j=1}^{N} \lambda^j = 1 \\
&\lambda^j \ge 0
\end{aligned} \tag{5.2}$$

式（5.1）与式（5.2）给出谢泼德投入距离函数的径向规划，按规模报酬假设分为CRS 与 VRS 两种情形。下文各 DEA 规划的规模报酬约束均按这两种方式处理，不再逐一说明。该设定下能源效率（EE）定义为：

$$EE = \theta$$

值得指出的是，方向距离函数取 $g = (x, e, 0)$（全部投入沿观测方向同步缩减、产出不变）时，投入约束化为 $(1-\beta)x$ 与 $(1-\beta)e$，与式（5.1）仅差变量替换 $\theta = 1-\beta$，两种表述数值等价，因而同样继承下述短板效应。

5.1.2 短板效应

径向口径要求所有投入**等比例**缩减，这一要求隐含着一个关键问题：可行的缩减比例由**节约潜力最小的投入**决定，正如木桶的容量由最短的木板决定。Lin and Du (2013) 将其称为" 短板效应"（bucket effect），并给出了一个直观的数值例子。

设想一个经济体的能源、资本、劳动在保持产出不变时分别可以缩减 30%、20% 与 10%。径向口径要求三者按同一比例 θ 压缩，可行的 θ 由最不易缩减的投入——此处为劳动——决定，θ 至多降至0.9。于是式（5.1）测得的" 能源效率" 为0.9，相应的结论是" 该经济体可节约 10% 的能源投入"；而能源本身真实的节

约潜力是30%。换言之，径向口径**系统性低估**能源专项节约潜力，且低估幅度取决于其他投入的冗余程度，与能源利用状况本身无关。Hu and Wang (2006) 的 TFEE 指标虽以" 目标能源投入/实际能源投入" 定义，但其DEA 实现要求能源与其他投入同比例缩减（能源的目标值随全投入径向投影一起得到），因而同样受短板效应影响。

短板效应的根源在于：能源节约与其他投入配置被捆绑在同一个比例因子上。要分离出能源本身的节约潜力，就必须允许能源**单独**缩减、其他投入保持不变——这正是下一节谢泼德能源距离函数的出发点。

5.2 谢泼德能源距离函数模型

为克服短板效应，应把能源投入从一般投入中单独剥离：固定非能源投入 x 和产出 y，仅沿能源投入 e 的方向径向缩减。Zhou, Ang and Zhou (2012) 定义了这一能源专项的谢泼德能源距离函数；Lin and Du (2013) 认为由此定义的能源效率" 更合适"，因为它直接度量能源本身的节约潜力，而不受其他投入冗余程度的干扰。该设定将能源利用效率与一般投入效率区分开来，适用于解释单位产出能源消耗、能源强度或能源约束政策下的技术改进空间。

式（5.3）通过最小化能源缩放比例 θ 来构造能源专项效率。非能源投入约束保持在观测水平 x_k^o，产出约束要求不低于观测产出 y_l^o，只有能源投入允许按 θ 比例缩减：

$$
\begin{aligned}
&\min_{\theta,\lambda^1,\ldots,\lambda^N} \theta \\
&\text{s.t.} \sum_{j=1}^{N} \lambda^j x_k^j \le x_k^o, \quad k = 1, \ldots, K \\
&\sum_{j=1}^{N} \lambda^j e_m^j \le \theta e_m^o, \quad m = 1, \ldots, M \\
&\sum_{j=1}^{N} \lambda^j y_l^j \ge y_l^o, \quad l = 1, \ldots, L \\
&\lambda^j \ge 0
\end{aligned}
\tag{5.3}
$$

规模报酬约束：CRS 下不加入凸性约束，VRS 下加入 $\sum_{j=1}^{N} \lambda^j = 1$（与式（5.1）及式（5.2）一致，下文各 DEA 规划均按此处理，不再逐一说明）。该设定下能源效率（EE）定义为：

$$EE = \theta$$

能源专项口径与全投入口径（5.1）的区别在于约束条件的非对称性：非能源投入不参与径向收缩，而能源投入单独收缩。两者的效率值存在明确的大小关系：式（5.3）的非能源约束 $\sum_j \lambda^j x_k^j \le x_k^o$ 比式（5.1）的 $\sum_j \lambda^j x_k^j \le \theta x_k^o$（$\theta \le 1$）更松，可行域更大，最小值因而不会更大，即 $\theta_{\text{能源}} \le \theta_{\text{全投入}}$。回到 5.1.2 节的数值例子：全投入口径测得 $\theta = 0.9$（节能10%），能源专项口径测得 $\theta = 0.7$（节能30%）——能源专项的效率得分**更低**，但揭示的节能空间**更大**，且后者才是能源本身的节约潜力。因此，$EE = \theta$ 反映的是在既定资本、劳动和产出条件下的能源节约潜力；若将其与全投入效率混合解释，会夸大能源专项技术进步的含义。从距离函数的一般形式看，式（5.3）正是第 3 章 3.2.2 节（投入距离函数）所涵盖的单投入谢泼德距离函数（3.6）——只是被单独径向缩减的投入由资本换成了能源。

5.3 方向距离函数模型

径向测度将调整比例限定于能源（或全部投入）单一维度。方向距离函数（Chambers, Chung and Färe, 1996）通过预设方向向量 $g=(g_x,g_e,g_y)$ 描述效率改进路径，允许研究者明确指定哪些变量缩减、哪些变量扩张以及各变量的调整幅度。在能源问题中，方向距离函数提供两类常用选择：**只调整能源一个方向**，或**让能源缩减与产出扩张同时发生**。两类选择的非能源投入方向分量均取 $g_x=0$，即资本与劳动保持在观测水平，从而延续 5.2 节克服短板效应的思路。

5.3.1 只调整能源一个方向

方向向量取 $g=(0,e,0)$：非能源投入方向分量 $g_x=0$ 固定资本与劳动的调整空间，能源方向分量 $g_e=e$ 沿能源投入原始规模设定压缩路径，产出方向分量 $g_y=0$ 保持产出不变。这一非对称设计使方向距离函数在度量口径上与 5.2 节的谢泼德能源距离函数一致——都衡量纯粹能源要素的节约潜力——但以加法形式刻画调整幅度，便于在不同投入维度上设定不同的方向权重。

固定非能源投入可隔离资本、劳动力等要素变动对能效评估的干扰，适用于评估既有生产设备条件下的能源节约空间；能源压缩方向 $g_e=e$ 与观测值同构，使模型度量的是”在当前能源投入规模下的最大可压缩比例”，与单位产出能耗、能源强度等指标的政策含义相对应。对应的DEA 规划通过最大化能源可压缩比例 β 构造生产前沿：

$$
\begin{aligned}
&\vec{D}\left(x^o,e^o,y^o;g_x^o,g_e^o,g_y^o\right)=\max_{\lambda^j,\beta}\beta\\
\text{s.t.}\quad&\textstyle\sum_{j=1}^{N}\lambda^j y_l^j\geqslant y_l^o,l=1,\cdots,L\\
&\textstyle\sum_{j=1}^{N}\lambda^j x_k^j\leqslant x_k^o,k=1,\cdots,K\\
&\textstyle\sum_{j=1}^{N}\lambda^j e_m^j\leqslant e_m^o-\beta g_{me}^o,m=1,\cdots,M\\
&\lambda^j\geqslant 0,j=1,\cdots,N
\end{aligned}
\tag{5.4}
$$

规模报酬约束：CRS 下不加入凸性约束，VRS 下加入 $\sum_{j=1}^{N}\lambda^j=1$。此时，能源效率（EE）定义为：

$$EE=1-\beta$$

因此，$1-\beta$ 越接近 1，说明能源投入越接近前沿水平；β 越大，则表示在保持非能源投入和产出不变时可节约的能源投入比例越高。该指标是本章后续案例所采用的主要静态能源效率口径。由于 $g_e=e$ 时能源约束化为 $(1-\beta)e$，式（5.4）与式（5.3）仅差变量替换 $\theta=1-\beta$，两种表述数值等价。

5.3.2 能源缩减与产出扩张同时进行

若研究问题不只关心”给定产出能节约多少能源”，还关心能源节约与产出提升能否兼得，可将产出方向分量打开：方向向量取 $g=(0,e,y)$，能源沿 $g_e=e$ 缩减、产出沿 $g_y=y$ 扩张，资本与劳动仍固定在观测水平。该设定刻画的是”节能”与”增产”的联合改进空间：β 越大，说明在不增加其他投入的前提下，能源压缩与产出扩张的同时改进幅度越大，与能源强度类指标（单位产出的能源消耗）的政策含义直接对应。

对应 DEA 规划通过最大化联合改进幅度 β 构造生产前沿：

$$\begin{aligned} & \vec{D}\left(x^o, e^o, y^o; g_x^o, g_e^o, g_y^o\right) = \max_{\lambda^j, \beta} \beta \\ \text{s.t.} \quad & \sum_{j=1}^N \lambda^j y_l^j \geqslant y_l^o + \beta g_{ly}^o, l = 1, \cdots, L \\ & \sum_{j=1}^N \lambda^j x_k^j \leqslant x_k^o, k = 1, \cdots, K \\ & \sum_{j=1}^N \lambda^j e_m^j \leqslant e_m^o - \beta g_{me}^o, m = 1, \cdots, M \\ & \lambda^j \geqslant 0, j = 1, \cdots, N \end{aligned} \tag{5.5}$$

规模报酬约束：CRS 下不加入凸性约束，VRS 下加入 $\sum_{j=1}^N \lambda^j = 1$。此时 β^* 同时包含能源端的压缩与产出端的扩张，若要把它折算成落在 $[0,1]$ 的绩效指数，应与第 3 章的口径一致，采用

$$EEI = \frac{1-\beta^*}{1+\beta^*}$$

或分别报告能源端的 $1-\beta^*$ 与产出端的 $1/(1+\beta^*)$。EEI 越接近1，说明联合改进空间越小、决策单元越接近方向前沿。需要提醒的是：$1-\beta^*$ 只刻画能源端的收缩比例，既不能单独当作联合改进口径下的能源效率，也不能与式(5.3)的 $EE = \theta$ 或式(5.4)的 $EE = 1-\beta$ 直接比较；本节的 $g = (0, e, y)$ 与 5.3.1 节纯能源方向 $g = (0, e, 0)$ 也不是同一指标。由于 β 同时反映投入和产出调整，结果解释时应说明方向向量的选择，否则不同研究之间的效率水平不具备直接可比性。

5.4 能源生产率

为分离决策单元自身追赶与技术前沿移动这两种来源，本节构造跨期能源生产率指数。Malmquist 指数(Caves, Christensen and Diewert, 1982；Färe, Grosskopf, Norris and Zhang, 1994)利用跨期距离函数比较不同时期的生产计划和技术前沿，并将生产率变化分解为效率变化(MEFFCH)和技术变化(MTECHCH)，其分解形式采用Färe, Grosskopf, Norris and Zhang (1994) 的 DEA 实现。其中，$MQ > 1$ 表示相邻期能源生产率提高，$MQ < 1$ 表示能源生产率下降；MEFFCH 反映决策单元相对于前沿的追赶或退步，MTECHCH 反映样本技术前沿的移动。

沿用前文符号，非能源投入、能源投入和产出分别记为 $x = (x_1, x_2, \cdots, x_K)^{\mathrm{T}} \in \mathfrak{R}_+^K$、$e = (e_1, e_2, \cdots, e_M)^{\mathrm{T}} \in \mathfrak{R}_+^M$ 和 $y = (y_1, y_2, \cdots, y_L)^{\mathrm{T}} \in \mathfrak{R}_+^L$。令 s 表示后一期、t 表示前一期，$\theta_o^\tau(q)$ 表示决策单元 o 在 q 期的生产计划相对于 τ 期技术前沿计算得到的效率型距离指标：谢泼德口径下为 Farrell 型效率分数，方向距离口径下为 $1-\vec{D}$ 型方向效率；数值越接近1，表示越接近参考前沿。由于本章程序的返回值均为效率分数而非距离函数本身，后文动态公式统一使用 θ 表示效率型距离指标。

Malmquist 指数的定义与口径无关。指数定义为两个相邻时期指数的几何平均：

$$M(s,t) = \sqrt{M^s M^t} = \sqrt{\frac{\theta_o^t(s)}{\theta_o^t(t)} \frac{\theta_o^s(s)}{\theta_o^s(t)}} \tag{5.6}$$

其中，$M(s,t) > 1$ 表示从前一期 t 到后一期 s 的能源生产率提高。该指数可分解为技术变化和效率变化(Färe, Grosskopf, Norris and Zhang, 1994)：

$$\begin{aligned} M(s,t) &= \sqrt{\frac{\theta_o^t(s)}{\theta_o^s(s)} \cdot \frac{\theta_o^t(t)}{\theta_o^s(t)} \cdot \frac{\theta_o^s(s)}{\theta_o^t(t)}} \\ &= MTECHCH(s,t) \cdot MEFFCH(s,t) \end{aligned} \tag{5.7}$$

动态指标的口径必须与静态效率指标一致：全投入口径（5.1 节）下的指数衡量全投入径向意义上的生产率变化，并继承短板效应；能源专项口径（5.2、5.3 节）下的指数衡量能源专项（或能源—产出联合）的生产率变化。只有在同一技术集、同一方向向量和同一规模报酬假设下，跨期指数及其分解才具有明确经济含义。

已有研究将该思路用于城市、国家和资源型经济体的能源绩效评价。例如，Zheng (2021) 基于 2012—2018 年 23 个城市面板数据测算全要素能源效率，并分解效率变化和技术变化；Shah, Hao, Yan, Yasmeen and Lu (2023) 考察 2001—2019 年南亚国家能源效率变化；Tachega, Yao, Liu, Ahmed, Li and Mintah (2021) 评估 2010—2017 年非洲产油国能源效率及其决定因素。

5.4.1 基于能源专项谢泼德距离函数的能源生产率

能源专项口径与 5.2 节的谢泼德能源距离函数对应：固定非能源投入和产出条件，仅允许能源投入缩减，考察能源投入相对于不同时期技术前沿的可压缩程度，因而更适合解释能源专项生产率变化。

Malmquist 指数的定义与分解同式（5.6）、（5.7）结构相同，区别仅在于跨期效率分数由能源专项距离函数给出：

$$M(s,t)=\sqrt{M^{s}M^{t}}=\sqrt{\frac{\theta_o^t(s)}{\theta_o^t(t)}\frac{\theta_o^s(s)}{\theta_o^s(t)}}=MTECHCH(s,t)\cdot MEFFCH(s,t) \tag{5.8}$$

同期效率计算模型： s 期技术下的同期效率分数由下式求解：

$$\begin{aligned}
&\theta_o^s(s)=\min_{\theta,\lambda^1,\ldots,\lambda^N}\theta\\
&\text{s.t.}\ \sum_{j=1}^{N}\lambda^j x_k^{js}\le x_k^{os},\quad k=1,\ldots,K\\
&\sum_{j=1}^{N}\lambda^j e_m^{js}\le \theta e_m^{os},\quad m=1,\ldots,M\\
&\sum_{j=1}^{N}\lambda^j y_l^{js}\ge y_l^{os},\quad l=1,\ldots,L\\
&\lambda^j\ge 0
\end{aligned} \tag{5.9}$$

跨期效率计算模型： t 期技术下评价 s 期观测值时，跨期效率分数由下式求解：

$$\begin{aligned}
&\theta_o^t(s)=\min_{\theta,\lambda^1,\ldots,\lambda^N}\theta\\
&\text{s.t.}\ \sum_{j=1}^{N}\lambda^j x_k^{jt}\le x_k^{os},\quad k=1,\ldots,K\\
&\sum_{j=1}^{N}\lambda^j e_m^{jt}\le \theta e_m^{os},\quad m=1,\ldots,M\\
&\sum_{j=1}^{N}\lambda^j y_l^{jt}\ge y_l^{os},\quad l=1,\ldots,L\\
&\lambda^j\ge 0
\end{aligned} \tag{5.10}$$

规模报酬约束：CRS 下不加入凸性约束，VRS 下加入 $\sum_{j=1}^{N}\lambda^j=1$。

5.4.2 基于能源专项方向距离函数的能源生产率

方向距离口径与 5.3 节对应。取方向向量 $g=(0,e,0)$（只调整能源方向）时，将式（5.6）、（5.7）中的效率分数 $\theta_o^\tau(q)$ 替换为方向效率 $1-\vec{D}_o^\tau(q)$，即得能源专项方向距离口径的 Malmquist 指数及其分解：

$$M(s,t)=\sqrt{\frac{1-\vec{D}_o^t(s)}{1-\vec{D}_o^t(t)}\frac{1-\vec{D}_o^s(s)}{1-\vec{D}_o^s(t)}}=MTECHCH(s,t)\cdot MEFFCH(s,t) \tag{5.11}$$

式（5.11）与式（5.6）具有相同的效率型比值方向，程序先将方向距离函数值转换为投入效率，再用跨期效率比率构造 `MQ`、`MEFFCH` 和 `MTECHCH`；后文案例中 `MQ>1` 仍解释为相邻期能源生产率提高。

在对应的方向距离模型中，s 期技术评价 s 期观测值的同期距离为：

$$\begin{aligned}&\vec{D}_o^s(s)=\max_{\lambda^j,\beta}\beta\\ \text{s.t.}\quad&\textstyle\sum_{j=1}^N\lambda^j y_l^{js}\geqslant y_l^{os},l=1,\cdots,L\\&\textstyle\sum_{j=1}^N\lambda^j x_k^{js}\leqslant x_k^{os},k=1,\cdots,K\\&\textstyle\sum_{j=1}^N\lambda^j e_m^{js}\leqslant e_m^{os}-\beta g_{me}^{os},m=1,\cdots,M\\&\lambda^j\geqslant 0,j=1,\cdots,N\end{aligned} \tag{5.12}$$

t 期技术评价 s 期观测值的跨期距离为：

$$\begin{aligned}&\vec{D}_o^t(s)=\max_{\lambda^j,\beta}\beta\\ \text{s.t.}\quad&\textstyle\sum_{j=1}^N\lambda^j y_l^{jt}\geqslant y_l^{os},l=1,\cdots,L\\&\textstyle\sum_{j=1}^N\lambda^j x_k^{jt}\leqslant x_k^{os},k=1,\cdots,K\\&\textstyle\sum_{j=1}^N\lambda^j e_m^{jt}\leqslant e_m^{os}-\beta g_{me}^{os},m=1,\cdots,M\\&\lambda^j\geqslant 0,j=1,\cdots,N\end{aligned} \tag{5.13}$$

规模报酬约束：CRS 下不加入凸性约束，VRS 下加入 $\sum_{j=1}^N\lambda^j=1$。若研究问题关注能源与产出的联合动态改进，方向向量亦可取 $g=(0,e,y)$（见 5.3.2 节），指数的构造方式完全相同，只需将方向效率相应替换；解释时应说明该口径同时包含能源端与产出端的调整。

跨期效率/方向距离函数（式（5.10）、（5.13））将一期观测值置于另一期技术前沿下评价，线性规划在某些决策单元—年份组合上可能无可行解，此时跨期距离值为缺失。这是DEA 跨期分析的固有现象，并不影响同期效率的可行性；后续案例中相应年份的 `MQ`、`MEFFCH` 和 `MTECHCH` 将按可得样本计算并予以说明。

式（5.6）—（5.13）给出了从静态能源效率到跨期能源生产率的完整动态框架。无论采用谢泼德距离函数还是方向距离函数，也无论采用全投入缩减还是能源专项口径，Malmquist 指数及其分解项的定义形式不变，差异仅体现在底层距离函数的取法上。这一统一结构是下一节 `deabook` 程序接口设计的依据。

5.5 能源效率与能源生产率模型的Python实现

前几节将能源效率与能源生产率的测算问题依次表述为若干线性规划：5.1 至 5.3 节给出静态效率的谢泼德与方向距离规划（式（5.1）、式（5.3）、式（5.4）与式（5.5）），5.4 节则在此基础上以跨期距离函数之比构造 Malmquist 指数（式（5.6）、式（5.7）、式（5.8）与式（5.11））。`deabook` 将这些规划封装为统一的求解接口，使同一套模型设定既能驱动静态效率求解，也能驱动跨期生产率分解。具体而言，两类静态能源效率分别由

DEA.ite 与 DEA.ddf 实现：谢泼德能源距离函数（5.2 节，只缩减能源、其余投入与产出固定）对应 DEA.ite；方向距离函数（5.3 节，沿指定方向向量调整）对应 DEA.ddf。跨期能源生产率（5.4 节）由专门的 DEA.ep（谢泼德口径，基于 DEA.ite）与 DEA.ep.ddf（方向距离口径，基于 DEA.ddf）实现，二者分别按 5.4 节的两个 Malmquist 公式组装相邻期效率分数，无需独立的动态模型类。下文先说明各接口如何把模型口径映射为代码设定，再给出与本章理论公式一一对应的调用示例。

5.5.1 数据接口约定

本章不涉及非期望产出进入 DEA 约束，二氧化碳变量仅在案例描述性统计中展示，故两类接口对生产结构的声明均沿用标准投入—产出写法。两类接口的分野源于它们对” 被缩减对象” 的不同处理方式：DEA.ite 把 sent 等号左侧的**第一个投入**固定为唯一被径向缩减的投入，因此能源专项口径要求将能源变量置于最前，例如 sent="E K L=Y"（E 为能源，K、L 为资本与劳动，Y 为期望产出）；DEA.ddf 则在 sent 中等号左侧列出全部投入、右侧列出全部产出，改由方向向量 gx/gy 指定调整对象，例如 sent="K L E=Y" 配合 gx=[0,0,1]、gy=[0] 即表示只缩减能源。这种设定上的差异，恰好对应 5.2 节” 固定其余、只缩能源” 与 5.3 节” 按方向向量调整” 两种理论口径。

调用前需导入分析环境与求解常量：

```
from deabook import data_path, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL
from deabook import DEA, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook.constant import RTS_CRS, RTS_VRS, OPT_LOCAL
import pandas as pd
```

上述两类接口共享的核心参数及其在本章案例中的常见取值汇总于表 5.1；该表既用于核对变量名映射，也标明了规模报酬、参考集与求解器等关键开关的所在位置。

表 5.1: 能源效率模型的主要参数

参数	适用接口	本章常用设置	说明
data	ite / ddf	DataFrame	包含K、L、E、Y 及识别变量的数据
sent	ite / ddf	ite: "E K L=Y"； ddf: "K L E=Y"	ite 首投入为被缩减的能源；ddf 全投入列于 = 左侧
gx	ddf	[0, 0, 1]	投入方向向量；第三个元素对应能源 E，1 表示沿能源方向缩减

参数	适用接口	本章常用设置	说明
gy	ddf	[0]	产出方向向量；本章能源专项案例产出保持不变
rts	ite / ddf	RTS_CRS、RTS_VRS	规模报酬不变 / 可变
evalindex	ite / ddf	None 或年份筛选	被评价 DMU 筛选条件，None 评价全部
refindex	ite / ddf	None 或年份筛选	参考技术集筛选条件，None 使用全部观测构造前沿
solver	ite / ddf	由 optimize() 指定	求解器，例如 "mosek"
email	ite / ddf	OPT_LOCAL	本地求解

5.5.2 测度口径与方向向量设定

DEA.ite 与 DEA.ddf 以两种互补的方式确定测度口径，其差别本质上反映了 5.2 节与 5.3 节两种理论表述的分野。DEA.ite 不需要显式方向向量，它始终只缩减 sent 中的第一个投入，其余投入与全部产出自动固定；只要将能源放在 sent 首位，便实现了能源专项谢泼德口径——例如 sent="E K L=Y" 令程序对 E 做径向缩减、K 与 L 保持观测水平、Y 不低于观测水平，对应 5.2 节的式(5.3)。DEA.ddf 则把调整对象完全交给研究者：以 sent="K L E=Y" 为例，gx=[0,0,1]、gy=[0] 表示仅缩减能源E，资本、劳动与产出保持不变，对应理论模型 $g = (0, e, 0)$（5.3.1 节）；若取 gx=[0,0,1]、gy=[1]，则能源缩减与产出扩张同时进行，对应 $g = (0, e, y)$（5.3.2 节）。表 5.2 汇总了 sent、方向向量与理论模型之间的对应关系及其经济含义。

表 5.2: 能源方向向量与经济含义

sent（首投入）	gx	gy	模型方向	经济含义	对应章节
ite: "E K L=Y"	—	—	能源专项谢泼德	只缩减能源 E，K、L、Y 固定	5.2
ddf: "K L E=Y"	[0,0,1]	[0]	只调整能源方向	仅能源 E 缩减，其余固定	5.3.1
ddf: "K L E=Y"	[0,0,1]	[1]	能源—产出混合方向	能源缩减同时产出扩张	5.3.2

sent（首投入）	gx	gy	模型方向	经济含义	对应章节
ddf: "K L E=Y"	[1,1,1]	[0]	全投入导向	资本、劳动、能源同时缩减	5.1

DEA.ddf 的 gx/gy 仅是方向选择向量，用于标记哪些变量参与缩减或扩张；程序在约束中实际代入的是" 对应投入/产出的观测值× 该分量" 作为方向，因此 gx=[0,0,1] 在数值上对应于理论中的 $g = (0, e, 0)$。这意味着方向向量决定的是调整维度而非调整幅度，调整幅度由规划自身求解得到。

5.5.3 返回结果与指标解读

明确 sent 与方向向量之后，还需理解两类接口返回的指标结构，方能将求解结果转换为本章的效率口径。DEA.ite 直接返回效率分数，而 DEA.ddf 返回的是方向距离值，需由研究者自行转换为效率——这一差异源于方向距离函数以加法形式刻画调整幅度、其取值本身不落在 $[0, 1]$ 区间。表 5.3 汇总了两类接口的主要返回列及其指标解释。

表 5.3: 静态能源模型的主要返回列

接口	主要返回列	指标解释
DEA.ite	optimization_status、rho、objective_value	求解状态、原始目标值
DEA.ite	te	能源专项谢泼德效率 θ，数值越接近 1 表示能源效率越高；本章将其直接记为 EE
DEA.ddf	optimization_status、rho、objective_value	求解状态、方向距离函数值 $\vec{D}$
DEA.ddf	（由 rho 转换）	投入方向效率 $EE = 1 - \rho$；产出方向效率 $EE = 1/(1 + \rho)$；混合方向两端分别按上述公式转换

返回列与 5.2、5.3 节公式符号的对应关系如下：DEA.ite 的 te 即式（5.3）的 θ，也是式（5.9）、式（5.10）中的同期/跨期效率分数；DEA.ddf 的 rho 即式（5.4）、式（5.12）、式（5.13）的方向距离 $\beta = \vec{D}$，能源效率按 $EE = 1 - \vec{D}$ 转换。

5.5.4 调用示例与理论公式的对应

以下示例说明如何将本章的能源效率与能源生产率模型逐一映射为 deabook 调用；示例仅演示接口映射，不预先绑定后续案例的表格或图形。静态效率由 DEA.ite/DEA.ddf 单次求解得到，跨期能源生产率则依据 5.4 节 Malmquist 公式，借助跨期 evalindex/refindex 调用组装。谢泼德能源距离函数口径（对应 5.2 节与式（5.3））将能源置于 sent 首位，其余投入与产出固定：

```
model = DEA.ite(data, sent="E K L=Y", rts=RTS_CRS)
res = model.optimize(solver="mosek")
res["te"]   # 能源效率 EE = theta
```

若改用规模报酬可变，只需将 `rts` 替换为 `RTS_VRS`。方向距离函数口径（对应 5.3.1 节与式（5.4））则通过方向向量指定仅沿能源缩减：

```
model = DEA.ddf(data, sent="K L E=Y", gx=[0, 0, 1], gy=[0], rts=RTS_CRS)
res = model.optimize(solver="mosek")
res["EE"] = 1 - res["rho"]   # 能源效率 EE = 1 - Dvec
```

若考察能源—产出联合方向（对应 5.3.2 节），只需将 `gy` 改为 `[1]`，并分别转换能源端与产出端效率：

```
model = DEA.ddf(data, sent="K L E=Y", gx=[0, 0, 1], gy=[1], rts=RTS_CRS)
res = model.optimize(solver="mosek")
res["EE_input"]  = 1 - res["rho"]         # 能源端效率
res["EE_output"] = 1 / (1 + res["rho"])   # 产出端效率
```

跨期能源生产率（对应 5.4 节公式）由专门的 `DEA.ep` 与 `DEA.ep.ddf` 实现，二者分别封装了谢泼德口径（5.4.1 节）与方向距离口径（5.4.2 节）的 Malmquist 指数及其 `MEFFCH/MTECHCH` 分解。其调用方式与 `DEA.mq/DEA.mqddf` 类似，只需额外指定 `id` 与 `year`，并沿用能源专项的 `sent`（谢泼德口径能源置于首位，方向距离口径用 `gx/gy` 指定能源方向）。谢泼德口径（基于 `DEA.ite`，对应式（5.8）—（5.10））：

```
model = DEA.ep(data, id=unit_col, year="year",
               sent="E K L=Y", rts=RTS_CRS)
res = model.optimize()
res[["MQ", "MEFFCH", "MTECHCH"]]   # 能源专项 Malmquist 指数及分解
```

方向距离口径（基于 `DEA.ddf`，对应式（5.11）—（5.13）），只需改为 `DEA.ep.ddf` 并给定能源方向向量：

```
model = DEA.ep.ddf(data, id=unit_col, year="year",
                   sent="K L E=Y", gx=[0, 0, 1], gy=[0], rts=RTS_CRS)
res = model.optimize()
res[["MQ", "MEFFCH", "MTECHCH"]]
```

两个接口返回的 `D11`、`D12`、`D21` 即式（5.6）、式（5.7）中的同期与跨期效率分数（谢泼德口径为 θ，方向距离口径为 $1-\vec{D}$），`MQ`、`MEFFCH`、`MTECHCH` 由同一组Malmquist 公式组装；首期没有上一期可比较，对应年份的 `MQ`、`MEFFCH`、`MTECHCH` 通常为空值，个别决策单元—年份组合的跨期距离可能无可行解（见 5.4 节末说明），此时相应年份的指数缺失。这与理论部分一致，整理结果时通常重点解释可得年份的 `MQ`、`MEFFCH` 和 `MTECHCH`。下一节用本章唯一主样本展示完整应用流程。

5.6 案例应用：中国省级能源效率与生产率

使用2016—2019年30个省级地区的120个观测。投入为能源E、资本K和劳动L，产出为Y。静态部分只压缩能源，固定其他投入与产出，参照集为四年混合CRS前沿；动态部分用各年当期前沿计算能源专项Malmquist指数。CO2不进入本章约束。各列沿用china data.xlsx的统计口径，并按全样本列均值缩放以改善

求解条件；该缩放不包含减均值。

```
from deabook.example.book_cases.run import FUN
FUN[5]()  # 重新计算本章，输出完整CSV、摘要表与PNG/PDF图
```

表 5.4: 中国省级能源效率与生产率：结果摘要

期末年份	指标	有效数	几何均值	中位数	P10	P90
2017	生产率指数	28	1.011	1.009	0.971	1.052
2017	效率变化	28	0.997	0.996	0.958	1.038
2017	技术变化	28	1.014	1.014	1.014	1.014
2018	生产率指数	28	1.003	1.007	0.934	1.054
2018	效率变化	28	0.999	1.003	0.930	1.050
2018	技术变化	28	1.004	1.004	1.004	1.004
2019	生产率指数	28	0.972	0.971	0.940	1.015
2019	效率变化	28	0.977	0.976	0.944	1.020
2019	技术变化	28	0.995	0.995	0.995	0.995

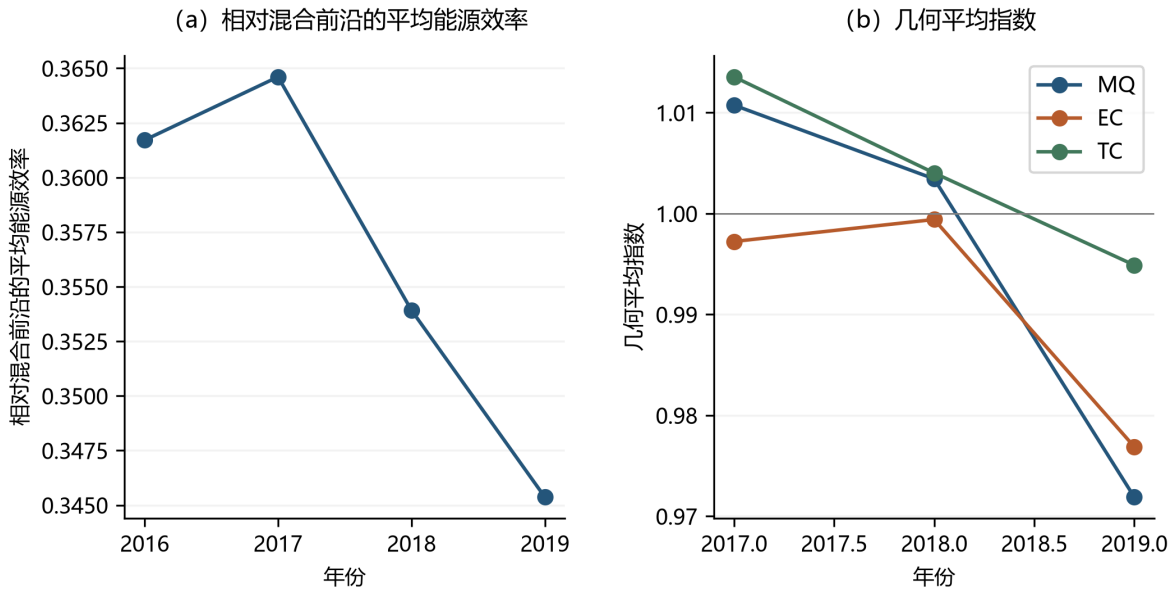


图 5.1: 中国省级能源效率与相邻期能源生产率

表按期末年份分别报告能源生产率及两项分解的有效数、几何均值和分位数。首期没有跨期指数，故不单列指数行。图左补充四个年度相对混合前沿的平均能源效率，图右展示相邻期指数；两幅图的参照技术和指标含义不同。

2016—2019年相对混合前沿的平均能源效率分别约为0.362、0.365、0.354和0.345。2019年能源生产率几何均值降至0.972，效率变化与技术变化均低于1。静态年度均值反映相对固定混合前沿的位置，动态分解反映相邻当期前沿的变化，两者不能混为同一时间趋势。首年没有相邻期指数，表中以“—”表示。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch05_full.csv`。表中P10和P90为第10和第90百分位数（如适用），展示分布而不占用逐个决策单元的行数。

5.7 本章小结

2016—2019年相对混合前沿的平均能源效率分别约为0.362、0.365、0.354和0.345。2019年能源生产率几何均值降至0.972，效率变化与技术变化均低于1。静态年度均值反映相对固定混合前沿的位置，动态分解反映相邻当期前沿的变化，两者不能混为同一时间趋势。首年没有相邻期指数，表中以“—” 表示。

参考文献

APERGIS N, AYE G C, BARROS C P, GUPTA R, WANKE H. Energy efficiency of selected OECD countries: a slacks based model with undesirable outputs [J]. Energy Economics, 2015, 51: 45-53. DOI: 10.1016/j.eneco.2015.05.022.

CAVES D W, CHRISTENSEN L R, DIEWERT W E. The economic theory of index numbers and the measurement of input, output, and productivity [J]. Econometrica, 1982, 50(6): 1393-1414. DOI: 10.2307/1913388.

CHAMBERS R G, CHUNG Y, FÄRE R. Benefit and distance functions [J]. Journal of Economic Theory, 1996, 70(2): 407-419. DOI: 10.1006/jeth.1996.0096.

CHARNES A, COOPER W W, RHODES E. Measuring the efficiency of decision making units [J]. European Journal of Operational Research, 1978, 2(6): 429-444. DOI: 10.1016/0377-2217(78)90138-8.

FEENSTRA R C, INKLAAR R, TIMMER M P. The next generation of the Penn World Table [J]. American Economic Review, 2015, 105(10): 3150-3182. DOI: 10.1257/aer.20130954.

FÄRE R, GROSSKOPF S, LOVELL C A K. The measurement of efficiency of production [M]. Boston: Kluwer-Nijhoff Publishing, 1985. DOI: 10.1007/978-94-015-7681-6.

FÄRE R, GROSSKOPF S, NORRIS M, ZHANG Z. Productivity growth, technical progress, and efficiency change in industrialized countries [J]. American Economic Review, 1994, 84(1): 66-83.

HU J L, WANG S C. Total-factor energy efficiency of regions in China [J]. Energy Policy, 2006, 34(17): 3206-3217. DOI: 10.1016/j.enpol.2005.06.015.

KIANI MAVI N, MAVI R K. Energy and environmental efficiency of OECD countries in the context of the circular economy: common weight analysis for Malmquist productivity index [J]. Journal of Environmental Management, 2019, 247: 651-661. DOI: 10.1016/j.jenvman.2019.05.003.

LIN B, DU K. Technology gap and China’s regional energy efficiency: a parametric metafrontier approach [J]. Energy Economics, 2013, 40: 529-536. DOI: 10.1016/j.eneco.2013.07.015.

SHAH W U H, HAO G, YAN H, YASMEEN R, LU L. Energy efficiency evaluation, changing trends and determinants of energy productivity growth across South Asian countries: SBM-DEA and Malmquist approach [J]. Environmental Science and Pollution Research, 2023, 30(8): 19890-19906. DOI: 10.1007/s11356-022-22642-2.

SHEPHARD R W. Theory of cost and production functions [M]. Princeton: Princeton University Press, 1970.

TACHEGA M A, YAO X, LIU Y, AHMED D, LI H, MINTAH C. Energy efficiency evaluation of oil producing economies in Africa: DEA, Malmquist and multiple regression approaches [J]. Cleaner Environmental Systems, 2021, 2: 100025. DOI: 10.1016/j.cesys.2021.100025.

ZHENG Z. Energy efficiency evaluation model based on DEA-SBM-Malmquist index [J]. Energy Reports, 2021, 7: 397-409. DOI: 10.1016/j.egyr.2021.10.020.

ZHOU P, ANG B W, ZHOU D Q. Measuring economy-wide energy efficiency performance: a parametric frontier approach [J]. Applied Energy, 2012, 90(1): 196-200. DOI: 10.1016/j.apenergy.2011.01.024.

第 6 章

环境绩效测度方法

环境绩效测度的核心问题，是在经济产出与污染排放共同生成的条件下，如何刻画决策单元相对于环境生产前沿的距离。传统DEA 模型通常只处理投入与期望产出，若将污染排放直接作为普通投入或可自由处置的产出，则会忽视污染与正常产出之间的联合生产关系，导致环境效率的解释出现偏误。

本章在概念、测算与指标解释三个层面扩展第 3 章的经典 DEA 框架。关键扩展在于引入非期望产出、弱可处置性和零结合性，并在此基础上比较谢泼德距离函数、双曲距离函数、方向距离函数和非径向方向距离函数。上述模型为碳排放、污染物排放等环境绩效问题提供可复现的前沿测度框架。

全章按照“生产技术设定—距离函数构造—程序调用—案例解释” 的逻辑展开。6.1 节给出包含非期望产出的环境生产技术及其规模报酬设定；6.2 节讨论不同距离函数对应的效率指标和经济含义；6.3 节给出 deabook 中弱可处置性模型的调用方式；6.4 节利用 OECD 国家和中国省份数据展示不同模型下环境效率结果的差异。不同距离函数、方向向量和规模报酬假设会生成不同量纲的指标，实证解释应始终回到模型设定本身。

6.1 生产技术

环境绩效测度首先需要回答“什么样的减排是技术上可行的”。在环境经济学语境下，污染排放和碳排放并不是可以任意丢弃的普通产出，也不宜简单地等同于投入。若假定非期望产出具有强可处置性，就意味着企业能够在投入和期望产出不变的情况下无成本降低排放，这与多数生产过程中的治理成本、技术约束和产出损失并不一致。更合适的设定是弱可处置性：在既定技术条件下，期望产出与非期望产出共同生成，降低污染排放通常需要压缩产出、增加治理投入或改变生产技术。因此，本节先给出包含非期望产出的环境生产技术，再说明规模报酬、减排因子和技术集修正对效率测算的影响。

考虑一个使用 K 种投入要素 $x=(x_1,x_2,\cdots,x_K)^{\mathrm{T}}\in\mathfrak{R}_+^K$，生产 L 种期望产出 $y=(y_1,y_2,\cdots,y_L)^{\mathrm{T}}\in\mathfrak{R}_+^L$ 和 M 种非期望产出 $b=(b_1,b_2,\cdots,b_M)^{\mathrm{T}}\in\mathfrak{R}_+^M$ 的生产过程。基于期望产出与非期望产出弱可处置性的生产技术集合 T 可表示为：

$$T=\{(x,y,b):x\text{ 能生产 }(y,b)\} \tag{6.1}$$

其包含了技术上所有可行的投入产出组合，即如果 x 能生产 (y,b)，那么 $(x,y,b)\in T$ 。

该生产技术通常满足以下假设：

1. **无产出可行性**：对于任意投入 x，均有 $(x,0,0)\in T$。这表示投入可以闲置，或在极端情形下未形成期

望产出和非期望产出。

2. **无免费午餐**：对于 $y > 0$ 和 $b > 0$，$(0, y, b) \notin T$。即没有投入不能产生正的期望产出和污染排放。
3. **零投入约束**：若 $(x, y, b) \in T$ 且 $x = 0$，则 $(y, b) = (0, 0)$。
4. **期望产出强可处置性**：若 $(x, y, b) \in T$ 且 $0 \leq y' \leq y$，则 $(x, y', b) \in T$。这意味着在投入和排放不变时，减少期望产出仍然可行。
5. **投入强可处置性**：若 $(x, y, b) \in T$ 且 $x' \geq x$，则 $(x', y, b) \in T$。这意味着使用更多投入不会破坏原有生产计划的可行性。
6. **封闭性**：技术集 T 是闭集，即包含自身边界；并且对任意给定投入 x，可行产出集 $P(x) = \{(y, b) : (x, y, b) \in T\}$ 有界，即有限投入不能产生无限的期望产出与非期望产出。注意有界性针对的是给定投入下的产出集，而不是 T 本身——CRS 下环境技术是一个锥，沿射线是无界的。
7. **期望产出与非期望产出弱可处置性**：若 $(x, y, b) \in T$ 且 $0 \leq \theta \leq 1$，则 $(x, \theta y, \theta b) \in T$。该假设要求期望产出和污染排放按相同比例收缩。
8. **零结合性**：若 $(x, y, b) \in T$ 且 $b = 0$，则 $y = 0$。该假设刻画污染排放与正常产出之间不可完全分离的生产关系。
9. **凸性**：可行生产计划的凸组合仍然可行。
10. **规模报酬设定**：技术集可设为规模报酬不变（CRS）或规模报酬可变（VRS）。

若去除假设 (7) 和 (8)，上述集合退化为经典 DEA 生产技术。假设 (1)—(3) 保证生产活动必须以投入为基础；假设 (4) 和 (5) 分别对应期望产出和投入的强可处置性；假设 (6) 保证技术集合封闭、且给定投入下的产出集有界；假设 (9) 表示凸性，即可行生产计划的加权平均仍然可行；假设 (10) 刻画规模报酬结构。加入假设 (7) 后，期望产出和非期望产出必须按同一比例收缩，体现弱可处置性；加入假设 (8) 后，若非期望产出为零，则期望产出也为零，体现零结合性。二者共同刻画了污染排放与正常产出之间的联合生产关系。

6.1.1 规模报酬性质

规模报酬（Return To Scale）刻画投入规模同比例变化时，期望产出和非期望产出组合如何变化。在包含非期望产出的生产技术中，规模报酬设定不仅改变效率前沿的位置，也决定效率差距是否包含规模因素。CRS 与 VRS 的选择直接关系到环境绩效指标的经济解释。下文分别讨论两类设定。

(1) 规模报酬不变(CRS)

在规模报酬不变（CRS）下，若某一投入—产出—排放组合可行，则其同比例扩张或收缩后的组合仍然可行。数学上，若 $(x, y, b) \in T$ 且 $\lambda > 0$，则 $(\lambda x, \lambda y, \lambda b) \in T$。该设定适用于所有决策单元面临同一规模技术的情形。

(2) 规模报酬可变(VRS)

在规模报酬可变（VRS）下，投入按比例变化时，期望产出和非期望产出不必按相同比例变化。若产出和排放扩张比例高于投入扩张比例，则对应规模报酬递增；反之则对应规模报酬递减。VRS 设定允许不同规模的决策单元具有不同的局部前沿，因此在地区或国家样本规模差异较大时更常用于解释不含规模效应的技术效率。

6.1.2 生产技术的参数刻画

集合形式说明了环境生产技术的经济含义，但实证测度还需要将其转化为可求解的DEA 生产前沿。根据规模报酬设定不同，包含非期望产出的 DEA 技术可分为 CRS 技术和 VRS 技术；根据弱可处置性的处理方式不同，又可进一步区分相同减排因子和不同减排因子两类模型。前者假定所有参考单元在同一减排比例下收缩期望产出和非期望产出，后者允许不同参考单元具有不同减排比例，从而更直接地刻画污染处置技术异质性。

(1) 规模报酬不变的生产技术

假设有 N 个决策单元 (decision-making unit, DMU)，每个 $\text{DMU}_i(i=1,2,\cdots,N)$ 的投入产出组合为 (x_i,y_i,b_i) 。根据 Färe, Grosskopf, Lovell and Pasurka (1989)，在规模报酬不变 (CRS) 假设下，基于期望产出和非期望产出之间弱可处置性的生产技术可表示为

$$T_{\text{CRS}}=\left\{(x,y,b):\begin{array}{l}\sum\limits_{i=1}^{N}z_iy_i\geqslant y,\\ \sum\limits_{i=1}^{N}z_ib_i=b,\\ \sum\limits_{i=1}^{N}z_ix_i\leqslant x,\\ z_i\geqslant 0,\quad i=1,2,\ldots,N\end{array}\right\} \tag{6.2}$$

式中，$z_i(i=1,2,\cdots,N)$ 表示构建生产技术前沿时各决策单元的权重。期望产出的“≥” 约束表明任一决策单元可在参考单元期望产出的凸组合范围内生产，体现期望产出的强可处置性；投入的“≤” 约束表明任一决策单元的投入可超出参考单元投入的凸组合，体现投入的强可处置性；非期望产出的“=” 约束表明任一决策单元的非期望产出等于参考单元非期望产出的凸组合，体现非期望产出的弱可处置性。

图 6.1 展示了规模报酬不变假设下的生产技术。由于生产技术满足规模报酬不变假设，各产出均可除以投入，相应的单位投入生产可能性区域为 $OABCDO$，生产技术前沿为 $OABC$。前沿段 OAB 体现了期望产出 y 和非期望产出 b 之间的弱可处置性。相比之下，生产可能性区域 $OEBCDO$ 对应非期望产出强可处置性假设下的情形，前沿段 EB 体现了在保持期望产出不变的情况下减少非期望产出的可能性。

(2) 规模报酬可变的生产技术

CRS 对所有决策单元施加同一规模技术约束，假设较强。VRS 技术通过加入 $\sum_{i=1}^{N}z_i=1$ 构造凸包前沿，并引入减排因子 θ 以保持期望产出和非期望产出的弱可处置性。若缺少减排因子，模型可能在 VRS 条件下破坏污染物与期望产出联合生产的结构。沿 Färe, Grosskopf, Lovell and Pasurka (1989) 弱可处置技术的 VRS 延伸、并按 Färe and Grosskopf (2003) 的表述，**相同**减排因子（即所有参考单元共用同一个 θ）的 VRS 技术可表示为：

$$T_{\text{VRS}}=\left\{(x,y,b):\begin{array}{l}\theta\sum_{i=1}^{N}z_iy_i\geqslant y,\\ \theta\sum_{i=1}^{N}z_ib_i=b,\\ \sum_{i=1}^{N}z_ix_i\leqslant x,\\ \sum_{i=1}^{N}z_i=1,\\ 0\leqslant\theta\leqslant 1,\ z_i\geqslant 0,\ i=1,2,\ldots,N\end{array}\right\} \tag{6.3}$$

式中各项约束的经济含义同式(6.2)；其中非期望产出的”=“约束与减排因子 θ、权重之和约束

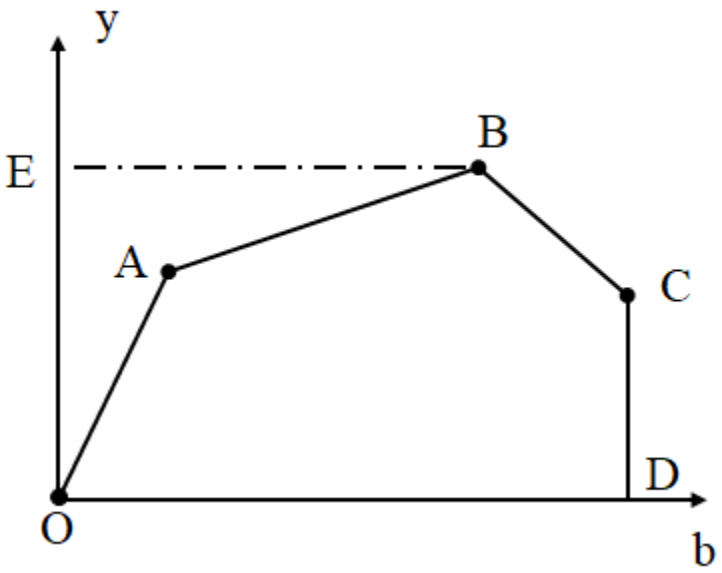


图 6.1: 规模报酬不变假设下的生产技术

$\sum_{i=1}^{N} z_i = 1$ 共同体现非期望产出的弱可处置性。

式 (6.3) 中的减排因子 θ 对所有 DMU 均相同。Kuosmanen (2005) 指出，共用同一个 θ 隐含参照集中所有单元按相同比例削减期望产出与非期望产出，这一限制在 VRS 下并非弱可处置性所必需；他给出的凸表述允许各 DMU 拥有不同的减排因子 θ_i。当 $\theta_1 = \theta_2 = \cdots = \theta_N$ 时，T^K_{VRS} 与 T_{VRS} 等价，即 T_{VRS} 是 T^K_{VRS} 的特例。

在规模报酬可变 (VRS) 假设下，具有不同减排因子 θ_i 的生产技术可表示为：

$$
\begin{aligned}
T^K_{\text{VRS}} = \{(x, y, b) : & \sum_{i=1}^{N} \theta_i z_i y_i \geqslant y \\
& \sum_{i=1}^{N} \theta_i z_i b_i = b \\
& \sum_{i=1}^{N} z_i x_i \leqslant x \\
& \sum_{i=1}^{N} z_i = 1, \\
& 0 \leqslant \theta_i \leqslant 1, z_i \geqslant 0, i = 1, 2, \cdots, N\}
\end{aligned}
\tag{6.4}
$$

图 6.2 展示了规模报酬可变假设下的生产技术，相应的生产可能性区域为 $OABCD$，生产技术前沿为 $OABC$。前沿段 OAB 体现了期望产出 y 和非期望产出 b 之间的弱可处置性。相比之下，若不加减排因子，则生产可能性区域为 $FABCD$，该情形违反产出的弱可处置性。

CRS 技术与 VRS 技术的关键区别在于是否施加权重约束 $\sum_{i=1}^{N} z_i = 1$。在 VRS 情形下，该约束使生产前沿能够反映局部规模差异；减排因子则保证期望产出与非期望产出的弱可处置关系不被破坏。

对非期望产出施加等式约束可能导致生产技术边界出现后弯现象（Chen and Delmas, 2012），如 图 6.2 中 BC 段所示。为避免后弯，Leleu (2013) 基于非期望产出影子价格非负的约束，利用对偶理论将非期望

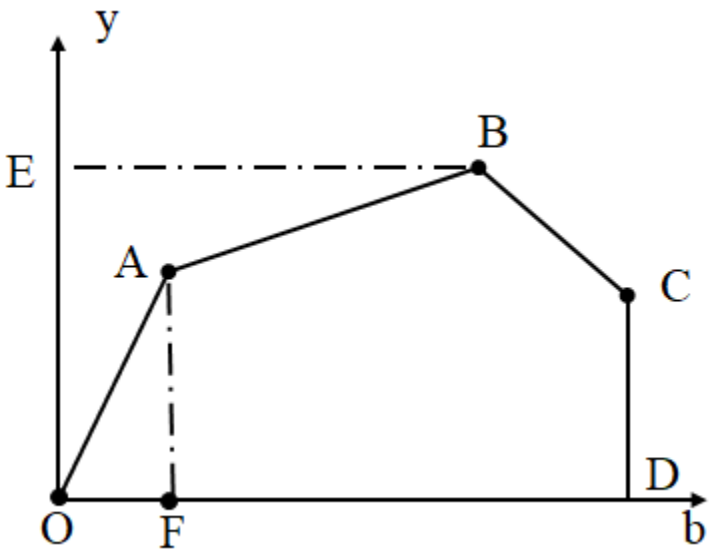


图 6.2: 规模报酬可变假设下的生产技术

产出的等式约束修正为不等式约束。相同减排因子和不同减排因子下的修正环境生产技术分别为：

$$T_{\text{VRS}}^{M}=\left\{\begin{array}{l}(x,y,b):\\ \theta\sum_{i=1}^{N}z_iy_i\geqslant y,\\ \theta\sum_{i=1}^{N}z_ib_i\leqslant b,\\ \sum_{i=1}^{N}z_ix_i\leqslant x,\\ \sum_{i=1}^{N}z_i=1,\\ 0\leqslant\theta\leqslant 1,\quad z_i\geqslant 0,\quad i=1,2,\ldots,N\end{array}\right\} \tag{6.5}$$

$$T_{\text{VRS}}^{MK}=\left\{\begin{array}{l}(x,y,b):\\ \sum_{i=1}^{N}\theta_iz_iy_i\geqslant y,\\ \sum_{i=1}^{N}\theta_iz_ib_i\leqslant b,\\ \sum_{i=1}^{N}z_ix_i\leqslant x,\\ \sum_{i=1}^{N}z_i=1,\\ 0\leqslant\theta_i\leqslant 1,\quad z_i\geqslant 0,\quad i=1,2,\ldots,N\end{array}\right\} \tag{6.6}$$

修正后，图 6.2 中的 BC 段将不存在，相应的产出可能性区域变为 $OABGD$，如 图 6.3 所示。

6.2 绩效测度

在给定环境生产技术后，绩效测度的任务是计算观测点相对于前沿的距离；距离函数的选择，决定了“改进空间” 究竟指向增产、节能、减排，还是这些目标的联合变化。本节比较谢泼德距离函数、双曲距离函数、方向距离函数和非径向方向距离函数。谢泼德距离函数通常沿单一方向进行径向调整；双曲距离函数同时扩张期望产出并压缩非期望产出或投入；方向距离函数通过方向向量统一描述投入缩减、期望产出扩张和非期望产出压缩；非径向方向距离函数进一步允许不同变量具有不同调整幅度。实证研究必须在模型

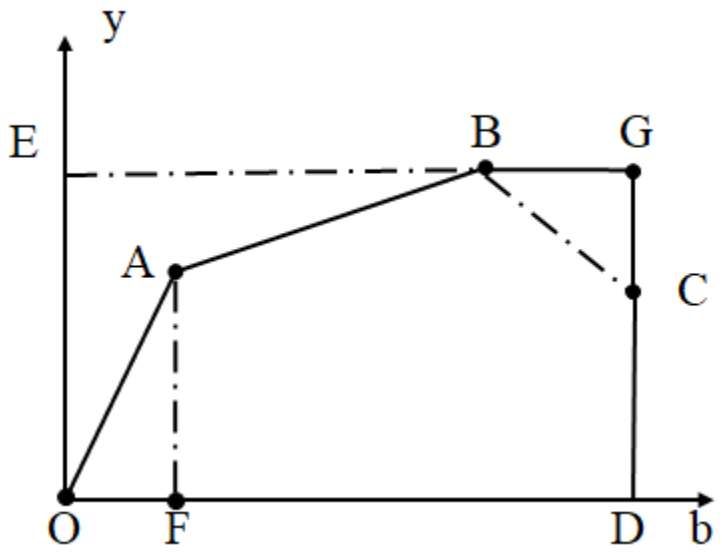


图 6.3: 规模报酬可变假设下修正的环境生产技术

设定、指标转换和结果解释之间保持一致。下文每一类距离函数都将在 6.1 节给出的 CRS、相同减排因子 VRS 与不同减排因子 VRS 三种基本技术集（式6.2–6.4）上分别求解；不同技术集对应不同效率前沿，但同一距离函数的指标口径保持一致。修正集（式6.5–6.6）主要用于避免技术后弯，本章OECD/省份样本未应用该修正；若样本中出现后弯（影子价格非负约束被破坏），应改用式(6.5)–(6.6) 重新估计。

6.2.1 谢泼德距离函数

谢泼德距离函数沿某一类投入或产出进行径向调整，因此适合度量单一方向上的效率。若沿期望产出方向扩张，则得到产出效率；若沿投入方向收缩，则得到投入效率；若沿非期望产出方向收缩，则得到环境效率。其优点是指标含义清晰、模型形式简洁；局限在于同一方向上的所有变量均按相同比例调整，难以反映变量层面的松弛差异。当投入或产出包含多个变量时，研究者还需要明确哪些变量被纳入调整方向。

（一）期望产出方向的谢泼德距离函数期望产出方向的谢泼德距离函数衡量在给定投入和非期望产出水平下，期望产出可按比例扩张的空间。其定义为：

$$D_o(x, y, b) = \inf\{\theta : (x, y/\theta, b) \in T\} \tag{6.7}$$

式中，T 为包含非期望产出的生产可能性集合；θ 是**谢泼德产出距离函数值**，即实际期望产出与前沿期望产出之比（前沿产出为 y/θ），而不是" 可扩张的比例"——最大扩张倍数是 $1/\theta$。$\theta = 1$ 表明被评价决策单元位于产出前沿面上，$\theta < 1$ 表明被评价决策单元的产出存在扩张空间，纯产出效率取 $PEO = \theta$。以图 6.4 所示的生产可能性区域为例，B 点位于生产前沿上，距离函数值为1；而 A 点位于边界下方，距离函数值 $\theta = OA/OA'$ 小于 1，存在扩张空间（该点的产出最多可扩张 $OA'/OA = 1/\theta$ 倍）。

在 CRS 假设下，期望产出方向的谢泼德距离函数可由下列规划求解。规划的决策变量 λ 是 Farrell 型

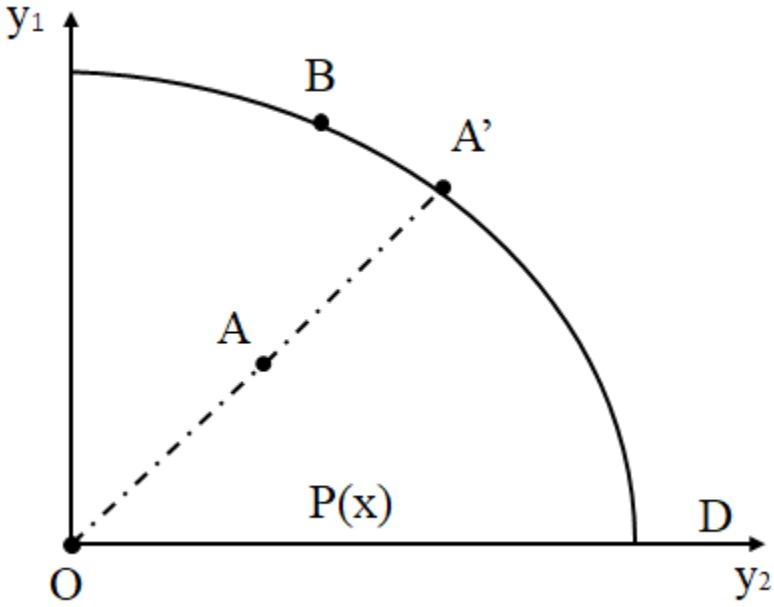


图 6.4: 期望产出方向的谢泼德距离函数

扩张倍数，与谢泼德距离互为倒数，即 $\lambda = 1/\theta$；因此下面求解的是 $1/D_o$，而不是 D_o 本身：

$$\begin{aligned} & \lambda^*_{\text{o,CRS}} = \max \lambda \\ \text{s.t.} \quad & \textstyle\sum_{i=1}^{N} z_i y_{li} \geqslant \lambda y_{l0}, l = 1, 2, \cdots, L, \\ & \textstyle\sum_{i=1}^{N} z_i b_{mi} = b_{m0}, m = 1, 2, \cdots, M, \\ & \textstyle\sum_{i=1}^{N} z_i x_{ki} \leqslant x_{k0}, k = 1, 2, \cdots, K, \\ & z_i \geqslant 0, i = 1, 2, \cdots, N \end{aligned} \tag{6.8}$$

式中，$z = (z_1, z_2, \cdots, z_N)$ 为权重向量。式(6.8)在保持投入和非期望产出不变的条件下，径向扩张所有期望产出。若目标函数值为 λ^*，则最佳期望产出量为 $\lambda^* y$，纯产出效率(PEO)为：

$$PEO = \frac{y}{\lambda^* y} = \frac{1}{\lambda^*}$$

式中，仅当 $\lambda^* = 1$ 时，有 $PEO = 1$，代表产出在生产前沿上；否则，当 $\lambda^* > 1$ 时，$PEO < 1$，表明产出效率存在提升空间。

在 VRS 假设下，需要分别讨论相同减排因子和不同减排因子两种弱可处置技术。

相同减排因子下的模型为：

$$\begin{aligned} & \lambda^*_{\text{o,VRS}} = \max \lambda \\ \text{s.t.} \quad & \theta \textstyle\sum_{i=1}^{N} z_i y_{li} \geqslant \lambda y_{l0}, l = 1, 2, \cdots, L, \\ & \theta \textstyle\sum_{i=1}^{N} z_i b_{mi} = b_{m0}, m = 1, 2, \cdots, M, \\ & \textstyle\sum_{i=1}^{N} z_i x_{ki} \leqslant x_{k0}, k = 1, 2, \cdots, K, \\ & \textstyle\sum_{i=1}^{N} z_i = 1, \\ & z_i \geqslant 0, 0 < \theta \leq 1, i = 1, 2, \cdots, N \end{aligned} \tag{6.9}$$

在投入 x 约束和权重之和约束的两边同乘 θ，并令 $z_i\theta$=$z_i'(i=1,...,N)$，可得与上述模型等价的新模型：

$$\begin{aligned}
&\lambda^*_{\text{o,VRS}} = \max \lambda \\
\text{s.t.}\quad &\textstyle\sum_{i=1}^{N} z_i' y_{li} \geqslant \lambda y_{l0}, l=1,2,\cdots,L, \\
&\textstyle\sum_{i=1}^{N} z_i' b_{mi} = b_{m0}, m=1,2,\cdots,M, \\
&\textstyle\sum_{i=1}^{N} z_i' x_{ki} \leqslant \theta x_{k0}, k=1,2,\cdots,K, \\
&\textstyle\sum_{i=1}^{N} z_i' = \theta, \\
&z_i' \geqslant 0, 0<\theta\leq 1, i=1,2,\cdots,N
\end{aligned}$$

其等价形式为：

$$\begin{aligned}
&\lambda^*_{\text{o,VRS}} = \max \lambda \\
\text{s.t.}\quad &\textstyle\sum_{i=1}^{N} z_i y_{li} \geqslant \lambda y_{l0}, l=1,2,\cdots,L, \\
&\textstyle\sum_{i=1}^{N} z_i b_{mi} = b_{m0}, m=1,2,\cdots,M, \\
&\textstyle\sum_{i=1}^{N} z_i x_{ki} \leqslant \theta x_{k0}, k=1,2,\cdots,K, \\
&\textstyle\sum_{i=1}^{N} z_i = \theta, \\
&z_i \geqslant 0, 0<\theta\leq 1, i=1,2,\cdots,N
\end{aligned} \tag{6.10}$$

若目标函数值为 λ^*，纯产出效率（PEO）为

$$PEO = \frac{y}{\lambda^* y} = \frac{1}{\lambda^*}$$

指标解释同式（6.8）后所述：$\lambda^*=1$ 时产出位于生产前沿，$\lambda^*>1$ 时存在效率提升空间。

不同减排因子下的模型为：

$$\begin{aligned}
&\lambda^{*,K}_{\text{o,VRS}} = \max \lambda \\
\text{s.t.}\quad &\textstyle\sum_{i=1}^{N} \theta_i z_i y_{li} \geqslant \lambda y_{l0}, l=1,2,\cdots,L, \\
&\textstyle\sum_{i=1}^{N} \theta_i z_i b_{mi} = b_{m0}, m=1,2,\cdots,M, \\
&\textstyle\sum_{i=1}^{N} z_i x_{ki} \leqslant x_{k0}, k=1,2,\cdots,K, \\
&\textstyle\sum_{i=1}^{N} z_i = 1, \\
&z_i \geqslant 0, 0<\theta_i\leq 1, i=1,2,\cdots,N
\end{aligned} \tag{6.11}$$

令 $\theta_i z_i = \rho_i$，并将 z_i 分解为 ρ_i 和 μ_i 两部分，即：$z_i=\rho_i+\mu_i$，则 $\theta_i = \frac{\rho_i}{\rho_i+\mu_i}$。可得上式的等价形式为：

$$\begin{aligned}
&\lambda^{*,K}_{\text{o,VRS}} = \max \lambda \\
\text{s.t.}\quad &\textstyle\sum_{i=1}^{N} \rho_i y_{li} \geqslant \lambda y_{l0}, l=1,2,\cdots,L, \\
&\textstyle\sum_{i=1}^{N} \rho_i b_{mi} = b_{m0}, m=1,2,\cdots,M, \\
&\textstyle\sum_{i=1}^{N} (\rho_i+\mu_i) x_{ki} \leqslant x_{k0}, k=1,2,\cdots,K, \\
&\textstyle\sum_{i=1}^{N} (\rho_i+\mu_i) = 1, \\
&\rho_i, \mu_i \geqslant 0, i=1,2,\cdots,N
\end{aligned} \tag{6.12}$$

求解上述线性规划，若目标函数值为 λ^*，纯产出效率（PEO）为

$$PEO = \frac{y}{\lambda^* y} = \frac{1}{\lambda^*}$$

指标解释同式（6.8）后所述，不再重复。

（二）投入方向的谢泼德距离函数（纯投入效率）与产出方向对应，投入方向把改进指向投入端，可识别资本、劳动和能源的可压缩比例。投入方向的谢泼德距离函数衡量在给定期望产出和非期望产出条件下，投入可按比例缩减的空间。其定义为：

$$D_i(x,y,b)=\sup\{\theta:(x/\theta,y,b)\in T\} \tag{6.13}$$

式中，T 为包含非期望产出的生产可能性集合；θ 是**谢泼德投入距离函数值**，即投入的最大可缩倍数（前沿投入为 x/θ），而不是”缩减的比例”——Farrell 型投入效率是它的倒数 $1/\theta$。$\theta=1$ 表明被评价决策单元位于生产前沿面上，$\theta>1$ 表明被评价决策单元的投入存在缩减空间。以图 6.5 所示的生产可能性区域为例，B 点位于生产前沿上，距离函数值为 1；而 A 点位于边界上方，距离函数值 $\theta=OA/OA'$ 大于1，存在缩减空间（投入最多可压缩到原来的 $OA'/OA=1/\theta$）。

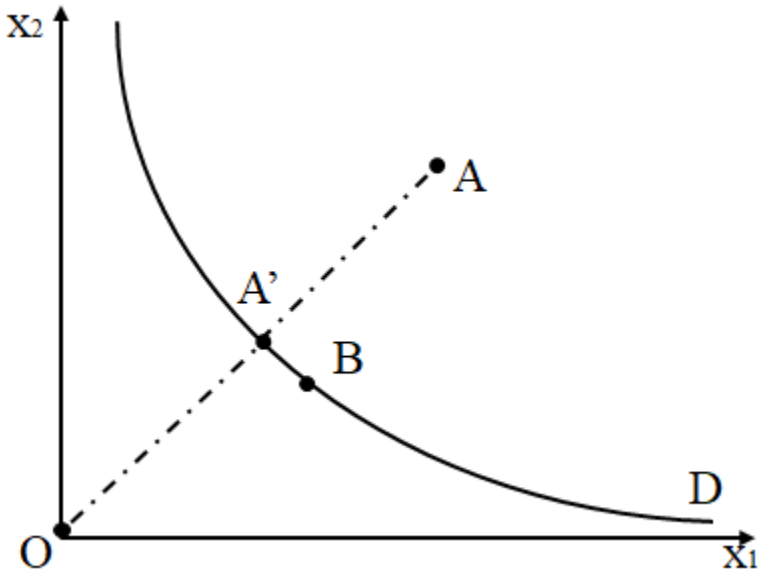


图 6.5: 投入方向的谢泼德距离函数

在 CRS 假设下，投入方向的谢泼德距离函数可由下列规划求解。与产出方向同理，规划的决策变量 λ 是 Farrell 型投入效率，与谢泼德投入距离互为倒数（$\lambda=1/\theta\leq 1$）：

$$\begin{aligned}
&\lambda^*_{\text{i,CRS}}=\min\lambda\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}z_iy_{li}\geqslant y_{l0},l=1,2,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}z_ib_{mi}=b_{m0},m=1,2,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant\lambda x_{k0},k=1,2,\cdots,K,\\
&z_i\geqslant 0,i=1,2,\cdots,N
\end{aligned} \tag{6.14}$$

式中，$z=(z_1,z_2,\cdots,z_N)$ 为权重向量。式（6.14）在保持期望产出和非期望产出不变的条件下，径向缩减所有投入。若目标函数值为 λ^*，则最佳投入量为 λ^*x，纯投入效率（PEI）为

$$PEI=\frac{\lambda^*x}{x}=\lambda^*$$

式中，仅当 $\lambda^* = 1$ 时，有 $PEI = 1$，代表投入利用是有效的；否则，当 $\lambda^* < 1$ 时，$PEI < 1$，表明投入效率存在提升空间。

VRS 下区分相同/不同减排因子两种情形：

相同减排因子下的模型为：

$$\begin{aligned} & \lambda^*_{\text{i,VRS}} = \min \lambda \\ \text{s.t.} \quad & \theta \sum_{i=1}^{N} z_i y_{li} \geqslant y_{l0}, l = 1, 2, \cdots, L, \\ & \theta \sum_{i=1}^{N} z_i b_{mi} = b_{m0}, m = 1, 2, \cdots, M, \\ & \sum_{i=1}^{N} z_i x_{ki} \leqslant \lambda x_{k0}, k = 1, 2, \cdots, K, \\ & \sum_{i=1}^{N} z_i = 1, \\ & z_i \geqslant 0, 0 < \theta \leq 1, i = 1, 2, \cdots, N \end{aligned} \tag{6.15}$$

该模型不能直接线性化，后续计算不采用这一设定。理由是：投入约束中含有同因子减排变量 θ 与效率变量 λ 的乘积项，重标定 $z_i \to z_i/\theta$ 后会留下 $\theta\lambda$ 双线性残差。本章对此类“投入方向 + 同因子” 设定一律仅列形式、不求解，下文 6.2.2 节–6.2.4 节遇到同类情形不再重复说明。

不同减排因子下的模型为：

$$\begin{aligned} & \lambda^{*,K}_{\text{i,VRS}} = \min \lambda \\ \text{s.t.} \quad & \sum_{i=1}^{N} \theta_i z_i y_{li} \geqslant y_{l0}, l = 1, 2, \cdots, L, \\ & \sum_{i=1}^{N} \theta_i z_i b_{mi} = b_{m0}, m = 1, 2, \cdots, M, \\ & \sum_{i=1}^{N} z_i x_{ki} \leqslant \lambda x_{k0}, k = 1, 2, \cdots, K, \\ & \sum_{i=1}^{N} z_i = 1, \\ & z_i \geqslant 0, 0 < \theta_i \leq 1, i = 1, 2, \cdots, N \end{aligned} \tag{6.16}$$

做与式 (6.12) 相同的替换 $\theta_i z_i = \rho_i,\ z_i = \rho_i + \mu_i$，得等价线性规划：

$$\begin{aligned} & \lambda^{*,K}_{\text{i,VRS}} = \min \lambda \\ \text{s.t.} \quad & \sum_{i=1}^{N} \rho_i y_{li} \geqslant y_{l0}, l = 1, 2, \cdots, L, \\ & \sum_{i=1}^{N} \rho_i b_{mi} = b_{m0}, m = 1, 2, \cdots, M, \\ & \sum_{i=1}^{N} (\rho_i + \mu_i) x_{ki} \leqslant \lambda x_{k0}, k = 1, 2, \cdots, K, \\ & \sum_{i=1}^{N} (\rho_i + \mu_i) = 1, \\ & \rho_i, \mu_i \geqslant 0, i = 1, 2, \cdots, N \end{aligned} \tag{6.17}$$

求解上述线性规划，若目标函数值为 λ^*，纯投入效率（PEI）为

$$PEI = \frac{\lambda^* x}{x} = \lambda^*$$

指标解释同式（6.14）后所述，不再重复。

（三）非期望产出方向的谢泼德距离函数沿非期望产出方向则把改进指向污染端，这是环境绩效测度区别于经典DEA 的关键环节；上述三个方向共同构成后续双曲距离函数与 DDF 联合改进测度的基础构件。非期望产出方向的谢泼德距离函数衡量在给定投入和期望产出水平下，非期望产出可按比例压缩的空间。依据Zhou, Ang and Han (2010)，非期望产出导向距离函数为：

$$D_b(x, y, b) = \sup \{\theta : (x, y, b/\theta) \in T\} \tag{6.18}$$

式中，T 为包含非期望产出的生产可能性集合；θ 是**谢泼德非期望产出距离函数值**，即排放的最大可缩倍数（前沿排放为 b/θ），而不是" 缩减的比例"。$\theta = 1$ 表明被评价单元位于生产前沿面上，$\theta > 1$ 表明存在缩减空间。在下文线性规划中，令 $\lambda = 1/\theta \leq 1$（即Farrell 型环境效率），则 $\sup\theta$ 等价于 $\min\lambda$，纯环境效率 $PEE = \lambda^* = 1/\theta^*$。以图 6.6 所示的生产可能性区域为例，$A$ 点位于生产前沿上，距离函数值为 1；而 M 点位于边界下方，过 M 点的水平线与纵轴交于 G 点、与生产前沿交于 F 点，距离函数值 $\theta = GM/GF$ 大于1，存在缩减空间（排放最多可压缩到原来的 $GF/GM = 1/\theta$）。

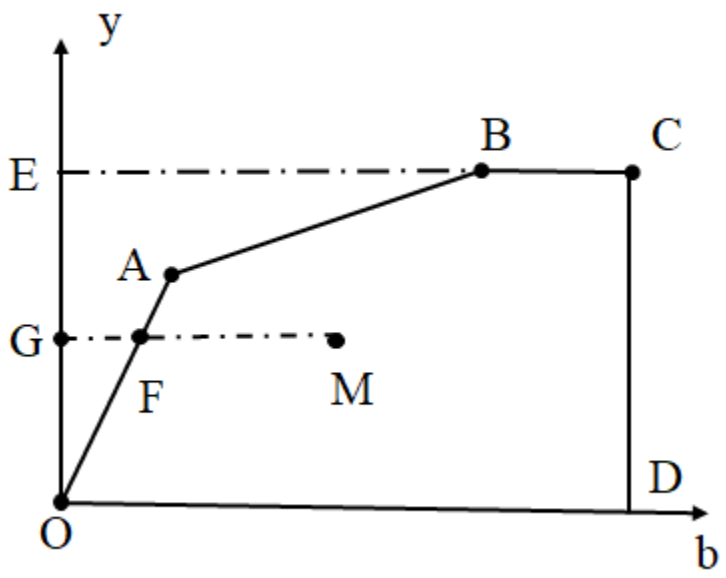


图 6.6: 非期望产出方向的谢泼德距离函数

规模报酬不变（CRS）假设下的非期望产出方向上的谢泼德距离函数可由下列规划求解：

$$\begin{aligned} & \lambda^*_{\text{b,CRS}} = \min\lambda \\ \text{s.t.} \quad & \textstyle\sum_{i=1}^{N} z_i y_{li} \geqslant y_{l0}, l = 1,2,\cdots,L, \\ & \textstyle\sum_{i=1}^{N} z_i b_{mi} = \lambda b_{m0}, m = 1,2,\cdots,M, \\ & \textstyle\sum_{i=1}^{N} z_i x_{ki} \leqslant x_{k0}, k = 1,2,\cdots,K, \\ & z_i \geqslant 0, i = 1,2,\cdots,N \end{aligned} \tag{6.19}$$

式中，$z = (z_1, z_2, \cdots, z_N)$ 为权重向量。

VRS 下区分相同/不同减排因子两种情形。相同减排因子下的模型为：

$$\begin{aligned} & \lambda^*_{\text{b,VRS}} = \min\lambda \\ \text{s.t.} \quad & \theta\textstyle\sum_{i=1}^{N} z_i y_{li} \geqslant y_{l0}, l = 1,2,\cdots,L, \\ & \theta\textstyle\sum_{i=1}^{N} z_i b_{mi} = \lambda b_{m0}, m = 1,2,\cdots,M, \\ & \textstyle\sum_{i=1}^{N} z_i x_{ki} \leqslant x_{k0}, k = 1,2,\cdots,K, \\ & \textstyle\sum_{i=1}^{N} z_i = 1, \\ & z_i \geqslant 0, 0 < \theta \leq 1, i = 1,2,\cdots,N \end{aligned} \tag{6.20}$$

在投入 x 约束和权重之和约束的两边同乘 θ，并令 $z_i\theta = z_i'(i = 1, ..., N)$，可得与上述模型等价的新模型：

$$\begin{aligned} & \lambda^*_{\text{b,VRS}} = \min \lambda \\ \text{s.t.} \quad & \sum_{i=1}^{N} z_i' y_{li} \geqslant y_{l0}, l = 1, 2, \cdots, L, \\ & \sum_{i=1}^{N} z_i' b_{mi} = \lambda b_{m0}, m = 1, 2, \cdots, M, \\ & \sum_{i=1}^{N} z_i' x_{ki} \leqslant \theta x_{k0}, k = 1, 2, \cdots, K, \\ & \sum_{i=1}^{N} z_i' = \theta, \\ & z_i' \geqslant 0, 0 < \theta \leq 1, i = 1, 2, \cdots, N \end{aligned} \tag{6.21}$$

其等价形式为：

$$\begin{aligned} & \lambda^*_{\text{b,VRS}} = \min \lambda \\ \text{s.t.} \quad & \sum_{i=1}^{N} z_i y_{li} \geqslant y_{l0}, l = 1, 2, \cdots, L, \\ & \sum_{i=1}^{N} z_i b_{mi} = \lambda b_{m0}, m = 1, 2, \cdots, M, \\ & \sum_{i=1}^{N} z_i x_{ki} \leqslant \theta x_{k0}, k = 1, 2, \cdots, K, \\ & \sum_{i=1}^{N} z_i = \theta, \\ & z_i \geqslant 0, 0 < \theta \leq 1, i = 1, 2, \cdots, N \end{aligned} \tag{6.22}$$

不同减排因子下的模型为：

$$\begin{aligned} & \lambda^{*,K}_{\text{b,VRS}} = \min \lambda \\ \text{s.t.} \quad & \sum_{i=1}^{N} \theta_i z_i y_{li} \geqslant y_{l0}, l = 1, 2, \cdots, L, \\ & \sum_{i=1}^{N} \theta_i z_i b_{mi} = \lambda b_{m0}, m = 1, 2, \cdots, M, \\ & \sum_{i=1}^{N} z_i x_{ki} \leqslant x_{k0}, k = 1, 2, \cdots, K, \\ & \sum_{i=1}^{N} z_i = 1, \\ & z_i \geqslant 0, 0 < \theta_i \leq 1, i = 1, 2, \cdots, N \end{aligned} \tag{6.23}$$

做与式 (6.12) 相同的替换 $\theta_i z_i = \rho_i,\ z_i = \rho_i + \mu_i$，得等价线性规划：

$$\begin{aligned} & \lambda^{*,K}_{\text{b,VRS}} = \min \lambda \\ \text{s.t.} \quad & \sum_{i=1}^{N} \rho_i y_{li} \geqslant y_{l0}, l = 1, 2, \cdots, L, \\ & \sum_{i=1}^{N} \rho_i b_{mi} = \lambda b_{m0}, m = 1, 2, \cdots, M, \\ & \sum_{i=1}^{N} (\rho_i + \mu_i) x_{ki} \leqslant x_{k0}, k = 1, 2, \cdots, K, \\ & \sum_{i=1}^{N} (\rho_i + \mu_i) = 1, \\ & \rho_i, \mu_i \geqslant 0, i = 1, 2, \cdots, N \end{aligned} \tag{6.24}$$

求解上述线性规划，若目标函数值为 λ^*，则纯环境效率（PEE）为：

$$PEE = \frac{\lambda^* b}{b} = \lambda^*$$

式中，仅当 $\lambda^* = 1$ 时，有 $PEE = 1$，表示非期望产出已经位于环境生产前沿；当 $\lambda^* < 1$ 时，$PEE < 1$，表明在保持投入和期望产出不变的条件下，非期望产出仍存在压缩空间。

6.2.2 双曲距离函数

双曲距离函数（Hyperbolic Distance Function，代码中标记为HYPER）是谢泼德距离函数的扩展，能够在同一模型中同时扩张期望产出并压缩非期望产出，或同时扩张期望产出并压缩投入。因此，它衡量的是

联合改进意义上的混合效率。与单一方向的谢泼德距离函数相比，双曲距离函数更适合分析“增产—减排”或“增产—节约投入”的协同改进空间。但也正因其同时调整多个端口，结果解释不能简单等同于纯产出效率、纯投入效率或纯环境效率。

（一）同时调整期望产出和非期望产出当研究重点是同时增加期望产出并减少非期望产出时，双曲距离函数可写为：

$$G(x,y,b)=\inf\{\lambda>0:(x,y/\lambda,\lambda b)\in T\} \tag{6.25}$$

G（即最优 λ）是双曲距离函数的值。当 $G=1$ 时，被评价决策单元位于生产前沿上；当 $G<1$ 时，$y/G>y$（期望产出扩张）、$Gb<b$（非期望产出缩减），决策单元位于生产前沿内部，存在联合改进空间。图 6.7 中的产出曲线表示给定非期望产出 b 对应的产出 y；经过 A 点的反比例函数刻画该决策单元同时扩张期望产出、缩减非期望产出的调整路径。双曲距离函数的值 $G=OA/OA'$ 小于 1，表明存在改进空间，可改进至 A' 点。

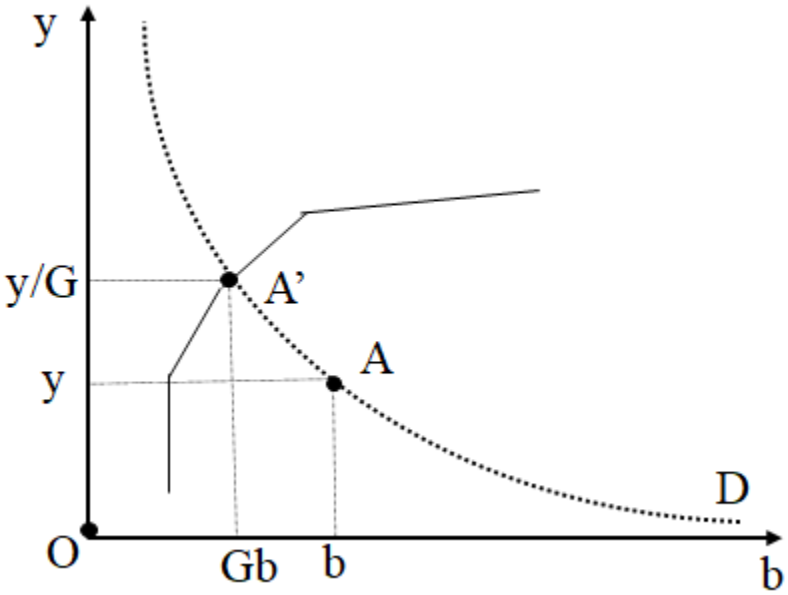


图 6.7: 同时调整期望产出和非期望产出时的产出曲线

在 CRS 假设下，同时调整期望产出和非期望产出的双曲距离函数可由下列规划求解：

$$\begin{aligned}
&\mathrm{D}_{\mathrm{yb,CRS}}=\min\lambda\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}z_iy_{li}\geqslant\frac{y_{l0}}{\lambda},l=1,2,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}z_ib_{mi}=\lambda b_{m0},m=1,2,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant x_{k0},k=1,2,\cdots,K,\\
&z_i\geqslant0,i=1,2,\cdots,N
\end{aligned} \tag{6.26}$$

式中，$z=(z_1,z_2,\cdots,z_N)$ 为权重向量。为便于求解，需将该非线性问题转化为线性规划：在每个约束两边

同乘 λ，并令 $\lambda^2=\delta$、$\lambda z_i=z_i'$，可得如下线性规划模型：

$$\begin{aligned}&\mathrm{D}_{\text{yb,CRS}}=\min\delta\\ \text{s.t.}\quad&\textstyle\sum_{i=1}^{N}z_i'y_{li}\geqslant y_{l0},l=1,2,\cdots,L,\\&\textstyle\sum_{i=1}^{N}z_i'b_{mi}=\delta b_{m0},m=1,2,\cdots,M,\\&\textstyle\sum_{i=1}^{N}z_i'x_{ki}\leqslant\lambda x_{k0},k=1,2,\cdots,K,\\&z_i'\geqslant 0,0<\lambda,\delta\leq 1,i=1,2,\cdots,N\end{aligned}\tag{6.27}$$

关于式 (6.27)/(6.34) 的退化说明：上述线性化定义 $\delta=\lambda^2$ 后，**并未在 LP 中显式施加 $\delta=\lambda^2$ 的链接约束**，δ 与 λ 在优化中事实上解耦。求解器会把 λ 推向1（放松投入约束），δ 退化为非期望产出方向的 Shephard 距离，期望产出扩张项 y/λ 因此从最优解中消失。结果是 δ^*（及相应的 MEE_{CRS}）只是非期望产出 Shephard 距离（即 PEE_{CRS}）的单调变换——可验证 $MEE_{CRS}\approx\sqrt{PEE_{CRS}}$，不携带" 同时扩张 y、压缩 b"的联合改进信息。精确的 CRS 双曲距离需借助 Charnes–Cooper 变换或二阶锥规划（SOCP）求解，本章沿用文献中的常见写法仅作示意；**6.4 节 案例中 CRS 双曲列应理解为坏产出端 Shephard 距离的单调变换，不可与 VRS 双曲或 DDF 作水平等同比较**。VRS 双曲（式 6.30/6.31）通过 Taylor 展开转化为 DDF 形式，不存在此退化问题。

在 VRS 且减排因子相同的情形下，模型为：

$$\begin{aligned}&\mathrm{D}_{\text{yb,VRS}}=\min\lambda\\ \text{s.t.}\quad&\theta\textstyle\sum_{i=1}^{N}z_iy_{li}\geqslant\frac{y_{l0}}{\lambda},l=1,2,\cdots,L,\\&\theta\textstyle\sum_{i=1}^{N}z_ib_{mi}=\lambda b_{m0},m=1,2,\cdots,M,\\&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant x_{k0},k=1,2,\cdots,K,\\&\textstyle\sum_{i=1}^{N}z_i=1,\\&z_i\geqslant 0,0<\theta\leq 1,i=1,2,\cdots,N\end{aligned}\tag{6.28}$$

将 λ 替换为 $e^{-\beta}$，则 $1/\lambda$ 可替换为 e^{β}。根据泰勒公式，$e^{-\beta}\approx 1-\beta$ 且 $e^{\beta}\approx 1+\beta$，此时可将非线性模型转化为线性模型：

$$\begin{aligned}&\mathrm{D}_{\text{yb,VRS}}=\max\beta\\ \text{s.t.}\quad&\theta\textstyle\sum_{i=1}^{N}z_iy_{li}\geqslant y_{l0}+\beta y_{l0},l=1,2,\cdots,L,\\&\theta\textstyle\sum_{i=1}^{N}z_ib_{mi}=b_{m0}-\beta b_{m0},m=1,2,\cdots,M,\\&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant x_{k0},k=1,2,\cdots,K,\\&\textstyle\sum_{i=1}^{N}z_i=1,\\&z_i\geqslant 0,0<\theta\leq 1,i=1,2,\cdots,N\end{aligned}\tag{6.29}$$

该线性规划与方向向量 $g=(g_y,g_b)=(y,b)$ 的方向距离函数相对应。因此，可按照第 3 章的处理方式，将双曲距离函数转换为方向距离函数形式。

其等价形式为：

$$\begin{aligned}&D_{yb,VRS}=\max_{z_i,\beta}\beta\\ \text{s.t.}\quad&\textstyle\sum_{i=1}^{N}z_iy_{li}\geqslant y_{l0}+\beta y_{l0},l=1,\cdots,L\\&\textstyle\sum_{i=1}^{N}z_ib_{mi}=b_{m0}-\beta b_{m0},m=1,\cdots,M\\&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant\theta x_{k0},k=1,\cdots,K,\\&\textstyle\sum_{i=1}^{N}z_i=\theta\\&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N\end{aligned}\tag{6.30}$$

求解上述线性规划，若目标函数值为 β^*，则期望产出端的混合效率 MEE_O 与非期望产出端的混合效率 MEE_{BO} 分别为：

$$MEE_O = \frac{1}{1+\beta^*}, \qquad MEE_{BO} = 1-\beta^*$$

两个公式对应同一 β^* 的两端：MEE_O 对应原始双曲比例 $1/e^{\beta}$ 在 $\beta=0$ 处的一阶展开 $1/(1+\beta^*)$，MEE_{BO} 为其对 β 的线性展开 $1-\beta^*$，二者在 β 较小时数值接近但量纲不同——前者刻画 y 扩张幅度，后者刻画 b 压缩幅度。式中，仅当 $\beta^*=0$ 时，$MEE_O = MEE_{BO} = 1$，代表产出是有效的；否则，当 $\beta^*>0$ 时，二者均小于 1，表明存在联合改进空间。本章 6.4 节案例在 VRS 设定下同时报告两者（代码列 `teo`、`teuo` 分别整理为 `MEE_O`、`MEE_BO`）；CRS 设定下因双曲线性化的退化（见式6.27 处说明），仅报告由坏产出 Shephard 距离单调变换得到的单一 MEE_{CRS}，不再区分两端。

双曲与 DDF 的等价性说明：式 (6.30) 经 Taylor 近似得到的线性规划，在结构上与 6.2.3 节方向向量取 $g=(g_y,g_b)=(y,b)$ 的 VRS 同因子方向距离函数（式6.47）**完全相同**——二者约束完全一致，β^* 相同，因而 $MEE_O = EO$、$MEE_{BO} = EB$。需要说明的是，这种一致性来自**线性化（一阶泰勒近似）之后**的规划形式，而不是说双曲距离函数本身是DDF 的特例：原始双曲的投影路径是双曲线 $(y/\lambda, \lambda b)$，DDF 的投影路径是直线 $(y+\beta g_y, b-\beta g_b)$，二者只在 β 很小时近似重合。本章仍将双曲单独列为一类，是因为其原始非线性形式（式6.25）导出了不同的线性化路径与近似处理；但读者在比较 6.4 节 的VRS 双曲结果（ 表 6.4 与 表 6.4）与 VRS DDF 结果（ 表 6.4 与表 6.4）时应知：两者在相同方向、相同规模报酬下数值一致，**不是相互独立的稳健性对照**。真正独立的模型比较应基于CRS 双曲 vs DDF、VRS 双曲 vs NDDF、或 DDF vs NDDF。

类似地，减排因子不同时的模型为：

$$\begin{aligned} & D_{yb,VRS} = \max_{z_i,\beta} \beta \\ \text{s.t.}\quad & \textstyle\sum_{i=1}^{N} \rho_i y_{li} \geqslant y_{l0} + \beta y_{l0}, l=1,\cdots,L \\ & \textstyle\sum_{i=1}^{N} \rho_i b_{mi} = b_{m0} - \beta b_{m0}, m=1,\cdots,M \\ & \textstyle\sum_{i=1}^{N} (\rho_i+\mu_i) x_{ki} \leqslant x_{k0}, k=1,\cdots,K, \\ & \textstyle\sum_{i=1}^{N} (\rho_i+\mu_i) = 1 \\ & \rho_i, \mu_i \geqslant 0, i=1,\cdots,N \end{aligned} \tag{6.31}$$

混合环境效率（MEE）的估计方式与相同减排因子的双曲距离函数相同。

（二）同时调整期望产出和投入当研究重点是同时扩张期望产出并缩减投入时，双曲距离函数为：

$$G(x,y,b) = \inf\{G : (Gx, y/G, b) \in T\} \tag{6.32}$$

G 是双曲距离函数的值。当 $G=1$ 时，被评价决策单元位于生产前沿上；当 $G<1$ 时，决策单元位于生产前沿内部，存在扩张期望产出和缩减投入的空间。图 6.8 中的产出曲线表示给定投入 x 对应的产出 y；经过 A 点的反比例函数刻画该决策单元同时扩张期望产出、缩减投入的调整路径。双曲距离函数的值 $G = OA/OA'$ 小于 1，表明存在改进空间，可改进至 A' 点。

该模型与“期望产出—非期望产出” 双曲模型的结构相似，区别在于被压缩的对象由非期望产出变为

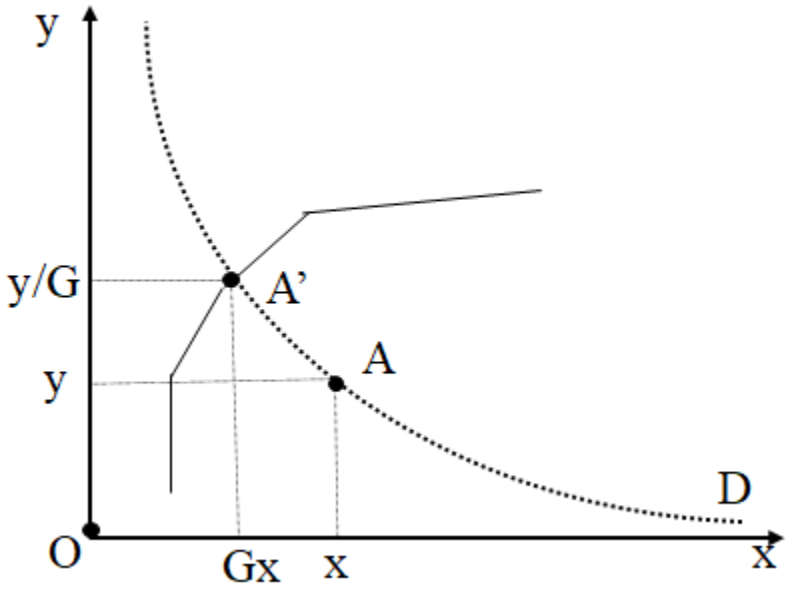


图 6.8: 同时调整期望产出和投入时的生产函数

投入。以CRS 假设为例，相应规划为：

$$\begin{aligned} & \mathrm{D}_{\mathrm{yx,CRS}} = \min \lambda \\ \text{s.t.} \quad & \textstyle\sum_{i=1}^{N} z_i y_{li} \geqslant \frac{y_{l0}}{\lambda}, l = 1, 2, \cdots, L, \\ & \textstyle\sum_{i=1}^{N} z_i b_{mi} = b_{m0}, m = 1, 2, \cdots, M, \\ & \textstyle\sum_{i=1}^{N} z_i x_{ki} \leqslant \lambda x_{k0}, k = 1, 2, \cdots, K, \\ & z_i \geqslant 0, i = 1, 2, \cdots, N \end{aligned} \tag{6.33}$$

式中， $z = (z_1, z_2, \cdots, z_N)$ 为权重向量。

为便于求解，需将该非线性问题转化为线性规划：在每个约束两边同乘 λ，并令 $\lambda^2 = \delta$、$\lambda z_i = z_i'$，可得如下线性规划模型（同式6.27，此处亦存在 δ 与 λ 解耦的退化问题，详见式 6.27 后的说明）：

$$\begin{aligned} & \mathrm{D}_{\mathrm{yx,CRS}} = \min \delta \\ \text{s.t.} \quad & \textstyle\sum_{i=1}^{N} z_i' y_{li} \geqslant y_{l0}, l = 1, 2, \cdots, L, \\ & \textstyle\sum_{i=1}^{N} z_i' b_{mi} = \lambda b_{m0}, m = 1, 2, \cdots, M, \\ & \textstyle\sum_{i=1}^{N} z_i' x_{ki} \leqslant \delta x_{k0}, k = 1, 2, \cdots, K, \\ & z_i' \geqslant 0, 0 < \lambda, \delta \leq 1, i = 1, 2, \cdots, N \end{aligned} \tag{6.34}$$

规模报酬可变（VRS）且减排因子相同时，同时调整期望产出和投入的双曲距离函数可由下列规划求解：

$$\begin{aligned} & \mathrm{D}_{\mathrm{yx,VRS}} = \min \lambda \\ \text{s.t.} \quad & \theta \textstyle\sum_{i=1}^{N} z_i y_{li} \geqslant \frac{y_{l0}}{\lambda}, l = 1, 2, \cdots, L, \\ & \theta \textstyle\sum_{i=1}^{N} z_i b_{mi} = b_{m0}, m = 1, 2, \cdots, M, \\ & \textstyle\sum_{i=1}^{N} z_i x_{ki} \leqslant \lambda x_{k0}, k = 1, 2, \cdots, K, \\ & \textstyle\sum_{i=1}^{N} z_i = 1, \\ & z_i \geqslant 0, 0 < \theta \leq 1, i = 1, 2, \cdots, N \end{aligned} \tag{6.35}$$

同样将 λ 替换为 $e^{-\beta}$ 并按泰勒公式作一阶近似（处理方式同式（6.29）），可将非线性模型转化为线性模型：

$$\begin{aligned} & \mathrm{D}_{\mathrm{yx,VRS}}=\max \beta \\ \text{s.t.}\quad & \theta \sum_{i=1}^{N} z_i y_{li} \geqslant y_{l0}+\beta y_{l0}, l=1,2,\cdots,L, \\ & \theta \sum_{i=1}^{N} z_i b_{mi}=b_{m0}, m=1,2,\cdots,M, \\ & \sum_{i=1}^{N} z_i x_{ki} \leqslant x_{k0}-\beta x_{k0}, k=1,2,\cdots,K, \\ & \sum_{i=1}^{N} z_i=1, \\ & z_i \geqslant 0, 0<\theta \leq 1, i=1,2,\cdots,N \end{aligned} \tag{6.36}$$

该模型不能直接转化为本章采用的线性规划形式，后续计算不采用这一设定。

减排因子不同时的模型为：

$$\begin{aligned} & D_{yx,VRS}=\max_{z_i,\beta} \beta \\ \text{s.t.}\quad & \sum_{i=1}^{N} \rho_i y_{li} \geqslant y_{l0}+\beta y_{l0}, l=1,\cdots,L \\ & \sum_{i=1}^{N} \rho_i b_{mi}=b_{m0}, m=1,\cdots,M \\ & \sum_{i=1}^{N} (\rho_i+\mu_i) x_{ki} \leqslant x_{k0}-\beta x_{k0}, k=1,\cdots,K, \\ & \sum_{i=1}^{N} (\rho_i+\mu_i)=1 \\ & \rho_i, \mu_i \geqslant 0, i=1,\cdots,N \end{aligned} \tag{6.37}$$

使用上述线性规划模型可求解联合改进幅度。若最优值为 β^*，则混合投入效率（MEI）为：

$$MEI=1-\beta^*$$

式中，仅当 $\beta^*=0$ 时，有 $MEI=1$，表示观测点位于相应方向前沿；当 $\beta^*>0$ 时，$MEI<1$，表明仍存在联合增产和节约投入的空间。

6.2.3 方向距离函数

双曲距离函数虽然能同时扩张期望产出并压缩非期望产出，但其扩张与压缩被绑定在互为倒数的同一比例（$1/G$ 与 G）上，无法独立权衡增产与减排的相对强度。方向距离函数（Directional Distance Function, DDF）以方向向量 (g_x, g_y, g_b) 替代固定的倒数比例，使增产、减排与节约投入的相对强度可以独立设定，从而更直接地刻画环境治理中的联合改进目标。需要强调的是，**原始的双曲距离函数并不是DDF 的特例**：双曲沿双曲线 $(y/G, Gb)$ 投影，DDF 沿直线 $(y+\beta g_y, b-\beta g_b)$ 投影，一条是曲线、一条是直线，二者只在改进幅度很小时近似重合。真正成立的是：VRS 下常用的双曲**线性化**形式（式 6.29/6.30，由 $e^{\pm\beta}$ 的一阶泰勒展开得到）与方向向量 $g=(y,b)$ 的DDF 是同一个线性规划，因此二者结果一致，不能互为稳健性对照（见式 6.30 处说明）；CRS 双曲的线性化则发生退化，只能读作坏产出端 Shephard 距离的单调变换（见式 6.27 处说明）。本章将双曲单独列为一类，是因为其原始非线性形式导出了不同的线性化路径与近似处理。方向向量的设定具有实质性含义：若只令非期望产出方向为正，指标反映减排空间；若同时令期望产出和非期望产出方向为正，指标反映增产减排的联合空间；若进一步纳入投入方向，则指标包含投入节约含义。根据 Chung, Färe and Grosskopf (1997)，其一般形式定义为：

$$\vec{D}\left(x,y,b;-g_x,g_{y,}-g_b\right)=\sup\left\{\beta:\left(x-\beta g_x,y+\beta g_y,b-\beta g_b\right)\in T\right\} \tag{6.38}$$

式中，T 为包含非期望产出的生产技术；$g_x=\left(g_{x_1},g_{x_2},\cdots,g_{x_K}\right)'\in R_+^K$，$g_y=\left(g_{y_1},g_{y_2},\cdots,g_{y_L}\right)'\in R_+^L$，$-g_b=\left(-g_{b_1},-g_{b_2},\cdots,-g_{b_M}\right)'\in \mathfrak{R}_+^M$，反映了方向距离函数的投射法则。$\beta$ 是方向距离函数的取值，表示扩张期望产出同时缩减投入和非期望产出的最大比例。

根据Chung, Färe and Grosskopf (1997)，方向距离函数满足以下性质：

(i) 若 $(x,y,b)\in T$，则 $\vec{D}\left(x,y,b;-g_x,g_y,-g_b\right)\geqslant 0$；

(ii) $\vec{D}\left(x-\alpha g_x,y+\alpha g_y,b-\alpha g_b;-g_x,g_y,-g_b\right)=\vec{D}\left(x,y,b;-g_x,g_y,-g_b\right)-\alpha$；

(iii) $\vec{D}\left(x,y,b;-\lambda g_x,\lambda g_y,-\lambda g_b\right)=(1/\lambda)\vec{D}\left(x,y,b;-g_x,g_y,-g_b\right),\lambda>0$；

(iv)规模报酬不变时，投入产出的一次齐次性：$\vec{D}\left(\lambda x,\lambda y,\lambda b;-g_x,g_y,-g_b\right)=\lambda\vec{D}\left(x,y,b;-g_x,g_y,-g_b\right),\lambda>0$；

(v) 如果 $x'\geqslant x$，则 $0\leqslant\vec{D}\left(x,y,b;-g_x,g_y,-g_b\right)\leqslant\vec{D}\left(x',y,b;-g_x,g_y,-g_b\right)$；

(vi) 如果 $y'\geqslant y$，则 $\vec{D}\left(x,y,b;-g_x,g_y,-g_b\right)\geqslant\vec{D}\left(x,y^{'},b;-g_x,g_y,-g_b\right)\geqslant 0$；

(vii) 如果 $b^{'}\geqslant b$，则 $0\leqslant\vec{D}\left(x,y,b;-g_x,g_y,-g_b\right)\leqslant\vec{D}\left(x,y,b^{'};-g_x,g_y,-g_b\right)$；

(viii) 如果 T 是凸的，那么 $\vec{D}\left(x,y,b;-g_x,g_y,-g_b\right)$ 对于 (x,y,b) 是凹的。

其中，性质 (i) 表明方向距离函数取非负值。性质 (ii) 说明，若决策单元沿原调整方向移动方向向量的 α 倍，方向距离函数值减少 α。性质 (iii) 表明，若方向向量变为原来的 λ 倍，方向距离函数变为原来的 $1/\lambda$ 倍。性质 (iv) 表明，若投入、期望产出和非期望产出同比例变为原来的 λ 倍，方向距离函数亦变为原来的 λ 倍。性质 (v)、性质 (vi) 和性质(vii) 表明投入增大时，方向距离函数增加；期望产出增大时，方向距离函数值减小；非期望产出增大时，方向距离函数值增加。这三个性质均反映了方向距离函数值随无效程度提高而变大。性质(viii) 表明生产技术为凸集时，方向距离函数是凹函数。

规模报酬不变（CRS）假设下的方向距离函数可通过下式求解：

$$\begin{aligned}&\vec{D}^r\left(x,y,b;-g_x,g_y,-g_b\right)=\max_{z_i,\beta}\beta\\ \text{s.t.}\quad&\textstyle\sum_{i=1}^N z_i y_{li}\geqslant y_{l0}+\beta g_{ly},l=1,\cdots,L\\&\textstyle\sum_{i=1}^N z_i b_{mi}=b_{m0}-\beta g_{mb},m=1,\cdots,M\\&\textstyle\sum_{i=1}^N z_i x_{ki}\leqslant x_{k0}-\beta g_{kx},k=1,\cdots,K\\&z_i\geqslant 0,i=1,\cdots,N\end{aligned}\tag{6.39}$$

VRS 下区分减排因子相同和不同两种情形：

（1）减排因子相同时的方向距离函数模型可表示为：

$$\begin{aligned}&\vec{D}^r\left(x,y,b;g_y,-g_b\right)=\max_{z_i,\beta}\beta\\ \text{s.t.}\quad&\theta\textstyle\sum_{i=1}^N z_i y_{li}\geqslant y_{l0}+\beta g_{ly},l=1,\cdots,L\\&\theta\textstyle\sum_{i=1}^N z_i b_{mi}=b_{m0}-\beta g_{mb},m=1,\cdots,M\\&\textstyle\sum_{i=1}^N z_i x_{ki}\leqslant x_{k0}-\beta g_{kx},k=1,\cdots,K,\\&\textstyle\sum_{i=1}^N z_i=1\\&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N\end{aligned}\tag{6.40}$$

其等价形式为（令 $\beta'=\theta\beta$，即投入方向的调整幅度经减排因子 θ 折算后与产出方向的 β 对应）：

$$\begin{aligned}&\vec{D}^r\left(x,y,b;-g_x,g_y,-g_b\right)=\max_{z_i,\beta,\theta}\beta'\\ \text{s.t.}\quad&\textstyle\sum_{i=1}^N z_i y_{li}\geqslant y_{l0}+\beta g_{ly},l=1,\cdots,L\\&\textstyle\sum_{i=1}^N z_i b_{mi}=b_{m0}-\beta g_{mb},m=1,\cdots,M\\&\textstyle\sum_{i=1}^N z_i x_{ki}\leqslant\theta x_{k0}-\beta' g_{kx},k=1,\cdots,K,\\&\textstyle\sum_{i=1}^N z_i=\theta\\&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N\end{aligned}\tag{6.41}$$

关于式 (6.41) 的弃用说明：式 (6.41) 通过 $\beta' = \theta\beta$ 的代入，把含投入方向的全方向 DDF 写成线性形式，但这一替换**丢弃了双线性链接** $\beta' = \theta\beta$，且目标从原模型的 $\max \beta$ 改为 $\max \beta' = \theta\beta$；由于 $\theta \in (0, 1]$，$\max \beta' \neq \max \beta$，因此式 (6.41) 与原始 DDF 定义并不等价。这与 6.2.3 节（二）式 (6.52) "投入方向同因子 DDF 不能直接线性化" 是同一类问题——只要投入约束中含效率变量（βg_{kx}），同因子下的 $z_i \to z_i/\theta$ 重标定就会留下 $\theta\beta$ 双线性残差。因此式(6.41) **仅作形式对照列出，本章后续不再采用：**同因子 DDF 的求解仅用于投入方向不参与调整的 y–b 方向（式 6.47）和 b 方向（式 6.58），这两种情形下投入约束不含 β，线性化严格成立。`DEAweak` 在代码层面也对同因子+投入方向的组合显式拒绝。

（2）减排因子不同时的方向距离函数模型可表示为：

$$
\begin{aligned}
& \vec{D}_o^r\left(x, y, b; g_y, -g_b\right) = \max_{z_i, \beta} \beta \\
\text{s.t.} \quad & \textstyle\sum_{i=1}^N \theta_i z_i y_{li} \geqslant y_{l0} + \beta g_{ly}, l = 1, \cdots, L \\
& \textstyle\sum_{i=1}^N \theta_i z_i b_{mi} = b_{m0} - \beta g_{mb}, m = 1, \cdots, M \\
& \textstyle\sum_{i=1}^N z_i x_{ki} \leqslant x_{k0} - \beta g_{kx}, k = 1, \cdots, K, \\
& \textstyle\sum_{i=1}^N z_i = 1 \\
& 0 \leqslant \theta_i \leqslant 1, z_i \geqslant 0, i = 1, \cdots, N
\end{aligned}
\tag{6.42}
$$

做与式 (6.12) 相同的替换 $\theta_i z_i = \rho_i,\ z_i = \rho_i + \mu_i$，得等价线性规划：

$$
\begin{aligned}
& \vec{D}_o^r\left(x, y, b; g_y, -g_b\right) = \max_{\rho_i, \mu_i, \beta} \beta \\
\text{s.t.} \quad & \textstyle\sum_{i=1}^N \rho_i y_{li} \geqslant y_{l0} + \beta g_{ly}, l = 1, \cdots, L \\
& \textstyle\sum_{i=1}^N \rho_i b_{mi} = b_{m0} - \beta g_{mb}, m = 1, \cdots, M \\
& \textstyle\sum_{i=1}^N (\rho_i + \mu_i) x_{ki} \leqslant x_{k0} - \beta g_{kx}, k = 1, \cdots, K, \\
& \textstyle\sum_{i=1}^N (\rho_i + \mu_i) = 1 \\
& \rho_i, \mu_i \geqslant 0, i = 1, \cdots, N
\end{aligned}
\tag{6.43}
$$

设模型最优解为 β^*。根据评价目标不同，可以分别构造投入、期望产出和非期望产出端的绩效指数，也可以进一步构造综合绩效指数。

投入的绩效指数可表示为：

$$EI = \frac{x - \beta^* x}{x} = 1 - \beta^*$$

期望产出的绩效指数可表示为：

$$EO = \frac{y}{y + \beta^* y} = \frac{1}{1 + \beta^*}$$

非期望产出的绩效指数可表示为：

$$EB = \frac{b - \beta^* b}{b} = 1 - \beta^*$$

还可将最优非期望产出强度与实际非期望产出强度之比定义为环境绩效指数：

$$EEI = \frac{1 - \beta^*}{1 + \beta^*}$$

（一）期望产出和非期望产出方向的方向距离函数方向向量的取值可以任意选择。当将方向向量调整为 $(-g_x, g_y, -g_b) = (0, y, -b)$ 时，得到期望产出和非期望产出同时调整的方向距离函数，定义如下（Chung, Färe and Grosskopf, 1997）：

$$\vec{D}\left(x, y, b; 0, g_y, -g_b,\right) = \sup\left\{\beta : \left(x, y + \beta g_y, b - \beta g_b\right) \in T\right\} \tag{6.44}$$

式中，T 为包含非期望产出的生产技术；$g_y=\left(g_{y_1},g_{y_2},\cdots,g_{y_L}\right)'\in R_+^L$，$-g_b=\left(-g_{b_1},-g_{b_2},\cdots,-g_{b_M}\right)'\in R_+^M$，反映了方向距离函数的投射法则。$\beta$ 是方向距离函数的取值，表示同时扩张期望产出和缩减非期望产出的最大比例。

以图 6.9 所示的单期望产出和单非期望产出技术为例，当取方向向量 $(-g_x,g_y,-g_b)=(0,1,-1)$ 时，M 点将沿 $(0,1,-1)$ 的方向投影至 F 点。

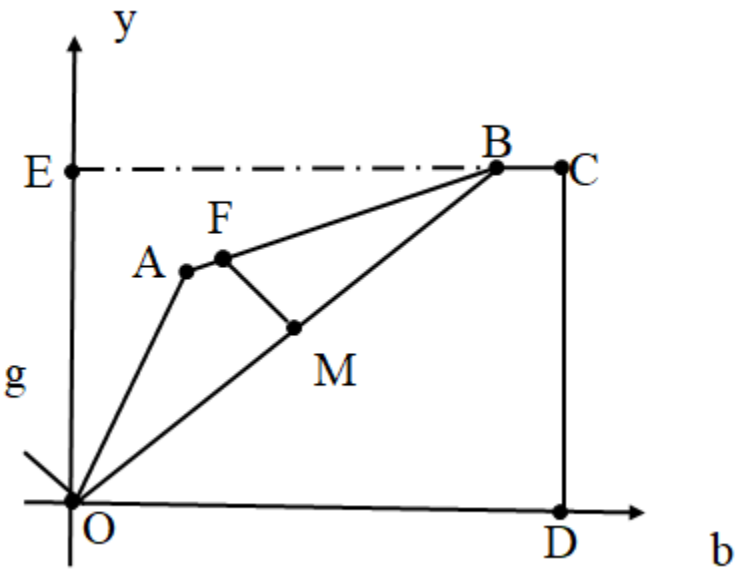


图 6.9: 单期望产出和单非期望产出技术的方向距离函数

规模报酬不变（CRS）假设下的期望产出和非期望产出方向上的方向距离函数可通过下式求解：

$$\begin{aligned}
&\vec{D}_o^r\left(x,y,b;0,g_y,-g_b\right)=\max_{z_i,\beta}\beta\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^N z_i y_{li}\geqslant y_{l0}+\beta g_{ly},l=1,\cdots,L\\
&\textstyle\sum_{i=1}^N z_i b_{mi}=b_{m0}-\beta g_{mb},m=1,\cdots,M\\
&\textstyle\sum_{i=1}^N z_i x_{ki}\leqslant x_{k0},k=1,\cdots,K\\
&z_i\geqslant 0,i=1,\cdots,N
\end{aligned}\tag{6.45}$$

VRS 下区分减排因子相同和不同两种情形：

（1）减排因子相同时的方向距离函数模型可表示为：

$$\begin{aligned}
&\vec{D}_o^r\left(x,y,b;0,g_y,-g_b\right)=\max_{z_i,\beta}\beta\\
\text{s.t.}\quad&\theta\textstyle\sum_{i=1}^N z_i y_{li}\geqslant y_{l0}+\beta g_{ly},l=1,\cdots,L\\
&\theta\textstyle\sum_{i=1}^N z_i b_{mi}=b_{m0}-\beta g_{mb},m=1,\cdots,M\\
&\textstyle\sum_{i=1}^N z_i x_{ki}\leqslant x_{k0},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^N z_i=1\\
&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N
\end{aligned}\tag{6.46}$$

其等价形式为：

$$\begin{aligned} &\vec{D}_o^r\left(x,y,b;0,g_y,-g_b\right)=\max_{z_i,\beta}\beta \\ \text{s.t.}\quad &\textstyle\sum_{i=1}^N z_i y_{li}\geqslant y_{l0}+\beta g_{ly}, l=1,\cdots,L \\ &\textstyle\sum_{i=1}^N z_i b_{mi}=b_{m0}-\beta g_{mb}, m=1,\cdots,M \\ &\textstyle\sum_{i=1}^N z_i x_{ki}\leqslant \theta x_{k0}, k=1,\cdots,K, \\ &\textstyle\sum_{i=1}^N z_i=\theta \\ &0\leqslant\theta\leqslant 1, z_i\geqslant 0, i=1,\cdots,N \end{aligned} \tag{6.47}$$

（2）减排因子不同时的方向距离函数模型可表示为：

$$\begin{aligned} &\vec{D}_o^r\left(x,y,b;0,g_y,-g_b\right)=\max_{z_i,\beta}\beta \\ \text{s.t.}\quad &\textstyle\sum_{i=1}^N \theta_i z_i y_{li}\geqslant y_{l0}+\beta g_{ly}, l=1,\cdots,L \\ &\textstyle\sum_{i=1}^N \theta_i z_i b_{mi}=b_{m0}-\beta g_{mb}, m=1,\cdots,M \\ &\textstyle\sum_{i=1}^N z_i x_{ki}\leqslant x_{k0}, k=1,\cdots,K, \\ &\textstyle\sum_{i=1}^N z_i=1 \\ &0\leqslant\theta_i\leqslant 1, z_i\geqslant 0, i=1,\cdots,N \end{aligned} \tag{6.48}$$

做与式 (6.12) 相同的替换 $\theta_i z_i=\rho_i,\ z_i=\rho_i+\mu_i$，得等价线性规划：

$$\begin{aligned} &\vec{D}_o^r\left(x,y,b;0,g_y,-g_b\right)=\max_{z_i,\beta}\beta \\ \text{s.t.}\quad &\textstyle\sum_{i=1}^N \rho_i y_{li}\geqslant y_{l0}+\beta g_{ly}, l=1,\cdots,L \\ &\textstyle\sum_{i=1}^N \rho_i b_{mi}=b_{m0}-\beta g_{mb}, m=1,\cdots,M \\ &\textstyle\sum_{i=1}^N (\rho_i+\mu_i) x_{ki}\leqslant x_{k0}, k=1,\cdots,K, \\ &\textstyle\sum_{i=1}^N (\rho_i+\mu_i)=1 \\ &\rho_i,\mu_i\geqslant 0, i=1,\cdots,N \end{aligned} \tag{6.49}$$

期望产出的绩效指数可表示为：

$$EO=\frac{y}{y+\beta^* y}=\frac{1}{1+\beta^*}$$

非期望产出的绩效指数可表示为：

$$EB=\frac{b-\beta^* b}{b}=1-\beta^*$$

还可将最优非期望产出强度与实际非期望产出强度之比定义为环境绩效指数：

$$EEI=\frac{1-\beta^*}{1+\beta^*}$$

（二）投入方向的方向距离函数前一情形同时调整期望产出和非期望产出；若只想识别投入端的压缩空间而不改变产出和排放，则需把方向向量限定在投入侧。当将方向向量调整为 $(-g_x,g_y,-g_b)=(x,0,0)$ 时，得到投入方向的方向距离函数，定义如下（Chung, Färe and Grosskopf, 1997）：

$$\vec{D}_i\left(x,y,b;-g_x,0,0\right)=\sup\left\{\beta:\left(x-\beta g_x,y,b\right)\in T\right\} \tag{6.50}$$

式中，T 为包含非期望产出的生产技术；$g_x=\left(g_{x_1},g_{x_2},\cdots,g_{x_K}\right)'\in R_+^K$，反映了方向距离函数的投射法则。$\beta$ 是方向距离函数的取值，表示缩减投入的最大比例。

规模报酬不变(CRS)假设下的投入方向上的方向距离函数可通过下式求解：

$$\begin{aligned} & \vec{D}_i^r\left(x, y, b ; -g_x, 0, 0\right)=\max_{z_i, \beta} \beta \\ \text{s.t.} \quad & \sum_{i=1}^N z_i y_{li} \geqslant y_{l0}, l=1, \cdots, L \\ & \sum_{i=1}^N z_i b_{mi}=b_{m0}, m=1, \cdots, M \\ & \sum_{i=1}^N z_i x_{ki} \leqslant x_{k0}-\beta g_{kx}, k=1, \cdots, K \\ & z_i \geqslant 0, i=1, \cdots, N \end{aligned} \tag{6.51}$$

VRS 下区分减排因子相同和不同两种情形：

(1)减排因子相同时的方向距离函数模型可表示为：

$$\begin{aligned} & \vec{D}_i^r\left(x, y, b ; -g_x, 0, 0\right)=\max_{z_i, \beta} \beta \\ \text{s.t.} \quad & \theta \sum_{i=1}^N z_i y_{li} \geqslant y_{l0}, l=1, \cdots, L \\ & \theta \sum_{i=1}^N z_i b_{mi}=b_{m0}, m=1, \cdots, M \\ & \sum_{i=1}^N z_i x_{ki} \leqslant x_{k0}-\beta g_{kx}, k=1, \cdots, K, \\ & \sum_{i=1}^N z_i=1 \\ & 0 \leqslant \theta \leqslant 1, z_i \geqslant 0, i=1, \cdots, N \end{aligned} \tag{6.52}$$

同理(见式 6.15 处说明)，该模型不能直接线性化，后续计算不采用。

(2)减排因子不同时的方向距离函数模型可表示为：

$$\begin{aligned} & \vec{D}_i^r\left(x, y, b ; -g_x, 0, 0\right)=\max_{z_i, \beta} \beta \\ \text{s.t.} \quad & \sum_{i=1}^N \theta_i z_i y_{li} \geqslant y_{l0}, l=1, \cdots, L \\ & \sum_{i=1}^N \theta_i z_i b_{mi}=b_{m0}, m=1, \cdots, M \\ & \sum_{i=1}^N z_i x_{ki} \leqslant x_{k0}-\beta g_{kx}, k=1, \cdots, K, \\ & \sum_{i=1}^N z_i=1 \\ & 0 \leqslant \theta_i \leqslant 1, z_i \geqslant 0, i=1, \cdots, N \end{aligned} \tag{6.53}$$

做与式 (6.12) 相同的替换 $\theta_i z_i=\rho_i,\ z_i=\rho_i+\mu_i$，得等价线性规划：

$$\begin{aligned} & \vec{D}_i^r\left(x, y, b ; -g_x, 0, 0\right)=\max_{z_i, \beta} \beta \\ \text{s.t.} \quad & \sum_{i=1}^N \rho_i y_{li} \geqslant y_{l0}, l=1, \cdots, L \\ & \sum_{i=1}^N \rho_i b_{mi}=b_{m0}, m=1, \cdots, M \\ & \sum_{i=1}^N (\rho_i+\mu_i) x_{ki} \leqslant x_{k0}-\beta g_{kx}, k=1, \cdots, K, \\ & \sum_{i=1}^N (\rho_i+\mu_i)=1 \\ & \rho_i, \mu_i \geqslant 0, i=1, \cdots, N \end{aligned} \tag{6.54}$$

投入的绩效指数可表示为：

$$EI=\frac{x-\beta^* x}{x}=1-\beta^*$$

(三)非期望产出方向的方向距离函数除期望产出与非期望产出同时调整的方向距离函数外，也可仅沿非期望产出方向定义方向距离函数：

$$\vec{D}_b\left(x, y, b ; 0, 0, -g_b\right)=\sup\left\{\beta:\left(x, y, b-\beta g_b\right) \in T\right\} \tag{6.55}$$

式中，T 为包含非期望产出的生产技术；$-g_b=\left(-g_{b_1},-g_{b_2},\cdots,-g_{b_M}\right)'\in\mathfrak{R}_+^M$，反映了方向距离函数的投射法则。$\beta$ 是方向距离函数的取值，表示缩减非期望产出的最大比例。

规模报酬不变（CRS）假设下的非期望产出方向上的方向距离函数可通过下式求解：

$$\begin{aligned}&\vec{D}_b^r\left(x,y,b;0,0,-g_b\right)=\max_{z_i,\beta}\beta\\ \text{s.t.}\quad&\sum_{i=1}^N z_i y_{li}\geqslant y_{l0},l=1,\cdots,L\\&\sum_{i=1}^N z_i b_{mi}=b_{m0}-\beta g_{mb},m=1,\cdots,M\\&\sum_{i=1}^N z_i x_{ki}\leqslant x_{k0},k=1,\cdots,K\\&z_i\geqslant 0,i=1,\cdots,N\end{aligned}\tag{6.56}$$

VRS 下区分减排因子相同和不同两种情形：

（1）减排因子相同时的方向距离函数模型可表示为：

$$\begin{aligned}&\vec{D}_b^r\left(x,y,b;0,0,-g_b\right)=\max_{z_i,\beta}\beta\\ \text{s.t.}\quad&\theta\sum_{i=1}^N z_i y_{li}\geqslant y_{l0},l=1,\cdots,L\\&\theta\sum_{i=1}^N z_i b_{mi}=b_{m0}-\beta g_{mb},m=1,\cdots,M\\&\sum_{i=1}^N z_i x_{ki}\leqslant x_{k0},k=1,\cdots,K,\\&\sum_{i=1}^N z_i=1\\&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N\end{aligned}\tag{6.57}$$

其等价形式为：

$$\begin{aligned}&\vec{D}_b^r\left(x,y,b;0,0,-g_b\right)=\max_{z_i,\beta}\beta\\ \text{s.t.}\quad&\sum_{i=1}^N z_i y_{li}\geqslant y_{l0},l=1,\cdots,L\\&\sum_{i=1}^N z_i b_{mi}=b_{m0}-\beta g_{mb},m=1,\cdots,M\\&\sum_{i=1}^N z_i x_{ki}\leqslant\theta x_{k0},k=1,\cdots,K,\\&\sum_{i=1}^N z_i=\theta\\&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N\end{aligned}\tag{6.58}$$

（2）经同样的转换，减排因子不同时的方向距离函数模型可表示为：

$$\begin{aligned}&\vec{D}_b^r\left(x,y,b;0,0,-g_b\right)=\max_{z_i,\beta}\beta\\ \text{s.t.}\quad&\sum_{i=1}^N \rho_i y_{li}\geqslant y_{l0},l=1,\cdots,L\\&\sum_{i=1}^N \rho_i b_{mi}=b_{m0}-\beta g_{mb},m=1,\cdots,M\\&\sum_{i=1}^N(\rho_i+\mu_i)x_{ki}\leqslant x_{k0},k=1,\cdots,K,\\&\sum_{i=1}^N(\rho_i+\mu_i)=1\\&\rho_i,\mu_i\geqslant 0,i=1,\cdots,N\end{aligned}\tag{6.59}$$

非期望产出的绩效指数可表示为：

$$EB=\frac{b-\beta^* b}{b}=1-\beta^*$$

6.2.4 非径向方向距离函数

标准方向距离函数对同一方向上的变量施加统一调整比例，因此仍带有径向测度特征。非径向方向距离函数（Non-radial Directional Distance Function, NDDF）允许不同变量具有不同调整幅度，能够把径向调整

之外的松弛信息纳入环境绩效测度。当投入、期望产出或非期望产出存在非零松弛时，径向效率可能高估决策单元的真实绩效（Fukuyama and Weber, 2009）。其代价是输出指标维度增多，结果解释必须逐一说明投入端、期望产出端和非期望产出端的含义。一般形式可定义为：

$$\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)=\max\left\{\omega'\beta:(x,y,b)+\left(-g_x,g_y,-g_b\right)\cdot\mathrm{diag}(\beta)\in T\right\} \tag{6.60}$$

式中，$\omega=\left(\omega_x,\omega_y,\omega_b\right)'=\left(\omega_{1x},\omega_{2x},\cdots,\omega_{Kx},\omega_{1y},\omega_{2y},\cdots,\omega_{Ly},\omega_{1b},\omega_{2b},\cdots,\omega_{Mb}\right)'$ 为权重向量，且满足 $\sum_{k=1}^{K}\omega_{kx}+\sum_{l=1}^{L}\omega_{ly}+\sum_{m=1}^{M}\omega_{mb}=1$；$\left(-g_x,g_y,-g_b\right)=\left((-g_{1x},-g_{2x},\cdots,-g_{Kx})',(g_{1y},g_{2y},\cdots,g_{Ly})',(-g_{1b},-g_{2b},\cdots\right.$ 为方向向量；$\beta=\left(\beta_x,\beta_y,\beta_b\right)'=\left(\beta_{1x},\beta_{2x},\cdots,\beta_{Kx},\beta_{1y},\beta_{2y},\cdots,\beta_{Ly},\beta_{1b},\beta_{2b},\cdots,\beta_{Mb}\right)'\geqslant 0$ 为各投入、期望产出和非期望产出在给定方向向量下的调整幅度。

非径向方向距离函数满足以下性质（Zhou, Ang and Wang, 2012）：

(i) 若 $(x,y,b)\in T$，则 $\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)\geqslant 0$；

(ii) 平移性质：$\vec{D}^{nr}\left(x-\alpha g_x,y+\alpha g_y,b-\alpha g_b;-g_x,g_y,-g_b\right)=\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)-\alpha$；(iii) 方向向量的负一次齐次性：$\vec{D}^{nr}\left(x,y,b;-g_x,\lambda g_y,-\lambda g_b\right)=(1/\lambda)\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right),\lambda>0$；

(iv)规模报酬不变时，投入产出的一次齐次性：$\vec{D}^{nr}\left(\lambda x,\lambda y,\lambda b;-g_x,g_y,-g_b\right)=\lambda\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right),\lambda>0$；(v) 如果 $y'\geqslant y$，则 $\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)\geqslant\vec{D}^{nr}\left(x,y',b;-g_x,g_y,-g_b\right)\geqslant 0$；

(vi) 如果 $b'\geqslant b$，则 $0\leqslant\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)\leqslant\vec{D}^{nr}\left(x,y,b';-g_x,g_y,-g_b\right)$；

(vii) 如果 $x'\geqslant x$，则 $0\leqslant\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)\leqslant\vec{D}^{nr}\left(x',y,b;-g_x,g_y,-g_b\right)$；

(viii) 如果 T 是凸的，那么 $\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)$ 对于 (x,y,b) 是凹的。

在 CRS 假设下，非径向方向距离函数可通过下式求解：

$$\begin{aligned}
&\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)=\max\textstyle\sum_{k=1}^{K}\omega_{kx}\beta_{kx}+\sum_{l=1}^{L}\omega_{ly}\beta_{ly}+\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}z_iy_{li}\geqslant y_{l0}+\beta_{ly}g_{ly},l=1,\cdots,L\\
&\textstyle\sum_{i=1}^{N}z_ib_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant x_{k0}-\beta_{kx}g_{kx},k=1,\cdots,K,\\
&z_i\geqslant 0,i=1,\cdots,N,\beta_{kx},\beta_{ly},\beta_{mb}\geqslant 0
\end{aligned} \tag{6.61}$$

VRS 下区分减排因子相同和不同两种情形：

（1）减排因子相同时的非径向方向距离函数模型可表示为：

$$\begin{aligned}
&\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)=\max\textstyle\sum_{k=1}^{K}\omega_{kx}\beta_{kx}+\sum_{l=1}^{L}\omega_{ly}\beta_{ly}+\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\theta\textstyle\sum_{i=1}^{N}z_iy_{li}\geqslant y_{l0}+\beta_{ly}g_{ly},l=1,\cdots,L,\\
&\theta\textstyle\sum_{i=1}^{N}z_ib_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant x_{k0}-\beta_{kx}g_{kx},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}z_i=1,\\
&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N,\beta_{kx},\beta_{ly},\beta_{mb}\geqslant 0
\end{aligned} \tag{6.62}$$

线性化后的模型为（令 $\beta'_{kx}=\theta\beta_{kx}$，即投入方向调整幅度经减排因子折算）：

$$\begin{aligned}
&\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)=\max\textstyle\sum_{k=1}^{K}\omega_{kx}\beta'_{kx}+\sum_{l=1}^{L}\omega_{ly}\beta_{ly}+\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}z_i y_{li}\geqslant y_{l0}+\beta_{ly}g_{ly},l=1,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}z_i b_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_i x_{ki}\leqslant\theta x_{k0}-\beta'_{kx}g_{kx},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}z_i=\theta,\\
&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N,\beta_{ly},\beta_{mb}\geqslant 0,\ \beta'_{kx}\geqslant 0
\end{aligned}\tag{6.63}$$

> **关于式 (6.63) 的弃用说明：**与式 (6.41) 同理，式 (6.63) 通过 $\beta'_{kx}=\theta\beta_{kx}$ 把含投入方向的同因子 NDDF 写成线性形式，但同样丢失了双线性链接，目标函数最大化的是 θ 加权后的 β'_{kx} 而非原始的 β_{kx}，因此与原 NDDF 定义不等价。本章后续不再采用式 (6.63)；同因子 NDDF 的求解仅用于投入方向不参与调整的 *y*–*b* 方向（式 6.69）和 *b* 方向（式 6.79），这两种情形下线性化严格成立。原式中残留的 $\beta_{kx}\geqslant 0$ 声明（实际不起作用）已替换为 $\beta'_{kx}\geqslant 0$。

（2）减排因子不同时的非径向方向距离函数模型可表示为：

$$\begin{aligned}
&\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)=\max\textstyle\sum_{k=1}^{K}\omega_{kx}\beta_{kx}+\sum_{l=1}^{L}\omega_{ly}\beta_{ly}+\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}\theta_i z_i y_{li}\geqslant y_{l0}+\beta_{ly}g_{ly},l=1,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}\theta_i z_i b_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_i x_{ki}\leqslant x_{k0}-\beta_{kx}g_{kx},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}z_i=1,\\
&0\leqslant\theta_i\leqslant 1,z_i\geqslant 0,i=1,\cdots,N,\beta_{kx},\beta_{ly},\beta_{mb}\geqslant 0
\end{aligned}\tag{6.64}$$

做与式 (6.12) 相同的替换 $\theta_i z_i=\rho_i,\ z_i=\rho_i+\mu_i$，得等价线性规划：

$$\begin{aligned}
&\vec{D}^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)=\max\textstyle\sum_{k=1}^{K}\omega_{kx}\beta_{kx}+\sum_{l=1}^{L}\omega_{ly}\beta_{ly}+\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}\rho_i y_{li}\geqslant y_{l0}+\beta_{ly}g_{ly},l=1,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}\rho_i b_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}(\rho_i+\mu_i)x_{ki}\leqslant x_{k0}-\beta_{kx}g_{kx},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}(\rho_i+\mu_i)=1,\\
&\rho_i,\mu_i\geqslant 0,i=1,\cdots,N,\beta_{kx},\beta_{ly},\beta_{mb}\geqslant 0
\end{aligned}\tag{6.65}$$

设模型最优解为 β^*_{kx}、β^*_{ly} 和 β^*_{mb}。各端口绩效指数可由前沿值与观测值的比值计算，也可进一步汇总为综合绩效指数。

关于 NDDF 效率指数的量纲与上界：NDDF 的效率指数取决于权重向量 ω 的归一化方式。本章案例在 *y*–*b* 方向下取 $\omega_y=\omega_b=1/2$（对应代码默认权重），目标函数为 $\frac{1}{2}\beta_{ly}+\frac{1}{2}\beta_{mb}$。需特别说明的是：前沿（有效）单元的 $\beta^*_{ly}=\beta^*_{mb}=0$（无法继续扩张 y 或压缩 b），其单变量效率 $teY_l=1/(1+0)=1$、$teB_m=1-0=1$。但由代码返回的变量级缩放因子 `beta` 在调用处加权汇总得到的 `EO`、`EB` 是**加权聚合**后的值，即 $EO=\sum_l\omega_{ly}\cdot teY_l$、$EB=\sum_m\omega_{mb}\cdot teB_m$；因此即便前沿单元，加权后 $EO=\omega_y\times 1=0.5$、$EB=\omega_b\times 1=0.5$。即 0.5 上界**来自权重** $\omega=1/2$**，而非调整幅度** β^*。这正是 6.4 节 表 6.4–6.16 中前沿单元（如Australia、Canada 全部年份，Switzerland 2018–2019 年的 `EO`）取0.5 的原因，与 DDF 前沿单元 $EO\to 1$ 的归一化不同。因此 NDDF 的 EO、EB 与 DDF 的同名指标**量纲不可直接横向等同**，应仅在排序和前沿识别层面比较。

此外，本章将调用处据 beta 计算的 EBO = EB/EO 定义为**两端加权效率的比值**（相对排放强度）而非 Farrell 效率，因而其上界不限于1：当某单元坏产出端的加权效率高于期望产出端时，EBO >1（如 6.4 节 表 6.4 中 Greece、Portugal、Slovenia 部分年份）。读者应将 EBO >1 解读为" 该单元在 b 端的相对效率高于 y 端"，而非" 超效率"。注意 EBO 与下文式 (6.71) 后定义的 EEI $=(1-\beta^*_{mb})/(1+\beta^*_{ly})$ 数学形式不同：前者是两个加权效率之比，后者是关于 β^* 的非线性分式；两者均反映相对排放强度，但量纲不同，不应混淆。

投入的单变量绩效指数可表示为：

$$teX_k = \frac{x_k - \beta^*_{kx} x_k}{x_k} = 1 - \beta^*_{kx}$$

期望产出的单变量绩效指数可表示为：

$$teY_l = \frac{y_l}{y_l + \beta^*_{ly} y_l} = \frac{1}{1+\beta^*_{ly}}$$

非期望产出的单变量绩效指数可表示为：

$$teB_m = \frac{b_m - \beta^*_{mb} b_m}{b_m} = 1 - \beta^*_{mb}$$

上述 teX_k、teY_l、teB_m 为**单变量效率**，由代码返回的变量级缩放因子 beta 在调用处换算得到（$teY_l = 1/(1+\beta^*_{ly})$、$teB_m = 1-\beta^*_{mb}$，对应代码列 beta_Y、beta_CO2），即未经加权的端口效率。本章案例报告的聚合指标 EI、EO、EB 是在此基础上按权重 ω 加权汇总：

$$EI = \sum_{k=1}^{K} \omega_{kx} \cdot teX_k, \qquad EO = \sum_{l=1}^{L} \omega_{ly} \cdot teY_l, \qquad EB = \sum_{m=1}^{M} \omega_{mb} \cdot teB_m$$

由调用处据 beta 与权重 ω 汇总得到（对应 EB、EO）。当 $L = M = 1$ 且 $\omega_{ly} = \omega_{mb} = 1/2$ 时（本章案例设定），$EO = \frac{1}{2} \cdot \frac{1}{1+\beta^*_{ly}}$、$EB = \frac{1}{2}(1-\beta^*_{mb})$，故前沿单元（$\beta^* = 0$）的 $EO = EB = 0.5$，详见前文量纲说明。根据 Zhang and Choi (2013)，综合绩效指数可定义为最优投入产出比相对于实际投入产出比、以及最优排放强度相对于实际排放强度的平均值，即：

$$UEI = \frac{1}{(K+M)} \left[\sum_{k=1}^{K} \frac{x_k - \beta^*_{kx} x_k}{y_l + \beta^*_{ly} y_l} \Big/ \frac{x_k}{y_l} + \sum_{m=1}^{M} \frac{b_m - \beta^*_{mb} b_m}{y_l + \beta^*_{ly} y_l} \Big/ \frac{b_m}{y_l} \right] = \frac{1}{(K+M)} \left[\sum_{k=1}^{K} \frac{1-\beta^*_{kx}}{1+\beta^*_{ly}} + \sum_{m=1}^{M} \frac{1-\beta^*_{mb}}{1+\beta^*_{ly}} \right]$$

（一）期望产出和非期望产出方向的非径向方向距离函数方向向量的取值可以任意选择。当将方向向量调整为 $(-g_x, g_y, -g_b) = (0, y, -b)$ 时，即得到期望产出和非期望产出方向的非径向方向距离函数：

$$\vec{D}^{nr}_o\left(x, y, b; 0, g_y, -g_b\right) = \max\left\{\omega' \beta : (x, y, b) + \left(0, g_y, -g_b\right) \cdot \mathrm{diag}(\beta) \in T\right\} \tag{6.66}$$

式中，$\omega = \left(\omega_y, \omega_b\right)' = \left(\omega_{1y}, \omega_{2y}, \cdots, \omega_{L_y}, \omega_{1b}, \omega_{2b}, \cdots, \omega_{M_b}\right)'$ 为权重向量，且满足 $\sum_{l=1}^{L} \omega_{ly} + \sum_{m=1}^{M} \omega_{mb} = 1$；$\left(g_y, -g_b\right) = \left(\left(g_{1y}, g_{2y}, \cdots, g_{Ly}\right)', (-g_{1b}, -g_{2b}, \cdots, -g_{Mb})'\right)$ 为方向向量；$\beta = \left(\beta_y, \beta_b\right)' = \left(\beta_{1y}, \beta_{2y}, \cdots, \beta_{Ly}, \beta_{1b}, \beta_{2b}, \cdots, \beta_{Mb}\right)' \geqslant 0$ 为各期望产出和非期望产出在给定方向向量下的调整幅度。

在 CRS 假设下，期望产出和非期望产出方向的非径向方向距离函数可通过下式求解：

$$\begin{aligned}
&\vec{D}_o^{nr}\left(x,y,b;0,g_y,-g_b\right)=\max\textstyle\sum_{l=1}^{L}\omega_{ly}\beta_{ly}+\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}z_iy_{li}\geqslant y_{l0}+\beta_{ly}g_{ly},l=1,\cdots,L\\
&\textstyle\sum_{i=1}^{N}z_ib_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant x_{k0},k=1,\cdots,K,\\
&z_i\geqslant 0,i=1,\cdots,N,\beta_{ly},\beta_{mb}\geqslant 0
\end{aligned}\tag{6.67}$$

VRS 下区分减排因子相同和不同两种情形：

（1）减排因子相同时的非径向方向距离函数模型可表示为：

$$\begin{aligned}
&\vec{D}_o^{nr}\left(x,y,b;0,g_y,-g_b\right)=\max\textstyle\sum_{l=1}^{L}\omega_{ly}\beta_{ly}+\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\theta\textstyle\sum_{i=1}^{N}z_iy_{li}\geqslant y_{l0}+\beta_{ly}g_{ly},l=1,\cdots,L,\\
&\theta\textstyle\sum_{i=1}^{N}z_ib_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant x_{k0},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}z_i=1,\\
&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N,\beta_{ly},\beta_{mb}\geqslant 0
\end{aligned}\tag{6.68}$$

线性化后的模型为：

$$\begin{aligned}
&\vec{D}_o^{nr}\left(x,y,b;0,g_y,-g_b\right)=\max\textstyle\sum_{l=1}^{L}\omega_{ly}\beta_{ly}+\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}z_iy_{li}\geqslant y_{l0}+\beta_{ly}g_{ly},l=1,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}z_ib_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant\theta x_{k0},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}z_i=\theta,\\
&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N,\beta_{ly},\beta_{mb}\geqslant 0
\end{aligned}\tag{6.69}$$

（2）减排因子不同时的非径向方向距离函数模型可表示为：

$$\begin{aligned}
&\vec{D}_o^{nr}\left(x,y,b;0,g_y,-g_b\right)=\max\textstyle\sum_{l=1}^{L}\omega_{ly}\beta_{ly}+\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}\theta_iz_iy_{li}\geqslant y_{l0}+\beta_{ly}g_{ly},l=1,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}\theta_iz_ib_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_ix_{ki}\leqslant x_{k0},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}z_i=1,\\
&0\leqslant\theta_i\leqslant 1,z_i\geqslant 0,i=1,\cdots,N,\beta_{ly},\beta_{mb}\geqslant 0
\end{aligned}\tag{6.70}$$

线性化后的模型为：

$$\begin{aligned}
&\vec{D}_o^{nr}\left(x,y,b;0,g_y,-g_b\right)=\max\textstyle\sum_{l=1}^{L}\omega_{ly}\beta_{ly}+\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}\rho_iy_{li}\geqslant y_{l0}+\beta_{ly}g_{ly},l=1,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}\rho_ib_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}(\rho_i+\mu_i)x_{ki}\leqslant x_{k0},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}(\rho_i+\mu_i)=1,\\
&\rho_i,\mu_i\geqslant 0,i=1,\cdots,N,\beta_{ly},\beta_{mb}\geqslant 0
\end{aligned}\tag{6.71}$$

期望产出的绩效指数可表示为：

$$teY_l = \frac{y_l}{y_l + \beta^*_{ly} y_l} = \frac{1}{1 + \beta^*_{ly}}$$

非期望产出的绩效指数可表示为：

$$teB_m = \frac{b_m - \beta^*_{mb} b_m}{b_m} = 1 - \beta^*_{mb}$$

上述 teY_l、teB_m 由代码返回的变量级缩放因子 `beta` 在调用处换算得到：$teY_l = 1/(1+\beta^*_{ly})$、$teB_m = 1 - \beta^*_{mb}$（对应代码列 `beta_Y`、`beta_CO2`）。案例报告的加权聚合指标 $EO = \sum_l \omega_{ly} \cdot teY_l$、$EB = \sum_m \omega_{mb} \cdot teB_m$ 同样由调用处据 `beta` 与权重 ω 汇总得到（标准等权设定下 $EO = 0.5/(1+\beta_Y)$、$EB = 0.5(1-\beta_{CO2})$），详见式(6.65) 后的量纲说明。还可将最优非期望产出强度与实际非期望产出强度之比定义为环境绩效指数：

$$EEI = \frac{1 - \beta^*_{mb}}{1 + \beta^*_{ly}}$$

注意 EEI 与调用处据 `beta` 计算的 EBO = EB/EO 数学形式不同：前者是单变量 β^* 的分式，后者是加权效率之比；本章案例报告 `EBO` 而非 `EEI`。

（二）投入方向的非径向方向距离函数当将方向向量调整为 $(-g_x, g_y, -g_b) = (x, 0, 0)$ 时，投入方向的非径向方向距离函数可定义为：

$$\vec{D}_i^{nr}(x, y, b; -g_x, 0, 0) = \max\left\{\omega' \beta : (x, y, b) + (-g_x, 0, 0) \cdot \mathrm{diag}(\beta) \in T\right\} \tag{6.72}$$

式中，$\omega = (\omega_x)' = (\omega_{1x}, \omega_{2x}, \cdots, \omega_{Kx})'$ 为权重向量，且满足 $\sum_{k=1}^K \omega_{kx} = 1$；$(-g_x) = (-g_{1x}, -g_{2x}, \cdots, -g_{Kx})'$ 为方向向量；$\beta = (\beta_x)' = (\beta_{1x}, \beta_{2x}, \cdots, \beta_{Kx})' \geqslant 0$ 为各投入变量在给定方向向量下的调整幅度。

在 CRS 假设下，投入方向的非径向方向距离函数可通过下式求解：

$$\begin{aligned} & \vec{D}_i^{nr}\left(x, y, b; -g_x, 0, 0\right) = \max \textstyle\sum_{k=1}^K \omega_{kx} \beta_{kx} \\ \text{s.t.} \quad & \textstyle\sum_{i=1}^N z_i y_{li} \geqslant y_{l0}, l = 1, \cdots, L \\ & \textstyle\sum_{i=1}^N z_i b_{mi} = b_{m0}, m = 1, \cdots, M, \\ & \textstyle\sum_{i=1}^N z_i x_{ki} \leqslant x_{k0} - \beta_{kx} g_{kx}, k = 1, \cdots, K, \\ & z_i \geqslant 0, i = 1, \cdots, N, \beta_{kx} \geqslant 0 \end{aligned} \tag{6.73}$$

VRS 下区分减排因子相同和不同两种情形：

（1）减排因子相同时的非径向方向距离函数模型可表示为：

$$\begin{aligned} & \vec{D}_i^{nr}\left(x, y, b; -g_x, 0, 0\right) = \max \textstyle\sum_{k=1}^K \omega_{kx} \beta_{kx} \\ \text{s.t.} \quad & \theta \textstyle\sum_{i=1}^N z_i y_{li} \geqslant y_{l0}, l = 1, \cdots, L, \\ & \theta \textstyle\sum_{i=1}^N z_i b_{mi} = b_{m0}, m = 1, \cdots, M, \\ & \textstyle\sum_{i=1}^N z_i x_{ki} \leqslant x_{k0} - \beta_{kx} g_{kx}, k = 1, \cdots, K, \\ & \textstyle\sum_{i=1}^N z_i = 1, \\ & 0 \leqslant \theta \leqslant 1, z_i \geqslant 0, i = 1, \cdots, N, \beta_{kx} \geqslant 0 \end{aligned} \tag{6.74}$$

该模型不能直接线性化，后续计算不采用这一设定（理由同式6.15 处说明）。

（2）减排因子不同时的非径向方向距离函数模型可表示为：

$$\begin{aligned}
&\vec{D}_i^{nr}\left(x,y,b;-g_x,0,0\right)=\max\textstyle\sum_{k=1}^{K}\omega_{kx}\beta_{kx}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}\theta_i z_i y_{li}\geqslant y_{l0},l=1,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}\theta_i z_i b_{mi}=b_{m0},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_i x_{ki}\leqslant x_{k0}-\beta_{kx}g_{kx},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}z_i=1,\\
&0\leqslant\theta_i\leqslant 1,z_i\geqslant 0,i=1,\cdots,N,\beta_{kx}\geqslant 0
\end{aligned}\tag{6.75}$$

线性化后的模型为：

$$\begin{aligned}
&\vec{D}_i^{nr}\left(x,y,b;-g_x,0,0\right)=\max\textstyle\sum_{k=1}^{K}\omega_{kx}\beta_{kx}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}\rho_i y_{li}\geqslant y_{l0},l=1,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}\rho_i b_{mi}=b_{m0},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}(\rho_i+\mu_i)x_{ki}\leqslant x_{k0}-\beta_{kx}g_{kx},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}(\rho_i+\mu_i)=1,\\
&\rho_i,\mu_i\geqslant 0,i=1,\cdots,N,\beta_{kx}\geqslant 0
\end{aligned}\tag{6.76}$$

投入的绩效指数可表示为：

$$EI=\frac{x_k-\beta_{kx}^{*}x_k}{x_k}=1-\beta_{kx}^{*}$$

（三）非期望产出方向的非径向方向距离函数除期望产出—非期望产出方向的非径向方向距离函数外，也可以只沿非期望产出方向定义非径向方向距离函数：

$$\vec{D}_o^{nr}\left(x,y,b;0,0,-g_b\right)=\max\left\{\omega'\beta:(x,y,b)+(0,0,-g_b)\cdot\mathrm{diag}(\beta)\in T\right\}\tag{6.77}$$

式中，$\omega=(\omega_b)'=\left(\omega_{1b},\omega_{2b},\cdots,\omega_{M_b}\right)'$ 为权重向量，且满足 $\sum_{m=1}^{M}\omega_{mb}=1$；$(-g_b)=(-g_{1b},-g_{2b},\cdots,-g_{Mb})'$ 为方向向量；$\beta=(\beta_b)'=(\beta_{1b},\beta_{2b},\cdots,\beta_{Mb})'\geqslant 0$ 为各非期望产出在给定方向向量下的调整幅度。

在 CRS 假设下，非期望产出方向的非径向方向距离函数可通过下式求解：

$$\begin{aligned}
&\vec{D}_o^{nr}\left(x,y,b;0,0,-g_b\right)=\max\textstyle\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}z_i y_{li}\geqslant y_{l0},l=1,\cdots,L\\
&\textstyle\sum_{i=1}^{N}z_i b_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_i x_{ki}\leqslant x_{k0},k=1,\cdots,K,\\
&z_i\geqslant 0,i=1,\cdots,N,\beta_{mb}\geqslant 0
\end{aligned}\tag{6.78}$$

VRS 下区分减排因子相同和不同两种情形：

（1）减排因子相同时的非径向方向距离函数模型可表示为：

$$\begin{aligned}
\vec{D}_o^{nr}\quad&(x,y,b;0,0,-g_b)=\max\textstyle\sum_{m=1}^{M}\omega_{mb}\beta_{mb}\\
\text{s.t.}\quad&\textstyle\sum_{i=1}^{N}z_i y_{li}\geqslant y_{l0},l=1,\cdots,L,\\
&\textstyle\sum_{i=1}^{N}z_i b_{mi}=b_{m0}-\beta_{mb}g_{mb},m=1,\cdots,M,\\
&\textstyle\sum_{i=1}^{N}z_i x_{ki}\leqslant\theta x_{k0},k=1,\cdots,K,\\
&\textstyle\sum_{i=1}^{N}z_i=\theta,\\
&0\leqslant\theta\leqslant 1,z_i\geqslant 0,i=1,\cdots,N,\beta_{mb}\geqslant 0
\end{aligned}\tag{6.79}$$

（2）减排因子不同时的非径向方向距离函数模型可表示为：

$$\begin{aligned} & \vec{D}_o^{nr}(x,y,b;0,0,-g_b) = \max \textstyle\sum_{m=1}^{M} \omega_{mb}\beta_{mb} \\ \text{s.t.} \quad & \textstyle\sum_{i=1}^{N} \rho_i y_{li} \geqslant y_{l0}, l = 1,\cdots,L, \\ & \textstyle\sum_{i=1}^{N} \rho_i b_{mi} = b_{m0} - \beta_{mb} g_{mb}, m = 1,\cdots,M, \\ & \textstyle\sum_{i=1}^{N} (\rho_i + \mu_i) x_{ki} \leqslant x_{k0}, k = 1,\cdots,K, \\ & \textstyle\sum_{i=1}^{N} (\rho_i + \mu_i) = 1, \\ & \rho_i, \mu_i \geqslant 0, i = 1,\cdots,N, \beta_{mb} \geqslant 0 \end{aligned} \tag{6.80}$$

非期望产出的绩效指数可表示为：

$$EB = \frac{b_m - \beta_{mb}^* b_m}{b_m} = 1 - \beta_{mb}^*$$

6.3 环境绩效测度模型的Python实现

6.1 节与 6.2 节已将非期望产出与弱可处置性约束纳入生产技术，并给出谢泼德距离函数、双曲距离函数、方向距离函数和非径向方向距离函数的线性规划形式。这些规划在代数上彼此平行，但其共同特征在于：除投入与期望产出外，约束中必须显式容纳非期望产出，并通过方向向量界定期望产出扩张、投入收缩与污染排放压缩的相对权重。本章的程序实现正是对上述线性规划的可计算转写——其关键差异并非算法本身，而在于须在 `sent` 中声明非期望产出，并借由方向向量 `gx/gy/gb` 说明调整方向。

`deabook` 提供的 `DEAweak` 模块即为这类弱可处置性模型而设计。为兼顾可读性与可复现性，该模块在底层承载类（`DEAweak`、`DDFweak`、`NDDFweak`）之上封装了四个面向距离函数的接口：`ushephard`、`hyper`、`ddf`、`nddf`。后续OECD 国家与中国省份两类案例均只调用这四个接口，而不直接使用底层承载类。因此，本节先阐明接口与变量的声明方式，再依次讨论方向向量与规模报酬的经济含义、各接口的返回指标及其效率换算，最后以四个可运行示例分别对应本章四类距离函数模型。

具体而言，四个接口与 6.2 节四类距离函数的对应关系为：`ushephard` 对应非期望产出方向的谢泼德距离函数（式 6.19/6.22/6.24）；`hyper` 对应包含非期望产出的双曲距离函数（式 6.27/6.30/6.31）；`ddf` 将方向距离函数扩展到非期望产出（式 6.45/6.47/6.49）；`nddf` 对应非径向方向距离函数（式 6.66 起）。四者共享同一套 `sent`、`gx/gy/gb` 与规模报酬设定，差异仅在于底层距离值的计算方式。其中 `ushephard` 与 `hyper` 为径向测度，默认方向由函数名隐式确定，使用者通常无需手写方向向量；`ddf` 与 `nddf` 则在方向距离框架下刻画期望产出扩张与非期望产出压缩，方向向量须与所研究的环境经济问题保持一致。

6.3.1 非期望产出的声明与参数框架

弱可处置性 DEA 模型与标准 DEA 调用的根本区别，集中体现在 `sent` 参数对产出结构的重新刻画上：冒号之前为期望产出，冒号之后为非期望产出。其余接口参数则沿用了既有DEA 模块的约定，只是在方向向量上增加了非期望产出的维度。

```
from deabook import data_path, PANEL
from deabook.constant import SEQUENTIAL
from deabook import DEAweak, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook.constant import RTS_CRS, RTS_VRS1, RTS_VRS2, OPT_LOCAL
```

以 sent="K L E=Y:CO2" 为例，等号左侧的 K L E 为投入变量，等号右侧冒号前的 Y 为期望产出，冒号后的 CO2 为非期望产出。方向向量 gx、gy 与 gb 分别作用于投入、期望产出与非期望产出，其元素顺序必须与 sent 中各变量的排列保持一致。完整的参数约定汇总于表 6.1；其中 data 为包含投入、两类产出与识别变量的 DataFrame，evalindex 与 refindex 分别用于筛选被评价样本与参考技术集（默认均为全部观测），solver 与 email 在 optimize() 中指定求解方式与求解位置。

表 6.1: 弱可处置性 DEA 模型的主要参数

参数	作用	本章常用设置	说明
data	指定待评价数据	DataFrame	包含投入、期望产出、非期望产出和识别变量
sent	声明投入产出结构	"K L E=Y:CO2"	= 左侧为投入，= 右侧且：前为期望产出，：后为非期望产出
gx	指定投入方向	[0,0,0]、[1,1,1]	元素顺序与投入变量 K、L、E 一致
gy	指定期望产出方向	[0]、[1]	[1] 表示沿期望产出方向扩张
gb	指定非期望产出方向	[0]、[1]	[1] 表示沿非期望产出方向缩减
rts	指定规模报酬和弱可处置性形式	RTS_CRS、RTS_VRS1、RTS_VRS2	分别表示规模报酬不变、相同减排因子和不同减排因子
evalindex	选择被评价样本	None	None 表示评价全部观测，也可用条件语句筛选特定年份或 DMU
refindex	选择参考技术集	None	None 表示使用全部观测构造前沿
solver	指定求解器	"mosek"	在 optimize() 中设置
email	指定求解方式	OPT_LOCAL	表示本地求解

方向向量 gx/gy/gb 与规模报酬 rts 共同决定环境绩效指标的经济含义，二者的取值逻辑在下文分别展开。

6.3.2 方向设定与规模报酬

方向向量决定了距离函数究竟在测度何种" 改进"。在弱可处置性框架下，若只令 gb=[1] 而保持 gx、gy 为零，模型评价的是在投入与期望产出不变时非期望产出可压缩的最大比例，即纯环境效率；若同时令 gy=[1]，则评价的是期望产出扩张与污染排放压缩的联合改进空间；若进一步加入投入方向，便得到涵盖投入缩减的全方向综合绩效。表 6.2 依此列出本章涉及的几类方向与规模报酬设定，并给出其对应的经济含义。

表 6.2: 方向向量、规模报酬与经济含义

设定	本章常用值	经济含义	对应模型或用途
非期望产出导向	gx=[0,0,0], gy=[0], gb=[1]	投入和期望产出不变，只压缩 CO2	纯环境效率
期望产出—非期望产出方向	gx=[0,0,0], gy=[1], gb=[1]	扩张 Y，同时压缩 CO2	混合环境效率、DDF 和 NDDF 案例
投入方向	gx=[1,1,1], gy=[0], gb=[0]	同时缩减资本、劳动和能源	投入效率扩展分析（仅CRS/VRS2 可解，VRS1 同因子不可线性化，见式 6.15 处说明）
全方向	gx=[1,1,1], gy=[1], gb=[1]	同时缩减投入、扩张期望产出并压缩非期望产出	综合绩效分析（仅 CRS/VRS2 可解）
RTS_CRS	规模报酬不变	所有 DMU 面临同一规模技术	基准模型
RTS_VRS1	规模报酬可变、相同减排因子	允许规模差异，但污染减排因子相同	同质弱可处置性模型
RTS_VRS2	规模报酬可变、不同减排因子	允许不同 DMU 面临不同污染处置难度	异质弱可处置性模型

本章实证部分集中采用两类标准方向：其一为 ushephard 的非期望产出导向，即投入与期望产出不变、仅压缩非期望产出（gy=[0]、gx=[0,0,0]、gb=[1]），对应纯环境效率；其二为 hyper、ddf 与 nddf 中的期望产出—非期望产出方向，即同时扩张期望产出并压缩非期望产出（gy=[1]、gx=[0,0,0]、gb=[1]）。当采用这两类标准方向时，四个接口的默认方向向量已与之吻合，调用时无需显式写出；若要偏离标准方向（如引入投入方向），再传入 gx/gy/gb 即可。方向设定确定之后，各接口究竟返回哪些列、列与效率指标的对应关系如何，便成为解读结果的关键。

6.3.3 返回指标及其效率换算

四个接口在类型上分别对应前述四类距离函数：ushephard 与 hyper 承载径向的弱可处置性 DEA 模型（分别为非期望产出方向的谢泼德距离函数与包含非期望产出的双曲距离函数），ddf 对应方向距离函数模型，nddf 对应非径向方向距离函数模型。值得强调的是，这些接口并不直接输出" 环境效率" 之类的

成品指标，而是返回线性规划的目标值 rho（及 objective_value），由调用方依据方向向量换算为具有经济含义的效率分数。各接口的返回列与换算口径汇总于表 6.3。

表 6.3: DEAweak 四类接口的主要返回列及解释

接口	主要返回列	指标解释
ushephard	optimization_status、rho、objective_value	方向固定为非期望产出导向（不支持设定方向），仅返回径向效率比 rho，即纯环境效率 PEE = rho（不生成 te 列）
hyper	optimization_status、rho、objective_value	双曲设定下仅返回线性规划目标值 rho；混合环境效率由调用方据方向换算：CRS 下 MEE = sqrt(rho)，VRS1/VRS2 下 MEE_BO = 1 - rho、MEE_O = 1/(1 + rho)（不生成 te/teuo/teo 列）
ddf	optimization_status、rho、objective_value	方向距离函数模型；与 DEA.ddf 一致，本接口**仅返回方向距离函数值 rho**（外加 objective_value），并不生成 teuo/teo 等效率分解列。环境效率 EB 与产出效率 EO 由调用方按方向在调用处换算：EB = 1 - rho、EO = 1/(1 + rho)
nddf	optimization_status、objective_value、beta_<变量名>	非径向模型为每个变量生成独立的缩放因子 beta（命名规则 beta_<变量名>，默认方向下为 beta_Y、beta_CO2），外加线性规划目标值 objective_value（各 beta 的加权和）。与 DEA.ddf 一致，本接口不生成 teY/teCO2/teuo/teo/teuo2o 等效率分解列；EB、EO、EBO 由调用方据 beta 与权重在调用处换算（标准等权下 EB = 0.5(1-beta_CO2)、EO = 0.5/(1+beta_Y)、EBO = EB/EO）

rho 与 te 分别对应哪个符号： rho 是线性规划的目标值，即 6.2 节各规划中的 Farrell 型比例因子 λ^*；四个接口均只返回 rho（nddf 另返回各变量缩放因子 beta），效率分数由调用方据方向在调用处换算，换算口径按方向而定。非期望产出导向与投入导向下 $\lambda^* \leq 1$ 本身就是效率，故 te = rho（对应 $PEE = \lambda^*$、$PEI = \lambda^*$，即谢泼德距离的倒数 $1/\theta^*$）；期望产出导向下 $\lambda^* \geq 1$ 是扩张倍数，故 te = 1/rho（对应 $PEO = 1/\lambda^* = \theta^*$，即谢泼德产出距离本身）。双曲设定下

CRS 的 rho 仍是径向比例因子，效率 $=\sqrt{\rho}$；VRS 的 rho 则是线性化后 DDF 形式的 β^*（目标为最大化），因而分解为坏产出端效率 $1-\rho$、期望产出端效率 $1/(1+\rho)$。简言之，除双曲 VRS 的 rho 是 β 外，rho 都是比例因子 λ，而效率分数才是与 θ 同口径（≤ 1）的指标。

整理结果时，应先剔除 optimization_status、rho/objective_value 等辅助列，再据研究目标重命名效率列。具体而言，本章将 ushephard 返回的 rho 直接视为纯环境效率 PEE（非期望产出导向下 $PEE=\rho$）；将 hyper 返回的 rho 据方向换算为混合环境效率（CRS 下 MEE = sqrt(rho)，VRS1/VRS2 下 MEE_BO = 1 - rho、MEE_O = 1/(1+rho)）；将 ddf 返回的 rho 按方向换算为环境效率 EB、EO（其中 EB = 1 - rho、EO = 1/(1 + rho)，与 DEA.ddf 的换算口径一致）；将 nddf 返回的变量级缩放因子 beta（列 beta_Y、beta_CO2）按权重汇总为环境效率 EB、产出效率 EO 与相对排放强度 EBO（标准等权下 EB = 0.5(1 - beta_CO2)、EO = 0.5/(1 + beta_Y)、EBO = EB/EO）。其中 EBO 是单位期望产出的相对排放强度指标，本质是两端效率的比值而非Farrell 效率，因而其上界不限于 1（详见 6.2.4 节 式 6.65 后的量纲说明）。底层变量级效率 teY、teCO2（由 beta_Y、beta_CO2 在调用处换算为 $teY=1/(1+\beta_Y^*)$、$teB=1-\beta_{CO2}^*$）是 EO、EB 的加权原料，本章不再单独整理。这一换算逻辑在下一小节的可运行示例中逐一体现。

6.3.4 四类距离函数的可运行示例

以下四个示例依次实现 6.2 节的四类距离函数：示例 1 对应谢泼德纯环境效率（式 6.19/6.22/6.24），示例 2 对应双曲混合效率（式6.27/6.30/6.31），示例 3 对应方向距离函数（式 6.45/6.47/6.49），示例 4 对应非径向方向距离函数（式 6.66 起）。四者在 CRS、VRS1（相同减排因子）与VRS2（不同减排因子）三种规模报酬下的调用方式完全一致，仅 rts 参数取值相异，故下文仅以 rts 区分三种技术设定。

示例 1：ushephard 非期望产出导向（对应 6.2.1 节模型6.19/6.22/6.24，即非期望产出方向的谢泼德距离函数。ushephard 的方向**固定**为非期望产出导向（gx=[0,0,0]、gy=[0]、gb=[1]：投入和期望产出不变、只压缩非期望产出），**不支持**设定方向向量，调用时无需（也不能）传入 gx/gy/gb；与 DEA.ddf 一致，本接口仅返回线性规划目标值 rho，在 rho≤1 下其本身就是效率，故纯环境效率 PEE = rho）：

```
# CRS
model = DEAweak.ushephard(data, sent="K L E=Y:CO2", rts=RTS_CRS)
res = model.optimize(solver="mosek")

# VRS1
model = DEAweak.ushephard(data, sent="K L E=Y:CO2", rts=RTS_VRS1)
res = model.optimize(solver="mosek")

# VRS2
model = DEAweak.ushephard(data, sent="K L E=Y:CO2", rts=RTS_VRS2)
res = model.optimize(solver="mosek")
```

示例 2：hyper 双曲距离函数（HYPER）（对应 6.2.2 节双曲距离函数模型 6.27/6.30/6.31。hyper 的默认方向向量为 gx=[0,0,0]、gy=[1]、gb=[1]，即同时扩张期望产出并压缩非期望产出；与 DEA.ddf 一致，本接口仅返回线性规划目标值 rho，混合环境效率由调用方据方向在调用处换算（CRS 下 MEE = sqrt(rho)，VRS1/VRS2 下 MEE_BO = 1 - rho、MEE_O = 1/(1+rho)），适用于” 增产与减排能否协同推进” 的联合改进分析。注意”Hyper” 是”Hyperbolic”（双曲）的缩写，与超效率super-efficiency 是不同概念。VRS1/VRS2 下

的双曲线性化等价于方向向量 $g = (y, -b)$ 的 DDF，详见 6.2.2 节 式 6.30 处的等价性说明）：

```
# CRS
model = DEAweak.hyper(data, sent="K L E=Y:CO2", rts=RTS_CRS)
res = model.optimize(solver="mosek")

# VRS1
model = DEAweak.hyper(data, sent="K L E=Y:CO2", rts=RTS_VRS1)
res = model.optimize(solver="mosek")

# VRS2
model = DEAweak.hyper(data, sent="K L E=Y:CO2", rts=RTS_VRS2)
res = model.optimize(solver="mosek")
```

示例 3：ddf 期望产出—非期望产出方向 — CRS/VRS1/VRS2（对应 6.2.3 节方向距离函数模型 6.45/6.47/6.49。ddf 的默认方向向量为 gx=[0,0,0]、gy=[1]、gb=[1]，同时调整期望产出与非期望产出；与 DEA.ddf 一致，本接口仅返回方向距离函数值 rho，需由调用方按方向换算为环境效率 EB = 1 - rho 与产出效率 EO = 1/(1 + rho)）：

```
# CRS
model = DEAweak.ddf(data, sent="K L E=Y:CO2", rts=RTS_CRS)
res = model.optimize(solver="mosek")

# VRS1
model = DEAweak.ddf(data, sent="K L E=Y:CO2", rts=RTS_VRS1)
res = model.optimize(solver="mosek")

# VRS2
model = DEAweak.ddf(data, sent="K L E=Y:CO2", rts=RTS_VRS2)
res = model.optimize(solver="mosek")

# 显式给出方向向量(与默认一致:投入不变、扩张期望产出、压缩非期望产出)
model = DEAweak.ddf(data, sent="K L E=Y:CO2",
                    gx=[0,0,0], gy=[1], gb=[1], rts=RTS_CRS)
res = model.optimize(solver="mosek")

# 也可设定其他非负方向,例如同时收缩第一个投入 K（gx 取非零）:
#   model = DEAweak.ddf(data, sent="K L E=Y:CO2",
#                       gx=[1,0,0], gy=[1], gb=[1], rts=RTS_CRS)
# 方向改变后，EB/EO 的换算口径需按对应方向重新确定(本书案例沿用上述默认方向)
```

ddf 使用方向距离函数，若不显式传入 gx/gy/gb 则取默认 gx=[0,0,0]、gy=[1]、gb=[1]：gy=[1] 扩张期望产出，gb=[1] 缩减非期望产出。

示例 4：nddf 期望产出—非期望产出方向 — CRS/VRS1/VRS2（对应 6.2.4 节非径向方向距离函数，式 6.66 起；与示例 3 方向设定相同，但每个变量独立调整，返回各变量的缩放因子 beta（列 beta_Y、beta_CO2）及线性规划目标值 objective_value，适用于径向松弛明显、需要识别变量层面改进空间的研究）：

```
# CRS
model = DEAweak.nddf(data, sent="K L E=Y:CO2", rts=RTS_CRS)
res = model.optimize(solver="mosek")

# VRS1
model = DEAweak.nddf(data, sent="K L E=Y:CO2", rts=RTS_VRS1)
res = model.optimize(solver="mosek")

# VRS2
model = DEAweak.nddf(data, sent="K L E=Y:CO2", rts=RTS_VRS2)
res = model.optimize(solver="mosek")

# 显式给出方向向量(与默认一致:投入不变、扩张期望产出、压缩非期望产出)
model = DEAweak.nddf(data, sent="K L E=Y:CO2",
                     gx=[0,0,0], gy=[1], gb=[1], rts=RTS_CRS)
res = model.optimize(solver="mosek")

# 也可设定其他非负方向,例如同时收缩第一个投入 K(gx 取非零):
#   model = DEAweak.nddf(data, sent="K L E=Y:CO2",
#                        gx=[1,0,0], gy=[1], gb=[1], rts=RTS_CRS)
# 方向改变后, EB/EO/EBO 的换算口径需按对应方向重新确定(本书案例沿用上述默认方向)
```

nddf 为非径向模型，每个变量独立计算缩放因子 beta（列 beta_Y、beta_CO2）；环境效率 EB、产出效率 EO 与相对排放强度 EBO 由调用方据 beta 与权重在调用处换算（标准等权下 EB = 0.5(1 - beta_CO2)、EO = 0.5/(1 + beta_Y)、EBO = EB/EO）。

上述调用规则只涉及语法层面。环境 DEA 模型的真正价值，在于不同距离函数、规模报酬与减排因子设定下结果如何分化。下面以本章主样本展示这种分化，并据此给出实证选择建议。

6.4 案例应用：OECD国家的碳排放效率

本节固定2019年33个OECD国家为一个截面，以K、L、E为投入，Y为期望产出，CO2为非期望产出。数据源沿用Penn World Table、EIA和Global Carbon Atlas所整理的oecd data.xlsx。方向只减少CO2，效率记为1−D；资本、劳动、能源与产出保持不变。主表只比较弱可处置CRS、共同减排因子VRS和强可处置CRS三种技术，不另设中国样本。

```
from deabook.example.book_cases.run import FUN
FUN[6]()  # 重新计算本章,输出完整CSV、摘要表与PNG/PDF图
```

表 6.4: OECD国家的碳排放效率：结果摘要

统计量	弱处置CRS	弱处置VRS共同因子	强处置CRS
有效样本数	33	33	33
均值	0.447	0.654	0.447
标准差	0.257	0.273	0.257
最小值	0.178	0.223	0.178
P10	0.207	0.317	0.207
中位数	0.361	0.670	0.361
P90	0.959	1.000	0.959
最大值	1.000	1.000	1.000

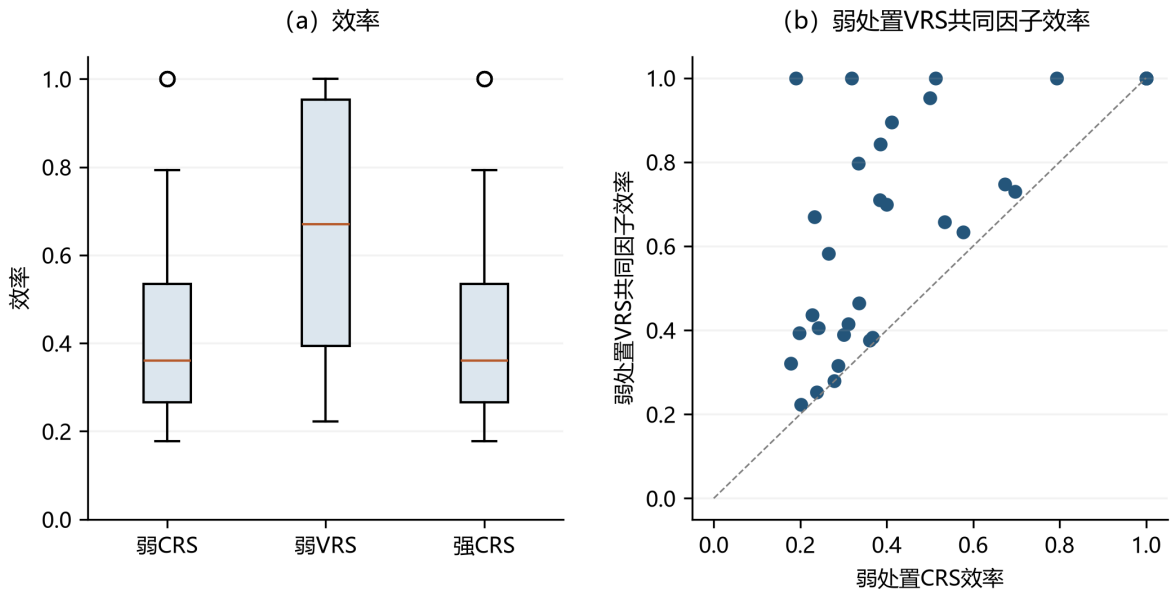


图 6.10: 2019年OECD碳排放方向效率的技术假设比较

表将三种技术设定并列，分别列出样本数、中心位置、离散程度及两端分位数。图中的散点用于逐个国家比较CRS与VRS结果，箱线图保留离群点。不能仅凭均值判断不同处置假设对所有国家的影响相同。

弱可处置CRS效率均值为0.447，共同减排因子VRS为0.654。本样本只沿单一坏产出调整时，强、弱可处置CRS给出相同最优值；这是本例结果，不证明两种技术集合等价。共同减排因子VRS同时满足强度之和等于减排因子、投入约束右端乘该因子，不能误写成异质减排因子的技术。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch06_full.csv`。表中P10和P90为第10和第90百分位数（如适用），展示分布而不占用逐个决策单元的行数。

6.5 本章小结

弱可处置CRS效率均值为0.447，共同减排因子VRS为0.654。本样本只沿单一坏产出调整时，强、弱可处置CRS给出相同最优值；这是本例结果，不证明两种技术集合等价。共同减排因子VRS同时满足强度之和等于减排因子、投入约束右端乘该因子，不能误写成异质减排因子的技术。

参考文献

CHEN C-M, DELMAS M A. Measuring eco-inefficiency: A new frontier approach [J]. Operations Research, 2012, 60(5): 1064-1079. DOI: 10.1287/opre.1120.1094.

CHUNG Y H, FÄRE R, GROSSKOPF S. Productivity and undesirable outputs: A directional distance function approach [J]. Journal of Environmental Management, 1997, 51(3): 229-240. DOI: 10.1006/jema.1997.0146.

FEENSTRA R C, INKLAAR R, TIMMER M P. Penn World Table version 10.01 [DS/OL]. Groningen Growth and Development Centre, University of Groningen, 2021. DOI: 10.34894/QT5BCC.

FEENSTRA R C, INKLAAR R, TIMMER M P. The next generation of the Penn World Table [J]. American Economic Review, 2015, 105(10): 3150-3182. DOI: 10.1257/aer.20130954.

FUKUYAMA H, WEBER W L. A directional slacks-based measure of technical inefficiency [J]. Socio-Economic Planning Sciences, 2009, 43(4): 274-287. DOI: 10.1016/j.seps.2008.12.001.

FÄRE R, GROSSKOPF S. Nonparametric productivity analysis with undesirable outputs: Comment [J]. American Journal of Agricultural Economics, 2003, 85(4): 1070-1074. DOI: 10.1111/1467-8276.00510.

FÄRE R, GROSSKOPF S, LOVELL C A K, PASURKA C A. Multilateral productivity comparisons when some outputs are undesirable: A nonparametric approach [J]. The Review of Economics and Statistics, 1989, 71(1): 90-98. DOI: 10.2307/1928055.

KUOSMANEN T. Weak disposability in nonparametric production analysis with undesirable outputs [J]. American Journal of Agricultural Economics, 2005, 87(4): 1077-1082. DOI: 10.1111/j.1467-8276.2005.00788.x.

LELEU H. Shadow pricing of undesirable outputs in nonparametric analysis [J]. European Journal of Operational Research, 2013, 231(2): 474-480. DOI: 10.1016/j.ejor.2013.05.028.

ZHANG N, CHOI Y. Total-factor carbon emission performance of fossil fuel power plants in China: A metafrontier non-radial Malmquist index analysis [J]. Energy Economics, 2013, 40: 549-559. DOI: 10.1016/j.eneco.2013.08.012.

ZHOU P, ANG B W, HAN J Y. Total factor carbon emission performance: A Malmquist index analysis [J]. Energy Economics, 2010, 32(1): 194-201. DOI: 10.1016/j.eneco.2009.10.003.

ZHOU P, ANG B W, WANG H. Energy and CO2 emission performance in electricity generation: A non-radial directional distance function approach [J]. European Journal of Operational Research, 2012, 221(3): 625-635. DOI: 10.1016/j.ejor.2012.04.022.

第 7 章

包含非期望产出的生产率

第 6 章讨论了非期望产出约束下的静态环境效率，本章将分析对象从单期效率水平扩展到跨期生产率变化。传统全要素生产率指标主要刻画投入节约与期望产出扩张。然而，在污染排放具有弱可处置性时，若不同时考虑非期望产出，容易高估经济增长的真实质量。基于这一问题意识，本章在环境生产技术集合的基础上，依次引入Malmquist 生产率指数、Malmquist-Luenberger 指数、Luenberger 指标以及非径向方向距离函数，并将非期望产出方向单独抽出，构造动态环境绩效指标，考察决策单元的环境绩效如何随效率追赶和技术前沿移动而变化。

本章按距离函数类型与调整方向依次展开，并以效率变化与技术变化的分解作为统一解释框架。首先，要明确投入、期望产出与非期望产出的可行组合；继而，以谢泼德距离函数、方向距离函数和非径向方向距离函数刻画观测点相对环境生产前沿的位置；再由相邻时期比较构造动态生产率指标与动态环境绩效指标；最后，将其分解为效率变化与技术变化。这样处理的优点在于，生产率增长被置于由污染约束与环境治理能力共同决定的技术边界之内进行评价，从而综合反映投入、产出与排放三方面的变化。

7.1 Malmquist 生产率指数

生产率的动态变化以单期距离函数为基础，并通过跨期比较加以刻画。Malmquist 生产率指数（Caves, Christensen and Diewert, 1982）以不同时期的生产技术为参照，通过比较同一决策单元在相邻时期相对于生产前沿的位置变化，将生产率变动分解为效率变化和技术变化；本章采用的DEA 实现及分解形式来自 Färe, Grosskopf, Norris and Zhang (1994)。本节先讨论期望产出方向的 Malmquist 指数，再给出投入方向的对称形式。Malmquist 指数在本章中首先作为基准框架出现：它可以将非期望产出纳入环境生产技术集合，但其调整方向仍主要围绕投入或期望产出展开，因此尚不能完全刻画” 扩张期望产出、压缩污染排放” 的绿色生产率含义。

7.1.1 期望产出方向

在包含非期望产出的环境生产技术中，期望产出方向的谢泼德距离函数（Shephard, 1970；环境方向扩展见 Färe, Grosskopf and Lovell, 1985）可定义为：

$$D_o(x, y, b) = \inf\{\theta : (x, y/\theta, b) \in T\} \tag{7.1}$$

式中，T 表示本章导语中定义的环境生产技术集合。设 t 期和 $s(s>t)$ 期决策单元观测值分别为 (x^t, y^t, b^t) 和 (x^s, y^s, b^s)，则以 t 期技术为参照的 Malmquist 生产率指数，可写为 s 期距离函数与 t 期距离函数之比：

$$M_o^t = \frac{D_o^t(x^s, y^s, b^s)}{D_o^t(x^t, y^t, b^t)} \tag{7.2}$$

相应地，以 s 期技术为参照的指数为：

$$M_o^s = \frac{D_o^s(x^s, y^s, b^s)}{D_o^s(x^t, y^t, b^t)} \tag{7.3}$$

为降低单一参照技术选择对指数的影响，产出导向 Malmquist 生产率指数通常取两个时期指数的几何平均：

$$M_o = \left[\frac{D_o^t(x^s, y^s, b^s)}{D_o^t(x^t, y^t, b^t)} \times \frac{D_o^s(x^s, y^s, b^s)}{D_o^s(x^t, y^t, b^t)}\right]^{1/2} \tag{7.4}$$

式(7.4)包含四个产出方向谢泼德距离函数，其中同一期观测与同一期技术的组合反映效率位置，跨期观测与异期技术的组合反映前沿移动。将二者重新整理，可得到如下分解：

$$\begin{aligned} M_o &= \frac{D_o^s(x^s, y^s, b^s)}{D_o^t(x^t, y^t, b^t)} \times \left[\frac{D_o^t(x^s, y^s, b^s)}{D_o^s(x^s, y^s, b^s)} \times \frac{D_o^t(x^t, y^t, b^t)}{D_o^s(x^t, y^t, b^t)}\right]^{1/2} \\ &= \text{MEFFCH} \times \text{MTECHCH} \end{aligned} \tag{7.5}$$

其中，$MEFFCH$ 表示效率变化，用于刻画决策单元相对于产出技术前沿的追赶或偏离；$MTECHCH$ 表示技术变化，用于刻画生产技术前沿本身的移动。在这一设定中，非期望产出通过技术集合进入模型，发挥的是可行性约束作用；指数本身仍沿期望产出方向评价生产率变化。因此，该指标适合作为传统生产率分析的环境约束扩展，对其解释应限于期望产出方向的生产率变化，污染减排绩效的测度留待后续指标处理。

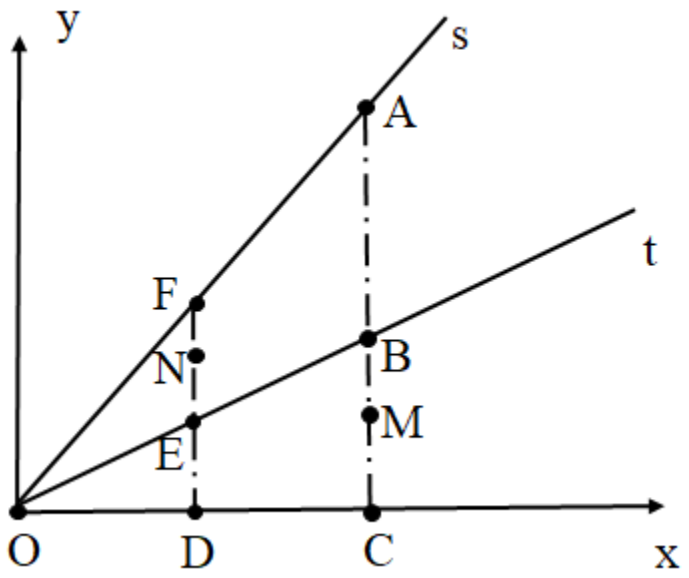


图 7.1: 单投入、单期望产出的产出导向 Malmquist 指数 M_o

图 7.1 给出了单投入、单期望产出情形下产出导向 Malmquist 指数 M_o 的几何含义。区域 $OABCD$ 和 $OA'B'C'D'$ 分别表示 t 期和 s 期环境 DEA 技术；点 M 和 N 表示同一决策单元在两个时期的投入—产出组合。图中，效率变化为 $\frac{DN/DF}{CM/CB}$，技术变化为 $[\frac{DN/DE}{DN/DF}\cdot\frac{CM/CB}{CM/CA}]^{1/2}=[\frac{GE}{GF'}\cdot\frac{G'F}{G'E'}]^{1/2}$，对应于给定投入水平下生产前沿的外移。

产出导向指数以期望产出的扩张刻画生产率变化，但当研究关注资源消耗约束更强、产出规模短期难以调整的情形时，更自然的参照是投入方向的节约能力，下一节给出投入导向的Malmquist 指数。

7.1.2 投入方向

在投入方向上，谢泼德距离函数（Shephard, 1970）可定义为：

$$D_i(x,y,b)=\sup\{\theta:(x/\theta,y,b)\in T\} \tag{7.6}$$

式中，T 表示环境生产技术集合。设 t 期和 $s(s>t)$ 期决策单元观测值分别为 (x^t,y^t,b^t) 和 (x^s,y^s,b^s)。投入方向谢泼德距离函数取值范围为 $[1,+\infty)$，其倒数可解释为投入效率，因此以 t 期技术为参照的投入导向指数写为：

$$M_i^t=\frac{D_i^t\left(x^t,y^t,b^t\right)}{D_i^t\left(x^s,y^s,b^s\right)} \tag{7.7}$$

相应地，以 s 期技术为参照的指数为：

$$M_i^s=\frac{D_i^s\left(x^t,y^t,b^t\right)}{D_i^s\left(x^s,y^s,b^s\right)} \tag{7.8}$$

为降低单一参照技术选择对指数的影响，投入导向 Malmquist 生产率指数通常取两个时期指数的几何平均：

$$M_i=\left[\frac{D_i^t\left(x^t,y^t,b^t\right)}{D_i^t\left(x^s,y^s,b^s\right)}\times\frac{D_i^s\left(x^t,y^t,b^t\right)}{D_i^s\left(x^s,y^s,b^s\right)}\right]^{1/2} \tag{7.9}$$

式（7.9）包含四个投入方向谢泼德距离函数。按照同一期技术参照和跨期技术参照重新分组，可分解为效率变化和技术变化两部分：

$$\begin{aligned}M_i&=\frac{D_i^t\left(x^t,y^t,b^t\right)}{D_i^s\left(x^s,y^s,b^s\right)}\times\left[\frac{D_i^s\left(x^s,y^s,b^s\right)}{D_i^t\left(x^s,y^s,b^s\right)}\times\frac{D_i^s\left(x^t,y^t,b^t\right)}{D_i^t\left(x^t,y^t,b^t\right)}\right]^{1/2}\\&=\text{MEFFCH}\times\text{MTECHCH}\end{aligned} \tag{7.10}$$

其中，$MEFFCH$ 表示效率变化，用于刻画决策单元相对于投入方向技术前沿的追赶或偏离；$MTECHCH$ 表示技术变化，用于刻画生产技术前沿本身的移动。与产出导向相比，投入导向更强调在既定期望产出和非期望产出水平下的投入节约能力，因此适用于研究资源消耗约束更强、产出规模短期较难调整的情形。

图 7.2 给出了投入导向 Malmquist 指数 M_i 的几何含义。射线 t 和 s 分别表示两个时期的环境 DEA 技术，点 M 和 N 表示同一决策单元在 t 期与 s 期、相同产出水平下的投入组合。A、E 分别是 M、N 在**本期**前沿上的对应点，B、F 分别是它们在**另一期**前沿上的对应点。于是 Farrell 型投入效率为 $\text{TE}_t=CA/CM$、$\text{TE}_s=DE/DN$，谢泼德距离为其倒数，即 $D_i^t(x^t)=CM/CA$、$D_i^s(x^s)=DN/DE$、$D_i^s(x^t)=CM/CB$、$D_i^t(x^s)=DN/DF$。据此，效率变化为 $\frac{DE/DN}{CA/CM}$（等价地写成谢泼德形式 $\frac{CM/CA}{DN/DE}$），与式 (7.10) 中的 $MEFFCH=D_i^t(x^t)/D_i^s(x^s)=\text{TE}_s/\text{TE}_t$ 一致；技术变化为 $\left[\frac{DN/DE}{DN/DF}\cdot\frac{CM/CB}{CM/CA}\right]^{1/2}=\left[\frac{DF}{DE}\cdot\frac{CA}{CB}\right]^{1/2}$，对应于给定产出水平下投入需求前沿的内移（前沿内移时该值大于1）。

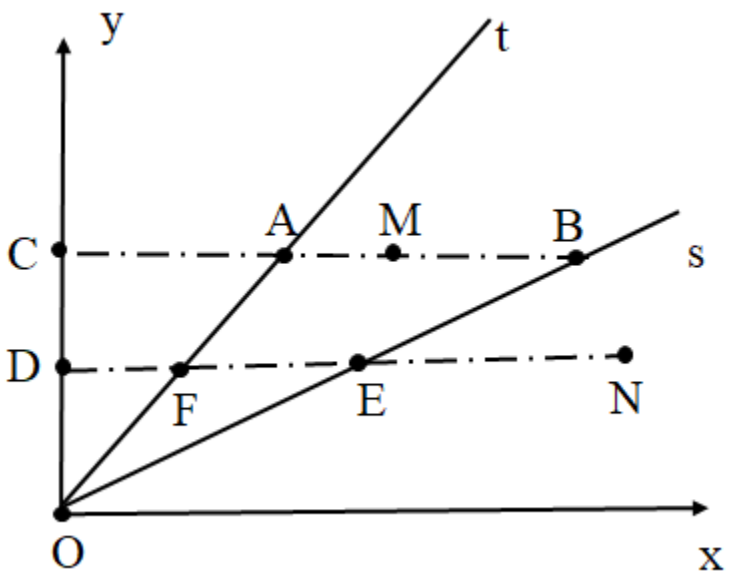


图 7.2: 单投入、单期望产出的投入导向 Malmquist 指数 M_i

上述 Malmquist 指数以谢泼德距离函数为基础，调整方向固定为投入或产出某一维度，难以同时刻画”扩张期望产出、压缩污染排放” 的绿色生产率含义。下一节引入方向距离函数，使调整方向可灵活设定，并据此构造Malmquist-Luenberger 指数与 Luenberger 指标。

7.2 基于方向距离函数的全要素生产率指数

方向距离函数的优势在于能够直接规定” 改善” 的经济方向。对于包含非期望产出的生产率测算，研究者通常关心给定污染约束下的最大产出，也关心在维持或节约投入的同时能否增加期望产出、减少污染排放（Nin, Arndt, Hertel and Preckel, 2003）。第 6 章已给出方向距离函数的四类设定：期望产出扩张与非期望产出压缩同时调整、投入缩减、仅非期望产出压缩，以及投入缩减—期望产出扩张—非期望产出压缩同时调整。其中，前两类方向可构造比值型Malmquist-Luenberger 指数或加法型 Luenberger 指标；综合方向由于同时涉及投入与产出调整，难以构造对应的谢泼德距离函数，通常采用加法型Luenberger 指标。仅沿非期望产出方向调整的情形更适合解释为动态环境绩效，将在 7.4 节单独讨论。

为便于后文叙述，此处统一交代两类指数的判读规则。比值型Malmquist-Luenberger 指数及其分解项以 1 为判读基准：大于 1 表示改善或正向贡献，小于 1 表示退步或负向贡献。加法型 Luenberger 指标及其分解项以 0 为判读基准：大于 0 表示改善或正向贡献，小于 0 表示退步或负向贡献，等于 0 表示无变化。下文涉及的方向向量不同，但同一类指数的判读基准不变；除各节首次出现比值型或加法型指数时略作提示外，不再逐一重述。

7.2.1 期望产出和非期望产出方向同时调整

（一）Malmquist-Luenberger 指数期望产出和非期望产出同时调整时，方向向量设为 $(-g_x, g_y, -g_b) = (0, y, -b)$，即在保持投入不变的同时扩张期望产出并压缩非期望产出。方向距离函数由Chambers, Chung

and Färe (1996) 提出，并在环境生产率框架中由 Chung, Färe and Grosskopf (1997) 加以应用。这一方向最直接对应绿色生产率改善的含义：同样的要素投入需同时创造更多期望产出并减少污染排放。

$$\vec{D}_o\left(x,y,b;0,y,-b\right)=\sup\left\{\beta:(x,y+\beta y,b-\beta b)\in T\right\} \tag{7.11}$$

方向距离函数与谢泼德产出距离函数之间存在一个常被引用的恒等关系。Chung, Färe and Grosskopf (1997) 给出的是：当谢泼德距离沿**好坏产出同向扩张**定义，即 $D_o^{yb}(x,y,b)=\inf\{\theta:(y,b)/\theta\in P(x)\}$，且方向向量取 $g=(0,y,b)$（好坏产出同向）时，有

$$D_o^{yb}(x,y,b)=\frac{1}{1+\overrightarrow{D}_o(x,y,b;0,y,b)} \tag{7.12}$$

需要特别说明的是，式 (7.12) **不适用于**式 (7.1) 的 D_o 与式 (7.11) 的增产减排方向距离函数：式(7.1) 只扩张期望产出而保持 b 不变，式(7.11) 的方向 $(0,y,-b)$ 是" 增产同时减污"，两者的投影点落在前沿的不同位置，因此二者之间不存在 $D_o=1/(1+\beta)$ 这样的恒等式，不能把 (7.1) 代入(7.12) 相互替换。第 6 章的纯产出效率 PEO 与环境DDF 的 $1/(1+\beta^*)$ 也因此不是同一个数。

Malmquist-Luenberger 指数的构造方式是：**类比**标准 Malmquist 指数把距离函数之比换成 $1+\overrightarrow{D}_o(\cdot;0,y,-b)$ 之比，直接定义为下式，而不是由 (7.1) 与 (7.12) 代入导出。令 t 和 $s(t<s)$ 表示相邻两个时期，其计算式为：

$$\mathrm{ML}_t^s=\left[\frac{1+\overrightarrow{D_o^t}\left(x_i^t,y_i^t,b_i^t;0,y_i^t,-b_i^t\right)}{1+\overrightarrow{D_o^t}\left(x_i^s,y_i^s,b_i^s;0,y_i^s,-b_i^s\right)}\times\frac{1+\overrightarrow{D_o^s}\left(x_i^t,y_i^t,b_i^t;0,y_i^t,-b_i^t\right)}{1+\overrightarrow{D_o^s}\left(x_i^s,y_i^s,b_i^s;0,y_i^s,-b_i^s\right)}\right]^{1/2} \tag{7.13}$$

其中，(x_i^s,y_i^s,b_i^s) 和 (x_i^t,y_i^t,b_i^t) 分别表示 s 期和 t 期观测值；方向向量分别取 $g=(0,y_i^s,-b_i^s)$ 和 $g=(0,y_i^t,-b_i^t)$；$\vec{D}_o^s$ 和 $\vec{D}_o^t$ 分别表示以 s 期和 t 期技术集合计算的方向距离函数。

ML 可分解为效率变化 $MLEFFCH$ 和技术变化 $MLTECHCH$：

$$MLEFFCH=\frac{1+\overrightarrow{D_o^t}\left(x_i^t,y_i^t,b_i^t;0,y_i^t,-b_i^t\right)}{1+\overrightarrow{D_o^s}\left(x_i^s,y_i^s,b_i^s;0,y_i^s,-b_i^s\right)} \tag{7.14}$$

$$\mathrm{MLTECHCH}=\left[\frac{1+\overrightarrow{D_o^s}\left(x_i^s,y_i^s,b_i^s;0,y_i^s,-b_i^s\right)}{1+\overrightarrow{D_o^t}\left(x_i^s,y_i^s,b_i^s;0,y_i^s,-b_i^s\right)}\times\frac{1+\overrightarrow{D_o^s}\left(x_i^t,y_i^t,b_i^t;0,y_i^t,-b_i^t\right)}{1+\overrightarrow{D_o^t}\left(x_i^t,y_i^t,b_i^t;0,y_i^t,-b_i^t\right)}\right]^{1/2} \tag{7.15}$$

其中，$MLEFFCH$ 衡量追赶效应，反映 DMU_i 从 t 期到 s 期相对于环境生产前沿的位置变化；$MLTECHCH$ 衡量前沿移动效应，反映环境生产技术边界的变化。就比值型指数而言，指标大于1 表示改善或进步，小于 1 表示退步。

（二）Luenberger 指标第二种做法是采用 Chambers, Färe and Grosskopf (1996) 以及 Briec and Kerstens (2009) 提出的加法型 Luenberger 生产率指标。其计算式为：

$$\begin{aligned}\mathrm{L}_t^s=&0.5\left[\overrightarrow{D_o^t}\left(x_i^t,y_i^t,b_i^t;0,y_i^t,-b_i^t\right)-\overrightarrow{D_o^t}\left(x_i^s,y_i^s,b_i^s;0,y_i^s,-b_i^s\right)\right]\\&+0.5\left[\overrightarrow{D_o^s}\left(x_i^t,y_i^t,b_i^t;0,y_i^t,-b_i^t\right)-\overrightarrow{D_o^s}\left(x_i^s,y_i^s,b_i^s;0,y_i^s,-b_i^s\right)\right]\end{aligned} \tag{7.16}$$

其中，(x_i^s,y_i^s,b_i^s) 和 (x_i^t,y_i^t,b_i^t) 分别为 s 期和 t 期观测值；方向向量分别为 $g=(0,y_i^s,-b_i^s)$ 和 $g=(0,y_i^t,-b_i^t)$；$\vec{D}_o^s$ 和 $\vec{D}_o^t$ 分别表示以 s 期和 t 期生产技术计算的方向距离函数。

按照 Luenberger 生产率指标的加法分解，L 可分解为效率变化 $LEFFCH$ 和技术变化 $LTECHCH$：

$$LEFFCH = \overrightarrow{D_o^t}\left(x_i^t, y_i^t, b_i^t; 0, y_i^t, -b_i^t\right) - \overrightarrow{D_o^s}\left(x_i^s, y_i^s, b_i^s; 0, y_i^s, -b_i^s\right) \tag{7.17}$$

$$\text{LTECHCH} = \left[\left(\overrightarrow{D_o^s}\left(x_i^s, y_i^s, b_i^s; 0, y_i^s, -b_i^s\right) - \overrightarrow{D_o^t}\left(x_i^s, y_i^s, b_i^s; 0, y_i^s, -b_i^s\right)\right)\right] \times \frac{1}{2} + \left[\overrightarrow{D_o^s}\left(x_i^t, y_i^t, b_i^t; 0, y_i^t, -b_i^t\right) - \overrightarrow{D_o^t}\left(x_i^t, y_i^t, b_i^t; 0\right.\right. \tag{7.18}$$

其中，$LEFFCH$ 和 $LTECHCH$ 分别衡量期望产出—非期望产出方向上的效率追赶与技术前沿移动，判读基准同本节开篇说明。

上述期望产出—非期望产出方向同时调整的设定，对应" 扩张产出、压缩排放" 的绿色生产率含义；若研究问题转向投入节约，则需将方向向量改为投入方向，下一节给出相应的Malmquist-Luenberger 指数与 Luenberger 指标。

7.2.2 投入方向

（一）Malmquist-Luenberger 指数当研究问题集中于投入节约能力时，可将方向向量设为 $(-g_x, g_y, -g_b) = (-x, 0, 0)$，表示在期望产出和非期望产出不变时缩减投入。该设定适合分析能源、资本或劳动力约束下的资源配置效率变化：

$$\overrightarrow{D_i}\left(x, y, b; -x, 0,0\right) = \sup\left\{\beta : (x - \beta x, y, b) \in T\right\} \tag{7.19}$$

与前述思路一致，可先利用投入方向距离函数与投入方向谢泼德距离函数之间的关系构造比值型指数。令 $\theta = \frac{1}{1-\beta}$，则 $(1-\beta)x = x/\theta$，且 β 关于 θ 单调递增，故 β 与 θ 的上确界在同一可行点达到：

$$\begin{aligned}\overrightarrow{D_i}(x, y, b; -x, 0, 0) &= \sup\{\beta : (x - \beta x, y, b) \in T\} \\ &= \sup\{\beta : ((1-\beta)x, y, b) \in T\}\end{aligned} \tag{7.20}$$

对应地，θ 的上确界为 $D_i(x, y, b) = \sup\{\theta : (x/\theta, y, b) \in T\}$。若记 $\overrightarrow{D}_i(x, y, b; -x, 0, 0) = \beta^*$，则 $\theta^* = \frac{1}{1-\beta^*}$ 达到 D_i 的上确界，因而二者满足：

$$D_i(x, y, b) = \frac{1}{1 - \overrightarrow{D}_i(x, y, b; -x, 0, 0)} \tag{7.21}$$

据此可将投入方向距离函数纳入 Malmquist 指数框架，得到投入方向的 Malmquist-Luenberger 生产率指数。令 t 和 $s(t < s)$ 表示相邻时期，其计算式为：

$$\text{ML}_t^s = \left[\frac{1 - \overrightarrow{D_i^t}\left(x_i^s, y_i^s, b_i^s; -x_i^s, 0, 0\right)}{1 - \overrightarrow{D_i^t}\left(x_i^t, y_i^t, b_i^t; -x_i^t, 0, 0\right)} \times \frac{1 - \overrightarrow{D_i^s}\left(x_i^s, y_i^s, b_i^s; -x_i^s, 0, 0\right)}{1 - \overrightarrow{D_i^s}\left(x_i^t, y_i^t, b_i^t; -x_i^t, 0, 0\right)}\right]^{1/2} \tag{7.22}$$

ML 可分解为效率变化 $MLEFFCH$ 和技术变化 $MLTECHCH$。由 (7.21) 可知，投入方向上与谢泼德距离函数同口径的效率分数为 $1/D_i = 1 - \overrightarrow{D_i}$，因此分解式中的每一项都必须写成 $1 - \beta$ 的形式（而不是产出方向的 $1 + \beta$）：

$$MLEFFCH = \frac{1 - \overrightarrow{D_i^s}\left(x_i^s, y_i^s, b_i^s; -x_i^s, 0, 0\right)}{1 - \overrightarrow{D_i^t}\left(x_i^t, y_i^t, b_i^t; -x_i^t, 0, 0\right)} \tag{7.23}$$

$$\text{MLTECHCH} = \left[\frac{1-\overrightarrow{D_i^t}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right)}{1-\overrightarrow{D_i^s}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right)} \times \frac{1-\overrightarrow{D_i^t}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)}{1-\overrightarrow{D_i^s}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)}\right]^{1/2} \tag{7.24}$$

其中，(7.23) 的分子与分母分别是 s 期和 t 期各自参照本期技术的投入效率分数，$MLEFFCH>1$ 表示效率追赶；(7.24) 对 s 期和 t 期两个观测点分别比较两期技术下的效率分数，等价于 $\left[\frac{D_i^s(x_i^s,\cdot)}{D_i^t(x_i^s,\cdot)} \times \frac{D_i^s(x_i^t,\cdot)}{D_i^t(x_i^t,\cdot)}\right]^{1/2}$，投入方向的前沿内移（即生产同样产出所需投入下降）时该值大于1。两式相乘即回到 (7.22)。投入方向的 ML 指数以比值型刻画投入节约的跨期变化；若研究者更关注改善的绝对幅度，可采用下文的加法型Luenberger 指标。

（二）Luenberger 指标投入方向也可采用加法型 Luenberger 指标进行测度，其计算式为：

$$\begin{aligned}\text{L}_t^s = 0.5&\left[\overrightarrow{D_i^t}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)-\overrightarrow{D_i^t}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right)\right]\\ +0.5&\left[\overrightarrow{D_i^s}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)-\overrightarrow{D_i^s}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right)\right]\end{aligned} \tag{7.25}$$

其中，(x_i^s,y_i^s,b_i^s) 和 (x_i^t,y_i^t,b_i^t) 分别为 s 期和 t 期观测值；方向向量分别为 $g=(-x_i^s,0,0)$ 和 $g=(-x_i^t,0,0)$；$\vec{D}_i^s$ 和 $\vec{D}_i^t$ 分别表示以 s 期和 t 期生产技术计算的投入方向距离函数。

L 可分解为效率变化 $LEFFCH$ 和技术变化 $LTECHCH$：

$$LEFFCH = \overrightarrow{D_i^t}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)-\overrightarrow{D_i^s}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right) \tag{7.26}$$

$$\text{LTECHCH} = \frac{1}{2}\left[\overrightarrow{D_i^s}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right)-\overrightarrow{D_i^t}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right)\right]+\frac{1}{2}\left[\overrightarrow{D_i^s}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)-\overrightarrow{D_i^t}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)\right] \tag{7.27}$$

其中，$LEFFCH$ 和 $LTECHCH$ 分别衡量投入方向上的效率追赶与投入节约技术前沿的移动，判读基准同本节开篇说明。

上述两类设定各自沿单一方向调整。若研究问题要求投入节约、产出扩张与污染压缩同时成立，则需采用综合方向向量，下一节给出相应的Luenberger 指标。

7.2.3 投入、期望产出、非期望产出同时调整

当研究需要同时缩减投入、扩张期望产出并压缩非期望产出时，可设方向向量为 $(-g_x,g_y,-g_b)=(-x,y,-b)$。这一综合方向对应更严格的环境生产率改进标准：同一决策单元只有在投入节约、产出扩张和污染压缩三个维度上共同改善，才会被评价为具有更高的动态绩效。相应方向距离函数定义如下（Chung et al., 1997）：

$$\vec{D}_g\left(x,y,b;-g_x,g_y,-g_b\right)=\sup\left\{\beta:\left(x-\beta g_x,y+\beta g_y,b-\beta g_b\right)\in T\right\} \tag{7.28}$$

在该综合方向下，投入与产出同时发生调整，难以直接构造与单一投入或产出方向相对应的谢泼德距离函数。因此，本节采用加法型Luenberger 指标刻画全要素生产率变化：

$$\begin{aligned}\text{L}_t^s = 0.5&\left[\overrightarrow{D_g^t}\left(x_i^t,y_i^t,b_i^t;-x_i^t,y_i^t,-b_i^t\right)-\overrightarrow{D_g^t}\left(x_i^s,y_i^s,b_i^s;-x_i^s,y_i^s,-b_i^s\right)\right]\\ +0.5&\left[\overrightarrow{D_g^s}\left(x_i^t,y_i^t,b_i^t;-x_i^t,y_i^t,-b_i^t\right)-\overrightarrow{D_g^s}\left(x_i^s,y_i^s,b_i^s;-x_i^s,y_i^s,-b_i^s\right)\right]\end{aligned} \tag{7.29}$$

其中，(x_i^s,y_i^s,b_i^s) 和 (x_i^t,y_i^t,b_i^t) 分别为 s 期和 t 期观测值；方向向量 $(-g_x,g_y,-g_b)$ 分别取 $(-x_i^s,y_i^s,-b_i^s)$ 和 $(-x_i^t,y_i^t,-b_i^t)$；$\vec{D}_g^s$ 和 $\vec{D}_g^t$ 分别表示以 s 期和 t 期生产技术计算的综合方向距离函数。

按照 Luenberger 生产率指标的加法分解，L 可分解为效率变化 $LEFFCH$ 和技术变化 $LTECHCH$：

$$LEFFCH = \overrightarrow{D_g^t}\left(x_i^t, y_i^t, b_i^t; -x_i^t, y_i^t, -b_i^t\right) - \overrightarrow{D_g^s}\left(x_i^s, y_i^s, b_i^s; -x_i^s, y_i^s, -b_i^s\right) \tag{7.30}$$

$$\text{LTECHCH} = \frac{1}{2}\left[\overrightarrow{D_g^s}\left(x_i^s, y_i^s, b_i^s; -x_i^s, y_i^s, -b_i^s\right) - \overrightarrow{D_g^t}\left(x_i^s, y_i^s, b_i^s; -x_i^s, y_i^s, -b_i^s\right)\right] + \frac{1}{2}\left[\overrightarrow{D_g^s}\left(x_i^t, y_i^t, b_i^t; -x_i^t, y_i^t, -b_i^t\right) - \overrightarrow{D_g^t}\left(x_i^t, y_i^t, \right.\right. \tag{7.31}$$

其中，$LEFFCH$ 和 $LTECHCH$ 分别衡量综合方向上的效率追赶与技术前沿移动，判读基准同 7.2 节开篇说明。至此，7.2 节的三类方向设定均沿用径向调整假设；当研究问题要求不同要素按差异化幅度调整时，需进一步放松该假设，下一节给出非径向扩展。

7.3 基于非径向方向距离函数的全要素生产率指标

传统 Malmquist 生产率指数和径向方向距离函数在方向选择与调整比例上均存在限制：一方面，投入缩减与产出扩张通常需要分别设定，难以同时刻画多方向调整；另一方面，径向模型要求同一方向上的各要素按相同比例变化。现实中，不同投入要素的节约空间、不同污染物的治理难度以及不同期望产出的扩张能力往往并不相同。若研究问题需要允许不同投入、期望产出或非期望产出以差异化幅度调整，则应采用非径向方向距离函数，并使用加法型Luenberger 指标进行跨期比较。

与 7.2 节相对应，本节依次给出期望产出—非期望产出方向（7.3.1）、投入方向（7.3.2）以及综合方向（7.3.3）下的非径向Luenberger 指标；三者的构造逻辑一致，差异仅在于方向向量与非径向松弛的设定。

7.3.1 期望产出和非期望产出方向同时调整

期望产出和非期望产出同时调整的非径向方向距离函数以 $(-g_x, g_y, -g_b) = (0, y, -b)$ 为方向向量（Mahlberg and Sahoo, 2011）：

$$\vec{D}_o^{nr}\left(x, y, b; -g_x, g_y, -g_b\right) = \max\left\{w'\beta : (x, y, b) + (0, y, -b) \cdot \text{diag}(\beta) \in T\right\} \tag{7.32}$$

其中，β 为由各方向调整比例构成的向量，w 为相应权重向量，$\text{diag}(\beta)$ 表示将各维度调整比例写入对角矩阵。非径向设定允许不同期望产出和非期望产出维度具有不同调整幅度。基于该非径向方向距离函数，可构造加法型Luenberger 生产率指标：

$$\begin{aligned} \text{L}_t^s = {} & 0.5\left[\overrightarrow{D_o^{nr,t}}\left(x_i^t, y_i^t, b_i^t; 0, y_i^t, -b_i^t\right) - \overrightarrow{D_o^{nr,t}}\left(x_i^s, y_i^s, b_i^s; 0, y_i^s, -b_i^s\right)\right] \\ & +0.5\left[\overrightarrow{D_o^{nr,s}}\left(x_i^t, y_i^t, b_i^t; 0, y_i^t, -b_i^t\right) - \overrightarrow{D_o^{nr,s}}\left(x_i^s, y_i^s, b_i^s; 0, y_i^s, -b_i^s\right)\right] \end{aligned} \tag{7.33}$$

其中，(x_i^s, y_i^s, b_i^s) 和 (x_i^t, y_i^t, b_i^t) 分别为 s 和 t 时期的观测值；$(-g_x, g_y, -g_b) = (0, y_i^s, -b_i^s)$ 和 $(-g_x, g_y, -g_b) = (0, y_i^t, -b_i^t)$ 分别为 s 和 t 时期的方向向量；$\vec{D}_o^{nr,s}$ 和 $\vec{D}_o^{nr,t}$ 分别表示以 s 期和 t 期生产技术计算的期望产出—非期望产出方向非径向距离函数值。

按照 Luenberger 生产率指标的加法分解，L 可分解为效率变化 $LEFFCH$ 和技术变化 $LTECHCH$：

$$LEFFCH = \overrightarrow{D_o^{nr,t}}\left(x_i^t, y_i^t, b_i^t; 0, y_i^t, -b_i^t\right) - \overrightarrow{D_o^{nr,s}}\left(x_i^s, y_i^s, b_i^s; 0, y_i^s, -b_i^s\right) \tag{7.34}$$

$$\text{LTECHCH}=\frac{1}{2}\left[\overrightarrow{D_o^{nr,s}}\left(x_i^s,y_i^s,b_i^s;0,y_i^s,-b_i^s\right)-\overrightarrow{D_o^{nr,t}}\left(x_i^s,y_i^s,b_i^s;0,y_i^s,-b_i^s\right)\right]+\frac{1}{2}\left[\overrightarrow{D_o^{nr,s}}\left(x_i^t,y_i^t,b_i^t;0,y_i^t,-b_i^t\right)-\overrightarrow{D_o^{nr,t}}\left(x_i^t\right.\right. \tag{7.35}$$

其中，$LEFFCH$ 和 $LTECHCH$ 分别衡量相应方向上的效率追赶与技术前沿移动，判读基准同 7.2 节开篇说明。

将非径向设定应用于投入方向，可刻画不同投入要素的差异化节约，下一节给出相应指标。

7.3.2 投入方向

投入方向的非径向方向距离函数以 $(-g_x,g_y,-g_b)=(-x,0,0)$ 为方向向量：

$$\vec{D}_i^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)=\max\left\{w'\beta:(x,y,b)+(-x,0,0)\cdot\text{diag}(\beta)\in T\right\} \tag{7.36}$$

基于该非径向方向距离函数，可构造加法型 Luenberger 生产率指标：

$$\begin{aligned}\text{L}_t^s=0.5&\left[\overrightarrow{D_i^{nr,t}}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)-\overrightarrow{D_i^{nr,t}}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right)\right]\\+0.5&\left[\overrightarrow{D_i^{nr,s}}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)-\overrightarrow{D_i^{nr,s}}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right)\right]\end{aligned} \tag{7.37}$$

其中，(x_i^s,y_i^s,b_i^s) 和 (x_i^t,y_i^t,b_i^t) 分别为 s 和 t 时期的观测值；$(-g_x,g_y,-g_b)=(-x_i^s,0,0)$ 和 $(-g_x,g_y,-g_b)=(-x_i^t,0,0)$ 分别为 s 和 t 时期的方向向量；$\vec{D}_i^{nr,s}$ 和 $\vec{D}_i^{nr,t}$ 分别表示以 s 时期和 t 时期生产技术计算的方向距离函数值。

按照 Luenberger 生产率指标的加法分解，L 可分解为效率变化 $LEFFCH$ 和技术变化 $LTECHCH$：

$$LEFFCH=\overrightarrow{D_i^{nr,t}}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)-\overrightarrow{D_i^{nr,s}}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right) \tag{7.38}$$

$$\text{LTECHCH}=\frac{1}{2}\left[\overrightarrow{D_i^{nr,s}}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right)-\overrightarrow{D_i^{nr,t}}\left(x_i^s,y_i^s,b_i^s;-x_i^s,0,0\right)\right]+\frac{1}{2}\left[\overrightarrow{D_i^{nr,s}}\left(x_i^t,y_i^t,b_i^t;-x_i^t,0,0\right)-\overrightarrow{D_i^{nr,t}}\left(x_i^t,\right.\right. \tag{7.39}$$

其中，$LEFFCH$ 和 $LTECHCH$ 分别衡量相应方向上的效率追赶与技术前沿移动，判读基准同 7.2 节开篇说明。

若研究问题要求投入节约、产出扩张与污染压缩同时以差异化幅度调整，则需采用综合方向下的非径向设定，下一节给出相应指标。

7.3.3 投入、期望产出、非期望产出同时调整

投入、期望产出和非期望产出同时调整时，非径向方向距离函数以 $(-g_x,g_y,-g_b)=(-x,y,-b)$ 为方向向量：

$$\vec{D}_g^{nr}\left(x,y,b;-g_x,g_y,-g_b\right)=\max\left\{w'\beta:(x,y,b)+(-x,y,-b)\cdot\text{diag}(\beta)\in T\right\} \tag{7.40}$$

基于该非径向方向距离函数，可构造加法型 Luenberger 生产率指标：

$$\begin{aligned}\text{L}_t^s=0.5&\left[\overrightarrow{D_g^{nr,t}}\left(x_i^t,y_i^t,b_i^t;-x_i^t,y_i^t,-b_i^t\right)-\overrightarrow{D_g^{nr,t}}\left(x_i^s,y_i^s,b_i^s;-x_i^s,y_i^s,-b_i^s\right)\right]\\+0.5&\left[\overrightarrow{D_g^{nr,s}}\left(x_i^t,y_i^t,b_i^t;-x_i^t,y_i^t,-b_i^t\right)-\overrightarrow{D_g^{nr,s}}\left(x_i^s,y_i^s,b_i^s;-x_i^s,y_i^s,-b_i^s\right)\right]\end{aligned} \tag{7.41}$$

其中，(x_i^s, y_i^s, b_i^s) 和 (x_i^t, y_i^t, b_i^t) 分别为 s 和 t 时期的观测值；$(-g_x, g_y, -g_b) = (-x_i^s, y_i^s, -b_i^s)$ 和 $(-g_x, g_y, -g_b) = (-x_i^t, y_i^t, -b_i^t)$ 分别为 s 和 t 时期的方向向量；$\vec{D}_g^{nr,s}$ 和 $\vec{D}_g^{nr,t}$ 分别表示以 s 期和 t 期生产技术计算的综合方向非径向距离函数值。

按照 Luenberger 生产率指标的加法分解，L 可分解为效率变化 $LEFFCH$ 和技术变化 $LTECHCH$：

$$LEFFCH = \overrightarrow{D_g^{nr,t}}\left(x_i^t, y_i^t, b_i^t; -x_i^t, y_i^t, -b_i^t\right) - \overrightarrow{D_g^{nr,s}}\left(x_i^s, y_i^s, b_i^s; -x_i^s, y_i^s, -b_i^s\right) \tag{7.42}$$

$$\text{LTECHCH} = \frac{1}{2}\left[\overrightarrow{D_g^{nr,s}}\left(x_i^s, y_i^s, b_i^s; -x_i^s, y_i^s, -b_i^s\right) - \overrightarrow{D_g^{nr,t}}\left(x_i^s, y_i^s, b_i^s; -x_i^s, y_i^s, -b_i^s\right)\right] + \frac{1}{2}\left[\overrightarrow{D_g^{nr,s}}\left(x_i^t, y_i^t, b_i^t; -x_i^t, y_i^t, -b_i^t\right) - \right. \tag{7.43}$$

其中，$LEFFCH$ 和 $LTECHCH$ 分别衡量相应方向上的效率追赶与技术前沿移动，判读基准同 7.2 节开篇说明。

以上设定使用综合方向向量 $(-g_x, g_y, -g_b) = (-x, y, -b)$。综合方向下仅Luenberger 指标适用，原因见 7.2.3 节。在具体应用中，可根据研究问题将方向向量的某些维度设为0；只要跨期比较中方向向量口径保持一致，Luenberger 指标的构造逻辑不变。

7.4 动态环境绩效

前述指标从全要素生产率角度比较投入、期望产出和非期望产出的共同变化。本节聚焦污染排放压缩能力本身，将非期望产出方向单独作为评价方向。此时，动态指标刻画的经济含义是：在给定投入和期望产出条件下，污染排放前沿是否向更清洁方向移动，以及各决策单元是否向该前沿追赶。本节先基于非期望产出方向谢泼德距离函数给出动态环境绩效指标，再用方向距离函数给出可相互验证的第二种实现。

7.4.1 基于非期望产出导向谢泼德距离函数的动态环境绩效

若研究重点是污染排放压缩能力的动态变化，可直接基于非期望产出方向构造动态环境绩效指标。非期望产出方向的谢泼德距离函数（Färe, Grosskopf, Lovell and Pasurka, 1989）定义为：

$$D_b(x, y, b) = \sup\{\theta : (x, y, b/\theta) \in T\} \tag{7.44}$$

令 t 和 $s(t < s)$ 表示相邻时期。$D_b^t(x^t, y^t, b^t)$ 与 $D_b^s(x^t, y^t, b^t)$ 分别表示 t 期观测值在 t 期和 s 期技术集合下的非期望产出方向谢泼德距离函数；$D_b^t(x^s, y^s, b^s)$ 与 $D_b^s(x^s, y^s, b^s)$ 分别表示 s 期观测值在 t 期和 s 期技术集合下的距离函数。相应动态环境绩效定义为：

$$\text{MEPI}_t^s = \left[\frac{D_b^t\left(x_i^t, y_i^t, b_i^t\right)}{D_b^t\left(x_i^s, y_i^s, b_i^s\right)} \times \frac{D_b^s\left(x_i^t, y_i^t, b_i^t\right)}{D_b^s\left(x_i^s, y_i^s, b_i^s\right)}\right]^{1/2} \tag{7.45}$$

式中，MEPI_t^s 衡量 DMU_i 从 t 期到 s 期的环境绩效变化。$\text{MEPI}_t^s > 1$ 表示环境绩效改善，$\text{MEPI}_t^s < 1$ 表示环境绩效下降。MEPI 可分解为效率变化 $MEFFCH$ 和技术变化 $MTECHCH$：

$$MEFFCH = \frac{D_b^t\left(x_i^t, y_i^t, b_i^t\right)}{D_b^s\left(x_i^s, y_i^s, b_i^s\right)} \tag{7.46}$$

$$\text{MTECHCH} = \left[\frac{D_b^s\left(x_i^s, y_i^s, b_i^s\right)}{D_b^t\left(x_i^s, y_i^s, b_i^s\right)} \times \frac{D_b^s\left(x_i^t, y_i^t, b_i^t\right)}{D_b^t\left(x_i^t, y_i^t, b_i^t\right)}\right]^{1/2} \tag{7.47}$$

其中，$MEFFCH$ 衡量效率追赶效应，反映 DMU_i 从 t 期到 s 期相对于环境生产前沿的位置变化；$MTECHCH$ 衡量环境技术前沿移动效应。作为比值型指数，二者大于 1 表示正向贡献，小于1 表示负向贡献。

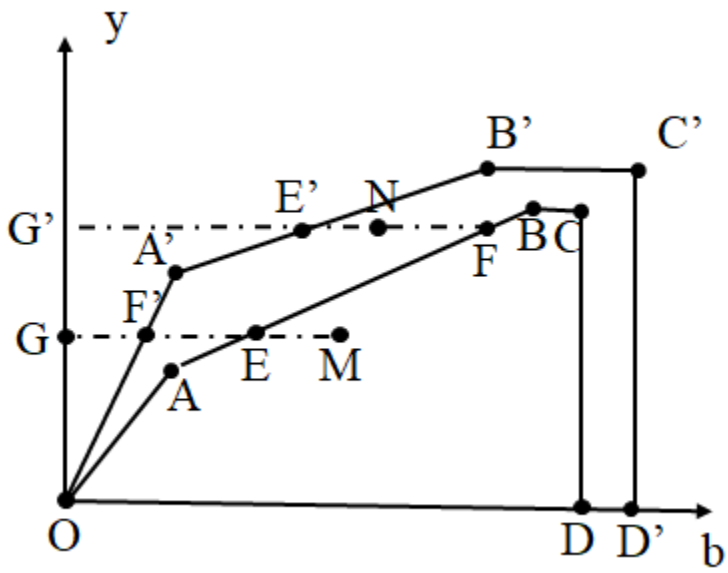


图 7.3: 单期望产出、单非期望产出的 $MEPI$

图 7.3 给出了单期望产出、单非期望产出情形下 $MEPI$ 的几何含义。区域 $OABCD$ 和 $OA'B'C'D'$ 分别表示 t 期和 s 期环境 DEA 技术，点 M 和 N 表示同一决策单元在两个时期的产出组合。效率变化为 $\frac{GM/GE}{G'N/G'E'}$，技术变化为 $\left[\frac{GM/GF'}{GM/GE}\cdot\frac{G'N/G'E'}{G'N/G'F}\right]^{1/2}=\left[\frac{GE}{GF'}\cdot\frac{G'F}{G'E'}\right]^{1/2}$，反映给定期望产出水平下污染排放前沿的移动。

上述 MEPI 基于谢泼德距离函数。若改用方向距离函数表示非期望产出方向，可与 7.2 节的 Malmquist-Luenberger 框架衔接，并在经验测算中提供可相互验证的第二种实现。

7.4.2 基于非期望产出导向方向距离函数的动态环境绩效

非期望产出方向也可用方向距离函数表示。此时方向向量为 $(-g_x, g_y, -g_b)=(0,0,-b)$，表示在投入和期望产出不变时压缩非期望产出：

$$\vec{D}_b\left(x,y,b;0,0,-b\right)=\sup\left\{\beta:(x,y,b-\beta b)\in T\right\} \tag{7.48}$$

非期望产出方向距离函数与非期望产出方向谢泼德距离函数之间存在可转换关系。

当方向向量为 $(-g_x, g_y, -g_b)=(0,0,-b)$ 时，上述关系可推导如下。令 $\theta=\frac{1}{1-\beta}$，则 $(1-\beta)b=b/\theta$，且 β 关于 θ 单调递增，故 β 取上确界与 θ 取上确界在同一可行点达到：

$$\begin{aligned}\overrightarrow{D_b}(x,y,b;0,0,-b)&=\sup\{\beta:(x,y,b-\beta b)\in T\}\\&=\sup\{\beta:(x,y,(1-\beta)b)\in T\}\end{aligned} \tag{7.49}$$

对应地，θ 的上确界为 $D_b(x,y,b)=\sup\{\theta:(x,y,b/\theta)\in T\}$。由此可知，若记 $\overrightarrow{D}_b(x,y,b;0,0,-b)=\beta^*$，则对应的 $\theta^*=\frac{1}{1-\beta^*}$ 即达到 D_b 的上确界，因此：

$$D_b(x,y,b)=\frac{1}{1-\overrightarrow{D}_b(x,y,b;0,0,-b)} \tag{7.50}$$

据此可将非期望产出方向距离函数纳入 Malmquist-Luenberger 框架，构造方向距离函数形式的动态环境绩效指数。令 t 和 $s(t<s)$ 表示相邻时期，计算式为：

$$\mathrm{ML}_t^s=\left[\frac{1-\overrightarrow{D}_b^t\left(x_i^s,y_i^s,b_i^s;0,0,-b_i^s\right)}{1-\overrightarrow{D}_b^t\left(x_i^t,y_i^t,b_i^t;0,0,-b_i^t\right)}\times\frac{1-\overrightarrow{D}_b^s\left(x_i^s,y_i^s,b_i^s;0,0,-b_i^s\right)}{1-\overrightarrow{D}_b^s\left(x_i^t,y_i^t,b_i^t;0,0,-b_i^t\right)}\right]^{1/2} \tag{7.51}$$

其中，(x_i^s,y_i^s,b_i^s) 和 (x_i^t,y_i^t,b_i^t) 分别为 s 和 t 时期的观测值；$g=(0,0,-b_i^s)$ 和 $g=(0,0,-b_i^t)$ 分别为 s 和 t 时期的方向向量；$\vec{D}_b^s$ 和 $\vec{D}_b^t$ 分别表示以 s 时期和 t 时期生产技术计算的方向距离函数值。

ML 可分解为效率变化 $MLEFFCH$ 和技术变化 $MLTECHCH$：

$$MLEFFCH=\frac{1-\overrightarrow{D}_b^s\left(x_i^s,y_i^s,b_i^s;0,0,-b_i^s\right)}{1-\overrightarrow{D}_b^t\left(x_i^t,y_i^t,b_i^t;0,0,-b_i^t\right)} \tag{7.52}$$

$$\mathrm{MLTECHCH}=\left[\frac{1-\overrightarrow{D}_b^t\left(x_i^s,y_i^s,b_i^s;0,0,-b_i^s\right)}{1-\overrightarrow{D}_b^s\left(x_i^s,y_i^s,b_i^s;0,0,-b_i^s\right)}\times\frac{1-\overrightarrow{D}_b^t\left(x_i^t,y_i^t,b_i^t;0,0,-b_i^t\right)}{1-\overrightarrow{D}_b^s\left(x_i^t,y_i^t,b_i^t;0,0,-b_i^t\right)}\right]^{1/2} \tag{7.53}$$

其中，$MLEFFCH$ 衡量非期望产出方向上的效率追赶，$MLTECHCH$ 衡量环境技术前沿移动。作为比值型指数，分解项大于 1 表示动态环境绩效改善的正向来源，小于 1 表示负向来源。上述指标体系已从理论层面刻画了动态环境绩效的构造逻辑；要将其用于实证比较，还需明确样本、变量、方向向量与技术假设的可复现设定，下一节转入测算实现。

7.5 包含非期望产出生产率模型的Python实现

前文已分别借助谢泼德距离函数、方向距离函数与非径向方向距离函数，从理论层面给出包含非期望产出的各类动态生产率指标及其效率变化—技术变化分解。这些指标在计算方法上最终都转化为一组线性规划：以环境生产技术集合的弱可处置性约束为基底，构造相邻时期的跨期距离函数或方向距离函数并求其最大投影。本章的案例测算即围绕这组规划展开。进入经验研究后，理论口径必须落实为可复现的样本结构、变量集合、方向向量、技术参照集、规模报酬假设与指数分解口径；这些设定在跨模型、跨地区之间保持一致，结果才具有可比性。

`deabook` 以 `DEAweak` 模块承载本章所需的弱可处置性动态环境绩效测算。该模块下，`DEAweak.dep` 对应 §7.4.1 中基于非期望产出导向谢泼德距离函数的动态环境绩效指数（MEPI），其经济含义是给定投入与期望产出时污染排放前沿的跨期移动与单元追赶；`DEAweak.depddf` 对应 §7.4.2 中基于方向距离函数的同口径指数（ML），二者在方向固定为非期望产出压缩时给出一致的比值型结论，可互为验证；`DEAweak.lddf` 对应 §7.2 与 §7.3 中基于方向距离函数的加性 Luenberger 指标，侧重以绝对幅度刻画” 增产同时减污” 的绿色生产率变化。三者共享同一求解引擎（`deabook.mqcore`），均支持当期、序列、窗口与全局四类技术参照集（`tech=CONTEMPORARY/SEQUENTIAL/WINDOW/GLOBAL`），并通过统一的 `.optimize(email, solver)` 接口返回结果。下文依” 样本结构—方向与技术口径—返回列对应—接口调用” 的顺序展开，以保证理论口径与代码实现的一一对应。

7.5.1 面板数据的样本结构与变量设定

动态模型要求数据同时包含决策单元标识和年份变量，以便识别同一国家或省份在相邻年份之间的变化。在程序中，`id` 对应决策单元列名，`year` 对应时间变量。经验应用中还应先检查样本是否形成平衡面板、年份是否连续、投入和产出变量是否为非负值；否则，相邻期指数的经济含义和可计算性都会受到影响。

```
from deabook import data_path, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook import DEAweak, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook.constant import RTS_CRS, RTS_VRS1, RTS_VRS2, OPT_LOCAL
import pandas as pd
```

本章案例设定投入向量为资本（K）、劳动（L）和能源消费（E），期望产出为GDP 或地区生产总值（Y），非期望产出为二氧化碳排放（CO2）。在程序中写作 `sent="K L E=Y:CO2"`：等号左侧为投入变量，等号右侧冒号前为期望产出，冒号后为非期望产出。

表 7.1: 动态环境绩效测算的主要设定

设定	适用函数	本章常用设置	含义
`data`	`DEAweak.dep` / `depddf` / `lddf`	`DataFrame`	面板数据，包含投入、期望产出、非期望产出、DMU 标识和年份
`id`	三者通用	`dmuname`	DMU 标识列，用于匹配同一国家或省份的跨期观测
`year`	三者通用	`'year'`	时间变量，模型按年份顺序计算相邻期指数
`sent`	三者通用	`"K L E=Y:CO2"`	K、L、E 为投入，Y 为期望产出，CO2 为非期望产出

设定	适用函数	本章常用设置	含义
rts	三者通用	RTS_CRS、RTS_VRS1、RTS_VRS2	分别表示规模报酬不变、规模报酬可变且相同减排因子、规模报酬可变且不同减排因子
tech	三者通用	CONTEMPORARY、SEQUENTIAL、WINDOW、GLOBAL	分别表示当期、序列、窗口、全局技术参照集
orient	lddf	"oo"、"io"、"both"	Luenberger 指标的方向设定；dep/depddf 方向固定为非期望产出导向，不接受该参数
solver	三者通用	"highs"（或 "mosek"）	运行环境参数，用于指定求解器
email	三者通用	OPT_LOCAL	运行环境参数，表示本地求解

表 7.1 汇总了动态环境绩效测算的全部参数。其中规模报酬与减排因子设定共同决定环境生产前沿的形状与可计算范围，下一节说明这些设定的经济含义及本章口径。需要注意的是，dep 与 depddf 的方向向量在内部固定为非期望产出导向（投入与期望产出保持不变，仅压缩CO2），因此不再通过 gx/gy/gb 暴露；lddf 则通过 orient 选择方向。

7.5.2 方向向量、技术参照集与环境绩效分解口径

方向向量刻画单期技术中的可调整方向，动态指数则比较相邻时期的距离函数或方向距离函数变化。本章 dep 与 depddf 在内部将方向向量仅指向非期望产出维度：理论上对应 $-g_b$，即在投入和期望产出不变时压缩排放。lddf 的方向由 orient 指定，可覆盖期望产出—非期望产出方向（oo）、投入方向（io）或综合方向（both）。规模报酬和减排因子设定共同决定环境生产前沿形状；技术参照集设定则决定采用分期前沿、累积前沿、滑动窗口前沿还是全局前沿。表 7.2 汇总了本章涉及的方向、技术集与指数分解口径。

表 7.2: 动态环境绩效模型的方向、技术集与指数设定

设定	本章常用值	经济含义	本章口径
非期望产出方向（内部固定）	dep/depddf 均为 gx=0, gy=0, gb=1	投入和期望产出不变，考察 CO2 压缩能力的跨期变化；gb=1 在理论上对应 $-g_b$	dep/depddf 默认口径
orient="oo"	期望产出—非期望产出方向	同时扩张期望产出并压缩污染排放	可用于 lddf 扩展测算
orient="io"	投入方向	沿投入方向调整并压缩污染排放	可用于 lddf 扩展测算
orient="both"	综合方向	投入、期望产出、非期望产出同时调整	lddf 综合方向口径
RTS_CRS	规模报酬不变	所有 DMU 面临同一规模技术	基准模型
RTS_VRS1	规模报酬可变、相同减排因子	不同规模下允许纯技术效率差异，但污染减排因子相同	同质弱可处置性模型
RTS_VRS2	规模报酬可变、不同减排因子	允许不同 DMU 的污染处置难度不同	异质弱可处置性模型
tech=CONTEMPORARY	当期技术	分期构造前沿，并可分解效率变化和技术变化	本章主要案例
tech=SEQUENTIAL	序列技术	截至各期的累积前沿，反映技术沿时间单向累积	稳健性比较
tech=WINDOW	窗口技术	以各期为中心的滑动窗口前沿，缓解单期波动	稳健性比较
tech=GLOBAL	全局技术	所有年份共同构造一个参考前沿	全局指数与 Pastor-Lovell 分解

本章案例主要使用非期望产出方向，并采用 tech=CONTEMPORARY 构造相邻时期的分期前沿，从而分解动态环境绩效变化。选择当期技术的原因在于，本章关注的是相邻年份环境生产前沿的移动和决策单元的效率追赶；若研究重点转向长期共同前沿下的相对位置变化，则可采用 tech=GLOBAL（全局 Malmquist，Pastor & Lovell, 2005），或采用 tech=SEQUENTIAL/tech=WINDOW 进行稳健性检验。在样本、方向向量与技术参照集均已明确后，测算结果将以一系列返回列呈现；正确解读这些列的理论口径，是后续均值统计与跨实现比较的前提。

7.5.3 理论指标与测算返回列的口径对应

测算结果包括距离函数值、动态环境绩效指数及其分解项，并与决策单元—年份观测一一对应。理论符号和软件返回列名并非完全相同：例如，理论中的 $MEPI$ 或 ML 在比值型实现中均以 MQ 形式返回。整

理结果时需要先确认返回列的理论口径，再进行跨年均值、国家或省份排序和图表展示，避免把距离函数值、总指数和分解项混作同一指标。表 7.3 汇总了本章使用到的主要结果列及其经济含义。

表 7.3: 理论指标与 deabook 返回列的对应关系

理论口径	软件函数	主要返回列	指标解释
基于 D_b 的动态环境绩效（§7.4.1）	`DEAweak.dep`, `CONTEMPORARY`	`D11`、`D12`、`D21`	分别表示当前观测—当前前沿、当前观测—相邻上期前沿、相邻下期观测—当前前沿等跨期距离项
$MEPI$	`DEAweak.dep`, `CONTEMPORARY`	`MQ`	动态环境绩效指数，大于 1 表示环境绩效提高
$MEFFCH$、$MTECHCH$	`DEAweak.dep`, `CONTEMPORARY`	`MEFFCH`、`MTECHCH`	分别表示效率变化和技术变化，满足 $MQ = MEFFCH \times MTECHCH$
`DEAweak.dep`, `GLOBAL`	`GLOBAL`	`D11`、`mqpi`、`MEFFCH`、`MTECHCH`	全局技术下的距离值、全局指数及 Pastor-Lovell 分解（§3.4）
基于 $\vec{D}_b$ 的 ML（§7.4.2）	`DEAweak.depddf`, `CONTEMPORARY`	`D11`、`D12`、`D21`、`MQ`、`MEFFCH`、`MTECHCH`	使用方向距离函数转换后的比值型 Malmquist-Luenberger 指数及其分解；返回列与 `dep` 完全一致，便于对照
`DEAweak.depddf`, `GLOBAL`	`GLOBAL`	`D11`、`mqpi`、`MEFFCH`、`MTECHCH`	全局技术下的方向距离函数动态指标及 Pastor-Lovell 分解
L、$LEFFCH$、$LTECHCH$	`DEAweak.lddf`	`D11`、`D12`、`D21`、`L`、`LEFFCH`、`LTECHCH`	加性 Luenberger 指标及其分解，满足 $L = LEFFCH + LTECHCH$，以 0 为判读基准；`GLOBAL` 下 `LTECHCH=0`、`L=LEFFCH`

由于跨期指数需要相邻期观测，首期通常没有上一期可供比较；涉及下一期技术参照的距离项在末期

也可能为空。后续表格展示的2017—2019 年 MQ、MEFFCH 和 MTECHCH，实质上对应 2016—2017、2017—2018 和 2018—2019 三个相邻期指数。表 7.4 汇总了返回列的方向后缀规则。

表 7.4: 动态模型返回列的方向后缀规则

情形	列名规则	示例	说明
dep / depddf（非期望产出导向）	无方向后缀	D11、MQ、MEFFCH、MTECHCH	方向在内部固定为非期望产出导向
lddf, orient="oo"	无方向后缀	D11、L、LEFFCH、LTECHCH	期望产出—非期望产出方向
lddf, orient="io"	无方向后缀	D11、L、LEFFCH、LTECHCH	投入方向
lddf, orient="both"	无方向后缀	D11、L、LEFFCH、LTECHCH	综合方向；加性指标使用目标函数差分，不按方向端拆列
tech=GLOBAL	仅保留全局距离和全局指数	D11、mqpi（比值型）或 L/LEFFCH/LTECHCH（加性）	比值型不返回 D12、D21 及当期技术分解项；加性下 LTECHCH=0

在整理实证结果时，本章主要保留 year、DMU 名称、MQ、MEFFCH 和 MTECHCH（或 L、LEFFCH、LTECHCH）。若采用全局技术，应改用 mqpi 列作为全局指数，并同时报告 MEFFCH 与 MTECHCH 的 Pastor-Lovell 分解。

7.5.4 测算接口的调用与结果获取

在样本、变量、方向向量、技术参照集与返回列口径均已明确后，即可调用模型并经由 .optimize(email, solver) 取得结果。下面依次给出三类接口的调用形式：DEAweak.dep 基于谢泼德距离函数（§7.4.1），DEAweak.depddf 基于方向距离函数（§7.4.2），DEAweak.lddf 则给出加性 Luenberger 分解。由于 dep 与 depddf 的方向已在内部固定为非期望产出导向，调用时无需再传入方向向量；lddf 通过 orient 选择方向。

DEAweak.dep（谢泼德距离函数，§7.4.1）：

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
# CRS + 当期技术
res = DEAweak.dep(
    data, id=dmuname, year='year',
    sent="K L E=Y:CO2", rts=RTS_CRS, tech=CONTEMPORARY,
).optimize(OPT_LOCAL, "highs")

# VRS1 + 当期技术
```

```
res = DEAweak.dep(
    data, id=dmuname, year='year',
    sent="K L E=Y:CO2", rts=RTS_VRS1, tech=CONTEMPORARY,
).optimize(OPT_LOCAL, "highs")

# VRS2 + 当期技术
res = DEAweak.dep(
    data, id=dmuname, year='year',
    sent="K L E=Y:CO2", rts=RTS_VRS2, tech=CONTEMPORARY,
).optimize(OPT_LOCAL, "highs")
```

DEAweak.depddf（方向距离函数，§7.4.2）：

`DEAweak.depddf` 与 `DEAweak.dep` 共用同一模块（`from deabook import DEAweak`），方向同样固定为非期望产出导向，仅底层距离函数由谢泼德距离改为方向距离函数。其余参数与 `dep` 一致：

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
# CRS + 当期技术
res = DEAweak.depddf(
    data, id=dmuname, year='year',
    sent="K L E=Y:CO2", rts=RTS_CRS, tech=CONTEMPORARY,
).optimize(OPT_LOCAL, "highs")

# VRS1 + 当期技术
res = DEAweak.depddf(
    data, id=dmuname, year='year',
    sent="K L E=Y:CO2", rts=RTS_VRS1, tech=CONTEMPORARY,
).optimize(OPT_LOCAL, "highs")

# VRS2 + 当期技术
res = DEAweak.depddf(
    data, id=dmuname, year='year',
    sent="K L E=Y:CO2", rts=RTS_VRS2, tech=CONTEMPORARY,
).optimize(OPT_LOCAL, "highs")
```

DEAweak.lddf（加性 Luenberger 指标）：

`DEAweak.lddf` 返回加性分解 `L/LEFFCH/LTECHCH`，通过 `orient` 选择方向；以下以期望产出—非期望产出方向（`orient="oo"`，§7.2.1）为例，该方向在三种规模报酬假设下均可求解。综合方向 `orient="both"` 与投入方向 `orient="io"` 仅支持 `RTS_CRS/RTS_VRS2`（VRS1 下不可线性化，见 §7.2.3）：

```
from deabook import PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
```

```
# CRS + 当期技术 + oo 方向
res = DEAweak.lddf(
    data, id=dmuname, year='year',
    sent="K L E=Y:CO2", orient="oo", rts=RTS_CRS, tech=CONTEMPORARY,
).optimize(OPT_LOCAL, "highs")

# VRS1 + 当期技术 + oo 方向
res = DEAweak.lddf(
    data, id=dmuname, year='year',
    sent="K L E=Y:CO2", orient="oo", rts=RTS_VRS1, tech=CONTEMPORARY,
).optimize(OPT_LOCAL, "highs")

# VRS2 + 当期技术 + oo 方向
res = DEAweak.lddf(
    data, id=dmuname, year='year',
    sent="K L E=Y:CO2", orient="oo", rts=RTS_VRS2, tech=CONTEMPORARY,
).optimize(OPT_LOCAL, "highs")
```

上述三个函数构成动态环境绩效测算的完整接口：`DEAweak.dep` 对应谢泼德距离函数实现，`DEAweak.depddf` 对应方向距离函数实现，二者在相同样本与口径下可相互验证；`DEAweak.lddf` 提供加性 Luenberger 分解作为补充。四种技术参照集（`CONTEMPORARY`/`SEQUENTIAL`/`WINDOW`/`GLOBAL`）均可通过 `tech` 参数切换，全局技术下比值型指数返回 `mqpi` 并附Pastor-Lovell 分解（`MEFFCH`/`MTECHCH`）。下一节将该接口分别应用于 OECD 国家与中国省级面板，考察动态环境绩效在不同样本与技术假设下的表现与稳健性。

7.6 案例应用：OECD动态环境绩效

使用第6章同源OECD数据的2016—2019年面板，共33个国家、132个观测。投入产出定义沿用第6章；本章仅研究碳排放方向的跨期变化。采用CRS时序技术，某年的参考集包含2016年至当年的全部观测。按同一国家匹配相邻年份，分别求解四个同期和跨期距离，并计算动态绩效MQ、效率变化EC与技术变化TC。

```
from deabook.example.book_cases.run import FUN
FUN[7]()  # 重新计算本章，输出完整CSV、摘要表与PNG/PDF图
```

表 7.5: OECD动态环境绩效：结果摘要

期末年份	指标	有效数	几何均值	中位数	P10	P90
2017	生产率指数	30	1.029	1.033	0.977	1.086
2017	效率变化	30	0.964	0.954	0.925	1.012
2017	技术变化	30	1.067	1.043	1.043	1.116
2018	生产率指数	30	1.039	1.038	0.996	1.076
2018	效率变化	30	0.950	0.961	0.886	1.001

期末年份	指标	有效数	几何均值	中位数	P10	P90
2018	技术变化	30	1.094	1.067	1.067	1.122
2019	生产率指数	30	1.054	1.041	0.991	1.106
2019	效率变化	30	1.035	1.026	0.973	1.090
2019	技术变化	30	1.018	1.014	1.014	1.023

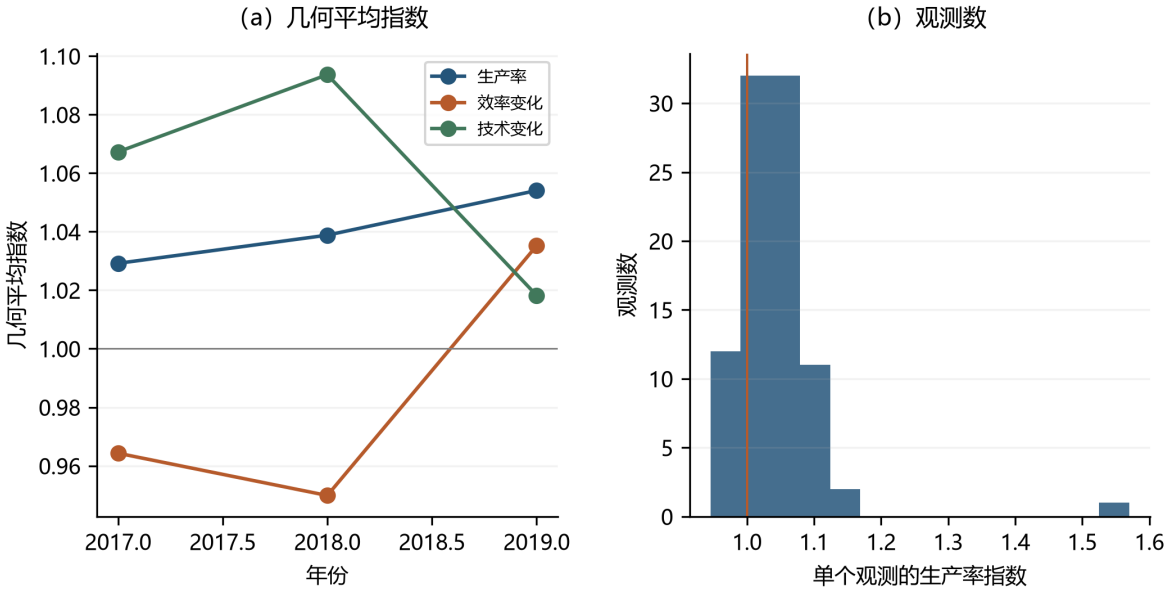


图 7.4: OECD动态环境绩效的年度分解与分布（时序CRS技术）

表按期末年份展开环境生产率及其分解，并报告各项有效观测数。跨期不可行的观测不参加统计，不按零值填补；因此有效数须与完整结果一起读取。图左比较时期变化，图右显示有效观测的生产率分布。

99个候选相邻期组合中，90个具有完整正距离，每年有效样本均为30。不可行跨期距离保留为缺失，不填0或1。MQ的年度几何均值依次为1.029、1.039和1.054；前两期EC低于1，2019年回升至1.035。表内三项使用同一有效集合取几何平均，因此乘法分解仍成立；完整文件同时列出四个距离，便于定位缺失。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch07_full.csv`。表中P10和P90为第10和第90百分位数（如适用），展示分布而不占用逐个决策单元的行数。

7.7 本章小结

99个候选相邻期组合中，90个具有完整正距离，每年有效样本均为30。不可行跨期距离保留为缺失，不填0或1。MQ的年度几何均值依次为1.029、1.039和1.054；前两期EC低于1，2019年回升至1.035。表内三项使用同一有效集合取几何平均，因此乘法分解仍成立；完整文件同时列出四个距离，便于定位缺失。

参考文献

BRIEC W, KERSTENS K. Infeasibility and directional distance functions with application to the determinateness of the Luenberger productivity indicator [J]. Journal of Optimization Theory and Applications, 2009, 141(1): 55-73. DOI: 10.1007/s10957-008-9503-2.

CAVES D W, CHRISTENSEN L R, DIEWERT W E. The economic theory of index numbers and the measurement of input, output, and productivity [J]. Econometrica, 1982, 50(6): 1393-1414. DOI: 10.2307/1913388.

CHAMBERS R G, CHUNG Y, FÄRE R. Benefit and distance functions [J]. Journal of Economic Theory, 1996, 70(2): 407-419. DOI: 10.1006/jeth.1996.0096.

CHAMBERS R G, FÄRE R, GROSSKOPF S. Productivity growth in APEC countries [J]. Pacific Economic Review, 1996, 1(3): 181-190. DOI: 10.1111/j.1468-0106.1996.tb00184.x.

CHARNES A, COOPER W W, RHODES E. Measuring the efficiency of decision making units [J]. European Journal of Operational Research, 1978, 2(6): 429-444. DOI: 10.1016/0377-2217(78)90138-8.

CHUNG Y H, FÄRE R, GROSSKOPF S. Productivity and undesirable outputs: a directional distance function approach [J]. Journal of Environmental Management, 1997, 51(3): 229-240. DOI: 10.1006/jema.1997.0146.

FEENSTRA R C, INKLAAR R, TIMMER M P. The next generation of the Penn world table [J]. American Economic Review, 2015, 105(10): 3150-3182. DOI: 10.1257/aer.20130954.

FÄRE R, GROSSKOPF S, LOVELL C A K, PASURKA C A. Multilateral productivity comparisons when some outputs are undesirable: a nonparametric approach [J]. The Review of Economics and Statistics, 1989, 71(1): 90-98. DOI: 10.2307/1928055.

FÄRE R, GROSSKOPF S, LOVELL C A K. The measurement of efficiency of production [M]. Boston: Kluwer-Nijhoff Publishing, 1985. DOI: 10.1007/978-94-015-7681-6.

FÄRE R, GROSSKOPF S, NORRIS M, ZHANG Z. Productivity growth, technical progress, and efficiency change in industrialized countries [J]. American Economic Review, 1994, 84(1): 66-83.

MAHLBERG B, SAHOO B K. Radial and non-radial decompositions of Luenberger productivity indicator with an illustrative application [J]. International Journal of Production Economics, 2011, 131(2): 721-726. DOI: 10.1016/j.ijpe.2011.02.021.

NIN A, ARNDT C, HERTEL T W, PRECKEL P V. Bridging the gap between partial and total factor productivity measures using directional distance functions [J]. American Journal of Agricultural Economics, 2003, 85(4): 928-945. DOI: 10.1111/0002-9092.00145.

SHEPHARD R W. Theory of cost and production functions [M]. Princeton: Princeton University Press, 1970.

第 8 章

基于松弛测度的效率模型

前面章节介绍的投入距离函数、产出距离函数、双曲距离函数和方向距离函数，大多采用径向（radial）效率测度。径向模型的核心假设是：投入收缩、产出扩张或非期望产出削减可以用一个共同的比例因子概括。这一处理方式直观，便于比较不同决策单元相对于前沿面的距离，但也隐含了较强的“等比例改进” 假设。在能源、环境与多投入生产问题中，不同投入要素、期望产出与污染排放往往分别存在冗余、不足或过剩。如果某个决策单元在完成径向调整后仍存在投入冗余、产出不足或非期望产出过剩，仅报告径向效率就可能高估其真实效率水平，并掩盖低效率的具体来源。

Tone（2001）提出的基于松弛测度的效率模型（Slacks-Based Measure, SBM）正是为处理这一问题而构建的。需要说明两点分工：Tone（2001）给出的是**只含投入与期望产出**的非径向SBM，把投入冗余 s^x 与期望产出不足 s^y 直接纳入效率指标（本章 8.3 节）；把**非期望产出松弛** s^b 也纳入目标函数的环境 SBM 出自 Tone（2003）的 GRIPS 研究报告（本章 8.4 节）。此外，“非径向” 是 SBM 各变体的共同属性，而” 非导向”（non-radial and non-oriented）只对应把两端（或三端）松弛同时纳入目标的式（8.1）与式（8.3）；8.3.1—8.3.2、8.4.3—8.4.5 的单一导向 SBM 仍是非径向的，但只把一端的松弛计入效率损失。SBM 的经济含义是：效率损失不仅来自整体比例上的落后，也来自具体变量层面的结构性冗余或短缺。能源、资本与劳动等投入要素的节约空间与污染排放的削减空间往往并不同步，单一比例因子难以同时刻画两端。除本章主线的相对松弛加权平均写法外，SBM 还可沿方向距离函数的思路推广为基于方向距离函数的松弛测度（SD，见 8.5 节），在允许零值与负值数据的同时提供比径向模型更强的判别力。

本章先说明径向模型遗漏松弛变量时可能产生的效率判断偏误，再分别给出不含和含非期望产出的 SBM 模型，最后结合 `deabook` 包的调用方式说明方向向量、规模报酬假设和返回结果的解释口径。全章重点包括：

1. 不含非期望产出的 SBM：投入导向、产出导向和非导向 SBM；
2. 含非期望产出的 SBM：投入导向、期望产出导向、非期望产出导向和综合非导向SBM；
3. CRS 与 VRS 两种规模报酬假设下技术集合的差异；
4. `deabook` 包中 `DEA.sbm` 与 `DEAweak.sbmu` 的参数设定、返回结果和案例解释，重点说明如何从研究问题出发选择效率改进口径；
5. 基于方向距离函数的松弛测度（SD）及其在非期望产出下的扩展（Färe & Grosskopf, 2010；Pham & Zelenyuk, 2018），作为把绝对松弛直接求和的SBM 推广形式，可在含零值或负值数据时保持判别力。

8.1 径向效率测度中的松弛问题

本节从径向 DEA（Charnes, Cooper and Rhodes（1978）提出；系统介绍见Cooper, Seiford and Tone（2007））中松弛变量的来源出发，说明 SBM 的必要性。设有 N 个决策单元（decision making unit, DMU），每个 DMU 使用 P 种投入 $x=(x_1,\dots,x_P)$ 生产 Q 种期望产出 $y=(y_1,\dots,y_Q)$。以投入导向 DEA 为例，CRS 下的径向模型可写为：

$$\begin{aligned}\min_{\theta,\lambda}\quad & \theta \\ \text{s.t.}\quad & \sum_{n=1}^{N}\lambda_n x_{np}\le \theta x_{0p},\quad p=1,\dots,P,\\ & \sum_{n=1}^{N}\lambda_n y_{nq}\ge y_{0q},\quad q=1,\dots,Q,\\ & \lambda_n\ge 0.\end{aligned}$$

令最优解为 θ^*,λ^*。即使 $\theta^*=1$，仍可能存在

$$s_p^- = x_{0p}-\sum_{n=1}^{N}\lambda_n^* x_{np}>0,\qquad s_q^+=\sum_{n=1}^{N}\lambda_n^* y_{nq}-y_{0q}>0.$$

其中，s_p^- 表示第 p 类投入的冗余，s_q^+ 表示第 q 类期望产出的不足。若一个 DMU 在径向比例调整后仍有非零松弛，则所谓“径向有效” 只说明不存在统一比例上的改进空间，并不意味着所有投入和产出变量都已达到前沿。松弛变量是反映资源配置结构的重要证据，而非求解过程中的附属信息。

若记同时考虑全部松弛的 SBM 效率为 ρ，则只要存在非零松弛就有 $\rho<\theta^*=1$；一般地，$\rho^*\le\theta^*$。该不等式即Tone（2001）定理 4，比较的是输入导向 CCR 径向效率 $\theta^*\in(0,1]$ 与非导向 SBM ρ^*。因此，存在松弛时径向效率系统性高于 SBM 效率，这正是“径向高估” 的形式化含义。SBM 的基本思想是：不再只用一个比例因子概括效率，而用所有相关松弛的平均相对大小构造效率指标，从而把非比例性改进空间纳入效率评价。

8.2 生产技术与符号设定

SBM 的松弛量需要在明确的生产技术集合上才能定义。本节统一给出该集合与变量记号，作为后续各类SBM 模型的共同基础，同时避免不同方向下的符号重复。不含非期望产出时，DEA 生产技术可以写为：

$$T=\{(x,y):x \text{ 能够生产 } y\}.$$

在样本数据下，CRS 技术集合为：

$$\begin{aligned}T_{CRS}=\{(x,y):\ & x_p\ge\sum_{n=1}^{N}\lambda_n x_{np},\quad p=1,\dots,P,\\ & y_q\le\sum_{n=1}^{N}\lambda_n y_{nq},\quad q=1,\dots,Q,\\ & \lambda_n\ge 0,\quad n=1,\dots,N\}.\end{aligned}$$

VRS 技术集合在 CRS 的基础上增加凸性约束：

$$\sum_{n=1}^{N}\lambda_n = 1.$$

从经济含义看，CRS 假设各 DMU 可以按任意比例复制生产活动，适合规模差异可被视为技术复制的情形；VRS 则要求参考组合位于样本凸包内，更适合比较规模差异较大的地区、企业或国家。CRS 与 VRS 的差异会影响前沿面的形状，进而影响效率水平和有效单元识别。下文用 0 表示被评价的决策单元，用 λ_n 表示参考前沿上第 n 个DMU 的强度变量。除非特别说明，公式先给出 CRS 情形，VRS 只需增加凸性约束。

8.3 不含非期望产出的 SBM 模型

本节先讨论没有污染等非期望产出时的 SBM。借助 8.2 节给出的生产技术 T 与符号 x, y, λ，下面先在最简的双松弛情形下给出 SBM 的目标函数，再逐步引入非期望产出。此时低效率主要表现为两类：一是给定期望产出下的投入冗余，二是给定投入下的期望产出不足。不同导向的SBM 对应不同的研究问题，效率值的数值大小本身无法说明哪个模型“更好”，关键在于明确效率值究竟衡量投入节约空间、产出扩张空间，还是二者的综合改进空间。

8.3.1 投入导向 SBM

投入导向 SBM 适用于研究资源节约、成本压缩或要素投入冗余等问题。其核心问题是：在期望产出至少不减少的条件下，各类投入分别可以减少多少。模型为：

$$\begin{aligned}
\min_{\rho^I,\lambda,s^-} \quad & \rho^I = 1 - \frac{1}{P}\sum_{p=1}^{P}\frac{s_p^-}{x_{0p}} \\
\text{s.t.} \quad & x_{0p} = \sum_{n=1}^{N}\lambda_n x_{np} + s_p^-, \quad p = 1, \dots, P, \\
& y_{0q} \le \sum_{n=1}^{N}\lambda_n y_{nq}, \quad q = 1, \dots, Q, \\
& \lambda_n \ge 0, \quad s_p^- \ge 0.
\end{aligned}$$

投入导向 SBM 的效率值满足 $0 < \rho^I \le 1$。如果 $s_p^- = 0$ 对所有投入都成立，则 $\rho^I = 1$，说明在保持期望产出不下降的前提下不存在投入冗余；若某些投入存在冗余，则 $\rho^I < 1$。ρ^I 是所有被纳入目标函数的投入相对松弛的平均结果，而非某一个投入的节约率。因此，在经验解释中应同时报告效率值和具体投入松弛，才能判断低效率主要来自资本、劳动、能源还是其他投入。该模型的目标函数本身是线性的，不需要 Charnes-Cooper 变换。

VRS 情形下增加：

$$\sum_{n=1}^{N}\lambda_n = 1.$$

8.3.2 产出导向 SBM

与投入导向对称地考察产出端：产出导向 SBM 关注另一类低效率来源——给定投入下的期望产出不足，适用于研究产出扩张、服务供给提升或创新产出不足等问题。其核心问题是：在投入不增加的条件下，各类期望产出分别可以增加多少。模型可写为：

$$\begin{aligned}\max_{\phi^O,\lambda,s^+}\quad & \phi^O = 1+\frac{1}{Q}\sum_{q=1}^{Q}\frac{s_q^+}{y_{0q}}\\ \text{s.t.}\quad & x_{0p}\geq\sum_{n=1}^{N}\lambda_n x_{np},\quad p=1,\dots,P,\\ & y_{0q}=\sum_{n=1}^{N}\lambda_n y_{nq}-s_q^+,\quad q=1,\dots,Q,\\ & \lambda_n\geq 0,\quad s_q^+\geq 0.\end{aligned}$$

$\phi^O\geq 1$ 表示平均产出扩张因子，其值越大，说明给定投入下期望产出的潜在扩张空间越大。为使不同方向下效率值都落在 $(0,1]$ 内，通常报告：

$$\rho^O=\frac{1}{\phi^O}.$$

若所有产出松弛均为 0，则 $\phi^O=1$ 且 $\rho^O=1$；若存在产出不足，则 $\phi^O>1$ 且 $\rho^O<1$。VRS 情形同样增加 $\sum_n\lambda_n=1$。

8.3.3 非导向 SBM

当研究对象同时存在投入冗余和产出不足时，可以使用非导向SBM。该模型同时考察投入端和产出端两类松弛，是 Tone（2001）最常用的 SBM 形式：

$$\begin{aligned}\min_{\rho,\lambda,s^-,s^+}\quad & \rho=\frac{1-\frac{1}{P}\sum_{p=1}^{P}s_p^-/x_{0p}}{1+\frac{1}{Q}\sum_{q=1}^{Q}s_q^+/y_{0q}}\\ \text{s.t.}\quad & x_{0p}=\sum_{n=1}^{N}\lambda_n x_{np}+s_p^-,\quad p=1,\dots,P,\\ & y_{0q}=\sum_{n=1}^{N}\lambda_n y_{nq}-s_q^+,\quad q=1,\dots,Q,\\ & \lambda_n\geq 0,\quad s_p^-\geq 0,\quad s_q^+\geq 0.\end{aligned}\tag{8.1}$$

分子刻画平均投入利用程度，分母刻画平均产出不足程度，只要任一投入或产出存在松弛，效率值都会下降。因此，非导向SBM 通常比单纯径向模型更严格，也更能揭示低效率的变量来源：记径向效率为 θ^*，则 $\rho_{SBM}\leq\theta^*$（Tone（2001）定理4），当且仅当不存在非径向松弛时取等。该模型尤其适合研究对象同时涉及资源节约和产出提升的情形。

8.3.4 非导向 SBM 的线性化

式(8.1)的目标函数是分式规划，直接求解不如线性规划稳定，可采用Charnes and Cooper（1962）提出的 Charnes-Cooper 变换。令

$$t=\left(1+\frac{1}{Q}\sum_{q=1}^{Q}\frac{s_q^+}{y_{0q}}\right)^{-1},\quad S_p^-=ts_p^-,\quad S_q^+=ts_q^+,\quad \Lambda_n=t\lambda_n.$$

则可转化为线性规划式(8.2)：

$$\begin{aligned}\min_{t,\Lambda,S^-,S^+}\quad & \tau=t-\frac{1}{P}\sum_{p=1}^{P}\frac{S_p^-}{x_{0p}}\\ \text{s.t.}\quad & 1=t+\frac{1}{Q}\sum_{q=1}^{Q}\frac{S_q^+}{y_{0q}},\\ & tx_{0p}=\sum_{n=1}^{N}\Lambda_n x_{np}+S_p^-,\quad p=1,\dots,P,\\ & ty_{0q}=\sum_{n=1}^{N}\Lambda_n y_{nq}-S_q^+,\quad q=1,\dots,Q,\\ & \Lambda_n\geq 0,\quad S_p^-\geq 0,\quad S_q^+\geq 0,\quad t>0.\end{aligned}\tag{8.2}$$

求解后有 $\rho=\tau^*$，原始松弛由

$$s_p^-=S_p^{-*}/t^*,\qquad s_q^+=S_q^{+*}/t^*$$

恢复。VRS 约束 $\sum_n\lambda_n=1$ 在线性化后变为：

$$\sum_{n=1}^{N}\Lambda_n=t.$$

8.4 含非期望产出的 SBM 模型

8.3 节的模型尚未考虑污染等非期望产出。在能源—环境问题中，许多生产过程会同时产生期望产出和非期望产出，例如企业在创造增加值或发电量的同时，也会排放二氧化碳、二氧化硫等污染物，因此需要把SBM 扩展到含非期望产出的情形。设非期望产出为 $b=(b_1,\dots,b_R)$。为同时区分投入、期望产出与非期望产出三类松弛，本节起将松弛记号由 s^-,s^+ 改写为 s^x,s^y,s^b，对应关系为 $s_p^x\equiv s_p^-$、$s_q^y\equiv s_q^+$，并新增非期望松弛 s_r^b。在环境 SBM 框架下，期望产出和非期望产出的经济方向相反：期望产出越多越好，非期望产出越少越好，因此非期望产出的松弛表示污染物过剩或可削减空间。环境效率测算的关键在于污染松弛是否进入效率损失函数：仅把污染变量列入生产技术（b 出现在约束中）并不改变效率口径，只有把 s^b 纳入目标函数，污染削减空间才真正进入效率评价。

本节先给出 Tone（2003）的综合非导向 SBM 作为环境 SBM 的主线模型并完成线性化（8.4.1–8.4.2）；随后 8.4.3–8.4.5 的投入导向、期望产出导向、非期望产出导向是其退化情形，目标函数本身为线性，无需 Charnes-Cooper 变换。

技术假设的差别：本节的非期望产出约束不是第 6—7 章的弱可处置技术。下文式（8.3）及 8.4.3—8.4.5 对非期望产出采用 $b_{0r} = \sum_n \lambda_n b_{nr} + s_r^b$（或 $b_{0r} \geq \sum_n \lambda_n b_{nr}$）的形式：参考组合的排放加上一个自由的非负松弛等于观测排放，因此污染可以在期望产出不变的条件下**单独削减**。这与把污染物写在约束不等式" 较小方向" 上的投入处理方式相同，属于强可处置的 bads-as-inputs 设定。第 6—7 章的环境技术采用弱可处置（$(y, b) \to (\theta y, \theta b)$，$0 \leq \theta \leq 1$）：减排必须与期望产出同比例收缩，减污是有机会成本的。两者不是同一个生产可能集 T，因此本章的效率值与第 6—7 章的环境效率不可直接比较，数值差异也不能解释为" 目标函数换了写法"。Tone（2003）在可分（separable）设定之外还给出好坏产出不可分（non-separable）的情形，以接近 Färe 等的联合生产刻画。若研究设计要求坚持联合生产，应改用不可分设定或直接在约束中写入弱可处置，而不是只改目标函数、不改技术。

8.4.1 含非期望产出的综合非导向 SBM

综合非导向环境 SBM 把投入冗余、期望产出不足和非期望产出过剩三类松弛同时纳入效率目标。Tone（2003）的含非期望产出SBM 为：

$$
\begin{aligned}
\min_{\rho,\lambda,s^x,s^y,s^b} \quad & \rho = \frac{1 - \frac{1}{P}\sum_{p=1}^{P} s_p^x / x_{0p}}{1 + \frac{1}{Q+R}\left(\sum_{q=1}^{Q} s_q^y / y_{0q} + \sum_{r=1}^{R} s_r^b / b_{0r}\right)} \\
\text{s.t.} \quad & x_{0p} = \sum_{n=1}^{N} \lambda_n x_{np} + s_p^x, \quad p = 1, \ldots, P, \\
& y_{0q} = \sum_{n=1}^{N} \lambda_n y_{nq} - s_q^y, \quad q = 1, \ldots, Q, \\
& b_{0r} = \sum_{n=1}^{N} \lambda_n b_{nr} + s_r^b, \quad r = 1, \ldots, R, \\
& \lambda_n \geq 0, \quad s_p^x \geq 0, \quad s_q^y \geq 0, \quad s_r^b \geq 0.
\end{aligned}
\tag{8.3}
$$

其中：

- s_p^x：投入冗余；
- s_q^y：期望产出不足；
- s_r^b：非期望产出过剩，即在保持生产可行的前提下可削减的污染物数量。

式（8.3）的分母同时惩罚期望产出不足和非期望产出过剩，分子则惩罚投入冗余。若某个DMU 的投入、期望产出和非期望产出都不存在松弛，则 $\rho = 1$。由于该模型使用相对松弛量，进入分式的投入、期望产出和非期望产出必须严格大于零（$x_{0p} > 0$、$y_{0q} > 0$、$b_{0r} > 0$），否则 s/x_0、s/y_0 或 s/b_0 会触发除零或失去经济可比性。

8.4.2 含非期望产出的线性化

与式（8.1）类似，式（8.3）的目标函数同样是分式规划，需经Charnes-Cooper 变换线性化后求解。令

$$
t = \left[1 + \frac{1}{Q+R}\left(\sum_{q=1}^{Q} \frac{s_q^y}{y_{0q}} + \sum_{r=1}^{R} \frac{s_r^b}{b_{0r}}\right)\right]^{-1},
$$

并令 $S_p^x = ts_p^x$、$S_q^y = ts_q^y$、$S_r^b = ts_r^b$、$\Lambda_n = t\lambda_n$，则得到线性规划式(8.4)：

$$
\begin{aligned}
\min_{t,\Lambda,S^x,S^y,S^b} \quad & \tau = t - \frac{1}{P}\sum_{p=1}^{P}\frac{S_p^x}{x_{0p}} \\
\text{s.t.} \quad & 1 = t + \frac{1}{Q+R}\left(\sum_{q=1}^{Q}\frac{S_q^y}{y_{0q}} + \sum_{r=1}^{R}\frac{S_r^b}{b_{0r}}\right), \\
& tx_{0p} = \sum_{n=1}^{N}\Lambda_n x_{np} + S_p^x, \quad p = 1,\dots,P, \\
& ty_{0q} = \sum_{n=1}^{N}\Lambda_n y_{nq} - S_q^y, \quad q = 1,\dots,Q, \\
& tb_{0r} = \sum_{n=1}^{N}\Lambda_n b_{nr} + S_r^b, \quad r = 1,\dots,R, \\
& \Lambda_n \geq 0, \quad S_p^x \geq 0, \quad S_q^y \geq 0, \quad S_r^b \geq 0, \quad t > 0.
\end{aligned}
\tag{8.4}
$$

VRS 情形增加 $\sum_n \Lambda_n = t$。

8.4.3 投入导向环境 SBM

8.4.1 节的综合非导向 SBM 把投入、期望产出和非期望产出三类松弛同时纳入目标函数，但许多研究只关心其中某一端的改进空间，因此下面分别给出投入导向、期望产出导向和非期望产出导向三种单一口径的环境SBM。投入导向环境 SBM 在期望产出不下降、非期望产出不增加的条件下衡量投入冗余，适合回答“在不降低经济产出、且不增加污染排放的前提下，资源投入可以节约多少”的问题：

$$
\begin{aligned}
\min \quad & \rho^{I,b} = 1 - \frac{1}{P}\sum_{p=1}^{P}\frac{s_p^x}{x_{0p}} \\
\text{s.t.} \quad & x_{0p} = \sum_{n=1}^{N}\lambda_n x_{np} + s_p^x, \quad p = 1,\dots,P, \\
& y_{0q} \leq \sum_{n=1}^{N}\lambda_n y_{nq}, \quad q = 1,\dots,Q, \\
& b_{0r} \geq \sum_{n=1}^{N}\lambda_n b_{nr}, \quad r = 1,\dots,R, \\
& \lambda_n \geq 0, \quad s_p^x \geq 0.
\end{aligned}
$$

在该模型中，期望产出和非期望产出作为技术可行性约束：参考组合的期望产出不得低于被评价DMU，非期望产出不得高于被评价DMU，只有投入松弛进入效率目标。该模型及以下 8.4.4、8.4.5 两个单一导向的目标函数均为线性，无需 Charnes-Cooper 变换；只有 8.4.1 的综合模型因含分式目标才需要线性化。

8.4.4 期望产出导向环境 SBM

把效率损失来源从投入端换到期望产出端，便得到期望产出导向环境SBM。它在投入不增加、非期望产出不增加的条件下衡量期望产出不足，适合回答“在资源投入和污染排放不增加的前提下，经济产出还有多大扩张空间”的问题：

$$
\begin{aligned}
\max \quad & \phi^{Y,b} = 1 + \frac{1}{Q}\sum_{q=1}^{Q}\frac{s_q^y}{y_{0q}} \\
\text{s.t.} \quad & x_{0p} \geq \sum_{n=1}^{N}\lambda_n x_{np}, \quad p = 1, \dots, P, \\
& y_{0q} = \sum_{n=1}^{N}\lambda_n y_{nq} - s_q^y, \quad q = 1, \dots, Q, \\
& b_{0r} \geq \sum_{n=1}^{N}\lambda_n b_{nr}, \quad r = 1, \dots, R, \\
& \lambda_n \geq 0, \quad s_q^y \geq 0.
\end{aligned}
$$

通常报告 $\rho^{Y,b} = 1/\phi^{Y,b}$，因此效率值仍位于 $(0, 1]$。这里的效率损失来自期望产出不足，而投入和非期望产出主要发挥约束作用。VRS 情形增加 $\sum_n \lambda_n = 1$（与 8.3.1、8.3.2 一致；该模型目标函数为线性，无需 Charnes-Cooper 变换）。

8.4.5 非期望产出导向 SBM

若关心的是污染削减空间而非经济产出扩张，则采用非期望产出导向SBM。它在投入不增加、期望产出不下降的条件下，衡量污染物的可削减空间。与式（8.3）保持一致，先求Tone 分式在非期望产出端的分母 ϕ^B：

$$
\begin{aligned}
\max \quad & \phi^{B} = 1 + \frac{1}{R}\sum_{r=1}^{R}\frac{s_r^b}{b_{0r}} \\
\text{s.t.} \quad & x_{0p} \geq \sum_{n=1}^{N}\lambda_n x_{np}, \quad p = 1, \dots, P, \\
& y_{0q} \leq \sum_{n=1}^{N}\lambda_n y_{nq}, \quad q = 1, \dots, Q, \\
& b_{0r} = \sum_{n=1}^{N}\lambda_n b_{nr} + s_r^b, \quad r = 1, \dots, R, \\
& \lambda_n \geq 0, \quad s_r^b \geq 0.
\end{aligned}
$$

其中，$\phi^B = 1 + \delta_b \geq 1$（记 $\delta_b = \frac{1}{R}\sum_r s_r^b/b_{0r}$ 为平均相对污染松弛）。报告的效率值为其倒数，从而仍位于 $(0, 1]$：

$$
\rho^B = \frac{1}{\phi^B} = \frac{1}{1+\delta_b}.
$$

ϕ^B 只是 Tone 分式在非期望产出端的分母，**不是径向意义上的污染物缩减因子**，ρ^B 也不是径向减排效率。这一点与期望产出导向不同：产出导向时投影产出为 $y + s^y = \phi^Y y$，故 $\rho^Y = 1/\phi^Y$ 恰好是 Farrell 意义上的"实际 / 前沿"比值；而污染端的投影是 $b - s^b$，剩余污染比例是 $1 - \delta_b$，与 $1/(1+\delta_b)$ 并不相等——对任意 $\delta_b > 0$ 恒有 $\frac{1}{1+\delta_b} > 1 - \delta_b$。因此本节及案例中报告的非期望产出导向效率应按 $\rho^B = 1/(1+\delta_b)$ 解释；若要读出"污染还剩多少比例"，须直接报告 $1 - \delta_b$ 或各维 s_r^b/b_{0r}（相关的跨方向可比性问题见 8.7 节）。

若 $\rho^B = 1$，表示在给定投入和期望产出约束下不存在污染物过剩；若 $\rho^B < 1$，说明非期望产出相对于前沿存在可削减空间。VRS 情形同样增加 $\sum_n \lambda_n = 1$。

综合 8.4.1–8.4.5 可见，含非期望产出的 SBM 各变体的差别仅在于“哪些松弛进入效率目标”：综合非导向（8.4.1）把三类松弛全部纳入，三个单一导向（8.4.3–8.4.5）各只纳入其中一类。此外，8.6.2 节 表 8.2 还将给出由方向向量组合得到的混合方向（如投入—污染联合改进型）。这一选择由方向向量 gx、gy、gb 编码，下一节说明 deabook 如何在代码中落实。

8.5 基于方向距离函数的松弛测度（SD）

8.3 节与 8.4 节给出的 SBM 以各变量相对松弛的加权平均构造效率，已能同时刻画投入、期望产出与非期望产出三端的非比例改进空间。不过，Tone（2001）的SBM 也是更一般框架的一个特例：Färe 与 Grosskopf（2010）指出，方向距离函数（directional distance function, DDF，见第 6、7 章）本身具有可加结构，据此可以构造一种“基于方向距离函数的松弛测度”（slack-based directional distance function，下文记为 SD）。SD 把方向向量取为单位向量 **1**，但允许各投入、各产出拥有相互独立的标量松弛，从而把 DDF 的方向性位移与 SBM 的松弛识别结合起来。Pham 与 Zelenyuk（2018）进一步把SD 扩展到含非期望产出的情形。本节先给出不含非期望产出的SD（8.5.1 节），再给出含非期望产出的扩展（8.5.2 节），并说明它与 8.3、8.4 节模型的关系。

8.5.1 不含非期望产出的 SD：Färe 与Grosskopf（2010）

回顾第 6 章的方向距离函数：给定方向向量 $g = (g_x, g_y)$，DDF 沿统一比例 τ 做位移

$$\vec{D}_T(x, y; g) = \max\{\tau : (x - \tau g_x,\ y + \tau g_y) \in T\}.$$

Färe 与 Grosskopf（2010）把方向向量取为全 1 向量 $g_x = \mathbf{1}_P$、$g_y = \mathbf{1}_Q$，但不再要求所有维度共享同一个位移系数，而是让每个投入、每个产出各自拥有一个独立的标量松弛 $\alpha_i \geq 0$、$\beta_r \geq 0$。于是投入可分别减少 α_i、期望产出可分别增加 β_r，效率损失由这些标量松弛的总和度量：

$$\begin{aligned} \max_{\alpha_i, \beta_r, \lambda_n} \quad & SD(x_0, y_0) = \sum_{i=1}^{P} \alpha_i + \sum_{r=1}^{Q} \beta_r \\ \text{s.t.} \quad & x_{0i} = \sum_{n=1}^{N} \lambda_n x_{ni} + \alpha_i, \quad i = 1, \ldots, P, \\ & y_{0r} = \sum_{n=1}^{N} \lambda_n y_{nr} - \beta_r, \quad r = 1, \ldots, Q, \\ & \lambda_n \geq 0, \quad \alpha_i \geq 0, \quad \beta_r \geq 0. \end{aligned} \tag{8.5}$$

VRS 情形下补充 $\sum_{n=1}^{N} \lambda_n = 1$。式（8.5）的目标函数是各维绝对松弛之和：投入松弛 α_i 以 x_i 的单位度量、产出松弛 β_r 以 y_r 的单位度量，方向向量取 **1** 使得各松弛都是标量，不依赖于相应变量的度量单位。这带来三点与 8.3 节 Tone SBM 不同的性质。

第一，SD 是 Tone（2001）SBM 的推广，Tone SBM 是 SD 的特例。二者都把松弛纳入效率定义；但 Tone SBM 采用相对松弛 s/x_0（或 s/y_0）的加权平均，并要求进入分式的变量严格为正，否则会触发除零；

而SD 直接对各维绝对松弛求和，无需除以观测值，因此能够处理零值乃至负值数据，也不必为变量符号而做数据变换。

第二，SD 具有平移不变性（translation invariance）。由于目标与约束都是投入、产出的可加形式，对数据做常数平移不会改变效率排序，这使其在处理含负值的金融数据（如以负值记账的贷款损失准备）时优于径向与相对松弛类模型。

第三，SD 比 DDF、增强双曲效率与 Farrell 型技术效率具有更强的判别力（discriminative power）。若某 DMU 被 SD 判定为完全有效（所有松弛均为零），则它同时也被 DDF、增强双曲效率与Farrell 技术效率判定为有效；反之不成立。原因在于 DDF 沿固定方向 g 只度量一个径向的位移比例，位于技术前沿“平面”（flat facet）上的 DMU 会被 DDF 判为有效，但 SD 仍能识别该平面内部的非径向松弛。因此，SD 在揭示低效率来源上比单一方向的DDF 更细致。

8.5.2 含非期望产出的 SD：Pham 与Zelenyuk（2018）

把 SD 用于环境绩效时，非期望产出（如污染物排放、银行不良贷款）必须被区别对待。Pham 与 Zelenyuk（2018）沿用第 6、7 章的联合弱可处置技术：期望产出强可处置，而期望产出与非期望产出联合弱可处置，即若 $(y,b)\in P(x)$，则对任意 $\tilde{y}\le y$ 与 $0\le\theta\le 1$ 均有 $(\tilde{y},\theta b)\in P(x)$。在该技术下，非期望产出可以在期望产出不变的条件下被削减，但减排并非无成本（减污与期望产出的扩张受联合生产约束）。

在 SD 框架中加入非期望产出松弛 $\gamma_r\ge 0$（$r=1,\dots,R$），即把污染削减空间也作为可独立取值的标量松弛纳入目标函数：

$$
\begin{aligned}
\max_{\alpha_i,\beta_r,\gamma_r,\lambda_n}\quad & SdD(x_0,y_0,b_0)=\sum_{i=1}^{P}\alpha_i+\sum_{r=1}^{Q}\beta_r+\sum_{r=1}^{R}\gamma_r\\
\text{s.t.}\quad & x_{0i}=\sum_{n=1}^{N}\lambda_n x_{ni}+\alpha_i,\quad i=1,\dots,P,\\
& y_{0r}=\sum_{n=1}^{N}\lambda_n y_{nr}-\beta_r,\quad r=1,\dots,Q,\\
& b_{0r}=\sum_{n=1}^{N}\lambda_n b_{nr}+\gamma_r,\quad r=1,\dots,R,\\
& \lambda_n\ge 0,\quad \alpha_i\ge 0,\quad \beta_r\ge 0,\quad \gamma_r\ge 0.
\end{aligned}
\tag{8.6}
$$

VRS 同理补充 $\sum_n\lambda_n=1$。式（8.6）中，非期望产出以正值存储，松弛 γ_r 表示在保持期望产出与其他投入可行的条件下可削减的污染量；其约束写法 $b_{0r}=\sum_n\lambda_n b_{nr}+\gamma_r$ 与 8.4 节环境 SBM 的 $b_{0r}=\sum_n\lambda_n b_{nr}+s_r^b$ 一致，都属于强可处置bads-as-inputs 的设定（参见 8.4 节的技术假设说明）。与 8.4 节 Tone（2003）环境 SBM 相比，SD 同样把污染松弛纳入目标，但用绝对松弛之和代替相对松弛的加权平均，因而同样不要求数据严格为正，并保持平移不变性与更强的判别力。

综合 8.5.1 与 8.5.2 节可见，SD 家族与 8.3、8.4 节 SBM 家族是同一“把松弛纳入效率” 思想的两种写法：SBM 用相对松弛的平均，SD 用绝对松弛的和。二者对“哪些松弛进入目标函数” 的判断完全一致——都可由投入、期望产出、非期望产出三端的松弛组合决定；差别仅在于松弛进入目标的函数形式（加权平均 vs 直接求和）以及对数据正值的依赖程度。因此，SD 特别适合数据类型复杂（含零值或负值）或需要更强判别力的环境效率研究。`deabook` 的 `DEAweak` 模块（如 `DEAweak.sbmu`、`DEAweak.ddf`）已提供含非期望产出的弱可处置技术与相应松弛求解，读者可在其生产可能集之上自行组装式（8.6）的SD 目标。

8.6 SBM 模型的Python实现

前几节已经从生产技术与线性规划两个层面给出了 SBM 的理论形式：不含非期望产出的 8.3 节（含式（8.1）及其Charnes–Cooper 线性化式（8.2））、含非期望产出的 8.4 节（含式（8.3）及式（8.4）），以及基于方向距离函数的松弛测度 8.5 节（含式（8.5）与式（8.6））。这些形式在数学上回答了“哪些松弛进入效率目标、约束如何书写、分式目标如何线性化”，而程序实现要回答的是同一组经济学选择在代码中如何落实。SBM 之所以需要专门的接口，正因为它把效率定义为各变量相对松弛的平均，而非一个统一的比例因子；因此编码时，研究者填写参数的过程本身就是在落实前文的理论选择——生产可能集由哪些投入、期望产出与非期望产出构成，哪些变量的松弛被视为效率损失来源，以及被评价对象相对于怎样的参考前沿进行比较。deabook 提供两个相互对照的 SBM 接口——`DEA.sbm`（不含非期望产出，对应 8.3 节，位于 `DEA` 模块）与 `DEAweak.sbmu`（含非期望产出，对应 8.4 节，位于 `DEAweak` 模块）——并把上述三类选择集中在 `sent`、`gx`、`gy`、`gb`、`rts`、`evalindex` 与 `refindex` 等参数中。下文先说明变量结构与方向向量的编码逻辑，再将求解结果与前文模型逐项对应。

8.6.1 接口模块与变量结构

SBM 的调用始于两个模块与一组常量的导入：`DEA` 与 `DEAweak` 分别承载不含与含非期望产出的 SBM（`DEA.sbm` 与 `DEAweak.sbmu`），规模报酬常量则用于显式指定前沿的凸性假设。整理表格、计算分组均值或输出结果通常借助 `pandas`，后文示例另以 `pathlib.Path` 统一管理数据路径。

```
from deabook import data_path, PANEL
from deabook.constant import WINDOW
from deabook import DEA, DEAweak, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook.constant import RTS_CRS, RTS_VRS1, OPT_LOCAL
from pathlib import Path
import pandas as pd
```

`sent` 参数用于声明投入产出结构。不含非期望产出时，例如 `sent="K L=Y"`，等号左侧的 `K L` 表示投入，右侧的 `Y` 表示期望产出。含非期望产出时，例如 `sent="K L E=Y:CO2"`，冒号前的 `Y` 仍为期望产出，冒号后的 `CO2` 为非期望产出。变量在 `sent` 中的顺序会决定 `gx`、`gy` 和 `gb` 中各元素所对应的变量。

表 8.1: `SBM` 模型的主要参数

参数	作用	本章常用设置	经济含义与解释重点
`data`	指定待评价数据	`DataFrame`	包含投入、期望产出、非期望产出和 DMU 标识变量，是构造生产可能集的样本基础
`sent`	声明投入产出结构	`"K L=Y"`、`"K L E=Y:CO2"`	`=` 左侧为投入，`=` 右侧且 `:` 前为期望产出，`:` 后为非期望产出
`gx`	指定投入松弛方向	`[0,0,0]`、`[1,1,1]`、`[0,0,1]`	元素顺序与投入变量一致；1 表示该投入冗余计入效率损失，0 表示该投入仍约束可行生产组合但不进入效率损失

参数	作用	本章常用设置	经济含义与解释重点
gy	指定期望产出松弛方向	[0]、[1]	1 表示该期望产出不足计入效率损失，0 表示该期望产出仅作为“不低于当前水平”的可行性约束
gb	指定非期望产出松弛方向	[0]、[1]	1 表示该非期望产出过剩计入效率损失，0 表示该非期望产出仅作为“不高于当前水平” 的可行性约束
rts	指定规模报酬假设	RTS_CRS、RTS_VRS1	分别表示规模报酬不变和规模报酬可变，决定参考前沿是否允许局部规模比较
evalindex	选择被评价样本	None 或筛选条件	决定哪些 DMU 被评价，即“评价谁”
refindex	选择参考技术集	None 或筛选条件	决定哪些DMU 构造前沿，即“与谁比较”
solver	指定求解器	"mosek"、"glpk"	在 optimize() 中设置，属于数值求解选项，不改变经济模型含义

使用 表 8.1 时尤其要核对方向向量的长度和变量顺序。例如， sent="K L E=Y:CO2" 有三个投入变量，因此 gx 必须包含三个元素；若只希望将能源投入 E 的冗余纳入目标函数，而资本 K 和劳动 L 仅作为“不增加” 的约束，应写作 gx=[0,0,1]，不能写作 gx=[1]。 gx=0、gy=0 或 gb=0 表示该变量的松弛不被计入本次效率损失，相应变量仍保留在生产技术中。进入相对松弛比的变量必须严格正值的要求已在 8.4.1 节说明，此处不再重复。

8.6.2 方向向量对效率口径的编码

deabook 中的 SBM 借用方向距离函数（DDF）的记号习惯，用 gx、gy 和 gb 三个向量表示” 哪些维度的松弛进入效率目标”。这里必须强调：这三个向量只是**开关**，测度本身仍是SBM，不是相应方向上的DDF——DDF 沿方向 g 做统一的径向位移 β，SBM 则让各维松弛独立取值后再代入分式（对照表 8.4）。即使把 gb=[1] 与第 6 章 $g=(0,0,-b)$ 的DDF 写成” 同一个方向”，两者的效率函数形式也不同：单一非期望产出时 SBM 给出 $\rho^B = 1/(1+\delta_b)$，第 6 章 DDF 常用的效率口径是 $1-\beta$。因此把本章结果与第 6 章 DDF 结果并列时，必须同时报告 β 与 δ_b（或同时给出 $1-\beta$ 与 $1/(1+\delta_b)$），不能因方向向量同名就当作同一口径的两次测算。三个向量的含义分别是：

- gx：投入方向向量。某个投入对应位置为1，表示该投入允许向下调整并进入 SBM 松弛目标。
- gy：期望产出方向向量。某个期望产出对应位置为1，表示该产出允许向上调整并进入 SBM 松弛目标。
- gb：非期望产出方向向量。某个非期望产出对应位置为1，表示该污染物允许向下调整并进入 SBM 松弛目标。

SBM 会根据 gx、gy 和 gb 的组合自动识别模型类型。可以概括为：某个方向向量中至少有一个元素为1，该方向的松弛量就会进入 SBM 效率目标；若该方向全部为0，则相应变量只作为技术可行性约束存在，不参与效率值的松弛惩罚。因此，方向向量本质上是研究问题的编码，须依研究目标设定。

表 8.2: SBM 方向向量与方向模式的对应关系

经济口径	方向向量条件	程序返回的方向名称	被直接纳入目标函数的松弛
投入冗余型	`sum(gx)>0, sum(gy)=0, sum(gb)=0`	`input_oriented`	投入冗余 s^x
期望产出不足型	`sum(gy)>0, sum(gx)=0, sum(gb)=0`	`output_oriented`	期望产出不足 s^y
污染过剩型	`sum(gb)>0, sum(gx)=0, sum(gy)=0`	`unoutput_oriented`	非期望产出过剩 s^b
投入—期望产出综合型	`sum(gx)>0, sum(gy)>0, sum(gb)=0`	`hyper_orientedyx`	s^x, s^y
期望产出—污染综合型	`sum(gy)>0, sum(gb)>0, sum(gx)=0`	`hyper_orientedyb`	s^y, s^b
投入—污染联合改进型	`sum(gx)>0, sum(gb)>0, sum(gy)=0`	`hyper_orientedxb`	s^x, s^b
投入—期望产出—污染综合型	`sum(gx)>0, sum(gy)>0, sum(gb)>0`	`hyper_orientedyxb`	s^x, s^y, s^b

方向向量还决定归一化基数，这一点与 Tone 标准形有实质差异。式（8.1）、式（8.3）等 Tone 标准形使用总维度（P、$Q+R$）作为相对松弛的平均基数；deabook 则在 gx、gy、gb 出现 0 元素（部分变量不进入目标函数）时，自动把归一化基数替换为活跃维度数（$|\{p: gx_p > 0\}|$、$|\{q: gy_q > 0\}| + |\{r: gb_r > 0\}|$）。例如，`gx=[0,0,1]`、`gy=[0]`、`gb=[1]` 实际求解的是 $\rho = \frac{1-s_E/x_E}{1+s^b/b_0}$（投入活跃数=1、产出活跃数=1）；而式（8.3）在该最优解（仅 s^x_E, s^b 非零）处的归一化基数仍为 $P=3$、$Q+R=2$，取值为 $\frac{1-s_E/(3x_E)}{1+s^b/(2b_0)}$。两种归一化在全变量活跃时完全等价，差异只出现在部分活跃设定下；本章8.5.4 的混合方向示例与 8.6.5 的能源—碳联合案例均采用活跃计数口径。

方向向量表示“是否允许该维度改进并进入目标函数” 的开关，与变量的实际大小或权重无关。投入导向模型使用 `gx=[1,1,1]`、`gy=[0]`、`gb=[0]`，表示只度量投入冗余；非期望产出导向模型使用 `gx=[0,0,0]`、`gy=[0]`、`gb=[1]`，表示只度量污染物过剩；能源投入与非期望产出联合方向使用 `gx=[0,0,1]`、`gy=[0]`、`gb=[1]`，表示能源投入和 `CO2` 两个维度进入效率目标，资本和劳动仅作为不增加的约束条件。

综合方向也应由研究问题决定，而不宜机械地把所有维度都设为1。例如，不含非期望产出且同时关注资源节约和产出扩张时，可设 `gx=[1,1]`、`gy=[1]`，对应 `hyper_orientedyx`；若研究绿色全要素效率，同时关注投入节约、期望产出扩张和污染削减，可设 `gx=[1,1,1]`、`gy=[1]`、`gb=[1]`，对应 `hyper_orientedyxb`。

这种设定方式的优点是理论模型与经验代码可以逐项对应。第 8.4 节的投入导向环境 SBM、期望产出导向环境 SBM、非期望产出导向SBM，以及综合方向 SBM，都可以通过改变这三个方向向量在 `DEA.sbm` / `DEAweak.sbmu` 同一接口下实现。方向向量设定后，下一步是调用求解并解读返回结果。

8.6.3 求解结果的解释口径

调用 `optimize()` 后，`DEA.sbm` / `DEAweak.sbmu` 返回一个 `DataFrame`，其主要列见表 8.3。解释顺序应当是：先检查求解状态是否为`"optimal"`，再用 `rho` 判断总体效率水平，最后结合 `slack_x_*`、`slack_y_*` 和 `slack_b_*` 识别效率损失来源。`objective_value`、`phi` 和 `t` 更多用于理解求解过程或特定方向下的变换关系，一般不直接作为经济结论。

表 8.3: `SBM.optimize()` 的主要返回列及解释

返回内容	适用方向	含义	解释注意
`optimization_status`	全部方向	求解状态	`"ok"` 或`"optimal"` 表示求解成功，具体取值取决于求解器接口
`direction`	全部方向	程序识别出的方向模式	名称与表 8.2 中的方向模式一致
`objective_value`	全部方向	原始优化目标值	通常作为求解过程信息，不直接作为效率指标解释
`rho`	全部方向	SBM 效率值	取值越接近1，表示相应方向上的松弛改进空间越小
`phi`	`output_oriented`、`unoutput_oriented`、`hyper_orientedyb`	产出扩张或非期望产出缩减因子	这些方向中通常有 $\rho = 1/\phi$，可辅助理解改进空间
`t`	`hyper_orientedyx`、`hyper_orientedxb`、`hyper_orientedyxb`	Charnes-Cooper 变换变量	属于线性化变量，不宜直接赋予经济含义
`slack_x_*`	投入方向活跃时	各投入变量的松弛量	`gx` 中对应元素为 1 时具有实质含义
`slack_y_*`	期望产出方向活跃时	各期望产出的松弛量	`gy` 中对应元素为 1 时具有实质含义
`slack_b_*`	非期望产出方向活跃时	各非期望产出的松弛量	`gb` 中对应元素为 1 时具有实质含义

此处要区分两类方向：投入导向、期望产出导向、非期望产出导向以及“期望产出+ 非期望产出” 方向的目标函数本身为线性，没有分式目标的线性化问题；只有 `hyper_orientedyx`、`hyper_orientedxb` 和 `hyper_orientedyxb` 这类同时包含投入方向和至少一个产出方向的模型采用 Charnes-Cooper 变换。`deabook` 在返回这些方向的 `slack_*` 列时已将变换后的松弛量除以 `t`，恢复为原始量纲。但由于不同方向纳入目标函数的变量集合不同，不同模型下的松弛量和效率值仍不宜脱离方向设定直接比较。比较前应说明比较的是“投入节约效率”“污染削减效率” 还是“综合环境效率”，否则排序差异可能来自评价口径差异，而不一定代表真实绩效差异。

表 8.4: SBM 与径向 DEA / DDF 的结果解释差异

比较维度	径向 DEA / DDF	SBM
调整方式	按统一比例因子收缩投入或扩张产出	允许不同变量具有不同松弛量
松弛变量的角色	多数情况下不直接进入效率值	直接进入效率值，是效率损失的重要来源
效率值含义	主要反映比例性改进空间	同时反映比例性和非比例性改进空间
结果解释重点	关注径向效率、方向距离或相应转换效率	同时关注 `rho` 和各变量的 `slack_*`
适用场景	改进可以近似看作等比例调整	存在非比例性投入冗余、产出不足或污染过剩

表 8.4 说明，SBM 的关键改进是把松弛量纳入效率定义本身。因此，在解释 SBM 结果时，仅看 `rho` 的高低并不充分，还应结合具体变量的 `slack_x_*`、`slack_y_*` 或 `slack_b_*` 判断改进空间来自哪里。

8.6.4 与理论模型的调用对应

下面把 8.3 节、8.4 节的公式与 `deabook` 调用逐一对应，每段代码注释标注其对应的公式或小节编号。为便于对照，前两组示例统一采用VRS，第三组演示 CRS 与 VRS 的切换。

不含非期望产出的 SBM（对应 8.3 节）：

```
# 投入与期望产出综合 SBM（hyper_orientedyx）—— 对应 8.3.3 节非导向 SBM,即式(8.1)
model_yx = DEA.sbm(
    data=ex4, sent="K L=Y", gy=[1], gx=[1, 1], rts=RTS_VRS1,
    evalindex="t=[1,2,3]", refindex="t=[1,2,3]",
)
result_yx = model_yx.optimize(OPT_LOCAL, "highs")

# 投入导向 SBM（input_oriented）—— 对应 8.3.1 节
model_i = DEA.sbm(
    data=ex4, sent="K L=Y", gy=[0], gx=[1, 1], rts=RTS_VRS1,
    evalindex="t=[1,2,3]", refindex="t=[1,2,3]",
```

```
)
result_i = model_i.optimize(OPT_LOCAL, "highs")

# 期望产出导向 SBM（output_oriented）—— 对应 8.3.2 节
model_o = DEA.sbm(
    data=ex4, sent="K L=Y", gy=[1], gx=[0, 0], rts=RTS_VRS1,
    evalindex="t=[1,2,3]", refindex="t=[1,2,3]",
)
result_o = model_o.optimize(OPT_LOCAL, "highs")
```

含非期望产出的 SBM（对应 8.4 节，把非期望产出加入 sent 结构）：

```
# 综合环境 SBM（hyper_orientedyxb）—— 对应 8.4.1 节，即式(8.3)
model_env_yxb = DEAweak.sbmu(
    data=ex4, sent="K L E=Y:CO2", gy=[1], gx=[1, 1, 1], gb=[1], rts=RTS_VRS1,
    evalindex="t=[1,2,3]", refindex="t=[1,2,3]",
)
result_env_yxb = model_env_yxb.optimize(OPT_LOCAL, "highs")

# 非期望产出导向 SBM（unoutput_oriented）—— 对应 8.4.5 节
result_env_b = DEAweak.sbmu(
    data=ex4, sent="K L E=Y:CO2", gy=[0], gx=[0, 0, 0], gb=[1], rts=RTS_VRS1,
    evalindex="t=[1,2,3]", refindex="t=[1,2,3]",
).optimize(OPT_LOCAL, "highs")

# 投入导向环境 SBM（input_oriented）—— 对应 8.4.3 节
result_env_i = DEAweak.sbmu(
    data=ex4, sent="K L E=Y:CO2", gy=[0], gx=[1, 1, 1], gb=[0], rts=RTS_VRS1,
    evalindex="t=[1,2,3]", refindex="t=[1,2,3]",
).optimize(OPT_LOCAL, "highs")

# 期望产出与非期望产出综合方向(hyper_orientedyb）—— 导向SBM的混合方向变体(活跃计数口径，见8.5.2节说明
result_env_yb = DEAweak.sbmu(
    data=ex4, sent="K L E=Y:CO2", gy=[1], gx=[0, 0, 0], gb=[1], rts=RTS_VRS1,
    evalindex="t=[1,2,3]", refindex="t=[1,2,3]",
).optimize(OPT_LOCAL, "highs")

# 投入与非期望产出综合方向(hyper_orientedxb）—— 导向SBM的混合方向变体(活跃计数口径，即 8.6.5 案例所用
result_env_xb = DEAweak.sbmu(
    data=ex4, sent="K L E=Y:CO2", gy=[0], gx=[1, 1, 1], gb=[1], rts=RTS_VRS1,
```

```
    evalindex="t=[1,2,3]", refindex="t=[1,2,3]",
).optimize(OPT_LOCAL, "highs")
```

CRS 与 VRS 的设定（前两组示例统一采用 VRS；只需把 `rts` 参数由 `RTS_VRS1` 改为 `RTS_CRS`，即可在两种规模报酬假设间切换）：

```
# CRS
res_crs = DEA.sbm(
    ex4, sent="K L=Y", gy=[1], gx=[1, 1], rts=RTS_CRS
).optimize(OPT_LOCAL, "highs")

# VRS
res_vrs = DEA.sbm(
    ex4, sent="K L=Y", gy=[1], gx=[1, 1], rts=RTS_VRS1
).optimize(OPT_LOCAL, "highs")
```

8.6.5 SD 模型的Python实现

8.5 节的两类基于方向距离函数的松弛测度——Färe 与Grosskopf（2010）不含非期望产出的式(8.5)，以及 Pham 与Zelenyuk（2018）含非期望产出的式(8.6）——在 `deabook` 中由 `DEA.sddf` 与 `DEAweak.sddf` 两个工厂函数实现，二者底层共享同一个 `_SDDF` 求解引擎。与 SBM 把效率定义为相对松弛的加权平均不同，SD 直接最大化各变量绝对松弛之和，且每个变量拥有独立的松弛变量（见 8.5 节讨论），因此它对零值与负值不敏感（可平移），并且比DDF、双曲线与 Farrell 测度具有更强的判别力：一个 SD 无效的 DMU 必定在这三种测度下也无效，反之不成立。

两个接口的参数结构与 SBM 一致：`sent` 声明投入产出结构（不含非期望产出用 `DEA.sddf`，含非期望产出用 `DEAweak.sddf`），`rts` 指定规模报酬，`evalindex/refindex` 选择评价与参考样本。SD 模型的方向固定为“单位方向”——所有变量的松弛都进入目标函数——因此一般无需（也不接受）`gx/gy/gb` 的精细设定；若需限定仅部分变量参与，可显式传入对应的方向向量。`DEA.sddf` 会拒绝带有 `:` 非期望产出成分的 `sent`，`DEAweak.sddf` 则要求必须包含非期望产出成分。

```
# 不含非期望产出的 SD: Färe & Grosskopf (2010)，对应 8.5.1 节式(8.5)
model_sd = DEA.sddf(
    data=ex4, sent="K L=Y", rts=RTS_VRS1,
    evalindex="t=[1,2,3]", refindex="t=[1,2,3]",
)
result_sd = model_sd.optimize(OPT_LOCAL, "highs")

# 含非期望产出的 SD: Pham & Zelenyuk (2018)，对应 8.5.2 节式(8.6)
model_sd_bad = DEAweak.sddf(
    data=ex4, sent="K L E=Y:CO2", rts=RTS_VRS1,
    evalindex="t=[1,2,3]", refindex="t=[1,2,3]",
)
```

```
result_sd_bad = model_sd_bad.optimize(OPT_LOCAL, "highs")
```

`optimize()` 返回的 `DataFrame` 与 SBM 类似，但 SD 以绝对松弛之和作为非效率度量（而非 SBM 的 `rho`）。主要列包括：`optimization_status`（求解状态）、`objective_value`（SD 非效率值，即各绝对松弛之和，取值为 0 表示 SD 有效）、`te`（由 `objective_value` 经 $1/(1+\text{SD})$ 归一化得到的效率值，SD 有效时为 1），以及逐变量的 `slack_x_*`、`slack_y_*`、`slack_b_*`（仅含非期望产出时出现）。由于SD 同时累计投入冗余、期望产出不足与非期望产出过剩，其 `objective_value` 可以跨变量直接相加，不像 SBM 的 `rho` 那样受归一化基数影响，因而更便于在变量之间比较松弛的绝对大小。但正因如此，SD 的 `te` 只是一种基于 $1/(1+\text{SD})$ 的归一化展示，并非 SD 文献中唯一的效率定义；若需与径向 DEA、DDF 的 `te` 并列比较，应同时报告 `objective_value` 与各 `slack_*` 的绝对值。

以上示例覆盖了 8.3、8.4 与 8.5 节的主要模型。下一节把这些调用应用到 OECD 国家的真实面板数据。

8.7 案例应用：美国电厂的投入冗余与发电不足

本章只使用美国1978年19家电厂样本。数据来自Färe、Grosskopf和Kokkelenberg（1989）在International Economic Review发表的“Measuring Plant Capacity, Utilization and Technical Change: A Nonparametric Approach”；公开数据文件见deaR的Electric_plants。投入为劳动Labor、燃料Fuel与装机容量Capital，产出为发电量Output。这份数据也在其他章节使用，但本章的模型比较、结果表和松弛图均使用同一19家电厂，不加入第二份样本。

本例计算非导向普通SBM的CRS和VRS版本。该样本没有污染排放变量，不补造SO2或CO2；含非期望产出的扩展在本章理论和接口部分说明。

```
from deabook.example.book_cases.run import FUN
FUN[8]()
```

表 8.5: 美国电厂SBM效率摘要

统计量	SBM-CRS	SBM-VRS	劳动松弛比例	燃料松弛比例	装机松弛比例	发电松弛比例
有效样本数	19	19	19	19	19	19
均值	0.677	0.826	0.208	0.000	0.176	0.067
标准差	0.165	0.161	0.247	0.000	0.178	0.072
最小值	0.421	0.626	-0.000	0.000	0.000	-0.000
P10	0.536	0.640	-0.000	0.000	0.000	-0.000
中位数	0.638	0.781	0.120	0.000	0.166	0.048
P90	1.000	1.000	0.572	0.000	0.367	0.155
最大值	1.000	1.000	0.737	0.000	0.564	0.212

表将SBM效率与四项VRS相对松弛并列，提供均值、标准差、分位数和极值。松弛比例在表中用小数表

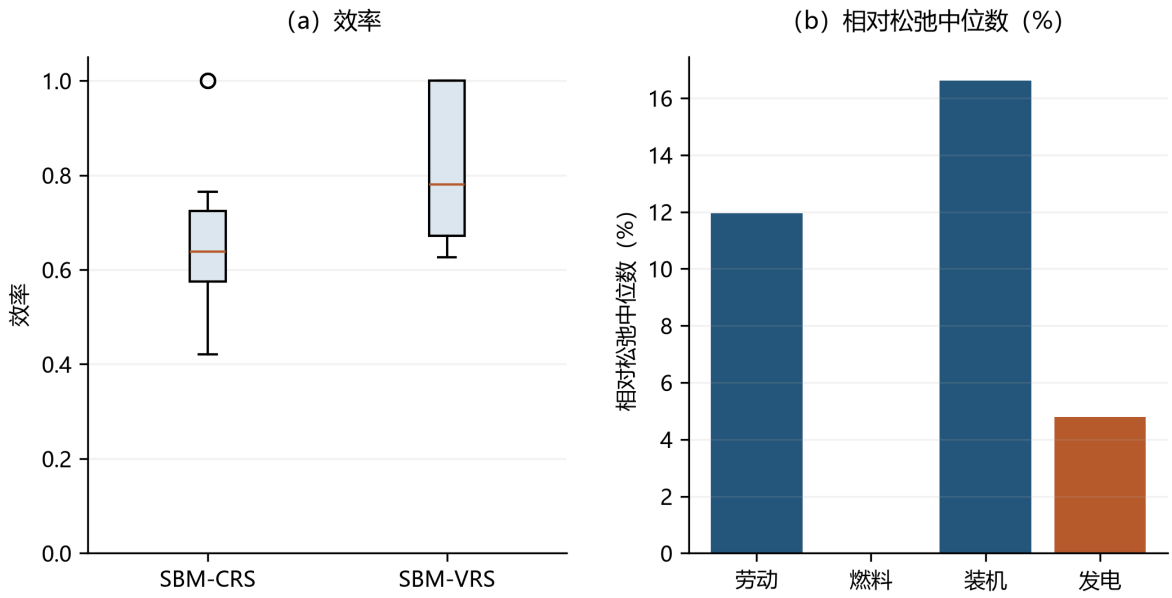


图 8.1: 美国19家电厂的SBM效率与分项松弛

示，图右换算为百分数。即使某项松弛中位数为零，也应检查均值、P90和最大值，不能据此认为所有电厂都没有改进空间。

图左比较两种规模报酬假设下的效率分布，图右报告VRS下各投入与产出的相对松弛中位数。中位数接近零不意味着所有电厂没有该项松弛，应结合完整观测判断。燃料冗余表示在样本技术和发电约束下的相对节约空间，不是工程测算的实际节煤量。完整计算结果保存在配套results/ch08_full.csv，正文同时报告效率与分项松弛的分布。

8.8 本章小结

SBM把投入冗余和产出不足纳入效率度量。本章在同一电厂样本内比较CRS与VRS，并用分项松弛图解释改进方向；污染排放扩展需要另有相应观测，不能从燃料变量直接推定。

参考文献

CHARNES A, COOPER W W, RHODES E. Measuring the efficiency of decision making units [J]. European Journal of Operational Research, 1978, 2(6): 429-444. DOI: 10.1016/0377-2217(78)90138-8.

CHARNES A, COOPER W W. Programming with linear fractional functionals [J]. Naval Research Logistics Quarterly, 1962, 9(3-4): 181-186. DOI: 10.1002/nav.3800090303.

COOPER W W, SEIFORD L M, TONE K. Data envelopment analysis: A comprehensive text with models, applications, references and DEA-solver software [M]. 2nd ed. New York: Springer, 2007. DOI: 10.1007/978-0-387-45283-8.

FEENSTRA R C, INKLAAR R, TIMMER M P. The next generation of the Penn World Table [J]. American Economic Review, 2015, 105(10): 3150-3182. DOI: 10.1257/aer.20130954.

TONE K. A slacks-based measure of efficiency in data envelopment analysis [J]. European Journal of Operational Research, 2001, 130(3): 498-509. DOI: 10.1016/S0377-2217(99)00407-5.

TONE K. Dealing with undesirable outputs in DEA: a slacks-based measure (SBM) approach [R]. Tokyo: National Graduate Institute for Policy Studies (GRIPS), GRIPS Research Report Series I-2003-0005, 2003. DOI: 10.24545/00000955. 可获取于: https://grips.repo.nii.ac.jp/record/955/files/I-2003-0005.pdf.

FÄRE R, GROSSKOPF S. Directional distance functions and slacks-based measures of efficiency [J]. European Journal of Operational Research, 2010, 200(1): 320-322. DOI: 10.1016/j.ejor.2009.01.031.

PHAM M D, ZELENYUK V. Slack-based directional distance function in the presence of bad outputs: theory and application to Vietnamese banking [J]. Empirical Economics, 2018, 54(1): 153-187. DOI: 10.1007/s00181-017-1232-7.

第 9 章

基于物质平衡的环境绩效测度方法

前几章已经讨论了含非期望产出的DEA、方向距离函数和动态生产率测度。这些方法能够将污染排放纳入效率评价，但在多数设定中，投入、期望产出和非期望产出之间的物质守恒关系并未被显式写入生产技术。对于能源、农业和工业排放等场景，如果忽略污染物质在投入端、产出端和排放端之间的流动关系，环境绩效测度可能将物理上不可行的投影点误判为可行技术，进而影响对效率得分、减排潜力和低效率来源的判断。

基于此，本章介绍物质平衡原则（Material Balance Principle, MBP）下的环境绩效测度方法。全章围绕“为什么需要物质守恒约束—物质平衡如何改变可行技术—如何形成非参数线性规划模型—如何在 deabook 中计算和解释分解指标” 这一主线展开。本章模型提供的是相对于样本前沿的测算结果，不对某项政策或外生冲击作因果识别；其核心贡献在于将环境绩效差异进一步分解为技术效率、污染投入配置、非污染投入配置和产出配置等可解释来源。

本章符号约定（与第 3–8、10 章不同）：本章案例以火电行业为对象，`E` 表示**电力（期望产出）**、`F` 表示**燃料（污染投入）**；而本书其他章节（第 3、5、6、7、8、10 章）的 `E` 一律表示**能源投入**。阅读本章案例（9.6 节、9.7 节）时，请注意 `sent="K L+F=E:CO2"` 中= 右侧的 `E` 是期望产出，而非投入。

9.1 物质平衡原则

在 DEA 文献中，非期望产出的处理大体有四类思路：第一，将非期望产出作为具有强可处置性的投入（Hailu and Veeman, 2001）；第二，将非期望产出作为具有弱可处置性的产出（Färe and Grosskopf, 2004; Färe et al., 1989）；第三，将期望产出和非期望产出分别置于不同子技术中刻画，例如副生产法以及自然处置/管理处置框架（Murty et al., 2012; Sueyoshi and Goto, 2012）；第四，在生产技术中引入物质平衡原则，约束污染物质从投入到产出和排放的流动。第四类思路中，物质平衡条件下的环境效率测度由Coelli, Lauwers and Van Huylenbroeck（2007）提出（以最小污染投入与观测污染投入之比定义环境效率），Welch and Barnum（2009）在此思路下考察发电的成本—环境联合效率；把**弱G 可处置性**（weak G-disposability，即只允许在满足物质平衡的方向上处置）确立为生产技术公理的是Rødseth（2016, 2017），其非参数 DEA 实现见 Hampf and Rødseth（2015）。

上述方法的核心差异在于是否承认污染物质具有明确的物理来源。将非期望产出直接当作投入，虽然形式上便于建模，却难以反映“投入经过生产过程形成期望产出和排放” 的经济含义。单纯的弱可处置设

定也可能在缺乏末端治理活动时允许不符合物质守恒的排放缩减，从而影响影子价格和效率测度（Hampf and Rødseth, 2015; Dakpo et al., 2016; Førsund, 2009）。因此，物质平衡模型的目的在于将污染物质从投入到产出和排放的转换关系纳入技术集本身，而非简单增加约束。下一节将通过证明传统径向效率度量在物质平衡条件下退化为1，说明为何必须重新定义生产技术与投影方向。

9.2 物质平衡条件

Ayres 和Kneese（1969）提出的物质平衡原则指出，在不考虑存量变化和外部减排活动时，投入中携带的特定物质或能量，应当等于期望产出中保留下来的相应物质或能量与非期望产出中排放出去的相应物质或能量之和（Coelli et al., 2007）。不考虑末端减排时，物质平衡条件可写为：

$$S_x x - S_y y = b \tag{9.1}$$

其中，x、y 和 b 分别表示投入、期望产出和非期望产出向量；S_x 和 S_y 分别表示投入端和期望产出端的单位物质含量系数。以养殖业为例，饲料和仔猪中的氮元素会在成品猪和粪便之间分配；以火力发电为例，燃料中的碳、硫、氮等元素会相应形成电力生产过程中的排放。为便于表述，下文将这类需要满足守恒关系的元素或污染载体统称为“污染物质”，并将携带污染物质的投入/产出简称**污染投入/污染产出**，不携带的简称**非污染投入/非污染产出**（详见 9.2.2 节）。

在式（9.1）中，不携带污染物质的投入对应的 S_x 分量为零，携带污染物质的投入对应分量大于零；同理，不携带污染物质的期望产出对应的 S_y 分量为零，携带污染物质的期望产出对应分量大于零。因此，降低非期望产出有多条途径：可以来自污染投入的减少、污染投入结构的变化，也可以来自污染物质更多地保留在期望产出中，而非仅靠“直接压减排放”一条路径。物质平衡模型正是通过这些约束区分不同来源的效率改进空间。

9.2.1 距离函数与物质平衡条件

前几章已经介绍了谢泼德产出距离函数、谢泼德投入距离函数、双曲线距离函数和方向距离函数等效率测度方式。在定义符合物质平衡原则的生产技术之前，有必要先考察这些距离函数在何种条件下与式（9.1）相容。基本结论是：当携带污染物质的期望产出分量为正（即 $S_y y \neq 0$）时，传统谢泼德距离函数和双曲线距离函数通常难以同时满足物质平衡条件；方向距离函数只有在投影方向满足特定物质守恒关系时，才与物质平衡条件相容。

为说明这一点，考虑一个生产单元使用 K 个投入 $x \in R_+^K$ 生产 L 个期望产出 $y \in R_+^L$，并产生非期望产出 b。排放量由物质平衡方程 $S_x x - S_y y = b$ 决定，其中 S_x 和 S_y 是已知的非负物质含量系数。

首先，考虑 Färe 等（1989）的双曲线效率度量。该方法寻找最大的标量 λ，使得放缩后的组合 $(\lambda y, x/\lambda, b/\lambda)$ 仍位于可行生产集中。若将该放缩方式代入物质平衡方程，可得：

$$b/\lambda = S_x x/\lambda - S_y y\lambda \tag{9.2}$$

这表示，当携带污染物质的期望产出分量为正（即 $S_y y \neq 0$）时，

$$b - S_x x = -S_y y\lambda^2 \tag{9.3}$$

此时，$\lambda = 1$ 是唯一能够同时满足双曲线效率测度和物质平衡方程的正值。由于观测点本身满足物质平衡条件 $S_x x - S_y y = b$，即 $S_y y = S_x x - b$，代入上式并由 $S_y y \neq 0$ 得 $\lambda^2 = 1$，故 $\lambda = 1$。模型不再允许低效率生产单元沿该比例方向投影到可行技术内部，效率测度由此失去区分低效率程度的作用。

Färe 等（1996）的投入导向效率度量也会出现类似问题。该方法寻找最大的标量 λ，使得 $(y, x/\lambda, b)$ 保持可行。将其代入物质平衡方程，可得：

$$b = S_x x/\lambda - S_y y \tag{9.4}$$

这表示，对任意 S_y（含 $S_y = 0$），

$$\left(b + S_y y\right)\lambda = S_x x \tag{9.5}$$

因此，$\lambda = 1$ 仍是唯一在数学上可行的值。由于观测点满足 $S_x x - S_y y = b$，即 $(b + S_y y) = S_x x$，代入上式得 $S_x x \cdot \lambda = S_x x$，故 $\lambda = 1$。

对 Reinhard 等（2000）使用的其他效率度量，即 $(y, x, b/\lambda)$、$(\lambda y, x, b)$ 和 $(y, x/\lambda, b/\lambda)$ 进行同样检查，也会得到相近结论：在 $S_y y \neq 0$ 的条件下，若投影路径没有同步满足物质守恒关系，与物质平衡条件一致的效率分数只能退化为1。这说明必须重新定义环境生产技术和允许的投影方向。

方向距离函数的优势在于可以显式指定投入、期望产出和非期望产出的联合调整方向。当方向向量设定为 $(-g_x, g_y, -g_b)$ 时，方向距离函数定义为（Chung et al., 1997）：

$$\vec{D}_i\left(x, y, b; -g_x, g_y, -g_b\right) = \sup\left\{\beta : \left(x - \beta g_x, y + \beta g_y, b - \beta g_b\right) \in T\right\} \tag{9.6}$$

式中，T 为包含非期望产出的生产技术；g_x、g_y 和 g_b 分别表示投入缩减、期望产出扩张和非期望产出缩减的方向；β 表示沿给定方向可改进的最大幅度。

投入和产出的变动需符合物质平衡条件，即：

$$(b - \beta g_b) = S_x(x - \beta g_x) - S_y(y + \beta g_y) \tag{9.7}$$

可得：

$$b - S_x x + S_y y = \beta(g_b - S_x g_x - S_y g_y) \tag{9.8}$$

由于观测点本身满足物质平衡条件 $S_x x - S_y y = b$，上式左端恒为零。为使**投影后的点** $(x - \beta g_x, y + \beta g_y, b - \beta g_b)$ 也满足物质平衡条件，必须有 $\beta(g_b - S_x g_x - S_y g_y) = 0$。该条件有两种满足方式：一是 $\beta = 0$，即生产单元无可改进空间（投影退化为不动点）；二是投影方向本身满足 $g_b - S_x g_x - S_y g_y = 0$。后一种情形允许 β 取非负值，是构造物质平衡方向距离函数的关键。当 $S_y = 0$ 时，上述条件退化为 $g_b - S_x g_x = 0$，表明期望产出的调整不改变污染物质守恒关系。

9.2.2 符合物质平衡的生产技术的公理

在得到方向距离函数与物质平衡条件相容的方向约束后，需要进一步定义符合该约束的生产技术。考虑一个生产过程，其中 K 个投入 $x \in R_+^K$ 被用于生产 L 个期望产出 $y \in R_+^L$。为便于下文表述，将“包含污染物质的投入”简称**污染投入**（x^P，共 K_1 个），“不包含污染物质的投入”简称**非污染投入**（x^{NP}，共 $K_2 = K - K_1$ 个），即 $x = \begin{bmatrix} x^P \\ x^{NP} \end{bmatrix}$；类似地，将期望产出划分为**污染产出**（$y^P$，共 L_1 个）和**非污染产出**（y^{NP}，共 $L_2 = L - L_1$ 个）。污染投入经生产过程转化后，会形成 M 个非期望产出 $b \in R_+^M$。生产技术集 T 定义为所有技术上可行的投入—产出组合：

$$T = \{(x, y, b) : x \text{ 能生产 } (y, b)\} \tag{9.9}$$

根据上一小节，方向距离函数满足物质平衡条件的关键在于方向向量满足 $g_b - S_x g_x - S_y g_y = 0$。Rødseth（2016）将这一关系纳入物质平衡生产技术的公理体系。其含义是，污染投入、污染产出和非期望产出的调整必须共同满足物质守恒约束，三者彼此相关。基于此，可将符合物质平衡条件的技术集设定为满足以下假设：

(1) T 是闭集；并且对任意给定的投入 x，可行产出集 $P(x) = \{(y,b) : (x,y,b) \in T\}$ 有界（有界性针对给定投入下的产出集，而不是 T 本身——CRS 下技术集是锥，沿射线无界）；

(2) 没有免费的午餐：若 $(y,b) \geq (0,0)$，则 $(0,y,b) \notin T$；

(3) 非污染投入具有强可处置性；

(4) 非污染产出具有强可处置性；

(5) 污染投入对非期望产出的要素必要性：若 $(x,y,b) \in T$，且 $x^P = 0$，则 $b = 0$；

(6) 非期望产出对污染投入的存在性对应：若 $(x,y,b) \in T$，且 $b = 0$，则 $x^P = 0$；

(7) 投入和产出满足弱 G 可处置性：若 $(x,y,b) \in T$，且方向向量满足 $g_b - S_x g_x - S_y g_y = 0$，则 $(x - g_x, y + g_y, b - g_b) \in T$。

(8) T 是凸的；

(9) 规模报酬可设定为可变（VRS）或不变（CRS）规模报酬。

公理 (5) 和 (6) 刻画了污染投入与非期望产出之间的**要素必要性（存在性对应）**：没有污染投入就不会产生相应非期望产出，反之，观测不到非期望产出也意味着没有污染投入。需要与第 6 章的**零结合性（null-jointness）**相区别：后者说的是好产出与坏产出之间的联合性，即若 $(x,y,b) \in T$ 且 $b = 0$，则 $y = 0$（无污染即无产出）；本章 (5)(6) 是物质守恒下 x^P 与 b 的对应，两者不是同一条公理，不应混用同一名称。公理 (7) 进一步刻画了二者之间的调整对应关系，是落实物质平衡的实质约束：投入和产出的调整必须满足物质守恒关系，即污染投入减少所降低的物质流入，以及污染产出扩张所吸收的物质量，必须与非期望产出的减少相匹配。该条件排除了污染投入和污染产出的任意强可处置性，但其他投入和产出仍可调整；非污染投入和非污染产出仍按照强可处置性处理。

9.3 物质平衡生产技术的非参数刻画

将公理 (7) 中的方向向量 (g_x, g_y, g_b) 替换为松弛向量 $(\epsilon_x, \epsilon_y, \epsilon_b)$，方向守恒条件 $g_b - S_x g_x - S_y g_y = 0$ 即对应为 $S_x \epsilon_x + S_y \epsilon_y = \epsilon_b$。在非参数DEA 框架中，可用这些松弛变量表示各维度相对于参考技术的调整幅度，并据此落实弱G 可处置性中的物质守恒要求。规模报酬不变下，物质平衡技术可估计为：

$$
\begin{aligned}
\widehat{T}_{CRS}^{MB} = \{ \quad & (x,y,b) : \\
& x = X\lambda + \epsilon_x, \\
& y = Y\lambda - \epsilon_y, \\
& b = B\lambda + \epsilon_b, \\
& S_x \epsilon_x + S_y \epsilon_y = \epsilon_b, \\
& \epsilon_x, \epsilon_y, \epsilon_b, \lambda \geqq \mathbf{0} \}
\end{aligned}
\tag{9.10}
$$

在规模报酬可变下，只需进一步加入强度变量之和等于 1 的约束：

$$\begin{aligned}\hat{T}_{VRS}^{MB} = \{ \quad & (x,y,b): \\ & x = X\lambda + \epsilon_x, \\ & y = Y\lambda - \epsilon_y, \\ & b = B\lambda + \epsilon_b, \\ & S_x\epsilon_x + S_y\epsilon_y = \epsilon_b, \\ & \mathbf{1}^T\lambda = 1, \\ & \epsilon_x, \epsilon_y, \epsilon_b, \lambda \geqq \mathbf{0}\}\end{aligned} \tag{9.11}$$

因此，物质平衡非参数技术的核心约束是 $S_x\epsilon_x + S_y\epsilon_y = \epsilon_b$。该式要求投入和期望产出松弛引起的污染物质变动必须与非期望产出松弛相一致。若进一步假定 $\epsilon_b = 0$，模型实际上考察的是在非期望产出不发生净变化时，投入和期望产出结构如何调整才能满足物质平衡。与一般联合生产技术相比，MB 技术的关键差异就在于增加了这一物质守恒约束，并据此形成弱 G 可处置性。

应用该模型时，S_x 和 S_y 需要在估计前给定。Wang 等（2018）在研究火电行业时，由于期望产出电力不携带碳元素，将 S_y 设为零，并用二氧化碳排放量与燃料投入之比构造投入端污染系数 S_x。这一处理是基于样本观测值的排放系数近似，而非独立于生产技术的结构参数。

9.4 绩效测度

9.4.1 非期望产出方向上的技术效率

基于上一节的非参数技术，本节构造可操作的环境绩效指标，首先考察非期望产出方向上的技术效率。对每个DMU，可以将目标设定为在给定物质平衡约束下最小化非期望产出。由于物质平衡约束把非期望产出与投入、期望产出绑定，“最小化非期望产出”本质上是寻找使排放最低的投入—产出再配置，而非对排放端的任意压缩。随着可调整变量范围逐步扩大，模型可将总体环境效率分解为技术效率、污染投入配置效率、非污染投入配置效率、污染产出配置效率和非污染产出配置效率。下面按调整成本递增的顺序逐层放松约束——先允许污染投入结构调整（如同质燃料替代），再允许非污染投入再配置（资本、劳动的重新组合），最后允许产出结构调整（产出组合改变涉及需求侧，通常调整成本较高）。这样，每一层级的效率损失对应着越来越难以调整的来源。

第一层级：污染投入保持不变。此时，最小可达非期望产出与实际非期望产出之比，用于衡量非期望产出方向上的技术效率。

在 CRS 假设下，该层级的最小可达排放可通过下式求解：

$$\begin{aligned} B_1 = \quad & \min_{\lambda,\epsilon,b_m} \tfrac{1}{M}\textstyle\sum_{m=1}^{M} b_m \\ & \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{P} + \epsilon_{kx^P} = x_{k0}^{P}, \quad k = 1,\dots,K_1 \\ & \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_{k0}^{NP}, \quad k = K_1+1,\dots,K \\ & \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{P} - \epsilon_{ly^P} = y_{l0}^{P}, \quad l = 1,\dots,L_1 \\ & \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_{l0}^{NP}, \quad l = L_1+1,\dots,L \\ & \textstyle\sum_{i=1}^{N} \lambda_i b_{mi} + \epsilon_{mb} = b_m, \quad m = 1,\dots,M \\ & \textstyle\sum_{k=1}^{K_1} S_{x_{km}}\epsilon_{kx^P} + \sum_{l=1}^{L_1} S_{y_{lm}}\epsilon_{ly^P} = (\epsilon_{mb} + b_{m0} - b_m), \quad m = 1,\dots,M \\ & \lambda_i \geq 0, \quad i = 1,\dots,N \end{aligned} \tag{9.12}$$

由于目标函数最小化非期望产出之和，最优解中恒有 $\epsilon_{mb}=0$。其机理是：将非期望产出约束 $\sum_i \lambda_i b_{mi}+\epsilon_{mb}=b_m$ 代入物质平衡约束 $\sum_k S_{x_{km}}\epsilon_{kx^P}+\sum_l S_{y_{lm}}\epsilon_{ly^P}=\epsilon_{mb}+b_{m0}-b_m$ 消元，ϵ_{mb} 恒等消去，不再独立约束 λ 与其他松弛；而 $\epsilon_{mb}\geq 0$ 仅以 $+\epsilon_{mb}$ 形式进入目标项 b_m，故极小化目标必然将其压至下界 0。据此在式(9.12)中令 $\epsilon_{mb}=0$ 并消去该项，即得等价的简约形式(式9.13)。该消去论证仅依赖非期望产出约束与目标函数的形式(二者在各层级不变)，故对各层级均成立。下文统一采用简约形式(deabook 的 DEAmb 模块实现的也是简约形式)。

$$
\begin{aligned}
B_1 = \quad & \min_{\lambda,\epsilon,b_m} \tfrac{1}{M}\textstyle\sum_{m=1}^{M} b_m \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^P + \epsilon_{kx^P} = x_{k0}^P, \quad k=1,\dots,K_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_{k0}^{NP}, \quad k=K_1+1,\dots,K \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^P - \epsilon_{ly^P} = y_{l0}^P, \quad l=1,\dots,L_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_{l0}^{NP}, \quad l=L_1+1,\dots,L \\
& \textstyle\sum_{i=1}^{N} \lambda_i b_{mi} + \epsilon_{mb} = b_m, \quad m=1,\dots M \\
& \textstyle\sum_{k=1}^{K_1} S_{x_{km}}\epsilon_{kx^P} + \sum_{l=1}^{L_1} S_{y_{lm}}\epsilon_{ly^P} = (b_{m0}-b_m), \quad m=1,\dots,M \\
& \lambda_i \geq 0 \quad i=1,\dots,N
\end{aligned}
\tag{9.13}
$$

第二层级：在第一层级基础上，允许污染投入调整。此时，新的最小可达非期望产出与第一层级下的最小可达非期望产出之比，用于衡量污染投入配置效率。

在 CRS 假设下，第二层级的最小可达排放可通过下式求解：

$$
\begin{aligned}
B_2 = \quad & \min_{\lambda,\epsilon,b_m,x^P} \tfrac{1}{M}\textstyle\sum_{m=1}^{M} b_m \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^P + \epsilon_{kx^P} = x_{k}^P, \quad k=1,\dots,K_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_{k0}^{NP}, \quad k=K_1+1,\dots,K \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^P - \epsilon_{ly^P} = y_{l0}^P, \quad l=1,\dots,L_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_{l0}^{NP}, \quad l=L_1+1,\dots,L \\
& \textstyle\sum_{i=1}^{N} \lambda_i b_{mi} + \epsilon_{mb} = b_m, \quad m=1,\dots M \\
& \textstyle\sum_{k=1}^{K_1} S_{x_{km}}\left(x_{k0}^P - x_k^P\right) + \sum_{l=1}^{L_1} S_{y_{lm}}\epsilon_{ly^P} = (b_{m0}-b_m), \quad m=1,\dots,M \\
& \lambda_i \geq 0 \quad i=1,\dots,N
\end{aligned}
\tag{9.14}
$$

由于污染投入被允许调整，污染物质流入的减少由实际污染投入从 x_{k0}^P 调整到 x_k^P 所带来的变化刻画，即 $S_{x_{km}}\left(x_{k0}^P-x_k^P\right)$，而不再仅由松弛项 ϵ_{kx^P} 表示。若把松弛项也显式写入物质平衡约束，可得到等价表达 $S_{x_{km}}\left(\epsilon_{kx^P}+x_{k0}^P-x_k^P\right)$；两种写法的最优值相同，因为目标函数极小化 b_m 且 $S_{x_{km}}\geq 0$，最优解中 ϵ_{kx^P} 被压至下界 0（污染投入被压至前沿组合 $\sum_i \lambda_i x_{ki}^P$）。两种形式可统一记为：

$$
\sum_{k=1}^{K_1} S_{x_{km}}\left(\epsilon_{kx^P}+x_{k0}^P-x_k^P\right) + \sum_{l=1}^{L_1} S_{y_{lm}}\epsilon_{ly^P} = (b_{m0}-b_m), \quad m=1,\dots,M \tag{9.15}
$$

其余约束与式(9.14)相同。

在第二层级的基础上，进一步允许非污染投入调整，可得到第三层级。

第三层级：非污染投入可以调整。此时，最小可达非期望产出与第二层级下最小可达非期望产出之比，用于衡量非污染投入配置效率。

在 CRS 假设下，第三层级的最小可达排放可通过下式求解：

$$
\begin{aligned}
B_3 = \quad & \min_{\lambda,\epsilon,b_m,x} \tfrac{1}{M}\textstyle\sum_{m=1}^{M} b_m \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{P} + \epsilon_{kx^P} = x_k^P, \quad k = 1,\dots,K_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_k^{NP}, \quad k = K_1+1,\dots,K \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{P} - \epsilon_{ly^P} = y_{l0}^P, \quad l = 1,\dots,L_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_{l0}^{NP}, \quad l = L_1+1,\dots,L \\
& \textstyle\sum_{i=1}^{N} \lambda_i b_{mi} + \epsilon_{mb} = b_m, \quad m = 1,\dots M \\
& \textstyle\sum_{k=1}^{K_1} S_{x_{km}} (x_{k0}^P - x_k^P) + \sum_{l=1}^{L_1} S_{y_{lm}} \epsilon_{ly^P} = (b_{m0} - b_m), \quad m = 1,\dots,M \\
& \lambda_i \geq 0 \quad i = 1,\dots,N
\end{aligned}
\tag{9.16}
$$

在第三层级的基础上，若进一步允许污染产出调整，可得到第四层级。

第四层级：污染产出可以调整。此时，最小可达非期望产出与第三层级下最小可达非期望产出之比，用于衡量污染产出配置效率。

在 CRS 假设下，第四层级的最小可达排放可通过下式求解：

$$
\begin{aligned}
B_4 = \quad & \min_{\lambda,\epsilon,b_m,x,y^P} \tfrac{1}{M}\textstyle\sum_{m=1}^{M} b_m \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{P} + \epsilon_{kx^P} = x_k^P, \quad k = 1,\dots,K_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_k^{NP}, \quad k = K_1+1,\dots,K \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{P} - \epsilon_{ly^P} = y_l^P, \quad l = 1,\dots,L_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_{l0}^{NP}, \quad l = L_1+1,\dots,L \\
& \textstyle\sum_{i=1}^{N} \lambda_i b_{mi} + \epsilon_{mb} = b_m, \quad m = 1,\dots M \\
& \textstyle\sum_{k=1}^{K_1} S_{x_{km}} (x_{k0}^P - x_k^P) + \sum_{l=1}^{L_1} S_{y_{lm}} (y_l^P - y_{l0}^P) = (b_{m0} - b_m), \quad m = 1,\dots,M \\
& \lambda_i \geq 0 \quad i = 1,\dots,N
\end{aligned}
\tag{9.17}
$$

同理，当污染产出可调整时，期望产出端所吸收的污染物质量需要由 $S_{y_{lm}}(y_l^P - y_{l0}^P)$ 表示，而不仅由 ϵ_{ly^P} 表示。在最小化非期望产出的目标下，若期望产出能够承载更多污染物质，扩大 y_l^P 会相应降低排放端的物质压力。

在第四层级的基础上，若继续允许非污染产出调整，可得到第五层级。

第五层级：非污染产出可以调整。此时，最小可达非期望产出与第四层级下最小可达非期望产出之比，用于衡量非污染产出配置效率；第五层级下的最小可达非期望产出与实际非期望产出之比，则用于衡量整体环境效率。需注意，第五层级允许产出调整可能引入**停产退化**：当产出可调时，模型可能选择产出与污染投入同步归零的平凡解，使 B_5 触零。此时 NPOAE=0 反映的是模型允许停产而非有经济意义的产出再配置空间；若研究目的在于排除停产情形，可在第五层级加入产出下限约束（如 $y_l^{NP} \geq y_{l0}^{NP}$）或改用 Hampf 和 Rødseth（2015）的最大化 E/b 目标。

在 CRS 假设下，第五层级的最小可达排放可通过下式求解：

$$
\begin{aligned}
B_5 = \quad & \min_{\lambda,\epsilon,b_m,x,y} \tfrac{1}{M}\textstyle\sum_{m=1}^{M} b_m \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{P} + \epsilon_{kx^P} = x_k^P, \quad k = 1,\dots,K_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_k^{NP}, \quad k = K_1+1,\dots,K \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{P} - \epsilon_{ly^P} = y_l^P, \quad l = 1,\dots,L_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_l^{NP}, \quad l = L_1+1,\dots,L \\
& \textstyle\sum_{i=1}^{N} \lambda_i b_{mi} + \epsilon_{mb} = b_m, \quad m = 1,\dots M \\
& \textstyle\sum_{k=1}^{K_1} S_{x_{km}} \left(x_{k0}^P - x_k^P\right) + \sum_{l=1}^{L_1} S_{y_{lm}} \left(y_l^P - y_{l0}^P\right) = (b_{m0} - b_m), \quad m = 1,\dots,M \\
& \lambda_i \geq 0 \quad i = 1,\dots,N
\end{aligned}
\tag{9.18}
$$

与式(9.15)同理，若把松弛项显式写入物质平衡约束，第五层级的物质平衡约束可等价写为：

$$
\sum_{k=1}^{K_1} S_{x_{km}} \left(\epsilon_{kx^P} + x_{k0}^P - x_k^P\right) + \sum_{l=1}^{L_1} S_{y_{lm}} \left(\epsilon_{ly^P} + y_l^P - y_{l0}^P\right) = (b_{m0} - b_m), \quad m = 1,\dots,M \tag{9.19}
$$

其余约束与式(9.18)相同；两种写法最优值一致(最优解处 $\epsilon_{kx^P} = 0$、$\epsilon_{ly^P} = 0$)。将上述五个层级得到的最优非期望产出分别记为 $B_1,\dots,B_5$，可得到如下分解指标：

$$
\text{整体效率}(OE) = \frac{B_5}{\frac{1}{M}\sum_{m=1}^{M} b_{m0}} \tag{9.20}
$$

$$
\text{非期望产出方向的技术效率}(TE) = \frac{B_1}{\frac{1}{M}\sum_{m=1}^{M} b_{m0}} \tag{9.21}
$$

$$
\text{包含污染物质的投入配置效率}(PIAE) = \frac{B_2}{B_1} \tag{9.22}
$$

$$
\text{不包含污染物质的投入配置效率}(NPIAE) = \frac{B_3}{B_2} \tag{9.23}
$$

$$
\text{包含污染物质的产出配置效率}(POAE) = \frac{B_4}{B_3} \tag{9.24}
$$

$$
\text{不包含污染物质的产出配置效率}(NPOAE) = \frac{B_5}{B_4} \tag{9.25}
$$

整体效率可以分解为非期望产出方向的技术效率、污染投入配置效率、非污染投入配置效率、污染产出配置效率和非污染产出配置效率：

$$
OE = TE \cdot PIAE \cdot NPIAE \cdot POAE \cdot NPOAE \tag{9.26}
$$

其中各缩写的英文全称为：OE=Overall Efficiency（整体效率），TE=Technical Efficiency（技术效率），$PIAE$=Polluting Input Allocative Efficiency（污染投入配置效率），$NPIAE$=Non-Polluting Input Allocative Efficiency（非污染投入配置效率），$POAE$=Polluting Output Allocative Efficiency（污染产出配置效率），$NPOAE$=Non-Polluting Output Allocative Efficiency（非污染产出配置效率）。在同一规模报酬假设下，每一层级相对上一层放松了约束，可行域单调扩大，故 $B_1 \geq B_2 \geq B_3 \geq B_4 \geq B_5$，各效率比值均落在 $[0,1]$

（下界 0 表示该层级最小可达非期望产出已降至零，如产出调整导致停产退化时 $B_5 = 0$，此时该比值失去常规经济解释力）。**必须强调：这一单调性以及各比值的配置效率解释，都要求各层级使用同一规模报酬假设。**若第一层级用VRS、第二层级用 CRS，则 B_2^{CRS}/B_1^{VRS} 同时混入了规模报酬变化，不再是纯粹的“放松污染投入约束”所带来的配置效率，不应称为PIAE（9.7 节给出该口径下的具体数值证据）。

还需注意上述简约形式的一个结构性含义：**在同一规模报酬假设下，$B_2 = B_1$ 恒成立，因而** $PIAE \equiv 1$。理由有两点。第一，把污染投入约束 $\sum_i \lambda_i x_{ki}^P + \epsilon_{kx^P} = x_k^P$ 代入第二层级的物质平衡约束（9.15），污染物质流入的减少量为 $\epsilon_{kx^P} + x_{k0}^P - x_k^P = x_{k0}^P - \sum_i \lambda_i x_{ki}^P$，与第一层级（9.13）中的 $\epsilon_{kx^P} = x_{k0}^P - \sum_i \lambda_i x_{ki}^P$ 完全相同——第一层级的投入松弛已经把“污染投入相对前沿组合的节约”计入了物质平衡，第二层级把 x^P 设为决策变量并未打开新的减排通道。第二，第二层级唯一多出的自由度是允许 $\sum_i \lambda_i x_{ki}^P > x_{k0}^P$（此时 $x_k^P > x_{k0}^P$），但这会使 $b_m > b_{m0}$；而取 $\lambda_0 = 1$、其余为零总是第一层级的可行解并给出 $b_m = b_{m0}$，故 $B_1 \le b_{m0}$，这一多出的自由度永不最优。这意味着污染投入配置效率在式（9.12）—（9.18）的**非径向**简约设定下不被单独识别：它已被吸收进第一层级的技术效率。若研究者需要一个非退化的PIAE，必须改变第一层级的设定本身（例如在第一层级要求 $\epsilon_{kx^P} = 0$、即技术效率不把投入松弛记作物质节约），而不是靠改变层级之间的规模报酬假设来制造差异。9.7 节案例将验证这一点。当采用规模报酬可变技术时，只需在相应线性规划中加入 $\sum_{i=1}^N \lambda_i = 1$。除最小化非期望产出外，文献中也可将目标函数设定为最大化期望产出与非期望产出的比值，相关处理可参见Hampf 和 Rødseth（2015）。

当 $S_y = 0$ 时，表示期望产出不携带所考察的污染物质。例如，在火力发电中，若考察燃料燃烧形成的 CO_2，期望产出电力本身不携带碳元素，则可令 $S_y = 0$。此时，污染物质守恒主要通过投入端排放系数与非期望产出之间的关系体现。

当 $S_y = 0$ 时，第四层级可简化为：

$$
\begin{aligned}
B_4' = \quad & \min_{\lambda,\epsilon,b_m,x,y^P} \tfrac{1}{M}\textstyle\sum_{m=1}^M b_m \\
& \textstyle\sum_{i=1}^N \lambda_i x_{ki}^P + \epsilon_{kx^P} = x_k^P, \quad k = 1,\dots,K_1 \\
& \textstyle\sum_{i=1}^N \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_k^{NP}, \quad k = K_1+1,\dots,K \\
& \textstyle\sum_{i=1}^N \lambda_i y_{li}^P - \epsilon_{ly^P} = y_l^P, \quad l = 1,\dots,L_1 \\
& \textstyle\sum_{i=1}^N \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_{l0}^{NP}, \quad l = L_1+1,\dots,L \\
& \textstyle\sum_{i=1}^N \lambda_i b_{mi} + \epsilon_{mb} = b_m, \quad m = 1,\dots M \\
& \textstyle\sum_{k=1}^{K_1} S_{x_{km}} (x_{k0}^P - x_k^P) = (b_{m0} - b_m), \quad m = 1,\dots,M \\
& \lambda_i \ge 0 \quad i = 1,\dots,N
\end{aligned}
\tag{9.27}
$$

同理，第五层级可简化为：

$$
\begin{aligned}
B_5' = \quad & \min_{\lambda,\epsilon,b_m,x,y} \tfrac{1}{M}\textstyle\sum_{m=1}^M b_m \\
& \textstyle\sum_{i=1}^N \lambda_i x_{ki}^P + \epsilon_{kx^P} = x_k^P, \quad k = 1,\dots,K_1 \\
& \textstyle\sum_{i=1}^N \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_k^{NP}, \quad k = K_1+1,\dots,K \\
& \textstyle\sum_{i=1}^N \lambda_i y_{li}^P - \epsilon_{ly^P} = y_l^P, \quad l = 1,\dots,L_1 \\
& \textstyle\sum_{i=1}^N \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_l^{NP}, \quad l = L_1+1,\dots,L \\
& \textstyle\sum_{i=1}^N \lambda_i b_{mi} + \epsilon_{mb} = b_m, \quad m = 1,\dots M \\
& \textstyle\sum_{k=1}^{K_1} S_{x_{km}} (x_{k0}^P - x_k^P) = (b_{m0} - b_m), \quad m = 1,\dots,M \\
& \lambda_i \ge 0 \quad i = 1,\dots,N
\end{aligned}
\tag{9.28}
$$

B_4' 和 B_5' 即 $S_y = 0$ 情形下的简化写法。两式关系需区分两种情形讨论：

第一，若数据中存在污染产出（$L_1 > 0$），式（9.27）放松 y^P、式（9.28）进一步放松 y^{NP}。由于 y^{NP} 不进入物质平衡约束，从第四层级到第五层级放松 y^{NP} 仅放松构造约束 $\sum_i \lambda_i y_{li}^{NP} \geq y_{l0}^{NP}$（由 $\epsilon_{ly^{NP}} \geq 0$ 得出），从而扩大 λ 的可行集，故 $B_5' \leq B_4'$（恒成立）。

第二，若数据中不存在污染产出（$L_1 = 0$，即本案例情形：电力 E 归入非污染产出，y^P 为空），第四层级（污染产出调整）为空操作，$B_4' = B_3$，POAE（B_4'/B_3）退化为1；此时 `deabook` 的 `level=4` 直接计算第五层级（非污染产出可调），为简化记法其输出仍记为 B_4。故本案例中 OE 由 B_4/CO2 计算、NPOAE 由 B_4/B_3 计算；因 $B_4' = B_3$，该 NPOAE 对应完整模型的NPOAE（$B_5'/B_4' = B_5'/B_3$）。需注意，POAE 退化为 1 的真正条件是 $L_1 = 0$（无污染产出），而非 $S_y = 0$；当 $L_1 > 0$ 但 $S_y = 0$ 时，POAE 仍可严格小于1。第五层级允许产出调整可能引入停产退化（见式 9.18 处说明），9.7 节案例中将进一步讨论。

9.4.2 投入方向上的技术效率

除非期望产出方向外，也可以从投入方向刻画效率。投入方向模型的基本思想是：在给定期望产出和非期望产出约束下，考察投入尤其是污染投入的最小可达水平，并用最小投入与实际投入之比衡量投入节约空间。需注意，由于投入方向模型最小化投入而非非期望产出，ϵ_{mb} 不被目标压至 0，故物质平衡约束保留完整形式（含 ϵ_{mb}）。在规模报酬不变下，可通过下式求解：

$$
\begin{aligned}
X_1 = \quad & \min_{\lambda,\epsilon,b_m} \tfrac{1}{K}\textstyle\sum_{k=1}^{K} x_k \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{P} + \epsilon_{kx^P} = x_k^{P}, \quad k = 1, \ldots, K_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_k^{NP}, \quad k = K_1+1, \ldots, K \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{P} - \epsilon_{ly^P} = y_{l0}^{P}, \quad l = 1, \ldots, L_1 \\
& \textstyle\sum_{i=1}^{N} \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_{l0}^{NP}, \quad l = L_1+1, \ldots, L \\
& \textstyle\sum_{i=1}^{N} \lambda_i b_{mi} + \epsilon_{mb} = b_{m0}, \quad m = 1, \ldots M \\
& \textstyle\sum_{k=1}^{K_1} S_{x_{km}} \left(x_{k0}^{P} - x_k^{P}\right) + \sum_{l=1}^{L_1} S_{y_{lm}} \epsilon_{ly^P} = \epsilon_{mb}, \quad m = 1, \ldots, M \\
& \lambda_i \geq 0 \quad i = 1, \ldots, N
\end{aligned}
\tag{9.29}
$$

相应的投入方向效率可按以下方式衡量：

$$
\text{投入方向的技术效率}(TE) = \frac{X_1}{\frac{1}{K}\sum_{k=1}^{K} x_{k0}} \tag{9.30}
$$

在此基础上，也可以逐步放松非污染投入、期望产出和非期望产出约束，构造与 9.4.1 节类似的分解指标。由于本章案例聚焦非期望产出方向，投入方向模型主要用于说明物质平衡框架的可扩展性。

9.4.3 期望产出方向上的技术效率

期望产出方向模型则将目标设定为最大化期望产出。在给定投入和非期望产出约束下，最大可达期望产出与实际期望产出之比反映产出扩张空间。在规模报酬不变下，可通过下式求解：

$$
\begin{aligned}
Y_1 = \quad & \max_{\lambda,\epsilon,b_m} \frac{1}{L}\sum_{l=1}^{L} y_l \\
& \sum_{i=1}^{N} \lambda_i x_{ki}^{P} + \epsilon_{kx^P} = x_{k0}^{P}, \quad k = 1,\dots,K_1 \\
& \sum_{i=1}^{N} \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_{k0}^{NP}, \quad k = K_1+1,\dots,K \\
& \sum_{i=1}^{N} \lambda_i y_{li}^{P} - \epsilon_{ly^P} = y_{l}^{P}, \quad l = 1,\dots,L_1 \\
& \sum_{i=1}^{N} \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_{l}^{NP}, \quad l = L_1+1,\dots,L \\
& \sum_{i=1}^{N} \lambda_i b_{mi} + \epsilon_{mb} = b_{m0}, \quad m = 1,\dots M \\
& \sum_{k=1}^{K_1} S_{x_{km}} \epsilon_{kx^P} + \sum_{l=1}^{L_1} S_{y_{lm}} \left(y_l^P - y_{l0}^P\right) = \epsilon_{mb}, \quad m = 1,\dots,M \\
& \lambda_i \geq 0 \quad i = 1,\dots,N
\end{aligned}
\tag{9.31}
$$

相应的期望产出扩张指标可写为：

$$
\text{期望产出方向的技术效率}(TE) = \frac{Y_1}{\frac{1}{L}\sum_{l=1}^{L} y_{l0}} \tag{9.32}
$$

该指标通常大于或等于1，表示沿期望产出方向的可扩张倍数；若需要构造取值不超过 1 的效率得分，可在具体应用中取其倒数。与投入方向类似，本章案例不使用该扩展指标。

9.4.4 混合方向上的技术效率

除上述三个单一方向外，还可以借鉴 SBM 模型，将投入压缩、期望产出扩张和非期望产出压缩放入同一目标函数，形成混合方向的物质平衡效率指标。本节模型仅作方法展示，本章案例仍采用 9.4.1 节的非期望产出方向分解。例如，可设定：

$$
\min \quad \rho = \frac{\frac{1}{K+M}\left(\sum_{k=1}^{K} \frac{x_k}{x_{k0}} + \sum_{m=1}^{M} \frac{b_m}{b_{m0}}\right)}{\frac{1}{L}\sum_{l=1}^{L} \frac{y_l}{y_{l0}}} \tag{9.33}
$$

约束条件为：

$$
\begin{aligned}
& \sum_{i=1}^{N} \lambda_i x_{ki}^{P} + \epsilon_{kx^P} = x_{k}^{P}, \quad k = 1,\dots,K_1 \\
& \sum_{i=1}^{N} \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_{k}^{NP}, \quad k = K_1+1,\dots,K \\
& \sum_{i=1}^{N} \lambda_i y_{li}^{P} - \epsilon_{ly^P} = y_{l}^{P}, \quad l = 1,\dots,L_1 \\
& \sum_{i=1}^{N} \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_{l}^{NP}, \quad l = L_1+1,\dots,L \\
& \sum_{i=1}^{N} \lambda_i b_{mi} + \epsilon_{mb} = b_{m}, \quad m = 1,\dots M \\
& \sum_{k=1}^{K_1} S_{x_{km}} \left(x_{k0}^P - x_k^P\right) + \sum_{l=1}^{L_1} S_{y_{lm}} \left(y_l^P - y_{l0}^P\right) = (b_{m0} - b_m), \quad m = 1,\dots,M \\
& \lambda_i \geq 0 \quad i = 1,\dots,N
\end{aligned}
\tag{9.34}
$$

物质平衡模型的有效性依赖于 S_x 和 S_y 的设定（关于其作为测算假设而非外生常数的性质，参见 9.3 节）。在实际研究中，这些系数往往由观测数据或工程排放系数推得，例如 Wang 等（2018）以火电行业 CO_2 排放量与燃料消耗量之比构造投入端排放系数。

上述基础模型假定不存在单独的末端减排活动。下一节放宽这一假定，介绍含末端减排活动的扩展模型。

9.5 物质平衡条件的发展：含末端减排活动的扩展

本节介绍物质平衡模型的一种扩展，用于刻画专门的末端减排活动。该扩展模型目前尚未在 deabook 的 DEAmb 模块中实现，9.6 节和 9.7 节的代码与案例对应的是不含减排活动 a 的基础模型；本节内容可作为进一步研究的理论参考。

在上述基础模型中，投入端进入生产过程的污染物质最终在期望产出和非期望产出之间分配。Hampf 和Rødseth（2015）进一步指出，现实生产中还可能存在专门的减排或处理活动：对某些DMU 而言，使用额外资源处理排放，可能比完全依靠低污染投入替代高污染投入的成本更低。例如，电厂可以安装脱硫、除尘或碳捕集设备，养殖场也可以通过粪便收集和外运利用减少本场剩余污染物。考虑这类减排活动时，物质平衡条件可扩展为：

$$S_x x - S_y y = b + a \tag{9.35}$$

其中，a 表示通过治理、回收或外运等活动减少的非期望产出向量；若不存在减排活动，则 $a = 0$；若存在减排活动，则 $a > 0$。

相应地，生产技术需要同时包括投入、期望产出、减排量和非期望产出。仍将投入划分为污染投入 x^P 与非污染投入 x^{NP}，将期望产出划分为污染产出 y^P 与非污染产出 y^{NP}。进一步假定前 M_1 个非期望产出是可控的，可通过治理活动形成减排量 a_m，$m = 1, \ldots, M_1$。此时技术集可写为：

$$T = \{(x, y, a, b) : x \text{ 能生产 } (y, a, b)\} \tag{9.36}$$

规模报酬不变下，物质平衡模型（MB）的相应非参数估计为：

$$\begin{aligned} \hat{T}^{MB}_{CRS} = \{ \quad & (x, y, a, b) : \\ & x = X\lambda + \epsilon_x, \\ & y = Y\lambda - \epsilon_y, \\ & b = B\lambda + \epsilon_b, \\ & a = A\lambda - \epsilon_a, \\ & S_x\epsilon_x + S_y\epsilon_y = \epsilon_b - \epsilon_a, \\ & \epsilon_x, \epsilon_y, \epsilon_b, \epsilon_a, \lambda \geqq \mathbf{0}\} \end{aligned} \tag{9.37}$$

规模报酬可变下，相应非参数技术为：

$$\begin{aligned} \hat{T}^{MB}_{VRS} = \{ \quad & (x, y, a, b) : \\ & x = X\lambda + \epsilon_x, \\ & y = Y\lambda - \epsilon_y, \\ & b = B\lambda + \epsilon_b, \\ & a = A\lambda - \epsilon_a, \\ & S_x\epsilon_x + S_y\epsilon_y = \epsilon_b - \epsilon_a, \\ & \mathbf{1}^T\lambda = 1, \\ & \epsilon_x, \epsilon_y, \epsilon_b, \epsilon_a, \lambda \geqq \mathbf{0}\} \end{aligned} \tag{9.38}$$

与无减排活动的模型相比，扩展模型的关键变化在于物质平衡约束变为 $S_x\epsilon_x + S_y\epsilon_y = \epsilon_b - \epsilon_a$。减排活动 ϵ_a 会吸收一部分原本进入非期望产出的物质流，全部污染物质变化不能再仅归因于投入和产出结构调整。当 $\epsilon_a = 0$ 时，该模型退化为前述无减排活动情形。

在规模报酬不变下，包含减排活动的非期望产出方向技术效率可通过下式求解：

$$
\begin{aligned}
B_1 = \quad & \min_{\lambda,\epsilon,b_m,a_m} \frac{1}{M}\sum_{m=1}^{M} b_m \\
& \sum_{i=1}^{N} \lambda_i x_{ki}^{P} + \epsilon_{kx^P} = x_{k0}^{P}, \quad k = 1,\dots,K_1 \\
& \sum_{i=1}^{N} \lambda_i x_{ki}^{NP} + \epsilon_{kx^{NP}} = x_{k0}^{NP}, \quad k = K_1+1,\dots,K \\
& \sum_{i=1}^{N} \lambda_i y_{li}^{P} - \epsilon_{ly^P} = y_{l0}^{P}, \quad l = 1,\dots,L_1 \\
& \sum_{i=1}^{N} \lambda_i y_{li}^{NP} - \epsilon_{ly^{NP}} = y_{l0}^{NP}, \quad l = L_1+1,\dots,L \\
& \sum_{i=1}^{N} \lambda_i b_{mi} + \epsilon_{mb} = b_m, \quad m = 1,\dots M \\
& \sum_{i=1}^{N} \lambda_i a_{mi} - \epsilon_{ma} = a_{m0}, \quad m = 1,\dots,M_1 \\
& \sum_{k=1}^{K_1} S_{x_{km}} \epsilon_{kx^P} + \sum_{l=1}^{L_1} S_{y_{lm}} \epsilon_{ly^P} = (b_{m0} - b_m) - \epsilon_{ma}, \quad m = 1,\dots,M_1 \\
& \sum_{k=1}^{K_1} S_{x_{km}} \epsilon_{kx^P} + \sum_{l=1}^{L_1} S_{y_{lm}} \epsilon_{ly^P} = (b_{m0} - b_m), \quad m = M_1+1,\dots,M \\
& \lambda_i \geq 0 \quad i = 1,\dots,N
\end{aligned}
\tag{9.39}
$$

式(9.39)仅展示第一层级作为示例；含减排活动的第二至五层级分解可类比 9.4.1 节构造。与 9.4.1 节同理，由于 ϵ_{mb} 在非期望产出约束与物质平衡约束相加后消去，且仅以 $+\epsilon_{mb}$ 进入目标，极小化目标将其压至0，简约形式与完整形式等价。本节模型为理论参考，下一节回到不含减排活动的基础模型，介绍其在 deabook 中的实现。

9.6 物质平衡模型的Python实现

前文已将物质平衡下的环境绩效测度严格写成一组逐层放松约束的线性规划（式 9.12）—(9.18)，并论证了在非径向简约形式下各层级 B 值如何经由比例指标分解为 OE、TE、PIAE、NPIAE、POAE 与 NPOAE。这些模型的经济含义能否在程序中正确落地，取决于一个前置的识别问题：必须把公理体系中的污染投入、污染产出与非期望产出明确地映射到求解接口，并保证五个层级在同一规模报酬假设下依次求解，方能维持 $B_1 \geq B_2 \geq B_3 \geq B_4 \geq B_5$ 的单调性与各配置效率的纯约束解释。

deabook 通过 DEAmb 模块实现本章 9.4.1 节所对应的基础物质平衡分解模型，即不显式引入减排活动 a 的情形（含减排活动的扩展见 9.5 节，目前尚未纳入该模块）。`DEAmb.MB.optimize()` 返回 `(result, info)`：其中 `result` 保存每个 DMU 的求解状态、目标函数值和当前层级的最优非期望产出；在当前基础 DEAmb 入口中，`info` 主要作为兼容性返回项，通常为 `None`，实际分析以 `result` 为准。下文先说明物质流声明与模型接口的经济含义，再依次说明分解层级、返回结果的整理与后处理指标的构造。

9.6.1 物质流声明与模型接口

模型接口的第一步是导入 DEAmb 模块及规模报酬常量。物质平衡模型区别于一般非期望产出 DEA 的关键，在于它要求调用者显式声明污染物质在投入端、产出端与排放端之间的守恒结构，而非仅提供一组变量列表。

```
from deabook import data_path, PANEL
from deabook.constant import GLOBAL
from deabook import DEAmb, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook.constant import RTS_CRS, RTS_VRS1
```

```
from pathlib import Path
import pandas as pd
```

sent 参数承担的正是这一物质流结构声明功能，而非普通变量清单。以 sent="K L+F=E:CO2" 为例，等号左侧加号前的 K 和 L 是非污染投入，加号后的 F 是污染投入；等号右侧冒号前的 E 是期望产出，冒号后的 CO2 是非期望产出。**再次提醒（见本章导语）：本章 E 表示电力（期望产出），F 表示燃料（污染投入），与本书其他章节将 E 用作能源投入的记法不同。**若期望产出亦需区分是否携带污染物质，则等号右侧加号前为非污染产出、加号后为污染产出。sx 与 sy 分别给出投入端与产出端的污染物质含量系数，二者共同决定投入、期望产出与非期望产出如何经由物质平衡约束相连。在式（9.1）的记号下，sx 对应 S_x、sy 对应 S_y，因此它们是连接前述公理体系与数值求解的接口参数。各参数的作用与经济含义汇总于表 9.1。

表 9.1: 物质平衡模型的主要参数

参数	作用	本章常用设置	经济含义与解释重点
data	待评价样本	DataFrame	包含投入、期望产出、非期望产出以及年份、省份等 DMU 标识变量；样本范围即全样本（全局）参考前沿
sent	物质流声明	"K L+F=E:CO2"	区分非污染投入、污染投入、期望产出和非期望产出
sx	投入端污染系数	[[0,0,sx]]	将燃料投入与 CO_2 排放连接起来；K、L 不携带碳元素，系数为 0
sy	产出端污染系数	[[0]]	本案例中电力 E 不携带碳元素，因此系数为 0；若期望产出保留污染物质，则需相应设定
rts	规模报酬假设	RTS_CRS、RTS_VRS1	决定生产单元在共同规模前沿还是局部规模前沿上比较
level	分解层级	1—5	决定依次放松哪些调整约束，并形成 B_1—B_5 的分解链条
evalindex	被评价样本	None 或条件表达式	可限定评价年份或 DMU，例如 "year=[2017]"
refindex	参考技术集	None 或条件表达式	可限定参考集年份或DMU；默认用全部样本（即全局）构造参考前沿，并非跨群组” 共同前沿”
solver	求解器	"mosek"、"glpk"	数值求解选项，不改变模型的经济含义

9.6.2 分解层级及其效率含义

level 是物质平衡模型中最关键的接口参数，它对应 9.4.1 节中逐层放松约束的五个层级，而非简单的模型开关。每一层级在上一个层级可行域的基础上进一步放开某一类调整约束，从而生成从 B_1 到 B_5 的分解链条；后文所有效率指标均由这些 B 值之间的比例给出。某一环节的比例明显低于1，即意味着整体效率损失主要源于该环节所对应的可调整来源——且调整成本通常随层级后移而递增。各层级、level 取值

与 B 值的对应关系见 表 9.2。

表 9.2: 物质平衡分解层级与 B 值含义

level	优化变量	返回的 B 值	对应效率含义	本章案例用途
1	b	B_1	技术效率TE，表示给定投入和产出约束下非期望产出的最小可达水平	计算 TE
2	b, x^P	B_2	在技术效率基础上允许污染投入调整，名义上刻画污染投入配置效率PIAE；但在简约形式下 $B_2 = B_1$ 恒成立（见 9.4.1 节式 9.26 后的说明），故同一 RTS 下 $PIAE \equiv 1$	计算 PIAE（本案例恒为 1）
3	b, x^P, x^{NP}	B_3	进一步允许非污染投入调整，刻画非污染投入配置效率 NPIAE	计算 NPIAE
4	b, x^P, x^{NP}, y^P	B_4	允许污染产出调整，刻画污染产出配置效率 POAE（B_4/B_3）；当 $L_1 = 0$（无污染产出，本案例）时第四层级为空操作、$B_4' = B_3$，POAE 退化为 1，level=4 直接计算第五层级（输出记为 B_4）	一般情形计算 POAE；$L_1 = 0$ 案例中兼作 OE、NPOAE 计算
5	全部变量	B_5	进一步允许非污染产出调整，刻画非污染产出配置效率 NPOAE（B_5/B_4）	计算 OE、NPOAE（$L_1 = 0$ 时改用 B_4/B_3）

本章案例中，数据不存在污染产出（$L_1 = 0$，电力 E 归入非污染产出；此即 POAE 退化的起作用条件）且 $S_y = 0$（sy=0，本案例附加特征），第四层级为空操作，POAE 退化为 1。deabook 的 level=4 在此情形下直接计算第五层级（非污染产出可调），其输出记为 B_4 仅为简便。因此程序运行到 level=4，将 $B_4/CO2$ 记为整体效率 OE，并将 B_4/B_3 记为非污染产出配置效率 NPOAE（对应完整模型的 B_5/B_4）。若研究对象

同时存在污染产出和非污染产出（$L_1 > 0$），则应继续运行 `level=5`，并按完整模型使用 B_5 计算 `OE`、用 B_4/B_3 计算 `POAE`、用 B_5/B_4 计算 `NPOAE`。

9.6.3 返回结果与后处理指标

`DEAmb.MB.optimize()` 在不同 `level` 下返回的结果列名保持一致，差异只体现在 `best of Undesirable` 所代表的 B 值层级上。因此，计算分解指标时需要分别运行各个层级，并将 `best of Undesirable` 按层级重命名为 `B1`、`B2`、`B3`、`B4` 或 `B5`；`obj` 为数值目标函数值，主要用于核对求解的一致性，经济解释应建立在各层级 B 值构造的比例指标之上。返回结果的主要字段见 表 9.3。

表 9.3: `DEAmb.MB.optimize()` 的返回结果

返回对象或列	类型	含义	使用方式
`result`	`DataFrame`	每个 DMU 的求解结果	与原始年份、省份和 CO_2 数据合并后计算分解指标
`optimization_status`	float	求解状态码	成功时为 `1.0`；整理结果前应检查是否存在非 1 的状态值
`obj`	float	目标函数值	对应当前层级的最小化目标，主要用于数值核验
`best of Undesirable`	float	当前 `level` 下的最优非期望产出值	按层级重命名为 `B1`—`B5` 后再计算效率比例
`info`	`None` 或模型信息	兼容性返回项	当前基础 MB 模型通常返回 `None`，实际分析主要使用 `result`

```
model_l1 = DEAmb.MB(
    data, sent="K L+F=E:CO2", sx=[[0, 0, sx]], sy=[[sy]],
    rts=RTS_CRS, evalindex=None, refindex=None, level=1,
)
result_l1, info_l1 = model_l1.optimize("mosek", "1")
result_l1[["optimization_status", "obj", "best of Undesirable"]].head()
```

将各层级的 B 值合并后，可以计算整体效率和分解效率。物质平衡环境绩效分解指标见表 9.4。本案例的电力产出不携带污染物质，因此将 `level=4` 的 B_4 作为最后一层可计算结果；若数据中同时存在污染产出和非污染产出，则应继续运行 `level=5` 并使用 B_5 完成完整分解。

表 9.4: 物质平衡环境绩效分解指标

指标	本案例公式	完整模型公式	含义与解释重点
OE	B4 / CO2	B5 / CO2	整体环境效率；越接近 1 表示整体越接近物质平衡前沿
TE	B1 / CO2	B1 / CO2	非期望产出方向的技术效率，反映给定投入产出约束下的排放压缩空间
PIAE	B2 / B1	B2 / B1	污染投入配置效率；在简约形式下 $B_2 = B_1$，故同一 RTS 下恒为 1，不提供额外信息（见 9.4.1 节说明与 9.7 节案例）
NPIAE	B3 / B2	B3 / B2	非污染投入配置效率，反映资本、劳动等非污染投入调整空间
POAE	不适用（$L_1 = 0$ 时 POAE 退化）	B4 / B3	污染产出配置效率
NPOAE	B4 / B3	B5 / B4	非污染产出配置效率；低值表示产出配置环节在乘法分解中距 1 最远，可视为整体效率偏低的主要贡献项（详见 9.7 节案例讨论）

注：本案例不存在污染产出（$L_1 = 0$，POAE 退化的起作用条件）且 $S_y = 0$（本案例附加特征），第四层级为空操作，POAE 退化为 1，故不单独计算；`level=4` 实算第五层级，输出记为 B_4 仅为简便，OE 与 NPOAE 直接用 B_4 计算。完整模型见式（9.20）—（9.26）。$B_1 \geq B_2 \geq B_3 \geq B_4$ 单调成立（要求各层级使用同一规模报酬假设；本章案例统一取CRS），各效率指标均落在 $[0, 1]$，故无需取绝对值；实际计算中若求解器返回负零（如 -0.000000）等数值噪声，可对 B 值取绝对值以消除显示问题。

```
MB_results["OE"] = MB_results["B4"] / MB_results["CO2"]
MB_results["TE"] = MB_results["B1"] / MB_results["CO2"]
MB_results["PIAE"] = MB_results["B2"] / MB_results["B1"]
MB_results["NPIAE"] = MB_results["B3"] / MB_results["B2"]
MB_results["NPOAE"] = MB_results["B4"] / MB_results["B3"]
```

9.6.4 多层级求解的依次调用

由于各层级须在同一规模报酬假设下分别求解（本章统一取 `RTS_CRS`，否则层间比值会混入规模报酬变化），实际计算时必须依次调用 `level=1` 至 `level=4`，再将各层级结果合并后方可构造分解指标。以基础模型在 CRS 下的四个层级为例，调用方式如下：

```
# Level 1
model = DEAmb.MB(data, sent="K L+F=E:CO2", sx=[[0,0,sx]], sy=[[sy]],
              rts=RTS_CRS, level=1)
res, info = model.optimize("mosek", "1")

# Level 2
model = DEAmb.MB(data, sent="K L+F=E:CO2", sx=[[0,0,sx]], sy=[[sy]],
              rts=RTS_CRS, level=2)
res, info = model.optimize("mosek", "1")

# Level 3
model = DEAmb.MB(data, sent="K L+F=E:CO2", sx=[[0,0,sx]], sy=[[sy]],
              rts=RTS_CRS, level=3)
res, info = model.optimize("mosek", "1")

# Level 4
model = DEAmb.MB(data, sent="K L+F=E:CO2", sx=[[0,0,sx]], sy=[[sy]],
              rts=RTS_CRS, level=4)
res, info = model.optimize("mosek", "1")
```

9.7 案例应用：中国火电行业的物质平衡分解

使用2015—2017年30个省级地区火电行业的90个观测。资本K为装机容量，L为相关行业就业人数，污染投入F为标准煤口径燃料，Y为发电量，CO2为依据燃料与排放因子计算的排放。三年观测共同构造CRS前沿。燃料系数取全样本CO2总量与F总量之比，电力不携带碳，产出物质系数为零。该平均系数是本例假设，不是逐省实测碳含量。

```
from deabook.example.book_cases.run import FUN
FUN[9]()  # 重新计算本章，输出完整CSV、摘要表与PNG/PDF图
```

表 9.5: 中国火电行业的物质平衡分解：结果摘要

统计量	TE	PIAE	NPIAE	NPOAE	保产出效率
有效样本数	90	90	90	90	90
均值	0.793	1.000	0.861	0.023	0.671
标准差	0.149	0.000	0.102	0.053	0.096
最小值	0.469	1.000	0.714	0.000	0.467
P10	0.591	1.000	0.753	0.000	0.541
中位数	0.810	1.000	0.840	0.000	0.684
P90	1.000	1.000	1.000	0.080	0.744

统计量	TE	PIAE	NPIAE	NPOAE	保产出效率
最大值	1.000	1.000	1.000	0.229	1.000

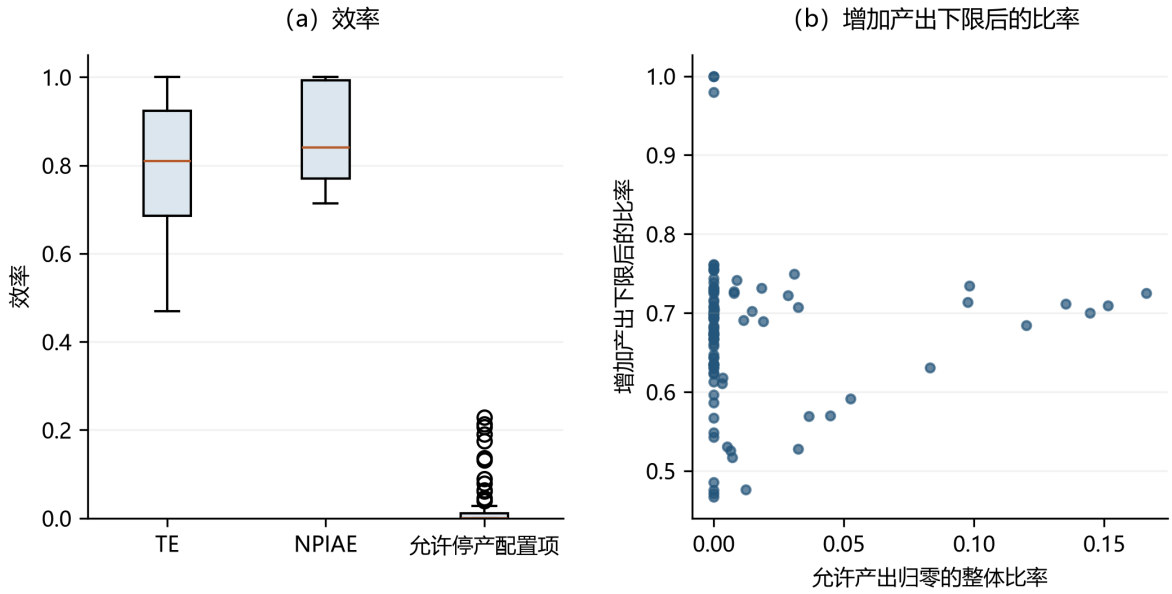


图 9.1: 物质平衡分解与产出下限敏感性（2015—2017年）

表列出各项效率分解与保产出约束下比率的分布，便于识别恒定项及对产出下限敏感的结果。图右对照允许产出归零与加入产出下限两种设定；不能把允许停产时的数学最优值直接理解为可实施的减排方案。

TE均值为0.793，PIAE恒为1，NPIAE均值为0.861。允许电力产出自由下降时，NPOAE中位数为0，不能把这种停产退化写成真实减排潜力。作为约束敏感性比较，最后一层增加Y*≥Y，重新求得的整体排放比率均值为0.671。图中分别报告原目标与保产出结果，后者属于增加约束后的重估，不与原分解指标混用。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch09_full.csv`。表中P10和P90为第10和第90百分位数（如适用），展示分布而不占用逐个决策单元的行数。

9.8 本章小结

TE均值为0.793，PIAE恒为1，NPIAE均值为0.861。允许电力产出自由下降时，NPOAE中位数为0，不能把这种停产退化写成真实减排潜力。作为约束敏感性比较，最后一层增加Y*≥Y，重新求得的整体排放比率均值为0.671。图中分别报告原目标与保产出结果，后者属于增加约束后的重估，不与原分解指标混用。

参考文献

AYRES R U, KNEESE A V. Production, consumption, and externalities [J]. American Economic Review, 1969, 59(3): 282-297.

CHUNG Y H, FÄRE R, GROSSKOPF S. Productivity and undesirable outputs: a directional distance function approach [J]. Journal of Environmental Management, 1997, 51(3): 229-240. DOI: 10.1006/jema.1997.0146.

COELLI T, LAUWERS L, VAN HUYLENBROECK G. Environmental efficiency measurement and the materials balance condition [J]. Journal of Productivity Analysis, 2007, 28(1-2): 3-12. DOI: 10.1007/s11123-007-0061-4.

DAKPO K H, JEANNEAUX P, LATRUFFE L. Modelling pollution-generating technologies in performance benchmarking: recent developments, limits and future prospects in the nonparametric framework [J]. European Journal of Operational Research, 2016, 250(2): 347-359. DOI: 10.1016/j.ejor.2015.09.031.

FÄRE R, GROSSKOPF S, LOVELL C A K, PASURKA C Jr. Multilateral productivity comparisons when some outputs are undesirable: a nonparametric approach [J]. The Review of Economics and Statistics, 1989, 71(1): 90-98. DOI: 10.2307/1928055.

FÄRE R, GROSSKOPF S, TYTECA D. An activity analysis model of the environmental performance of firms: application to fossil-fuel-fired electric utilities [J]. Ecological Economics, 1996, 18(2): 161-175. DOI: 10.1016/0921-8009(96)00019-5.

FÄRE R, GROSSKOPF S. Modeling undesirable factors in efficiency evaluation: comment [J]. European Journal of Operational Research, 2004, 157(1): 242-245. DOI: 10.1016/S0377-2217(03)00205-4.

FØRSUND F R. Good modelling of bad outputs: pollution and multiple-output production [J]. International Review of Environmental and Resource Economics, 2009, 3(1): 1-38. DOI: 10.1561/101.00000019.

HAILU A, VEEMAN T S. Non-parametric productivity analysis with undesirable outputs: an application to the Canadian pulp and paper industry [J]. American Journal of Agricultural Economics, 2001, 83(3): 605-616. DOI: 10.1111/0002-9092.00181.

HAMPF B, RØDSETH K L. Carbon dioxide emission standards for U.S. power plants: an efficiency analysis perspective [J]. Energy Economics, 2015, 50: 140-153. DOI: 10.1016/j.eneco.2015.04.001.

MURTY S, RUSSELL R R, LEVKOFF S B. On modeling pollution-generating technologies [J]. Journal of Environmental Economics and Management, 2012, 64(1): 117-135. DOI: 10.1016/j.jeem.2012.01.001.

REINHARD S, LOVELL C A K, THIJSSEN G J. Environmental efficiency with multiple environmentally detrimental variables; estimated with SFA and DEA [J]. European Journal of Operational Research, 2000, 121(2): 287-303. DOI: 10.1016/S0377-2217(99)00115-2.

RØDSETH K L. Environmental efficiency measurement and the materials balance condition reconsidered [J]. European Journal of Operational Research, 2016, 250(1): 342-346. DOI: 10.1016/j.ejor.2015.10.061.

RØDSETH K L. Axioms of a polluting technology: a materials balance approach [J]. Environmental and Resource Economics, 2017, 67(1): 1-22. DOI: 10.1007/s10640-015-9974-1.

SUEYOSHI T, GOTO M. Data envelopment analysis for environmental assessment: comparison between public and private ownership in petroleum industry [J]. European Journal of Operational Research, 2012, 216(3): 668-678. DOI: 10.1016/j.ejor.2011.07.046.

WANG K, WEI Y-M, HUANG Z. Environmental efficiency and abatement efficiency measurements of China's thermal power industry: a data envelopment analysis based materials balance approach [J]. European Journal of Operational Research, 2018, 269(1): 35-50. DOI: 10.1016/j.ejor.2017.04.053.

WELCH E, BARNUM D. Joint environmental and cost efficiency analysis of electricity generation [J]. Ecological Economics, 2009, 68(8-9): 2336-2343. DOI: 10.1016/j.ecolecon.2009.02.011.

第 10 章

随机非参数数据包络分析

在效率分析领域，尽管尚未形成统一的理论框架，数据包络分析（DEA）和随机前沿分析（SFA）已发展为两种具有互补特征的主流方法，二者在处理效率评估问题时呈现出不同的技术路径，方法的选择通常取决于研究数据的特征与具体问题的需求。

数据包络分析（DEA, Farrell 1957; Charnes et al. 1978）是一种基于公理、采用数学规划的效率分析方法。与计量经济学中基于回归的工具相比，DEA的主要优势在于对前沿的非参数化处理：它建立在生产理论的公理之上，主要包括自由可处置性（单调性）与凸性，而不假定边界或无效率分布的任何特定函数形式。规模报酬不变（CRS）并非必有公理，实践中可在不变、可变（VRS）等设定之间选择。这种直接的、以数据为导向的方法也有助于将效率分析的结果传达给决策者。然而，DEA的主要缺点在于它把所有偏离前沿的情况都归结为无效率，这往往是一个非常强的假设。

SFA（Aigner, Lovell and Schmidt, 1977）的主要优势在于其对偏离前沿部分的概率建模：这些偏离被分解为一个非负的无效率项和一个随机扰动项 v_i，后者用以吸收企业及经营环境的不可观测异质性、测量误差、设定误差等噪声来源。与DEA相比，SFA采用参数化的回归技术，需要事先对前沿的函数形式和无效率分布进行规范。由于经济理论很少证明一个特定的函数形式是合理的，实践中常使用灵活的函数形式，如translog（超对数函数）。然而，灵活的函数形式往往违反生产理论的公理，而施加这些公理则会降低灵活性。

鉴于两类方法论之间的界限与区别，Kuosmanen（2008）提出的凸非参数最小二乘法（Convex Nonparametric Least Squares, CNLS）开创了新的研究路径。CNLS通过二次规划估计满足单调性与凹性约束的非参数生产前沿（对应生产可能性集的凸性），误差项为均值为零的噪声 v_i，从而在不预设函数形式的前提下继承了DEA的公理化建模。Kuosmanen and Johnson（2010）进一步表明，对残差施加符号约束后，CNLS与DEA等价（corrected convex nonparametric least squares, C²NLS）。在此基础上，Kuosmanen and Kortelainen（2012）将偏离前沿的部分分解为非负无效率 u_i 与噪声 v_i，形成随机非参数数据包络（Stochastic Nonparametric Envelopment of Data, StoNED）。该框架把生产技术的非参数估计与复合误差的统计分解结合起来，为效率分析提供了更为严谨的计量经济学基础。

10.1 单产出模型设定

本节首先考察多投入、单产出的情形，并与SFA模型进行比较，以便理解二者的差异。多投入、多产出以及多投入、多产出且有非期望产出的情形均为单产出模型的扩展，将在本节之后讨论。

假设生产技术由前沿生产函数 $f(x)$ 表示，其中 x 是一个m维的投入向量。前沿 $f(x)$ 表示用投入 x 可以生产的最大产出，因此，函数 $f(x)$ 表征了生产可能性集的边界。假设函数 $f(x)$ 属于连续的、单调递增的、全局凹的函数类，且可以是不可微的（将此类记为 F_2），这等价于生产可能性集满足自由可处置性和凸性的经典DEA假设。与SFA不同的是，这里没有假定 $f(x)$ 的具体函数形式。

由于无效率和噪声的影响，决策单元（decision making unit, DMU）$_i$ 的观测产出 y_i 可能与 $f(x_i)$ 不同。遵循SFA文献，引入一个综合误差项 $\varepsilon_i = v_i - u_i$，它由非效率项 u_i 和随机噪声项 v_i 组成，形式为

$$y_i = f(x_i) + \varepsilon_i = f(x_i) + v_i - u_i, \quad i = 1, \ldots, n \tag{10.1}$$

式中，变量 u_i 和 v_i 是随机变量，假定二者相互独立，且都与投入 x 无关。另外，假定 $E(u_i) = \mu > 0$，$Var(u_i) = \sigma_u^2 < \infty$，且 $E(v_i) = 0$，$Var(v_i) = \sigma_v^2 < \infty$。假设 σ_u^2 和 σ_v^2 在不同DMU间是恒定的，这被称为同方差性；具有异方差性的非效率和噪声模型需要对 u_i 和 v_i 的分布进一步施加假设，这里不做讨论，可见Kuosmanen(2008) 的进一步讨论。在这个模型中，没有对 $f(x_i)$ 的具体函数形式施加限制，这与DEA文献的方法相同；同时，假设综合误差项 ε_i 由非效率项 u_i 和随机噪声项 v_i 组成，这与SFA文献的方法相同。因此，这个模型结合了DEA和SFA方法的优点。然而，与DEA模型相比，这个模型对非效率项 u_i 的分布施加了假设；与SFA模型相比，对 u_i 和 v_i 的分布的假设可能无法进一步精细化。

除了假设综合误差项 $\varepsilon_i = v_i - u_i$ 外，可以施加一个替代性的假设：$\varepsilon_i = -u_i$，从而与DEA模型的假设保持相似。需要注意的是，这里的非效率项 u_i 不是最终需要的非效率，非效率的计算方法为 u_i 除以前沿（$y_i + u_i$）。此外，还可以施加另一种替代性假设：$\varepsilon_i = v_i$，从而与凸非参数最小二乘法的假设相同。下面逐一介绍这些模型。

10.1.1 凸非参数最小二乘法

当把模型（10.1）中综合误差项 $\varepsilon_i = v_i - u_i$ 的假设替换为综合误差项 $\varepsilon_i = v_i$ 时，便可得到凸非参数最小二乘的模型，即：

$$y_i = f(x_i) + \varepsilon_i = f(x_i) + v_i, \quad i = 1, \ldots, n$$

其中，变量 v_i 是随机变量，假定 v_i 与投入 x 无关，$E(v_i) = 0$，$Var(v_i) = \sigma_v^2 < \infty$，且 σ_v^2 在不同DMU间是恒定的。此时，$E(y_i|X) = f(x_i) + E(v_i|X) = f(x_i) + 0$。在满足这种假设的条件下，凸非参数最小二乘法估计量是以下广义回归问题的解：

$$\min_{f \in F_2} \sum_{i=1}^{n} \left(y_i - f\left(\mathbf{x}_i\right)\right)^2, \tag{10.2}$$

其中，F_2 代表满足单调性与凹性的函数类。模型（10.2）是一个关于函数的优化问题，可以等效地写成一个有限维的凹二次优化问题，其中变量是函数值和每个点 $x_1, \ldots, x_n$ 的子梯度(Boyd and Vandenberghe, 2004; Kuosmanen, 2008)。作为约束条件的一部分，需施加凹性条件；也就是说，凹函数的图像位于其任一切平面之下。更确切地说，给定任何一点 x_i，f 在 x_i 处的值小于或等于在任何一点处估计的切平面在 x_i 上的取值（在 x_i 自身的切平面处二者相等）。显然，任何凹函数在每一点都有一个非空的子梯度，因而这种切平面的存在是有保证的。对于这个问题，在有限样本下，对所有 $n(n-1)$ 对观测点 $x_i, x_j\,(1 \le i, j \le n, i \ne j)$

施加凹性约束条件：$f(x_j) \leq f(x_i) + \nabla f(x_i)^\top (x_j - x_i)$，即可保证函数 f 在整个定义域上的全局凹性。这是因为凹函数在任意两点间的线性插值均不高于其实际值，而样本外点的性质可通过插值/外推自动满足（无需显式约束）。由此形成带有变量 $\alpha, \beta, \varepsilon$ 的二次优化问题如下：

$$
\begin{aligned}
&\min_{\alpha,\beta,\varepsilon} \sum_{i=1}^{n} \left(\varepsilon_i^{CNLS}\right)^2 \\
&\text{s.t.} \\
&y_i = \alpha_i + \mathbf{x}_i'\beta_i + \varepsilon_i^{CNLS} \quad \forall i = 1, \ldots, n \\
&\alpha_i + \mathbf{x}_i'\beta_i \leq \alpha_h + \mathbf{x}_i'\beta_h \quad \forall h, i = 1, \ldots, n \\
&\beta_i \geq \mathbf{0} \quad \forall i = 1, \ldots, n
\end{aligned}
\tag{10.3}
$$

其中，变量 $\alpha_i + \mathbf{x}_i'\beta_i$ 代表 $f(x_i)$ 的切平面，β_i 属于凹函数 f 在每个 x_i 处的超梯度，也就是边际产量。这个问题的解 $\hat{\alpha} + \mathbf{x}'\hat{\beta}$ 被称为凸非参数最小二乘法估计量(CNLS)。注意，通过设置 $\alpha_i = \alpha, \beta_i = \beta \forall i$ 和 $f(x_i) = \alpha + \mathbf{x}_i'\beta$ 可恢复普通最小二乘线性回归问题。也就是说，CNLS估计量估计了n条不同的回归线，其截距和斜率系数因DMU而异，而不是像OLS那样只拟合一条回归线。模型的第一个约束施加了产出观测值与超平面之间的关系；第二个约束通过应用Afriat 不等式系统（Afriat, 1972）施加凹性约束；第三个约束施加单调性约束。

可以证明，在满足上述假设时，$\widehat{f}^{CNLS}(x_i)$ 是 $f(x_i)$ 的一致估计。特别是，$E(v_i) = 0$ 的假设对一致性至关重要，当违反这个假设时，CNLS估计量将不是一致的估计。需注意，带Afriat 不等式的凹非参数最小二乘是**受约束**的估计量：凹性和单调性约束在有限样本中一般会使拟合值产生偏差，因此CNLS 只有一致性，没有 OLS 那样的有限样本无偏性。

另外，$\widehat{f}^{CNLS}(x_i)$ 在样本内观察值 (x_i, y_i) 上是 $f(x_i)$ 的唯一估计。虽然CNLS估计量对于样本内观察值是唯一的，但没有唯一的方法将其外推到样本外观察值。CNLS外推的不唯一性导致样本外预测结果波动（类似过拟合中高方差模型对噪声敏感），因模型对样本内结构过度适配而缺乏稳健性。为了使CNLS估计量适用于样本外观察值，Bertsimas and Mundru(2021)建议使用正则化，即加入 β_i 的二范数，得到的需要求解的问题如下（其中 $\rho > 0$ 为正则化参数）：

$$
\begin{aligned}
&\min_{\alpha,\beta,\varepsilon} \sum_{i=1}^{n} \left(\varepsilon_i^{CNLS}\right)^2 + \rho \sum_{i=1}^{n} \left\|\beta_i\right\|^2 \\
&\text{s.t.} \\
&y_i = \alpha_i + \mathbf{x}_i'\beta_i + \varepsilon_i^{CNLS} \quad \forall i = 1, \ldots, n \\
&\alpha_i + \mathbf{x}_i'\beta_i \leq \alpha_h + \mathbf{x}_i'\beta_h \quad \forall h, i = 1, \ldots, n \\
&\beta_i \geq \mathbf{0} \quad \forall i = 1, \ldots, n
\end{aligned}
\tag{10.4}
$$

另一方面，Kuosmanen and Kortelainen(2012)呼吁使用最小外推原则，并建议使用下限值：

$$
\widehat{f}_{min}^{CNLS}(x) = \min_{\alpha,\beta} \{\alpha + \beta'\mathbf{x} \mid \alpha + \beta'\mathbf{x_i} \geq \widehat{f}^{CNLS}(\mathbf{x_i}), \forall i = 1, ..., n\} \tag{10.5}
$$

10.1.2 确定性前沿

在这一节中，考虑模型（10.1）的另一种特殊情况，即综合误差项 $\varepsilon_i = v_i - u_i$ 的假设替换为综合误差项 $\varepsilon_i = -u_i$ 时，便可得到确定性模型。确定性模型假设投入产出数据无测量误差（即 $v_i = 0$），唯一的随机性在于观测样本是从生产可能性集中抽样得到的。因此，DEA 的 bootstrap 置信区间不能直接与其他真正具有随机性的模型的置信区间（如SFA 置信区间）进行比较。如果确定性模型的无噪声假设 $(v_i = 0)$ 不成立，DEA 的统计推断基础将不再成立；即便是面向小样本修正的 bootstrap 方法，也无法解决噪声带来的问题。

在单一产出的情况下，第 o 个DMU的生产函数 f_i 的规模报酬可变（VRS）DEA估计值可以被表述为：

$$\max_{\theta_o,\lambda}\left\{\theta_o \mid \begin{array}{ll} \sum_{i=1}^{n}\lambda_i y_i \geq \theta_o y_o & \\ \sum_{i=1}^{n}\lambda_i x_{im} \leq x_{om} & \forall m=1,\ldots,M \\ \sum_{i=1}^{n}\lambda_i = 1 & \\ \lambda_i \geq 0 & \forall i=1,\ldots,n \end{array}\right\} \tag{10.6}$$

乘法模式可以转换为加法模式：

$$\max_{\phi_o,\lambda,\mu}\left\{\phi_o \mid \begin{array}{ll} \sum_{i=1}^{n}\lambda_i y_i \geq y_o + \phi_o & \\ \sum_{i=1}^{n}\lambda_i x_{im} \leq x_{om} & \forall m=1,\ldots,M \\ \sum_{i=1}^{n}\lambda_i = 1 & \\ \lambda_i \geq 0 & \forall i=1,\ldots,n \end{array}\right\}$$

其中 $\theta_o = 1 + \frac{\phi_o}{y_o}$。

上述线性规划问题的对偶为：

$$\min_{\alpha,}\left\{\alpha + x_o^{'}\beta - y_o \mid \begin{array}{ll} \alpha + x_i^{'}\beta \geq y_i & \forall i=1,\ldots,n \\ \geq 0 & \end{array}\right\}$$

引入一个新变量 $\varepsilon_o = y_o - (\alpha + x_o^{'}\beta)$，并添加这个新约束，上述线性规划问题可以被写为：

$$\min_{\alpha,,\varepsilon}\left\{-\varepsilon_o \mid \begin{array}{ll} \varepsilon_o = y_o - (\alpha + x_o^{'}\beta) & \\ \alpha + x_i^{'}\beta \geq y_i & \forall i=1,\ldots,n \\ \geq 0 & \end{array}\right\}$$

DEA问题是对每一个DMU分别求解上述线性规划问题，使用新的线性规划问题可同时求解所有DMU的线性规划问题：

$$\min_{\alpha_i,\mathbf{i},\varepsilon_i}\left\{-\sum_{i=1}^{n}\varepsilon_i \mid \begin{array}{ll} \varepsilon_i = y_i - (\alpha_i + x_i^{'}\beta_i) & \forall i=1,\ldots,n \\ \alpha_h + x_i^{'}\beta_h \geq y_i & \forall i,h=1,\ldots,n \\ \mathbf{i} \geq 0 & \forall i=1,\ldots,n \end{array}\right\}$$

通过构造，可知 $\varepsilon_i \leq 0$。另外，由于单调性，上述线性规划问题为：

$$\min_{\alpha_i,\mathbf{i},\varepsilon_i}\left\{-\sum_{i=1}^{n}\varepsilon_i \mid \begin{array}{ll} \varepsilon_i = y_i - (\alpha_i + x_i^{'}\beta_i) & \forall i=1,\ldots,n \\ \alpha_h + x_i^{'}\beta_h \geq \alpha_i + x_i^{'}\beta_i & \forall i,h=1,\ldots,n \\ \mathbf{i} \geq 0, \varepsilon_i \leq 0 & \forall i=1,\ldots,n \end{array}\right\} \tag{10.7}$$

这是模型（10.3）的限制符号的版本。因此，得到的结论是：限制符号版本的CNLS估计量等同于DEA VRS估计量。另外，可以将其他规模报酬假定的DEA估计量与限制符号版本的CNLS估计量等价：若假定DEA的规模报酬不变（CRS），则可以设定 $\alpha_i = 0$；若假定DEA的规模报酬非递增（NIRS），则可以设定 $\alpha_i \geq 0$；若假定DEA的规模报酬非递减（NDRS），则可以设定 $\alpha_i \leq 0$。在这一单产出、不含非期望产出的设定中，截距 α_i 的符号约束是规模报酬的**唯一**载体，与减排因子无关；deabook 中 `RTS_VRS1` 与 `RTS_VRS2` 的区别只出现在含非期望产出的弱可处置技术里（见 10.3 节式(10.44) 处的说明）。另外，这种方法与Aigner and Chu(1968)的参数化规划 (PP) 模型的非参数化扩展类似。

在参数模型中，除了限制综合误差项的符号，还可以通过修正的OLS（COLS）来估计非效率。COLS 是一个两阶段的过程：在第一阶段，前沿通过普通最小二乘法 (OLS) 回归估计；在第二阶段，前沿向上移动，使得产生的 COLS 前沿包含所有数据。普通最小二乘 (OLS) 对误差项没有限制，OLS 残差既有正值也有负值。OLS 残差记为 ε_i^{OLS}。在 COLS 模型中，这些误差被归因于效率低下；因此误差估计量被调整为

$$\hat{\varepsilon}_i^{COLS} = \varepsilon_i^{OLS} - \underset{i}{max}\varepsilon_i^{OLS} \quad \forall i = 1, \dots, n \tag{10.8}$$

其中，$\hat{\varepsilon}_i^{COLS}$ 的值范围为 $[-\infty, 0]$，0 表示位于最优前沿，即完全有效。类似地，将截距项调整为

$$\hat{\alpha}^{COLS} = \alpha^{OLS} + \underset{i}{max}\varepsilon_i^{OLS} \tag{10.9}$$

斜率系数 $\hat{\beta}_i^{COLS}$ 直接从OLS估计中获得，因为 $\hat{\beta}_i^{COLS} = \hat{\beta}_i^{OLS}$。

因CNLS是OLS的非参数扩展，可以将COLS中的第一步OLS估计替换为CNLS估计，并同样将COLS的第二步用于估计非效率。Kuosmanen and Johnson(2010) 将这种方法称为 C^2NLS，完整的估计步骤如下：

第一步：通过求解式 (10.3) 的 CNLS 估计量来估计 $E(y_i|x_i)$，将得到的残差记为 $\hat{\varepsilon}_i^{CNLS}$。注意，这里的 $E(\varepsilon_i|x) = -\mu \neq 0$，所以，这里估计的是 $h = f - \mu$。

第二步：类似于 COLS 过程移动残差，C^2NLS 效率估计量是

$$\hat{\varepsilon}_i^{C^2NLS} = \varepsilon_i^{CNLS} - \underset{i}{max}\varepsilon_i^{CNLS} \quad \forall i = 1, \dots, n \tag{10.10}$$

其中，$\hat{\varepsilon}_i^{C^2NLS}$ 的值范围为 $[-\infty, 0]$，0 表示位于最优前沿，即完全有效。虽然已经说明CNLS 估计量在观察点 $x_i, \forall i = 1, \dots, n$ 上是唯一的，但在未观察的 x 上不是。因此，可以使用公式（10.5）中的方法来得到 $h = f - \mu$ 的下限 $\hat{h}_{min}^{CNLS}(x)$：

$$\hat{h}_{min}^{CNLS}(x) = \min_{\alpha,\beta}\{\alpha + \beta'\mathbf{x} \mid \alpha + \beta'\mathbf{x_i} \geq \hat{h}^{CNLS}(\mathbf{x_i}), \forall i = 1, ..., n\}$$

类似地，将 C^2NLS 的截距项调整为

$$\hat{\alpha}_i^{C^2NLS} = \alpha_i^{CNLS} + \underset{i}{max}\varepsilon_i^{CNLS} \quad \forall i = 1, \dots, n \tag{10.11}$$

C^2NLS 的斜率系数 $\hat{\beta}_i^{C^2NLS}$ 直接从CNLS估计中获得，因为 $\hat{\beta}_i^{C^2NLS} = \hat{\beta}_i^{CNLS}$。$C^2NLS$ 估计的前沿为：

$$\hat{f}^{C^2NLS}(x_i) = \hat{\alpha}_i^{C^2NLS} + x_i^{'}\hat{\beta}_i^{C^2NLS} \quad \forall i = 1, \dots, n \tag{10.12}$$

Kuosmanen and Johnson(2010)证明了 C^2NLS 估计量的一致性，即前沿估计量 $\hat{h}^{C^2NLS}(\cdot)$ 在样本点上一致收敛到 $h(\cdot) = f(\cdot) - \mu$。值得说明的是，$C^2NLS$ 的一致性要求所有 DMU 的非效率项都来自相同的分布（同方差）。如果违反i.i.d.假设（例如，不同DMU的非效率具有不同分布），则无法保证 C^2NLS 的一致性。

10.1.3 随机非参数(StoNED)模型

现在，可以使用模型（10.1）中综合误差项 $\varepsilon_i = v_i - u_i$ 来进行分析，这样的模型是随机非参数模型，Kuosmanen and Kortelainen(2012)提出一种StoNED方法来估计这个问题。可以发现，$E(y_i|x) = f(x) + E(\varepsilon_i|x) = f(x) + (E(v_i|x) - E(u_i|x)) = f(x) + (-\mu)$，其中 $E(\varepsilon_i|x) = -\mu$，所以，对前沿 f 的估

计将不是一致的。为此，可以将综合误差项的条件期望中的非0项与前沿 f 合并估计，这样得到的新的误差项的条件期望就为0，即估计：

$$\begin{aligned} y_i &= f(x_i) + v_i - u_i, \\ &= (f(x_i) - \mu) + (\mu + v_i - u_i) \\ &= g(x_i) + (\mu + v_i - u_i) \quad i = 1, \dots, n \end{aligned} \tag{10.13}$$

此时，$E(y_i|x) = g(x) + E(\mu + v_i - u_i|x) = g(x) + 0$，其中 $E(\mu + v_i - u_i|x) = \mu - E(u_i) = 0$（因为 $E(v_i) = 0$，且此处 $\mu \equiv E(u_i)$ 即非效率的无条件均值）。注意，这里分析的是同方差的情况；如果是异方差的情况，还需要对综合误差项各成分的分布施加进一步假设。

在同方差的条件下，可以将StoNED方法的第一步设定为：使用CNLS估计量对模型（10.13）进行估计，得到 $g(x) = f(x) - \mu$ 的一致估计。估计的方程和公式（10.3）一致，但需要注意的是，这里使用CNLS估计量估计的残差为对 $\mu + v_i - u_i$ 的估计。

进一步，只需得到 μ，就可以得到前沿 f 的一致估计。因此，第二步使用不同方法来估计非效率。下面介绍两类路径：（一）参数化路径——使用Fan et al.(1996)提出、后经 Johnson and Kuosmanen(2015)系统整理的准似然估计（QLE），在 Stevenson 截断正态设定下估计 $(\mu_0, \sigma_u, \sigma_v)$ 进而得到 $\hat{m}_u$ 和企业特定条件非效率；（二）非参数路径——使用核密度估计（KDE）直接恢复 $\hat{f}_u$ 和 $\hat{f}_v$，通过数值积分计算条件非效率 $\hat{u}_i^{KDE} = E(u_i \mid e_i)$，绕过参数化的 μ 直接进入 JLMS 型条件期望。两种路径最终都产出企业特定条件非效率 $\hat{u}_i$，用于第三步的效率恢复。

（一）准似然估计准似然估计法需要额外的参数化分布假设。本章采用Stevenson (1980)的截断正态—正态设定：噪声项服从正态分布，非效率项来自潜在正态分布并在0 处左截断。形式上，

$$\begin{aligned} u_i &\sim N(\mu_0, \sigma_u^2) \text{ truncated at } 0, \\ v_i &\sim N(0, \sigma_v^2), \end{aligned} \tag{10.14}$$

其中 μ_0 是截断前潜在正态分布的均值，可以不等于 0；u_i 的无条件均值不是 μ_0，而是

$$m_u = E(u_i) = \mu_0 + \sigma_u \frac{\phi(\mu_0/\sigma_u)}{\Phi(\mu_0/\sigma_u)}, \tag{10.15}$$

这里 $\phi(\cdot)$ 和 $\Phi(\cdot)$ 分别为标准正态密度函数和分布函数。半正态设定只是 $\mu_0 = 0$ 的特例。需要特别指出，前文 10.1 节中用于 $g = f - \mu$ 分解的非效率均值 μ，在 Stevenson 截断正态设定下即为截断后均值 $m_u = E(u_i)$，它由潜在均值 μ_0 经 (10.15) 决定，而非 μ_0 本身。

在生产型模型中，复合误差可写为 $\varepsilon_i = v_i - u_i$。CNLS 残差 $\hat{\varepsilon}_i^{CNLS}$ 是对 $m_u + v_i - u_i$（零均值）的估计；为进入截断正态—正态复合误差似然，需先将其转回原始复合误差尺度：

$$e_i = \hat{\varepsilon}_i - m_u \tag{10.16}$$

给定 $\sigma = (\sigma_u^2 + \sigma_v^2)^{1/2}$，单个观测的对数似然可写为

$$\ell_i = -\ln \sigma + \ln \phi \left(\frac{e_i + \mu_0}{\sigma} \right) + \ln \Phi \left(\frac{\mu_0 \sigma_v^2 - e_i \sigma_u^2}{\sigma \sigma_u \sigma_v} \right) - \ln \Phi \left(\frac{\mu_0}{\sigma_u} \right). \tag{10.17}$$

通过最大化 $\sum_i \ell_i$ 可得到 $\hat{\mu}_0$、$\hat{\sigma}_u$ 和 $\hat{\sigma}_v$，并进一步计算 $\hat{m}_u = E(u_i)$。代码中保留 `mu_latent` 表示 $\hat{\mu}_0$，保留 `mu` 表示截断后均值 $\hat{m}_u$，以避免将潜在均值和非效率均值混用。

（二）非参数核密度估计法与准似然估计不同，非参数核密度估计法不施加Stevenson 截断正态参数化假设。需要注意的是，仅凭复合误差 $e_i = v_i - u_i$ 本身并不能在没有额外识别约束时唯一分解出 f_u 与 f_v。因此，`deabook` 的 `RED_KDE` 被定位为非参数稳健性路径：先按残差偏态构造非负单边非效率初始样本，在 $u \geq 0$ 上用边界反射 KDE 估计 $\hat{f}_u(u)$，并由中心化噪声样本估计 $\hat{f}_v(v)$，再通过 Bayes 公式的数值积分计算企业特定条件非效率 $E(u_i \mid e_i)$。

具体地，先由 CNLS 残差构造非效率种子样本（取残差位于零均值右侧的部分并截断为非负）和噪声种子样本，再用高斯核分别估计 $\hat{f}_u(u)$ 和 $\hat{f}_v(v)$。在 $u_i \geq 0$ 的约束下，条件非效率的核密度数值估计为

$$\hat{u}_i^{KDE} = E(u_i \mid e_i) = \frac{\int_0^\infty u\, \hat{f}_v(e_i + u)\, \hat{f}_u(u)\, du}{\int_0^\infty \hat{f}_v(e_i + u)\, \hat{f}_u(u)\, du}, \tag{10.18}$$

其中 $e_i = r_i - \hat{m}_u$ 为中心化后的复合误差（与 (10.16) 同形），$\hat{f}_v(e_i + u)$ 反映噪声把真实非效率 u 平滑到观测误差 $e_i + u$ 处的概率权重。分子是带 u 加权的积分（一阶矩），分母是归一化积分（零阶矩），二者之比即截断非效率的条件期望。

需要强调的是，KDE 路径**不再估计** Stevenson 截断正态的 $(\mu_0, \sigma_u, \sigma_v)$，而是基于单边残差初始化与 KDE 数值积分得到企业特定条件非效率 $\hat{u}_i^{KDE}$；它不返回 Stevenson 意义下的无条件均值 $\hat{\mu}$，也不应被解释为形式化盲反卷积估计。因此KDE 结果应理解为非参数条件均值估计，而不是 Stevenson 参数估计的近似。这一路径与 Hall and Simar(2002) 提出的残差密度不连续点检测法在目标上相通（均不依赖参数分布假设），但实现上用数值条件期望替代了导数最大变化点定位。

接下来，便可以使用第一步得到的 g 的CNLS估计量和第二步得到的非效率的期望来计算前沿 f。但是，如前所述，CNLS 估计量在观察点 $x_i, \forall i = 1, ..., n$ 上是唯一的，但在未观察的 x 上不是。因此，Kuosmanen and Kortelainen(2012)建议应用 g 的下限（类似于公式（10.5）），定义为

$$\hat{g}_{min}^{CNLS}(x) = \min_{\alpha,\beta}\{\alpha + \beta' \mathbf{x} \mid \alpha + \beta' \mathbf{x_i} \geq \hat{g}^{CNLS}(\mathbf{x_i}), \forall i = 1, ..., n\}$$

随后，可以提出StoNED方法的第三步：在 g 的CNLS估计量中加入非效率的期望 μ 来估计前沿 f，使用：

$$\hat{f}^{StoNED}(x) = \hat{g}_{min}^{CNLS}(x) + \hat{\mu}$$

必须强调，即使第三步在统计上使用一致的方法来估计前沿的 f，企业特定非效率 u_i 本身仍需要基于条件分布进一步估计。在Stevenson 截断正态—正态设定下，`deabook` 先将 CNLS 残差写成零均值残差

$$r_i = \hat{\varepsilon}_i^{CNLS} \approx v_i - u_i + m_u,$$

再转回原始复合误差尺度

$$e_i = r_i - \hat{m}_u \approx v_i - u_i.$$

令

$$\sigma_*^2 = \frac{\sigma_u^2 \sigma_v^2}{\sigma_u^2 + \sigma_v^2}, \qquad \mu_{*i} = \frac{\mu_0 \sigma_v^2 - e_i \sigma_u^2}{\sigma_u^2 + \sigma_v^2},$$

其中 $e_i = \hat{\varepsilon}_i - \hat{m}_u$。则 $u_i \mid e_i$ 服从在 0 处截断的正态分布，其条件期望（JLMS 型非效率估计，即 Jondrow、Lovell、Materov 和 Schmidt (1982) 提出的条件期望法在截断正态下的推广，参见 Kumbhakar 和 Lovell (2000)）为

$$\hat{u}_i = E(u_i \mid e_i) = \mu_{*i} + \sigma_* \frac{\phi(\mu_{*i}/\sigma_*)}{\Phi(\mu_{*i}/\sigma_*)}.$$

得到企业特定非效率 $\hat{u}_i = E(u_i \mid e_i)$ 后，便可以恢复效率值。对于普通产出型加性模型，效率可理解为实际产出与条件前沿产出的比值：

$$TE_i = \frac{y_i}{y_i + \hat{u}_i}.$$

对于乘法误差或 Shephard 型距离函数，条件无效率进入对数尺度，`deabook` 使用指数转换恢复径向效率：

$$TE_i = \exp(-\hat{u}_i).$$

对于 DDF 模型，$\hat{u}_i$ 在归一化方向距离尺度（如 CO2 量级）上取值，直接取指数会导致数值趋近于 0。因此 DDF 模型先用真实变量归一化：对非期望产出方向 g^b，$TE_i = (b_i - \hat{u}_i)/b_i$；对期望产出方向 g^y，$TE_i = y_i/(y_i + \hat{u}_i \cdot g^y)$；对投入方向 g^x，$TE_i = (x_i - \hat{u}_i \cdot g^x)/x_i$。多方向同时存在时取绑定（最小）分量比率。所有TE 经 clip 后满足 $0 \leq TE_i \leq 1$。

因此，StoNED 的“恢复效率” 分成两步：第一步由 JLMS 条件期望（式(10.35)/(10.46)）从复合误差中分离出条件非效率 $\hat{u}_i = E(u_i \mid e_i)$；第二步按距离函数类型把 $\hat{u}_i$ 转换成效率——Shephard 乘法模型取 $TE_i = \exp(-\hat{u}_i)$（式(10.36)），DDF 加性模型取式 (10.36) 后续的方向归一化比率。两步的输出即为技术非效率 $\hat{u}_i$ 与技术效率 TE_i。

Kuosmanen and Kortelainen(2012)提出，StoNED模型也可以通过对不限制符号的CNLS中改变 α_i 的限制条件来获得（CRS、NIRS、NDRS设定方式与第 10.1.2 节相同，详见 10.1.2 节末说明）。

10.1.4 乘法模型

虽然 CNLS 回归很容易适应替代 RTS 假设，但对效率估计的影响有些棘手。具体而言，如果估计受 CRS 约束的平均实践技术 g，再将前沿向上平移预期无效率的估计值 $\hat{\mu}$，则所得最佳实践前沿通常不满足 CRS。这是由于 (10.1) 中假设的非效率与噪声项的加性结构与规模属性的乘法性质不匹配。如果施加CRS、NIRS 或 NDRS 假设，则采用非效率与噪声的乘法设定在逻辑上是一致的，接下来将讨论。

可以将模型 (10.1) 改写为：

$$y_i = f(\mathbf{x}_i) \cdot \exp(\varepsilon_i) = f(\mathbf{x}_i) \cdot \exp(v_i - u_i) \tag{10.19}$$

如上所述，CRS 假设需要乘法误差结构。此外，乘法误差规范可能有助于缓解不同尺度大小的异方差性（参见 Caudill and Ford (1993)）。

对生产函数 f 和复合误差项保持与模型 (10.1) 中相同的假设。对等式（10.19）应用对数变换，得到

$$\ln y_i = \ln f(\mathbf{x}_i) + \varepsilon_i. \tag{10.20}$$

仍然要求CNLS估计量估计的误差项的期望为0。因此，将公式（10.20）改写为

$$\ln y_i = (\ln f(\mathbf{x}_i) - \mu) + (\varepsilon_i + \mu) = \ln g(\mathbf{x}_i) + (\varepsilon_i + \mu) \tag{10.21}$$

其中 μ 是预期的低效率，为了通过 CNLS 估计 $\ln g$，可以将（10.3）改写为如下非线性规划（NLP）问题（因 $\ln(\phi_i+1)$ 关于变量 ϕ_i 非线性，需采用非线性求解器，如Knitro）：

$$\begin{aligned}&\min_{\alpha,\beta,\phi,\varepsilon}\sum_{i=1}^{n}\left(\varepsilon_i^{CNLS}\right)^2\\&\text{s.t.}\\&\ln y_i=\ln\left(\phi_i+1\right)+\varepsilon_i^{CNLS}\forall i\\&\phi_i+1=\alpha_i+\beta_i'\mathbf{x}_i\forall i\\&\alpha_i+\beta_i'\mathbf{x}_i\leq\alpha_h+\beta_h'\mathbf{x}_i\forall h,i\\&\beta_i\geq\mathbf{0}\forall i\end{aligned}\tag{10.22}$$

其中 ϕ_i+1 是 $\ln g(\mathbf{x}_i)$ 取指数后的CNLS 估计量（近似 $g(\mathbf{x}_i)$；对 $\exp(\cdot)$ 变换应用Jensen 不等式可知严格地 $E(y_i|\mathbf{x}_i)=E(f\cdot e^{\varepsilon_i})\neq g(\mathbf{x}_i)\cdot e^{E(\varepsilon_i)}$）。这里加上1的值是为了确保计算算法不会试图取零的对数。第一个等式可以解释为对数转换的回归方程（使用自然对数函数 $ln(.)$）。第二至第五个约束条件与(10.3)相似，但(10.3)中观察到的产出被替换为 ϕ_i+1。使用 ϕ_i 可以估计产出和投入之间的乘法关系，同时保证原始投入产出空间中生产可能性集的凸性。

在乘法模型下，可以应用CNLS估计量作为第 10.1.3 节描述的StoNED估计程序的第一步。接着使用第 10.1.3 节描述的StoNED估计程序的第二步估计非效率的均值 μ。在进行第三步之前，仍然使用最小外推方法，估计 g 的下限：

$$\hat{g}_{min}^{CNLS}(x)=\min_{\alpha,\beta}\{\alpha+\beta'\mathbf{x}\mid\alpha+\beta'\mathbf{x_i}\geq\hat{g}^{CNLS}(\mathbf{x_i}),\forall i=1,...,n\}$$

第三步，鉴于 $(\ln f(\mathbf{x}_i)-\mu)=\ln g(\mathbf{x}_i)$，等价于 $f(\mathbf{x}_i)/exp(\mu)=g(\mathbf{x}_i)$，使用 $\hat{f}^{StoNED}(\mathbf{x}_i)=\hat{g}_{\min}^{CNLS}(\mathbf{x})\cdot\exp(\hat{\mu})$ 作为前沿 f 的估计量。

10.1.5 背景变量

以乘法模型为起点，引入背景变量（contextual variable）。用r维向量 $\mathbf{z}_i$ 表示背景变量的观测值，假设综合误差项为 $\varepsilon_i=v_i-u_i+\mathbf{z}_i'\eta$，从而得到以下半非参数化的偏对数线性方程（此处用 η 表示背景变量系数，以避免与 10.3 节中用作非期望产出系数的 δ 混淆）：

$$\ln y_i=\ln f(\mathbf{x}_i)+\eta'\mathbf{z}_i+v_i-u_i\tag{10.23}$$

其中，$\eta=(\eta_1,...,\eta_r)'$ 是一个待估计的参数向量，表示背景变量 $\mathbf{z}_i$ 对绩效的平均影响。还有其他的设置背景变量的方法，如将 $\mathbf{z}$ 变量影响随机非效率项 u_i 的概率分布。需要指出的是，这两种设置具有非常大的差异。这里主要涉及公式（10.23）中的设置方法。Johnson and Kuosmanen(2011)认为公式（10.23）中的设置方法需要更少的假设条件。

背景变量 $\mathbf{z}_i$ 可以直接纳入StoNED问题中的目标函数，公式（10.23）可以使用与公式（10.3）相似的方程求解：

$$\begin{aligned}&\min_{\alpha,\beta,\eta,\phi,\varepsilon}\sum_{i=1}^{n}\left(\varepsilon_i^{CNLS}\right)^2\\&\text{s.t.}\\&\ln y_i=\ln\left(\phi_i+1\right)+\eta'\mathbf{z}_i+\varepsilon_i^{CNLS}\forall i\\&\phi_i+1=\alpha_i+\beta_i'\mathbf{x}_i\forall i\\&\alpha_i+\beta_i'\mathbf{x}_i\leq\alpha_h+\beta_h'\mathbf{x}_i\forall h,i\\&\beta_i\geq\mathbf{0}\forall i\end{aligned}\tag{10.24}$$

其中，$\hat{\eta}^{\text{StoNEZD}}$ 表示作为模型（10.23）的最优解得到的背景变量的系数。Johnson and Kuosmanen(2011)详细研究了这个估计量的统计特性，显示了其 $\sqrt{n}$ 一致性、渐近正态性和渐近有效性（该文并未主张有限样本无偏性）。

10.1.6 凸非参数分位数回归

进一步定义凸非参数分位数回归模型，从而在数据内部分析投入的边际产出。

令 $Y \mid \mathbf{x}$ 是给定投入水平 $\mathbf{x}$ 的随机产出 Y，其条件分布为 $F_{Y|\mathbf{x}}$，即：$F_{Y|\mathbf{x}}(y) \triangleq \text{Prob}\{Y \leqslant y \mid \mathbf{x}\}$。那么，在投入水平 $\mathbf{x}$ 的 τ-分位前沿被定义为：

$$\varphi_\tau(\mathbf{x}) \triangleq \sup\left\{y \in \mathbb{R}_+ \mid F_{Y|\mathbf{x}}(y) \leqslant 1-\tau\right\}$$

如果 $F_{Y|\mathbf{x}}$ 是连续且严格增加的，有 $\varphi_\tau(\mathbf{x}) = F^{-1}_{Y|\mathbf{x}}(1-\tau)$。$\tau$-分位前沿的经济含义很直观：对于任意给定的投入水平 $\mathbf{x}$，有效与非有效观测的占比分别为 $1-\tau$ 和 τ。

Wang et al.(2014)通过类比线性分位数回归中关于目标函数的设定和凸非参数最小二乘中关于凸性和单调性假设的设定，定义了如下凸非参数分位数回归模型(CNQR)的求解方法：

$$\begin{aligned}
&\min_{\alpha,\beta,\varepsilon^+,\varepsilon^-} \tau\sum_{i=1}^n \varepsilon_i^+ + (1-\tau)\sum_{i=1}^n \varepsilon_i^- \\
&\text{s.t.} \\
&y_i = \alpha_i + \beta_i'\mathbf{x}_i + \varepsilon_i^+ - \varepsilon_i^- \forall i \\
&\alpha_i + \beta_i'\mathbf{x}_i \leq \alpha_h + \beta_h'\mathbf{x}_i \forall h,i \\
&\beta_i \geq \mathbf{0} \forall i \\
&\varepsilon_i^+ \geq 0 \forall i \\
&\varepsilon_i^- \geq 0 \forall i
\end{aligned} \tag{10.25}$$

与CNLS相比，CNQR的一个主要的缺点是该问题的最优解不一定是唯一的，甚至对于观察到的数据点 $(\mathbf{x}_i, y_i), i = 1, \ldots, n$ 也是如此。Kuosmanen and Johnson(2017)建议使用以下的目标函数的二次方表述来求得唯一解：

$$\begin{aligned}
&\min_{\alpha,\beta,\varepsilon^+,\varepsilon^-} \tau\sum_{i=1}^n (\varepsilon_i^+)^2 + (1-\tau)\sum_{i=1}^n (\varepsilon_i^-)^2 \\
&\text{s.t.} \\
&y_i = \alpha_i + \beta_i'\mathbf{x}_i + \varepsilon_i^+ - \varepsilon_i^- \forall i \\
&\alpha_i + \beta_i'\mathbf{x}_i \leq \alpha_h + \beta_h'\mathbf{x}_i \forall h,i \\
&\beta_i \geq \mathbf{0} \forall i \\
&\varepsilon_i^+ \geq 0 \forall i \\
&\varepsilon_i^- \geq 0 \forall i
\end{aligned} \tag{10.26}$$

对异方差的稳健性被普遍认为是分位数回归的主要优势之一（例如Koenker, 2005）。如果 ε_i 是同方差的，那么根据经验估计的分位数前沿的形状在不同的 τ 下将是相似的。在这种情况下，估计一个单一的分位数指标或条件平均值就足以得到任何感兴趣的分位数指标。通过实证估计与不同绩效水平相关的多个分位数指标，可以有效地放松同方差假设，因为分位数前沿的形状可以在不同的 τ 之间自由地变化。

10.2 多产出StoNED模型的设定

到目前为止，CNLS和StoNED框架一直是在单产出、多投入情形下展开的。在本节中，将在Chambers et al.(1996, 1998) 的方向距离函数（DDF）框架内描述CNLS估计量。CNLS 的表述从结构上满足了 DDF 的公理特性，对多投入和多产出进行建模，并明确说明了随机噪声，解决了 DEA 和参数化方法的关键限制。下面将简要介绍CNLS对DDF的估计。

考虑一种联合生产技术，将投入向量 $\mathbf{x}=(x_1 \dots x_m)' \in \mathfrak{R}_+^m$ 转换为期望产出 $\mathbf{y}=(y_1 \dots y_s)' \in \mathfrak{R}_+^s$ 。基准技术的特点是生产可能性集，定义为：

$$T=\left\{(\mathbf{x},\mathbf{y}) \in \mathfrak{R}_+^{m+s} \mid \mathbf{X} \text{ 可以生产 } (\mathbf{y})\right\}. \tag{10.27}$$

本章考虑的基准技术的经典公理包括自由可处置性（即 $T=T+\mathfrak{R}_+^m \times \mathfrak{R}_-^s$）和凸性（即 $T=\mathrm{Conv}(T)$）（见，例如， Färe and Primont, 1995）。

方向距离函数被用来表示多产出的生产函数。Chambers et al.(1996, 1998) 介绍的方向距离函数（DDF）定义为：

$$\vec{D}\left(\mathbf{x},\mathbf{y},\mathbf{g}^x,\mathbf{g}^y\right)=\sup\left\{\theta \mid (\mathbf{x}-\theta\mathbf{g}^x,\mathbf{y}+\theta\mathbf{g}^y) \in T\right\} \tag{10.28}$$

其中 $(\mathbf{g}^x,\mathbf{g}^y) \in \mathfrak{R}_+^{m+s}$ 是一个方向向量。

DDF继承了生产可能性集的公理属性（见 Chambers, Chung and Färe, 1998, Lemma 2.2）。任何DDF必须满足的两个基本属性包括转换属性（即 $\vec{D}\left(\mathbf{x}-\lambda\mathbf{g}^x,\mathbf{y}+\lambda\mathbf{g}^y,\mathbf{g}^x,\mathbf{g}^y\right)=\vec{D}\left(\mathbf{x},\mathbf{y},\mathbf{g}^x,\mathbf{g}^y\right)-\lambda$）和DDF对方向向量的-1度齐次性（即 $\vec{D}\left(\mathbf{x},\mathbf{y},\lambda\mathbf{g}^x,\lambda\mathbf{g}^y\right)=(1/\lambda)\cdot\vec{D}\left(\mathbf{x},\mathbf{y},\mathbf{g}^x,\mathbf{g}^y\right),\lambda>0$）。这些属性对DDF的内部一致性至关重要，必须在DDF的经验估计中施加这些约束。

Kuosmanen and Johnson(2017)引入了一个DDF的概率模型，其中所有的投入和产出 $(\mathbf{x},\mathbf{y})$ 都受到综合误差项 ε_i 和位于生产前沿的投入产出 $(\mathbf{x}^*,\mathbf{y}^*)$ 的影响，其中综合误差项 $\varepsilon_i=u_i+v_i$ 包含非负非效率项 u_i 和随机噪声项 v_i，即（注意：此处DDF 框架下 $\varepsilon_i=u_i+v_i$ 为非负值，与 10.1 节单产出模型 $\varepsilon_i=v_i-u_i$ 的符号约定相反；后续 JLMS 公式与残差调整均按 ε_i 取负、统一回到 v_i-u_i 尺度处理）：

$$(\mathbf{x}_i,\mathbf{y}_i)=(\mathbf{x}_i^*+\varepsilon_i\mathbf{g}_i^x,\mathbf{y}_i^*-\varepsilon_i\mathbf{g}_i^y)\,\forall i=1,\dots,n,$$

由于观察到的数据受到非效率和噪声的干扰，第i个DMU的DDF值是一个随机变量：

$$\vec{D}_T\left(\mathbf{x}_i,\mathbf{y}_i,\mathbf{g}_i^x,\mathbf{g}_i^y\right)=\varepsilon_i \quad \forall i \tag{10.29}$$

由于所有投入和产出都受到非效率和噪声的影响，因此，所有投入和产出都是内生变量，在进行估计时需要解决这一问题。可以利用DDF的转换属性，估计：

$$\vec{D}_T\left(\vec{\mathbf{x}}_i,\vec{\mathbf{y}}_i,\mathbf{g}_i^x,\mathbf{g}_i^y\right)-\frac{y_{1i}}{g_1^y}=\varepsilon_i \tag{10.30}$$

即：

$$\frac{y_{1i}}{g_1^y}=\vec{D}_T\left(\vec{\mathbf{x}}_i,\vec{\mathbf{y}}_i,\mathbf{g}_i^x,\mathbf{g}_i^y\right)-\varepsilon_i=\vec{D}_T\left(\vec{\mathbf{x}}_i,\vec{\mathbf{y}}_i,\mathbf{g}_i^x,\mathbf{g}_i^y\right)-(u_i+v_i) \tag{10.31}$$

其中，$\vec{\mathbf{x}}_i=\mathbf{x_i}+\frac{y_{1i}}{g_1^y}\mathbf{g}^x$，$\vec{\mathbf{y}}_i=\mathbf{y_i}-\frac{y_{1i}}{g_1^y}\mathbf{g}^y$。可以证明，此时，$\mathrm{Cov}\left(\vec{\mathbf{x}}_i,\varepsilon_i\right)=0$，$\mathrm{Cov}\left(\vec{\mathbf{y}}_i,\varepsilon_i\right)=0$。这里以 y_{1i}/g_1^y 作平移基准，前提是 $g_1^y\neq 0$；若方向向量在期望产出上全为零（例如只压缩投入），则须换用其他非零分量对应的变量作基准，一般形式见 10.3 节 t_i 的定义。

类比单产出的StoNED问题，可以在第一步中使用二次规划问题计算方向距离函数的超平面。需要注意的是 $E(\varepsilon_i) = E(u_i + v_i) = \mu \neq 0$，所以，需要先估计：

$$\frac{y_{1i}}{g_1^y} = (\vec{D}_T(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y) - \mu) - (\varepsilon_i - \mu) = \vec{D}_{T2}(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y) - (\varepsilon_i - \mu) \quad \forall i = \vec{D}_{T2}(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y) + \varepsilon_i^{CNLS} \quad \forall i \tag{10.32}$$

可以发现上式与单产出的StoNED模型（10.13）一致，因此，可以按照同样的方法计算超平面、无条件非效率 μ 和条件非效率 $E(u_i|\varepsilon_i)$。

$\vec{D}_{T2}$ 的估计值 $\hat{\vec{D}}_{T2}$ 的计算方法如下：

$$\begin{aligned}
&\min_{\alpha,\beta,\gamma,\varepsilon^{CNLS}}, \sum_{i=1}^n \left(\varepsilon_i^{CNLS}\right)^2 \\
&\text{s.t.} \\
&\tfrac{y_{1i}}{g_1^y} = \alpha_i + \beta_i'\vec{\mathbf{x}}_i - \gamma_i'\vec{\mathbf{y}}_i + \varepsilon_i^{CNLS} && \forall i \\
&\alpha_i + \beta_i'\vec{\mathbf{x}}_i - \gamma_i'\vec{\mathbf{y}}_i \leq \alpha_h + \beta_h'\vec{\mathbf{x}}_i - \gamma'_h\vec{\mathbf{y}}_i && \forall i, h \\
&\beta_i'\mathbf{g}^x + \gamma_i'\mathbf{g}^y = 1 && \forall i \\
&\beta_i \geq \mathbf{0}, \gamma_i \geq \mathbf{0} && \forall i
\end{aligned} \tag{10.33}$$

第二步是像单产出的模型中一样，使用准似然估计（Fan et al., 1996；Stevenson 截断正态设定）或非参数核密度估计（KDE 数值条件期望，参见 10.1.3 节(二)）来估计企业特定条件非效率 $\hat{u}_i$。

第三步是从 $\vec{D}_{T2}$ 的估计值 $\hat{\vec{D}}_{T2}$ 中减去 μ 的估计值 $\hat{\mu}$。但首先，请注意，CNLS 估计量在观察点 $(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i), \forall i = 1, \ldots, n$ 上是唯一的，但在未观察的 $(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i)$ 上不是。为了解决非唯一性问题，Kuosmanen and Kortelainen(2012) 求助于最小外推原则，并提出估计 CNLS 估计量拟合值的最小包络。同样，对 $\hat{\vec{D}}_{T2}^*$ 的最小外推估计 $\hat{\vec{D}}_{T2,min}^*$ 由下式给出：

$$\begin{aligned}
&\min_{\alpha,\beta,\gamma} \alpha + \beta'\mathbf{x} - \gamma'\mathbf{y} \\
&\text{s.t.} \\
&\alpha + \beta'\hat{\mathbf{x}}_i - \gamma'\hat{\mathbf{y}}_i \geq 0 \qquad \forall i \\
&\beta'\mathbf{g}^x + \gamma'\mathbf{g}^y = 1 \\
&\beta \geq \mathbf{0}, \gamma \geq \mathbf{0}
\end{aligned} \tag{10.34}$$

其中，$\hat{\mathbf{x}}_i = \mathbf{x_i} - (\hat{\mu} + \hat{\varepsilon}_i^{CNLS})\mathbf{g}^x$，$\hat{\mathbf{y}}_i = \mathbf{y_i} + (\hat{\mu} + \hat{\varepsilon}_i^{CNLS})\mathbf{g}^y$。

第四步是用 Stevenson 截断正态—正态残差分解结果恢复企业特定无效率和效率。deabook 先根据方向符号得到统一尺度下的 CNLS 残差 r_i^{DDF}，再转回原始复合误差尺度

$$e_i = r_i^{DDF} - \hat{m}_u.$$

令

$$\hat{\sigma}_*^2 = \frac{\hat{\sigma}_u^2\hat{\sigma}_v^2}{\hat{\sigma}_u^2 + \hat{\sigma}_v^2}, \qquad \hat{\mu}_{*i} = \frac{\hat{\mu}_0\hat{\sigma}_v^2 - e_i\hat{\sigma}_u^2}{\hat{\sigma}_u^2 + \hat{\sigma}_v^2}.$$

则 Stevenson-JLMS 条件非效率为

$$\hat{u}_i = E(u_i \mid e_i) = \hat{\mu}_{*i} + \hat{\sigma}_* \frac{\phi(\hat{\mu}_{*i}/\hat{\sigma}_*)}{\Phi(\hat{\mu}_{*i}/\hat{\sigma}_*)}. \tag{10.35}$$

其中 ϕ 是标准正态分布 $N(0,1)$ 的密度函数，Φ 是相应的累积分布函数。

得到条件非效率 $\widehat{u}_i$ 后，对 Shephard 乘法模型（对数尺度），技术效率由指数转换恢复：

$$TE_i = \exp(-\widehat{u}_i). \tag{10.36}$$

对 DDF 加性模型，$\widehat{u}_i$ 带有方向变量单位，需按对应真实变量归一化后形成有界效率比率：期望产出方向用 $TE_i = y_i/(y_i + \widehat{u}_i g^y)$，投入方向用 $TE_i = (x_i - \widehat{u}_i g^x)/x_i$，非期望产出方向用 $TE_i = (b_i - \widehat{u}_i g^b)/b_i$；多方向同时存在时取绑定的最小方向比率 $TE_i = \min\{TE_i^x, TE_i^y, TE_i^b\}$ 作为总效率。方向距离本身记为 $\widehat{u}_i$，辅助距离分数 $1/(1+\widehat{u}_i)$ 仅用于跨 DMU 排序。

记号说明：$\widehat{u}_i$ 表示 JLMS 条件非效率，定义为 $\widehat{u}_i = E(u_i \mid e_i)$，由式 (10.35) 或 (10.46) 给出。非 DDF 模型的技术效率为式 (10.36) 的 $TE_i = \exp(-\widehat{u}_i)$；DDF 模型则按上述方向归一化比率恢复效率。注意，DDF 框架下，当方向作用于投入（g^x）或非期望产出（g^b）时，CNLS 残差会先取负（对应式 (10.29) 处的符号约定：把 $\varepsilon_i = u_i + v_i$ 映射回 $v_i - u_i$ 尺度）；当方向作用于期望产出（g^y）时不取负。这一方向相关的符号处理的目的，是统一回到 $v_i - u_i$ 尺度后，再代入上述 (10.35) 形式的 JLMS 公式。

需要强调的是，两种效率形式均满足 $0 \le TE_i \le 1$。对 Shephard 乘法模型，$\widehat{u}_i$ 在对数尺度上，$TE_i = \exp(-\widehat{u}_i)$ 即为径向技术效率；对 DDF 加性模型，$\widehat{u}_i$ 在归一化方向距离尺度上，需通过上述方向归一化比率得到有界的 TE_i。除 TE_i 之外，方向距离 $\widehat{u}_i$ 与辅助距离分数 $1/(1+\widehat{u}_i)$ 也可作为排序指标。

10.3 有非期望产出的StoNED模型的设定

考虑一种联合生产技术，将投入向量 $\mathbf{x} = (x_1 \dots x_m)' \in \mathfrak{R}_+^m$ 转换为期望产出 $\mathbf{y} = (y_1 \dots y_s)' \in \mathfrak{R}_+^s$ 和非期望产出 $\mathbf{b} = (b_1 \dots b_q)' \in \mathfrak{R}_+^q$。基准技术的特点是生产可能性集，定义为：

$$T = \left\{(\mathbf{x}, \mathbf{y}, \mathbf{b}) \in \mathfrak{R}_+^{m+s+q} \mid \mathbf{X} \text{ 可以生产 } (\mathbf{y}, \mathbf{b})\right\}. \tag{10.37}$$

本章考虑的基准技术的经典公理包括自由可处置性（即 $T = T + \mathfrak{R}_+^m \times \mathfrak{R}_-^s \times \mathfrak{R}_+^q$）和凸性（即 $T = \text{Conv}(T)$）（见，例如，Färe and Primont, 1995）。另外，假设非期望产出 $\mathbf{b}$ 和投入 $\mathbf{x}$ 是单调递增的，且期望产出是单调递减的。对非期望产出的单调性显然与物质平衡原则不一致：对于给定的物质投入，非期望产出不能无限增加（例如，Rødseth, 2016, 2017）。然而，这里假设单调性可以保证非期望产出的影子价格是非负的。另外，也可以不假设非期望产出的单调性，从而施加非期望产出的弱可处置性假设。

与 10.2 节同理，方向距离函数被用来表示多产出的生产函数。Chambers, Chung and Färe(1996, 1998) 介绍的方向距离函数（DDF）在含非期望产出的情形下定义为：

$$\vec{D}\left(\mathbf{x}, \mathbf{y}, \mathbf{b}, \mathbf{g}^x, \mathbf{g}^y, \mathbf{g}^b\right) = \sup\left\{\theta \mid (\mathbf{x} - \theta\mathbf{g}^x, \mathbf{y} + \theta\mathbf{g}^y, \mathbf{b} - \theta\mathbf{g}^b,) \in T\right\}, \tag{10.38}$$

其中 $\left(\mathbf{g}^x, \mathbf{g}^y, \mathbf{g}^b\right) \in \mathfrak{R}_+^{m+s+q}$ 是一个方向向量。

DDF继承了生产可能性集的公理属性（见 Chambers, Chung and Färe, 1998, Lemma 2.2）。任何DDF必须满足的两个基本属性包括转换属性（即 $\vec{D}\left(\mathbf{x} - \lambda\mathbf{g}^x, \mathbf{y} + \lambda\mathbf{g}^y, \mathbf{b} - \lambda\mathbf{g}^b, \mathbf{g}^x, \mathbf{g}^y, \mathbf{g}^b\right) = \vec{D}\left(\mathbf{x}, \mathbf{y}, \mathbf{b}, \mathbf{g}^x, \mathbf{g}^y, \mathbf{g}^b\right) - \lambda$）和DDF对方向向量的-1度齐次性（即 $\vec{D}\left(\mathbf{x}, \mathbf{y}, \mathbf{b}, \lambda\mathbf{g}^x, \lambda\mathbf{g}^y, \lambda\mathbf{g}^b\right) = (1/\lambda) \cdot \vec{D}\left(\mathbf{x}, \mathbf{y}, \mathbf{b}, \mathbf{g}^x, \mathbf{g}^y, \mathbf{g}^b\right), \lambda > 0$）。这些属性对DDF的内部一致性至关重要，必须在DDF的经验估计中施加这些约束。

Kuosmanen and Johnson(2017)引入了一个DDF的概率模型，其中所有的投入和产出 $(\mathbf{x}, \mathbf{y}, \mathbf{b})$ 都受到综合误差项 ε_i 和位于生产前沿的投入产出 $(\mathbf{x}^*, \mathbf{y}^*, \mathbf{b}^*)$ 的影响，其中综合误差项 $\varepsilon_i = u_i + v_i$ 包含非负非效

率项 u_i 和随机噪声项 v_i，即：

$$(\mathbf{x}_i, \mathbf{y}_i, \mathbf{b}_i) = (\mathbf{x}_i^* + \varepsilon_i \mathbf{g}_i^x, \mathbf{y}_i^* - \varepsilon_i \mathbf{g}_i^y, \mathbf{b}_i^* + \varepsilon_i \mathbf{g}_i^b) \; \forall i = 1, \ldots, n \tag{10.39}$$

由于观察到的数据受到非效率和噪声的干扰，第i个DMU的DDF值是一个随机变量：

$$\vec{D}_T\left(\mathbf{x}_i, \mathbf{y}_i, \mathbf{b}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) = \varepsilon_i \quad \forall i \tag{10.40}$$

如前所述，所有投入和产出均为内生变量，估计时需要解决这一问题。可以利用DDF的转换属性把其中一个变量移到方程左端作为因变量。**这里的关键是：作为平移基准的变量，其对应的方向向量分量必须非零**。记该基准为 t_i，它可以取自方向向量任一非零分量所对应的变量：

$$t_i = \begin{cases} y_{ji}/g_j^y, & \text{若存在 } g_j^y \neq 0 \\ -x_{ji}/g_j^x, & \text{若 } \mathbf{g}^y = \mathbf{0} \text{ 且存在 } g_j^x \neq 0 \\ -b_{ji}/g_j^b, & \text{若 } \mathbf{g}^y = \mathbf{g}^x = \mathbf{0} \text{ 且存在 } g_j^b \neq 0 \end{cases}$$

（`deabook` 的 `assert_CNLSDDFweak1` 正是按 $\mathbf{g}^y \to \mathbf{g}^x \to \mathbf{g}^b$ 的顺序取第一个正分量构造 `basexyb`，见 `utils/tools.py`。）于是要估计的是：

$$\vec{D}_T\left(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \vec{\mathbf{b}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) - t_i = \varepsilon_i \tag{10.41}$$

即：

$$t_i = \vec{D}_T\left(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \vec{\mathbf{b}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) - \varepsilon_i = \vec{D}_T\left(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \vec{\mathbf{b}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) - (u_i + v_i) \tag{10.42}$$

其中，$\vec{\mathbf{x}}_i = \mathbf{x_i} + t_i \mathbf{g}^x$，$\vec{\mathbf{y}}_i = \mathbf{y_i} - t_i \mathbf{g}^y$，$\vec{\mathbf{b}}_i = \mathbf{b_i} + t_i \mathbf{g}^b$；按 t_i 的构造，作为基准的那一个变量在平移后恰好等于零。可以证明，此时，$\mathrm{Cov}\left(\vec{\mathbf{x}}_i, \varepsilon_i\right) = 0$，$\mathrm{Cov}\left(\vec{\mathbf{y}}_i, \varepsilon_i\right) = 0$，$\mathrm{Cov}\left(\vec{\mathbf{b}}_i, \varepsilon_i\right) = 0$。本节以下各式沿用 t_i 这一记号：只压缩期望产出（$g_1^y = 1$）时 $t_i = y_{1i}$，只压缩非期望产出（$g_1^b = 1$、$\mathbf{g}^y = \mathbf{g}^x = \mathbf{0}$）时 $t_i = -b_{1i}$，此时因变量为**负的**非期望产出，10.6 节的案例即属于后者。

与单产出 StoNED 问题类似，第一步可以使用二次规划问题计算方向距离函数的超平面。需要注意的是 $E(\varepsilon_i) = E(u_i + v_i) = \mu \neq 0$，所以，需要先估计：

$$t_i = (\vec{D}_T\left(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \vec{\mathbf{b}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) - \mu) - (\varepsilon_i - \mu) = \vec{D}_{T2}\left(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \vec{\mathbf{b}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) - (\varepsilon_i - \mu) \quad \forall i = \vec{D}_{T2}\left(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \vec{\mathbf{b}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) \tag{10.43}$$

上式同样与单产出的StoNED模型（10.13）一致，因此，可以按照同样的方法计算超平面、无条件非效率 μ 和条件非效率 $E(u_i|\varepsilon_i)$。

$\vec{D}_{T2}$ 的估计值 $\hat{\vec{D}}_{T2}$ 的计算方法如下：

$$\begin{aligned} &\min\nolimits_{\alpha,\beta,\gamma,\varepsilon^{CNLS}}, \textstyle\sum_{i=1}^n \left(\varepsilon_i^{CNLS}\right)^2 \\ &\text{s.t.} \\ &t_i = \alpha_i + \beta_i' \vec{\mathbf{x}}_i - \gamma_i' \vec{\mathbf{y}}_i + \delta_i' \vec{\mathbf{b}}_i + \varepsilon_i^{CNLS} && \forall i \\ &\alpha_i + \beta_i' \vec{\mathbf{x}}_i - \gamma_i' \vec{\mathbf{y}}_i + \delta_i' \vec{\mathbf{b}}_i \leq \alpha_h + \beta_h' \vec{\mathbf{x}}_i - \gamma'_h \vec{\mathbf{y}}_i + \delta_h' \vec{\mathbf{b}}_i && \forall i, h \\ &\alpha_h + \beta_h' \vec{\mathbf{x}}_i \geq 0 && \forall i, h \\ &\beta_i' \mathbf{g}^x + \gamma_i' \mathbf{g}^y + \delta_i' \mathbf{g}^b = 1 && \forall i \\ &\beta_i \geq \mathbf{0}, \gamma_i \geq \mathbf{0}, \delta_i \geq \mathbf{0} && \forall i \end{aligned} \tag{10.44}$$

相比于多产出StoNED模型对应公式(10.33)，公式(10.44)中多出了第三个约束条件，这个约束条件对应非期望产出和期望产出的弱可处置性。这里也是" 减排因子" 进入模型的地方：`deabook` 中 `RTS_VRS1` 表示规模报酬可变且**相同减排因子**（所有 DMU 共享同一 θ），`RTS_VRS2` 表示规模报酬可变且**不同减排因子**（各 DMU 拥有独立的 θ_i）。二者的差别仅存在于含非期望产出的弱可处置技术中，与 10.1.2 节里 α_i 的符号约束（CRS/NIRS/NDRS）不是同一件事。具体地，公式 (10.44) 与 (10.51) 是 `deabook` 在 `RTS_VRS2` 设定下的完整约束结构（回归约束含 α_i、凹性约束含 $+\alpha_i/+\alpha_h$、并施加弱可处置性约束）。在 `RTS_CRS` 下取 $\alpha_i = 0$；在 `RTS_VRS1` 下，所有 DMU 共享相同减排因子，弱可处置性被方向归一化等式约束隐含满足，故回归方程不含 α_i、凹性约束以 $-\alpha_i/-\alpha_h$ 形式出现且不施加额外的弱可处置性不等式约束（详见 `CNLSSDFDDFweak.py` 中`__regression_rule`、`__afriat_rule`、`__reduction_factor_rule` 的实现）。此外，公式 (10.44) 与 (10.51) 第三约束（弱可处置性）理论上写作 $\alpha_h + \beta_h'\vec{\mathbf{x}}_i \geq 0\ \forall i,h$（约 n^2 个约束，$i=h$ 时跳过，对应 `CNLSSDweak` 的实现）；而 `CNLSDDFweak` 源码出于计算效率做了对角简化 $\alpha_i + \beta_i'\vec{\mathbf{x}}_i \geq 0\ \forall i$（$n$ 个约束），两者在样本内等价。

第二步与单产出模型相同，使用准似然估计或非参数核密度估计（参见 10.1.3 节）来估计企业特定条件非效率 $\hat{u}_i$。

第三步是从 $\vec{D}_{T2}$ 的估计值 $\hat{\vec{D}}_{T2}$ 中减去 μ 的估计值 $\hat{\mu}$。同样，CNLS 估计量在观察点 $\left(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \vec{\mathbf{b}}_i\right), \forall i = 1, \ldots, n$ 上是唯一的，但在未观察的 $\left(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \vec{\mathbf{b}}_i\right)$ 上不是。为了解决非唯一性问题，Kuosmanen and Kortelainen(2012)求助于最小外推原则，并提出估计 CNLS 估计量拟合值的最小包络。同样，对 $\hat{\vec{D}}^*_{T2}$ 的最小外推估计 $\hat{\vec{D}}^*_{T2,min}$ 由下式给出：

$$
\begin{aligned}
&\min\nolimits_{\alpha,\beta,\gamma,\delta}\ \alpha + \beta'\mathbf{x} + \delta'\mathbf{b} - \gamma'\mathbf{y} \\
&\text{s.t.} \\
&\alpha + \beta'\hat{\mathbf{x}}_i + \delta'\hat{\mathbf{b}}_i - \gamma'\hat{\mathbf{y}}_i \geq 0 \qquad \forall i \\
&\alpha + \beta'\hat{\mathbf{x}}_i \geq 0 \qquad \forall i \\
&\beta'\mathbf{g}^x + \delta'\mathbf{g}^b + \gamma'\mathbf{g}^y = 1 \\
&\beta \geq \mathbf{0}, \gamma \geq \mathbf{0}, \delta \geq \mathbf{0}
\end{aligned}
\tag{10.45}
$$

其中，$\hat{\mathbf{x}}_i = \mathbf{x_i} - (\hat{\mu} + \hat{\varepsilon}_i^{CNLS})\mathbf{g}^x$，$\hat{\mathbf{y}}_i = \mathbf{y_i} + (\hat{\mu} + \hat{\varepsilon}_i^{CNLS})\mathbf{g}^y$，$\hat{\mathbf{b}}_i = \mathbf{b_i} - (\hat{\mu} + \hat{\varepsilon}_i^{CNLS})\mathbf{g}^b$。

第四步是用 Stevenson 截断正态—正态残差分解结果恢复企业特定无效率和效率。令经方向符号处理后的CNLS 残差为 r_i^{DDF}，并转回原始复合误差尺度

$$e_i = r_i^{DDF} - \hat{m}_u.$$

令

$$\hat{\sigma}_*^2 = \frac{\hat{\sigma}_u^2\hat{\sigma}_v^2}{\hat{\sigma}_u^2 + \hat{\sigma}_v^2}, \qquad \hat{\mu}_{*i} = \frac{\hat{\mu}_0\hat{\sigma}_v^2 - e_i\hat{\sigma}_u^2}{\hat{\sigma}_u^2 + \hat{\sigma}_v^2}.$$

则 Stevenson-JLMS 条件非效率为

$$\hat{u}_i = E(u_i \mid e_i) = \hat{\mu}_{*i} + \hat{\sigma}_* \frac{\phi(\hat{\mu}_{*i}/\hat{\sigma}_*)}{\Phi(\hat{\mu}_{*i}/\hat{\sigma}_*)}. \tag{10.46}$$

注意，DDF 框架下，当方向作用于投入（g^x）或非期望产出（g^b）时，源码会先将 $\hat{\varepsilon}_i^{CNLS}$ 取负（对应式 (10.29) 处的符号约定）；当方向作用于期望产出（g^y）时不取负。再代入与(10.35) 同形的 Stevenson-JLMS 公

式(10.46)。得到非效率的条件期望值 $\hat{u}_i$ 后，可以使用方向基准变量与条件前沿值之间的比值得到条件效率。在 $\sum g^y \geq 1$ 时，$\frac{y_{1i}}{g_1^y}$ 为产出基准，条件前沿值为 $\frac{y_{1i}}{g_1^y} + \hat{u}_i$。因此，效率可以被计算为：

$$TE_i = \frac{\frac{y_{1i}}{g_1^y}}{\frac{y_{1i}}{g_1^y} + \hat{u}_i}. \tag{10.47}$$

当方向作用于投入或非期望产出时，效率公式与 (10.47) 同形，只是把产出基准 $\frac{y_{1i}}{g_1^y}$ 替换为对应方向的观测变量：投入方向用 $TE_i = (x_i - \hat{u}_i g^x)/x_i$，非期望产出方向用 $TE_i = (b_i - \hat{u}_i g^b)/b_i$。本章案例的 `gb=[1]` 属于非期望产出方向，效率由 $TE_i = (b_i - \hat{u}_i)/b_i$ 恢复，其中 b_i 为原始非期望产出观测值。

如果方向距离函数同时调整多个方向，例如 `gx=[1]`、`gy=[1]` 且 `gb=[1]`，则同一个方向距离 $\hat{u}_i$ 同时对应“压缩投入”“增加期望产出” 和“减少非期望产出” 的联合移动。此时不应只报告一个 y 或 b 口径的效率，而应同时报告每个方向分量上的非效率、投影值和效率比值。

对每个正方向分量，`deabook` 先把共同的方向距离 $\hat{u}_i$ 转成该变量上的调整量：

$$\Delta x_{ji} = \hat{u}_i g_j^x, \qquad \Delta y_{ki} = \hat{u}_i g_k^y, \qquad \Delta b_{li} = \hat{u}_i g_l^b.$$

因此，联合方向下的条件前沿投影为

$$x_{ji}^* = x_{ji} - \Delta x_{ji}, \qquad y_{ki}^* = y_{ki} + \Delta y_{ki}, \qquad b_{li}^* = b_{li} - \Delta b_{li}.$$

对应的分项效率比值可写为

$$TE_{ji}^x = \frac{x_{ji}^*}{x_{ji}} = \frac{x_{ji} - \hat{u}_i g_j^x}{x_{ji}}, \qquad TE_{ki}^y = \frac{y_{ki}}{y_{ki}^*} = \frac{y_{ki}}{y_{ki} + \hat{u}_i g_k^y}, \qquad TE_{li}^b = \frac{b_{li}^*}{b_{li}} = \frac{b_{li} - \hat{u}_i g_l^b}{b_{li}}.$$

多方向同时存在时，总效率取上述各方向分量效率的绑定（最小）值：

$$TE_i = \min\left\{TE_{ji}^x : g_j^x > 0;\ TE_{ki}^y : g_k^y > 0;\ TE_{li}^b : g_l^b > 0\right\}.$$

因此，DDF 模型在多方向同时存在时，总效率取各方向分量效率的最小值 $TE_i = \min\{TE_i^x, TE_i^y, TE_i^b\}$（即“绑定方向” 的效率）；若需逐方向查看，可分别计算投入方向效率 TE_i^x、期望产出方向效率 TE_i^y 和非期望产出方向效率 TE_i^b，以及对应的目标点 $(x_i - \hat{u}_i g^x,\ y_i + \hat{u}_i g^y,\ b_i - \hat{u}_i g^b)$。这样在 $g^x + g^y + g^b$ 同时非零时，即可分别查看投入压缩、期望产出扩张和非期望产出压缩三个口径下的效率与非效率。

10.3.1 有背景变量和非期望产出的StoNED模型

以第 10.3 节描述的非期望产出模型为起点，引入背景变量。用r维向量 $\mathbf{z}_i$ 表示背景变量的观测值，假设综合误差项为 $\varepsilon_i = v_i + u_i + \eta'\mathbf{z}$（沿用 10.3 节 DDF 框架 $\varepsilon_i = u_i + v_i$ 的符号约定），从而得到以下方程：

$$\vec{D}_T\left(\mathbf{x}_i, \mathbf{y}_i, \mathbf{b}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) = \varepsilon_i = v_i + u_i + \eta'\mathbf{z} \quad \forall i \tag{10.48}$$

同样由于所有投入和产出都受到非效率和噪声的影响，它们都是内生变量，在进行估计时需要解决这一问题。可以利用DDF的转换属性，估计：

$$\vec{D}_T\left(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \vec{\mathbf{b}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) - t_i = \varepsilon_i = v_i + u_i + \eta'\mathbf{z} \tag{10.49}$$

其中，平移基准 t_i 的定义与 10.3 节相同（取方向向量任一非零分量对应的变量），$\vec{\mathbf{x}}_i = \mathbf{x_i} + t_i\mathbf{g}^x$，$\vec{\mathbf{y}}_i = \mathbf{y_i} - t_i\mathbf{g}^y$，$\vec{\mathbf{b}}_i = \mathbf{b_i} + t_i\mathbf{g}^b$。可以证明，此时，$\mathrm{Cov}\left(\vec{\mathbf{x}}_i, \varepsilon_i\right) = 0$，$\mathrm{Cov}\left(\vec{\mathbf{y}}_i, \varepsilon_i\right) = 0$，$\mathrm{Cov}\left(\vec{\mathbf{b}}_i, \varepsilon_i\right) = 0$。

可以将 $\eta'\mathbf{z}$ 移到等式左边，并使用DDF的平移属性，可得：

$$\vec{D}_T\left(\vec{\mathbf{x}}_i, \vec{\mathbf{y}}_i, \vec{\mathbf{b}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) - t_i - \eta'\mathbf{z} = \vec{D}_T\left(\tilde{\vec{\mathbf{x}}}_i, \tilde{\vec{\mathbf{y}}}_i, \tilde{\vec{\mathbf{b}}}_i, \mathbf{g}_i^x, \mathbf{g}_i^y, \mathbf{g}_i^b\right) - t_i = v_i + u_i \quad \forall i \tag{10.50}$$

其中，$\tilde{\vec{\mathbf{x}}}_i = \vec{\mathbf{x}}_i - (\eta'\mathbf{z})\mathbf{g}^x$，$\tilde{\vec{\mathbf{y}}}_i = \vec{\mathbf{y}}_i + (\eta'\mathbf{z})\mathbf{g}^y$，$\tilde{\vec{\mathbf{b}}}_i = \vec{\mathbf{b}}_i - (\eta'\mathbf{z})\mathbf{g}^b$。

有背景变量和非期望产出的StoNED模型可以通过如下模型求解，

$$\begin{aligned}
&\min_{\alpha,\beta,\gamma,\varepsilon^{CNLS}}, \textstyle\sum_{i=1}^{n}\left(\varepsilon_i^{CNLS}\right)^2 \\
&\text{s.t.} \\
&t_i = \alpha_i + \beta_i'\vec{\mathbf{x}}_i - \gamma_i'\vec{\mathbf{y}}_i + \delta_i'\vec{\mathbf{b}}_i + \eta'\mathbf{z} + \varepsilon_i^{CNLS} && \forall i \\
&\alpha_i + \beta_i'\vec{\mathbf{x}}_i - \gamma_i'\vec{\mathbf{y}}_i + \delta_i'\vec{\mathbf{b}}_i \le \alpha_h + \beta_h'\vec{\mathbf{x}}_i - \gamma'_h\vec{\mathbf{y}}_i + \delta_h'\vec{\mathbf{b}}_i && \forall i, h \\
&\alpha_h + \beta_h'\vec{\mathbf{x}}_i \ge 0 \\
&\beta_i'\mathbf{g}^x + \gamma_i'\mathbf{g}^y + \delta_i'\mathbf{g}^b = 1 && \forall i \\
&\beta_i \ge \mathbf{0}, \gamma_i \ge \mathbf{0}, \delta_i \ge \mathbf{0} && \forall i
\end{aligned} \tag{10.51}$$

在得到 ε_i^{CNLS} 的估计值后，无条件非效率、条件非效率以及效率的值的估计都与第 10.2 节中的步骤一致。

有背景变量和非期望产出的 StoNED 模型可进一步扩展为meta-前沿（meta-frontier）设定：先用不含背景变量的 DDF 得到第一阶段方向距离、效率比值和残差，再在第二阶段的 CNLS-DDF 回归中加入背景变量项 $\eta'\mathbf{z}_i$。在StoNED 估计阶段，meta-DDF 分支返回包含技术差距比 TGR、组别技术效率 GTE 和共同前沿技术效率 MTE = TGR × GTE 的结果；其分项效率明细与普通 DDF 模型一致，并额外保留 meta 模型所需的共同前沿比值解释。

10.3.2 有非期望产出的凸非参数分位数回归

在第 10.1.6 节的凸非参数分位数回归和第 10.3 节的非期望产出框架基础上，进一步定义含非期望产出的凸非参数分位数回归模型，从而可以在数据内部分析投入的边际产出。通过求助于DDF的概率分位数特征，对含有低效率和噪音的随机数据进行建模：

$$\begin{aligned}
&\vec{D}_\tau\left(\mathbf{x}_i, \mathbf{y}_i, \mathbf{b}_i, \mathbf{g}^x, \mathbf{g}^y, \mathbf{g}^b\right) \\
&\quad = \sup\left\{\theta \mid \Pr\cdot\left[\left(\mathbf{x}_i - \theta\mathbf{g}^x, \mathbf{y}_i + \theta\mathbf{g}^y, \mathbf{b}_i - \theta\mathbf{g}^b\right) \in T\right] \ge 1 - \tau\right\}
\end{aligned} \tag{10.52}$$

其中参数 $\tau \in [0, 1]$。请注意，确定性DEA技术是通过设置 $\tau \to 0$ 得到的极限特例（此时要求 $\Pr[\cdot] \ge 1$ 为确定性约束）。对分位数DDF的解释直接类似于标准线性分位数回归（Koenker and Bassett, 1978）。本公式可以看作是Wang et al.(2014)在形状约束下提出的分位数前沿公式在一般多投入多产出设置中的延伸。

类比第 10.1.6 节的单产出的凸非参数分位数回归模型，可以定义如下有非期望产出的凸非参数分位

数回归模型(CNQR)的求解方法：

$$
\begin{aligned}
&\min_{\alpha,\beta,\gamma,\delta,\varepsilon^-,\varepsilon^+}(1-\tau)\sum_{i=1}^{n}\varepsilon_i^- + \tau\sum_{i=1}^{n}\varepsilon_i^+ \\
&\text{s.t.} \\
&\gamma_i'\mathbf{y}_i = \alpha_i + \beta_i'\mathbf{x}_i + \delta_i'\mathbf{b}_i - \varepsilon_i^- + \varepsilon_i^+ && \forall i \\
&\alpha_i + \beta_i'\mathbf{x}_i + \delta_i'\mathbf{b}_i - \gamma_i'\mathbf{y}_i \le \alpha_h + \beta_h'\mathbf{x}_i + \delta'_h\mathbf{b}_i - \gamma'_h\mathbf{y}_i && \forall i,h \\
&\beta_i'\mathbf{g}^x + \delta_i'\mathbf{g}^b + \gamma_i'\mathbf{g}^y = 1 && \forall i \\
&\beta_i \ge \mathbf{0}, \gamma_i \ge \mathbf{0}, \delta_i \ge \mathbf{0} && \forall i \\
&\varepsilon_i^- \ge 0, \varepsilon_i^+ \ge 0 && \forall i
\end{aligned}
\tag{10.53}
$$

与CNLS相比，CNQR的一个主要的缺点是该问题的最优解不一定是唯一的，甚至对于观察到的数据点 $(\mathbf{x}_i, y_i), i = 1, \ldots, n$ 也是如此。Kuosmanen and Johnson(2017)建议使用以下的目标函数的二次方表述来求得唯一解：

$$
\begin{aligned}
&\min_{\alpha,\beta,\gamma,\delta,\varepsilon^-,\varepsilon^+}(1-\tilde{\tau})\sum_{i=1}^{n}\left(\varepsilon_i^-\right)^2 + \tilde{\tau}\sum_{i=1}^{n}\left(\varepsilon_i^+\right)^2, \\
&\text{s.t.} \\
&\gamma_i'\mathbf{y}_i = \alpha_i + \beta_i'\mathbf{x}_i + \delta_i'\mathbf{b}_i - \varepsilon_i^- + \varepsilon_i^+ && \forall i \\
&\alpha_i + \beta_i'\mathbf{x}_i + \delta_i'\mathbf{b}_i - \gamma_i'\mathbf{y}_i \le \alpha_h + \beta_h'\mathbf{x}_i + \delta'_h\mathbf{b}_i - \gamma'_h\mathbf{y}_i && \forall i,h \\
&\beta_i'\mathbf{g}^x + \delta_i'\mathbf{g}^b + \gamma_i'\mathbf{g}^y = 1 && \forall i \\
&\beta_i \ge \mathbf{0}, \gamma_i \ge \mathbf{0}, \delta_i \ge \mathbf{0} && \forall i \\
&\varepsilon_i^- \ge 0, \varepsilon_i^+ \ge 0 && \forall i
\end{aligned}
\tag{10.54}
$$

同理，有非期望产出的凸非参数分位数回归可以添加背景变量，以分析背景变量的影响。

10.3.3 基于 Shephard 距离函数的弱可处置 StoNED 模型

前文的弱可处置 StoNED 模型均在方向距离函数（DDF）框架下展开。deabook 还提供基于Shephard 距离函数的弱可处置乘法实现（对应类 CNLSSDweak，应用见 10.6 节）。与 DDF 沿给定方向进行非径向调整不同，Shephard 型距离函数强调对期望产出的比例缩放。一个常见的 Shephard 产出距离函数可表示为：

$$
D_o(x, y, b) = \sup\{\theta > 0 \mid (x, y/\theta, b) \in T\}.
$$

CNLSSDweak 在弱可处置生产技术下估计 Shephard 型距离函数，并在乘法复合误差结构下通过 StoNED.get_technical_efficiency() 将条件无效率转换为近似径向的技术效率（参见 10.2 节式(10.36)的指数转换 $TE_i = \exp(-\hat{u}_i)$）。最终效率值通常位于 $(0, 1]$，越接近 1 表示越接近前沿。

在乘法误差结构下，其 CNLS 估计可概括为（此处公式不参与正文编号，仅说明 CNLSSDweak 的约束结构；与 10.1.4 节乘法模型(10.22) 同属乘法族）：

$$
\begin{aligned}
&\min \sum_{i=1}^{n}\left(\varepsilon_i^{CNLS}\right)^2 \\
&\text{s.t.} \\
&\varepsilon_i^{CNLS} - \ln(\text{base}_i) = \ln\left(\hat{D}_i + 1\right) \quad \forall i \\
&\hat{D}_i + 1 = \alpha_i + \beta_i' x_i - \gamma_i' y_i + \kappa_i' b_i \quad \forall i \\
&\alpha_h + \beta_h' x_i - \gamma_h' y_i + \kappa_h' b_i \le \hat{D}_h + 1 \quad \forall i,h \\
&\beta_i' g^x + \gamma_i' g^y + \kappa_i' g^b = 1 \quad \forall i \\
&\beta_i \ge 0,\ \gamma_i \ge 0,\ \kappa_i \ge 0 \quad \forall i.
\end{aligned}
$$

其中，base$_i$ 为由方向向量决定的基准变量（与 DDF 框架(10.41)–(10.43) 中的 t_i 同源，用于把内生投入产出转换为外生回归变量）。例如，当仅设定 $g_b \neq 0$ 时，基准变量来自非期望产出方向，即 base$_i = -b_{1i}/g_1^b$。使用 $\ln(\hat{D}_i + 1)$ 是为了避免对零取对数，与 10.1.4 节乘法模型中 $\ln(\phi_i + 1)$ 的处理一致。该模型的 StoNED 三步估计（CNLS 前沿、残差分解、效率恢复）与 10.3 节 DDF 框架完全平行，区别仅在于距离函数类型与误差结构。

10.4 计算问题

对于模型（10.3），在样本中增加一个新的DMU会使未知参数的数量增加m+2（m为投入的维度），而Afriat不等式约束的数量增加2n（n为DMU的个数）。引入一个额外的投入变量会使未知参数的数量增加n，但对约束的数量没有影响。由于这些原因，标准的QP算法不足以处理有几百或几千个观测值的大样本。可以使用基于切割平面的算法来求解模型（10.3）。下面解释该算法的各个步骤。

切割面算法是解决约束条件数量非常多的大规模优化问题的有效工具。在继续之前，定义一些在大规模优化文献中常用的术语。在这里，主问题指的是具有n(n-1)个约束条件的主问题（10.3），而减少的主问题指的是具有相同目标和变量、但约束条件只是主问题约束条件子集的问题。这些方法的主要思想是：从最初只有几个约束条件的问题，开始求解；然后，找到被违反的约束条件，并以延迟的方式迭代添加它们：在每次迭代中，解决一个减少的主问题（但约束条件逐渐多于最初的减少的主问题）。因此，这种方法在大规模优化文献中也被称为延迟约束生成（Bertsimas and Tsitsiklis, 1997）。这种方法的成功在很大程度上取决于寻找被违反的约束条件的效率。

从n(n-1)个约束中的一小部分开始（一个初始的减少的主问题）。通常情况下，只有n(n-1)个约束中的一小部分在主问题的最优解下是有效的，而且只用这些有效的约束来解决这个问题显然等同于解决完整问题。然而，这些有效的约束条件是事先不知道的。从一个“接近” 有效约束集的约束集开始，一个关键的好处是，它可以大大减少在以后的迭代中增加的切割数量，并减少净计算负担。

有两种常用的方法来形成初始的n-1个约束：基于观测值之间的欧氏距离的跨越路径（SP）和观测值之间的最小生成树（MST）。此外，还有Lee等(2013)提出的Sweet spot 方法。问题（10.3）的初始减少的主问题是：

$$
\begin{aligned}
&\min\nolimits_{\alpha,\beta,\varepsilon} \textstyle\sum_{i=1}^{n} \left(\varepsilon_i^{CNLS}\right)^2 \\
&\text{s.t.} \\
&\alpha_{i_1} + \beta'_{i_1} x_{i_2} \geq \alpha_{i_2} + \beta'_{i_2} x_{i_2} \\
&\alpha_{i_2} + \beta'_{i_2} x_{i_3} \geq \alpha_{i_3} + \beta'_{i_3} x_{i_3} \\
&\alpha_{i_{n-1}} + \beta'_{i_{n-1}} x_{i_n} \geq \alpha_{i_n} + \beta'_{i_n} x_{i_n} \\
&y_i = \alpha_i + \beta'_i \mathbf{x}_i + \varepsilon_i^{CNLS} \qquad \forall i \\
&\beta_i \geq \mathbf{0} \qquad \forall i
\end{aligned}
\tag{10.55}
$$

对于由 $\hat{\alpha}, \hat{\beta}, \hat{\varepsilon}$ 给出的减少的主问题（10.55）的解，需要检查这是否是主问题的可行解。如果它确实是可行的，那么显然它也是主问题的最优解。另一方面，如果它不可行，那么需要有效地找到一个被违反的约束条件。这个寻找违反约束的问题也被称为分离问题，因为这相当于找到一个超平面，将 $\hat{\alpha}, \hat{\beta}, \hat{\varepsilon}$ 从可行集合中分离出来（Bertsimas and Tsitsiklis, 1997）。因此，对于每一个i，第i个分离问题是找到最小的索引 $m(i)$，满足

$$
m(i) = \arg \min_{1 \leq m \leq n} \left\{ (\alpha_i + \beta'_i x_m) - (\alpha_m + \beta'_m x_m) < 0 \right\} \tag{10.56}
$$

此外，当且仅当最小值小于一个给定的阈值时，违反的约束才会被添加到减少的主问题中。

添加的分离问题对应的约束条件为：

$$\alpha_i + \beta_i' x_m \geq \alpha_m + \beta_m' x_m \quad \forall i$$

用 T_k 表示在第 k 次迭代时加入的被违反的约束的索引对。因此，在第k次迭代后，所解决的问题为：

$$\begin{aligned}
&\min_{\alpha,\beta,\varepsilon} \textstyle\sum_{i=1}^{n} \left(\varepsilon_i^{CNLS}\right)^2 \\
&\text{s.t.} \\
&\alpha_i + \beta_i' x_h \geq \alpha_h + \beta_h' x_h \quad \forall (i,h) \in T_0 \\
&\alpha_i + \beta_i' x_h \geq \alpha_h + \beta_h' x_h \quad \forall (i,h) \in T_1 \\
&\alpha_i + \beta_i' x_h \geq \alpha_h + \beta_h' x_h \quad \forall (i,h) \in T_k \\
&y_i = \alpha_i + \beta_i' \mathbf{x}_i + \varepsilon_i^{CNLS} \quad \forall i \\
&\beta_i \geq \mathbf{0} \quad \forall i
\end{aligned} \tag{10.57}$$

同样，在凸非参数分位数回归中，可以使用上述算法来求解大样本问题。需要做的是将目标函数对应，并改变相应的Afriat条件。问题（10.25）对应的减少的主问题为：

$$\begin{aligned}
&\min_{\alpha,\beta,\varepsilon^+,\varepsilon^-} \tau \textstyle\sum_{i=1}^{n} \varepsilon_i^+ + (1-\tau) \sum_{i=1}^{n} \varepsilon_i^- \\
&\text{s.t.} \\
&\alpha_{i_1} + \beta_{i_1}' x_{i_2} \geq \alpha_{i_2} + \beta_{i_2}' x_{i_2} \\
&\alpha_{i_2} + \beta_{i_2}' x_{i_3} \geq \alpha_{i_3} + \beta_{i_3}' x_{i_3} \\
&\alpha_{i_{n-1}} + \beta_{i_{n-1}}' x_{i_n} \geq \alpha_{i_n} + \beta_{i_n}' x_{i_n} \\
&y_i = \alpha_i + \beta_i' \mathbf{x}_i + \varepsilon_i^+ - \varepsilon_i^- \quad \forall i \\
&\beta_i \geq \mathbf{0} \quad \forall i \\
&\varepsilon_i^+, \varepsilon_i^- \geq 0 \quad \forall i
\end{aligned} \tag{10.58}$$

在第k次迭代后，所解决的问题为：

$$\begin{aligned}
&\min_{\alpha,\beta,\varepsilon^+,\varepsilon^-} \tau \textstyle\sum_{i=1}^{n} \varepsilon_i^+ + (1-\tau) \sum_{i=1}^{n} \varepsilon_i^- \\
&\text{s.t.} \\
&\alpha_i + \beta_i' x_h \geq \alpha_h + \beta_h' x_h \quad \forall (i,h) \in T_0 \\
&\alpha_i + \beta_i' x_h \geq \alpha_h + \beta_h' x_h \quad \forall (i,h) \in T_1 \\
&\alpha_i + \beta_i' x_h \geq \alpha_h + \beta_h' x_h \quad \forall (i,h) \in T_k \\
&y_i = \alpha_i + \beta_i' \mathbf{x}_i + \varepsilon_i^+ - \varepsilon_i^- \quad \forall i \\
&\beta_i \geq \mathbf{0} \quad \forall i \\
&\varepsilon_i^+, \varepsilon_i^- \geq 0 \quad \forall i
\end{aligned} \tag{10.59}$$

需要说明的是，切割面（延迟约束生成）算法主要面向数千观测以上的大样本场景，用于缓解Afriat 不等式约束数随 n^2 增长带来的求解压力。本章10.5–10.6 节的案例样本量 $n = 120$，由求解器（mosek/knitro）内置的约束生成机制已可高效处理，因此代码层不显式调用切割面算法。`deabook` 在 `CQERDFDDFweak` 模块中实现了含非期望产出的凸非参数分位数回归接口（`CQRweak` 为Shephard 型、`CQRDDFweak` 为方向距离函数型，对应 10.1.6 与 10.3.2 节的理论公式）；本章 10.6.3 与 10.6 节将给出其应用示例。若需大样本加速的约束生成版本，可参考 Lee, Johnson, Moreno-Centeno and Kuosmanen (2013) 的算法。

10.5 随机非参数前沿模型的Python实现

本章前述理论将 StoNED 问题表述为带 Afriat 凸性约束的约束优化形式：CNLS 对应式 (10.3) 的二次规划，凸非参数分位数回归对应式(10.25)、式 (10.53) 等线性或凸规划。deabook 沿用了这一数学结构，将实现组织为“先构造前沿、后分解残差” 的两阶段流程：第一阶段用 CNLSSDFDDFweak 构造并求解满足凸性、单调性和弱可处置性的 CNLS 前沿；第二阶段将 CNLS 残差传入 StoNED，在复合误差设定下完成随机噪声与技术无效率的分解。这一组织方式并非工程惯例，而是直接反映模型结构本身的两点经济—计算含义。其一，CNLS 前沿的求解器并非 Python 标准库组件：加性方向距离函数（DDF）对应凸二次规划，通常由 mosek 等求解；乘法 Shephard 距离函数对应非线性规划，通常需 knitro （或 amplpy）等外部/商业求解器，因此对象的构造（CNLSSDFDDFweak.CNLSDDFweak）与求解（model.optimize(solver="...")）被刻意分离，以便在不同求解环境下复用同一建模逻辑。其二，效率表需经“准备数据—设定方向—求解前沿—分解残差—解释效率” 的串行处理才能获得，而非由单一函数直接返回，这正对应StoNED 将偏离前沿的部分进一步识别为随机扰动与技术无效率的两步推断思想。

10.5.1 调用前的数据与模型设定

deabook 为两类距离函数分别提供了生产前沿类：CNLSSDFDDFweak.CNLSDDFweak 对应方向距离函数，其经济含义是沿研究者指定的方向（压缩非期望产出、调整投入或扩张期望产出）度量无效率；CNLSSDFDDFweak.CNLSSDweak 对应 Shephard 距离函数，采用乘法复合误差结构，通常与 StoNED.get_technical_efficiency() 配合以指数形式恢复径向技术效率。二者共享同一份数据与方向设定，差别只在于距离函数类型与误差结构所决定的效率解释（详见 10.5.3 节对照）。此外，CNLSSDFDDF.CNLSDDF 是不含非期望产出（即无非期望产出方向 g^b、无非期望产出系数 δ）的多产出 DDF 类，对应 10.2 节公式(10.33)；本章重点是含非期望产出的 CNLSDDFweak，其弱可处置设定对应式(10.44)。无论采用哪一类，调用前都需先将投入、期望产出与非期望产出汇入同一 DataFrame，并以 sent 字符串声明各变量的生产角色——这是后续所有方向向量与规模报酬设定得以对齐的前提。

```
from deabook import data_path, PANEL
from deabook.constant import GLOBAL
from pathlib import Path

import numpy as np
import pandas as pd
from deabook import CNLSSDFDDFweak, StoNED, PANEL
from deabook.constant import CONTEMPORARY, GLOBAL, SEQUENTIAL, WINDOW
from deabook.constant import CET_MULT, FUN_PROD, OPT_LOCAL, RED_KDE, RED_MOM, RED_QLE, RTS_CRS, RTS_VRS
```

下表 表 10.1 汇总了构造与求解这两类前沿时最具实质性的接口参数及其典型取值。其中 data 与 sent 规定了样本与变量角色，cet 与 solver 则共同决定模型的数学类型与所需求解器——CNLSDDFweak 默认加性误差（CET_ADDI）、由凸二次规划求解；CNLSSDweak 必须设为乘法误差（CET_MULT）、并由非线性求解器（通常为 knitro）求解。

表 10.1: 本章 StoNED/CNLS 调用的核心参数

参数	典型设定	含义
data	DataFrame	DMU 数据，需包含 sent 中出现的全部变量。
sent	"K L E=Y:CO2"	= 左侧为投入，右侧冒号前为期望产出，冒号后为非期望产出。
gy	[0] 或 [1]	期望产出方向向量；[0] 表示不沿期望产出方向调整。
gx	[0,0,0] 或 [1,0,0]	投入方向向量；元素顺序与 sent 中投入变量顺序一致。
gb	[0] 或 [1]	非期望产出方向向量；本章碳排放案例设为 [1]。
fun	FUN_PROD	前沿类型；本章使用生产前沿。
rts	RTS_CRS、RTS_VRS1、RTS_VRS2	规模报酬设定，分别表示规模报酬不变、规模报酬可变且减排因子相同、规模报酬可变且减排因子不同。
email	OPT_LOCAL 或邮箱地址	本地求解或远程求解通知设置。
solver	"mosek"、"knitro"	外部优化求解器名称；需在本地环境中安装并配置授权。
cet	CET_ADDI、CET_MULT	复合误差项类型。CNLSDDFweak 默认加性（CET_ADDI）；CNLSSDweak **必须**设为乘法 CET_MULT，且求解器通常用"knitro"（非线性）。

方向向量是 StoNED 设定中真正承载经济含义的部分，它把“无效率” 具体化为沿某一技术维度的改进空间，而非一个笼统的标量。以 sent="K L E=Y:CO2" 为例，gx 的三个元素分别对应资本、劳动与能源，gy 对应 GDP，gb 对应碳排放；若研究问题关注在既定投入与期望产出下压缩碳排放，便采用 gy=[0]、gx=[0,0,0]、gb=[1]。下文表 10.2 将方向、规模报酬与模型类型一并列出，以便理解每种组合所对应的经济识别对象。

表 10.2: 方向、规模报酬与模型类型设定

设定对象	可选设定	解释
期望产出方向	`gy=[0]`；`gy=[1]`	不扩张期望产出；或沿期望产出方向扩张。
投入方向	`gx=[0,0,0]`；`gx=[1,0,0]`	不压缩投入；或沿指定投入方向压缩。
非期望产出方向	`gb=[0]`；`gb=[1]`	不调整非期望产出；或沿非期望产出方向压缩。
规模报酬	`RTS_CRS`；`RTS_VRS1`；`RTS_VRS2`	规模报酬不变；规模报酬可变且减排因子相同；规模报酬可变且减排因子不同。
Shephard 距离函数	`CNLSSDweak(..., cet=CET_MULT)`	使用乘法复合误差项，求解器通常设为 `"knitro"`。
方向距离函数	`CNLSDDFweak(...)`	使用方向距离函数，求解器通常设为 `"mosek"`。

`"mosek"` 和 `"knitro"` 均属外部优化求解器，不在 Python 标准库中。运行示例前，应先确认求解器已安装、授权有效，并能被 Pyomo 调用；否则，模型可能在优化阶段报错。对于乘法复合误差模型，`CNLSSDweak` 通常需要可用的非线性求解器。

10.5.2 从 CNLS 前沿到 StoNED 效率

前沿求解与残差分解在 `deabook` 中分属两个对象，这一分离正是StoNED “先估计确定性前沿、再对残差做随机分解” 两步法的代码对应。CNLS 模型的 `optimize()` 方法只负责求解式 (10.3) 所示的前沿，结果保存在模型对象内部，并不直接给出可解释的效率表；只有将求解后的 `model` 传入 `StoNED`，才进入噪声与无效率的分解阶段。因此在实际分析中，应先核验 `optimization_status` 以及 Pyomo 返回的求解器状态，确认前沿已正常收敛，再开展残差分解。以下片段假定 `model` 已经按 10.5.1 节的方式构造完毕：

```
model.optimize(solver="mosek")
print(model.optimization_status)
print(model.problem_status.solver.status)
print(model.problem_status.solver.termination_condition)

rd = StoNED.StoNED(model)
res = rd.get_technical_efficiency(RED_QLE)
```

StoNED 残差分解与效率获取的完整方法链如 表 10.3 所示。

表 10.3: StoNED 残差分解与效率获取方法

步骤或方法	deabook 调用	作用
求解 CNLS 前沿	`model.optimize(solver="...")`	根据模型类型调用线性或非线性求解器，结果保存在模型对象内部。
查看求解状态	`model.optimization_status`、`model.problem_status.solver.status`、`model.problem_status.solver.termination_condition`	检查模型是否进入已优化状态、求解器是否正常终止。
创建 StoNED 对象	`StoNED.StoNED(model)`	读取 CNLS 残差并准备进行随机噪声与无效率分解。
准似然估计	`RED_QLE`	通过 Stevenson 截断正态准似然估计 μ_0、σ_u 和 σ_v；本章案例采用该方法。
矩估计	`RED_MOM`	采用 Stevenson 截断正态三矩匹配，估计潜在均值 μ_0、σ_u 和 σ_v；可进一步计算 JLMS 条件非效率。
核密度条件期望	`RED_KDE`	不再套用 Stevenson 截断正态解析JLMS；先基于单边残差初始化估计 $f_u(u)$ 与 $f_v(v)$，再通过数值积分计算 $\hat{u}_i^{KDE} = E(u_i \mid e_i)$。

步骤或方法	deabook 调用	作用
Shephard 效率	`get_technical_efficiency(method)`	用于 `CNLSSDweak`，返回 $TE_i = \exp(-\hat{u}_i) \in (0,1]$。
DDF 效率	`get_technical_efficiency(method)`	用于 `CNLSDDFweak`，返回方向归一化比率 $TE_i \in [0,1]$（如 gb 方向为 $(b_i - \hat{u}_i)/b_i$）。
DDF 效率恢复明细	`get_technical_efficiency_components(method)`	用于 `CNLSDDFweak`，返回残差、JLMS 条件无效率 $\hat{u}_i$、方向距离和TE 等精简列。

对于 CQER/CER 类，`get_technical_efficiency()` 不带 `method` 参数，因为它不是 StoNED 残差分解接口；`get_technical_efficiency_components()` 返回确定性前沿值、方向距离或分项效率，便于和 StoNED-JLMS 效率区分。

三种残差分解方法的对比。`RED_QLE`（准似然）、`RED_MOM`（矩估计）、`RED_KDE`（核密度）是从CNLS 残差中分解无效率和噪声的三条路径。`RED_QLE` 与 `RED_MOM` 均采用 Stevenson 截断正态—正态设定：前者最大化截断正态复合误差准似然（式(10.17)），后者匹配残差的二阶、三阶和四阶累积量，二者估计 μ_0、σ_u 和 σ_v 后使用 Stevenson-JLMS 条件期望公式恢复 $\hat{u}_i = E(u_i \mid e_i)$。`RED_KDE` 则作为非参数核密度路径：它不再用 Stevenson 截断正态解析JLMS，而是基于单边残差初始化构造 u 与 v 的样本，再用 KDE 估计非效率密度 $f_u(u)$ 与噪声密度 $f_v(v)$，最后通过数值积分计算 $E(u_i \mid e_i)$。因此，`RED_KDE` 的结果应理解为 KDE 条件均值恢复，而不是 Stevenson 参数估计结果。以下代码片段展示如何在同一CNLS 前沿上切换三种路径（以 `CNLSDDFweak` 为例）：

```
rd = StoNED.StoNED(model)  # 复用同一个已求解的 CNLS 前沿
te_qle = rd.get_technical_efficiency(RED_QLE)  # 准似然(本章主用)
te_mom = rd.get_technical_efficiency(RED_MOM)  # Stevenson 截断正态三矩匹配
te_kde = rd.get_technical_efficiency(RED_KDE)  # KDE 数值条件期望

# 简要对比三种方法的全样本均值
import numpy as np
print("MOM:", np.mean(te_mom), " QLE:", np.mean(te_qle), " KDE:", np.mean(
```

```
    te_kde))
```

若三种方法得到的效率排序高度一致（例如秩相关系数接近1），说明残差分解方法的选择对排序影响有限；若差异明显，则需结合样本规模、残差偏态、异方差和研究目的判断哪种路径更稳健。由于 RED_KDE 不施加 Stevenson 截断正态分布，它尤其适合作为非参数稳健性对照，但不应把它解释为 μ_0、σ_u、σ_v 的 Stevenson 参数估计。

RED_KDE 的企业特定无效率恢复公式为：令 e_i 表示中心化后的复合残差，满足 $e_i \approx v_i - u_i$，且 $u_i \geq 0$。用核密度估计得到 $\hat{f}_u(u)$ 与 $\hat{f}_v(v)$ 后，deabook 通过数值积分计算

$$\hat{u}_i^{KDE} = E(u_i \mid e_i) = \frac{\int_0^\infty u\, \hat{f}_v(e_i + u)\hat{f}_u(u)\, du}{\int_0^\infty \hat{f}_v(e_i + u)\hat{f}_u(u)\, du}.$$

这里的 $\hat{f}_u(u)$ 在 $u \geq 0$ 上由非负单边非效率初始样本和边界反射 KDE 估计，$\hat{f}_v(v)$ 用中心化噪声样本的 KDE 估计。得到 $\hat{u}_i^{KDE}$ 后，效率转换步骤与 RED_QLE、RED_MOM 完全一致：普通Shephard/乘法模型使用 $TE_i = \exp(-\hat{u}_i)$；DDF 模型则按激活方向用真实变量归一化，例如非期望产出方向为 $TE_i = (b_i - \hat{u}_i g^b)/b_i$。

本章案例的实际表现。在本章 n=120、gb=[1] 的设定下，由于三种残差分解方法的识别信息不同，效率水平的绝对值不宜跨方法直接比较，真正应考察的是**排序（rank）一致性**而非水平一致；本章后续表 10.5 至表 10.5 的结果均基于 RED_QLE。

此外，get_mean_of_inefficiency(RED_MOM) 和 get_mean_of_inefficiency(RED_QLE) 返回的是截断后无效率均值 $m_u = E(u_i)$，而潜在均值 μ_0 保存在 mu_latent 中。RED_KDE 的 get_mean_of_inefficiency(RED_KDE) 返回 KDE 初始非效率样本的均值，用于把 CNLS 残差从 $r_i \approx v_i - u_i + E(u_i)$ 中心化到 $e_i \approx v_i - u_i$，但它不对应 Stevenson 潜在均值 μ_0。这些无效率量都处于 DDF 回归因变量 basexyb 的量纲（gb=[1] 时 basexyb=-b_i，即 CO2 量级），不宜与 表 10.5–10.8 中的效率比值直接比较量级。

10.5.3 两类前沿的调用结构

前述参数与分解方法最终汇成两类可运行的建模结构。为突出核心调用逻辑，下面仅在规模报酬不变（CRS）下展示每类模型的最小骨架；三种规模报酬设定（CRS、VRS1、VRS2）在碳排放案例中的完整对照见 10.6 节（DDF）与 10.6 节（Shephard）。两种结构在经济含义上的本质差异，在于距离函数类型与误差结构所决定的效率解释，详见下文的对照说明。

CNLSSDweak（Shephard 距离函数，乘法误差，knitro）：

```
# 以 CRS 为例；改 rts=RTS_VRS1 / RTS_VRS2 即可切换规模报酬
model = CNLSSDFDDFweak.CNLSSDweak(
    data, sent="K L E=Y:CO2", gy=[0], gx=[0,0,0], gb=[1],
    cet=CET_MULT, rts=RTS_CRS, fun=FUN_PROD
)
model.optimize(solver="knitro")
rd = StoNED.StoNED(model)
res = rd.get_technical_efficiency(RED_QLE)
```

CNLSDDFweak（方向距离函数,加性误差，mosek）:

```
# 以 CRS 为例;改 rts=RTS_VRS1 / RTS_VRS2 即可切换规模报酬
model = CNLSSDFDDFweak.CNLSDDFweak(
    data, sent="K L E=Y:CO2", gy=[0], gx=[0,0,0], gb=[1],
    fun=FUN_PROD, rts=RTS_CRS
)
model.optimize(solver="mosek")
rd = StoNED.StoNED(model)
res = rd.get_technical_efficiency(RED_QLE)
```

两类前沿在距离函数类型、误差结构、方向向量用法、效率计算方法与结果解释上的系统差异,可归结为效率指标经济含义的不同,汇总于表 10.4。`CNLSSDweak` 经由指数转换得到的是更接近径向技术效率的测度,适用于需要与传统径向模型比较的场合；`CNLSDDFweak` 则把无效率显式解释为沿某一方向的减排、增产或投入压缩空间,更适合在环境效率研究中区分投入、期望产出与非期望产出的方向性调整。10.6 节（DDF）与 10.6 节（Shephard）将基于同一份碳排放数据对两类方法进行数值对照,以检验结论对距离函数设定的稳健性。

表 10.4: CNLSSDweak 与 CNLSDDFweak 的方法差异

特征	CNLSSDweak（Shephard 距离函数）	CNLSDDFweak（方向距离函数）
距离函数类型	径向、比例缩放	非径向、沿指定方向调整
误差结构	乘法模型,必须使用 `CET_MULT`	加性方向距离函数模型
方向向量	使用 `gy`、`gx`、`gb` 确定基准方向	使用 `gy`、`gx`、`gb` 确定调整方向
效率计算方法	`StoNED.get_technical_efficiency(RED_QLE)`	`StoNED.get_technical_efficiency(RED_QLE)`
结果解释	指数转换后的技术效率,通常在 $(0,1]$	方向归一化后的技术效率 $(b-\hat{u})/b$,在 $[0,1]$ 区间
适用场景	需要径向效率测度或与传统径向模型比较	需要明确区分投入、期望产出和非期望产出的方向性调整

10.6 案例应用:芬兰配电企业的随机成本前沿

Kuosmanen（2012）将StoNED用于芬兰配电监管,公开数据包含89家企业。现有electricityFirms.csv与pyStoNED公开文件逐项一致。本节以TOTEX为成本,以加权输配电量Energy、网络长度Length和客户数Customers为服务规模。Energy是服务产出,不是能源消耗投入。TOTEX不等于文件中OPEX与CAPEX的简单相加,保留原始口径。CSV没有企业年份和名称,本节仅按原始行号标识观测,不构造年度趋势。

```
from deabook.example.book_cases.run import FUN
FUN[10]()  # 重新计算本章,输出完整CSV、摘要表与PNG/PDF图
```

表 10.5: 芬兰配电企业的随机成本前沿：结果摘要

统计量	DEA成本效率	StoNED成本效率
有效样本数	89	89
均值	0.828	0.795
标准差	0.119	0.194
最小值	0.485	0.000
P10	0.662	0.601
中位数	0.836	0.843
P90	1.000	0.975
最大值	1.000	0.998

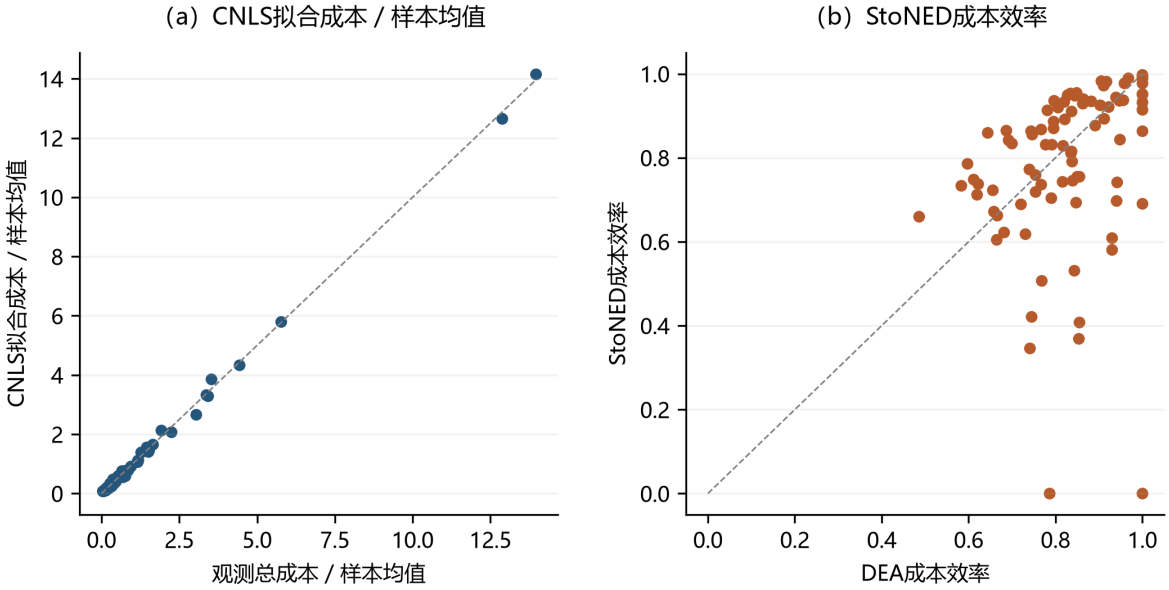


图 10.1: 芬兰89家配电企业的成本拟合与效率比较

表并列报告DEA与StoNED成本效率的中心位置和分布范围。图左检查CNLS拟合成本与观测成本的关系，图右逐企业比较两种效率估计。两种方法处理随机噪声的方式不同，均值差异不等于某一种方法必然更准确。

本例估计加法误差的凸、单调成本CNLS，再以正态噪声与半正态非效率的中心化似然分解残差。CNLS残差三阶偏度约0.194；估得σu=0.0684、σv=0.0738，单位均为样本平均TOTEX。用JLMS条件均值û计算加法成本效率(C−û)/C，有2个低成本观测得到负值，主图将其截到0，同时在完整结果保留û和残差。这说明共同绝对噪声尺度对小企业的适配有限；0值是模型诊断，不能解释为企业没有生产能力。DEA与StoNED成本效率均值分别为0.828和0.795，两者不是必然谁高谁低。本例是透明的半正态教学重估，不是芬兰监管公式或原文结果的完整复现，也不同于本章其他部分的Stevenson截断正态扩展。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch10_full.csv`。表中P10和P90为第10和第90百分位数（如适用），展示分布而不占用逐个决策单元的行数。

10.7 本章小结

本例估计加法误差的凸、单调成本CNLS，再以正态噪声与半正态非效率的中心化似然分解残差。CNLS残差三阶偏度约0.194；估得σu=0.0684、σv=0.0738，单位均为样本平均TOTEX。用JLMS条件均值û计算加法成本效率(C−û)/C，有2个低成本观测得到负值，主图将其截到0，同时在完整结果保留û和残差。这说明共同绝对噪声尺度对小企业的适配有限；0值是模型诊断，不能解释为企业没有生产能力。DEA与StoNED成本效率均值分别为0.828和0.795，两者不是必然谁高谁低。本例是透明的半正态教学重估，不是芬兰监管公式或原文结果的完整复现，也不同于本章其他部分的Stevenson截断正态扩展。

参考文献

AFRIAT S N. Efficiency estimation of production functions [J]. International Economic Review, 1972, 13(3): 568-598. DOI: 10.2307/2525845.

AIGNER D J, CHU S F. On estimating the industry production function [J]. The American Economic Review, 1968, 58(4): 826-839.

AIGNER D, LOVELL C A K, SCHMIDT P. Formulation and estimation of stochastic frontier production function models [J]. Journal of Econometrics, 1977, 6(1): 21-37. DOI: 10.1016/0304-4076(77)90052-5.

BERTSIMAS D, MUNDRU N. Sparse convex regression [J]. INFORMS Journal on Computing, 2021, 33(1): 262-279. DOI: 10.1287/ijoc.2020.0954.

BERTSIMAS D, TSITSIKLIS J N. Introduction to linear optimization [M]. Belmont: Athena Scientific, 1997.

BOYD S P, VANDENBERGHE L. Convex optimization [M]. Cambridge: Cambridge University Press, 2004.

CAUDILL S B, FORD J M. Biases in frontier estimation due to heteroscedasticity [J]. Economics Letters, 1993, 41(1): 17-20. DOI: 10.1016/0165-1765(93)90104-K.

CHAMBERS R G, CHUNG Y, FÄRE R. Benefit and distance functions [J]. Journal of Economic Theory, 1996, 70(2): 407-419. DOI: 10.1006/jeth.1996.0096.

CHAMBERS R G, CHUNG Y, FÄRE R. Profit, directional distance functions, and Nerlovian efficiency [J]. Journal of Optimization Theory and Applications, 1998, 98(2): 351-364. DOI: 10.1023/A:1022637501082.

CHARNES A, COOPER W W, RHODES E. Measuring the efficiency of decision making units [J]. European Journal of Operational Research, 1978, 2(6): 429-444. DOI: 10.1016/0377-2217(78)90138-8.

FAN Y, LI Q, WEERSINK A. Semiparametric estimation of stochastic production frontier models [J]. Journal of Business & Economic Statistics, 1996, 14(4): 460-468. DOI: 10.1080/07350015.1996.10524675.

FARRELL M J. The measurement of productive efficiency [J]. Journal of the Royal Statistical Society: Series A (General), 1957, 120(3): 253-290. DOI: 10.2307/2343100.

FÄRE R, PRIMONT D. Multi-output production and duality: Theory and applications [M]. Dordrecht: Springer, 1995. DOI: 10.1007/978-94-011-0651-1.

HALL P, SIMAR L. Estimating a changepoint, boundary, or frontier in the presence of observation error [J]. Journal of the American Statistical Association, 2002, 97(458): 523-534. DOI: 10.1198/016214502760047050.

JOHNSON A L, KUOSMANEN T. An introduction to CNLS and StoNED methods for efficiency analysis: Economic insights and computational aspects [M]//RAY S C, KUMBHAKAR S C, DUA P. Benchmarking for

performance evaluation: A production frontier approach. New Delhi: Springer, 2015: 117-186. DOI: 10.1007/978-81-322-2253-8_3.

JOHNSON A L, KUOSMANEN T. One-stage estimation of the effects of operational conditions and practices on productive performance: Asymptotically normal and efficient, root-n consistent StoNEZD method [J]. Journal of Productivity Analysis, 2011, 36(2): 219-230. DOI: 10.1007/s11123-011-0231-5.

JONDROW J, LOVELL C A K, MATEROV I S, SCHMIDT P. On the estimation of technical inefficiency in the stochastic frontier production function model [J]. Journal of Econometrics, 1982, 19(2-3): 233-238. DOI: 10.1016/0304-4076(82)90004-5.

KOENKER R, BASSETT G. Regression quantiles [J]. Econometrica, 1978, 46(1): 33-50. DOI: 10.2307/1913643.

KOENKER R. Quantile regression [M]. Cambridge: Cambridge University Press, 2005. DOI: 10.1017/CBO9780511754098.

KUMBHAKAR S C, LOVELL C A K. Stochastic frontier analysis [M]. Cambridge: Cambridge University Press, 2000. DOI: 10.1017/CBO9781139174411.

KUOSMANEN T, JOHNSON A L. Data envelopment analysis as nonparametric least-squares regression [J]. Operations Research, 2010, 58(1): 149-160. DOI: 10.1287/opre.1090.0722.

KUOSMANEN T, JOHNSON A L. Modeling joint production of multiple outputs in StoNED: Directional distance function approach [J]. European Journal of Operational Research, 2017, 262(2): 792-801. DOI: 10.1016/j.ejor.2017.04.014.

KUOSMANEN T, KORTELAINEN M. Stochastic non-smooth envelopment of data: Semi-parametric frontier estimation subject to shape constraints [J]. Journal of Productivity Analysis, 2012, 38(1): 11-28. DOI: 10.1007/s11123-010-0201-3.

KUOSMANEN T. Representation theorem for convex nonparametric least squares [J]. Econometrics Journal, 2008, 11(2): 308-325. DOI: 10.1111/j.1368-423X.2008.00239.x.

LEE C Y, JOHNSON A L, MORENO-CENTENO E, KUOSMANEN T. A more efficient algorithm for convex nonparametric least squares [J]. European Journal of Operational Research, 2013, 227(2): 391-400. DOI: 10.1016/j.ejor.2012.11.054.

RØDSETH K L. Axioms of a polluting technology: A materials balance approach [J]. Environmental and Resource Economics, 2017, 67(1): 1-22. DOI: 10.1007/s10640-015-9974-1.

RØDSETH K L. Environmental efficiency measurement and the materials balance condition reconsidered [J]. European Journal of Operational Research, 2016, 250(1): 342-346. DOI: 10.1016/j.ejor.2015.10.061.

STEVENSON R E. Likelihood functions for generalized stochastic frontier estimation [J]. Journal of Econometrics, 1980, 13(1): 57-66. DOI: 10.1016/0304-4076(80)90042-1.

WANG Y, WANG S, DANG C, GE W. Nonparametric quantile frontier estimation under shape restriction [J]. European Journal of Operational Research, 2014, 232(3): 671-678. DOI: 10.1016/j.ejor.2013.06.049.

第 11 章

DEA模型编程的基本思路

代码运行约定：在项目根目录运行，先安装 `python -m pip install numpy pandas scipy matplotlib pyomo highspy openpyxl`。各段Python 代码按本章顺序在同一会话执行，后文复用前文定义；默认采用HiGHS。公式讲解片段另标为 `text`，不作为独立程序执行。

在前面的章节中，我们介绍了多个用于效率评价的DEA模型。这些模型虽然形式各异，但基本结构相同，均由两部分组成：技术集和效率评价准则。在数学形式上，二者共同构成一个线性规划问题：效率评价准则体现为线性规划的目标函数，技术集体现为线性规划的约束条件。

第3章至第10章已系统建立了导向、规模报酬假设与技术集构造各异的一系列DEA模型，并论证了它们在生产理论上的含义。本章承接上述建模工作，进一步回答一个方法论问题：如何将形式化的DEA规划翻译为优化求解器可识别、并能在整个数据集中自动执行的计算机程序。具体而言，本章将阐明单个决策单元模型向Pyomo建模语言的映射逻辑、以决策单元为单位的循环求解结构，以及如何将这一流程封装为具有明确接口的Python函数。所建立的编程思路，将直接支撑第12章的实战演练。

11.1 DEA模型求解的程序化思路

根据DEA模型的结构特征与Python语言的表达能力，将一个数学形式的DEA模型转化为可运行的求解程序，在逻辑上可分解为三个相互衔接的环节。第一，将针对单个决策单元的模型翻译为Pyomo等建模语言能够识别的对象，即明确决策变量、目标函数与约束条件，并声明其索引集合。第二，将这一单单元求解过程在全部待评价单元的数据集上循环推进，从而完成整体效率评价。第三，将循环过程进一步封装为具有明确输入与输出的Python函数，以提升代码的复用性与可维护性。

本章的代码演进遵循” 迭代式教学” 策略：从基础版本开始，逐步增加功能，使读者在实践中理解函数封装、数据框操作、公式解析等核心编程技能。各版本的演进路线如下：

步骤1：单个DMU的CRS产出导向DEA（核心求解代码，11.2节）

↓ + 循环迭代

步骤2：所有DMU的CRS产出导向DEA（增加for循环，11.3节）

↓ + 函数封装

dea1：封装为函数（numpy数组输入，11.4.1节）

↓ + 数据框支持

dea2: 支持pandas DataFrame（增加数据框操作，11.4.2节）
↓ + 查询条件
dea3: 支持查询条件指定评价单元（11.4.3节）
↓ + 参照集选择
dea4: 支持分别指定评价单元和参照单元（11.4.4节）
↓ + 索引保留
dea5: 保留原始索引（解决索引错位问题，11.4.5节）
↓ + 公式解析
dea6: 支持公式字符串"Output=Labor Fuel Capital"（增加字符串解析，11.4.6节）
↓ + 规模报酬选择
dea7: 支持CRS/VRS切换（增加条件约束，11.4.7节）
↓ + 方向选择
dea8: 支持投入导向/产出导向（增加方向参数，11.4.8节）

图11.1:代码演进路线图

图11.1展示了本章代码的演进路径。每个新版本均在前一版本的基础上增加一项具体功能，这种渐进式的安排有助于读者理解每一行代码的作用与必要性。

11.2 单个决策单元规划问题的建模映射

在DEA模型中，效率评价的测度通常可转化为一个线性规划问题。Python的pyomo包可用于构建并求解线性规划问题，读者可访问 pyomo的文档查看说明文档和技术细节。下面以求解基于谢泼德产出导向距离函数的技术效率为例，进行详细说明。假定给定的数据集如表 11.1 所示。

表 11.1: 原文件前五家电厂（程序检查子样本）

电厂	Labor	Fuel	Capital	Output
Coffeen	179.0	3969.1	1005.5	3531.4
Grant Tower	96.0	896.32	194.6	841.3
Gudsonville	94.0	803.17	200.0	690.5
Meredosia	141.0	1629.9	564.1	1498.1
Newton	147.0	3264.4	617.4	3347.7

将规模报酬不变技术下的谢泼德产出导向距离函数表示为如下线性规划问题：

$$
\begin{aligned}
D(\mathbf{X}^*, \mathbf{Y}^*)^{-1} = \max_{\theta, \lambda_1, \lambda_2, \ldots, \lambda_5} \quad & \theta \\
s.t. \sum_{i=1}^{5} \lambda_i X_{ki} \leq X_k^*, k = 1, 2, 3 \\
\sum_{i=1}^{5} \lambda_i Y_{li} \geq \theta Y_l^*, l = 1 \\
\lambda_i \geq 0, i = 1, 2, \ldots, 5; -\infty \leq \theta \leq +\infty
\end{aligned}
$$

其中，$X^* \in R_+^3$ 和 $Y^* \in R_+^1$ 分别为被评价决策单元的投入和产出向量。

具体而言，以被评价决策单元为Coffeen、技术参照集为表中五家电厂为例，该规划向求解器输入的转化可按三个要素展开。首先是决策变量：需明确其维度与上下界。其次是目标函数：需明确优化的方向（极大或极小）以及各变量对应的系数。最后是约束条件：需明确其维度、各变量对应的系数以及约束符号（$\geq$、$\leq$ 与 $=$）。

就上述谢泼德产出导向距离函数而言，决策变量由 θ 与 λ_i 构成：θ 为一维且取值无约束，λ_i 为 1×5 的多维强度向量，满足 $\lambda_i \geq 0$。目标函数中 θ 的系数为 1，λ_i 的系数为 $\mathbf{0}_{1\times5}$，故整体取极大值。约束条件包含两个维度不同的部分：其一对应 3 个投入变量 X（x_1、x_2、x_3 的第1—5行），符号为 $\leq$，其中 θ 不出现；其二对应 1 个产出变量 Y（y_1 的第1—5行），符号为 $\geq$，θ 的系数为被评价单元A的产出（y_1 的第1行）。

在确定上述决策变量、约束和目标函数后，还需将它们转换为pyomo定义的建模语言，才能用代码实现优化模型。在转换之前，需要先介绍pyomo模块中另一个不可或缺的要素——**索引集合**。索引集合用于在定义变量和约束时声明其索引，从而可以通过索引调用变量和约束的具体内容。索引集合可采用如下方法定义：

```
import numpy as np
from pyomo.environ import *
model = ConcreteModel()
model.I = Set(initialize = range(5)) # 采用range初始化技术参照决策单元个数的集合
model.K = Set(initialize = range(3)) # 采用range初始化投入变量个数的集合
model.L = Set(initialize = range(1)) # 采用range初始化产出变量个数的集合
```

常用参数 `initialize`表示初始化参数，可以采用列表、range对象、元组和集合等可迭代对象来给一个 `Set`初始化。上面的代码使用 `range`对象进行初始化。可以通过 `data`属性查看索引集合的内容。

```
print(model.I.data())
print(model.K.data())
print(model.L.data())
```

11.2.1 决策变量的定义

在定义完索引后，使用 `Var`方法定义决策变量。常用的参数有四个：`index_set`表示决策变量的索引，这个索引可以是上面定义的索引集合；`bounds`表示决策变量的上下界；`within`表示决策变量的类型；`doc`为该决策变量取名。`bounds`传入一个元组，元组的第一个值表示变量的下界，第二个值表示变量的上界，且都包含等号，∞ 对应传入 `None`；`within`默认为实数，可以选择的类型有：

传入值	含义
`Reals`	实数
`PositiveReals`	正实数
`NonPositiveReals`	非正实数
`NegativeReals`	负实数
`NonNegativeReals`	非负实数
`PercentFraction`	0到1之间实数

传入值	含义
UnitInterval	0到1之间实数
Integers	整数
PositiveIntegers	正整数
NonPositiveIntegers	非正整数
NegativeIntegers	负整数
NonNegativeIntegers	非负整数
Boolean	布尔变量
Binary	0,1变量

由于DEA对应的规划问题为线性规划问题，且变量的范围为正实数或非负实数，所以 within一般不需要传入参数，只需要在 bounds中传入变量的上下界即可。决策变量是一维还是多维，对应 Var函数的两种调用方法：当决策变量是一维变量时，只需要传入一个索引变量；当决策变量是多维变量时，则需要根据决策变量的维度传入多个索引变量。比如，下面两行代码分别定义了一维变量和二维变量：

```
model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
model.intensity = Var(
    model.I, bounds=(0.0, None), doc='intensity variables'
)  # 强度变量(原lambda)
```

> **命名说明：** model.intensity是DEA模型中的**强度变量**（intensity variable），对应数学公式中的 λ_i。在Python中， lambda是关键字(用于定义匿名函数)，因此不能作为变量名。这里使用 intensity而非 lamda，因为 intensity是DEA文献中的标准术语，含义明确。

11.2.2 目标函数的构造

使用 Objective方法定义目标函数。常用参数有三个： rule表示目标函数的定义规则； sense表示最大化问题还是最小化问题， maximize代表极大化问题， minimize代表极小化问题； doc为该目标函数取名。下面的代码定义了上述DEA模型的目标函数。由于 λ_i 的系数为0，故可将其省略。

```
def obj_rule(model):
    return model.theta
    # return model.theta*1 + sum(model.intensity[i] * 0 for i in model.I)
model.obj = Objective(rule=obj_rule, sense=maximize, doc='objective function')
```

11.2.3 约束条件的定义

使用 Constraint方法定义约束条件。参数有三个： index_set表示约束的索引集合，可以是上面定义的索引集合； rule表示约束的定义规则，可以传入一个函数表达式，一般用来定义多维约束； expr表示约束表达式，一般用来定义一维约束。下面使用 rule定义上述DEA模型的约束，这是一个多维约束。此时需要以技术参照决策单元和被评价决策单元的投入产出变量作为决策变量的系数。

```
from deabook.example.book_cases.loaders import plants
data = plants().head(5)[["Labor", "Fuel", "Capital", "Output"]].to_numpy()
x = data[0, 0:3]      # 被评价电厂Coffeen的投入
y = data[0, 3:4]      # 被评价电厂Coffeen的产出
xref = data[:, 0:3]   # 所有参照决策单元的投入
yref = data[:, 3:4]   # 所有参照决策单元的产出

def input_rule(model, k):
    """投入约束:参照单元的加权投入不超过被评价单元的投入"""
    return (
        sum(model.intensity[i] * xref[i, k] for i in model.I)
        <= x[k]
    )

def output_rule(model, l):
    """产出约束:参照单元的加权产出不低于theta倍的被评价单元产出"""
    return (
        sum(model.intensity[i] * yref[i, l] for i in model.I)
        >= model.theta * y[l]
    )

model.input = Constraint(model.K, rule=input_rule, doc='input constraint')
model.output = Constraint(model.L, rule=output_rule, doc='output constraint')
```

下面的例子定义模型 model2中的一维约束。模型 model2中变量的维度为 4×5，第一个一维约束为 $x_{11} + 4x_{13} \geq 2$，第二个一维约束为 $3x_{34} + 5x_{45} = 10$。需要注意，Python中的下标从0开始。

```
model2 = ConcreteModel()
model2.I = Set(initialize = range(4))
model2.J = Set(initialize = range(5))
model2.x = Var(model2.I, model2.J, within = Reals)
model2.c1 = Constraint(
    expr=2.0 <= model2.x[0, 0] + 4 * model2.x[0, 2]
)      # 不等式约束 2 <= x[0,0] + 4*x[0,2]
model2.c2 = Constraint(
    expr=3 * model2.x[2, 3] + 5 * model2.x[3, 4] == 10.0
)    # 等式约束 3*x[2,3] + 5*x[3,4] = 10
```

可以使用 Constraint.Skip方法跳过使用某一个索引值对应的变量来组成约束。如下面定义的多维约束为 $\sum_{j,j\neq 1} x_{ij} \leq 2,\ i = 1,2,3,4$：

```
def c3_rule(model2, i):
    if i == 1:
        return Constraint.Skip
    return (
        sum([model2.x[i, j] for j in model2.J])
        <= 2
    )
model2.c3 = Constraint(model2.I, rule=c3_rule)
```

11.2.4 模型求解

Pyomo只能完成建模的任务，求解还需将Pyomo模型导入专业的优化求解器来实现，例如 glpk、ipopt、Cplex、SCIP、mosek等。如下代码就是调用 highs求解器来实现求解：

```
opt = SolverFactory('highs')       # 指定 HiGHS 作为求解器
solution = opt.solve(model)       # 调用求解器求解
```

另外，可以在 solve方法中传入 report_timing=True来查看模型运行的时间：

```
opt = SolverFactory('highs')
solution = opt.solve(model, report_timing=True)
```

本章采用开源线性规划求解器 HiGHS，可用 python -m pip install highspy 安装。运行时间取决于机器、模型规模与求解器版本，读者可用上面的计时代码在自己的环境测量。其他求解器须单独安装，并满足其许可证要求。

```
opt = SolverFactory('highs')
assert opt.available(exception_flag=False), "请安装 highspy"
solution = opt.solve(model)
```

求解器安装提示：关于各求解器的详细安装步骤，请参阅本书附录A.1.3 节（求解器安装详细指南）。下面给出一段代码，用于**验证求解器是否已正确安装：**

```
# 验证求解器是否可用
from pyomo.environ import SolverFactory
for solver in ['highs', 'glpk', 'mosek']:
    opt = SolverFactory(solver)
    print(f"{solver}: {' 可用' if opt.available(exception_flag=False) else ' 不可用'}")
```

11.2.5 优化结果的提取

```
theta = np.asarray(list(model.theta[:].value))          # 提取决策变量 theta（效率倒数）
intensity_vals = np.asarray(list(model.intensity[:].value))  # 提取强度变量 lambda
obj = value(model.obj)                                  # 提取目标函数值
```

```
print("optimum theta: \n {} ".format(theta))
print("optimum intensity: \n {} ".format(intensity_vals))
print("optimal objective: {}".format(obj))
```

图11.2:代码运行结果

求解一个决策单元的DEA线性规划问题的完整代码如下：

```
from pyomo.environ import *
import numpy as np

from deabook.example.book_cases.loaders import plants
data = plants().head(5)[["Labor", "Fuel", "Capital", "Output"]].to_numpy()

x = data[0, 0:3]       # 被评价单元A的投入
y = data[0, 3:4]       # 被评价单元A的产出
xref = data[:, 0:3]    # 参照单元投入
yref = data[:, 3:4]    # 参照单元产出

model = ConcreteModel()
model.I = Set(initialize=range(5))    # 技术参照决策单元集合
model.K = Set(initialize=range(3))    # 投入变量集合
model.L = Set(initialize=range(1))    # 产出变量集合

model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
model.intensity = Var(model.I, bounds=(0.0, None), doc='intensity variables')

def obj_rule(model):
    return model.theta
model.obj = Objective(rule=obj_rule, sense=maximize, doc='objective function')

def input_rule(model, k):
    return (
        sum(model.intensity[i] * xref[i, k] for i in model.I)
        <= x[k]
    )
def output_rule(model, l):
    return (
        sum(model.intensity[i] * yref[i, l] for i in model.I)
        >= model.theta * y[l]
    )
```

```
model.input = Constraint(model.K, rule=input_rule, doc='input constraint')
model.output = Constraint(model.L, rule=output_rule, doc='output constraint')

opt = SolverFactory('highs')
solution = opt.solve(model)

theta = np.asarray(list(model.theta[:].value))
intensity_vals = np.asarray(list(model.intensity[:].value))
obj = value(model.obj)
print("optimum theta: \n {} ".format(theta))
print("optimum intensity: \n {} ".format(intensity_vals))
print("optimal objective: {}".format(obj))
```

11.3 效率评价的循环迭代

在前面的步骤中，已经完成了决策单元A的技术效率求解，接下来将求解决策单元B的技术效率。决策单元B的投入和产出数据在数组的第二行，因此只要改变被评价决策单元的投入和产出数据（X和Y所在的行），即将步骤1代码中的第8-9行修改为 `x=data[1,0:3]`、`y=data[1,3:4]`，求解得到的就是第2个决策单元的技术效率。求解所有决策单元技术效率的基本思路，就是让步骤1的线性规划求解在所有决策单元上进行迭代，这可以用 `for`循环来实现。具体代码如下：

```
from pyomo.environ import *
import numpy as np

from deabook.example.book_cases.loaders import plants
data = plants().head(5)[["Labor", "Fuel", "Capital", "Output"]].to_numpy()

theta_results = np.empty(5)    # 定义 theta_results 用于存储计算结果（原thetalt）

for j in range(5):             # 在 data 的行上循环
    x = data[j, 0:3]
    y = data[j, 3:4]
    xref = data[:, 0:3]
    yref = data[:, 3:4]

    model = ConcreteModel()
    model.I = Set(initialize=range(5))
    model.K = Set(initialize=range(3))
    model.L = Set(initialize=range(1))
```

```
    model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
    model.intensity = Var(model.I, bounds=(0.0, None),
        doc='intensity variables')

    def obj_rule(model):
        return model.theta
    model.obj = Objective(rule=obj_rule, sense=maximize,
        doc='objective function')

    def input_rule(model, k):
        return (
            sum(model.intensity[i] * xref[i, k] for i in model.I)
            <= x[k]
        )
    def output_rule(model, l):
        return (
            sum(model.intensity[i] * yref[i, l] for i in model.I)
            >= model.theta * y[l]
        )

    model.input = Constraint(model.K, rule=input_rule, doc='input constraint')
    model.output = Constraint(model.L, rule=output_rule,
        doc='output constraint')

    opt = SolverFactory('highs')
    solution = opt.solve(model)

    theta = np.asarray(list(model.theta[:].value))
    intensity_vals = np.asarray(list(model.intensity[:].value))
    obj = value(model.obj)

    theta_results[j] = theta[0]

te = 1 / theta_results
print(te)
```

图11.3：代码运行结果

命名说明：`theta_results`存储每个决策单元的 θ 值（效率的倒数），`te`（technical efficiency）为最终的技术效率值，两者关系为 $\text{te} = 1/\theta$。

在上述代码中，第8行提前定义了一个变量 `theta_results`（其元素值未初始化，为内存中的残留值，

仅用于预先分配存储空间），用以存储循环计算得到的结果。这部分代码与步骤1中的核心计算代码完全相同，只是增加了 for循环语句。运行之后， te变量就存储了5个决策单元的技术效率值。从计算逻辑上看，DEA的效率评价本质上是对每个决策单元求解一个以其余单元为参照集的规划问题，因而以决策单元为单位的循环迭代并非权宜之计，而是DEA方法论的内在要求：每个单元既是被评价对象，又作为参照集的组成部分参与其余单元前沿的构造。

11.4 封装为可复用函数

将以上代码写成函数，可以提高代码应用的灵活性，实现代码的重复使用。

11.4.1 dea1：基础函数

```
from pyomo.environ import *
import numpy as np

def dea1(data, dataref, numk):
    """
    计算DEA技术效率（基础版本）

    Parameters
    ----------
    data : np.ndarray
        待评价决策单元的投入产出数据
    dataref : np.ndarray
        参照决策单元的投入产出数据
    numk : int
        投入变量的个数

    Returns
    -------
    np.ndarray
        各决策单元的技术效率值（te）
    """
    n = data.shape[0]                    # 待评价决策单元个数
    theta_results = np.empty(n)          # 存储theta值的数组

    for j in range(n):                   # 在每个决策单元上循环
        x = data[j, 0:numk]
        y = data[j, numk:]
        xref = dataref[:, 0:numk]
```

```
        yref = dataref[:, numk:]

        model = ConcreteModel()
        model.I = Set(initialize=range(dataref.shape[0]))
        model.K = Set(initialize=range(numk))
        model.L = Set(initialize=range(data.shape[1] - numk))

        model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
        model.intensity = Var(model.I, bounds=(0.0, None),
            doc='intensity variables')

        def obj_rule(model):
            return model.theta
        model.obj = Objective(rule=obj_rule, sense=maximize,
            doc='objective function')

        def input_rule(model, k):
            return (
                sum(model.intensity[i] * xref[i, k] for i in model.I)
                <= x[k]
            )
        def output_rule(model, l):
            return (
                sum(model.intensity[i] * yref[i, l] for i in model.I)
                >= model.theta * y[l]
            )

        model.input = Constraint(model.K, rule=input_rule,
            doc='input constraint')
        model.output = Constraint(model.L, rule=output_rule,
            doc='output constraint')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        theta = np.asarray(list(model.theta[:].value))
        theta_results[j] = theta[0]

    te = 1.0 / theta_results
```

```
    return te
```

对以上代码简要解释如下：

- `def dea1(data, dataref, numk)` 定义了函数，三个参数分别是被评价单元的投入产出数据、参照单元的投入产出数据、投入变量的个数。
- `np.empty(n)` 定义一个空数组，`n` 使用 `data.shape[0]` 确定（shape属性可获得数据的行数和列数，`[0]`表示行数）。
- 循环中对每个被评价决策单元求解线性规划，最后返回技术效率值数组 `te`。

将这一过程封装为函数，使效率评价从一次性的脚本操作转变为可重复调用的计算接口：使用者只需提供数据与变量结构，便可获得标准化的效率结果，而无须重复书写建模与求解的细节。

仍以 表 11.1 所示数据为例，`dea1` 的调用与效率提取过程如下：

```
from deabook.example.book_cases.loaders import plants
data = plants().head(5)[["Labor", "Fuel", "Capital", "Output"]].to_numpy()
dataref = data
numk = 3
te = dea1(data, dataref, numk)
print(te)
```

图11.4：代码运行结果

11.4.2 dea2：支持pandas DataFrame

`dea1`函数使用numpy数组传入数据，无法保留变量名称。接下来使用pandas的 **DataFrame**（数据框）传入投入产出数据，并用数据框的变量名确定投入产出变量。下面以美国电厂数据（见图11.5）为例进行说明。该数据集包含19家电厂，三个投入为Labor、Fuel、Capital，产出为Output。

图11.5：美国电厂数据

```
from pyomo.environ import *
import pandas as pd
from deabook.example.book_cases.loaders import plants
import numpy as np

def dea2(dataframe, varname, numk):
    """
    计算DEA技术效率（支持DataFrame版本）

    Parameters
    ----------
    dataframe : pd.DataFrame
        待评价决策单元的投入产出数据框
    varname : list[str]
        投入和产出变量的列表，如["K","L","Y"]
```

```
    numk : int
        前numk个变量为投入数据

    Returns
    -------
    pd.DataFrame
        包含theta值和技术效率值的数据框
    """
    theta_results = {}           # 使用字典存储结果(保留原始索引)
    data = dataframe.loc[:, varname]
    dataref = dataframe.loc[:, varname]

    for j in range(data.shape[0]):
        x = data.iloc[j, 0:numk]
        y = data.iloc[j, numk:]
        xref = dataref.iloc[:, 0:numk]
        yref = dataref.iloc[:, numk:]
        xcol = xref.columns
        ycol = yref.columns

        model = ConcreteModel()
        model.I = Set(initialize=range(dataref.shape[0]))
        model.K = Set(initialize=range(numk))
        model.L = Set(initialize=range(len(ycol)))

        model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
        model.intensity = Var(model.I, bounds=(0.0, None),
            doc='intensity variables')

        def obj_rule(model):
            return model.theta
        model.obj = Objective(rule=obj_rule, sense=maximize,
            doc='objective function')

        def input_rule(model, k):
            return (
                sum(model.intensity[i] * xref.iloc[i, k] for i in model.I)
                <= x.iloc[k]
            )
```

```
        def output_rule(model, l):
            return (
                sum(model.intensity[i] * yref.iloc[i, l] for i in model.I)
                >= model.theta * y.iloc[l]
            )

        model.input = Constraint(model.K, rule=input_rule,
            doc='input constraint')
        model.output = Constraint(model.L, rule=output_rule,
            doc='output constraint')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        theta = np.asarray(list(model.theta[:].value))
        theta_results[j] = theta[0]

    thetadf = pd.DataFrame(theta_results, index=["theta"]).T
    thetadf["te"] = 1 / thetadf["theta"]
    return thetadf
```

dea2()函数与 dea1()函数的主要区别：

1. 使用数据框的 loc方法取出投入产出数据，使用 iloc方法索引行和列。
2. 约束条件中对 xref和 yref的调用使用**位置下标**（xref.iloc[i, k]）。这里 model.I 初始化为 range(dataref.shape[0])，是行的位置而不是行标签，因此第二维也必须用整数位置 k。若写成 xref.iloc[i, xcol[k]]，由于 xcol[k] 是列名字符串（如'K'），iloc 的第二维不接受字符串，运行时会抛出 TypeError；需要用列名时应改用标签索引 xref.loc[i, xcol[k]]（见 dea4 起的写法，此时 model.I 也须改为行标签）。
3. 使用字典存储结果，再转换为DataFrame，保留了数据框的操作特性。

以 美国电厂数据 为例，dea2 的调用方式如下：

```
import pandas as pd
from deabook.example.book_cases.loaders import plants
ex3 = plants()
te = dea2(ex3, ["Labor", "Fuel", "Capital", "Output"], 3)
print(te)
```

图11.6：代码运行结果

11.4.3 dea3：支持查询条件指定评价单元

通过数据框的.query()方法来指定待评价的决策单元：

```
def dea3(dataframe, varname, numk, eval_query):
    """
    计算DEA技术效率(支持查询条件指定评价单元)

    Parameters
    ----------
    eval_query : str or None
        传入数据框.query()方法的参数,如"dmu==1","dmu in [1,2,3]"
        若为None,则评价所有决策单元
    """
    theta_results = {}

    if eval_query is None:
        index_list = dataframe.index
    else:
        index_list = dataframe.query(eval_query).index

    data = dataframe.loc[index_list, varname]
    dataref = dataframe.loc[index_list, varname]

    for j in range(data.shape[0]):
        x = data.iloc[j, 0:numk]
        y = data.iloc[j, numk:]
        xref = dataref.iloc[:, 0:numk]
        yref = dataref.iloc[:, numk:]
        xcol = xref.columns
        ycol = yref.columns

        model = ConcreteModel()
        model.I = Set(initialize=range(dataref.shape[0]))
        model.K = Set(initialize=range(numk))
        model.L = Set(initialize=range(data.shape[1] - numk))

        model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
        model.intensity = Var(model.I, bounds=(0.0, None),
            doc='intensity variables')

        def obj_rule(model):
            return model.theta
```

```
        model.obj = Objective(rule=obj_rule, sense=maximize,
            doc='objective function')

        def input_rule(model, k):
            return (
                sum(model.intensity[i] * xref.iloc[i, k] for i in model.I)
                <= x.iloc[k]
            )
        def output_rule(model, l):
            return (
                sum(model.intensity[i] * yref.iloc[i, l] for i in model.I)
                >= model.theta * y.iloc[l]
            )

        model.input = Constraint(model.K, rule=input_rule,
            doc='input constraint')
        model.output = Constraint(model.L, rule=output_rule,
            doc='output constraint')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        theta = np.asarray(list(model.theta[:].value))
        theta_results[j] = theta[0]

    thetadf = pd.DataFrame(theta_results, index=["theta"]).T
    thetadf["te"] = 1 / thetadf["theta"]
    return thetadf
```

命名说明：`eval_query`（evaluation query）表示用于筛选待评价决策单元的查询条件字符串。使用下划线分隔的命名方式（snake_case）是Python的PEP 8标准命名规范。

该函数的典型调用方式如下：

```
import pandas as pd
from deabook.example.book_cases.loaders import plants
ex3 = plants()
te = dea3(ex3, ["Labor", "Fuel", "Capital", "Output"], 3, "dmu in [1,2,3,4,5,6,7,8,9,10]")
print(te)
```

其他 `eval_query`的用法：

```
te = dea3(ex3, ["Labor", "Fuel", "Capital", "Output"], 3, "dmu<=10")     # 评价前10个DMU
te = dea3(ex3, ["Labor", "Fuel", "Capital", "Output"], 3, None)          # 评价所有DMU
```

名单筛选用 `in`，不要用 `<=` 接列表。`DataFrame.query` 里的`<=`、`<`、`>=`、`>` 只做逐元素比较：右侧写成列表时，pandas 会把它当成一列长度为列表长度的数据去和整列对齐，长度不等就抛 `ValueError: ('Lengths must match to compare', (20,), (10,))`；长度恰好相等时不报错，但比较的是" 第1行的dmu ≤ 1、第2行的 dmu ≤ 2……"，与" 取出这10个DMU" 的本意毫无关系。按名单筛选必须用集合运算 `dmu in [1,2,3,4,5,6,7,8,9,10]`（`==` 右侧接列表时pandas会自动等价成 `in`/`isin`，也可用，但 `in` 的语义更直白）。上例这10个 DMU 的编号连续，因此也可以直接写标量比较 `dmu<=10`。

11.4.4 dea4：支持分别指定评价单元和参照单元

```
def dea4(dataframe, varname, numk, eval_query, ref_query):
    """
    计算DEA技术效率（支持分别指定评价单元和参照单元）

    Parameters
    ----------
    eval_query : str or None
        筛选待评价决策单元的查询条件
    ref_query : str or None
        筛选参照决策单元的查询条件
    """
    theta_results = {}

    if eval_query is None:
        index_list = dataframe.index
    else:
        index_list = dataframe.query(eval_query).index

    if ref_query is None:
        index_list_ref = dataframe.index
    else:
        index_list_ref = dataframe.query(ref_query).index

    data = dataframe.loc[index_list, varname]
    dataref = dataframe.loc[index_list_ref, varname]

    for j in range(data.shape[0]):
```

```
        x = data.iloc[j, 0:numk]
        y = data.iloc[j, numk:]
        xref = dataref.iloc[:, 0:numk]
        yref = dataref.iloc[:, numk:]
        xcol = xref.columns
        ycol = yref.columns

        model = ConcreteModel()
        model.I = Set(initialize=dataref.index)    # 使用参照单元的原始索引
        model.K = Set(initialize=range(numk))
        model.L = Set(initialize=range(data.shape[1] - numk))

        model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
        model.intensity = Var(model.I, bounds=(0.0, None),
            doc='intensity variables')

        def obj_rule(model):
            return model.theta
        model.obj = Objective(rule=obj_rule, sense=maximize,
            doc='objective function')

        def input_rule(model, k):
            return (
                sum(model.intensity[i] * xref.loc[i, xcol[k]] for i in model.I)
                <= x.iloc[k]
            )
        def output_rule(model, l):
            return (
                sum(model.intensity[i] * yref.loc[i, ycol[l]] for i in model.I)
                >= model.theta * y.iloc[l]
            )

        model.input = Constraint(model.K, rule=input_rule,
            doc='input constraint')
        model.output = Constraint(model.L, rule=output_rule,
            doc='output constraint')

        opt = SolverFactory('highs')
        solution = opt.solve(model)
```

```
        theta = np.asarray(list(model.theta[:].value))
        theta_results[j] = theta[0]

    thetadf = pd.DataFrame(theta_results, index=["theta"]).T
    thetadf["te"] = 1 / thetadf["theta"]
    return thetadf
```

该函数的典型调用方式如下：

```
import pandas as pd
from deabook.example.book_cases.loaders import plants
ex3 = plants()
te = dea4(ex3, ["Labor", "Fuel", "Capital", "Output"], 3, "dmu<=10", "dmu<=20")
print(te)
```

图11.7：代码运行结果

其他调用方式：

```
te = dea4(ex3, ["Labor", "Fuel", "Capital", "Output"], 3, "dmu==[10,11,2,3,4,5,6,18]", "dmu<=20")
```

11.4.5 dea5：保留原始索引

从dea4的结果中可以发现，即使选择了非连续的DMU（如第10、11、2、3、4、5、6、18个），得到的te的索引却是0-7。我们期望得到和原始数据框中相同的索引，以便将优化结果与原始数据合并。这需要对dea4进行修改：

```
def dea5(dataframe, varname, numk, eval_query, ref_query):
    """
    计算DEA技术效率(保留原始索引版本)
    """
    theta_results = {}

    if eval_query is None:
        index_list = dataframe.index
    else:
        index_list = dataframe.query(eval_query).index

    if ref_query is None:
        index_list_ref = dataframe.index
    else:
        index_list_ref = dataframe.query(ref_query).index

    xcol = varname[0:numk]
```

```
    ycol = varname[numk:]

    data = dataframe.loc[index_list, varname]
    dataref = dataframe.loc[index_list_ref, varname]
    xref = dataref.loc[:, xcol]
    yref = dataref.loc[:, ycol]

    for j in data.index:                    # 在data的原始索引上循环(关键改动)
        x = data.loc[j, xcol]               # 使用loc而非iloc
        y = data.loc[j, ycol]

        model = ConcreteModel()
        model.I = Set(initialize=dataref.index)
        model.K = Set(initialize=range(numk))
        model.L = Set(initialize=range(len(ycol)))

        model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
        model.intensity = Var(model.I, bounds=(0.0, None),
            doc='intensity variables')

        def obj_rule(model):
            return model.theta
        model.obj = Objective(rule=obj_rule, sense=maximize,
            doc='objective function')

        def input_rule(model, k):
            return (
                sum(model.intensity[i] * xref.loc[i, xcol[k]] for i in model.I)
                <= x.loc[xcol[k]]
            )
        def output_rule(model, l):
            return (
                sum(model.intensity[i] * yref.loc[i, ycol[l]] for i in model.I)
                >= model.theta * y.loc[ycol[l]]
            )

        model.input = Constraint(model.K, rule=input_rule,
            doc='input constraint')
        model.output = Constraint(model.L, rule=output_rule,
```

```
            doc='output constraint')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        theta = np.asarray(list(model.theta[:].value))
        theta_results[j] = theta[0]             # j为原始索引值

    thetadf = pd.DataFrame(theta_results, index=["theta"]).T
    thetadf["te"] = 1 / thetadf["theta"]
    return thetadf
```

关键改动说明：

1. 循环变量从 `range(data.shape[0])`改为 `data.index`，使 `j`取原始索引值
2. 数据切片从 `iloc`（位置索引）改为 `loc`（标签索引）
3. `xcol`和 `ycol`提前在循环外通过 `varname`切片获得

该函数的典型调用方式如下：

```
import pandas as pd
from deabook.example.book_cases.loaders import plants
ex3 = plants()
te = dea5(ex3, ["Labor", "Fuel", "Capital", "Output"], 3, "dmu==[10,11,2,3,4,5,6,18]", "dmu<=20")
print(te)
```

图11.8：代码运行结果

此时，`te`的索引与电厂数据中的原始索引一致，可以通过索引将二者直接合并。

11.4.6 dea6：支持公式字符串

`dea5`通过 `varname`和 `numk`传递投入产出变量。接下来优化输入方式，使用类似R语言的 **formula表达式格式**来传递变量，例如`"Output=Labor Fuel Capital"`。在这个表达式中，以`=`作为分隔符，等式左边是产出变量，等式右边是投入变量。

```
import re

s = "Y = K     L  "
input_vars = s.split('=')[1].strip(' ')
input_vars = re.compile(' +').sub(' ', input_vars).split(' ')
output_vars = s.split('=')[0].strip(' ')
output_vars = re.compile(' +').sub(' ', output_vars).split(' ')
```

第3行代码中的 `split("=")`将 `s`切分成列表 `["Y ", " K     L  "]`（注意首元素带有尾随空格），`[1]`选择第二个元素；`strip(' ')`去掉两端空格，`re.sub`将多个空格替换为一个空格，最后用 `split(' ')`得到列表 `['K', 'L']`。

```
from pyomo.environ import *
import pandas as pd
from deabook.example.book_cases.loaders import plants
import numpy as np
import re

def dea6(formula, dataframe, eval_query, ref_query):
    """
    计算DEA技术效率(支持公式字符串版本)

    Parameters
    ----------
    formula : str
        公式字符串,格式为 "产出 = 投入1 投入2 ..."
        例如: "Output=Labor Fuel Capital"、"Y = K     L"
    """
    theta_results = {}

    if eval_query is None:
        index_list = dataframe.index
    else:
        index_list = dataframe.query(eval_query).index

    if ref_query is None:
        index_list_ref = dataframe.index
    else:
        index_list_ref = dataframe.query(ref_query).index

    # 解析公式字符串
    input_vars = formula.split('=')[1].strip(' ')
    xcol = re.compile(' +').sub(' ', input_vars).split(' ')
    output_vars = formula.split('=')[0].strip(' ')
    ycol = re.compile(' +').sub(' ', output_vars).split(' ')

    data = dataframe.loc[index_list, xcol + ycol]
    dataref = dataframe.loc[index_list_ref, xcol + ycol]
    xref = dataref.loc[:, xcol]
    yref = dataref.loc[:, ycol]
```

```
    for j in data.index:
        x = data.loc[j, xcol]
        y = data.loc[j, ycol]

        model = ConcreteModel()
        model.I = Set(initialize=dataref.index)
        model.K = Set(initialize=range(len(xcol)))
        model.L = Set(initialize=range(len(ycol)))

        model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
        model.intensity = Var(model.I, bounds=(0.0, None),
            doc='intensity variables')

        def obj_rule(model):
            return model.theta
        model.obj = Objective(rule=obj_rule, sense=maximize,
            doc='objective function')

        def input_rule(model, k):
            return (
                sum(model.intensity[i] * xref.loc[i, xcol[k]] for i in model.I)
                <= x.loc[xcol[k]]
            )
        def output_rule(model, l):
            return (
                sum(model.intensity[i] * yref.loc[i, ycol[l]] for i in model.I)
                >= model.theta * y.loc[ycol[l]]
            )

        model.input = Constraint(model.K, rule=input_rule,
            doc='input constraint')
        model.output = Constraint(model.L, rule=output_rule,
            doc='output constraint')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        theta = np.asarray(list(model.theta[:].value))
        theta_results[j] = theta[0]
```

```
    thetadf = pd.DataFrame(theta_results, index=["theta"]).T
    thetadf["te"] = 1 / thetadf["theta"]
    return thetadf
```

该函数的典型调用方式如下：

```
import pandas as pd
from deabook.example.book_cases.loaders import plants
ex3 = plants()
te = dea6("Output=Labor Fuel Capital", ex3, "dmu==[10,11,2,3,4,5,6,18]", "dmu<=20")
print(te)
```

图11.9：代码运行结果

11.4.7 dea7：支持CRS/VRS切换

dea6可以完成产出方向且规模报酬不变条件下基于谢泼德距离函数的技术效率计算。若要支持规模报酬可变（VRS），需要添加约束条件：$\sum_{i=1}^{n}\lambda_i = 1$。

规模报酬可变的产出导向模型：

$$
\begin{aligned}
D(\mathbf{X}^*, \mathbf{Y}^*)^{-1} = \max_{\theta,\lambda} \quad & \theta \\
s.t. \sum_{i=1}^{n} \lambda_i X_{ki} \leq X_k^*, \quad & k = 1,2,3 \\
\sum_{i=1}^{n} \lambda_i Y_{li} \geq \theta Y_l^*, \quad & l = 1 \\
\sum_{i=1}^{n} \lambda_i = 1 & \\
\lambda_i \geq 0 &
\end{aligned}
$$

```
from pyomo.environ import *
import pandas as pd
from deabook.example.book_cases.loaders import plants
import numpy as np
import re

def dea7(formula, dataframe, rts, eval_query, ref_query):
    """
    计算DEA技术效率（支持CRS/VRS切换版本）

    Parameters
    ----------
    rts : str
```

```
    规模报酬假设: "crs"（规模报酬不变）或"vrs"（规模报酬可变）
"""
theta_results = {}

if eval_query is None:
    index_list = dataframe.index
else:
    index_list = dataframe.query(eval_query).index

if ref_query is None:
    index_list_ref = dataframe.index
else:
    index_list_ref = dataframe.query(ref_query).index

# 解析公式
input_vars = formula.split('=')[1].strip(' ')
xcol = re.compile(' +').sub(' ', input_vars).split(' ')
output_vars = formula.split('=')[0].strip(' ')
ycol = re.compile(' +').sub(' ', output_vars).split(' ')

data = dataframe.loc[index_list, xcol + ycol]
dataref = dataframe.loc[index_list_ref, xcol + ycol]
xref = dataref.loc[:, xcol]
yref = dataref.loc[:, ycol]

for j in data.index:
    x = data.loc[j, xcol]
    y = data.loc[j, ycol]

    model = ConcreteModel()
    model.I = Set(initialize=dataref.index)
    model.K = Set(initialize=range(len(xcol)))
    model.L = Set(initialize=range(len(ycol)))

    model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
    model.intensity = Var(model.I, bounds=(0.0, None),
        doc='intensity variables')

    def obj_rule(model):
```

```
            return model.theta
        model.obj = Objective(rule=obj_rule, sense=maximize,
            doc='objective function')

        def input_rule(model, k):
            return (
                sum(model.intensity[i] * xref.loc[i, xcol[k]] for i in model.I)
                <= x.loc[xcol[k]]
            )
        def output_rule(model, l):
            return (
                sum(model.intensity[i] * yref.loc[i, ycol[l]] for i in model.I)
                >= model.theta * y.loc[ycol[l]]
            )
        def vrs_rule(model):
            """规模报酬可变约束:强度变量之和为1"""
            return sum(model.intensity[i] for i in model.I) == 1

        model.input = Constraint(model.K, rule=input_rule,
            doc='input constraint')
        model.output = Constraint(model.L, rule=output_rule,
            doc='output constraint')

        # 根据rts参数条件添加VRS约束
        if rts == "vrs":
            model.vrs = Constraint(rule=vrs_rule, doc='VRS constraint')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        theta = np.asarray(list(model.theta[:].value))
        theta_results[j] = theta[0]

    thetadf = pd.DataFrame(theta_results, index=["theta"]).T
    thetadf["te"] = 1 / thetadf["theta"]
    return thetadf
```

该函数的典型调用方式如下：

```
import pandas as pd
from deabook.example.book_cases.loaders import plants
```

```
ex3 = plants()
te = dea7("Output=Labor Fuel Capital", ex3, "vrs", "dmu==[10,11,2,3,4,5,6,18]", "dmu<=20")
print(te)
```

图11.10：代码运行结果

11.4.8 dea8：支持投入导向/产出导向

接下来以投入导向进行研究。投入方向的 Farrell 投入效率（即谢泼德投入距离函数的倒数 D_I^{-1}，见第 3 章式(3.12)）公式为：

CRS投入导向：

$$
\begin{aligned}
D_I(\mathbf{X}^*, \mathbf{Y}^*)^{-1} = \min_{\theta,\lambda} \quad & \theta \\
s.t. \sum_{i=1}^{n} \lambda_i X_{ki} \le \theta X_k^*, \quad & k = 1,2,3 \\
\sum_{i=1}^{n} \lambda_i Y_{li} \ge Y_l^*, \quad & l = 1 \\
\lambda_i \ge 0 &
\end{aligned}
$$

这里的 θ 是投入收缩比，取值范围为 $(0,1]$，技术效率 $\text{TE} = \theta = 1/D_I$；而谢泼德投入距离函数本身满足 $D_I \ge 1$。这与产出侧的写法（式中 $D^{-1} = \max\theta$、$\theta \ge 1$ 为扩张倍数）是同一套对应关系的两侧，不要把投入侧的 $\min\theta$ 直接称作谢泼德距离 D。

VRS投入导向：增加约束 $\sum_{i=1}^{n} \lambda_i = 1$

```
from pyomo.environ import *
import pandas as pd
from deabook.example.book_cases.loaders import plants
import numpy as np
import re

def dea8(formula: str,
         dataframe: pd.DataFrame,
         rts: str = "crs",
         orient: str = "oo",
         eval_query: str | None = None,
         ref_query: str | None = None) -> pd.DataFrame:
    """
    计算DEA技术效率（完整版本：支持CRS/VRS、投入/产出导向）

    Parameters
    ----------
    formula : str
```

```
    公式字符串,格式为 "产出 = 投入1 投入2 ..."
dataframe : pd.DataFrame
    包含投入产出数据的DataFrame
rts : str, default "crs"
    规模报酬假设: "crs"(规模报酬不变)或"vrs"(规模报酬可变)
orient : str, default "oo"
    导向类型: "oo"(产出导向, output-oriented)或"io"(投入导向, input-oriented)
eval_query : str or None, default None
    筛选待评价决策单元的查询条件
ref_query : str or None, default None
    筛选参照决策单元的查询条件

Returns
-------
pd.DataFrame
    包含theta值和技术效率值的数据框
"""
theta_results = {}

# 处理查询条件(None表示使用全部数据)
if eval_query is None:
    index_list = dataframe.index
else:
    index_list = dataframe.query(eval_query).index

if ref_query is None:
    index_list_ref = dataframe.index
else:
    index_list_ref = dataframe.query(ref_query).index

# 解析公式字符串
input_vars = formula.split('=')[1].strip(' ')
xcol = re.compile(' +').sub(' ', input_vars).split(' ')
output_vars = formula.split('=')[0].strip(' ')
ycol = re.compile(' +').sub(' ', output_vars).split(' ')

data = dataframe.loc[index_list, xcol + ycol]
dataref = dataframe.loc[index_list_ref, xcol + ycol]
xref = dataref.loc[:, xcol]
```

```
    yref = dataref.loc[:, ycol]

    for j in data.index:
        x = data.loc[j, xcol]
        y = data.loc[j, ycol]

        model = ConcreteModel()
        model.I = Set(initialize=dataref.index)
        model.K = Set(initialize=range(len(xcol)))
        model.L = Set(initialize=range(len(ycol)))

        model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
        model.intensity = Var(model.I, bounds=(0.0, None),
            doc='intensity variables')

        # 目标函数:根据导向确定最大/最小化
        def obj_rule(model):
            return model.theta
        model.obj = Objective(
            rule=obj_rule,
            sense=(maximize if orient == "oo" else minimize),
            doc='objective function'
        )

        # 投入约束:根据导向变化
        def input_rule(model, k):
            if orient == "oo":           # 产出导向:投入不超过被评价单元
                return (
                    sum(model.intensity[i] * xref.loc[i, xcol[
                        k]] for i in model.I)
                    <= x.loc[xcol[k]]
                )
            elif orient == "io":         # 投入导向:投入不超过theta倍被评价单元
                return (
                    sum(model.intensity[i] * xref.loc[i, xcol[
                        k]] for i in model.I)
                    <= model.theta * x.loc[xcol[k]]
                )
```

```
    # 产出约束:根据导向变化
    def output_rule(model, l):
        if orient == "oo":           # 产出导向:产出不低于theta倍被评价单元
            return (
                sum(model.intensity[i] * yref.loc[i, ycol[
                    l]] for i in model.I)
                >= model.theta * y.loc[ycol[l]]
            )
        elif orient == "io":         # 投入导向:产出不低于被评价单元
            return (
                sum(model.intensity[i] * yref.loc[i, ycol[
                    l]] for i in model.I)
                >= y.loc[ycol[l]]
            )

    # VRS约束
    def vrs_rule(model):
        return sum(model.intensity[i] for i in model.I) == 1

    model.input = Constraint(model.K, rule=input_rule,
        doc='input constraint')
    model.output = Constraint(model.L, rule=output_rule,
        doc='output constraint')

    if rts == "vrs":
        model.vrs = Constraint(rule=vrs_rule, doc='VRS constraint')

    opt = SolverFactory('highs')
    solution = opt.solve(model)

    theta = np.asarray(list(model.theta[:].value))
    theta_results[j] = theta[0]

thetadf = pd.DataFrame(theta_results, index=["theta"]).T
# 技术效率计算:产出导向取倒数,投入导向直接取theta
thetadf["te"] = (1 / thetadf["theta"] if orient == "oo" else thetadf[
    "theta"])
return thetadf
```

dea8相比dea7的改动：

1. **参数 orient**：控制产出导向（"oo"）或投入导向（"io"）
2. **目标函数方向**：产出导向为 maximize，投入导向为 minimize
3. **约束条件**：根据导向自动调整不等式形式
4. **技术效率计算**：产出导向 te = 1/theta，投入导向 te = theta
5. **查询条件默认值**：eval_query和 ref_query默认为 None，表示使用全部数据

至此，dea1至 dea8逐步形成了一套在导向、规模报酬与样本选择上均可灵活配置的求解接口。其演进的意义在于：将DEA建模中原本分散、易错的重复代码收敛为少数可控的参数，使研究者得以把注意力从编程细节转移到模型设定本身。该函数的典型调用方式如下：

```
import pandas as pd
from deabook.example.book_cases.loaders import plants
ex3 = plants()

# VRS投入导向
te = dea8("Output=Labor Fuel Capital", ex3, "vrs", "io", "dmu==[10,11,2,3,4,5,6,18]", "dmu<=20")
print(te)
```

图11.11：代码运行结果

11.5 代码重构：用类封装DEA模型

从 dea1到 dea8，代码逐步增加了功能，但也带来了一个问题：**大量代码重复**。每个版本的函数中，模型定义、求解器调用、结果提取等核心逻辑几乎完全相同。这违背了软件工程中的 **DRY原则**（Don't Repeat Yourself，不要重复自己）。

本节展示如何用 **Python类**来重构代码，将通用逻辑封装起来，使代码更简洁、更易维护。这是从" 脚本编程" 到" 模块化设计" 的进阶，也是从入门练习走向专业编程的关键一步。

11.5.1 异常处理：让代码更健壮

在重构之前，先介绍一项重要的工程化技能——**异常处理**。在 dea1到 dea8中，如果求解器未安装、数据格式错误或线性规划无可行解，程序会直接崩溃并抛出晦涩的错误信息。通过 try-except块来捕获这些异常，可以给用户提供有意义的错误提示：

```
from pyomo.opt import SolverStatus, TerminationCondition

def solve_with_check(model, solver="highs"):
    """
    带状态检查的求解器调用

    检查求解器是否可用、求解是否成功，提供清晰的错误信息。
    """
    opt = SolverFactory(solver)
```

```
    # 检查求解器是否已安装
    if not opt.available(exception_flag=False):
        raise RuntimeError(
            f"求解器 '{solver}' 未安装或不可用。"
            f"请运行以下命令安装：\n"
            f"  conda install {solver}\n"
            f"或使用其他求解器：'highs'、'ipopt'"
        )

    # 调用求解器
    solution = opt.solve(model, load_solutions=False)

    # 检查求解状态
    if solution.solver.status != SolverStatus.ok:
        raise RuntimeError(f"求解器状态异常：{solution.solver.status}")

    # 检查求解终止条件
    if solution.solver.termination_condition != TerminationCondition.optimal:
        if (solution.solver.termination_condition ==
            TerminationCondition.infeasible):
            raise RuntimeError(
                "线性规划无可行解——请检查数据是否存在异常值(如0或负数)，"
                "或投入产出变量是否设置正确。"
            )
        else:
            raise RuntimeError(
                f"求解未达到最优。终止条件：{solution.solver.termination_condition}"
            )

    model.solutions.load_from(solution)
    return solution
```

教学价值：异常处理是工程化编程的核心能力。这段代码向读者展示了”如何给用户提供有意义的错误信息”，而非让Python抛出晦涩的堆栈跟踪。

11.5.2 DEAModel类

下面用类来封装DEA模型：

```
from pyomo.environ import *
from pyomo.opt import SolverStatus, TerminationCondition
import pandas as pd
from deabook.example.book_cases.loaders import plants
import numpy as np
import re


class DEAModel:
    """
    DEA效率评价模型(类封装版本)

    将DEA模型的构建、求解、结果提取封装为类,减少代码重复,
    提高代码的可维护性和可扩展性。

    Parameters
    ----------
    formula : str
        公式字符串,格式为 "产出 = 投入1 投入2 ..."
    dataframe : pd.DataFrame
        包含投入产出数据的DataFrame
    orient : str, default "oo"
        导向类型: "oo"(产出导向)或"io"(投入导向)
    rts : str, default "crs"
        规模报酬假设: "crs"或"vrs"
    """

    def __init__(self, formula, dataframe, orient="oo", rts="crs"):
        self.formula = formula
        self.dataframe = dataframe
        self.orient = orient
        self.rts = rts
        self._parse_formula()

    def _parse_formula(self):
        """解析公式字符串,提取投入变量名和产出变量名"""
        input_vars = self.formula.split('=')[1].strip(' ')
        self.xcol = re.compile(' +').sub(' ', input_vars).split(' ')
        output_vars = self.formula.split('=')[0].strip(' ')
```

```
        self.ycol = re.compile(' +').sub(' ', output_vars).split(' ')

    def _prepare_data(self, eval_query=None, ref_query=None):
        """
        根据查询条件准备评价数据和参考数据

        Returns
        -------
        data, dataref : pd.DataFrame
            评价数据和参照数据
        """
        if eval_query is None:
            index_list = self.dataframe.index
        else:
            index_list = self.dataframe.query(eval_query).index

        if ref_query is None:
            index_list_ref = self.dataframe.index
        else:
            index_list_ref = self.dataframe.query(ref_query).index

        cols = self.xcol + self.ycol
        data = self.dataframe.loc[index_list, cols]
        dataref = self.dataframe.loc[index_list_ref, cols]
        return data, dataref

    def _build_model(self, x, y, xref, yref):
        """
        构建Pyomo模型(通用部分)

        Parameters
        ----------
        x, y : pd.Series
            被评价单元的投入和产出
        xref, yref : pd.DataFrame
            参照单元的投入和产出

        Returns
        -------
```

```
        model : ConcreteModel
            构建好的Pyomo模型
        """
        model = ConcreteModel()
        model.I = Set(initialize=xref.index)
        model.K = Set(initialize=range(len(self.xcol)))
        model.L = Set(initialize=range(len(self.ycol)))

        model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
        model.intensity = Var(model.I, bounds=(0.0, None),
            doc='intensity variables')

        # 目标函数
        def obj_rule(model):
            return model.theta
        model.obj = Objective(
            rule=obj_rule,
            sense=(maximize if self.orient == "oo" else minimize)
        )

        # 投入约束
        def input_rule(model, k):
            if self.orient == "oo":
                return (
                    sum(model.intensity[i] * xref.loc[i, self.xcol[k]]
                          for i in model.I)
                    <= x.loc[self.xcol[k]]
                )
            else:  # io
                return (
                    sum(model.intensity[i] * xref.loc[i, self.xcol[k]]
                          for i in model.I)
                    <= model.theta * x.loc[self.xcol[k]]
                )

        # 产出约束
        def output_rule(model, l):

            if self.orient == "oo":
```

```
                return (
                    sum(model.intensity[i] * yref.loc[i, self.ycol[l]]
                          for i in model.I)
                    >= model.theta * y.loc[self.ycol[l]]
                )
            else:  # io
                return (
                    sum(model.intensity[i] * yref.loc[i, self.ycol[l]]
                          for i in model.I)
                    >= y.loc[self.ycol[l]]
                )

        model.input = Constraint(model.K, rule=input_rule)
        model.output = Constraint(model.L, rule=output_rule)

        # VRS约束
        if self.rts == "vrs":
            def vrs_rule(model):
                return sum(model.intensity[i] for i in model.I) == 1
            model.vrs = Constraint(rule=vrs_rule)

        return model

    def solve(self, eval_query=None, ref_query=None, solver="highs"):
        """
        求解DEA模型并返回效率结果

        Parameters
        ----------
        eval_query : str or None
            筛选待评价决策单元的查询条件
        ref_query : str or None
            筛选参照决策单元的查询条件
        solver : str, default "highs"
            求解器名称

        Returns
        -------
        pd.DataFrame
```

```
            包含theta值和技术效率值的数据框
        """
        data, dataref = self._prepare_data(eval_query, ref_query)
        xref = dataref.loc[:, self.xcol]
        yref = dataref.loc[:, self.ycol]

        theta_results = {}

        for j in data.index:
            x = data.loc[j, self.xcol]
            y = data.loc[j, self.ycol]

            model = self._build_model(x, y, xref, yref)

            # 调用带检查的求解
            opt = SolverFactory(solver)
            if not opt.available(exception_flag=False):
                raise RuntimeError(f"求解器 '{solver}' 未安装。请安装后重试。")

            solution = opt.solve(model)

            # 检查求解状态
            if (solution.solver.termination_condition
                != TerminationCondition.optimal):
                print(f"警告: DMU {j} 的求解未达到最优, "
                      f"终止条件: {solution.solver.termination_condition}")

            theta = np.asarray(list(model.theta[:].value))
            theta_results[j] = theta[0]

        thetadf = pd.DataFrame(theta_results, index=["theta"]).T
        thetadf["te"] = (1 / thetadf["theta"]
                         if self.orient == "oo" else thetadf["theta"])
        return thetadf
```

11.5.3 使用DEAModel类

```
import pandas as pd
from deabook.example.book_cases.loaders import plants
```

```
# 读取数据
ex3 = plants()

# 创建模型实例(产出导向、VRS)
model = DEAModel("Output=Labor Fuel Capital", ex3, orient="oo", rts="vrs")

# 求解
te = model.solve(eval_query="dmu in [1,2,3,4,5]", ref_query="dmu<=20")
print(te)

# 切换到CRS投入导向,无需重新定义类
te_io = DEAModel("Output=Labor Fuel Capital", ex3, orient="io", rts="crs").solve()
print(te_io)
```

类的优势：

1. **消除重复**：模型构建逻辑只写一次，多个DMU共用
2. **易于扩展**：增加新功能(如SBM模型、Malmquist指数)只需继承或修改类
3. **状态封装**：公式解析、数据准备等中间状态存储在实例属性中
4. **接口清晰**：`solve()`方法的调用方式直观简洁

11.6 常见错误与调试技巧

在编写DEA代码的过程中，初学者常会遇到各种错误。本节汇总了最常见的错误现象、原因和解决方案，帮助读者快速定位和修复问题。

表 11.3: 常见错误与调试方法

错误现象	可能原因	解决方案
`SolverFactory('highs')` 报错	未安装 highspy	`python -m pip install highspy`
`RuntimeError: 求解器不可用`	求解器未正确安装	运行验证代码检查可用求解器；参见附录A.1.3 节
`ValueError: No objective`	目标函数定义错误	检查 `Objective`的 `rule`参数是否返回有效表达式
`Constraint.Skip` 无效	索引变量未传入	确保 `rule`函数接受索引参数(如 `def rule(model, k)`)
效率值全为1.0	指标过多而DMU过少(维数灾难)，或各单元均具独特投入产出组合，样本内无法区分	增加样本量、合并或删减指标；检查参照集是否过小(是否误把每个DMU当作自己的唯一参照)

错误现象	可能原因	解决方案
模型无可行解或求解失败	数据含0、负值,或产出全为0	检查并处理非正值;确认模型对零值的假设;检查投入产出变量方向
VRS结果与CRS相同	VRS约束未生效	检查 `if rts == "vrs"`条件拼写;确认约束已添加到模型
`KeyError` 在数据索引时	列名拼写错误	检查formula中的变量名是否与DataFrame列名一致
`TypeError: 'NoneType'`	eval_query或ref_query为无效字符串	检查query字符串语法;确保列名存在
内存溢出(大样本)	约束数量过大	使用切割平面算法或降采样;考虑使用专业包如 `deabook`
产出导向效率值大于1	theta计算方向错误	产出导向效率 = 1/theta;投入导向效率 = theta
索引错位(结果索引从0开始)	未使用原始索引循环	将 `range(n)`改为 `data.index`;将 `iloc`改为 `loc`

调试建议:

1. **从简单开始:**先用 `dea1`在5个DMU的小数据集上测试,确认代码正确后再扩展
2. **逐步验证:**每增加一个功能,检查结果是否合理(如VRS效率应不低于CRS效率,因为CRS技术集包含VRS技术集, CRS可行域更大)
3. **打印中间变量:**在循环中打印 `x`、`y`的值,确认数据切片正确
4. **检查求解状态:**使用 `solve_with_check`函数捕获求解异常

11.7 案例应用:把单个电厂求解扩展为循环

本章只使用美国19家电厂。先从原文件抽取前5行检查输入与索引,再以全部19行为参照集循环求解。投入为Labor、Fuel、Capital,产出为Output;结果图使用VRS产出导向效率1/φ。表只展示原文件前5行的结果,这不是前五名排名,完整结果仍包含19家电厂。

```
from deabook.example.book_cases.run import FUN
FUN[11]()  # 重新计算本章,输出完整CSV、摘要表与PNG/PDF图
```

表 11.4: 把单个电厂求解扩展为循环:结果摘要

电厂	劳动	燃料	装机	发电量	VRS产出效率
Coffeen	179.000	3969.100	1005.500	3531.400	0.873
Grant Tower	96.000	896.320	194.600	841.300	1.000
Gudsonville	94.000	803.170	200.000	690.500	1.000
Meredosia	141.000	1629.900	564.100	1498.100	0.926

电厂	劳动	燃料	装机	发电量	VRS产出效率
Newton	147.000	3264.400	617.400	3347.700	1.000
Fisk	261.000	1427.500	347.100	1263.200	0.900
Joliet	448.000	6365.300	1680.400	5734.000	0.893
Kinkaid	223.000	4911.000	1319.400	4107.700	0.825

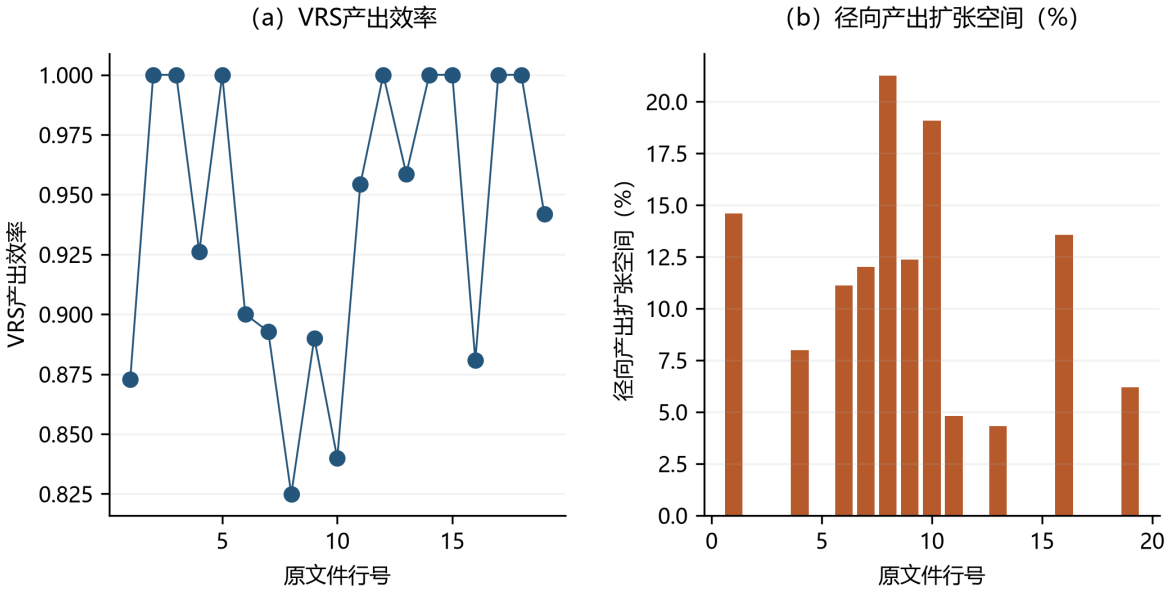


图 11.1: 从单个电厂求解扩展到19家电厂循环

表按原文件顺序展示前8家电厂的投入、产出及VRS产出效率，便于将循环计算的返回索引与原始记录逐一对应。图左展示全部19家电厂的效率，图右按100×(1/效率−1)计算径向产出扩张空间；该比例未包含非径向松弛。

循环中用原始行号追踪结果，参照集始终保持全部19行。仅改变显示行数不会改变参照技术；若为了缩短表格而只拿前5家构造前沿，效率会改变。教材前面的索引、函数封装和类封装都复用同一数据入口。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch11_full.csv`。表中P10和P90为第10和第90百分位数（如适用），展示分布而不占用逐个决策单元的行数。

11.8 本章小结

本章通过**迭代式教学**的方法，从单个决策单元的线性规划求解出发，逐步构建了一个功能完整的DEA效率评价函数。主要内容包括：

1. **Pyomo建模基础**：索引集合、决策变量、目标函数、约束条件的定义
2. **循环迭代**：用 `for`循环对所有决策单元进行效率评价
3. **函数封装**：将代码组织为可复用的函数（`dea1`到 `dea8`）
4. **功能扩展**：DataFrame支持、查询条件、公式解析、 CRS/VRS切换、投入/产出导向

5. **代码重构**：用Python类封装DEA模型，实现模块化设计
6. **异常处理**：添加求解状态检查，提高代码健壮性

11.8.1 从自定义函数到deabook包

本章构建的自定义DEA函数虽然功能完整，但在实际研究中推荐使用成熟的软件包（如 `deabook`），原因有三：

1. **可靠性**：软件包经过大量测试，边界情况处理更完善；
2. **效率**：软件包使用优化的求解策略（如切割平面算法），大样本下速度更快；
3. **功能**：软件包实现了更多模型（SBM、DDF、Malmquist指数等），自定义函数难以覆盖。

理解本章的编程原理，有助于读者在使用 `deabook`时调试错误、扩展功能，甚至在软件包无法满足需求时自行开发。

11.8.2 变量命名对照表

表 11.5: 本章主要变量命名对照

变量名	含义	备注
`theta`	目标函数的直接优化结果：产出导向为产出扩张倍数（$\theta \geq 1$，即效率的倒数）；投入导向为投入收缩比（$\theta \in (0, 1]$，即技术效率本身）	两种导向下 `theta` 的经济含义不同，不可混称” 效率倒数”
`te`	技术效率值（technical efficiency）	产出导向：te = 1/theta；投入导向：te = theta
`intensity`	强度变量（λ_i）	DEA文献标准术语，替代 `lamda`/`lambda_var`
`theta_results`	存储各DMU的theta值的字典/数组	替代原 `thetalt`
`eval_query`	评价单元查询条件字符串	snake_case命名，替代原 `evaquery`
`ref_query`	参照单元查询条件字符串	snake_case命名，替代原 `refquery`
`xcol`	投入变量名列表	从formula解析得到
`ycol`	产出变量名列表	从formula解析得到
`orient`	导向方向参数	`"oo"`=产出导向，`"io"`=投入导向
`rts`	规模报酬假设参数	`"crs"`=不变，`"vrs"`=可变

命名规范建议：遵循Python的PEP 8标准，使用 **snake_case**（小写下划线分隔）命名变量和函数，使用 **CamelCase** 命名类。选择具有描述性的名称，避免缩写（除非缩写是领域通用术语如 `te`、`rts`）。

附录：关于各求解器（glpk、cbc、ipopt、mosek、gurobi、AMPL/Knitro、scip）的详细安装步骤，请参阅本书**附录A.1.3 节（求解器安装详细指南）**。

第 12 章

DEA程序编写的实战演练

代码运行约定：在项目根目录运行，先安装 `python -m pip install numpy pandas scipy matplotlib pyomo highspy openpyxl`。各段Python 代码按本章顺序在同一会话执行，后文复用前文定义；默认采用HiGHS。公式讲解片段另标为 `text`，不作为独立程序执行。

上一章详细介绍了如何在Python中逐步编写DEA程序。本章将通过6个代表性实例开展程序编写的实战演练，涵盖SBM模型、含非期望产出的SBM模型、DDF模型、NDDF模型、Malmquist生产率指数以及Malmquist-Luenberger生产率指数。

第三章至第十章已系统建立了各类DEA模型的理论基础，第十一章则阐述了运用Python与Pyomo将这些模型转化为线性规划求解程序的一般编程逻辑。本章在上述理论与编程逻辑之上开展实战演练，依次实现SBM、含非期望产出的SBM、方向距离函数（DDF）、非径向方向距离函数（NDDF）、Malmquist以及Malmquist-Luenberger六类具有代表性的模型。需要强调的是，尽管这些模型在经济学含义与数学形式上各有侧重，其在计算结构上却共享同一骨架——公式解析、数据准备、模型构建、求解与结果提取——差异仅局部地体现在目标函数与约束条件的设定之上。本章将先以两个完整示例呈现这一骨架的全貌，进而通过提取公共工具函数说明如何在保持模型特异性的同时消除重复代码，最后在后续模型中复用这些工具，将注意力集中于各模型特有的逻辑。

12.1 SBM模型的程序编写

Tone(2001)提出了基于松弛测度的效率模型（Slacks-Based Measure, SBM）。假设决策单元以 P 种投入要素 X 生产出 Q 种产出 Y，相应的技术效率测度模型定义如下：

$$
\begin{aligned}
\min \quad & \rho = \frac{1 - \frac{1}{P}\sum_{p=1}^{P} s_p^X / X_{0p}}{1 + \frac{1}{Q}\sum_{q=1}^{Q} s_q^Y / Y_{0q}} \\
\text{s.t.} \quad & X_{0p} = \sum_{i=1}^{N} \lambda_i X_{ip} + s_p^X, \qquad p = 1, \dots, P, \\
& Y_{0q} = \sum_{i=1}^{N} \lambda_i Y_{iq} - s_q^Y, \qquad q = 1, \dots, Q, \\
& \lambda_i, s_p^X, s_q^Y \geq 0.
\end{aligned}
$$

上述目标函数为非线性形式，无法直接运用线性规划技术求解，因此在求解前需对其进行如下**Charnes-Cooper线性转换**：

$$
\begin{aligned}
\min \quad & \tau = t - \frac{1}{P}\sum_{p=1}^{P} S_p^X / X_{0p} \\
\text{s.t.} \quad & 1 = t + \frac{1}{Q}\sum_{q=1}^{Q} S_q^Y / Y_{0q}, \\
& tX_{0p} = \sum_{i=1}^{N} \Lambda_i X_{ip} + S_p^X, \quad p = 1, \ldots, P, \\
& tY_{0q} = \sum_{i=1}^{N} \Lambda_i Y_{iq} - S_q^Y, \quad q = 1, \ldots, Q, \\
& \Lambda_i, S_p^X, S_q^Y \geq 0,\ t > 0.
\end{aligned}
$$

在这个线性规划问题中：

(1) **变量**为$\{t, S_p^X, S_q^Y, \Lambda_i\}$，其中 t 是一维的，S_p^X 的维度是 P，S_q^Y 的维度是 Q，Λ_i 的维度是 N；$\Lambda_i, S_p^X, S_q^Y \geq 0, t > 0$；

(2) **目标函数**中变量 t 和 S_p^X 对应的系数分别为1和 $\frac{1}{P \cdot X_{0p}}$；求最小值；

(3) **约束条件**有三个：

- 第一个约束（Charnes-Cooper转换约束）对应的行数有 1 个，其中 t 和 S_q^Y 的系数分别为1和 $\frac{1}{Q \cdot Y_{0q}}$；
- 第二个约束（投入约束）对应的行数有 P 个，其中 t、Λ_i、S_p^X 的系数分别为 X_{0p}、X_{ip} 和1；
- 第三个约束（产出约束）对应的行数有 Q 个，其中 t、Λ_i、S_q^Y 的系数分别为 Y_{0q}、Y_{iq} 和1；
- 三个约束的约束符号都为 $=$。

以下给出求解SBM效率的完整实现。作为本章第一个完整示例，该函数意在呈现从公式解析、数据准备到模型求解与结果整理的完整流程，并为后续各模型提供一个可对照的计算骨架。

```
from deabook import data_path
from pyomo.environ import *
from pyomo.opt import SolverStatus, TerminationCondition
import pandas as pd
from deabook.example.book_cases.loaders import china
import numpy as np
import re
import warnings

warnings.filterwarnings("ignore")


def sbm(formula, dataframe, eval_query=None, ref_query=None, solver="highs"):
    """
    计算SBM（Slacks-Based Measure）效率值。
```

```
    基于Tone(2001)的SBM模型,通过Charnes-Cooper线性转换将非线性
    规划问题转化为线性规划问题求解。

    Parameters
    ----------
    formula : str
        公式字符串,格式为 "产出 = 投入1 投入2 ..."
        例如: "Y = K L"
    dataframe : pd.DataFrame
        待评价决策单元的投入产出数据框
    eval_query : str or None, default None
        筛选待评价决策单元的query字符串。None表示评价全部
    ref_query : str or None, default None
        筛选参照决策单元的query字符串。None表示使用全部
    solver : str, default "highs"
        线性规划求解器名称

    Returns
    -------
    pd.DataFrame
        包含效率值(obj)和各投入产出松弛变量的DataFrame

    Examples
    --------
    >>> import pandas as pd
from deabook.example.book_cases.loaders import china
    >>> ex4 = china(scaled=True)
    >>> result = sbm("Y = K L", ex4, "t==[1,2,3]", "t==[1,2,3]")
    """
    # === Step 1: 解析公式字符串 ===
    input_vars = formula.split("=")[1].strip(" ")
    xcol = re.compile(" +").sub(" ", input_vars).split(" ")
    output_vars = formula.split("=")[0].strip(" ")
    ycol = re.compile(" +").sub(" ", output_vars).split(" ")

    # === Step 2: 根据查询条件准备数据 ===
    if eval_query is None:
        index_eval = dataframe.index
    else:
```

```
    index_eval = dataframe.query(eval_query).index

if ref_query is None:
    index_ref = dataframe.index
else:
    index_ref = dataframe.query(ref_query).index

data = dataframe.loc[index_eval, xcol + ycol]
dataref = dataframe.loc[index_ref, xcol + ycol]

xref = dataref.loc[:, xcol]   # 参照单元投入
yref = dataref.loc[:, ycol]   # 参照单元产出

# === Step 3: 数据验证 ===
assert data[xcol + ycol].notna().all().all(), "数据包含缺失值"
assert (data[xcol] > 0).all().all(), "投入数据必须严格为正"
assert (data[ycol] > 0).all().all(), "产出数据必须严格为正"

# 初始化结果存储字典
slack_x_results = {}   # 投入松弛变量
slack_y_results = {}   # 产出松弛变量
obj_results = {}       # 目标函数值

# === Step 4: 对每个决策单元构建并求解线性规划 ===
for j in data.index:
    x = data.loc[j, xcol]   # 被评价单元投入
    y = data.loc[j, ycol]   # 被评价单元产出

    # 构建Pyomo模型
    model = ConcreteModel()
    model.I = Set(initialize=dataref.index)          # 参照决策单元集合
    model.K = Set(initialize=range(len(xcol)))       # 投入变量集合
    model.L = Set(initialize=range(len(ycol)))       # 产出变量集合

    # 定义决策变量
    model.t = Var(domain=PositiveReals, doc="CC transformation variable")
    model.slack_x = Var(model.K, bounds=(0.0, None), doc="input slack")
    model.slack_y = Var(model.L, bounds=(0.0, None), doc="output slack")
    model.intensity = Var(model.I, bounds=(0.0, None), doc="intensity variables")
```

```
        # 目标函数: min t - (1/P) * sum(slack_x[p] / x[p])
        def objective_rule(model):
            return model.t - sum(
                model.slack_x[k] / x.iloc[k] for k in model.K
            ) / len(xcol)
        model.obj = Objective(rule=objective_rule, sense=minimize)

        # Charnes-Cooper转换约束
        def cc_trans_rule(model):
            return 1 == model.t + sum(
                model.slack_y[l] / y.iloc[l] for l in model.L
            ) / len(ycol)
        model.cc_trans = Constraint(rule=cc_trans_rule)

        # 投入约束: t * x[k] = sum(intensity[i] * xref[i,k]) + slack_x[k]
        def input_rule(model, k):
            return sum(
                model.intensity[i] * xref.loc[i, xcol[k]] for i in model.I
            ) == x.iloc[k] * model.t - model.slack_x[k]
        model.input = Constraint(model.K, rule=input_rule)

        # 产出约束: t * y[l] = sum(intensity[i] * yref[i,l]) - slack_y[l]
        def output_rule(model, l):
            return sum(
                model.intensity[i] * yref.loc[i, ycol[l]] for i in model.I
            ) == y.iloc[l] * model.t + model.slack_y[l]
        model.output = Constraint(model.L, rule=output_rule)

        # === Step 5: 求解模型 ===
        opt = SolverFactory(solver)
        if not opt.available(exception_flag=False):
            raise RuntimeError(f"求解器 '{solver}' 未安装。请安装后重试。")

        solution = opt.solve(model)

        if solution.solver.termination_condition == "optimal":
            # 提取结果(线性变换后松弛变量 slack = Slack / t)
            t_val = value(model.t)
            slack_x_results[j] = np.asarray(
```

```
                list(model.slack_x[:].value)
            ) / t_val
            slack_y_results[j] = np.asarray(
                list(model.slack_y[:].value)
            ) / t_val
            obj_results[j] = value(model.obj)

    # === Step 6: 格式化输出结果 ===
    obj_df = pd.DataFrame(obj_results, index=["obj"]).T

    slack_x_df = pd.DataFrame(slack_x_results, index=xcol).T
    slack_x_df.columns = slack_x_df.columns.map(lambda x: "slack " + str(x))

    slack_y_df = pd.DataFrame(slack_y_results, index=ycol).T
    slack_y_df.columns = slack_y_df.columns.map(lambda y: "slack " + str(y))

    result = pd.concat([obj_df, slack_x_df, slack_y_df], axis=1)
    return result
```

上述SBM函数沿用了第十一章所阐述的一般编程逻辑。函数首先解析公式字符串以分离投入与产出变量，并依据 `eval_query`与 `ref_query`筛选评价单元与参照单元；随后对数据的完整性与正性进行校验，以保证线性规划的可行。核心部分是针对每个被评价单元构建并求解对应的Pyomo模型——声明强度变量、各项松弛变量与Charnes-Cooper变换变量，定义以 τ 为目标、以转化后约束为条件的线性规划，并调用求解器提取效率值与各项松弛。最后，函数将结果整理为结构化的DataFrame予以返回。下文两个说明性注记分别解释了强度变量的命名缘由，以及标量变量与带索引变量在结果提取时的取值差异。

命名说明： `model.intensity`是DEA模型中的**强度变量**（intensity variable），对应数学公式中的 λ_i。在Python中，`lambda`是关键字（用于定义匿名函数），因此不能作为变量名。使用 `intensity` 而非 `lamda`，因为 `intensity`是DEA文献中的标准术语，含义明确。`slack_x`和 `slack_y`分别表示投入和产出的松弛变量。

标量变量与索引变量的取值方式不同。 `model.t` 是Charnes-Cooper变换引入的**标量** `Var`（声明时没有索引集），求解后 `model.t.value` 已经是一个浮点数，因此直接写 `t_val = value(model.t)`（或 `float(model.t.value)`）即可；若照搬索引变量的写法 `list(model.t.value)`，会得到 `TypeError: 'float' object is not iterable`。相反，`model.slack_x`、`model.slack_y`、`model.intensity` 是带索引集的`Var`，`model.slack_x[:].value` 返回的是切片视图，需要 `list(...)` 或 `np.asarray(list(...))` 才能展开成数组。判断依据只有一条：`Var` 声明时是否传入了索引集。

下面以本章省级面板中第1至3期为例调用 `sbm`函数，其运行结果如**图12.1** 所示：

```
import pandas as pd
from deabook.example.book_cases.loaders import china
```

```
# 读取数据
ex4 = china(scaled=True)

# 计算SBM效率(评价第1-3期数据,参照集为第1-3期)
sbm_result = sbm("Y = K L", ex4, "t==[1,2,3]", "t==[1,2,3]")
print(sbm_result)
```

图12.1：SBM模型代码运行结果

结果DataFrame包含：`obj`（SBM效率值）、`slack K`（资本投入松弛）、`slack L`（劳动投入松弛）、`slack Y`（产出松弛）。效率值越小表示技术效率越低，SBM效率的取值范围为 $(0, 1]$。当某决策单元的 `obj` 等于1时，表明其已落在生产前沿上、技术有效；`obj`偏离1的程度与各项松弛变量的大小共同刻画了该单元的无效来源与改进方向。

上述基础SBM未考虑生产过程中的非期望产出。在环境与发展的议题下，污染等副产品往往与经济产出相伴而生，忽略它们会使效率评价失真。下一节将扩展该框架，把非期望产出纳入前沿构造。

12.2 包含非期望产出的SBM模型的程序编写

Tone(2003)对原始SBM模型进行了扩展，使其能够用于包含非期望产出情形下的技术效率测度。假设决策单元以 P 种投入要素 X 生产出 Q 种产出 Y，同时伴随产生 R 种非期望产出 B，相应的技术效率测度模型定义如下：

$$
\begin{aligned}
\min \quad & \rho = \frac{1 - \frac{1}{P}\sum_{p=1}^{P} s_p^X / X_{0p}}{1 + \frac{1}{Q+R}\left(\sum_{q=1}^{Q} s_q^Y / Y_{0q} + \sum_{r=1}^{R} s_r^B / B_{0r}\right)} \\
\text{s.t.} \quad & X_{0p} = \sum_{i=1}^{N} \lambda_i X_{ip} + s_p^X, \quad p = 1, \ldots, P, \\
& Y_{0q} = \sum_{i=1}^{N} \lambda_i Y_{iq} - s_q^Y, \quad q = 1, \ldots, Q, \\
& B_{0r} = \sum_{i=1}^{N} \lambda_i B_{ir} + s_r^B, \quad r = 1, \ldots, R, \\
& \lambda_i, s_p^X, s_q^Y, s_r^B \geq 0.
\end{aligned}
$$

利用线性转换方法，上述优化问题可以转换为如下线性规划问题：

$$
\begin{aligned}
\min \quad & \tau = t - \frac{1}{P}\sum_{p=1}^{P} S_p^X / X_{0p} \\
\text{s.t.} \quad & 1 = t + \frac{1}{Q+R}\left(\sum_{q=1}^{Q} S_q^Y / Y_{0q} + \sum_{r=1}^{R} S_r^B / B_{0r}\right), \\
& tX_{0p} = \sum_{i=1}^{N} \Lambda_i X_{ip} + S_p^X, \quad p = 1, ..., P, \\
& tY_{0q} = \sum_{i=1}^{N} \Lambda_i Y_{iq} - S_q^Y, \quad q = 1, ..., Q, \\
& tB_{0r} = \sum_{i=1}^{N} \Lambda_i B_{ir} + S_r^B, \quad r = 1, ..., R, \\
& \Lambda_i, S_p^X, S_q^Y, S_r^B \geq 0, \quad t > 0.
\end{aligned}
$$

与 12.1 节的SBM模型相比，含非期望产出的SBM模型增加了：1. **非期望产出松弛变量** S_r^B 及其对应约束 2. **非期望产出项** 加入到Charnes-Cooper转换约束的分母中（$Q+R$）

```
def sbm_undesirable(formula, dataframe, eval_query=None, ref_query=None,
                    solver="highs"):
    """
    计算含非期望产出的SBM效率值(Tone, 2003)。

    Parameters
    ----------
    formula : str
        公式字符串,格式为 "产出:非期望产出 = 投入1 投入2 ..."
        例如: "Y:CO2 = K L"
    dataframe : pd.DataFrame
        待评价决策单元的投入产出数据框
    eval_query : str or None, default None
        筛选待评价决策单元的query字符串
    ref_query : str or None, default None
        筛选参照决策单元的query字符串
    solver : str, default "highs"
        线性规划求解器名称

    Returns
    -------
    pd.DataFrame
        包含效率值(obj)、投入松弛、期望产出松弛、非期望产出松弛的DataFrame

    Examples
```

```
    --------
    >>> result = sbm_undesirable("Y:CO2 = K L", ex4, "t==[1,2,3]", "t==[1,2,3]")
    """
    # === Step 1: 解析公式字符串(投入:非期望产出 = 投入变量) ===
    input_vars = formula.split("=")[1].strip(" ")
    xcol = re.compile(" +").sub(" ", input_vars).split(" ")

    output_vars = formula.split("=")[0].split(":")[0].strip(" ")
    ycol = re.compile(" +").sub(" ", output_vars).split(" ")

    undesirable_vars = formula.split("=")[0].split(":")[1].strip(" ")
    bcol = re.compile(" +").sub(" ", undesirable_vars).split(" ")

    # === Step 2: 根据查询条件准备数据 ===
    if eval_query is None:
        index_eval = dataframe.index
    else:
        index_eval = dataframe.query(eval_query).index

    if ref_query is None:
        index_ref = dataframe.index
    else:
        index_ref = dataframe.query(ref_query).index

    data = dataframe.loc[index_eval, xcol + ycol + bcol]
    dataref = dataframe.loc[index_ref, xcol + ycol + bcol]

    xref = dataref.loc[:, xcol]   # 参照单元投入
    yref = dataref.loc[:, ycol]   # 参照单元期望产出
    bref = dataref.loc[:, bcol]   # 参照单元非期望产出(新增)

    # === Step 3: 数据验证 ===
    assert data[xcol + ycol + bcol].notna().all().all(), "数据包含缺失值"
    assert (data[xcol] > 0).all().all(), "投入数据必须严格为正"
    assert (data[ycol] > 0).all().all(), "期望产出数据必须严格为正"
    assert (data[bcol] >= 0).all().all(), "非期望产出不能为负"

    # 初始化结果存储
    slack_x_results = {}
```

```
    slack_y_results = {}
    slack_b_results = {}   # 非期望产出松弛(新增)
    obj_results = {}

    # === Step 4: 对每个决策单元构建并求解线性规划 ===
    for j in data.index:
        x = data.loc[j, xcol]   # 被评价单元投入
        y = data.loc[j, ycol]   # 被评价单元期望产出
        b = data.loc[j, bcol]   # 被评价单元非期望产出(新增)

        model = ConcreteModel()
        model.I = Set(initialize=dataref.index)
        model.K = Set(initialize=range(len(xcol)))
        model.L = Set(initialize=range(len(ycol)))
        model.M = Set(initialize=range(len(bcol)))   # 非期望产出集合(新增)

        model.t = Var(domain=PositiveReals, doc="CC transformation variable")
        model.slack_x = Var(model.K, bounds=(0.0, None), doc="input slack")
        model.slack_y = Var(model.L, bounds=(0.0, None), doc="desirable output slack")
        model.slack_b = Var(model.M, bounds=(0.0, None), doc="undesirable output slack")
        model.intensity = Var(model.I, bounds=(0.0, None), doc="intensity variables")

        # 目标函数(与基础SBM相同)
        def objective_rule(model):
            return model.t - sum(
                model.slack_x[k] / x.iloc[k] for k in model.K
            ) / len(xcol)
        model.obj = Objective(rule=objective_rule, sense=minimize)

        # CC转换约束(分母变为 Q+R,增加非期望产出项)
        def cc_trans_rule(model):
            return 1 == model.t + (
                sum(model.slack_y[l] / y.iloc[l] for l in model.L)
                + sum(model.slack_b[m] / b.iloc[m] for m in model.M)
            ) / (len(ycol) + len(bcol))
        model.cc_trans = Constraint(rule=cc_trans_rule)

        # 投入约束(与基础SBM相同)
        def input_rule(model, k):
```

```
            return sum(
                model.intensity[i] * xref.loc[i, xcol[k]] for i in model.I
            ) == x.iloc[k] * model.t - model.slack_x[k]
        model.input = Constraint(model.K, rule=input_rule)

        # 期望产出约束(与基础SBM相同)
        def output_rule(model, l):
            return sum(
                model.intensity[i] * yref.loc[i, ycol[l]] for i in model.I
            ) == y.iloc[l] * model.t + model.slack_y[l]
        model.output = Constraint(model.L, rule=output_rule)

        # 非期望产出约束(新增): t*b = sum(lambda*bref) + slack_b
        def undesirable_output_rule(model, m):
            return sum(
                model.intensity[i] * bref.loc[i, bcol[m]] for i in model.I
            ) == b.iloc[m] * model.t - model.slack_b[m]
        model.undesirable_output = Constraint(model.M, rule=undesirable_output_rule)

        # === Step 5: 求解模型 ===
        opt = SolverFactory(solver)
        if not opt.available(exception_flag=False):
            raise RuntimeError(f"求解器 '{solver}' 未安装。请安装后重试。")

        solution = opt.solve(model)

        if solution.solver.termination_condition == "optimal":
            t_val = value(model.t)
            slack_x_results[j] = np.asarray(list(model.slack_x[:].value)) / t_val
            slack_y_results[j] = np.asarray(list(model.slack_y[:].value)) / t_val
            slack_b_results[j] = np.asarray(list(model.slack_b[:].value)) / t_val
            obj_results[j] = value(model.obj)

    # === Step 6: 格式化输出结果 ===
    obj_df = pd.DataFrame(obj_results, index=["obj"]).T

    slack_x_df = pd.DataFrame(slack_x_results, index=xcol).T
    slack_x_df.columns = slack_x_df.columns.map(lambda x: "slack " + str(x))
```

```
    slack_y_df = pd.DataFrame(slack_y_results, index=ycol).T
    slack_y_df.columns = slack_y_df.columns.map(lambda y: "slack " + str(y))

    result = pd.concat([obj_df, slack_x_df, slack_y_df], axis=1)

    slack_b_df = pd.DataFrame(slack_b_results, index=bcol).T
    slack_b_df.columns = slack_b_df.columns.map(lambda b: "slack " + str(b))
    result = pd.concat([result, slack_b_df], axis=1)

    return result
```

`sbm_undesirable`函数通过冒号分隔期望产出与非期望产出（公式左侧写作"Y:CO2"），从而在解析阶段同时得到 `xcol`、`ycol`与 `bcol`三个变量列表，其余公式解析与数据准备的流程与基础SBM一致。相较于 12.1 节的基础SBM，该扩展主要体现在五处模型特有逻辑之上：在Charnes-Cooper转换约束的分母中以 $Q + R$ 取代 Q，将非期望产出纳入归一化项；相应地引入非期望产出集合 `M`与松弛变量 `slack_b`，并增列一条形如 $t\,B_0 = \sum \lambda_i B_i + S^B$ 的非期望产出约束；最后在结果中追加 `slack_b`列。除这几处外，函数的其余部分（公式解析、数据准备、求解与格式化）与基础SBM保持同一结构。

以下仍以本章省级面板第1至3期数据演示该函数的调用：

```
import pandas as pd
from deabook.example.book_cases.loaders import china

ex4 = china(scaled=True)
result = sbm_undesirable("Y:CO2 = K L", ex4, "t==[1,2,3]", "t==[1,2,3]")
print(result)
```

图12.2：含非期望产出SBM模型代码运行结果

与图12.1相比，此处效率值已将CO_2 等非期望产出纳入前沿构造，因而一个在传统SBM下有效的决策单元，一旦计入污染排放后可能转为无效——`obj`与 `slack CO2`共同揭示了环境维度的无效率来源。

12.3 公共工具模块的建立

12.3.1 从代码重复到模块化的动机

完成 12.1 节（SBM模型）和 12.2 节（含非期望产出的SBM模型）的编程练习后，细心的读者可能已经注意到一个不容忽视的事实：**两个函数的代码存在大量重复**。不妨先做一个简单的对比：

表 12.1: SBM与含非期望产出SBM的代码重复分析

代码段	SBM (12.1)	SBM+非期望产出 (12.2)	是否重复
导入语句 (`import ...`)	有	有	是

代码段	SBM (12.1)	SBM+非期望产出 (12.2)	是否重复
公式解析（字符串分割）	`split("=")`	`split("=")` + `split(":")`	类似
数据准备（query筛选）	相同逻辑	相同逻辑	是
数据验证（缺失值/正数检查）	`assert`语句	`assert`语句	是
求解器调用	`SolverFactory` + `solve`	`SolverFactory` + `solve`	是
结果格式化（DataFrame构造）	相同逻辑	相同逻辑	是
异常处理	`opt.available(exception_flag=False)` 检查	`opt.available(exception_flag=False)` 检查	是

由 表 12.1 可以清晰地看出，**约70%的代码是重复的**，真正体现模型差异的部分（非期望产出的约束定义和变量声明）只占约30%。这种重复带来了一系列问题：

1. **维护困难**：如果发现数据验证逻辑有bug，需要在两个函数中分别修改；
2. **一致性风险**：修改时可能遗漏某个函数，导致行为不一致；
3. **可读性下降**：读者需要在大量重复代码中寻找差异点；
4. **扩展成本高**：每增加一个新模型，都需要” 复制-粘贴-修改” 一整套流程。

软件工程中有一条著名的原则——**DRY原则**（Don’t Repeat Yourself，不要重复自己）。遵循这一原则，本节将两个示例中重复的代码提取为**公共工具函数**，构建一套共享的工具模块。后续所有模型（DDF、NDDF、Malmquist）都将复用这些工具，仅需关注各自模型特有的逻辑。

12.3.2 公共导入约定

本章后续各示例均依赖如下公共导入。在实际研究中，可将其置于脚本或Notebook起始处统一执行：

```
"""DEA实战演练——公共导入模块"""
from pyomo.environ import *
from pyomo.opt import SolverStatus, TerminationCondition
import pandas as pd
from deabook.example.book_cases.loaders import china
import numpy as np
import re
import warnings

warnings.filterwarnings("ignore")
```

12.3.3 公式字符串的解析

DEA模型通过公式字符串指定投入和产出变量。为了保证代码的健壮性，使用正则表达式解析公式，并添加输入规范化处理。

```
def normalize_formula(formula_str):
    """
    规范化公式字符串:统一分隔符、去除多余空格。

    Parameters
    ----------
    formula_str : str
        原始公式字符串,如 " Y  = K     L " 或 "Y:CO2 = K L"

    Returns
    -------
    str
        规范化后的公式字符串
    """
    # 统一全角符号为半角符号
    formula_str = formula_str.replace(": ", ":").replace("＝", "=")
    # 将多个连续空格替换为单个空格
    formula_str = re.sub(r'\s+', ' ', formula_str).strip()
    return formula_str


def parse_formula(formula_str):
    """
    解析DEA公式字符串,提取投入变量名和产出变量名。

    支持格式:
    - 基础格式: "产出 = 投入1 投入2"（如 "Y = K L"）
    - 含非期望产出格式: "产出:非期望产出 = 投入1 投入2"（如 "Y:CO2 = K L"）

    Parameters
    ----------
    formula_str : str
        公式字符串,等号左边为产出变量,右边为投入变量

    Returns
    -------
```

```
    tuple
        (xcol, ycol, bcol) —— 分别为投入变量列表、期望产出变量列表、
        非期望产出变量列表。若无非期望产出，bcol为空列表。
    """
    formula_str = normalize_formula(formula_str)

    parts = formula_str.split("=")
    if len(parts) != 2:
        raise ValueError(
            f"公式格式错误: '{formula_str}'。应包含一个 '=' 分隔符"
        )

    left_side = parts[0].strip()
    right_side = parts[1].strip()

    xcol = [v.strip() for v in right_side.split(" ") if v.strip()]

    if ":" in left_side:
        output_parts = left_side.split(":")
        ycol = [v.strip() for v in output_parts[0].split(" ") if v.strip()]
        bcol = [v.strip() for v in output_parts[1].split(" ") if v.strip()]
    else:
        ycol = [v.strip() for v in left_side.split(" ") if v.strip()]
        bcol = []

    return xcol, ycol, bcol
```

12.3.4 数据验证与准备

```
def validate_data(df, xcol, ycol, bcol=None):
    """
    验证投入产出数据是否符合DEA模型要求。

    Parameters
    ----------
    df : pd.DataFrame
        包含投入产出数据的DataFrame
    xcol : list of str
        投入变量列名列表
```

```
    ycol : list of str
        期望产出变量列名列表
    bcol : list of str or None, default None
        非期望产出变量列名列表

    Raises
    ------
    AssertionError
        数据不符合DEA模型要求时抛出
    """
    cols = xcol + ycol
    if bcol:
        cols = cols + bcol

    assert df[cols].notna().all().all(), \
        f"数据包含缺失值(NA),请检查以下变量: {cols}"
    assert (df[xcol] > 0).all().all(), \
        f"投入变量 {xcol} 中存在非正值。SBM和DDF模型要求投入数据严格为正。"
    assert (df[ycol] > 0).all().all(), \
        f"期望产出变量 {ycol} 中存在非正值。"
    if bcol:
        assert (df[bcol] >= 0).all().all(), \
            f"非期望产出变量 {bcol} 中存在负值。"


def prepare_data(formula, dataframe, eval_query=None, ref_query=None):
    """
    根据公式和查询条件准备评价数据和参照数据。

    Parameters
    ----------
    formula : str
        公式字符串
    dataframe : pd.DataFrame
        包含投入产出数据的DataFrame
    eval_query : str or None, default None
        筛选待评价决策单元的query字符串
    ref_query : str or None, default None
        筛选参照决策单元的query字符串
```

```
    Returns
    -------
    tuple
        (data, dataref, xcol, ycol, bcol)
    """
    xcol, ycol, bcol = parse_formula(formula)

    if eval_query is None:
        index_eval = dataframe.index
    else:
        index_eval = dataframe.query(eval_query).index

    if ref_query is None:
        index_ref = dataframe.index
    else:
        index_ref = dataframe.query(ref_query).index

    all_cols = xcol + ycol + (bcol if bcol else [])
    data = dataframe.loc[index_eval, all_cols]
    dataref = dataframe.loc[index_ref, all_cols]

    return data, dataref, xcol, ycol, bcol
```

12.3.5 求解与结果格式化

```
def solve_dea_model(model, solver_name="highs", timeout=60):
    """
    求解DEA模型并返回求解结果,包含完整的异常处理和状态检查。

    Parameters
    ----------
    model : pyomo.environ.ConcreteModel
        已构建的Pyomo模型对象
    solver_name : str, default "highs"
        求解器名称。推荐 "highs";备选 "glpk"、"ipopt"
    timeout : int, default 60
        求解超时时间(秒)

    Returns
```

```
    -------
    pyomo.opt.results.SolverResults
        求解器返回的结果对象

    Raises
    ------
    RuntimeError
        求解器未安装、求解失败或未达到最优时抛出
    """
    solver = SolverFactory(solver_name)

    if not solver.available(exception_flag=False):
        raise RuntimeError(
            f"求解器 '{solver_name}' 未安装或不可用。\n"
            f"安装建议: conda install {solver_name}"
        )

    results = solver.solve(model, timelimit=timeout, load_solutions=False)

    if results.solver.status != SolverStatus.ok:
        raise RuntimeError(f"求解器状态异常: {results.solver.status}")

    if results.solver.termination_condition != TerminationCondition.optimal:
        if (results.solver.termination_condition ==
            TerminationCondition.infeasible):
            raise RuntimeError("线性规划无可行解。请检查数据是否存在异常值。")
        else:
            raise RuntimeError(
                f"求解未达到最优: {results.solver.termination_condition}"
            )

    model.solutions.load_from(results)
    return results


def format_sbm_results(obj_dict, slack_x_dict, slack_y_dict,
                       xcol, ycol, bcol=None, slack_b_dict=None):
    """
    将SBM模型的求解结果格式化为标准DataFrame输出。
```

```
    Parameters
    ----------
    obj_dict : dict
        目标函数值字典
    slack_x_dict, slack_y_dict : dict
        松弛变量字典
    xcol, ycol : list of str
        变量名列表
    bcol : list of str or None, default None
        非期望产出变量名列表
    slack_b_dict : dict or None, default None
        非期望产出松弛变量字典

    Returns
    -------
    pd.DataFrame
        包含效率值和各松弛变量的结果DataFrame
    """
    obj_df = pd.DataFrame(obj_dict, index=["obj"]).T

    slack_x_df = pd.DataFrame(slack_x_dict, index=xcol).T
    slack_x_df.columns = slack_x_df.columns.map(lambda x: "slack " + str(x))

    slack_y_df = pd.DataFrame(slack_y_dict, index=ycol).T
    slack_y_df.columns = slack_y_df.columns.map(lambda y: "slack " + str(y))

    result = pd.concat([obj_df, slack_x_df, slack_y_df], axis=1)

    if bcol and slack_b_dict:
        slack_b_df = pd.DataFrame(slack_b_dict, index=bcol).T
        slack_b_df.columns = slack_b_df.columns.map(lambda b: "slack " + str(
            b))
        result = pd.concat([result, slack_b_df], axis=1)

    return result


def format_ddf_results(obj_dict, index_list):
    """
```

```
    将DDF/NDDF模型的求解结果格式化为标准DataFrame输出。

    Parameters
    ----------
    obj_dict : dict
        目标函数值字典
    index_list : list
        DMU索引列表

    Returns
    -------
    pd.DataFrame
        包含方向距离值的结果DataFrame
    """
    return pd.DataFrame(obj_dict, index=["te"]).T
```

12.3.6 求解器的选择

本章默认使用开源 HiGHS 求解线性规划，安装命令为 `python -m pip install highspy`。其他求解器须按其安装与许可要求配置；不预设不同机器上的速度排序。

求解器安装提示：关于各求解器的详细安装步骤，请参阅本书**附录A.1.3 节（求解器安装详细指南）**。以下代码片段可用于核验所需求解器是否可用：

```
# 验证求解器是否可用
from pyomo.environ import SolverFactory
for solver in ["highs", "glpk", "ipopt"]:
    opt = SolverFactory(solver)
    print(f"{solver}: {' 可用' if opt.available(exception_flag=False) else ' 不可用'}")
```

说明：公共工具模块的建立体现了软件工程思维在DEA编程中的应用。前两个完整示例已揭示重复代码带来的维护负担，12.4 节起后续各模型将复用本节定义的工具函数，仅聚焦于各模型特有的逻辑。研究者可将本节代码保存为独立的`.py`文件（如 `dea_utils.py`），以便在实际研究中重复调用。

12.4 DDF模型的程序编写

本节复用 12.3 节的公共工具函数，仅展示模型特有逻辑。

方向距离函数（Directional Distance Function, DDF）由 Chambers, Chung and Färe (1996, 1998) 提出，是测度技术效率的另一个常用工具；Chung, Färe and Grosskopf (1997) 则把它用于含非期望产出的生产技术，并在此基础上构造了Malmquist-Luenberger 生产率指数（见 12.6.5 节）。假设决策单元以 P 种投入要素 X 生产出 Q 种产出 Y，同时伴随产生 R 种非期望产出 B，方向距离函数的定义如下：

$$\vec{D}(X,Y,B;g)=\max\{\beta:(X,Y,B)+\beta\,g\circ(X,Y,B)\in T\}$$

其中 T 是生产技术集，$g=(g_X,g_Y,g_B)'$ 是投入变量、期望产出变量和非期望产出变量调整的方向向量。

本章（及配套代码）统一采用**无量纲方向系数**口径：g 的每个分量取自 $\{-1,0,1\}$，实际的调整幅度由该系数与被评价单元自身的观测值相乘得到。因此投影点写作 $X_0+\beta\,g_X\circ X_0$（"$\circ$" 表示逐分量相乘），例如 $g_X=-1$ 对应投入缩减到 $(1-\beta)X_0$。按这一口径，方向距离函数可以由下面的线性规划问题求解：

$$\begin{aligned}
\max_{\beta,\lambda}\quad & \beta \\
\text{s.t.}\quad & \sum_{i=1}^{N}\lambda_i X_i\le X_0+\beta\,g_X\circ X_0, \\
& \sum_{i=1}^{N}\lambda_i Y_i\ge Y_0+\beta\,g_Y\circ Y_0, \\
& \sum_{i=1}^{N}\lambda_i B_i=B_0+\beta\,g_B\circ B_0, \\
& \lambda_i\ge 0,\quad i=1,\dots,N.
\end{aligned}$$

若采用文献中常见的" 方向向量与变量同量纲" 口径（即 g 直接是一个投入产出向量），则上式右端为 $X_0+\beta g_X$，此时等比例调整对应 $g_X=-X_0$。两种写法在等比例设定下给出相同的 β，但 g 的取值含义不同，混用会把方向距离的单位读错。

在这个线性规划问题中：

(1) **变量**为$\{\beta,\lambda_i\}$，其中 β 是一维的，λ_i 的维度是 N；$\lambda_i\ge 0$，β 无约束；

(2) **目标函数**中变量 β 对应的系数为1；求最大值；

(3) **约束条件**有三个，分别对应投入、期望产出和非期望产出；约束符号分别为 $\le$、$\ge$ 和 $=$。

12.4.1 从SBM到DDF：模型逻辑的转变

在正式编写代码之前，有必要厘清DDF模型与 12.1 节SBM模型的根本差异，因为这决定了代码结构的显著不同。

第一，效率测度方式不同。SBM模型通过**松弛变量**直接度量各投入产出的超额或不足，其效率值是一个介于0和1之间的比值。而DDF模型通过**方向距离** β 来度量被评价DMU沿指定方向 g 到生产前沿的" 距离"。β 越大，表示该DMU距离前沿越远，效率越低。从经济学直觉上看，SBM回答的是" 各变量需要调整多少比例"，而DDF回答的是" 沿某个特定方向还能走多远"。

第二，目标函数的结构差异。SBM模型由于涉及分式形式，需要通过Charnes-Cooper转换引入额外的变量 t 和相应的转换约束。DDF模型的目标函数直接就是最大化 β，是一个标准的线性目标函数，无需任何转换。这大大简化了模型构建——不再需要处理CC转换变量 t，也无需在结果提取时进行 slack$/t$ 的逆变换。

第三，约束条件的方向性。SBM模型的所有约束都是等式（投入约束和产出约束均为 $=$），松弛变量以显式形式出现在约束中。DDF模型的约束则使用不等号：投入约束是 $\le$（参照单元的投入组合不超过被评价单元的投入加上方向调整），期望产出约束是 $\ge$（参照单元的产出组合不少于被评价单元的产出加上方

向调整），非期望产出约束是 $=$（通常假设非期望产出需要严格匹配）。这种不等式的使用意味着松弛变量被” 隐含” 在约束中，而非显式声明。

第四，方向向量的引入。DDF模型最大的特色是方向向量 g，它决定了效率改进的方向。在环境效率分析中，最典型的方向向量设定是**同时减少投入和非期望产出、增加期望产出**，按本节的无量纲系数口径即 $g_X = -1$，$g_Y = 1$，$g_B = -1$（等价于同量纲口径下的 $g_X = -X_0$、$g_Y = Y_0$、$g_B = -B_0$）。这一设定具有明确的经济学含义：β 可以解释为” 所有投入和非期望产出等比例减少 β、同时期望产出等比例增加 β“的潜在改进空间。

基于以上分析，可以预见 `ddf`函数的代码结构将与 `sbm`函数有显著差异：不再需要 `t`变量和CC转换约束，不再提取松弛变量，而是直接提取 β 作为效率度量。同时，函数需要接受额外的方向向量参数 `gx`、`gy`、`gb`，并提供合理的默认值。

```
def ddf(formula, dataframe, gx=None, gy=None, gb=None,
        eval_query=None, ref_query=None, solver="highs"):
    """
    计算方向距离函数(Directional Distance Function, DDF)效率值。

    基于Chambers, Chung and Färe (1996, 1998)的DDF模型(含非期望产出的
    应用见Chung, Färe and Grosskopf, 1997),支持投入、期望产出和非期望产出
    的自定义方向向量。

    Parameters
    ----------
    formula : str
        公式字符串,格式为 "产出:非期望产出 = 投入1 投入2 ..."
        例如: "Y:CO2 = K L"
    dataframe : pd.DataFrame
        待评价决策单元的投入产出数据框
    gx : list or None, default None
        投入方向向量。None表示默认 [-1, -1, ...](投入缩减)
    gy : list or None, default None
        期望产出方向向量。None表示默认 [1, 1, ...](产出扩张)
    gb : list or None, default None
        非期望产出方向向量。None表示默认 [-1, -1, ...](非期望产出缩减)
    eval_query : str or None, default None
        筛选待评价决策单元的query字符串
    ref_query : str or None, default None
        筛选参照决策单元的query字符串
    solver : str, default "highs"
        线性规划求解器名称
```

```
    Returns
    -------
    pd.DataFrame
        包含方向距离值(te列)的结果DataFrame

    Notes
    -----
    方向向量默认取:
    - gx = [-1, -1, ...]:投入按比例缩减
    - gy = [1, 1, ...]:期望产出按比例扩张
    - gb = [-1, -1, ...]:非期望产出按比例缩减
    """
    # 复用公共工具:解析公式并准备数据
    data, dataref, xcol, ycol, bcol = prepare_data(
        formula, dataframe, eval_query, ref_query
    )
    validate_data(dataframe, xcol, ycol, bcol)

    xref = dataref.loc[:, xcol]
    yref = dataref.loc[:, ycol]
    bref = dataref.loc[:, bcol]

    # 设置默认方向向量(若未提供)
    if gx is None:
        gx = [-1] * len(xcol)
    if gy is None:
        gy = [1] * len(ycol)
    if gb is None:
        gb = [-1] * len(bcol)

    obj_results = {}

    for j in data.index:
        x = data.loc[j, xcol]
        y = data.loc[j, ycol]
        b = data.loc[j, bcol]

        model = ConcreteModel()
        model.I = Set(initialize=dataref.index)
```

```
    model.K = Set(initialize=range(len(xcol)))
    model.L = Set(initialize=range(len(ycol)))
    model.M = Set(initialize=range(len(bcol)))

    # DDF特有变量:方向距离 beta（无约束）
    model.beta = Var(bounds=(None, None), within=Reals,
                     doc='directional distance')
    model.intensity = Var(model.I, bounds=(0.0, None),
                          doc='intensity variables')

    # 目标函数: maximize beta
    def objective_rule(model):
        return model.beta
    model.obj = Objective(rule=objective_rule, sense=maximize)

    # 投入约束: sum(lambda * xref) <= x + beta * gx * x
    def input_rule(model, k):
        return (sum(model.intensity[i] * xref.loc[i, xcol[k]]
                    for i in model.I)
                - model.beta * gx[k] * x.iloc[k] <= x.iloc[k])
    model.input = Constraint(model.K, rule=input_rule)

    # 期望产出约束: sum(lambda * yref) >= y + beta * gy * y
    def output_rule(model, l):
        return (-sum(model.intensity[i] * yref.loc[i, ycol[l]]
                     for i in model.I)
                + model.beta * gy[l] * y.iloc[l] <= -y.iloc[l])
    model.output = Constraint(model.L, rule=output_rule)

    # 非期望产出约束: sum(lambda * bref) = b + beta * gb * b
    def undesirable_output_rule(model, m):
        return (sum(model.intensity[i] * bref.loc[i, bcol[m]]
                    for i in model.I)
                - model.beta * gb[m] * b.iloc[m] == b.iloc[m])
    model.undesirable_output = Constraint(model.M,
        rule=undesirable_output_rule)

    # 求解并提取结果
    try:
```

```
            solution = solve_dea_model(model, solver)
            obj_results[j] = value(model.obj)
        except RuntimeError as e:
            print(f"DMU {j} 求解失败: {e}")
            obj_results[j] = np.nan

    # 复用公共工具:格式化输出
    return format_ddf_results(obj_results, data.index)
```

以下围绕若干关键设计选择，剖析 ddf函数相对于SBM模型的取舍。

关于方向向量的默认值设定（第42-48行）。当用户不提供方向向量时，函数自动采用”投入缩减、产出扩张、非期望产出缩减”的经典设定。具体来说，`gx = [-1] * len(xcol)`为每种投入分配-1的方向系数，这意味着在投入约束中，被评价DMU的右端项变为 $x + \beta \cdot (-1) \cdot x = x(1-\beta)$，即投入允许减少到原来的 $(1-\beta)$ 倍。同理，`gy = [1] * len(ycol)`使期望产出增加到原来的 $(1+\beta)$ 倍，`gb = [-1] * len(bcol)`使非期望产出减少到原来的 $(1-\beta)$ 倍。这种默认设定符合环境效率分析中”少投入、多产出、少污染”的直觉。

关于变量的声明（第61-65行）。DDF模型只声明了两个变量：`model.beta`和 `model.intensity`。与SBM模型相比，这里缺少了 `t`变量和各类松弛变量。`model.beta`被声明为 `within=Reals`且 `bounds=(None, None)`，即 β 是一个无约束的实数变量。理论上 $\beta \geq 0$（距离不能为负），但即使不显式设置下界，最优解也会自动满足非负性，因为目标函数是最大化 β 且 $\beta = 0$ 总是可行解（取 λ_i 使得参照单元恰好复制被评价单元即可）。

关于约束的数学转换（第71-91行）。代码中的约束形式与数学公式略有不同，这是为了适配Pyomo的标准形式 expression $\leq$ constant 或 expression $=$ constant。以投入约束为例，数学形式是 $\sum \lambda_i X_{ik} \leq X_{0k} + \beta g_{Xk} X_{0k}$，代码中将其整理为 $\sum \lambda_i X_{ik} - \beta g_{Xk} X_{0k} \leq X_{0k}$。这种等价变形使得所有与变量相关的项都在不等式左侧，常数项在右侧，符合线性规划的标准矩阵形式。

关于期望产出约束的符号处理（第81-85行）。细心的读者可能注意到，期望产出约束被改写成了”小于等于”形式，且 y 项带有负号。这是因为数学上的 $\geq$ 约束在移项后变成了 $-\sum \lambda_i Y_{ik} + \beta g_{Yk} Y_{0k} \leq -Y_{0k}$。Pyomo内部会将这种形式正确处理，但理解这种符号转换有助于调试约束错误。

关于异常处理的设计（第94-99行）。与SBM模型直接使用 `if`判断求解状态不同，`ddf`函数使用 `try-except`块捕获 `solve_dea_model`抛出的 `RuntimeError`。这种设计更加Pythonic——将求解和状态检查封装在工具函数中，模型函数只需关注”求解成功时记录结果，失败时记录NaN并打印警告”。这也体现了分层错误处理的思想：底层工具函数负责诊断具体问题（求解器未安装、无可行解、未达最优），上层模型函数负责决定失败后的业务逻辑（跳过该DMU、使用默认值等）。

以下给出 `ddf`函数的调用示例：

```
import pandas as pd
from deabook.example.book_cases.loaders import china
ex4 = china(scaled=True)
ddf_result = ddf("Y:CO2 = K L", ex4)
print(ddf_result)
```

图12.3：DDF模型代码运行结果

返回的 te列即为方向距离 β；$\beta = 0$ 表示决策单元已位于前沿、方向调整空间耗尽，而 β 越大意味着该单元沿既定方向距前沿越远、潜在改进幅度越大。

12.5 NDDF模型的程序编写

本节复用 12.3 节的公共工具函数，仅展示模型特有逻辑。

Zhou et al. (2012)提出的非径向方向距离函数（Non-radial Directional Distance Function, NDDF）允许投入和产出变量进行不同比例的调整。假设决策单元以 P 种投入要素 X 生产出 Q 种产出 Y，同时伴随产生 R 种非期望产出 B，NDDF的定义如下：

$$\vec{D}(X, Y, B; g) = \max\{w'\beta : (X, Y, B) + \beta' \text{diag}(g \circ (X, Y, B)) \in T\}$$

其中 $g = (g_X, g_Y, g_B)'$ 是方向系数向量（口径与 12.4 节一致：各分量取自 $\{-1, 0, 1\}$，实际调整幅度由该系数乘以被评价单元的观测值得到），$\beta = (\beta_X, \beta_Y, \beta_B)'$ 是各变量对应的调整因子向量，$w = (\omega_X, \omega_Y, \omega_B)'$ 是权重向量。利用数据包络分析的方法，NDDF可以由以下线性规划问题求解：

$$\begin{aligned}
&\max \quad w'\beta \\
\text{s.t.} \quad &\sum_{i=1}^{N} \lambda_i X_i \le x + \text{diag}(g_X \circ x)\beta_X, \\
&\sum_{i=1}^{N} \lambda_i Y_i \ge y + \text{diag}(g_Y \circ y)\beta_Y, \\
&\sum_{i=1}^{N} \lambda_i B_i = b + \text{diag}(g_B \circ b)\beta_B, \\
&\lambda_i \ge 0, \quad \beta \ge 0.
\end{aligned}$$

即第 k 个投入的约束为 $\sum_i \lambda_i X_{ik} \le x_k(1 + g_{Xk}\beta_{Xk})$，与代码中的`- gx[k] * x.iloc[k] * model.beta_x[k] <= x.iloc[k]`一一对应。

NDDF与DDF的主要区别在于：1. **每个变量有独立的调整因子** $\beta_X, \beta_Y, \beta_B$（而非单一的 β）2. **目标函数是加权求和** $w'\beta$ 3. **所有调整因子非负**

12.5.1 从DDF到NDDF：非径向化的设计哲学

如果说DDF模型是DEA效率测度的" 径向版本"——所有投入按同一比例 β 调整、所有期望产出按同一比例 β 调整，那么NDDF模型则解除了这一" 等比例" 约束，允许每种投入、每种产出拥有**各自独立的调整因子**。这种非径向化的设计具有重要的经济学意义：在现实生产中，不同投入要素（如资本和劳动）的替代弹性各不相同，不同产出（如GDP和CO_2）的减排成本也存在差异，强制它们按相同比例调整显然过于粗糙。NDDF通过为每种变量分配独立的 β 和权重 w，能够更精确地刻画技术效率的结构性特征。**权重向量 w 的设计考量**是NDDF模型编程中的核心问题。从数学上看，w 的作用是聚合各变量的调整因子为一个标量目标值。权重设定的经济学含义是：研究者对不同变量改进的**相对重视程度**。例如，在环境效率评价中，如果研究者认为减少CO_2 排放比减少资本投入更为重要，就可以给CO_2 对应的 β_B 赋予更大的权重。当用户不提供权重时，函数采用**等权重**作为默认设定（代码第52-65行）。等权重的计算逻辑是：首先统计非零方

向向量的变量类型数(投入、期望产出、非期望产出三类中哪些参与了方向调整),然后将总权重1均分给各类变量,再在各类型内部均分给每种具体变量。例如,当投入、期望产出、非期望产出三类均参与调整且各有1种变量时,每种变量获得的权重为 $1/3$。这种设定保证了目标函数具有对称性和可比性,且当方向向量变化时权重自动适应。**目标函数中系数符号的处理**需要特别关注(代码第72-78行)。观察代码可以发现,目标函数表达式为:

```
-sum(w_x[k] * gx[k] * model.beta_x[k])
+ sum(w_y[l] * gy[l] * model.beta_y[l])
- sum(w_b[m] * gb[m] * model.beta_b[m])
```

为什么投入项和非期望产出项带有负号,而期望产出项带有正号?这是因为 β 的非负约束与方向向量的符号共同决定了改进方向。当 $g_X = -1$(投入缩减)时,$-w_X \cdot (-1) \cdot \beta_X = w_X \beta_X$,目标函数中该项为正值,最大化目标函数意味着鼓励投入调整因子 β_X 增大。同理,当 $g_Y = 1$(产出扩张)时,$w_Y \cdot 1 \cdot \beta_Y = w_Y \beta_Y$ 也为正值。这种符号设计确保了最大化 $w'\beta$ 等价于最大化各变量的有效改进幅度。**从变量声明看模型结构差异。**NDDF模型声明了三种独立的调整因子变量(`beta_x`、`beta_y`、`beta_b`),分别对应投入、期望产出和非期望产出,每种都是向量形式(以各自变量集合为索引)。这与DDF模型中单一的 `beta`标量形成鲜明对比。此外,NDDF要求所有 $\beta \geq 0$,因此在变量声明时显式设置了 `bounds=(0.0, None)`。

```
def nddf(formula, dataframe, gx=None, gy=None, gb=None,
         weight=None, eval_query=None, ref_query=None, solver="highs"):
    """
    计算非径向方向距离函数(NDDF)效率值(Zhou et al., 2012)。

    NDDF允许投入和产出进行不同比例的调整,通过权重向量w对各调整因子
    进行加权求和作为目标函数。

    Parameters
    ----------
    formula : str
        公式字符串,格式为 "产出:非期望产出 = 投入1 投入2 ..."
    dataframe : pd.DataFrame
        待评价决策单元的投入产出数据框
    gx, gy, gb : list or None, default None
        投入、期望产出、非期望产出的方向向量。None时使用默认值
    weight : list or None, default None
        权重向量 [w_X, w_Y, w_B]。None时自动计算等权重
    eval_query : str or None, default None
        筛选待评价决策单元的query字符串
    ref_query : str or None, default None
        筛选参照决策单元的query字符串
    solver : str, default "highs"
```

```
        线性规划求解器名称

    Returns
    -------
    pd.DataFrame
        包含目标函数值(obj)和各变量调整因子(scale)的结果DataFrame
    """
    # 复用公共工具:解析公式并准备数据
    data, dataref, xcol, ycol, bcol = prepare_data(
        formula, dataframe, eval_query, ref_query
    )
    validate_data(dataframe, xcol, ycol, bcol)

    xref = dataref.loc[:, xcol]
    yref = dataref.loc[:, ycol]
    bref = dataref.loc[:, bcol]

    # 设置默认方向向量
    if gx is None:
        gx = [-1] * len(xcol)
    if gy is None:
        gy = [1] * len(ycol)
    if gb is None:
        gb = [-1] * len(bcol)

    # 设置默认权重(等权重)
    if weight is None:
        weight = []
        # 计算非零方向向量的种类数
        num_directions = (
            int(gx[0] != 0) + int(gy[0] != 0) + int(gb[0] != 0)
        )
        # 每种变量类型内部等权重
        for _ in range(len(xcol)):
            weight.append(1.0 / num_directions / len(xcol))
        for _ in range(len(ycol)):
            weight.append(1.0 / num_directions / len(ycol))
        for _ in range(len(bcol)):
            weight.append(1.0 / num_directions / len(bcol))
```

```
# 分离各变量类型的权重
w_x = weight[0:len(xcol)]
w_y = weight[len(xcol):len(xcol) + len(ycol)]
w_b = weight[len(xcol) + len(ycol):len(xcol) + len(ycol) + len(bcol)]

# 初始化结果存储
beta_x_results = {}
beta_y_results = {}
beta_b_results = {}
obj_results = {}

for j in data.index:
    x = data.loc[j, xcol]
    y = data.loc[j, ycol]
    b = data.loc[j, bcol]

    model = ConcreteModel()
    model.I = Set(initialize=dataref.index)
    model.K = Set(initialize=range(len(xcol)))
    model.L = Set(initialize=range(len(ycol)))
    model.M = Set(initialize=range(len(bcol)))

    # NDDF特有变量:各变量类型的独立调整因子
    model.beta_x = Var(model.K, bounds=(0.0, None), doc='scale factor x')
    model.beta_y = Var(model.L, bounds=(0.0, None), doc='scale factor y')
    model.beta_b = Var(model.M, bounds=(0.0, None), doc='scale factor b')
    model.intensity = Var(model.I, bounds=(0.0, None),
                          doc='intensity variables')

    # NDDF特有目标函数:加权求和 w' * beta
    def objective_rule(model):
        return (
            -sum(w_x[k] * gx[k] * model.beta_x[k] for k in model.K)
            + sum(w_y[l] * gy[l] * model.beta_y[l] for l in model.L)
            - sum(w_b[m] * gb[m] * model.beta_b[m] for m in model.M)
        )
    model.obj = Objective(rule=objective_rule, sense=maximize)

    # 投入约束
```

```
        def input_rule(model, k):
            return (sum(model.intensity[i] * xref.loc[i, xcol[k]]
                        for i in model.I)
                    - gx[k] * x.iloc[k] * model.beta_x[k] <= x.iloc[k])
        model.input = Constraint(model.K, rule=input_rule)

        # 期望产出约束
        def output_rule(model, l):
            return (-sum(model.intensity[i] * yref.loc[i, ycol[l]]
                         for i in model.I)
                    + gy[l] * y.iloc[l] * model.beta_y[l] <= -y.iloc[l])
        model.output = Constraint(model.L, rule=output_rule)

        # 非期望产出约束
        def undesirable_output_rule(model, m):
            return (sum(model.intensity[i] * bref.loc[i, bcol[m]]
                        for i in model.I)
                    - gb[m] * b.iloc[m] * model.beta_b[m] == b.iloc[m])
        model.undesirable_output = Constraint(model.M,
            rule=undesirable_output_rule)

        # 求解并提取结果
        try:
            solution = solve_dea_model(model, solver)

            beta_x_results[j] = np.asarray(list(model.beta_x[:].value))
            beta_y_results[j] = np.asarray(list(model.beta_y[:].value))
            beta_b_results[j] = np.asarray(list(model.beta_b[:].value))
            obj_results[j] = value(model.obj)

        except RuntimeError as e:
            print(f"DMU {j} 求解失败: {e}")
            beta_x_results[j] = np.full(len(xcol), np.nan)
            beta_y_results[j] = np.full(len(ycol), np.nan)
            beta_b_results[j] = np.full(len(bcol), np.nan)
            obj_results[j] = np.nan

    # 格式化输出
    obj_df = pd.DataFrame(obj_results, index=["obj"]).T
```

```
    beta_x_df = pd.DataFrame(beta_x_results, index=xcol).T
    beta_x_df.columns = beta_x_df.columns.map(lambda x: "scale " + str(x))

    beta_y_df = pd.DataFrame(beta_y_results, index=ycol).T
    beta_y_df.columns = beta_y_df.columns.map(lambda y: "scale " + str(y))

    beta_b_df = pd.DataFrame(beta_b_results, index=bcol).T
    beta_b_df.columns = beta_b_df.columns.map(lambda b: "scale " + str(b))

    result = pd.concat([obj_df, beta_x_df, beta_y_df, beta_b_df], axis=1)
    return result
```

12.5.2 结果解读与使用建议

NDDF函数返回的结果DataFrame包含 `obj`（加权目标函数值）和各变量的 `scale`值（调整因子）。与DDF模型返回的单一 β 不同，NDDF为每种投入和产出变量都提供了独立的调整因子，这意味着研究者可以进行**结构性效率分析**。例如，如果某DMU的 `scale K`远大于 `scale L`，说明该DMU的资本投入冗余比劳动投入更为严重，政策建议应优先针对资本配置进行优化。在结果格式化部分（代码第118-130行），使用 `scale` 作为列名前缀（而非DDF中的 `te`或SBM中的 `slack`），以强调这些值是" 调整比例因子" 而非直接的距离或松弛度量。这种命名约定有助于用户正确理解结果的经济学含义。以下给出 `nddf`函数的调用示例：

```
import pandas as pd
from deabook.example.book_cases.loaders import china

ex4 = china(scaled=True)
nddf_result = nddf("Y:CO2 = K L", ex4)
print(nddf_result)
```

图12.4：NDDF模型代码运行结果

12.6 Malmquist生产率指数的程序编写

Malmquist生产率指数是衡量跨期生产率变化的重要工具。本节将产出导向与投入导向的Malmquist指数合并讲解，并给出Malmquist分解公式及其代码实现。

12.6.1 Malmquist指数的基本原理

产出方向的Malmquist生产率指数定义如下：

$$M(X_t, Y_t, X_{t+1}, Y_{t+1}) = \left[\frac{D^t(X_{t+1}, Y_{t+1})}{D^t(X_t, Y_t)} \cdot \frac{D^{t+1}(X_{t+1}, Y_{t+1})}{D^{t+1}(X_t, Y_t)} \right]^{1/2}$$

其中 $D^t(\cdot)$ 和 $D^{t+1}(\cdot)$ 分别表示以时期 t 和时期 $t+1$ 的技术为参照的产出距离函数。计算Malmquist指数需要估计**四个**距离函数：

距离函数	被评价DMU	参照前沿	含义
$D^t(X_t, Y_t)$	时期 t	时期 t 前沿	时期 t 的同期效率
$D^{t+1}(X_{t+1}, Y_{t+1})$	时期 $t+1$	时期 $t+1$ 前沿	时期 $t+1$ 的同期效率
$D^t(X_{t+1}, Y_{t+1})$	时期 $t+1$	时期 t 前沿	**混合效率**（跨期）
$D^{t+1}(X_t, Y_t)$	时期 t	时期 $t+1$ 前沿	**混合效率**（跨期）

图12.5：Malmquist指数的四类距离函数示意

```
时期 t 的效率                          时期 t+1 的效率
被评价：时期 t DMU                     被评价：时期 t+1 DMU
参照前沿：V(t)                         参照前沿：V(t+1)
├─ D^t(X_t, Y_t)                      ├─ D^{t+1}(X_{t+1}, Y_{t+1})
│                                     │
混合效率（跨期前沿）                    混合效率（跨期前沿）
被评价：时期 t+1 DMU                   被评价：时期 t DMU
参照前沿：V(t)                         参照前沿：V(t+1)
├─ D^t(X_{t+1}, Y_{t+1})              ├─ D^{t+1}(X_t, Y_t)
```

跨期前沿的构造方法：混合效率计算中，被评价DMU和参照DMU来自不同时期。在代码实现中，这通过分别指定 `eval_query`（被评价DMU）和 `ref_query`（参照DMU）来实现。例如，$D^t(X_{t+1}, Y_{t+1})$ 的 `eval_query`为 `t==t+1`，`ref_query`为 `t==t`。

12.6.2 Malmquist分解

Malmquist指数可以分解为**效率变化**（efficiency change，追赶效应）和**技术变化**（technical change，前沿移动）两个部分：

$$
\begin{aligned}
M &= EC \times TC, \\
EC &= \frac{D^{t+1}(X_{t+1}, Y_{t+1})}{D^t(X_t, Y_t)}, \\
TC &= \left[\frac{D^t(X_{t+1}, Y_{t+1})}{D^{t+1}(X_{t+1}, Y_{t+1})} \cdot \frac{D^t(X_t, Y_t)}{D^{t+1}(X_t, Y_t)} \right]^{1/2}.
\end{aligned}
$$

其中，EC 为两个时期同期效率的比值，反映效率追赶；TC 比较同一观测值在两个时期前沿下的距离，反映技术前沿的移动。

经济学解释：- 效率变化 (EC) > 1：DMU向生产前沿靠近（管理水平提升）- **技术变化 (TC) > 1：**生产前沿向外扩张（技术进步）- **Malmquist指数(MPI) > 1：**总生产率提升

12.6.3 从距离函数到代码实现：Malmquist计算的整体架构

在深入代码之前，先从宏观层面理解Malmquist指数计算的**架构设计**。与SBM、DDF、NDDF等单期效率模型不同，Malmquist指数是一个" 元模型"——它本身不直接定义新的线性规划形式，而是**复用现有的距离函数模型**（如谢泼德距离函数），通过组合多个不同时间设定下的效率值来构造跨期生产率指标。

这种" 元模型" 的设计带来了独特的编程挑战。首先，Malmquist指数的计算不是" 一次线性规划求解"，而是**多次线性规划求解的编排**——对于 T 个时期的面板数据，需要求解约 $3T$ 次（当期效率 T 次、两种混合效率各 $T-1$ 次）。其次，每次求解的 `eval_query`和 `ref_query`组合不同，需要根据生产技术设定（当期、时序、全局、视窗）动态生成查询字符串。最后，求解结果需要按照时间顺序对齐，通过Pandas的 `shift`操作进行时序匹配。

基于以上分析，Malmquist指数的代码实现自然地划分为三个层次：**基础距离函数层**（`dea8`函数，负责单次DEA求解）、**距离函数计算层**（`mpi`函数中的三个循环，负责生成所有需要的距离函数值）、**指数计算层**（`mpi`函数末尾的公式计算，负责组合距离函数并分解为EC和TC）。这种分层设计使代码逻辑清晰，也便于后续扩展（如增加新的生产技术设定或导向类型）。

12.6.4 基础DEA函数

Malmquist指数的计算依赖于基础的距离函数估计函数。本节复用并改进第 11 章的 `dea8`函数：

```
def dea8(formula, dataframe, rts, orient,
         eval_query=None, ref_query=None, solver="highs"):
    """
    计算谢泼德距离函数效率值(Malmquist计算的基础函数)。

    支持CRS/VRS规模假设和产出导向(oo)/投入导向(io)两种方向。

    Parameters
    ----------
    formula : str
        公式字符串,如 "Y = K L"
    dataframe : pd.DataFrame
        投入产出数据框
    rts : str
        规模报酬假设: "crs"（不变）或 "vrs"（可变）
    orient : str
        导向类型: "oo"（产出导向）或 "io"（投入导向）
    eval_query : str or None, default None
        筛选待评价决策单元的query字符串
    ref_query : str or None, default None
        筛选参照决策单元的query字符串
    solver : str, default "highs"
```

```
    求解器名称

Returns
-------
pd.DataFrame
    包含theta值和技术效率值(te)的DataFrame
"""
# 复用公共工具:解析公式并准备数据
data, dataref, xcol, ycol, _ = prepare_data(
    formula, dataframe, eval_query, ref_query
)

xref = dataref.loc[:, xcol]
yref = dataref.loc[:, ycol]

theta_results = {}   # 存储各DMU的theta值(原thetalt)

for j in data.index:
    x = data.loc[j, xcol]
    y = data.loc[j, ycol]

    model = ConcreteModel()
    model.I = Set(initialize=dataref.index)
    model.K = Set(initialize=range(len(xcol)))
    model.L = Set(initialize=range(len(ycol)))

    model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
    model.intensity = Var(model.I, bounds=(0.0, None),
        doc='intensity variables')

    # 目标函数:根据导向确定最大/最小化
    def obj_rule(model):
        return model.theta
    model.obj = Objective(
        rule=obj_rule,
        sense=(maximize if orient == "oo" else minimize)
    )

    # 投入约束:根据导向变化
```

```
        def input_rule(model, k):
            if orient == "oo":       # 产出导向:投入不超过被评价单元
                return sum(
                    model.intensity[i] * xref.loc[i, xcol[k]] for i in model.I
                ) <= x.iloc[k]
            elif orient == "io":     # 投入导向:投入不超过theta倍被评价单元
                return sum(
                    model.intensity[i] * xref.loc[i, xcol[k]] for i in model.I
                ) <= model.theta * x.iloc[k]

        # 产出约束:根据导向变化
        def output_rule(model, l):
            if orient == "oo":       # 产出导向:产出不低于theta倍被评价单元
                return sum(
                    model.intensity[i] * yref.loc[i, ycol[l]] for i in model.I
                ) >= model.theta * y.iloc[l]
            elif orient == "io":     # 投入导向:产出不低于被评价单元
                return sum(
                    model.intensity[i] * yref.loc[i, ycol[l]] for i in model.I
                ) >= y.iloc[l]

        # VRS约束
        def vrs_rule(model):
            return sum(model.intensity[i] for i in model.I) == 1

        model.input = Constraint(model.K, rule=input_rule)
        model.output = Constraint(model.L, rule=output_rule)

        if rts == "vrs":
            model.vrs = Constraint(rule=vrs_rule)

        # 求解
        try:
            solution = solve_dea_model(model, solver)
            theta = np.asarray(list(model.theta[:].value))
            theta_results[j] = theta[0]
        except RuntimeError as e:
            print(f"DMU {j} 求解失败: {e}")
            theta_results[j] = np.nan
```

```
    # 格式化输出
    thetadf = pd.DataFrame(theta_results, index=["theta"]).T
    # 技术效率:产出导向取倒数,投入导向直接取theta
    thetadf["te"] = (
        1.0 / thetadf["theta"] if orient == "oo" else thetadf["theta"]
    )
    return thetadf
```

12.6.5 dea8函数的设计解析

dea8函数是Malmquist指数计算的" 基石",它封装了经典的**谢泼德距离函数**模型(Shephard distance function),支持两种规模报酬假设(CRS/VRS)和两种导向(产出导向/投入导向)。理解该函数的设计,是理解整个Malmquist计算框架的关键。

导向类型对约束条件的影响是dea8函数设计的核心。产出导向(output-oriented)和投入导向(input-oriented)在约束条件上呈现" 对偶" 关系:

在**产出导向**下,投入约束保持固定(参照单元的投入组合不超过被评价单元的实际投入),而产出约束要求参照单元的产出达到被评价单元产出的 θ 倍($\theta \geq 1$)。此时 θ 衡量的是" 产出可以扩大多少倍",技术效率取其倒数 $1/\theta$ (使得效率值落在 $(0, 1]$ 区间)。

在**投入导向**下,产出约束保持固定(参照单元的产出组合不少于被评价单元的实际产出),而投入约束要求参照单元的投入不超过被评价单元投入的 θ 倍($\theta \leq 1$)。此时 θ 直接就是技术效率值,无需取倒数。

这种约束条件的" 交叉" 设计体现在代码第51-71行:投入约束在产出导向时使用 $\leq x_{0k}$,在投入导向时使用 $\leq \theta \cdot x_{0k}$;产出约束在产出导向时使用 $\geq \theta \cdot y_{0l}$,在投入导向时使用 $\geq y_{0l}$。通过 `if orient == "oo"`的条件判断,同一套代码框架可以同时支持两种导向。

VRS约束的条件化添加(代码第77-78行)体现了DEA模型中规模报酬假设的灵活性。当 `rts="vrs"`时,添加 $\sum \lambda_i = 1$ 约束;当 `rts="crs"`时,不添加此约束。值得注意的是, Malmquist指数的标准定义基于CRS假设,因此 `mpi`函数在调用 `dea8`时固定传入`"crs"`(参见 12.6.6 节代码第37、56、74行)。VRS支持的存在使得 `dea8`函数可以作为通用工具,在Malmquist之外的其他分析场景中复用。

技术效率的换算逻辑(代码第88-90行)需要特别注意。产出导向时返回的 `theta`是产出扩张倍数(≥ 1),而研究者通常期望的效率值是 $\in (0, 1]$ 的形式,因此需要取倒数。投入导向时 `theta`直接就是效率值(≤ 1),无需转换。这种换算保证了无论使用哪种导向,最终 `te`列的数值都在同一尺度上可比。

12.6.6 产出导向的Malmquist指数

```
def mpi(formula, data, id_var, t_var, orient="oo", tech=None):
    """
    计算Malmquist生产率指数(支持多种生产技术设定)。

    基于Fare et al. (1994)的Malmquist指数,支持当期(com)、时序(seq)、
    视窗(window)和全局(global)四种生产技术设定。
```

```
    Parameters
    ----------
    formula : str
        公式字符串,如 "Y = K L"
    data : pd.DataFrame
        面板数据(包含id_var和t_var)
    id_var : str
        个体标识变量名
    t_var : str
        时间标识变量名
    orient : str, default "oo"
        导向类型: "oo"(产出导向)或 "io"(投入导向)
    tech : str or None, default None
        生产技术设定:
        - None / "com":当期生产技术(默认)
        - "seq":时序生产技术
        - "window h":视窗生产技术(h为窗宽)
        - "global":全局生产技术

    Returns
    -------
    pd.DataFrame
        原始数据附加Malmquist指数及其分解(effch, techch, mpi)

    Examples
    --------
    >>> ex4 = china(scaled=True)
    >>> result = mpi("Y = K L", ex4, "id", "t", "oo", "com")
    >>> result_seq = mpi("Y = K L", ex4, "id", "t", "oo", "seq")
    >>> result_win = mpi("Y = K L", ex4, "id", "t", "oo", "window 2")
    """
    # 获取时间序列(去重并排序)
    t_series = pd.Series(data[t_var]).drop_duplicates().sort_values()
    t_values = t_series.tolist()

    # ---------- 计算 D^t(X_t, Y_t):同期效率 ----------
    D11_list = []
    for t_val in t_values:
        eval_q = "{}=={}".format(t_var, t_val)
```

```
    # 根据生产技术设定确定参照集
    if tech is None or tech == "com":
        ref_q = "{}=={}".format(t_var, t_val)
    elif tech == "seq":
        ref_q = "{}<={}".format(t_var, t_val)
    elif tech == "global":
        ref_q = None
    elif "window " in str(tech):
        h = int(tech.split(" ")[1])
        t_idx = t_values.index(t_val)
        t_min = t_values[max(0, t_idx - h)]
        t_max = t_values[min(len(t_values) - 1, t_idx + h)]
        ref_q = "{}<={}<={}".format(t_min, t_var, t_max)

    D11 = dea8(formula, data, "crs", orient, eval_q, ref_q)[["te"]]
    D11_list.append(D11)

D11_df = pd.concat(D11_list)
D11_df.rename(columns={"te": "D11"}, inplace=True)

# 全局技术只需D11即可计算
if tech == "global":

    result = pd.merge(data, D11_df, left_index=True, right_index=True,
                      how="left")
    result["mpi"] = (
        result["D11"] / result.groupby(id_var)["D11"].shift(1)
    )
    result.drop(columns=["D11"], inplace=True)
    return result

# ---------- 计算 D^t(X_{t+1}, Y_{t+1}):混合效率 ----------
D12_list = []
for t_val in t_values[1:]:   # 从第2个时期开始
    eval_q = "{}=={}".format(t_var, t_val)

    if tech is None or tech == "com":
        ref_q = "{}=={}".format(t_var, t_values[t_values.index(t_val) - 1])
    elif tech == "seq":
```

```
            ref_q = "{}<={}".format(t_var, t_values[t_values.index(t_val) - 1])
        elif "window " in str(tech):
            h = int(tech.split(" ")[1])
            t_idx = t_values.index(t_val)
            t_min = t_values[max(0, t_idx - 1 - h)]
            t_max = t_values[min(len(t_values) - 1, t_idx - 1 + h)]
            ref_q = "{}<={}<={}".format(t_min, t_var, t_max)

        D12 = dea8(formula, data, "crs", orient, eval_q, ref_q)[["te"]]
        D12_list.append(D12)

    D12_df = pd.concat(D12_list)
    D12_df.rename(columns={"te": "D12"}, inplace=True)

    # ---------- 计算 D^{t+1}(X_t, Y_t):混合效率 ----------
    D21_list = []
    for t_val in t_values[1:]:
        eval_q = "{}=={}".format(t_var, t_values[t_values.index(t_val) - 1])

        if tech is None or tech == "com":
            ref_q = "{}=={}".format(t_var, t_val)
        elif tech == "seq":
            ref_q = "{}<={}".format(t_var, t_val)
        elif "window " in str(tech):
            h = int(tech.split(" ")[1])
            t_idx = t_values.index(t_val)
            t_min = t_values[max(0, t_idx - h)]
            t_max = t_values[min(len(t_values) - 1, t_idx + h)]
            ref_q = "{}<={}<={}".format(t_min, t_var, t_max)

        D21 = dea8(formula, data, "crs", orient, eval_q, ref_q)[["te"]]
        D21_list.append(D21)

    D21_df = pd.concat(D21_list)
    D21_df.rename(columns={"te": "D21"}, inplace=True)

    # ---------- 合并并计算Malmquist指数及其分解 ----------
    result_df = pd.concat([D11_df, D12_df, D21_df], axis=1)
    result = pd.merge(data, result_df, left_index=True, right_index=True,
```

```
                    how="left")

    # Malmquist指数
    # dea8 在两种导向下返回的 te 都是取值于 (0,1] 的效率:
    #   产出导向 te = 1/theta = 谢泼德产出距离 D_O;
    #   投入导向 te = theta = Farrell 投入效率 = 1/D_I。
    # 因此两种导向共用同一套比率公式,效率改善时 mpi>1,与 effch 方向一致。
    # MPI = [D^t(X_{t+1},Y_{t+1}) / D^t(X_t,Y_t)
    #        * D^{t+1}(X_{t+1},Y_{t+1}) / D^{t+1}(X_t,Y_t)]^{1/2}
    # 即 sqrt(D12/D11.shift(1) * D11/D21.shift(1))
    result["mpi"] = np.sqrt(result["D12"] / result.groupby(id_var)["D11"].shift(1)
                            * result["D11"] / result.groupby(id_var)["D21"].shift(1))

    # Malmquist分解
    # 效率变化 EC = D_{t+1}(X_{t+1},Y_{t+1}) / D_t(X_t,Y_t)
    # 即 同一DMU相邻期 D11 之比(按个体分组 shift,首期为 NaN)
    result["effch"] = result["D11"] / result.groupby(id_var)["D11"].shift(1)

    # 技术变化 TC = MPI / EC
    result["techch"] = result["mpi"] / result["effch"]

    # 清理中间结果列
    result.drop(columns=["D11", "D12", "D21"], inplace=True)

    return result
```

12.6.7 mpi函数的深度解析:从理论公式到代码实现

mpi函数是整个Malmquist计算框架的" 指挥中心",其代码虽较长,但逻辑层次分明。下面逐层剖析。

第一层:时间序列的提取与对齐(代码第37-39行)。函数首先对 t_var列进行去重和排序,得到完整的时间序列 t_values。这一操作至关重要,因为后续所有循环都依赖 t_values的顺序来正确匹配" 时期 t"和" 时期 $t+1$"。如果数据中存在时间跳跃(如2010、2012、2015), t_values会忠实反映这种非均匀间隔,代码会自动处理,无需假设时间等距。

第二层:同期效率 $D^t(X_t, Y_t)$ 的计算(代码第42-66行,标记为D11)。这是Malmquist计算中最直观的环节——对每个时期 t,被评价DMU和参照DMU都来自同一时期。代码通过循环遍历 t_values,为每个时期构造 eval_q和 ref_q。关键技术设定差异体现在 ref_q的构造上:

- **当期技术(com)**: ref_q与 eval_q相同,只使用时期 t 的DMU构建前沿。这是最严格的设定,假设技术在不同期间完全独立。
- **时序技术(seq)**: ref_q使用"{}<={}",包含从第一期到时期 t 的所有DMU。这种设定假设技术具有累积性,过去的技术在当期仍然可用。

- **全局技术（global）**：`ref_q=None`，使用全部时期的全部DMU构建一个统一的前沿。这是最不严格的设定，假设所有技术在所有时期都可用。
- **视窗技术（window）**：`ref_q`使用以当前时期为中心、窗宽为 h 的时间窗口。这种设定在当期和全局之间取得平衡，假设技术在相邻时期之间可以共享。

第三层：混合效率的计算（代码第69-102行，标记为D12和D21）。混合效率是Malmquist指数的精髓所在，它衡量的是"跨期技术适应性"。$D^t(X_{t+1}, Y_{t+1})$（D12）回答的问题是：如果用时期 t 的技术来评价时期 $t+1$ 的DMU，效率会如何？$D^{t+1}(X_t, Y_t)$（D21）则反之。这两种混合效率共同构成了技术变化（TC）的度量基础。

在代码实现上，D12和D21的循环从 `t_values[1:]`开始（即跳过第一个时期），因为混合效率需要"时期 t 和时期 $t+1$"两个时间点的数据。每种生产技术设定下，`eval_q`和 `ref_q`的组合逻辑需要仔细对照——例如，在当期技术设定下，D12的 `eval_q`是 `t==t_{k}`而 `ref_q`是 `t==t_{k-1}`；D21则正好相反。

第四层：全局技术的特殊处理（代码第54-58行）。全局技术（global）是四种设定中最简单的一种——由于所有时期的DMU共享同一个全局前沿，混合效率的计算变得多余。在全局技术下，Malmquist指数简化为相邻时期同期效率的比值：$M = D^{t+1}(X_{t+1}, Y_{t+1}) / D^t(X_t, Y_t)$。因此，代码在第54行检测 `tech == "global"`后，直接计算 `mpi`并返回，跳过D12和D21的计算。

第五层：Malmquist指数的计算与分解（代码第105-121行）。这是理论公式与代码实现衔接最紧密的部分。下面验证公式与代码的对应关系——由于 `dea8`在两种导向下返回的 `te`都是取值于 $(0, 1]$ 的效率（产出导向为 D_O，投入导向为Farrell投入效率 $1/D_I$），**同一套比率公式同时适用于产出导向与投入导向**，代码中不再按导向分支：

Malmquist指数的数学定义是（式中 D 统一表示上述 $(0, 1]$ 区间的效率度量）：

$$M = \left[\frac{D^t(X_{t+1}, Y_{t+1})}{D^t(X_t, Y_t)} \cdot \frac{D^{t+1}(X_{t+1}, Y_{t+1})}{D^{t+1}(X_t, Y_t)} \right]^{1/2}$$

代码中的实现是（第111-112行）：

```
result["mpi"] = np.sqrt(result["D12"] / result.groupby(id_var)["D11"].shift(1)
                        * result["D11"] / result.groupby(id_var)["D21"].shift(
                            1))
```

下面逐项对应：- $D^t(X_{t+1}, Y_{t+1})$ → `D12`（时期 $t+1$ 的DMU到时期 t 前沿的距离）- $D^t(X_t, Y_t)$ → `D11.shift(1)`（时期 t 的DMU到时期 t 前沿的距离，按个体分组的shift(1)将其与同一DMU的D12对齐到同一行）- $D^{t+1}(X_{t+1}, Y_{t+1})$ → `D11`（时期 $t+1$ 的DMU到时期 $t+1$ 前沿的距离）- $D^{t+1}(X_t, Y_t)$ → `D21.shift(1)`（时期 t 的DMU到时期 $t+1$ 前沿的距离）

代码中通过 `np.sqrt`显式取平方根（$[\cdot]^{1/2}$），与上面的数学定义严格一致。开方是定义的一部分：M 是分别以时期 t 与时期 $t+1$ 生产技术为参照的两期指数的几何平均。若省略开方，得到的是 M^2——虽然 $M^2 > 1$ 与 $M > 1$ 的方向判断一致，但增长幅度会失真（如 $M = 1.05$ 对应 5% 的生产率增长，$M^2 = 1.1025$ 则会误读为 10.25%），且技术变化分解 $TC = M/EC$ 不再成立。取平方根后，`effch`、`techch`、`mpi` 三列严格满足 $M = EC \times TC$。

关于 `shift(1)`的时序对齐原理需要特别说明。公式中所有 `shift(1)`都不是对整列直接平移，而是先按个体分组：`result.groupby(id_var)["D11"].shift(1)` 取**同一DMU** 上一时期的取值。这样无论数据按个体排序还是按时期排序，时期 $t+1$ 的D12都能与同一DMU时期 t 的D11落到同一行；同时，每个DMU的首期没有上一时期，分组shift返回NaN，`effch`、`mpi`相应为NaN——若不分组而直接对整列 `shift(1)`，

个体主序数据中每个DMU的首期会错误地除以**前一个体**末期的取值（如Beijing t1的效率变化被算成D11(Beijing, t1)/D11(Anhui, t3)），产生无意义的跨个体数值。这种按个体分组的对齐方式保证了分子和分母的时间对应关系正确。

效率变化（EC）的计算（第117行）使用 `D11 / groupby(id_var)["D11"].shift(1)`，即同一DMU相邻时期同期效率的比值。技术变化（TC）则通过 `mpi / effch` 间接计算（第120行），这种分解方式保证了 $M = EC \times TC$ 的恒等关系成立。

为什么投入导向不需要另写一套倒比公式。上式中的 D 是取值于 $(0, 1]$ 的效率度量。`dea8` 在产出导向下返回 te $= 1/\theta = D_O \in (0, 1]$，在投入导向下返回 te $= \theta = 1/D_I \in (0, 1]$（Farrell 投入效率，见 12.6.3 节），两者都已经是" 实际 / 前沿" 意义上的效率，而不是谢泼德投入距离 $D_I \geq 1$。教科书中投入导向 MPI 之所以把比率倒过来写，是因为那里代入的是 $D_I \geq 1$。若对已经是 Farrell 效率的 `te` 再取一次倒比，效率改善时会得到 $M < 1$，与 $EC = D_{11}/D_{11,-1}$（改善时 > 1）方向相反，$TC = M/EC$ 也随之被带偏。因此 `mpi` 对两种导向使用同一套公式；只有当 `dea8` 投入侧改为返回 $\theta^{-1} = D_I$ 时，才需要同时把 `mpi` 换成倒比公式，两处必须成对修改。

以下给出不同生产技术设定下的调用示例：

```
import pandas as pd
from deabook.example.book_cases.loaders import china

ex4 = china(scaled=True)

# 产出导向,当期生产技术
result_com = mpi("Y = K L", ex4, "id", "t", orient="oo", tech="com")

# 产出导向,时序生产技术
result_seq = mpi("Y = K L", ex4, "id", "t", orient="oo", tech="seq")

# 产出导向,视窗生产技术(窗宽=2)
result_win = mpi("Y = K L", ex4, "id", "t", orient="oo", tech="window 2")

# 产出导向,全局生产技术
result_glb = mpi("Y = K L", ex4, "id", "t", orient="oo", tech="global")

# 投入导向
result_io = mpi("Y = K L", ex4, "id", "t", orient="io", tech="com")
```

图12.6：Malmquist指数代码运行结果

结果中的 `effch`、`techch`与 `mpi`三列分别刻画效率追赶、技术前沿移动与总的全要素生产率变化，三者满足恒等式 $mpi = effch \times techch$；任一分指数大于1即表示对应维度有所改善，小于1则表示恶化。

12.6.8 Malmquist-Luenberger生产率指数

Malmquist-Luenberger (ML) 指数将Malmquist框架扩展到包含非期望产出的情形，使用方向距离函数替代谢泼德距离函数：

$$ML_t^{t+1} = \left[\frac{1 + D^t(x_t, y_t, b_t; g)}{1 + D^t(x_{t+1}, y_{t+1}, b_{t+1}; g)} \times \frac{1 + D^{t+1}(x_t, y_t, b_t; g)}{1 + D^{t+1}(x_{t+1}, y_{t+1}, b_{t+1}; g)} \right]^{1/2}$$

12.6.9 从Malmquist到ML：DDF替换的设计考量

ML指数与标准Malmquist指数的关系，恰如含非期望产出的SBM与基础SBM的关系——前者是后者在”非期望产出”情境下的自然扩展。从编程角度看，这种扩展的实现异常简洁：**只需将距离函数从 `dea8`替换为 `ddf`**。

这种替换之所以可行，根源在于两个函数具有**一致的接口契约**。观察 `ddf`函数的返回格式：它返回一个包含 `te`列的DataFrame，与 `dea8`函数的返回格式完全一致。这意味着 `mpi`函数中对距离函数结果的所有后续操作（提取 `te`列、重命名为 `D11`/`D12`/`D21`、时序对齐、公式计算）都可以不经修改地应用于 `ddf`的结果。

$(1 + D)$ **形式的公式由来**是ML指数与MPI在计算层面的唯一差异。在DDF模型中，β（即 `te`列）表示方向距离，$\beta = 0$ 意味着DMU已经位于前沿上。ML指数的公式使用 $(1 + \beta)$ 而非直接使用 β，是为了确保当DMU位于前沿时（$\beta = 0$），比值项为1，不影响指数的乘积结构。代码中这种转换体现在第143行的公式：`(1 + result.groupby(id_var)["D11"].shift(1)) / (1 + result["D12"])`。

ML指数的函数结构选择体现了”代码复用最大化”的设计哲学。`mlpi`函数没有调用 `mpi`函数本身（虽然理论上可以封装一个接受”距离函数工厂”作为参数的通用函数），而是直接复用了 `mpi`函数的框架结构。这种选择权衡了代码复用性和可读性——直接复用框架使得读者可以清晰地看到ML与MPI的对应关系，每个 `ddf`调用对应一个 `dea8`调用的位置，便于逐行对比理解。

```
def mlpi(formula, data, id_var, t_var, tech=None):
    """
    计算Malmquist-Luenberger生产率指数（含非期望产出）。

    复用 mpi() 函数的框架，将距离函数从谢泼德距离函数替换为
    方向距离函数（DDF）。

    Parameters
    ----------
    formula : str
        公式字符串，如 "Y:CO2 = K L"
    data : pd.DataFrame
        面板数据
    id_var : str
        个体标识变量名
    t_var : str
```

```
    时间标识变量名
tech : str or None, default None
    生产技术设定(同mpi函数)

Returns
-------
pd.DataFrame
    原始数据附加ML指数的结果
"""
t_series = pd.Series(data[t_var]).drop_duplicates().sort_values()
t_values = t_series.tolist()

# D^t(x_t, y_t, b_t)
D11_list = []
for t_val in t_values:
    eval_q = "{}=={}".format(t_var, t_val)

    if tech is None or tech == "com":
        ref_q = "{}=={}".format(t_var, t_val)
    elif tech == "seq":
        ref_q = "{}<={}".format(t_var, t_val)
    elif tech == "global":
        ref_q = None
    elif "window " in str(tech):
        h = int(tech.split(" ")[1])
        t_idx = t_values.index(t_val)
        t_min = t_values[max(0, t_idx - h)]
        t_max = t_values[min(len(t_values) - 1, t_idx + h)]
        ref_q = "{}<={}<={}".format(t_min, t_var, t_max)

    D11 = ddf(formula, data, eval_query=eval_q, ref_query=ref_q)[["te"]]
    D11_list.append(D11)

D11_df = pd.concat(D11_list)
D11_df.rename(columns={"te": "D11"}, inplace=True)

if tech == "global":
    result = pd.merge(data, D11_df, left_index=True, right_index=True,
                      how="left")
```

```
        result["mlpi"] = ((1 + result.groupby(id_var)["D11"].shift(1)) / (1 +
            result["D11"]))
        result.drop(columns=["D11"], inplace=True)
        return result

    # D^t(x_{t+1}, y_{t+1}, b_{t+1})
    D12_list = []
    for t_val in t_values[1:]:
        eval_q = "{}=={}".format(t_var, t_val)

        if tech is None or tech == "com":
            ref_q = "{}=={}".format(t_var, t_values[t_values.index(t_val) - 1])
        elif tech == "seq":
            ref_q = "{}<={}".format(t_var, t_values[t_values.index(t_val) - 1])
        elif "window " in str(tech):
            h = int(tech.split(" ")[1])
            t_idx = t_values.index(t_val)
            t_min = t_values[max(0, t_idx - 1 - h)]
            t_max = t_values[min(len(t_values) - 1, t_idx - 1 + h)]
            ref_q = "{}<={}<={}".format(t_min, t_var, t_max)

        D12 = ddf(formula, data, eval_query=eval_q, ref_query=ref_q)[["te"]]
        D12_list.append(D12)

    D12_df = pd.concat(D12_list)
    D12_df.rename(columns={"te": "D12"}, inplace=True)

    # D^{t+1}(x_t, y_t, b_t)
    D21_list = []
    for t_val in t_values[1:]:
        eval_q = "{}=={}".format(t_var, t_values[t_values.index(t_val) - 1])

        if tech is None or tech == "com":
            ref_q = "{}=={}".format(t_var, t_val)
        elif tech == "seq":
            ref_q = "{}<={}".format(t_var, t_val)
        elif "window " in str(tech):
            h = int(tech.split(" ")[1])
            t_idx = t_values.index(t_val)
```

```
            t_min = t_values[max(0, t_idx - h)]
            t_max = t_values[min(len(t_values) - 1, t_idx + h)]
            ref_q = "{}<={}<={}".format(t_min, t_var, t_max)

        D21 = ddf(formula, data, eval_query=eval_q, ref_query=ref_q)[["te"]]
        D21_list.append(D21)

    D21_df = pd.concat(D21_list)
    D21_df.rename(columns={"te": "D21"}, inplace=True)

    # 合并并计算ML指数
    result_df = pd.concat([D11_df, D12_df, D21_df], axis=1)
    result = pd.merge(data, result_df, left_index=True, right_index=True,
                      how="left")

    # ML指数公式: sqrt[(1+D11.shift)/(1+D12) * (1+D21.shift)/(1+D11)]（shift 按个体分组）
    result["mlpi"] = np.sqrt((1 + result.groupby(id_var)["D11"].shift(1)) / (
        1 + result["D12"])
                             * (1 + result.groupby(id_var)["D21"].shift(1)) / (
                                 1 + result["D11"]))

    # Malmquist-Luenberger分解
    result["effch"] = (1 + result.groupby(id_var)["D11"].shift(1)) / (1 +
        result["D11"])
    result["techch"] = result["mlpi"] / result["effch"]

    result.drop(columns=["D11", "D12", "D21"], inplace=True)
    return result
```

12.6.10 ML与MPI的代码对比总结

为了清晰展示ML指数与标准Malmquist指数的关系，下表对比了两者的核心差异：

对比维度	Malmquist (`mpi`)	Malmquist-Luenberger (`mlpi`)
距离函数	`dea8`（谢泼德距离函数）	`ddf`（方向距离函数）
支持非期望产出	否	是
公式格式	`"Y = K L"`	`"Y:CO2 = K L"`
同期效率计算	`dea8(...)["te"]`	`ddf(...)["te"]`

对比维度	Malmquist (mpi)	Malmquist-Luenberger (mlpi)
MPI计算公式	`sqrt(D12/D11.shift * D11/D21.shift)`	`sqrt((1+D11.shift)/(1+D12) * (1+D21.shift)/(1+D11))`
EC计算公式	`D11/D11.shift`	`(1+D11.shift)/(1+D11)`
结果列名	`mpi`	`mlpi`

表中 `.shift` 均指按个体分组的 `groupby(id_var)[•].shift(1)`——同一 DMU 相邻时期配对，每个 DMU 首期为 NaN。

跨期参照技术可能无法覆盖被评价观测。程序对未求得最优解的距离保留NaN，相关指数也保留NaN，不能把它们替换成零或有效值。

以下给出 `mlpi`函数的调用示例：

```
import pandas as pd
from deabook.example.book_cases.loaders import china

ex4 = china(scaled=True)

# 计算ML指数
ml_result = mlpi("Y:CO2 = K L", ex4, "id", "t", tech="com")
print(ml_result)
```

图12.7：Malmquist-Luenberger指数代码运行结果

与Malmquist的情形类似，`mlpi`、`effch`、`techch`三者同样满足恒等式 `mlpi = effch × techch`；区别仅在于方向距离 β 经 $(1+\beta)$ 变换后替代了谢泼德距离函数，从而将污染约束一并纳入生产率变化率的测度。

12.7 结果验证与敏感性分析

本节为新增内容，介绍如何验证DEA计算结果的正确性。

计算效率值只是第一步，更重要的是确保结果是正确的。本节提供一套结果验证和敏感性分析的方法。

12.7.1 为什么需要专门的结果验证？

在DEA编程实践中，“代码运行通过”绝不等于“结果正确”。线性规划求解器总会返回一个数值结果，但该结果可能是错误的——原因包括：约束条件设置错误（如将 $\geq$ 写成 $\leq$）、变量声明遗漏（如忘记声明松弛变量）、方向向量符号错误、数据预处理失误等。更糟糕的是，某些错误不会引发运行时异常，而是静默地产生看似合理但实则错误的效率值。例如，如果投入约束的不等号方向写反，所有DMU的效率值都可能被计算为1.0（有效），这显然是不合理的，但代码会正常运行完毕。

结果验证的核心目标是建立**多层次的防御体系**：在代码层面通过断言检查不变量，在模型层面通过与基准实现交叉验证，在数据层面通过敏感性分析评估结果的稳健性。这三层验证分别对应“快速发现明显

错误”、“确认计算逻辑正确” 和” 评估结果的可靠性” 三个递进目标。

12.7.2 效率值合理性检查

```
def validate_efficiency_results(result_df, model_type="sbm", same_frontier=True):
    """
    验证DEA效率结果的合理性。

    Parameters
    ----------
    result_df : pd.DataFrame
        包含效率值的结果DataFrame
    model_type : str, default "sbm"
        模型类型: "sbm"、"ddf"、"nddf"、"dea"

    Returns
    -------
    dict
        验证结果报告

    Raises
    ------
    AssertionError
        验证不通过时抛出警告
    """
    report = {"model_type": model_type, "checks_passed": True, "warnings": []}

    if model_type == "sbm":
        eff = result_df["obj"]
        # SBM效率应在 (0, 1] 区间
        assert (eff.dropna() > 0).all(), "SBM效率值应为正"
        assert (eff.dropna() <= 1.001).all(), "SBM效率值不应超过1（允许数值误差）"

        # 松弛变量应非负
        slack_cols = [c for c in result_df.columns if c.startswith("slack ")]
        for col in slack_cols:
            assert (result_df[col].dropna() >= -1e-6).all(), \
                f"松弛变量 {col} 出现负值"

    elif model_type in ["ddf", "nddf", "dea"]:
```

```
        eff = result_df["te"] if "te" in result_df.columns else result_df[
            "obj"]
        # DDF方向距离应 >= 0
        if same_frontier:
            assert (eff.dropna() >= -1e-6).all(), "同期方向距离不应为负"

    else:
        raise ValueError("未知模型类型")

    # 检查NaN比例
    nan_ratio = eff.isna().mean()
    if nan_ratio > 0.1:
        report["warnings"].append(
            f"警告: {nan_ratio*100:.1f}% 的DMU求解失败,请检查数据质量"
        )
        report["checks_passed"] = False

    return report
```

12.7.3 合理性检查的设计依据

`validate_efficiency_results`函数的设计基于DEA模型的**理论不变量**——即无论数据如何变化，某些性质必须始终成立的数学规律。这些不变量构成了结果验证的"黄金标准"。

SBM模型的不变量检查包括两个层面。首先是**效率值范围检查**：SBM效率值 ρ 的理论范围是 $(0, 1]$，$\rho = 1$ 表示该DMU位于生产前沿上（无效率损失），$\rho < 1$ 表示存在效率改进空间。代码中使用 `eff.dropna() > 0`和 `eff.dropna() <= 1.001`进行验证，后者允许0.001的数值误差容忍度，这是因为线性规划求解器的数值精度限制可能导致效率值略微超过1（如1.0000001）。其次是**松弛变量非负检查**：松弛变量代表投入可缩减量或产出可扩张量，理论上必须非负。代码中使用`-1e-6`而非 `0`作为阈值，同样是出于数值精度考虑。

DDF/NDDF模型的不变量检查聚焦于方向距离的非负性。$\beta \geq 0$ 是DDF模型的基本性质——距离不能为负，$\beta = 0$ 表示DMU已经位于前沿上。如果检查发现有负值，通常意味着方向向量设置错误或约束条件符号错误。

NaN比例检查是数据层面的防御机制。当大量DMU求解失败（返回NaN）时，可能表明数据存在系统性问题，如异常值导致模型不可行、变量间完全共线性、或数据尺度差异过大等。设定10%为警告阈值是一个经验规则——在实际应用中，研究者应根据样本大小和问题复杂度调整此阈值。

12.7.4 与deabook包的交叉验证

使用 deabook包作为基准，验证自编程结果的正确性：

```
def cross_validate_with_deabook(data, xcol, ycol, model="SBM"):
    """
    使用deabook包交叉验证自编程结果。

    Parameters
    ----------
    data : pd.DataFrame
        投入产出数据
    xcol : list of str
        投入变量名
    ycol : list of str
        产出变量名
    model : str, default "SBM"
        模型类型

    Returns
    -------
    bool
        交叉验证是否通过
    """
    from deabook import DEA
    from deabook.constant import RTS_CRS, OPT_LOCAL
    if model.upper() != "SBM":
        raise ValueError("此函数只核验普通CRS SBM")
    formula = " ".join(ycol) + " = " + " ".join(xcol)
    benchmark = DEA.sbm(data, " ".join(xcol) + "=" + " ".join(ycol),
                        gx=[1]*len(xcol), gy=[1]*len(ycol),
                        rts=RTS_CRS).optimize(OPT_LOCAL, 'highs')
    actual = sbm(formula, data)["obj"]
    return bool(np.allclose(actual.to_numpy(), benchmark["rho"].to_numpy(),
                            atol=1e-6, rtol=1e-6))
```

12.7.5 交叉验证策略的选择

交叉验证是确认计算逻辑正确性的**最有效手段**。其基本原理是：使用另一个独立实现的、经过充分测试的DEA计算工具（如 deabook包）作为基准，比较两者的计算结果。如果结果在数值误差范围内一致，就有很高的置信度认为自编程代码的逻辑是正确的。

选择 deabook作为基准工具基于以下考量：它是本书配套开发的专用DEA工具包，实现了本章涉及的所有模型，且经过了广泛的测试验证。`np.allclose`函数用于比较两个结果数组，其 `atol=1e-4`参数设置了绝对误差容忍度。这个值的选择权衡了检测灵敏度和数值噪声——线性规划求解器（尤其是内点法）的结

果通常有 10^{-6} 到 10^{-8} 量级的数值误差，10^{-4} 的容忍度足以过滤噪声，同时又能检测到真正的逻辑错误。

交叉验证的**覆盖策略**也值得讨论。理想情况下，应对每种模型（SBM、SBM+非期望产出、DDF、NDDF）都进行交叉验证。对于Malmquist指数，可以先验证其底层距离函数（`dea8`和 `ddf`）的正确性，再手动验证几个DMU的MPI计算结果。这种“自下而上”的验证策略比直接验证最终指数更高效——如果底层距离函数正确，且MPI的公式计算逻辑清晰可审计，那么MPI的结果可信度就很高。

12.7.6 敏感性分析示例

```
def sensitivity_analysis_drop_dmu(formula, data, eval_query=None):
    """
    敏感性分析：逐一剔除每个DMU，观察其他DMU效率变化。

    若剔除某DMU后其他DMU效率大幅变化，则该DMU为"关键观测值"，
    对效率前沿有重要影响。

    Parameters
    ----------
    formula : str
        公式字符串
    data : pd.DataFrame
        投入产出数据
    eval_query : str or None, default None
        评价单元筛选条件

    Returns
    -------
    pd.DataFrame
        敏感性分析结果
    """
    # 基准效率
    base_result = sbm(formula, data, eval_query, eval_query)
    base_eff = base_result["obj"]

    # 准备数据
    if eval_query is not None:
        eval_data = data.query(eval_query)
    else:
        eval_data = data

    sensitivity = {}
```

```
    for dmu in eval_data.index:
        # 剔除该DMU
        data_drop = data.drop(index=dmu)
        result_drop = sbm(formula, data_drop, eval_query, eval_query)

        # 计算效率变化幅度(均方根变化)
        common_idx = base_eff.index.intersection(result_drop.index)
        if len(common_idx) > 0:
            rms_change = np.sqrt(
                np.mean((base_eff[common_idx] - result_drop.loc[common_idx,
                    "obj"])**2)
            )
            sensitivity[dmu] = rms_change

    sens_df = pd.DataFrame.from_dict(
        sensitivity, orient="index", columns=["rms_change"]
    )
    sens_df = sens_df.sort_values("rms_change", ascending=False)

    print("敏感性分析结果(影响最大的前5个DMU): ")
    print(sens_df.head())

    return sens_df


def sensitivity_analysis_perturbation(formula, data, eval_query=None,
                                      noise_level=0.01, n_replicates=10):
    """
    敏感性分析:对投入数据施加随机扰动,观察效率稳定性。

    Parameters
    ----------
    formula : str
        公式字符串
    data : pd.DataFrame
        投入产出数据
    eval_query : str or None, default None
        评价单元筛选条件
    noise_level : float, default 0.01
```

```
        扰动幅度(如0.01表示1%)
    n_replicates : int, default 10
        重复次数

    Returns
    -------
    pd.DataFrame
        每次重复的效率结果
    """
    _, _, xcol, ycol, _ = prepare_data(formula, data)

    all_results = []
    for rep in range(n_replicates):
        # 复制数据并施加随机扰动
        data_perturbed = data.copy()
        for col in xcol:
            noise = np.random.normal(0, noise_level, size=len(data_perturbed))
            data_perturbed[col] = data_perturbed[col] * (1 + noise)

        result = sbm(formula, data_perturbed, eval_query, eval_query)
        result["replicate"] = rep
        all_results.append(result[["obj", "replicate"]])

    all_df = pd.concat(all_results)

    # 计算变异系数(CV = std / mean)
    cv = all_df.groupby(all_df.index)["obj"].agg(["mean", "std"])
    cv["cv"] = cv["std"] / cv["mean"]

    print(f"效率值的平均变异系数: {cv['cv'].mean():.4f}")
    print(f"最大变异系数: {cv['cv'].max():.4f}")

    return all_df
```

12.7.7 敏感性分析的方法论解读

敏感性分析是DEA研究中不可或缺的环节，它回答的问题是：**结果的稳健性如何？**具体而言，如果数据发生微小变化，效率值是否会大幅波动?如果某个DMU被剔除，其他DMU的效率是否会显著改变?这些问题的答案直接影响研究结论的可信度。

剔除DMU分析（“留一法”）的经济学含义是识别” 关键观测值”——即对生产前沿构造有重要影响的DMU。在DEA中，生产前沿由一组” 有效”DMU（效率值为1的DMU）的凸包构成。如果某个DMU是前沿的” 顶点”（即移除它会改变前沿的形状），那么剔除它会导致其他DMU的效率值发生变化。`sensitivity_analysis_drop_dmu`函数通过计算” 剔除某DMU后其他DMU效率的均方根变化” 来量化这种影响。影响最大的DMU通常是极端值或异常值，研究者应仔细检查这些观测值的数据质量。

数据扰动分析则模拟了测量误差对效率值的影响。在实际数据中，投入产出变量往往存在测量误差（如统计口径不一致、数据四舍五入等）。通过对投入数据施加1%-5%的随机扰动，观察效率值的变化幅度，可以评估结果的统计稳健性。变异系数（CV = 标准差 / 均值）是衡量稳健性的常用指标。CV越小，说明效率值对数据扰动越不敏感，结果越可靠。在实际应用中，如果某DMU的CV显著高于其他DMU，可能表明该DMU位于效率前沿的” 边缘” 位置，其效率值对数据变化特别敏感。

12.8 从脚本到模块：代码组织进阶

本节为新增内容，展示如何将本章代码组织为可复用的Python模块。

经过本章的学习，读者已经掌握了多种DEA模型的编程实现。在实际研究中，将这些代码组织成模块或软件包可以大幅提高工作效率。本节展示如何从Jupyter Notebook脚本进阶到模块化设计。

12.8.1 为什么需要从脚本走向模块？

在学习和探索阶段，将所有代码写在一个Jupyter Notebook中是高效且直观的，可以即时运行、快速迭代、方便可视化。然而，当研究进入批量应用阶段（如需处理多个数据集、比较多种模型、进行稳健性检验）时，脚本式编程的弊端便会显现：代码分散在多个Notebook中难以维护，相同功能的代码被反复复制粘贴，修改一处需要手动同步到所有副本，且不同Notebook中的代码版本可能不一致。

模块化的核心价值在于**关注点分离**（Separation of Concerns）：将代码按照功能职责划分为独立的单元，每个单元只负责一件事，单元之间通过清晰的接口进行交互。对于DEA工具包而言，这意味着将” 公式解析”、“数据验证”、“模型求解”、“结果格式化” 等通用功能与”SBM模型逻辑”、“DDF模型逻辑” 等特有功能分离，使得新增一个模型时只需关注该模型的数学定义，而无需重复编写数据准备和结果格式化的” 样板代码”。

12.8.2 推荐项目结构

```
my_dea_toolkit/                    # 项目根目录
├── __init__.py                    # 包初始化
├── models/                        # 模型模块
│   ├── __init__.py
│   ├── sbm.py                     # SBM模型（含非期望产出）
│   ├── ddf_nddf.py                # DDF和NDDF模型
│   └── malmquist.py               # Malmquist和ML指数
├── utils/                         # 工具模块
│   ├── __init__.py
│   ├── common.py                  # 公共工具函数（求解、验证、格式化）
```

```
│   ├── formula_parser.py          # 公式解析
│   └── validation.py              # 结果验证
├── tests/                         # 单元测试
│   ├── __init__.py
│   └── test_models.py             # 模型测试
└── examples/                      # 示例脚本
    ├── run_sbm.py                 # SBM示例
    └── run_malmquist.py           # Malmquist示例
```

12.8.3 项目结构的设计哲学

上述项目结构遵循了Python社区广泛认可的**包组织惯例**（Package Organization Convention），其核心原则可以概括为"高内聚、低耦合"。

models/目录包含了所有DEA模型的实现，每个模型一个文件。这种组织方式的优点是：- **清晰的模型边界**：每个文件只关注一种（或一类紧密相关的）模型，如 `ddf_nddf.py`将DDF和NDDF放在一起，因为它们共享方向向量的概念；- **独立的版本演进**：当某个模型需要修改（如SBM模型增加新的约束变体）时，不会影响到其他模型文件；- **按需导入**：用户可以根据需要只导入特定模型，如 `from models.sbm import sbm`，避免加载不必要的代码。

utils/目录包含了所有模型共享的通用工具函数。将其与模型逻辑分离的决策基于 **DRY原则**——在12.3 节中已经看到，约70%的代码是通用的（公式解析、数据验证、求解器调用、结果格式化）。将这些通用代码集中到 `utils/`中，不仅避免了重复，更重要的是建立了**单一事实来源**（Single Source of Truth）——当需要修改数据验证逻辑时，只需修改 `utils/common.py`中的一个函数，所有模型自动受益。

tests/目录是专业软件开发的标志。单元测试不仅用于验证代码正确性，更是一种**可执行的文档**——通过阅读测试用例，用户可以清晰地了解每个函数的预期输入和输出。对于DEA工具包，测试用例应该覆盖边界情况（如单投入单产出、含零值数据、缺失值处理等）。

12.8.4 完整模块代码

以下是本章所有公共工具函数的汇总，可作为 `utils/common.py`的完整内容：

```python
"""DEA工具包——公共工具模块

本模块封装了DEA模型求解的通用功能，包括：
- 公式解析（parse_formula, normalize_formula）
- 数据验证（validate_data, prepare_data）
- 模型求解（solve_dea_model）
- 结果格式化（format_sbm_results, format_ddf_results）
"""

from pyomo.environ import *
from pyomo.opt import SolverStatus, TerminationCondition
```

```
import pandas as pd
from deabook.example.book_cases.loaders import china
import numpy as np
import re


# ========== 公式解析 ==========

def normalize_formula(formula_str):
    """规范化公式字符串(详见12.3.3节)"""
    formula_str = formula_str.replace(': ', ':').replace(' =', '=')
    formula_str = re.sub(r'\s+', ' ', formula_str).strip()
    return formula_str


def parse_formula(formula_str):
    """解析DEA公式字符串(详见12.3.3节)"""
    formula_str = normalize_formula(formula_str)
    parts = formula_str.split('=')
    if len(parts) != 2:
        raise ValueError(f"公式格式错误: '{formula_str}'")

    left_side = parts[0].strip()
    right_side = parts[1].strip()

    xcol = [v.strip() for v in right_side.split(' ') if v.strip()]

    if ':' in left_side:
        output_parts = left_side.split(':')
        ycol = [v.strip() for v in output_parts[0].split(' ') if v.strip()]
        bcol = [v.strip() for v in output_parts[1].split(' ') if v.strip()]
    else:
        ycol = [v.strip() for v in left_side.split(' ') if v.strip()]
        bcol = []

    return xcol, ycol, bcol


# ========== 数据验证与准备 ==========
```

```
def validate_data(df, xcol, ycol, bcol=None):
    """验证数据是否符合DEA模型要求(详见12.3.4节)"""
    cols = xcol + ycol
    if bcol:
        cols = cols + bcol

    assert df[cols].notna().all().all(), "数据包含缺失值"
    assert (df[xcol] > 0).all().all(), "投入数据必须严格为正"
    assert (df[ycol] > 0).all().all(), "期望产出数据必须严格为正"
    if bcol:
        assert (df[bcol] >= 0).all().all(), "非期望产出不能为负"


def prepare_data(formula, dataframe, eval_query=None, ref_query=None):
    """根据公式和查询条件准备数据(详见12.3.4节)"""
    xcol, ycol, bcol = parse_formula(formula)

    if eval_query is None:
        index_eval = dataframe.index
    else:
        index_eval = dataframe.query(eval_query).index

    if ref_query is None:
        index_ref = dataframe.index
    else:
        index_ref = dataframe.query(ref_query).index

    all_cols = xcol + ycol + (bcol if bcol else [])
    data = dataframe.loc[index_eval, all_cols]
    dataref = dataframe.loc[index_ref, all_cols]

    return data, dataref, xcol, ycol, bcol


# ========== 求解与结果提取 ==========

def solve_dea_model(model, solver_name="highs", timeout=60):
    """求解DEA模型并返回结果(详见12.3.5节)"""
    solver = SolverFactory(solver_name)
```

```
    if not solver.available(exception_flag=False):
        raise RuntimeError(f"求解器 '{solver_name}' 未安装或不可用。")

    results = solver.solve(model, timelimit=timeout, load_solutions=False)

    if results.solver.status != SolverStatus.ok:
        raise RuntimeError(f"求解器状态异常: {results.solver.status}")

    if results.solver.termination_condition != TerminationCondition.optimal:
        raise RuntimeError(
            f"求解未达到最优: {results.solver.termination_condition}"
        )

    model.solutions.load_from(results)
    return results


def format_sbm_results(obj_dict, slack_x_dict, slack_y_dict,
                       xcol, ycol, bcol=None, slack_b_dict=None):
    """格式化SBM结果为DataFrame（详见12.3.5节）"""
    obj_df = pd.DataFrame(obj_dict, index=["obj"]).T

    slack_x_df = pd.DataFrame(slack_x_dict, index=xcol).T
    slack_x_df.columns = slack_x_df.columns.map(lambda x: "slack " + str(x))

    slack_y_df = pd.DataFrame(slack_y_dict, index=ycol).T
    slack_y_df.columns = slack_y_df.columns.map(lambda y: "slack " + str(y))

    result = pd.concat([obj_df, slack_x_df, slack_y_df], axis=1)

    if bcol and slack_b_dict:
        slack_b_df = pd.DataFrame(slack_b_dict, index=bcol).T
        slack_b_df.columns = slack_b_df.columns.map(lambda b: "slack " + str(
            b))
        result = pd.concat([result, slack_b_df], axis=1)

    return result
```

```
def format_ddf_results(obj_dict, index_list):
    """格式化DDF结果为DataFrame（详见12.3.5节）"""
    return pd.DataFrame(obj_dict, index=["te"]).T
```

12.8.5 使用示例

```
# 按本章顺序执行时,直接复用前文已经定义的模型函数
import pandas as pd
from deabook.example.book_cases.loaders import china

# 读取数据
data = china(scaled=True)

# SBM模型
result_sbm = sbm("Y = K L", data, "t==[1,2,3]", "t==[1,2,3]")

# DDF模型
result_ddf = ddf("Y:CO2 = K L", data)

# Malmquist指数
result_mpi = mpi("Y = K L", data, "id", "t", orient="oo", tech="com")
```

12.8.6 函数式设计与面向对象设计的权衡

本章所有代码均采用**纯函数式**设计——每个DEA模型都是一个独立的Python函数,输入数据框和参数,返回结果数据框。这种设计选择并非唯一正确的答案,而是基于DEA计算特性作出的审慎决策。

函数式设计的优势在于**简单性和透明性**:给定相同的输入,函数总是产生相同的输出(无副作用),这使得代码易于理解和调试。对于DEA这种"数据进、结果出"的计算密集型任务,函数式设计非常自然。

然而,在某些场景下,**面向对象设计**(OOP)可能更合适。例如,当需要开发一个支持多种求解器切换、自动缓存结果或提供丰富配置选项的DEA框架时,使用类可以更好地封装状态和行为。以下是一个简化的OOP设计示例,展示了何时以及如何从函数转向类:

```
class DEAModel:
    """DEA模型的基类,展示面向对象设计的思路"""

    def __init__(self, formula, data, solver="highs"):
        self.formula = formula
        self.data = data
        self.solver = solver
        self.xcol, self.ycol, self.bcol = parse_formula(formula)
```

```
        validate_data(data, self.xcol, self.ycol, self.bcol)

    def fit(self, eval_query=None, ref_query=None):
        """拟合模型(子类必须实现)"""
        raise NotImplementedError

    def summary(self):
        """输出结果摘要(子类必须实现)"""
        raise NotImplementedError


class SBMModel(DEAModel):
    """SBM模型的面向对象实现"""

    def fit(self, eval_query=None, ref_query=None):
        # 复用函数式sbm的核心逻辑
        self.result = sbm(self.formula, self.data, eval_query, ref_query,
            self.solver)
        return self

    def summary(self):
        print(f"SBM模型结果 (N={len(self.result)}):")
        print(f"  平均效率: {self.result['obj'].mean():.4f}")
        print(f"  有效DMU数: {(self.result['obj'] >= 0.999).sum()}")
```

何时使用类而非函数？以下是决策参考：

场景	推荐方案	理由
单次分析、快速探索	函数	简单直接，无需额外的类定义
需要保存中间状态(如缓存)	类	状态封装自然
多模型比较(同一数据用多种模型)	类	共享数据准备逻辑，通过多态切换模型
构建可发布的软件包	类+函数	对外提供统一的类接口，内部使用函数实现
需要方法链式调用(如`.fit().summary().plot()`)	类	支持fluent interface

12.8.7 单元测试示例

```
"""DEA模型单元测试"""
import pandas as pd
from deabook.example.book_cases.loaders import china
import numpy as np


class TestFormulaParser:
    """测试公式解析函数"""

    def test_basic_formula(self):
        xcol, ycol, bcol = parse_formula("Y = K L")
        assert xcol == ["K", "L"]
        assert ycol == ["Y"]
        assert bcol == []

    def test_undesirable_formula(self):
        xcol, ycol, bcol = parse_formula("Y:CO2 = K L")
        assert xcol == ["K", "L"]
        assert ycol == ["Y"]
        assert bcol == ["CO2"]

    def test_normalize_whitespace(self):
        xcol, ycol, bcol = parse_formula("  Y   =  K     L  ")
        assert xcol == ["K", "L"]
        assert ycol == ["Y"]


class TestSBMModel:
    """测试SBM模型"""

    def test_sbm_efficiency_range(self):
        """SBM效率值应在(0, 1]区间"""
        data = china(scaled=True).query("t==4").copy()
        result = sbm("Y = K L", data)
        assert (result["obj"] > 0).all()
        assert (result["obj"] <= 1 + 1e-7).all()

    def test_sbm_efficient_dmu(self):
```

```
        """效率前沿上的DMU效率应为1"""
        data = china(scaled=True).query("t==4").copy()
        result = sbm("Y = K L", data)
        assert np.isclose(result["obj"].max(), 1.0, atol=1e-7)

# 不依赖测试框架也可直接执行全部检查。
for test_class in (TestFormulaParser, TestSBMModel):
    instance = test_class()
    for name in dir(instance):
        if name.startswith("test_"):
            getattr(instance, name)()
```

12.8.8 常见错误排查速查表

表 12.5: 常见错误与调试方法

错误现象	可能原因	解决方案
`SolverFactory('highs')` 报错	未安装 highspy	`python -m pip install highspy`
`RuntimeError: 求解器不可用`	求解器未正确安装	运行验证代码检查可用求解器；参见附录A.1.3 节
`ValueError: 公式格式错误`	公式字符串不含=	检查公式格式是否为 “产出 = 投入”
`AssertionError: 投入数据必须严格为正`	数据含零或负值	检查数据预处理，确保投入产出严格为正
`TerminationCondition.infeasible`	模型约束矛盾	检查方向（投入/产出）是否设置正确；检查数据是否有异常值
`KeyError: 'x1'`	变量名不匹配	检查DataFrame列名与公式中名称是否一致（区分大小写）
效率值全为1.0	变量遗漏或约束错误	检查是否所有投入产出都纳入了约束
Malmquist指数异常大	跨期前沿构造错误	检查混合数据集的eval_query和ref_query是否正确
`TypeError: 'NoneType'`	eval_query或ref_query为无效字符串	检查query字符串语法；确保列名存在
内存溢出（大样本）	约束数量过大	使用切割平面算法或降采样；考虑使用专业包如 `deabook`
SBM效率值大于1	CC转换约束错误	检查约束是否使用==而非<=或>=

错误现象	可能原因	解决方案
结果索引错位	未使用原始索引循环	将 `range(n)`改为 `data.index`;将 `iloc`改为 `loc`

12.9 案例应用:同一省级样本的程序核验

实战使用china data.xlsx的2016—2019年30省面板;静态结果固定2019年30个观测,跨期函数仍使用同源面板。K、L、E为投入,Y为期望产出,CO2为非期望产出。下表在同一2019年CRS样本上比较普通SBM、含污染SBM与只沿排放方向调整的DDF效率。比较的目的是核实程序如何反映目标函数与约束变化,不能把不同定义的效率简单解释为同一指标的稳健性。

```
from deabook.example.book_cases.run import FUN
FUN[12]()  # 重新计算本章,输出完整CSV、摘要表与PNG/PDF图
```

表 12.6: 同一省级样本的程序核验:结果摘要

统计量	普通SBM	含污染SBM	减排方向效率
有效样本数	30	30	30
均值	0.404	0.302	0.227
标准差	0.173	0.170	0.179
最小值	0.190	0.128	0.037
P10	0.240	0.167	0.085
中位数	0.381	0.271	0.193
P90	0.584	0.446	0.377
最大值	1.000	1.000	1.000

表将三个模型的有效数、均值、离散程度和分位数并列,便于从代码输出检查效率范围及分布变化。图右逐省对照普通SBM与含污染SBM。三个指标的目标函数和约束不同,不能将数值差异直接当作同一个指标的稳健性结论。

结果表报告三个模型的详细分布统计。普通SBM不包含CO2约束,含污染SBM增加污染松弛项,排放DDF固定其他方向;它们回答的问题不同。复算脚本同时检查线性规划原始约束残差,完整CSV保留全部30个地区的结果。书中只展示摘要不能替代程序检验。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch12_full.csv`。表中P10和P90为第10和第90百分位数(如适用),展示分布而不占用逐个决策单元的行数。

12.10 本章小结

本章通过六个代表性实例,系统展示了如何将DEA的理论模型转化为可执行的Python代码。从基础的SBM模型及其基于松弛的效率测度出发,逐步扩展到包含非期望产出的SBM环境效率评价、借助方向向量

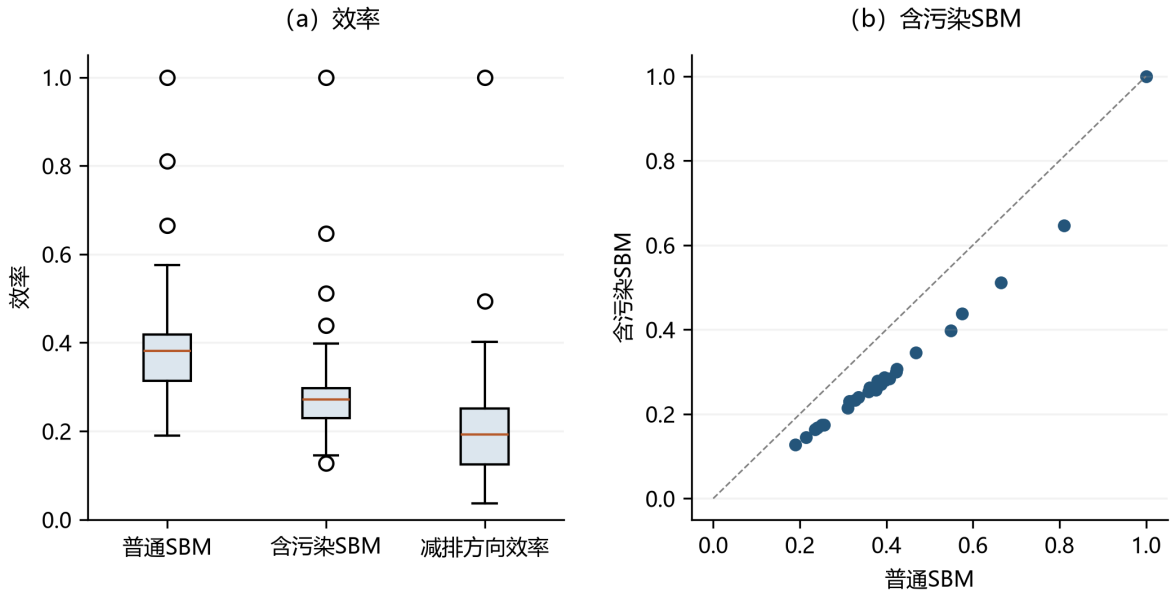


图 12.1: 同一中国省级样本中不同目标函数的结果比较（2019年）

调整的方向距离函数DDF、允许各变量按不同比例加权的非径向方向距离函数NDDF，再到刻画生产率动态变化的跨期Malmquist指数，以及将方向距离函数与跨期思想相结合的ML指数，这些实例覆盖了从径向到非径向、从静态效率到动态生产率、从合意产出到非期望产出的主要建模场景，并相应体现了非线性模型经Charnes-Cooper转换实现线性化、增加非期望产出松弛、最大化单一或多方向距离函数等核心求解思路。在编程实现层面，本章强调以 `prepare_data`、`validate_data`、`solve_dea_model`、`format_*_results` 等公共工具函数实现模块化设计以避免重复并提升可维护性，借助 `try-except`块与求解状态检查增强代码在真实数据下的健壮性，利用正则表达式规范公式输入以提高对用户表达的容错能力，遵循PEP 8 以 `snake_case`命名变量并用 `intensity`替代 `lamda`，并通过合理性检查、交叉验证与敏感性分析确保计算结果的可靠性。最后，本章演示了如何将上述代码组织为可复用的Python模块，建议读者将公共工具函数集中置于 `utils/common.py`、将各模型分别存放于 `models/`目录下的独立文件、并辅以单元测试保障正确性，在处理大规模问题时也可直接借助 `deabook`这类专业软件包。理解这些编程原理，有助于读者在使用 `deabook`时调试错误、扩展功能，乃至在现有软件包无法满足需求时自行开发定制化的DEA求解程序。

参考文献

Chambers, R. G., Chung, Y., & Färe, R. (1996). Benefit and distance functions. *Journal of Economic Theory*, 70(2), 407-419. https://doi.org/10.1006/jeth.1996.0096

Chambers, R. G., Chung, Y., & Färe, R. (1998). Profit, directional distance functions, and Nerlovian efficiency. *Journal of Optimization Theory and Applications*, 98(2), 351-364. https://doi.org/10.1023/A:1022637501082

Chung, Y. H., Fare, R., & Grosskopf, S. (1997). Productivity and Undesirable Outputs: A Directional Distance Function Approach. *Journal of Environmental Management*, 51(3), 229-240. https://doi.org/10.1006/jema.1997.0146

Tone, K. (2001). A slacks-based measure of efficiency in data envelopment analysis. *European Journal of Operational Research*, 130(3), 498-509. https://doi.org/10.1016/s0377-2217(99)00407-5

Tone, K. (2003). Dealing with undesirable outputs in DEA: A slacks-based measure (SBM) approach (GRIPS Discussion Paper No. I-2003-0005). GRIPS Policy Research Center. https://doi.org/10.24545/00000955

Zhou, P., Ang, B. W., & Wang, H. (2012). Energy and CO2 emission performance in electricity generation: A non-radial directional distance function approach. *European Journal of Operational Research*, 221(3), 625-635. https://doi.org/10.1016/j.ejor.2012.04.022

第 13 章

基于DEA对偶问题的影子价格估计

代码运行约定：在项目根目录运行，先安装 `python -m pip install numpy pandas scipy matplotlib pyomo highspy openpyxl`。各段Python 代码按本章顺序在同一会话执行，后文复用前文定义；默认采用HiGHS。公式讲解片段另标为 `text`，不作为独立程序执行。

第 3 章至第 10 章依次建立了DEA的一系列原始模型——从径向的CCR/BCC，到非径向的SBM（第 8 章）与方向性距离函数DDF（第 7 章）——并通过包络形式（envelopment form）求解效率值；这些章节的重心始终在于" 如何构建线性规划并测度效率"。需要指出的是，上述每一个包络形式都对应着一个等价的乘数形式，二者经线性规划对偶理论相互关联：对偶问题不改变最优值，却把" 效率" 这一无量纲测度转换为各投入与产出的**隐含价值**。本章即承接这一对偶关系展开。

本章将深入探讨对偶变量的经济含义，**影子价格（shadow price）**，以及如何稳健地估计和解释这些价格。在资源环境经济学中，影子价格具有特殊的重要性：当某些资源（如清洁空气、碳排放权）缺乏市场交易价格时，DEA对偶问题提供的影子价格成为估计其" 隐含价值" 的重要方法。

原始问题测度效率，对偶问题揭示价值。这句话凝练地概括了DEA原始问题（包络形式）与对偶问题（乘数形式）的分工。具体而言：

- **包络形式（原始问题）**回答："在给定技术下，某个决策单元（DMU）最多能减少多少投入？或最多能增加多少产出？"——它测度的是**效率值**。
- **乘数形式（对偶问题）**回答："这些投入和产出的隐含价值（影子价格）是多少？"——它揭示的是**资源的边际价值**。

影子价格在资源环境经济学中的核心应用包括：

1. **非期望产出的定价：**当污染（如CO2、SO2）缺乏市场价格时，DEA影子价格可估计其减排的边际成本，为环境税（如碳税）的制定提供参考；
2. **资源错配诊断：**通过比较影子价格与市场价格，可以识别哪些资源被低估或高估；
3. **政策模拟：**基于影子价格构建边际减排成本曲线，评估不同减排目标的经济可行性。

本章将介绍如何应用Python编写基于DEA对偶问题的影子价格计算程序，并讨论影子价格估计中的数值稳定性问题、模型选择对价格估计的影响，以及重抽样敏感性诊断与置信区间的区别。

术语说明：在DEA文献中，**对偶变量（dual variables）**与**乘数（multipliers）**是同义词，均指对偶线性规划问题中的决策变量。这些变量在经济上被解释为**影子价格（shadow prices）**，表示约束条件的边际价值。本章将统一使用" 影子价格" 这一术语。

13.1 线性规划的对偶问题

13.1.1 从经济学直觉出发：资源定价视角

在介绍数学推导之前，让我们先用一个具体的数值例子建立对偶问题的经济学直觉。

假设一个简单的单投入单产出生产系统：某工厂使用1单位能源（x）生产1单位产品（y）。现有A、B、C三个工厂的数据如下：

工厂	能源投入 x	产品产出 y
A	2	1
B	4	2
C	6	2

在投入导向的DEA模型中，原始问题（包络形式）旨在最小化工厂C的能源投入缩减比例 θ：

$$\begin{aligned} \min_{\theta,\lambda} \quad & \theta \\ \text{s.t.} \quad & 2\lambda_A + 4\lambda_B + 6\lambda_C \le 6\theta \\ & \lambda_A + 2\lambda_B + 2\lambda_C \ge 2 \\ & \lambda_A, \lambda_B, \lambda_C \ge 0 \end{aligned}$$

求解该问题，得到 $\theta^* = 2/3$，即工厂C最多可缩减 $1/3$ 的能源投入而保持产出不变。这意味着工厂C的效率值为 $2/3$。

现在从另一个角度思考：假设不知道能源和产品的市场价格，而是要给它们各”标”一套内部价格（即影子价格）——能源的影子价格记为 v，产品的影子价格记为 u。这套价格需要满足两个要求：

- **参照集的可行性约束**：这套价格下，样本中没有任何工厂能”无本获利”，即每个工厂的产出价值不超过其投入成本：
  - 工厂A：$1 \cdot u \le 2v$，即 $u - 2v \le 0$；
  - 工厂B：$2u \le 4v$，即 $2u - 4v \le 0$；
  - 工厂C：$2u \le 6v$，即 $2u - 6v \le 0$。
- **归一化约束**：影子价格只有相对大小有意义（把 u, v 同时放大 k 倍，上述约束不变），因此需固定一个尺度。投入导向的做法是把被评价单元C的投入总价值标准化为1，即 $6v = 1$。

在此基础上，我们为工厂C寻找**对它最有利**的一套价格，也就是让它的产出价值 $2u$ 尽可能大。这便构成了对偶问题（乘数形式）：

$$\begin{aligned} \max_{u,v} \quad & 2u \\ \text{s.t.} \quad & 6v = 1 \\ & u - 2v \le 0 \\ & 2u - 4v \le 0 \\ & 2u - 6v \le 0 \\ & v \ge 0,\ u \ge 0 \end{aligned}$$

其中 v 是投入的影子价格，u 是产出的影子价格。归一化约束给出 $v^* = 1/6$，最紧的一条参照约束是工厂A（$u \le 2v$），故 $u^* = 1/3$，目标值 $2u^* = 2/3$——与包络形式的 $\theta^* = 2/3$ 相等，这验证了**强对偶定理（strong duality theorem）**：原始问题的最优值等于对偶问题的最优值。

需要强调归一化约束 $6v = 1$ 不可省略。若把目标写成无归一化的 $\max(2u - 6v)$，则在 $u \le 2v$ 下有 $2u - 6v \le 4v - 6v = -2v \le 0$，而 $u = v = 0$ 可行，最优值是 **0** 而不是 $2/3$，强对偶关系无从体现。一般地，投入导向CCR的乘数形式是 $\max\ \mu' y_0$ s.t. $\nu' x_0 = 1$、$\mu' Y_i - \nu' X_i \le 0$、$\mu, \nu \ge 0$，它是一个**带归一化约束的**加权产出最大化问题，而不是无约束的" 虚拟利润" 最大化。

通过这个例子，可以看到：- **原始问题**告诉我们" 工厂C可以缩减多少能源投入"（效率测度）；- **对偶问题**告诉我们" 每单位能源的影子价格是多少"（价值揭示）。

在市场竞争中，影子价格趋向于市场价格；在市场失灵（如污染无定价）时，影子价格提供了" 社会最优价格" 的估计——这正是DEA影子价格方法在环境经济学中的核心价值。

13.1.2 DEA的两种形式：包络形式与乘数形式

在深入数学推导之前，有必要澄清DEA文献中的一对核心术语：包络形式与乘数形式是同一模型的两种表述，二者在数学上通过线性规划对偶理论相互关联，构成一对对偶线性规划问题。

特征	包络形式（Primal / Envelopment Form）	乘数形式（Dual / Multiplier Form）
决策变量	强度变量 λ_i（intensity variables）	权重 (μ, ν)，即影子价格
约束条件	投入产出约束（参照集凸组合）	权重非负约束、DMU效率约束
目标函数	最小化/最大化效率测度	最大化/最小化加权和比率
直观解释	"参照哪个标杆DMU？"	"各指标值多少钱？"
计算优势	通常变量更少（N 个 λ）	有时约束更少（M 个权重）
经济学含义	效率值（距前沿的距离）	影子价格（资源的隐含价值）

术语澄清：在DEA文献中，有时会将包络形式称为" 对偶问题"、乘数形式称为" 原始问题"——这是因为线性规划对偶理论中的原始-对偶关系与DEA的命名习惯恰好相反。本章遵循DEA文献的主流约定：包络形式为原始问题（envelopment primal），乘数形式为对偶问题（multiplier dual）。

13.1.3 线性规划对偶的数学推导

考虑如下问题：企业A拥有3种投入要素，最大可用量分别为8、16和12。企业计划生产2种商品，商品1每单位需要投入的3种生产要素分别为1、4和0，商品2每单位需要投入的3种生产要素分别为2、0和4。两种商品的市场价格分别为2和3。企业应如何安排生产，才能使其利润最大化？

上述问题可以总结为 表 13.3：

表 13.3: 生产要素与商品信息

	商品1	商品2	要素最大可用量
要素1	1	2	8
要素2	4	0	16
要素3	0	4	12
市场价格	2	3	

设企业两种商品的产量分别为 x_1 和 x_2，则企业的决策问题可以表示为：

$$\begin{aligned} \max \quad & z = 2x_1 + 3x_2 \\ \text{s.t.} \quad & x_1 + 2x_2 \leq 8 \\ & 4x_1 \leq 16 \\ & 4x_2 \leq 12 \\ & x_1 \geq 0,\ x_2 \geq 0 \end{aligned} \tag{13.1}$$

现在换一个角度分析。假设企业B想将企业A的所有生产资源收买过来，其需要支付的最小代价是多少?换言之，企业A放弃其拥有的生产资源需要得到多少补偿?这要求企业A出让生产一单位某种商品的资源所获得的回报不低于该商品的价格，即 $y_1 + 4y_2 \geq 2$、$2y_1 + 4y_3 \geq 3$（y_1, y_2, y_3 为一单位生产资源的影子价格）。由此得到以下线性规划问题：

$$\begin{aligned} \min \quad & w = 8y_1 + 16y_2 + 12y_3 \\ \text{s.t.} \quad & y_1 + 4y_2 \geq 2 \\ & 2y_1 + 4y_3 \geq 3 \\ & y_1 \geq 0,\ y_2 \geq 0,\ y_3 \geq 0 \end{aligned} \tag{13.2}$$

以上即为线性规划的对偶问题。

接下来，从数学表达式的角度分析原问题的对偶问题。为了获得目标函数式(13.1)的最大值，可以对式(13.1)的约束条件进行线性组合，进而对目标函数进行放缩，获得目标函数的上限。例如，将第一个约束乘以 3/2、第二个约束乘以 1/8 后相加，得 $2x_1 + 3x_2 = z \leq 14$，可知在这种组合情况下，目标函数的上限为 14（这恰好是该问题的最优值）。

如何找到一个最佳的组合方式?可以设定三个约束分别乘上 y_1、y_2 和 y_3，且 $y_1 \geq 0, y_2 \geq 0, y_3 \geq 0$，即考虑约束1 $\times y_1$ + 约束2 $\times y_2$ + 约束3 $\times y_3$，可得：

$$(y_1 + 4y_2) \times x_1 + (2y_1 + 4y_3) \times x_2 \leq 8y_1 + 16y_2 + 12y_3 \tag{13.3}$$

要使式(13.3)构成式(13.1)目标函数的上界，需要限定式(13.3)中 x_1 的系数不小于式(13.1)中 x_1 的系数、x_2 的系数不小于式(13.1)中 x_2 的系数，可得：

$$z = 2x_1 + 3x_2 \leq (y_1 + 4y_2) \times x_1 + (2y_1 + 4y_3) \times x_2 \leq 8y_1 + 16y_2 + 12y_3 \tag{13.4}$$

若求得 $8y_1 + 16y_2 + 12y_3$ 的最小值，则 z 的最大值即可确定。此时得到了另一个线性规划问题，即式(13.2)。

13.1.4 对偶转换的一般规则

基于上述推导，可以总结得到以下要点：

（1）原问题有几个约束，对偶问题就有几个决策变量（用于在约束两边同时乘该决策变量）；原问题有几个决策变量，对偶问题就有几个约束。

（2）原问题的决策变量的系数对应对偶问题的右端项；原问题的右端项对应对偶问题的决策变量的系数（原问题的右端项乘了对偶问题的决策变量）。

（3）原问题的系数矩阵的转置对应对偶问题的系数矩阵（原问题的左端项乘了对偶问题的决策变量，合并同类项后便可得到对偶问题的系数矩阵）。

（4）原问题是最大化问题，对偶问题就是最小化问题；原问题是最小化问题，对偶问题就是最大化问题。

（5）原问题与对偶问题之间约束条件符号方向与决策变量符号范围的对应关系。其探讨的核心逻辑围绕" 正常情况" 展开，通过约束条件与变量符号的映射关系建立原问题与对偶问题的联系。

首先，线性规划问题的" 正常情况" 是其最基础形式，为对偶转换提供基准，主要包括：- **最大化问题**的" 正常情况" 要求所有决策变量非负（$x_j \geq 0$），且约束条件为小于等于型（$\leq$）。这一设计的逻辑在于：最大化目标需上界约束以防止解趋向无穷大，而非负变量符合实际资源的物理意义（如投入不可为负）。- **最小化问题**的" 正常情况" 则要求变量非负（$x_j \geq 0$），且约束为大于等于型（$\geq$），以确保资源投入满足最低需求的下界限制。

其次，原问题的约束类型按照以下规则直接决定对偶变量的符号范围：- **原问题约束为 $\leq$（最大化）或 $\geq$（最小化）**：对偶变量 $y_i \geq 0$。此时约束代表资源的有效限制，影子价格反映其边际贡献或成本节省，非负性符合经济学直觉。- **原问题约束为等式（=）**：对偶变量 y_i 无约束（可正可负）。等式约束意味着资源被严格固定，其影子价格可能因冗余或短缺呈现不同符号。

在此基础上，可以进一步讨论原问题变量的符号范围如何决定对偶约束的方向：

（1）**原问题变量非负（$x_j \geq 0$）**：- 若原问题为**最大化**，对偶约束为 $\leq c_j$（资源消耗的边际收益上限）；- 若原问题为**最小化**，对偶约束为 $\geq c_j$（资源投入的边际成本下限）。

（2）**原问题变量非正（$x_j \leq 0$）**：需通过变量替换（如 $x'_j = -x_j \geq 0$）转化为非负形式。替换后原问题的目标系数与约束矩阵相应列符号反转，对偶约束方向随之反向。

（3）**原问题变量无约束**：对偶约束必须为等式（=）。无约束变量可自由取值，其对偶约束需严格平衡目标函数与资源消耗的关系，避免解无界。

除上述情况外，还可能遇到原问题包含非标准约束或变量符号的情况。此时需通过数学变形转化为正常情况：

（1）**约束条件符号不符合正常情况**：- 最大化问题中出现 $\geq$ 约束时，等式两边乘以 -1 转化为 $\leq$；- 最小化问题中出现 $\leq$ 约束时，同样乘以 -1 转化为 $\geq$。- 转化后，对偶变量仍按正常情况的符号规则处理。

例如，原问题为最大化但约束为 $\geq$ 时，乘以 -1 后约束变为 $\leq$。按正常情况的规则，转化后这一行的对偶变量 $\hat{y}_i \geq 0$；而原约束行的对偶变量是它的相反数 $y_i = -\hat{y}_i \leq 0$（约束行整体乘以 -1 等价于对应对偶变量变号），即最大化问题的 $\geq$ 约束对应对偶变量非正。

（2）**变量符号非正常**：- 非正变量（$x_j \leq 0$）通过替换 $x_j = -x'_j$（$x'_j \geq 0$）转化为非负形式，此时原问

题的目标系数与约束矩阵对应列符号反转。- 变量无约束时直接对应对偶约束为等式，无需额外调整。

上述规则的本质是通过数学对称性揭示资源的隐含价值（影子价格）。可将原问题与对偶问题的符号对应归纳为以下原则：

（1）原问题约束类型： - $\leq$（最大化）或 $\geq$（最小化）$\rightarrow$ 对偶变量非负（$y_i \geq 0$）；- $=$ $\rightarrow$ 对偶变量无约束。

（2）原问题变量符号： - $x_j \geq 0 \rightarrow$ 对偶约束为 $\leq$（最大化）或 $\geq$（最小化）；- $x_j \leq 0 \rightarrow$ 对偶约束方向反向；- x_j 无约束 $\rightarrow$ 对偶约束为等式。

13.1.5 强对偶定理与互补松弛条件

对于DEA模型对应的线性规划问题，**强对偶定理（Strong Duality Theorem）**适用，因为DEA模型满足线性规划的标准形式要求。强对偶定理保证了：若原始问题存在最优解，则对偶问题也存在最优解，且两者的最优目标函数值相等。

与强对偶定理密切相关的**互补松弛条件（Complementary Slackness Conditions）**具有重要的经济学含义。设 s_i 为原始问题第 i 个约束的松弛变量，y_i 为对应的对偶变量，则互补松弛条件要求：

$$s_i \cdot y_i = 0, \quad \forall i$$

这一条件的经济含义极为直观：**如果某投入要素未充分发挥作用（存在松弛，即 $s_i > 0$），则其影子价格为零（$y_i = 0$）**——因为该资源不稀缺，增加一单位该资源不会改变最优效率。反之，如果某资源的影子价格为正（$y_i > 0$），则该约束必须是紧的（$s_i = 0$），即该资源被充分利用。

在DEA效率评价中，互补松弛条件意味着：处于生产前沿（效率值为1）的DMU，其至少一个投入或产出约束的影子价格为正；而存在松弛的DMU（即使是前沿上的DMU，也可能在某些维度上有松弛），对应松弛变量的影子价格必为零。

13.2 SBM模型的对偶问题

了解如何从原问题导出对偶问题后，下面考察Tone（2003）提出的SBM模型的线性规划问题：

$$\begin{aligned}
\min \quad & \tau = t - \frac{1}{P}\sum_{p=1}^{P} S_p^X / X_{0p} \\
\text{s.t.} \quad & 1 = t + \frac{1}{Q+R}\left(\sum_{q=1}^{Q} S_q^Y / Y_{0q} + \sum_{r=1}^{R} S_r^B / B_{0r}\right) \\
& tX_{0p} = \sum_{i=1}^{N} \Lambda_i X_{ip} + S_p^X, \quad p = 1, \ldots, P \\
& tY_{0q} = \sum_{i=1}^{N} \Lambda_i Y_{iq} - S_q^Y, \quad q = 1, \ldots, Q \\
& tB_{0r} = \sum_{i=1}^{N} \Lambda_i B_{ir} + S_r^B, \quad r = 1, \ldots, R \\
& \Lambda_i, S_p^X, S_q^Y, S_r^B \geq 0, \quad t > 0
\end{aligned}$$

首先，由于 $t>0$ 已暗含在第一个约束条件中，可以去掉原线性规划中 $t>0$ 的约束。此外，为便于从原问题写出对偶问题，可以将原问题的目标函数和约束条件中的决策变量按照固定的顺序排列，系数为0的项补0。从而得到如下线性规划问题：

$$
\begin{aligned}
\min \quad & \tau=\sum_{i=1}^{N}\Lambda_i 0-\frac{1}{P}\sum_{p=1}^{P}\frac{S_p^X}{X_{0p}}+S_q^Y 0+S_r^B 0+t \\
\text{s.t.} \quad & \sum_{i=1}^{N}\Lambda_i 0+S_p^X 0+\frac{1}{Q+R}\sum_{q=1}^{Q}\frac{S_q^Y}{Y_{0q}}+\frac{1}{Q+R}\sum_{r=1}^{R}\frac{S_r^B}{B_{0r}}+t=1 \\
& \sum_{i=1}^{N}\Lambda_i X_{ip}+S_p^X+S_q^Y 0+S_r^B 0-tX_{0p}=0, \quad p=1,\dots,P \\
& \sum_{i=1}^{N}\Lambda_i Y_{iq}+S_p^X 0-S_q^Y+S_r^B 0-tY_{0q}=0, \quad q=1,\dots,Q \\
& \sum_{i=1}^{N}\Lambda_i B_{ir}+S_p^X 0+S_q^Y 0+S_r^B-tB_{0r}=0, \quad r=1,\dots,R \\
& \Lambda_i, S_p^X, S_q^Y, S_r^B \geq 0
\end{aligned}
$$

下面根据原问题写出对偶问题。机械对偶得到的乘数中，投入等式的对偶变量本身取负号；为了让所有乘数都读作**经济价格**（投入、期望产出、非期望产出的边际价值均为正），下式统一取 $v=-v_{\text{机械}}$，并相应地把与之关联的约束一并变号。整套符号如下：

$$
\begin{aligned}
\max \quad & \xi \\
\text{s.t.} \quad & -v'X_i+u^{g\prime}Y_i-u^{b\prime}B_i\leq 0, \quad i=1,\dots,N \\
& v_p\geq\frac{1}{PX_{0p}}, \quad p=1,\dots,P \\
& u_q^g\geq\frac{\xi}{(Q+R)Y_{0q}}, \quad q=1,\dots,Q \\
& u_r^b\geq\frac{\xi}{(Q+R)B_{0r}}, \quad r=1,\dots,R \\
& \xi+v'X_0-u^{g\prime}Y_0+u^{b\prime}B_0=1 \\
& \xi\in\mathbb{R}
\end{aligned}
\tag{13.5}
$$

其中，ξ 为对偶问题的目标函数值，v、u^g 和 u^b 分别为投入、期望产出和非期望产出的影子价格向量。原问题中 t 是自由变量，故其对应的对偶约束（第五式）取等号；ξ 在对偶中同样是自由变量，其非负性来自最优值 $\xi^*=\tau^*>0$ 而非人为施加。式(13.5)的第五个约束对应原始问题中的Charnes-Cooper变换约束 $t+\frac{1}{Q+R}(\sum S^Y/Y_0+\sum S^B/B_0)=1$；第二至第四式则说明，只要 $\xi>0$，三类影子价格都自动严格为正，无需额外附加非负约束。

关于式(13.5)有三点需要说明。

第一，**符号必须成套使用**。第一至第四个约束与第五个约束里的 v 是同一套变量，若把第一个约束写成 $+v'X_i$（机械对偶的符号），则第二个约束必须同时改为 $v_p\leq -1/(PX_{0p})$、第五个约束改为 $\xi-v'X_0-\cdots$。两套符号混写在同一个线性规划里，既推不出下一节的代码，也解不出正的投入影子价格。

第二，**非期望产出被当作投入处理**。第一个约束中 B_i 与 X_i 同号（都带负号），这正是Tone口径下"坏产出计入投入侧"的体现，因此 u^b 在本模型中与投入影子价格同类、取非负值（不同技术设定下坏产出对

偶的符号差异见 13.4.1 节）。

第三，ξ **就是效率值本身，不是它的倒数**。ξ 与线性化SBM原问题的目标 τ 通过强对偶相等，而Charnes-Cooper变换又保证 $\tau^* = \rho^*$（分式形式的SBM效率），故 $\xi^* = \tau^* = \rho^* \in (0, 1]$。

13.3 基于SBM模型对偶问题的影子价格计算程序

13.3.1 数值注意事项与数据预处理

本章与第12章共用2016—2019年中国省级面板。程序示范按各列全样本均值缩放K、L、E、Y、CO2，保留正值并避免相差过大的系数。缩放不含中心化；效率保持原定义，对偶乘数则随单位改变。若原变量除以s得到缩放变量，原单位乘数应为缩放乘数除以s。任何货币化解释还需要产出价格和物理单位，不能沿用其他数据文件的千克或万元口径。

```
from deabook.example.book_cases.loaders import china
raw = china()
columns = ["K", "L", "E", "Y", "CO2"]
scale = raw[columns].mean()
data = raw.copy()
data[columns] = raw[columns] / scale
```

13.3.2 单个DMU的影子价格计算

在式(13.5)给出SBM对偶问题之后，本节以 本章省级面板数据为算例，展示如何借助Pyomo将该对偶问题转化为可求解的线性规划，并从中提取影子价格。代码首先读取同一省级面板并按列均值缩放（理由见 13.3.1 节），随后将投入、期望产出与非期望产出整理为参照集与被评价单元的列块。实现沿用本章统一的技术约定：采用NumPy风格docstring与snake_case命名以增强可读性，求解器默认采用 `highs`，默认参数以 `None` 表示。模型中的 `px`、`py`、`pb`、`pomega` 四组变量正是式(13.5)中的 v、u^g、u^b 与 ξ；五个约束 `first` 至 `fifth` 逐条对应式(13.5)的五个条件。

```
"""
基于SBM对偶问题的影子价格计算模块。

本模块提供基于SBM模型对偶问题估计投入、产出和非期望产出
影子价格的函数。求解器默认采用 highs。
"""
from pyomo.environ import (
    ConcreteModel, Set, Var, Objective, Constraint, Reals,
    maximize, minimize, SolverFactory, value
)
import pandas as pd
from deabook.example.book_cases.loaders import china, china_lower
import numpy as np
```

```
import re
import warnings

warnings.filterwarnings("ignore")

# 读取数据
ex4 = china(scaled=True)
data = ex4[["K", "L", "Y", "CO2"]].copy()  # 提取投入、期望产出和非期望产出变量

# 设定变量个数
K = 2  # 投入变量个数
L = 1  # 期望产出变量个数
M = 1  # 非期望产出变量个数

# 准备参照集数据
xref = data.iloc[:, 0:K]
yref = data.iloc[:, K:K+L]
bref = data.iloc[:, K+L:K+L+M]

# 准备被评价DMU数据(以第1个DMU为例)
x = data.iloc[0, 0:K]
y = data.iloc[0, K:K+L]
b = data.iloc[0, K+L:K+L+M]

# 构建Pyomo模型
model = ConcreteModel()
model.I = Set(initialize=range(xref.shape[0]))  # 参照决策单元集合
model.K = Set(initialize=range(len(x)))          # 投入变量集合
model.L = Set(initialize=range(len(y)))          # 期望产出变量集合
model.M = Set(initialize=range(len(b)))          # 非期望产出变量集合

# 定义影子价格变量(施加非负约束以确保经济学含义)
model.px = Var(model.K, bounds=(0, None), within=Reals, doc='投入的影子价格')
model.py = Var(model.L, bounds=(0, None), within=Reals, doc='期望产出的影子价格')
model.pb = Var(model.M, bounds=(0, None), within=Reals, doc='非期望产出的影子价格')
model.pomega = Var(bounds=(0, None), within=Reals, doc='SBM效率值(式(13.5)中的 xi)')

# 目标函数:最大化 pomega
def objective_rule(model):
```

```
    """返回目标函数表达式"""
    return model.pomega * 1

# 约束1:对偶约束(参照集加权影子价格之和非正)
def first_rule(model, i):
    """第i个参照DMU的对偶约束"""
    return (
        -sum(model.px[k] * xref.iloc[i, k] for k in model.K)
        + sum(model.py[l] * yref.iloc[i, l] for l in model.L)
        - sum(model.pb[m] * bref.iloc[i, m] for m in model.M)
        <= 0
    )

# 约束2:投入影子价格的下界约束
def second_rule(model, k):
    """第k个投入变量的影子价格下界"""
    return -model.px[k] <= -1 / (len(model.K) * x.iloc[k])

# 约束3:期望产出影子价格的上界约束
def third_rule(model, l):
    """第l个期望产出变量的影子价格上界"""
    return (
        model.pomega / (len(model.L) + len(model.M)) / y.iloc[l] - model.py[l]
        <= 0
    )

# 约束4:非期望产出影子价格的上界约束
def forth_rule(model, m):
    """第m个非期望产出变量的影子价格上界"""
    return (
        model.pomega / (len(model.L) + len(model.M)) / b.iloc[m] - model.pb[m]
        <= 0
    )

# 约束5: Charnes-Cooper变换的归一化约束
def fifth_rule(model):
    """归一化约束"""
    return (
        model.pomega
```

```
        + sum(model.px[k] * x.iloc[k] for k in model.K)
        - sum(model.py[l] * y.iloc[l] for l in model.L)
        + sum(model.pb[m] * b.iloc[m] for m in model.M)
        <= 1
    )

# 组装模型
model.obj = Objective(rule=objective_rule, sense=maximize, doc='目标函数')
model.first = Constraint(model.I, rule=first_rule, doc='对偶约束')
model.second = Constraint(model.K, rule=second_rule, doc='投入下界约束')
model.third = Constraint(model.L, rule=third_rule, doc='期望产出上界约束')
model.forth = Constraint(model.M, rule=forth_rule, doc='非期望产出上界约束')
model.fifth = Constraint(rule=fifth_rule, doc='归一化约束')

# 求解
opt = SolverFactory('highs')  # 指定 HiGHS 作为求解器
solution = opt.solve(model)   # 调用求解器求解

# 提取结果
px = np.asarray(list(model.px[:].value))
py = np.asarray(list(model.py[:].value))
pb = np.asarray(list(model.pb[:].value))
obj = value(model.obj)

print(f"投入影子价格 (px): {px}")
print(f"期望产出影子价格 (py): {py}")
print(f"非期望产出影子价格 (pb): {pb}")
print(f"最优目标值: {obj}")
```

需要特别指出的是，上述代码对决策变量施加了下界为0的约束，而机械对偶得到的乘数是自由变量。这一处理是**额外施加的**，而不是Tone对偶本身的要求，其依据是经济学含义：影子价格通常不应小于零——投入的影子价格为正表示资源的稀缺性，产出的影子价格为正表示其价值。在式(13.5)的经济价格口径下，第二至第四个约束已经蕴含 $v>0$、$u^g>0$、$u^b>0$，因此这组下界在正常数据下不会真正起作用，只是在数据含零值或求解退化时起到兜住符号的作用。

代码与式(13.5)的另一处差别是第五个约束写成了`<= 1`而不是等号。二者的最优值相同：目标函数与第一至第四个约束在 (ξ, v, u^g, u^b) 上都是正齐次的（把整组变量同比例放大 $k>1$ 倍，第一式仍 ≤ 0，第二至第四式的下界要求同样满足），而第五式左端是一次齐次的。若最优解处第五式留有松弛（左端 $=1-s<1$，$s>0$），把整组变量放大 $1/(1-s)$ 倍即可得到更大的 ξ，与最优性矛盾。故最优解处该式必取等号，写成不等式只是给求解器留出数值余量。

命名说明：代码中 `px`、`py`、`pb` 分别对应投入（x）、期望产出（y）和非期望产出（b）的影子价格。

`pomega` 对应式(13.5)中的 ξ，即 **SBM效率值本身**（$\xi^* = \tau^* = \rho^* \in (0,1]$），不是效率的倒数。`first` 至 `fifth` 分别对应式(13.5)的五个约束条件。

13.3.3 迭代求解所有DMU

将单一DMU的求解推广至全部样本，只需在每次循环中对一个DMU按式(13.5)重建并求解对偶线性规划，并把最优解存入字典 `px`、`py`、`pb` 与 `obj`。其中 `termination_condition == "optimal"` 用于过滤求解失败的情形；但须注意，如 13.3.1 节所示，在量纲未校正的病态条件下求解器仍可能报 `optimal` 而返回错误价格，故该判断本身不足以保证估计质量。

```
px, py, pb, obj = {}, {}, {}, {}

for j in range(data.shape[0]):
    x = data.iloc[j, 0:K]
    y = data.iloc[j, K:K+L]
    b = data.iloc[j, K+L:K+L+M]

    model = ConcreteModel()
    model.I = Set(initialize=range(xref.shape[0]))
    model.K = Set(initialize=range(len(x)))
    model.L = Set(initialize=range(len(y)))
    model.M = Set(initialize=range(len(b)))

    model.px = Var(model.K, bounds=(0, None), within=Reals, doc='投入的影子价格')
    model.py = Var(model.L, bounds=(0, None), within=Reals, doc='期望产出的影子价格')
    model.pb = Var(model.M, bounds=(0, None), within=Reals, doc='非期望产出的影子价格')
    model.pomega = Var(bounds=(0, None), within=Reals,
        doc='SBM效率值(式(13.5)中的 xi)')

    def objective_rule(model):
        return model.pomega * 1

    def first_rule(model, i):
        return (
            -sum(model.px[k] * xref.iloc[i, k] for k in model.K)
            + sum(model.py[l] * yref.iloc[i, l] for l in model.L)
            - sum(model.pb[m] * bref.iloc[i, m] for m in model.M)
            <= 0
        )

    def second_rule(model, k):
```

```
        return -model.px[k] <= -1 / (len(model.K) * x.iloc[k])

    def third_rule(model, l):
        return (
            model.pomega / (len(model.L) + len(model.M)) / y.iloc[
                l] - model.py[l]
            <= 0
        )

    def forth_rule(model, m):
        return (
            model.pomega / (len(model.L) + len(model.M)) / b.iloc[
                m] - model.pb[m]
            <= 0
        )

    def fifth_rule(model):
        return (
            model.pomega
            + sum(model.px[k] * x.iloc[k] for k in model.K)
            - sum(model.py[l] * y.iloc[l] for l in model.L)
            + sum(model.pb[m] * b.iloc[m] for m in model.M)
            <= 1
        )

    model.obj = Objective(rule=objective_rule, sense=maximize, doc='目标函数')
    model.first = Constraint(model.I, rule=first_rule, doc='对偶约束')
    model.second = Constraint(model.K, rule=second_rule, doc='投入下界约束')
    model.third = Constraint(model.L, rule=third_rule, doc='期望产出上界约束')
    model.forth = Constraint(model.M, rule=forth_rule, doc='非期望产出上界约束')
    model.fifth = Constraint(rule=fifth_rule, doc='归一化约束')

    opt = SolverFactory('highs')
    solution = opt.solve(model)

    # 终止条件判断：optimal（最优）、infeasible（不可行）、unbounded（无界）
    if solution.solver.termination_condition == "optimal":
        px[j] = np.asarray(list(model.px[:].value))
        py[j] = np.asarray(list(model.py[:].value))
```

```
        pb[j] = np.asarray(list(model.pb[:].value))
        obj[j] = value(model.obj)
```

13.3.4 面向复用的函数封装

为便于在实证研究中重复调用，上述逐DMU循环可被一般化为一个返回数据框的函数 `sbm_dual`。其函数首部以docstring说明参数与返回值，内部逻辑与 13.3.3 节完全一致，仅将参照集与被评价单元的切片以 `varname`、`num_inputs`、`num_outputs` 一般化，使任意变量组合均可直接传入。

```
from pyomo.environ import (
    ConcreteModel, Set, Var, Objective, Constraint, Reals,
    maximize, SolverFactory, value
)
import pandas as pd
from deabook.example.book_cases.loaders import china, china_lower
import numpy as np
import re


def sbm_dual(dataframe, varname, num_inputs, num_outputs):
    """
    基于SBM模型对偶问题估计影子价格。

    求解SBM模型的对偶问题(乘数形式),获得各决策单元的投入、
    期望产出和非期望产出的影子价格。

    Parameters
    ----------
    dataframe : pd.DataFrame
        待评价决策单元的投入产出数据框,变量按投入在前、
        期望产出在中、非期望产出在后的顺序排列。
    varname : list[str]
        投入变量、期望产出变量和非期望产出变量的名称列表,
        如 ["K", "L", "Y", "CO2"]。
    num_inputs : int
        投入变量的个数(P)。
    num_outputs : int
        期望产出变量的个数(Q)。

    Returns
```

```
    -------
    pd.DataFrame
        包含各DMU的影子价格估计结果的数据框。列包括：
        - 'obj': SBM效率值（式(13.5)中的 xi，取值于 (0, 1]）
        - 'shadow price <varname>': 各变量的影子价格

    Notes
    -----
    - 本函数对影子价格施加非负约束，以确保经济学含义的合理性。
    - 求解器默认使用 highs。
    - 关于SBM模型及其含非期望产出扩展的原问题形式，参见第8章（式(8.1)与式(8.3)）。

    Examples
    --------
    >>> ex4 = china(scaled=True)
    >>> result = sbm_dual(ex4, ["K", "L", "Y", "CO2"], 2, 1)
    >>> print(result.head())
    """
    data = dataframe.loc[:, varname]
    num_inputs_ = num_inputs
    num_outputs_ = num_outputs
    num_bad = data.shape[1] - num_inputs_ - num_outputs_

    xref = data.iloc[:, 0:num_inputs_]
    yref = data.iloc[:, num_inputs_:num_inputs_+num_outputs_]
    bref = data.iloc[:, num_inputs_+num_outputs_:num_inputs_+num_outputs_+num_bad]

    xcol = varname[0:num_inputs_]
    ycol = varname[num_inputs_:num_inputs_+num_outputs_]
    bcol = varname[num_inputs_+num_outputs_:num_inputs_+num_outputs_+num_bad]

    px, py, pb, obj = {}, {}, {}, {}

    for j in range(data.shape[0]):
        x = data.iloc[j, 0:num_inputs_]
        y = data.iloc[j, num_inputs_:num_inputs_+num_outputs_]
        b = data.iloc[j, num_inputs_+num_outputs_:num_inputs_+num_outputs_+num_bad]

        model = ConcreteModel()
```

```
    model.I = Set(initialize=xref.index)
    model.K = Set(initialize=range(len(x)))
    model.L = Set(initialize=range(len(y)))
    model.M = Set(initialize=range(len(b)))

    model.px = Var(model.K, bounds=(0, None), within=Reals, doc='投入的影子价格')
    model.py = Var(model.L, bounds=(0, None), within=Reals, doc='期望产出的影子价格')
    model.pb = Var(model.M, bounds=(0, None), within=Reals, doc='非期望产出的影子价格')
    model.pomega = Var(bounds=(0, None), within=Reals, doc='SBM效率值(式(13.5)中的 xi)')

    def objective_rule(model):
        return model.pomega * 1

    def first_rule(model, i):
        return (
            -sum(model.px[k] * xref.loc[i, xcol[k]] for k in model.K)
            + sum(model.py[l] * yref.loc[i, ycol[l]] for l in model.L)
            - sum(model.pb[m] * bref.loc[i, bcol[m]] for m in model.M)
            <= 0
        )

    def second_rule(model, k):
        return -model.px[k] <= -1 / (len(model.K) * x.iloc[k])

    def third_rule(model, l):
        return (
            model.pomega / (len(model.L) + len(model.M)) / y.iloc[l] - model.py[l]
            <= 0
        )

    def forth_rule(model, m):
        return (
            model.pomega / (len(model.L) + len(model.M)) / b.iloc[m] - model.pb[m]
            <= 0
        )

    def fifth_rule(model):
        return (
            model.pomega
```

```
                + sum(model.px[k] * x.iloc[k] for k in model.K)
                - sum(model.py[l] * y.iloc[l] for l in model.L)
                + sum(model.pb[m] * b.iloc[m] for m in model.M)
                <= 1
            )

        model.obj = Objective(rule=objective_rule, sense=maximize, doc='目标函数')
        model.first = Constraint(model.I, rule=first_rule, doc='对偶约束')
        model.second = Constraint(model.K, rule=second_rule, doc='投入下界约束')
        model.third = Constraint(model.L, rule=third_rule, doc='期望产出上界约束')
        model.forth = Constraint(model.M, rule=forth_rule, doc='非期望产出上界约束')
        model.fifth = Constraint(rule=fifth_rule, doc='归一化约束')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        if solution.solver.termination_condition == "optimal":
            px[j] = np.asarray(list(model.px[:].value))
            py[j] = np.asarray(list(model.py[:].value))
            pb[j] = np.asarray(list(model.pb[:].value))
            obj[j] = value(model.obj)

    objdf = pd.DataFrame(obj, index=["obj"]).T
    pxdf = pd.DataFrame(px, index=xcol).T
    pxdf.columns = pxdf.columns.map(lambda x: "shadow price " + str(x))
    pydf = pd.DataFrame(py, index=ycol).T
    pydf.columns = pydf.columns.map(lambda y: "shadow price " + str(y))
    pbdf = pd.DataFrame(pb, index=bcol).T
    pbdf.columns = pbdf.columns.map(lambda b: "shadow price " + str(b))

    pdf = pd.concat([pxdf, pydf], axis=1)
    pdf = pd.concat([pdf, pbdf], axis=1)
    redata = pd.concat([objdf, pdf], axis=1)

    return redata
```

以 本章省级面板 调用 `sbm_dual`，可得各DMU的影子价格估计数据框：

```
import pandas as pd
from deabook.example.book_cases.loaders import china, china_lower
```

```
ex4 = china(scaled=True)
result = sbm_dual(ex4, ["K", "L", "Y", "CO2"], 2, 1)
print(result.head())
```

13.3.5 支持样本和参照集限定

实证分析中往往只需对部分DMU定价(如特定行业或时期),或希望以特定子集作为技术参照集。`sbm_dual_select` 在 `sbm_dual` 的基础上新增 `eval_query` 与 `ref_query` 两个参数,分别经 `pandas.query` 筛选待评价单元与参照集,从而使同一函数既能完成全样本估计,也能刻画受限技术前沿下的影子价格。

```
def sbm_dual_select(dataframe, varname, num_inputs, num_outputs,
                    eval_query, ref_query):
    """
    基于SBM对偶问题估计影子价格(支持评价单元和参照集筛选)。

    Parameters
    ----------
    dataframe : pd.DataFrame
        待评价决策单元的投入产出数据框。
    varname : list[str]
        投入、期望产出和非期望产出变量的名称列表,
        如 ["K", "L", "Y", "CO2"]。
    num_inputs : int
        投入变量的个数。
    num_outputs : int
        期望产出变量的个数。
    eval_query : str
        传入数据框.query()方法的参数,用于筛选待评价DMU,
        如 "dmu==1"、"dmu==[1,2,3]"。
    ref_query : str
        传入数据框.query()方法的参数,用于筛选参照集DMU,
        如 "dmu==1"、"dmu==[1,2,3]"。

    Returns
    -------
    pd.DataFrame
        包含各DMU影子价格估计结果的数据框。
    """
    index_list = dataframe.query(eval_query).index
    index_list_ref = dataframe.query(ref_query).index
```

```
data = dataframe.loc[index_list, varname]
dataref = dataframe.loc[index_list_ref, varname]

num_inputs_ = num_inputs
num_outputs_ = num_outputs
num_bad = data.shape[1] - num_inputs_ - num_outputs_

xref = dataref.iloc[:, 0:num_inputs_]
yref = dataref.iloc[:, num_inputs_:num_inputs_+num_outputs_]
bref = dataref.iloc[:, num_inputs_+num_outputs_:num_inputs_+num_outputs_+
    num_bad]

xcol = varname[0:num_inputs_]
ycol = varname[num_inputs_:num_inputs_+num_outputs_]
bcol = varname[num_inputs_+num_outputs_:num_inputs_+num_outputs_+num_bad]

px, py, pb, obj = {}, {}, {}, {}

for j in data.index:
    x = data.loc[j, xcol]
    y = data.loc[j, ycol]
    b = data.loc[j, bcol]

    model = ConcreteModel()
    model.I = Set(initialize=dataref.index)
    model.K = Set(initialize=range(len(x)))
    model.L = Set(initialize=range(len(y)))
    model.M = Set(initialize=range(len(b)))

    model.px = Var(model.K, bounds=(0, None), within=Reals, doc='投入的影子价格')
    model.py = Var(model.L, bounds=(0, None), within=Reals,
        doc='期望产出的影子价格')
    model.pb = Var(model.M, bounds=(0, None), within=Reals,
        doc='非期望产出的影子价格')
    model.pomega = Var(bounds=(0, None), within=Reals,
        doc='SBM效率值(式(13.5)中的 xi)')

    def objective_rule(model):
        return model.pomega * 1
```

```
    def first_rule(model, i):
        return (
            -sum(model.px[k] * xref.loc[i, xcol[k]] for k in model.K)
            + sum(model.py[l] * yref.loc[i, ycol[l]] for l in model.L)
            - sum(model.pb[m] * bref.loc[i, bcol[m]] for m in model.M)
            <= 0
        )

    def second_rule(model, k):
        return -model.px[k] <= -1 / (len(model.K) * x.iloc[k])

    def third_rule(model, l):
        return (
            model.pomega / (len(model.L) + len(model.M)) / y.iloc[
                l] - model.py[l]
            <= 0
        )

    def forth_rule(model, m):
        return (
            model.pomega / (len(model.L) + len(model.M)) / b.iloc[
                m] - model.pb[m]
            <= 0
        )

    def fifth_rule(model):
        return (
            model.pomega
            + sum(model.px[k] * x.iloc[k] for k in model.K)
            - sum(model.py[l] * y.iloc[l] for l in model.L)
            + sum(model.pb[m] * b.iloc[m] for m in model.M)
            <= 1
        )

    model.obj = Objective(rule=objective_rule, sense=maximize, doc='目标函数')
    model.first = Constraint(model.I, rule=first_rule, doc='对偶约束')
    model.second = Constraint(model.K, rule=second_rule, doc='投入下界约束')
    model.third = Constraint(model.L, rule=third_rule, doc='期望产出上界约束')
    model.forth = Constraint(model.M, rule=forth_rule, doc='非期望产出上界约束')
```

```
        model.fifth = Constraint(rule=fifth_rule, doc='归一化约束')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        if solution.solver.termination_condition == "optimal":
            px[j] = np.asarray(list(model.px[:].value))
            py[j] = np.asarray(list(model.py[:].value))
            pb[j] = np.asarray(list(model.pb[:].value))
            obj[j] = value(model.obj)

    objdf = pd.DataFrame(obj, index=["obj"]).T
    pxdf = pd.DataFrame(px, index=xcol).T
    pxdf.columns = pxdf.columns.map(lambda x: "shadow price " + str(x))
    pydf = pd.DataFrame(py, index=ycol).T
    pydf.columns = pydf.columns.map(lambda y: "shadow price " + str(y))
    pbdf = pd.DataFrame(pb, index=bcol).T
    pbdf.columns = pbdf.columns.map(lambda b: "shadow price " + str(b))

    pdf = pd.concat([pxdf, pydf], axis=1)
    pdf = pd.concat([pdf, pbdf], axis=1)
    redata = pd.concat([objdf, pdf], axis=1)

    return redata
```

命名说明：`eval_query`（evaluation query）表示用于筛选待评价决策单元的查询条件字符串，`ref_query`（reference query）表示用于筛选参照集决策单元的查询条件字符串。使用下划线分隔的命名方式（snake_case）是Python的PEP 8标准命名规范。原稿中的 `evaquery` 和 `refquery` 已修正。

加入筛选后，索引必须"标签对标签、位置对位置"。一旦用 `query` 筛选参照集，`dataref.index` 里装的就是**原始行标签**（例如筛出第3期后可能是60、61、…、89），而不再是 $0, 1, \ldots, N-1$。因此 `model.I = Set(initialize=dataref.index)` 之后，取参照单元的数据只能用标签索引 `xref.loc[i, xcol[k]]`；若沿用位置索引 `xref.iloc[i, k]`，标签会被当成行号：标签超出行数时抛 `IndexError`，没超出时更危险——程序照样跑完，但每一条对偶约束都对应了错误的参照DMU，影子价格与被评价单元完全对不上。反过来，若想用位置索引，就要先 `dataref = dataref.reset_index(drop=True)` 再令 `model.I = Set(initialize=range(xref.shape[0]))`，两者不能混搭（13.3.2 节的整段示例是"位置对位置"的写法，可以对照）。同理，被评价单元的 `x = data.loc[j, xcol]` 是以 `K`、`L` 等**列名**为索引的Series，`model.K` 里却是 $0, 1, \ldots$，所以要写 `x.iloc[k]`；`x[k]` 在pandas 2下只是靠"整数键回退按位置查找"的兼容行为侥幸可用，会触发 `FutureWarning`，pandas 3起将直接按标签查找并

抛 KeyError。

13.3.6 支持公式字符串调用

为降低调用时的变量顺序约束，可进一步提供以公式字符串"期望产出变量：非期望产出变量= 投入变量" 声明模型结构的接口。sbm_dual_formula 解析该字符串自动完成变量分组，并同样支持 eval_query、ref_query 限定评价与参照范围，使模型设定与数据调用相分离。

```
def sbm_dual_formula(formula, dataframe, eval_query=None, ref_query=None):
    """
    基于SBM对偶问题估计影子价格(支持公式字符串调用)。

    Parameters
    ----------
    formula : str
        公式字符串,格式为 "Y : CO2 = K L"。
        等号左边为产出变量(冒号前为期望产出,冒号后为非期望产出),
        等号右边为投入变量。
    dataframe : pd.DataFrame
        待评价决策单元的投入产出数据框。
    eval_query : str or None, default None
        传入数据框.query()方法的参数,用于筛选待评价DMU。
        若为None,则评价所有DMU。
    ref_query : str or None, default None
        传入数据框.query()方法的参数,用于筛选参照集DMU。
        若为None,则使用全部DMU作为参照集。

    Returns
    -------
    pd.DataFrame
        包含各DMU影子价格估计结果的数据框。
    """
    px, py, pb, obj = {}, {}, {}, {}

    if eval_query is None:
        index_list = dataframe.index
    else:
        index_list = dataframe.query(eval_query).index

    if ref_query is None:
        index_list_ref = dataframe.index
```

```
    else:
        index_list_ref = dataframe.query(ref_query).index

    # 解析公式字符串
    input_vars = formula.split('=')[1].strip(' ')
    xcol = re.compile(' +').sub(' ', input_vars).split(' ')
    output_vars = formula.split('=')[0].split(':')[0].strip(' ')
    ycol = re.compile(' +').sub(' ', output_vars).split(' ')
    unoutput_vars = formula.split('=')[0].split(':')[1].strip(' ')
    bcol = re.compile(' +').sub(' ', unoutput_vars).split(' ')

    data = dataframe.loc[index_list, xcol + ycol + bcol]
    dataref = dataframe.loc[index_list_ref, xcol + ycol + bcol]

    xref = dataref.loc[:, xcol]
    yref = dataref.loc[:, ycol]
    bref = dataref.loc[:, bcol]

    for j in data.index:
        x = data.loc[j, xcol]
        y = data.loc[j, ycol]
        b = data.loc[j, bcol]

        model = ConcreteModel()
        model.I = Set(initialize=dataref.index)
        model.K = Set(initialize=range(len(x)))
        model.L = Set(initialize=range(len(y)))
        model.M = Set(initialize=range(len(b)))

        model.px = Var(model.K, bounds=(0, None), within=Reals, doc=' 投入的影子价格')
        model.py = Var(model.L, bounds=(0, None), within=Reals,
            doc=' 期望产出的影子价格')
        model.pb = Var(model.M, bounds=(0, None), within=Reals,
            doc=' 非期望产出的影子价格')
        model.pomega = Var(bounds=(0, None), within=Reals,
            doc='SBM效率值(式(13.5)中的 xi) ')

        def objective_rule(model):
            return model.pomega * 1
```

```
    def first_rule(model, i):
        return (
            -sum(model.px[k] * xref.loc[i, xcol[k]] for k in model.K)
            + sum(model.py[l] * yref.loc[i, ycol[l]] for l in model.L)
            - sum(model.pb[m] * bref.loc[i, bcol[m]] for m in model.M)
            <= 0
        )

    def second_rule(model, k):
        return -model.px[k] <= -1 / (len(model.K) * x.iloc[k])

    def third_rule(model, l):
        return (
            model.pomega / (len(model.L) + len(model.M)) / y.iloc[
                l] - model.py[l]
            <= 0
        )

    def forth_rule(model, m):
        return (
            model.pomega / (len(model.L) + len(model.M)) / b.iloc[
                m] - model.pb[m]
            <= 0
        )

    def fifth_rule(model):
        return (
            model.pomega
            + sum(model.px[k] * x.iloc[k] for k in model.K)
            - sum(model.py[l] * y.iloc[l] for l in model.L)
            + sum(model.pb[m] * b.iloc[m] for m in model.M)
            <= 1
        )

    model.obj = Objective(rule=objective_rule, sense=maximize, doc='目标函数')
    model.first = Constraint(model.I, rule=first_rule, doc='对偶约束')
    model.second = Constraint(model.K, rule=second_rule, doc='投入下界约束')
    model.third = Constraint(model.L, rule=third_rule, doc='期望产出上界约束')
    model.forth = Constraint(model.M, rule=forth_rule, doc='非期望产出上界约束')
```

```
        model.fifth = Constraint(rule=fifth_rule, doc='归一化约束')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        if solution.solver.termination_condition == "optimal":
            px[j] = np.asarray(list(model.px[:].value))
            py[j] = np.asarray(list(model.py[:].value))
            pb[j] = np.asarray(list(model.pb[:].value))
            obj[j] = value(model.obj)

    objdf = pd.DataFrame(obj, index=["obj"]).T
    pxdf = pd.DataFrame(px, index=xcol).T
    pxdf.columns = pxdf.columns.map(lambda x: "shadow price " + str(x))
    pydf = pd.DataFrame(py, index=ycol).T
    pydf.columns = pydf.columns.map(lambda y: "shadow price " + str(y))
    pbdf = pd.DataFrame(pb, index=bcol).T
    pbdf.columns = pbdf.columns.map(lambda b: "shadow price " + str(b))

    pdf = pd.concat([pxdf, pydf], axis=1)
    pdf = pd.concat([pdf, pbdf], axis=1)
    redata = pd.concat([objdf, pdf], axis=1)

    return redata
```

上述函数即可按公式字符串与查询条件直接调用，例如仅以第1—3期作为参照、评价第1—2期：

```
import pandas as pd
from deabook.example.book_cases.loaders import china, china_lower

ex4 = china(scaled=True)
result = sbm_dual_formula("Y : CO2 = K L", ex4, "t==[1,2]", "t==[1,2,3]")
print(result)
```

13.4 影子价格的合理解释范围

13.4.1 影子价格的经济含义

影子价格是线性规划对偶问题的解，在经济上表示约束条件的边际价值。在DEA框架下，影子价格的具体含义如下：

- **投入的影子价格为正**（$v > 0$）：该投入是稀缺的，减少一单位该投入会带来效率的改善（或需要付出代

价）。影子价格的绝对值越大，该投入越关键。

- **期望产出的影子价格为正**（$u^g > 0$）：该产出是有价值的，增加一单位该产出会提升效率评价。
- **非期望产出的影子价格**：符号取决于生产技术如何处理坏产出，见下文。

非期望产出影子价格的符号取决于技术设定。这一点在环境经济学应用中最容易被误读，必须分两种口径来看。

第一种是 **Tone口径的SBM（坏产出计入投入侧）**，即本章13.2—13.3 节的模型：式(13.5)第一个约束中 B_i 与 X_i 同号，坏产出与投入同类，其对偶 u^b 在经济价格口径下**非负**（$u^b > 0$ 表示减少一单位污染可改善效率评价）。13.5 节NDDF对偶(13.8)也采用同一口径：坏产出以 $+b'u_b$ 的形式进入" 虚拟利润"，作为成本项，故 $u_b \geq 0$。此时边际减排成本直接取 u_b/u_y。

第二种是 **Färe-Grosskopf弱可处置技术 / 方向距离函数**，即第6—7章使用的技术：坏产出作为产出出现在产出约束一侧，其对偶变量**一般为负**。这不是错误，而是" 多产一单位坏产出必须以少产好产出为代价" 的直接体现；此时边际减排成本报告为相对价格 $-u_b/u_y$（或 $-u_b/v$），负号是定义的一部分，并需注明单位（如元/吨）。文献中碳影子价格常被报告为负值，正是出自这一口径。

因此，判断 u^b 的符号是否" 正常"，第一步是确认所用技术属于上述哪一种，而不是先怀疑变量顺序设错。

13.4.2 异常值的识别与处理

在实际计算中，影子价格可能出现零值、负值或极端大值。这些异常值需要谨慎解释。

表 13.4: 影子价格的合理解释范围

影子价格值	经济含义	常见原因	处理建议
正值	资源稀缺，具有边际价值	正常情况	直接使用
零值	资源不稀缺（约束非紧致）	投入冗余；VRS模型中规模非有效区域	检查互补松弛条件；报告为" 无定价"
负值	投入与期望产出为负值时通常表示模型设定错误或数据问题；非期望产出为负值可能是技术设定的正常结果	投入产出方向设定错误；数据异常值；退化解；弱可处置/DDF技术下坏产出对偶本就为负	先按 13.4.1 节区分技术口径；再检查数据方向；检查退化
极端大值	数值不稳定	数据量纲差异大；DMU数量少；约束近似线性相关	按列缩放数据（不中心化）；增加样本量；检查多重共线性

影子价格为零的解释：

根据互补松弛条件，如果某投入变量存在松弛（即该投入未充分发挥作用，存在冗余），则其对应的影子价格必为零。这并不意味着该投入" 没有价值"，而是在**当前生产前沿下**，该投入不制约效率提升。常见情况包括：- **投入导向模型**中某些投入大量过剩的DMU；- **VRS模型**中处于规模收益递增或递减区域的DMU；- **非期望产出**对于某些已处于环境前沿的DMU。

影子价格为负的解释：

在本章的Tone口径（坏产出计入投入侧、影子价格施加非负下界）下，负影子价格通常表明模型设定或数据问题，可能原因包括：1. **投入产出方向设定错误**：例如将投入误标为产出，或将产出误标为投入；2. **数据中存在异常值**：某些DMU的数据与其他DMU显著不同，导致前沿面扭曲；3. **对偶问题的退化解（degenerate solution）**：多个约束同时起作用，导致对偶变量不唯一，求解器可能返回负值。

但如 13.4.1 节所述，若使用第6—7章的弱可处置/DDF技术，非期望产出的对偶为负是技术设定的正常结果，不应按上述三条排查。

13.4.3 退化问题与处理建议

退化（Degeneracy）是线性规划中的一个常见问题：当多个约束同时起作用（紧致）时，基变量的选择不唯一，导致对偶变量（影子价格）的解不唯一。在DEA中，退化非常常见，因为生产前沿由多个DMU共同构成，前沿上的DMU通常同时满足多个约束。

退化对影子价格估计的影响：- 同一DMU使用不同求解器可能得到不同的影子价格；- 即使使用同一求解器，不同的初始点也可能导致不同的解；- 影子价格的点估计可能不可靠，需要报告区间估计。

处理退化的建议：

1. **微小扰动法**（Epsilon-perturbation）：在约束右端项添加微小扰动 ϵ，使约束不再同时紧致，从而获得唯一的影子价格：

$$\sum_i \lambda_i X_{ip} + S_p^X = X_{0p} + \epsilon$$

2. **报告影子价格区间而非点估计**：通过求解多个辅助线性规划，获得影子价格的可能范围。
3. **使用两阶段法**：
   - 第一阶段：求解原始DEA问题，固定效率值；
   - 第二阶段：在固定效率值下，以影子价格的平方和最小为目标，选择唯一的影子价格。
4. **施加非负约束**：如本章代码所示，对影子价格施加非负约束可以在一定程度上避免退化导致的负值。

13.4.4 在同一省级样本中检查退化

对偶乘数不唯一时，不应把一次求解器返回的数值当作唯一市场价格。主案例对同一2019年样本检查产出乘数是否接近零，并固定方向向量做约束右端有限差分。归一化不可用的观测保留缺失，检验结果见表 13.6，不另造第二套行业数据。

13.5 NDDF模型的对偶问题

13.5.1 NDDF模型的原始形式

下面给出用于估计非径向方向性距离函数（Non-radial Directional Distance Function, NDDF）的线性规划问题：

$$
\begin{aligned}
\vec{D}(x,y,b;g) = \max \quad & \omega_x^{'}\beta_X + \omega_y^{'}\beta_Y + \omega_b^{'}\beta_B \\
\text{s.t.} \quad & \sum_{i=1}^{N}\lambda_i X_i \le x + \text{diag}(g_X)\beta_X \\
& \sum_{i=1}^{N}\lambda_i Y_i \ge y + \text{diag}(g_Y)\beta_Y \\
& \sum_{i=1}^{N}\lambda_i B_i = b + \text{diag}(g_B)\beta_B \\
& \lambda_i \ge 0,\ \beta \ge 0, \quad i = 1,\dots,N
\end{aligned}
\tag{13.6}
$$

其中：- $g = (g_X, g_Y, g_B)$ 为方向向量，决定效率改进的方向；- $\beta = (\beta_X, \beta_Y, \beta_B)$ 为各变量沿方向 g 的改进量；- $\omega = (\omega_X, \omega_Y, \omega_B)$ 为各变量改进量的权重。

为便于确定对偶问题，将目标函数中决策变量 λ_i 的系数补上 0，把目标函数和约束条件中的决策变量按顺序排列，并将约束条件的符号统一变为 $\le$。变换后的模型为：

$$
\begin{aligned}
\vec{D}(x,y,b;g) = \max \quad & \sum_{i=1}^{N}\lambda_i 0 + \omega_x^{'}\beta_X + \omega_y^{'}\beta_Y + \omega_b^{'}\beta_B \\
\text{s.t.} \quad & \sum_{i=1}^{N}\lambda_i X_i - \text{diag}(g_X)\beta_X + 0\beta_Y + 0\beta_B \le x \\
& -\sum_{i=1}^{N}\lambda_i Y_i + 0\beta_X + \text{diag}(g_Y)\beta_Y + 0\beta_B \le -y \\
& \sum_{i=1}^{N}\lambda_i B_i + 0\beta_X + 0\beta_Y - \text{diag}(g_B)\beta_B = b \\
& \lambda_i \ge 0,\ \beta \ge 0, \quad i = 1,\dots,N
\end{aligned}
\tag{13.7}
$$

NDDF模型的对偶形式 {#sec-13-5-2}

根据线性规划原问题与对偶问题的对应关系，可写出如下对偶问题：

$$
\begin{aligned}
\min \quad & x^{'}v_x - y^{'}u_y + b^{'}u_b \\
\text{s.t.} \quad & Xv_X - Yu_Y + Bu_B \ge 0 \\
& -\text{diag}(g_X)v_X \ge \omega_X \\
& \text{diag}(g_Y)u_Y \ge \omega_Y \\
& -\text{diag}(g_B)u_B \ge \omega_B \\
& v_x, u_y, u_b \ge 0
\end{aligned}
\tag{13.8}
$$

其中，v_x、u_y 和 u_b 分别为投入、期望产出和非期望产出的影子价格向量。对偶问题(13.8)的经济含义是：在给定方向向量 g 和权重 ω 的条件下，寻找一组影子价格，使得被评价DMU的" 虚拟利润"（投入成本减产出价值加非期望产出成本）最小化，同时满足所有参照DMU的利润非正约束。

与SBM模型的对偶问题相比，NDDF对偶问题的特点在于：1. 目标函数直接是被评价DMU的虚拟利润，无需Charnes-Cooper变换；2. 影子价格的下界由方向向量 g 和权重 ω 共同决定；3. 方向向量的选择直接影响影子价格的估计结果。

13.6 基于NDDF模型对偶问题的影子价格计算程序

13.6.1 单个DMU的影子价格计算

式(13.8)给出NDDF对偶问题后，本节以 同一省级面板（小写列名视图）为算例，展示其Pyomo实现。与SBM对偶不同，NDDF对偶直接以" 虚拟利润" 为目标，无需Charnes-Cooper归一化，故模型仅含一个对偶约束（`first`）与三类由方向向量 g 和权重 ω 决定的下界约束（`second`—`forth`）。代码先按默认方向（投入减少、期望产出增加、非期望产出减少）与等权重设定 g 与 ω，再将其映射为下界约束右端项的 `iweight`、`oweight`、`bweight`。

```
from pyomo.environ import (
    ConcreteModel, Set, Var, Objective, Constraint, Reals,
    minimize, SolverFactory, value
)
import pandas as pd
from deabook.example.book_cases.loaders import china, china_lower
import numpy as np
import warnings

warnings.filterwarnings("ignore")

# 读取数据
ex5 = china_lower(scaled=True)
data = ex5[['labor', 'capital', 'energy', 'gdp', 'co2']]

# 设定变量个数
K = 3  # 投入变量个数（labor, capital, energy）
L = 1  # 期望产出变量个数（gdp）
M = 1  # 非期望产出变量个数（co2）

# 准备参照集数据
xref = data.iloc[:, 0:K]
yref = data.iloc[:, K:K+L]
bref = data.iloc[:, K+L:K+L+M]

# 准备被评价DMU数据（以第1个DMU为例）
x = data.iloc[0, 0:K]
```

```
y = data.iloc[0, K:K+L]
b = data.iloc[0, K+L:K+L+M]

# 变量名
xcol = xref.columns
ycol = yref.columns
bcol = bref.columns

# 设定方向向量(默认:投入减少、产出增加、非期望产出减少)
gx = [-1] * len(xcol)
gy = [1] * len(ycol)
gb = [-1] * len(bcol)

# 计算权重(默认等权重)
weight = []
denominator = int(gx[0] != 0) + int(gy[0] != 0) + int(gb[0] != 0)
for _ in range(len(xcol)):
    weight.append(1.0 / denominator / len(xcol))
for _ in range(len(ycol)):
    weight.append(1.0 / denominator / len(ycol))
for _ in range(len(bcol)):
    weight.append(1.0 / denominator / len(bcol))

iweight = weight[0:len(xcol)]
oweight = weight[len(xcol):len(xcol)+len(ycol)]
bweight = weight[len(xcol)+len(ycol):len(xcol)+len(ycol)+len(bcol)]

# 构建Pyomo模型
model = ConcreteModel()
model.I = Set(initialize=xref.index)              # 参照决策单元集合
model.K = Set(initialize=range(len(xcol)))        # 投入变量集合
model.L = Set(initialize=range(len(ycol)))        # 期望产出变量集合
model.M = Set(initialize=range(len(bcol)))        # 非期望产出变量集合

# 定义影子价格变量(施加非负约束)
model.px = Var(model.K, bounds=(0, None), within=Reals, doc='投入的影子价格')
model.py = Var(model.L, bounds=(0, None), within=Reals, doc='期望产出的影子价格')
model.pb = Var(model.M, bounds=(0, None), within=Reals, doc='非期望产出的影子价格')
model.pomega = Var(bounds=(0, None), within=Reals, doc='方向距离函数值')
```

```
# 目标函数:最小化虚拟利润
def objective_rule(model):
    """返回目标函数表达式(虚拟利润) """
    return (
        sum(model.px[k] * x.iloc[k] for k in model.K)
        - sum(model.py[l] * y.iloc[l] for l in model.L)
        + sum(model.pb[j] * b.iloc[j] for j in model.M)
        + model.pomega * 1
    )

# 约束1:对偶约束(参照集加权影子价格之和非负)
def first_rule(model, i):
    """第i个参照DMU的对偶约束"""
    return (
        sum(model.px[k] * xref.loc[i, xcol[k]] for k in model.K)
        - sum(model.py[l] * yref.loc[i, ycol[l]] for l in model.L)
        + sum(model.pb[m] * bref.loc[i, bcol[m]] for m in model.M)
        + model.pomega * 1
        >= 0
    )

# 约束2:投入影子价格的下界约束
def second_rule(model, k):
    """第k个投入变量的影子价格下界"""
    return -gx[k] * x.iloc[k] * model.px[k] >= iweight[k]

# 约束3:期望产出影子价格的下界约束
def third_rule(model, l):
    """第l个期望产出变量的影子价格下界"""
    return gy[l] * y.iloc[l] * model.py[l] >= oweight[l]

# 约束4:非期望产出影子价格的下界约束
def forth_rule(model, j):
    """第j个非期望产出变量的影子价格下界"""
    return -gb[j] * b.iloc[j] * model.pb[j] >= bweight[j]

# 组装模型
model.obj = Objective(rule=objective_rule, sense=minimize, doc='目标函数')
```

```
model.first = Constraint(model.I, rule=first_rule, doc='对偶约束')
model.second = Constraint(model.K, rule=second_rule, doc='投入下界约束')
model.third = Constraint(model.L, rule=third_rule, doc='期望产出下界约束')
model.forth = Constraint(model.M, rule=forth_rule, doc='非期望产出下界约束')

# 求解
opt = SolverFactory('highs')
solution = opt.solve(model)

# 提取结果
px = np.asarray(list(model.px[:].value))
py = np.asarray(list(model.py[:].value))
pb = np.asarray(list(model.pb[:].value))
obj = value(model.obj)

print(f"投入影子价格 (px): {px}")
print(f"期望产出影子价格 (py): {py}")
print(f"非期望产出影子价格 (pb): {pb}")
print(f"最优目标值: {obj}")
```

在以上代码中，约束2-4是按照方向向量 g 和权重 ω 设定的下界约束。与SBM模型的对偶问题不同，NDDF对偶问题不需要Charnes-Cooper变换的归一化约束，因此模型结构更简洁。

13.6.2 迭代求解所有DMU

与SBM情形类似，将NDDF对偶推广至全部样本只需在循环中逐DMU重建并求解式(13.8)，并把最优解存入 `px`、`py`、`pb` 与 `obj`：

```
px, py, pb, obj = {}, {}, {}, {}

for j in range(data.shape[0]):
    x = data.iloc[j, 0:K]
    y = data.iloc[j, K:K+L]
    b = data.iloc[j, K+L:K+L+M]

    model = ConcreteModel()
    model.I = Set(initialize=xref.index)
    model.K = Set(initialize=range(len(xcol)))
    model.L = Set(initialize=range(len(ycol)))
    model.M = Set(initialize=range(len(bcol)))
```

```
model.px = Var(model.K, bounds=(0, None), within=Reals, doc=' 投入的影子价格')
model.py = Var(model.L, bounds=(0, None), within=Reals, doc=' 期望产出的影子价格')
model.pb = Var(model.M, bounds=(0, None), within=Reals, doc=' 非期望产出的影子价格')
model.pomega = Var(bounds=(0, None), within=Reals, doc=' 方向距离函数值')

def objective_rule(model):
    return (
        sum(model.px[k] * x.iloc[k] for k in model.K)
        - sum(model.py[l] * y.iloc[l] for l in model.L)
        + sum(model.pb[jj] * b.iloc[jj] for jj in model.M)
        + model.pomega * 1
    )

def first_rule(model, i):
    return (
        sum(model.px[k] * xref.loc[i, xcol[k]] for k in model.K)
        - sum(model.py[l] * yref.loc[i, ycol[l]] for l in model.L)
        + sum(model.pb[m] * bref.loc[i, bcol[m]] for m in model.M)
        + model.pomega * 1
        >= 0
    )

def second_rule(model, k):
    return -gx[k] * x.iloc[k] * model.px[k] >= iweight[k]

def third_rule(model, l):
    return gy[l] * y.iloc[l] * model.py[l] >= oweight[l]

def forth_rule(model, jj):
    return -gb[jj] * b.iloc[jj] * model.pb[jj] >= bweight[jj]

model.obj = Objective(rule=objective_rule, sense=minimize, doc=' 目标函数')
model.first = Constraint(model.I, rule=first_rule, doc=' 对偶约束')
model.second = Constraint(model.K, rule=second_rule, doc=' 投入下界约束')
model.third = Constraint(model.L, rule=third_rule, doc=' 期望产出下界约束')
model.forth = Constraint(model.M, rule=forth_rule, doc=' 非期望产出下界约束')

opt = SolverFactory('highs')
solution = opt.solve(model)
```

```
    if solution.solver.termination_condition == "optimal":
        px[j] = np.asarray(list(model.px[:].value))
        py[j] = np.asarray(list(model.py[:].value))
        pb[j] = np.asarray(list(model.pb[:].value))
        obj[j] = value(model.obj)
```

由此，NDDF对偶问题的逐DMU求解已完整给出。为便于重复使用，以下将其封装为函数。

13.6.3 面向复用的函数封装

```
def nddf_dual(dataframe, varname, num_inputs, num_outputs):
    """
    基于NDDF模型对偶问题估计影子价格。

    求解NDDF模型的对偶问题(乘数形式),获得各决策单元的投入、
    期望产出和非期望产出的影子价格。

    Parameters
    ----------
    dataframe : pd.DataFrame
        待评价决策单元的投入产出数据框,变量按投入在前、
        期望产出在中、非期望产出在后的顺序排列。
    varname : list[str]
        投入变量、期望产出变量和非期望产出变量的名称列表,
        如 ["labor", "capital", "energy", "gdp", "co2"]。
    num_inputs : int
        投入变量的个数。
    num_outputs : int
        期望产出变量的个数。

    Returns
    -------
    pd.DataFrame
        包含各DMU的影子价格估计结果的数据框。

    Notes
    -----
    - 默认方向向量:投入减少(-1)、期望产出增加(+1)、非期望产出减少(-1)。
    - 默认权重:等权重分配。
    - 如需自定义方向向量和权重,请使用 nddf_dual_advanced 函数。
```

```
    """
    data = dataframe.loc[:, varname]
    num_inputs_ = num_inputs
    num_outputs_ = num_outputs
    num_bad = data.shape[1] - num_inputs_ - num_outputs_

    xref = data.iloc[:, 0:num_inputs_]
    yref = data.iloc[:, num_inputs_:num_inputs_+num_outputs_]
    bref = data.iloc[:, num_inputs_+num_outputs_:num_inputs_+num_outputs_+
        num_bad]

    xcol = xref.columns
    ycol = yref.columns
    bcol = bref.columns

    # 默认方向向量
    gx = [-1] * len(xcol)
    gy = [1] * len(ycol)
    gb = [-1] * len(bcol)

    # 默认权重
    weight = []
    denominator = int(gx[0] != 0) + int(gy[0] != 0) + int(gb[0] != 0)
    for _ in range(len(xcol)):
        weight.append(1.0 / denominator / len(xcol))
    for _ in range(len(ycol)):
        weight.append(1.0 / denominator / len(ycol))
    for _ in range(len(bcol)):
        weight.append(1.0 / denominator / len(bcol))

    iweight = weight[0:len(xcol)]
    oweight = weight[len(xcol):len(xcol)+len(ycol)]
    bweight = weight[len(xcol)+len(ycol):len(xcol)+len(ycol)+len(bcol)]

    px, py, pb, obj = {}, {}, {}, {}

    for j in range(data.shape[0]):
        x = data.iloc[j, 0:num_inputs_]
        y = data.iloc[j, num_inputs_:num_inputs_+num_outputs_]
```

```
    b = data.iloc[j, num_inputs_+num_outputs_:num_inputs_+num_outputs_+
        num_bad]

    model = ConcreteModel()
    model.I = Set(initialize=xref.index)
    model.K = Set(initialize=range(len(xcol)))
    model.L = Set(initialize=range(len(ycol)))
    model.M = Set(initialize=range(len(bcol)))

    model.px = Var(model.K, bounds=(0, None), within=Reals, doc='投入的影子价格')
    model.py = Var(model.L, bounds=(0, None), within=Reals,
        doc='期望产出的影子价格')
    model.pb = Var(model.M, bounds=(0, None), within=Reals,
        doc='非期望产出的影子价格')
    model.pomega = Var(bounds=(0, None), within=Reals, doc='方向距离函数值')

    def objective_rule(model):
        return (
            sum(model.px[k] * x.iloc[k] for k in model.K)
            - sum(model.py[l] * y.iloc[l] for l in model.L)
            + sum(model.pb[jj] * b.iloc[jj] for jj in model.M)
            + model.pomega * 1
        )

    def first_rule(model, i):
        return (
            sum(model.px[k] * xref.loc[i, xcol[k]] for k in model.K)
            - sum(model.py[l] * yref.loc[i, ycol[l]] for l in model.L)
            + sum(model.pb[m] * bref.loc[i, bcol[m]] for m in model.M)
            + model.pomega * 1
            >= 0
        )

    def second_rule(model, k):
        return -gx[k] * x.iloc[k] * model.px[k] >= iweight[k]


    def third_rule(model, l):
        return gy[l] * y.iloc[l] * model.py[l] >= oweight[l]
```

```
        def forth_rule(model, jj):
            return -gb[jj] * b.iloc[jj] * model.pb[jj] >= bweight[jj]

        model.obj = Objective(rule=objective_rule, sense=minimize, doc='目标函数')
        model.first = Constraint(model.I, rule=first_rule, doc='对偶约束')
        model.second = Constraint(model.K, rule=second_rule, doc='投入下界约束')
        model.third = Constraint(model.L, rule=third_rule, doc='期望产出下界约束')
        model.forth = Constraint(model.M, rule=forth_rule, doc='非期望产出下界约束')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        if solution.solver.termination_condition == "optimal":
            px[j] = np.asarray(list(model.px[:].value))
            py[j] = np.asarray(list(model.py[:].value))
            pb[j] = np.asarray(list(model.pb[:].value))
            obj[j] = value(model.obj)

    objdf = pd.DataFrame(obj, index=["obj"]).T
    pxdf = pd.DataFrame(px, index=xcol).T
    pxdf.columns = pxdf.columns.map(lambda x: "shadow price " + str(x))
    pydf = pd.DataFrame(py, index=ycol).T
    pydf.columns = pydf.columns.map(lambda y: "shadow price " + str(y))
    pbdf = pd.DataFrame(pb, index=bcol).T
    pbdf.columns = pbdf.columns.map(lambda b: "shadow price " + str(b))

    pdf = pd.concat([pxdf, pydf], axis=1)
    pdf = pd.concat([pdf, pbdf], axis=1)
    redata = pd.concat([objdf, pdf], axis=1)

    return redata
```

13.6.4 高级版本：支持方向向量、权重和查询条件

`nddf_dual_advanced` 进一步将方向向量、权重及评价/参照集筛选统一进一个接口，并以公式字符串"期望产出:非期望产出=投入"传递变量结构。方向向量 g 与权重 ω 的默认值分别为投入减少、期望产出增加、非期望产出减少与等权重，用户可按研究问题覆盖；`eval_query`、`ref_query` 则用于限定评价单元与参照集。

```
def nddf_dual_advanced(formula, dataframe, gx=None, gy=None, gb=None,
                       weight=None, eval_query=None, ref_query=None):
    """
    基于NDDF对偶问题估计影子价格(高级版本)。

    支持自定义方向向量、权重,以及评价单元和参照集的筛选。

    Parameters
    ----------
    formula : str
        公式字符串,格式为 "期望产出:非期望产出=投入1 投入2 ...",
        如 "gdp : co2 = labor capital energy"。
    dataframe : pd.DataFrame
        待评价决策单元的投入产出数据框。
    gx : list or None, default None
        投入方向向量。默认为 [-1, -1, ...],表示投入减少方向。
    gy : list or None, default None
        期望产出方向向量。默认为 [1, 1, ...],表示期望产出增加方向。
    gb : list or None, default None
        非期望产出方向向量。默认为 [-1, -1, ...],表示非期望产出减少方向。
    weight : list or None, default None
        各变量的权重向量。若为None,则使用等权重分配。
    eval_query : str or None, default None
        传入数据框.query()方法的参数,用于筛选待评价DMU。
        若为None,则评价所有DMU。
    ref_query : str or None, default None
        传入数据框.query()方法的参数,用于筛选参照集DMU。
        若为None,则使用全部DMU作为参照集。

    Returns
    -------
    pd.DataFrame
        包含各DMU影子价格估计结果的数据框。
    """
    px, py, pb, obj = {}, {}, {}, {}

    # 处理查询条件
    if eval_query is None:
        index_list = dataframe.index
```

```
    else:
        index_list = dataframe.query(eval_query).index

    if ref_query is None:
        index_list_ref = dataframe.index
    else:
        index_list_ref = dataframe.query(ref_query).index

    # 解析公式字符串
    input_vars = formula.split('=')[1].strip(' ')
    xcol = re.compile(' +').sub(' ', input_vars).split(' ')
    output_vars = formula.split('=')[0].split(':')[0].strip(' ')
    ycol = re.compile(' +').sub(' ', output_vars).split(' ')
    unoutput_vars = formula.split('=')[0].split(':')[1].strip(' ')
    bcol = re.compile(' +').sub(' ', unoutput_vars).split(' ')

    data = dataframe.loc[index_list, xcol + ycol + bcol]
    dataref = dataframe.loc[index_list_ref, xcol + ycol + bcol]

    xref = dataref.loc[:, xcol]
    yref = dataref.loc[:, ycol]
    bref = dataref.loc[:, bcol]

    # 处理方向向量(使用None判断默认值)
    if gx is None:
        gx = [-1] * len(xcol)
    if gy is None:
        gy = [1] * len(ycol)
    if gb is None:
        gb = [-1] * len(bcol)

    # 处理权重(使用None判断默认值)
    if weight is None:
        weight = []
        denominator = int(gx[0] != 0) + int(gy[0] != 0) + int(gb[0] != 0)
        for _ in range(len(xcol)):
            weight.append(1.0 / denominator / len(xcol))
        for _ in range(len(ycol)):
            weight.append(1.0 / denominator / len(ycol))
```

```
        for _ in range(len(bcol)):
            weight.append(1.0 / denominator / len(bcol))

    iweight = weight[0:len(xcol)]
    oweight = weight[len(xcol):len(xcol)+len(ycol)]
    bweight = weight[len(xcol)+len(ycol):len(xcol)+len(ycol)+len(bcol)]

    for j in data.index:
        x = data.loc[j, xcol]
        y = data.loc[j, ycol]
        b = data.loc[j, bcol]

        model = ConcreteModel()
        model.I = Set(initialize=xref.index)
        model.K = Set(initialize=range(len(xcol)))
        model.L = Set(initialize=range(len(ycol)))
        model.M = Set(initialize=range(len(bcol)))

        model.px = Var(model.K, bounds=(0, None), within=Reals, doc='投入的影子价格')
        model.py = Var(model.L, bounds=(0, None), within=Reals,
            doc='期望产出的影子价格')
        model.pb = Var(model.M, bounds=(0, None), within=Reals,
            doc='非期望产出的影子价格')
        model.pomega = Var(bounds=(0, None), within=Reals, doc='方向距离函数值')

        def objective_rule(model):
            return (
                sum(model.px[k] * x.iloc[k] for k in model.K)
                - sum(model.py[l] * y.iloc[l] for l in model.L)
                + sum(model.pb[jj] * b.iloc[jj] for jj in model.M)
                + model.pomega * 1
            )

        def first_rule(model, i):
            return (
                sum(model.px[k] * xref.loc[i, xcol[k]] for k in model.K)
                - sum(model.py[l] * yref.loc[i, ycol[l]] for l in model.L)
                + sum(model.pb[m] * bref.loc[i, bcol[m]] for m in model.M)
                + model.pomega * 1
```

```
                >= 0
            )

        def second_rule(model, k):
            return -gx[k] * x.iloc[k] * model.px[k] >= iweight[k]

        def third_rule(model, l):
            return gy[l] * y.iloc[l] * model.py[l] >= oweight[l]

        def forth_rule(model, jj):
            return -gb[jj] * b.iloc[jj] * model.pb[jj] >= bweight[jj]

        model.obj = Objective(rule=objective_rule, sense=minimize, doc='目标函数')
        model.first = Constraint(model.I, rule=first_rule, doc='对偶约束')
        model.second = Constraint(model.K, rule=second_rule, doc='投入下界约束')
        model.third = Constraint(model.L, rule=third_rule, doc='期望产出下界约束')
        model.forth = Constraint(model.M, rule=forth_rule, doc='非期望产出下界约束')

        opt = SolverFactory('highs')
        solution = opt.solve(model)

        if solution.solver.termination_condition == "optimal":
            px[j] = np.asarray(list(model.px[:].value))
            py[j] = np.asarray(list(model.py[:].value))
            pb[j] = np.asarray(list(model.pb[:].value))
            obj[j] = value(model.obj)

objdf = pd.DataFrame(obj, index=["obj"]).T
pxdf = pd.DataFrame(px, index=xcol).T
pxdf.columns = pxdf.columns.map(lambda x: "shadow price " + str(x))
pydf = pd.DataFrame(py, index=ycol).T
pydf.columns = pydf.columns.map(lambda y: "shadow price " + str(y))
pbdf = pd.DataFrame(pb, index=bcol).T
pbdf.columns = pbdf.columns.map(lambda b: "shadow price " + str(b))

pdf = pd.concat([pxdf, pydf], axis=1)
pdf = pd.concat([pdf, pbdf], axis=1)
redata = pd.concat([objdf, pdf], axis=1)
```

```
    return redata
```

命名说明：`gx`、`gy`、`gb` 分别对应投入、期望产出和非期望产出的方向向量（g-direction），`iweight`、`oweight`、`bweight` 分别对应三者的权重。参数默认值使用 `None` 而非 `type(None)` 判断，这是Python的推荐做法。

该函数默认设置下即可直接调用，例如：

```
import pandas as pd
from deabook.example.book_cases.loaders import china, china_lower

ex5 = china_lower(scaled=True)
result = nddf_dual_advanced(
    'gdp : co2 = labor capital energy',
    ex5,
    gx=[-1, -1, -1],
    gy=[1],
    gb=[-1],
    weight=None,
    eval_query=None,
    ref_query=None
)
print(result.head())
```

13.7 不同模型设定下的影子价格差异

影子价格的估计结果受模型设定的显著影响。本节系统比较不同模型设定（CRS vs VRS、投入导向 vs 产出导向）对影子价格的影响，并提供模型选择的决策参考。

13.7.1 CRS与VRS的影响

规模报酬不变（CRS） 假设所有DMU都在最优规模上运营，其影子价格因而隐含了最优规模的前提。在CRS下，影子价格反映的是技术效率与规模效率的综合效应。

规模报酬可变（VRS） 放松了这一假设，允许DMU在不同规模水平上运营。在VRS下：- 规模效率非有效的DMU，其影子价格还包含了**规模扭曲**的成分；- VRS模型的影子价格通常更为分散，因为不同规模水平的DMU面临不同的边际替代率；- 对于处于规模收益递增区域的DMU，投入的影子价格倾向于偏低（因为增加投入带来的产出增加超过比例）。

实践建议：资源定价分析通常偏好VRS，除非有充分理由相信所有DMU都在最优规模上运营（如高度竞争的市场）。这是因为VRS下的影子价格更准确地反映了各DMU面临的局部技术条件。

13.7.2 投入导向与产出导向的影响

导向的作用是决定**归一化约束放在哪一侧、目标函数取哪一侧的加权和**，而不是决定哪一侧的乘数存在。径向CCR/BCC乘数形式中，ν 与 μ 一般都严格为正：

投入导向（Input-oriented）：把被评价单元的投入价值归一化，$\nu' x_0 = 1$，目标为 $\max \mu' y_0$（见 13.1.1 节的数值例子）。在这种导向下：- 投入乘数由归一化约束标定其尺度，反映各投入在" 单位投入预算" 中所占的份额；- 期望产出的影子价格进入目标函数，一般严格为正——虽然产出在包络形式中不做径向调整，但产出约束（参照集产出不得低于 y_0）依然存在，其对偶不必为零；- 非期望产出的影子价格反映减少该非期望产出的边际收益。

产出导向（Output-oriented）：把被评价单元的产出价值归一化，$\mu' y_0 = 1$，目标为 $\min \nu' x_0$。在这种导向下：- 期望产出乘数由归一化约束标定尺度；- 投入的影子价格进入目标函数，一般严格为正——投入约束（参照集投入不得超过 x_0）同样存在；- 非期望产出的影子价格反映增加期望产出时伴随的非期望产出的" 代价"。

什么时候影子价格才真正为零：零影子价格来自**互补松弛条件**，而不是导向。某一维投入或产出在最优解处存在松弛（该约束不紧）时，对应的乘数为零；这种情况在两种导向下都可能出现，也可能都不出现。把" 未选导向的那一侧乘数为零" 当成一般规律，会让读者误以为投入导向下无法报告产出的影子价格（或反之），从而在环境定价中只敢报告一侧的结果。

非期望产出影子价格的符号：在环境绩效评估中，非期望产出影子价格的符号主要由**技术设定**（坏产出计入投入侧，还是弱可处置/DDF技术）决定，而不是由导向决定，详见 13.4.1 节。

13.7.3 模型选择决策树

以下决策树为影子价格估计的模型选择提供参考：

```
影子价格估计的模型选择：
|
|-- 研究目的是估算投入要素的稀缺价值（如能源、资本的隐含成本）?
|    |-- 是 --> 投入导向
|    |-- 否 --> 继续
|
|-- 研究目的是估算产出的边际价值（如GDP的环境成本）?
|    |-- 是 --> 产出导向
|    |-- 否 --> 继续
|
|-- 样本包含多种规模的组织（如不同规模的企业、省份）?
|    |-- 是 --> VRS（规模报酬可变）
|    |-- 否 --> CRS（规模报酬不变）
|
|-- 需要区分技术效率与规模效率的影响?
|    |-- 是 --> VRS + 效率分解
|    |-- 否 --> 继续
|
```

```
|-- 价格估计用于政策模拟（如碳税、环境税的制定）?
|   |-- 是 --> 需专门的前沿统计推断；重抽样诊断见13.9节
|   |-- 否 --> 报告点估计即可
|
|-- 存在非期望产出（如CO2、SO2）?
|   |-- 是 --> 使用SBM模型或NDDF模型
|   |-- 否 --> 使用传统CCR/BCC模型
```

表 13.5: 模型设定对影子价格估计的影响

模型设定	投入影子价格	期望产出影子价格	非期望产出影子价格	适用场景
CRS投入导向	由归一化约束标定（反映技术+规模效率）	进入目标函数	减排边际成本	行业整体资源定价
VRS投入导向	由归一化约束标定（反映纯技术效率）	进入目标函数	减排边际成本	异质性组织资源定价
CRS产出导向	进入目标函数	由归一化约束标定	增产的污染代价	产出价值评估
VRS产出导向	进入目标函数	由归一化约束标定（含规模效应）	增产的污染代价	异质性组织产出评估
SBM非径向	综合反映	综合反映	综合反映	非期望产出定价
NDDF	方向依赖	方向依赖	方向依赖	政策模拟

表中”由归一化约束标定”指该侧乘数的尺度被 $\nu' x_0 = 1$ 或 $\mu' y_0 = 1$ 固定，“进入目标函数”指该侧乘数构成被优化的加权和。两侧乘数在四种设定下一般都为正；只有当对应的投入或产出约束在最优解处存在松弛时，该维乘数才由互补松弛条件取零。

13.8 案例应用：中国省级排放约束的边际价值

使用第12章同一省级数据的2019年截面，变量与样本均保持一致。以CRS碳排放方向DDF为主程序，读取产出约束与排放约束的对偶乘数。各列按样本均值缩放；表中的乘数比是标准化产出与排放之间的边际替代率，不是元/吨碳价。只有明确产出货币基年、排放物理单位和产出价格归一化，才能进一步换算货币影子价格。

```
from deabook.example.book_cases.run import FUN
FUN[13]()  # 重新计算本章，输出完整CSV、摘要表与PNG/PDF图
```

表 13.6: 中国省级排放约束的边际价值：结果摘要

统计量	效率	产出约束乘数	排放约束乘数	标准化边际替代率
有效样本数	30	30	30	30
均值	0.227	0.339	1.742	5.138
标准差	0.179	0.352	1.807	0.000
最小值	0.037	0.075	0.387	5.138
P10	0.085	0.089	0.456	5.138
中位数	0.193	0.246	1.264	5.138
P90	0.377	0.487	2.501	5.138
最大值	1.000	1.639	8.422	5.138

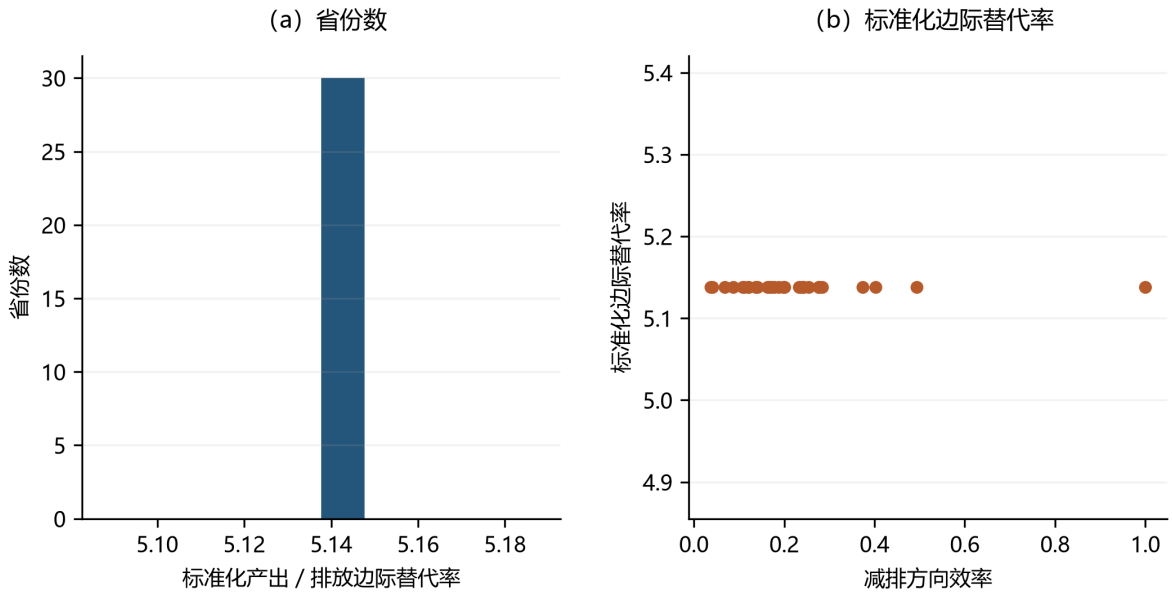


图 13.1: 中国省级样本的排放约束边际价值与效率（2019年）

表分别报告减排方向效率、产出与排放约束乘数及标准化边际替代率的分布。乘数比高度集中时，不应制作省份碳价排名；图如实保留这种集中性。乘数比仍是标准化变量间的边际替代率，不是元/吨价格。有限差分核验误差保存在完整结果中。

当前CRS单一坏产出设定下，可归一化的边际替代率几乎相同，图如实显示其集中程度，不据此制作省份“碳价排名”。对偶检验固定方向向量，只微调排放约束右端，比较目标值差商与对偶乘数；若同时改变方向向量，差商不再是同一个约束的影子价格。DEA给出生产技术的边际机会成本，不能直接等同于污染的边际社会损害或最优庇古税。

配套程序 `deabook/example/book_cases/run.py` 保存全部观测到 `results/ch13_full.csv`。表中P10和P90为第10和第90百分位数（如适用），展示分布而不占用逐个决策单元的行数。

13.9 影子价格的重抽样诊断与统计推断

13.9.1 为什么需要置信区间？

影子价格作为从样本数据估计得到的统计量，具有**样本不确定性**。仅报告点估计而不提供置信区间，会误导读者将估计值视为精确值。在环境政策制定中，了解影子价格的置信区间至关重要：

- 如果某污染物影子价格的95%置信区间包含零，则无法确认该污染物的边际成本显著为正；
- 如果两个地区的影子价格置信区间大幅重叠，则不能得出两地区边际成本显著不同的结论。

13.9.2 重抽样敏感性诊断及其边界

Simar-Wilson 的平滑 Bootstrap 需要处理前沿估计的边界偏差。简单有放回抽取数据行，不能直接称为该方法；重抽样后行号也不再对应原来的省份，不能按行位置计算各省置信区间。

13.9.3 Python实现：样本中位数的重抽样诊断

下面程序只考察影子价格样本中位数对重抽样的敏感性，输出重复估计的分位数，不将其解释为省级置信区间。每次重新估计整个重抽样样本；失败次数和有效次数单独报告。

```
import numpy as np
import pandas as pd

def resample_shadow_summary(dataframe, varname, num_inputs, num_outputs,
                            model_func, b=100, alpha=0.05, seed=42):
    if b < 2 or not 0 < alpha < 1 or dataframe.empty:
        raise ValueError("需至少两次重复、非空数据及合法alpha")
    rng = np.random.default_rng(seed)
    original = model_func(dataframe, varname, num_inputs, num_outputs)
    cols = [c for c in original if c.startswith('shadow price ')]
    estimates = []
    failures = 0
    for _ in range(b):
        sample = dataframe.iloc[rng.integers(len(dataframe), size=len(dataframe))]
        try:
            result = model_func(sample.reset_index(drop=True), varname,
                                num_inputs, num_outputs)
            if not np.isfinite(result[cols].to_numpy()).all():
                raise RuntimeError("重抽样结果含非有限值")
            estimates.append(result[cols].median())
        except RuntimeError:
            failures += 1
    if len(estimates) < 2:
```

```
        raise RuntimeError("有效重复不足,无法计算分位数")
    draws = pd.DataFrame(estimates)
    return pd.DataFrame({
        'sample_median': original[cols].median(),
        'resample_q_low': draws.quantile(alpha / 2),
        'resample_q_high': draws.quantile(1 - alpha / 2),
        'successful': len(estimates), 'failed': failures
    })

# 小规模教学运行;正式敏感性分析可提高重复次数。
resampling_result = resample_shadow_summary(
    china(scaled=True).query('t==4'), ['K', 'L', 'Y', 'CO2'],
    2, 1, sbm_dual, b=5)
print(resampling_result)
```

13.9.4 Bootstrap在DEA中的特殊性

与传统统计方法的Bootstrap不同，DEA的Bootstrap需要特别注意以下几点：

1. **边界估计的偏差**：DEA是确定性前沿方法，效率估计量存在向上的固有偏差（因为样本前沿位于真实前沿之下）。Bootstrap需要修正这一偏差，Simar-Wilson方法通过反射法（reflection method）或平滑法（smoothed bootstrap）处理边界问题。
2. **前沿面支撑集特性**：Bootstrap样本的效率分布可能与原始样本不同，需要确保Bootstrap样本的前沿面与原始前沿面具有相同的支撑集特性。
3. **收敛性**：Bootstrap DEA的收敛速度较慢（$O(n^{-2/(p+q+1)})$，其中 p 和 q 分别为投入和产出的维度），这意味着在小样本或高维情况下，Bootstrap置信区间可能较宽。
4. **推荐实现**：Python生态中针对Bootstrap DEA的成熟专用包较少：`pyStoNED` 包提供了随机非参数方法（StoNED）的实现，`pyfrontier` 包提供了径向与SBM类DEA估计；偏差修正的Bootstrap DEA可参考R语言的 `rDEA` 包，或基于 `pyomo/deabook` 的框架自行实现。

13.10 数值稳定性与求解器选择

13.10.1 约束缩放

如 13.3.1 节所述，DEA对偶问题的数值稳定性受数据量纲影响很大：投入产出数据的量纲差异显著时，约束矩阵的条件数变大，数值误差随之累积。因此，建议在建模前统一各列量纲。这里只能**按列缩放**（每列除以一个正常数），不能中心化——理由见 13.3.1 节的说明。若变量本身有现成的自然单位，直接换单位最直观（如 13.3.1 节按列均值缩放）；下面的函数用于没有合适单位可换时，按统一规则取缩放常数——注意 `method="std"` 只是除以标准差、不减均值，与z-score标准化不是一回事。具体实现如下：

```
import numpy as np


def scale_for_shadow_price(data, input_cols, output_cols, bad_cols,
                           method="mean"):
    """
    按列缩放投入产出数据,以提高数值稳定性。

    只做缩放、不做中心化: SBM效率对列缩放严格不变,对平移则不然,
    因此不能使用z-score标准化(会把投入产出变成非正数)。

    Parameters
    ----------
    data : pd.DataFrame
        原始数据。
    input_cols, output_cols, bad_cols : list[str]
        投入、期望产出、非期望产出的列名。
    method : {"mean", "std", "max"}, default "mean"
        缩放基准:列均值、列标准差或列最大值,三者均为正常数。

    Returns
    -------
    scaled_data : pd.DataFrame
        缩放后的数据,各列量纲统一到1附近。
    scaler_params : dict
        缩放参数,键 'scale' 为各列除数(用于将影子价格反变换回
        原始尺度: price_original = price_scaled / scale)。
    """
    all_cols = input_cols + output_cols + bad_cols

    if method == "mean":
        scale = data[all_cols].mean()
    elif method == "std":
        scale = data[all_cols].std(ddof=0)
    elif method == "max":
        scale = data[all_cols].max()
    else:
        raise ValueError("method 只能取 'mean'、'std' 或 'max'")
```

```
    assert (scale > 0).all(), "缩放基准必须为正,请检查数据是否含非正列"

    scaled_data = data.copy()
    scaled_data[all_cols] = data[all_cols] / scale

    scaler_params = {
        'scale': scale,
        'input_cols': input_cols,
        'output_cols': output_cols,
        'bad_cols': bad_cols
    }

    return scaled_data, scaler_params
```

求解完成后,把各乘数除以对应列的缩放常数,再按已核实的产出价格归一化。结果是否稳定还需检查约束残差与对偶退化,不能只比较效率得分。

13.10.2 求解器选择与容差设置

表 13.7: 求解器选择

求解器	安装与用途
HiGHS	`pip install highspy`;本章线性规划默认求解器
MOSEK / CPLEX	需单独安装并配置相应许可证
IPOPT / SCIP	可用于非线性或整数扩展,须另行配置

求解器容差设置建议:

```
opt = SolverFactory('highs')

# 设置严格容差
opt.options['primal_feasibility_tolerance'] = 1e-9   # 原始可行性容差
opt.options['dual_feasibility_tolerance'] = 1e-9   # 对偶可行性容差
opt.options['ipm_optimality_tolerance'] = 1e-9  # 内点法最优性容差

solution = opt.solve(model)
```

13.10.3 退化检测与处理

检测DEA对偶问题是否退化的简单方法:

```
def check_degeneracy(model, tol=1e-8):
    """
    检查DEA对偶问题是否存在退化。

    通过检查对偶约束的松弛变量是否接近零来判断退化。
    如果多个约束同时紧致,可能存在退化。

    Parameters
    ----------
    model : pyomo.environ.ConcreteModel
        已求解的Pyomo模型。
    tol : float, default 1e-8
        判断约束是否紧致的容差。

    Returns
    -------
    bool
        是否存在退化。
    """
    tight_count = 0
    for constr in model.first.values():
        slack = constr.uslack()  # 上界松弛
        if abs(slack) < tol:
            tight_count += 1

    # 如果超过维度数的约束同时紧致,可能存在退化
    is_degenerate = tight_count > len(model.K)
    return is_degenerate
```

13.11 进一步阅读与前沿方法

本章介绍了基于SBM和NDDF对偶问题的影子价格估计方法。以下列出影子价格估计领域的若干前沿方向,供读者深入学习:

13.11.1 超效率DEA的影子价格

传统DEA模型的一个局限是:**位于生产前沿上的DMU(效率值为1)的影子价格可能为零或不唯一**,因为这些DMU是参照集的组成部分,而非被评价对象。超效率(Super-efficiency)DEA模型通过将被评价DMU从参照集中排除,解决了这一问题,使得所有DMU(包括原本效率为1的DMU)都可以获得唯一的影子价格估计。

Andersen and Petersen（1993）提出的超效率模型是应用最广泛的方法。其思想是：在评价DMU k 时，将DMU k 从参照集中移除，从而使原本效率为1的DMU也能获得大于1的效率值和对应的影子价格。

13.11.2 网络DEA的影子价格

在多阶段生产系统中，**中间产品（intermediate products）**的影子价格具有特殊的经济含义。网络DEA（Network DEA）将生产过程分解为多个相互连接的子过程，每个子过程都有自己的投入、产出和中间产品。通过求解网络DEA的对偶问题，可以获得中间产品的影子价格，揭示生产系统内部的价值转移机制。

Tone and Tsutsui（2009）提出的松弛基础网络DEA（Network SBM）是应用最广泛的网络DEA模型之一。

13.11.3 DEA与SFA影子价格的对比

随机前沿分析（Stochastic Frontier Analysis, SFA）与DEA是估计效率的两种主流方法，它们在影子价格估计上也各有优劣：

特征	DEA影子价格	SFA影子价格
函数形式	非参数（数据驱动）	参数（需预设函数形式）
统计推断	无（需Bootstrap辅助）	有（渐近理论）
噪声处理	无（所有偏离视为无效率）	有（区分噪声与无效率）
影子价格唯一性	可能不唯一（退化问题）	通常唯一
样本要求	适用于小样本	需要较大样本
灵活性	高（不受函数形式限制）	低（受函数形式约束）

13.11.4 StoNED的影子价格

凸非参数最小二乘法（Stochastic Non-smooth Envelopment of Data, StoNED）是Kuosmanen and Kortelainen（2012）提出的方法，它结合了DEA的非参数灵活性与SFA的统计推断能力。StoNED通过凸非参数最小二乘估计生产前沿，并允许存在随机噪声，因此其估计的影子价格既具有非参数方法的灵活性，又具有统计置信区间。

关于StoNED的详细方法，请参阅本书第 10 章。

13.12 本章小结

当前CRS单一坏产出设定下，可归一化的边际替代率几乎相同，图如实显示其集中程度，不据此制作省份“碳价排名”。对偶检验固定方向向量，只微调排放约束右端，比较目标值差商与对偶乘数；若同时改变方向向量，差商不再是同一个约束的影子价格。DEA给出生产技术的边际机会成本，不能直接等同于污染的边际社会损害或最优庇古税。

参考文献

Andersen, P., & Petersen, N. C. (1993). A procedure for ranking efficient units in data envelopment analysis. *Management Science*, 39(10), 1261-1264.

Charnes, A., Cooper, W. W., & Rhodes, E. (1978). Measuring the efficiency of decision making units. *European Journal of Operational Research*, 2(6), 429-444.

Du, L., Hanley, A., & Wei, C. (2015). Estimating the marginal abatement cost curve of CO_2 emissions in China: Provincial panel data analysis. *Energy Economics*, 48, 217-229.

Kuosmanen, T., & Kortelainen, M. (2012). Stochastic non-smooth envelopment of data: Semi-parametric frontier estimation subject to shape constraints. *Journal of Productivity Analysis*, 38(1), 11-28.

Pigou, A. C. (1920). *The Economics of Welfare*. Macmillan.

Revesz, R. L., Howard, P. H., Arrow, K., Goulder, L. H., Kopp, R. E., Livermore, M. A., … & Sterner, T. (2014). Global warming: Improve economic models of climate change. *Nature*, 508(7495), 173-175.

Simar, L., & Wilson, P. W. (1998). Sensitivity analysis of efficiency scores: How to bootstrap in nonparametric frontier models. *Management Science*, 44(1), 49-61.

Simar, L., & Wilson, P. W. (2000). A general methodology for bootstrapping in non-parametric frontier models. *Journal of Applied Statistics*, 27(6), 779-802.

Tone, K. (2003). Dealing with undesirable outputs in DEA: A slacks-based measure (SBM) approach (GRIPS Discussion Paper No. I-2003-0005). GRIPS Policy Research Center. https://doi.org/10.24545/00000955

Tone, K., & Tsutsui, M. (2009). Network DEA: A slacks-based measure approach. *European Journal of Operational Research*, 197(1), 243-252.

Wei, C., Ni, J., & Du, L. (2012). Regional allocation of carbon dioxide abatement in China. *China Economic Review*, 23(3), 552-565.

附录 A

附录. Python编写的DEA程序的调试

第11—13章系统给出了本书各类DEA模型的Python实现，并借助 deabook 包逐项复现了正文中的数值算例。为使读者能够独立、可靠地重现已完成的全部计算，本附录提供一套与之配套的程序调试与错误排查方法。在将书中代码付诸实践的进程中，环境配置、模型构建、求解与结果提取的任一环节都可能引入错误；随着模型复杂度的提升，出错的概率与定位难度都会显著上升。因此，掌握系统化的调试手段，是将其转化为可复现研究结论的必要前提。

本附录从DEA编程实践出发，依次涵盖开发环境配置、常见错误排查、性能优化、代码质量保障等内容，旨在为读者提供一份可随时查阅的DEA编程排错参考。

速查索引

问题场景	参考章节
Python环境安装与配置	A.1
求解器安装与验证	A.1.3
Python基础语法错误（SyntaxError、IndentationError等）	A.2
包导入错误（ModuleNotFoundError）	A.2
Pyomo模型构建失败	A.3
约束条件返回None/约束不符合预期	A.3.3
求解器报错或求解失败	A.3.2
效率值超出合理范围（>1或<0）	A.3.1
Jupyter Notebook内核/显示问题	A.4
程序运行太慢（大规模DEA问题）	A.5
输入数据验证	A.6.1
环境配置导出与复现	A.7
获取社区帮助	A.8

A.1 A.1 开发环境配置指南

A.1.1 A.1.1 Python环境管理

推荐使用 **Miniconda** 或 **Anaconda** 管理Python环境。Conda可以方便地创建隔离的虚拟环境，避免包版本冲突，特别适合科学计算和数据处理工作流。

安装Miniconda：

- **Windows**：下载安装程序（`.exe`）并运行，安装时勾选"Add to PATH"
- **macOS**：下载`.pkg`安装程序，或使用命令行：

```
curl -O https://repo.anaconda.com/miniconda/Miniconda3-latest-MacOSX-x86_64.sh
bash Miniconda3-latest-MacOSX-x86_64.sh
```

- **Linux**：

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
```

创建DEA专用环境：

```
# 创建名为dea的Python 3.11环境
conda create -n dea python=3.11

# 激活环境（每次使用前必须执行）
conda activate dea
```

跨平台注意事项：Windows用户使用 `conda activate dea`，部分旧版本conda使用 `activate dea`（无需 `conda`前缀）。macOS/Linux用户如果在激活时遇到问题，可尝试先运行 `source ~/miniconda3/etc/profile.d/conda.sh`。

A.1.2 A.1.2 核心包安装

激活环境后，按以下顺序安装所需包：

```
# 基础科学计算包
conda install numpy pandas scipy matplotlib

# Pyomo和基础求解器（conda-forge渠道）
conda install -c conda-forge pyomo glpk

# Jupyter Notebook
conda install jupyter

# 注册Notebook内核（可选，便于在Jupyter中切换环境）
python -m ipykernel install --user --name=dea --display-name "Python (dea)"
```

如果使用 pip安装：

```
pip install pyomo pandas numpy scipy matplotlib jupyter
```

各包的作用说明：

包名	用途	本书关联
pyomo	Python优化建模语言	所有DEA模型的核心建模工具
pandas	数据处理与分析	数据框操作（第11章）
numpy	数值计算	数组操作和线性代数
scipy	科学计算	辅助数值运算
matplotlib	数据可视化	效率分布图绘制
glpk	线性规划求解器	基础DEA求解

A.1.3 A.1.3 求解器安装详细指南

Pyomo作为建模语言本身不包含求解器，需要额外安装。以下是本书涉及求解器的安装方法：

（一）GLPK（GNU Linear Programming Kit）

特点：开源免费，安装最简单，适合初学者和小规模问题。

```
# 通过conda-forge安装（推荐）
conda install -c conda-forge glpk

# 验证安装
python -c "from pyomo.environ import SolverFactory; print(SolverFactory('glpk').available())"
```

（二）CBC（Coin-or Branch and Cut）

特点：开源免费，比GLPK更稳定，求解速度更快，推荐作为免费求解器的首选。

```
conda install -c conda-forge coincbc
```

（三）IPOPT（Interior Point OPTimizer）

特点：开源免费，擅长处理非线性规划问题，适用于非线性DEA模型（如二次约束）。

```
conda install -c conda-forge ipopt
```

（四）MOSEK（商业求解器）

特点：求解速度最快，稳定性最高。提供免费学术许可证，是本书**推荐**的求解器。

安装步骤：

1. 访问 MOSEK学术许可申请页面申请免费许可证
2. 下载并安装MOSEK：

```
pip install mosek
```

3. 将获得的许可证文件（`mosek.lic`）放置在`~/mosek/`目录下

求解器性能对比：下表展示了各求解器求解单个DMU效率评价问题所需的时间。`mosek`求解器消耗的时间约为`glpk`的1/7.5、`ipopt`的1/4，因此**强烈推荐使用mosek**。

求解器	时间
glpk	约 0.15s
ipopt	约 0.08s
mosek	约 0.02s

（五）Gurobi（商业求解器）

特点：学术界广泛使用，性能优异。同样提供免费学术许可。

```
conda install -c gurobi gurobi
# 或使用 pip
pip install gurobipy
```

获取学术许可证需在Gurobi官网注册并验证学术身份。

（六）AMPL/Knitro（商业求解器，经 amplpy 调用）

特点：AMPL（A Mathematical Programming Language）是专为数学规划问题设计的高级代数建模语言，旨在通过抽象的代数表达式简化复杂优化模型的构建。AMPL本身不直接求解问题，而是将模型转换为标准格式（如`.nl`文件）后调用外部求解器，支持的求解器包括Knitro、Gurobi、MINOS、IPOPT等，涵盖线性、非线性、整数规划等多种问题类型。本书第10 章的 CNLS/StoNED 类非线性模型可使用其分发的 Knitro 求解器求解——免费的 ipopt 虽能求解非线性规划，但速度较慢。AMPL 提供30 天学术许可证，过期后可使用学生身份申请延长。

安装步骤：

1. 注册并登录：进入 AMPL 门户，使用教育邮箱注册并登录。

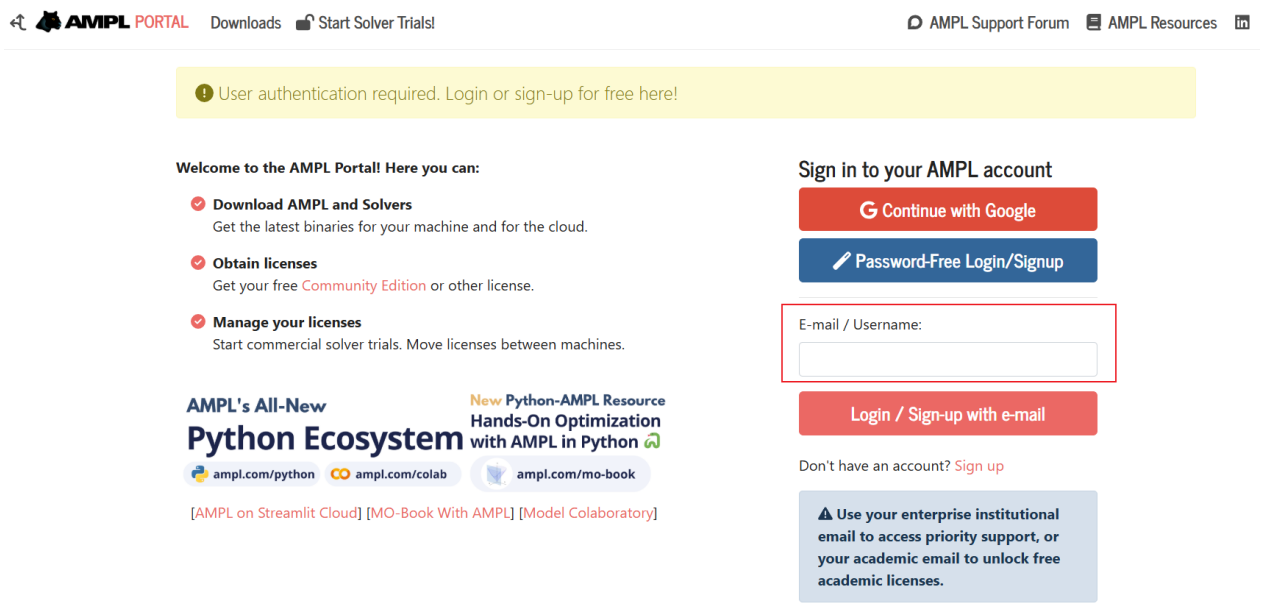


图 A.1: AMPL 注册页面

2. 登录后进入许可证列表页，可查看AMPL 在 Python 环境中的安装步骤与许可证 UUID。

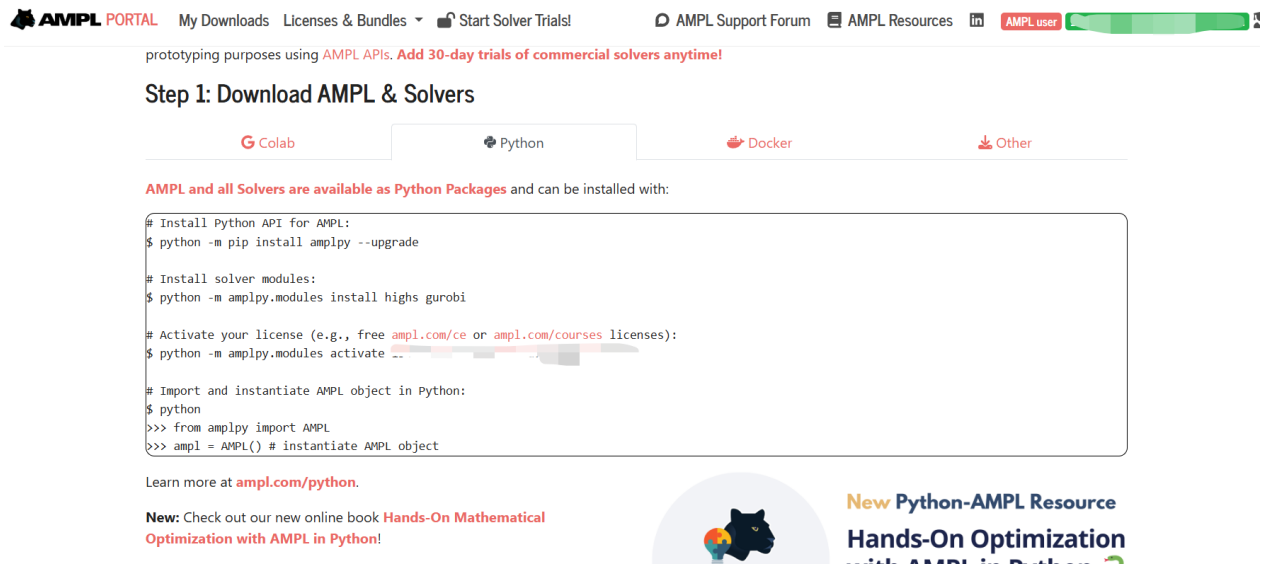


图 A.2: AMPL 许可证与安装步骤页面

3. 在 anaconda 自带的 `Anaconda Powershell Prompt` 中依次执行以下命令（即官网给出的安装步骤）：

```
# 安装 AMPL 的 Python API
python -m pip install amplpy --upgrade
# 安装求解器模块（含非线性求解器 knitro）
python -m amplpy.modules install highs gurobi knitro
```

```
# 激活许可证(UUID 见许可证列表页)
python -m amplpy.modules activate <your-license-uuid>
```

安装完成后，求解器可执行文件位于 Python 环境的 `ampl_module_base` 目录下；在 pyomo 中以 `SolverFactory('knitro')` 调用时，需确保该目录已加入系统 PATH，并将 `AMPL_LICFILE` 指向许可证文件 `ampl.lic`。

（七）SCIP（开源求解器）

特点：开源免费，擅长混合整数规划问题(本书第 11 章求解器对比表中提及)。

```
conda install -c conda-forge scip
# 配合 Pyomo 使用需安装 PySCIPOpt 接口
pip install pyscipopt
```

安装后可通过 `SolverFactory('scip').available()` 验证是否可用。

A.1.4 A.1.4 验证安装

安装完成后，运行以下代码验证所有组件是否正常工作：

```
"""
DEA开发环境验证脚本
运行此脚本可检查Python环境、Pyomo和求解器是否正确安装。
"""
import sys

print("=" * 50)
print("DEA开发环境验证")
print("=" * 50)

# 1. Python版本
print(f"\nPython版本: {sys.version}")
assert sys.version_info >= (3, 9), "Python版本需>=3.9"

# 2. 核心包导入测试
try:
    import numpy as np
    print(f"numpy: {np.__version__}  导入成功")
except ImportError as e:
    print(f"numpy: 导入失败 - {e}")

try:
    import pandas as pd
```

```
    print(f"pandas: {pd.__version__}  导入成功")
except ImportError as e:
    print(f"pandas: 导入失败 - {e}")

try:
    from pyomo.environ import *
    print("pyomo: 导入成功")
except ImportError as e:
    print(f"pyomo: 导入失败 - {e}")

# 3. 求解器可用性测试
print("\n求解器可用性检查:")
from pyomo.environ import SolverFactory

for solver_name in ['glpk', 'cbc', 'ipopt', 'mosek']:
    solver = SolverFactory(solver_name)
    status = '可用' if solver.available() else '不可用'
    print(f"  {solver_name}: {status}")

# 4. 求解一个简单的LP测试
print("\n简单LP求解测试:")
model = ConcreteModel()
model.x = Var(within=NonNegativeReals)
model.y = Var(within=NonNegativeReals)
model.obj = Objective(expr=model.x + 2 * model.y, sense=minimize)
model.c1 = Constraint(expr=model.x + model.y >= 1)

for solver_name in ['mosek', 'cbc', 'glpk']:
    solver = SolverFactory(solver_name)
    if solver.available():
        result = solver.solve(model)
        print(f"  {solver_name}: 目标值 = {value(model.obj):.4f}")
        break

print("\n" + "=" * 50)
print("环境验证完成！")
print("=" * 50)
```

正常运行后，应输出各包版本号和求解器可用状态，以及简单线性规划的求解结果（目标值约为1.0）。

A.1.5 A.1.5 跨平台注意事项

本书读者可能使用Windows、macOS或Linux等不同操作系统，以下为跨平台开发的关键注意事项：

文件路径处理：

不同操作系统使用不同的路径分隔符（Windows使用\，macOS/Linux使用/）。推荐使用Python的`pathlib`模块处理文件路径，它会自动适配当前操作系统：

```
from pathlib import Path

# 自动使用正确的路径分隔符
from deabook import data_path
data_file = Path(data_path("us_electric_plants.csv"))
# data_path定位安装包中的案例数据，Path适配各操作系统。

# 兼容三种操作系统的写法
data_file = Path.home() / 'Documents' / 'dea_data' / 'sample.csv'
```

终端命令差异：

操作	Windows (CMD)	Windows (PowerShell)	macOS/Linux
列出目录	`dir`	`ls`	`ls`
切换目录	`cd`	`cd`	`cd`
激活conda	`activate dea`	`conda activate dea`	`conda activate dea`
安装包	`conda install`	`conda install`	`conda install`
路径分隔符	`\`	`\`或`/`	`/`

Jupyter Notebook路径问题：

在Jupyter Notebook中，!命令在当前环境的shell中执行。Windows用户若使用默认的CMD而非PowerShell，部分命令可能不兼容。建议在Jupyter中使用Python的`pathlib`或`os`模块处理文件路径，而非直接调用系统命令。

A.2 A.2 Python基础错误速查

在环境就绪之后，调试工作通常始于对错误类型的识别。本节以速查表形式汇总DEA编程中最为常见的通用Python错误。这些错误本身与DEA建模无关，属Python语言基础问题；如需更详尽的解释，请参阅第2章或Python官方教程（docs.python.org/zh-cn/3/tutorial/errors.html）。

A.2.1 A.2.1 常见错误速查表

错误类型	典型错误信息	常见原因	解决方案
SyntaxError	invalid syntax	冒号遗漏、括号不匹配、中文标点	检查行末冒号(:)、括号匹配,确保使用英文标点
IndentationError	unexpected indent	缩进不一致(混用空格和Tab)	统一使用4个空格缩进,编辑器设置"将Tab转为空格"
NameError	name 'xxx' is not defined	变量名拼写错误、变量未定义就使用、作用域错误	检查拼写;确认变量在使用前已赋值;注意函数内/外作用域
TypeError	unsupported operand type	类型不匹配(如字符串+数字)、函数参数类型错误	使用 type()检查变量类型;必要时进行类型转换
IndexError	list index out of range	索引超出序列范围	检查索引值和序列长度;使用 len()确认
KeyError	'key' not found	字典中不存在该键	使用.get()方法或先检查键是否存在
ModuleNotFoundError	No module named 'xxx'	包未安装、环境未激活	激活正确环境;使用 conda install或 pip install安装
ImportError	cannot import name 'xxx'	包版本不兼容、循环导入	检查包版本;检查导入路径
FileNotFoundError	No such file or directory	文件路径错误、工作目录不对	使用绝对路径;检查 os.getcwd()确认当前工作目录
ZeroDivisionError	division by zero	除数为0	添加除零保护(如 if denominator != 0)

A.2.2 A.2.2 包导入错误的DEA场景

在DEA编程中，ModuleNotFoundError最常发生在以下场景：

```
# 错误示例:环境未激活导致pyomo找不到
from pyomo.environ import *        # ModuleNotFoundError: No module named 'pyomo'

# 解决方法:
# 1. 确认已激活正确的conda环境: conda activate dea
# 2. 如果仍未解决,重新安装: conda install -c conda-forge pyomo
```

```
# 错误示例:拼写错误
from pyomo.environ import Solverfactory
   # ImportError: cannot import name 'Solverfactory' from 'pyomo.environ'
# 正确应为 SolverFactory（注意F大写）

from pyomo.environ import ConcreteModel, Var, Objective, Constraint, maximize
```

A.2.3 A.2.3 使用pathlib处理文件路径

数据读取时的路径问题是跨平台开发中最常见的错误来源。以下代码展示了DEA场景下的推荐做法：

```
from pathlib import Path
import pandas as pd

# 推荐做法:使用pathlib自动适配操作系统
data_dir = Path('data')
ex3_path = data_dir / 'Ex3.dta'

# 检查文件是否存在
if not ex3_path.exists():
    raise FileNotFoundError(f"数据文件未找到: {ex3_path.absolute()}")

ex3 = pd.read_stata(ex3_path)
```

A.3 A.3 Pyomo与DEA特有错误排查

通用Python错误之外，DEA程序最常见的故障源于Pyomo建模与DEA模型本身的特殊性。本节对这类特有错误作深入分析。从排查路径来看，它们通常依次出现在数据准备、模型构建、求解与结果提取四个阶段，因而需要结合DEA模型的专业知识逐一甄别。

A.3.1 A.3.1 DEA程序调试决策树

当DEA程序运行出错时，可按以下决策树系统性地定位问题：

```
程序运行出错？
|
|-- 错误发生在导入阶段？
|   |-- ModuleNotFoundError --> 检查包安装（conda list / pip list）
|   |-- ImportError --> 检查Python路径和包版本兼容性
|
```

```
|-- 错误发生在数据读取阶段？
|   |-- FileNotFoundError --> 检查文件路径（使用pathlib.Path）
|   |-- UnicodeDecodeError --> 指定encoding='utf-8'或'gbk'
|   |-- KeyError --> 检查列名是否与代码中一致（注意空格、大小写）
|
|-- 错误发生在模型构建阶段？
|   |-- ValueError: Invalid constraint --> 检查变量维度是否匹配
|   |-- TypeError: Unsupported operand --> 检查Pyomo变量与Python原生类型混用
|   |-- NameError --> 检查约束规则函数中的变量名拼写
|   |-- 公式解析错误 --> 检查公式字符串格式
|
|-- 错误发生在求解阶段？
|   |-- SolverNotAvailable --> 安装/配置求解器（见A.1.3节）
|   |-- TerminationCondition.infeasible --> 检查数据是否有零/负值、方向是否正确
|   |-- TerminationCondition.unbounded --> 检查是否遗漏了关键约束
|   |-- 求解器崩溃/内存错误 --> 减少DMU数量或换用更高效的求解器
|
|-- 错误发生在结果提取阶段？
    |-- KeyError/AttributeError --> 检查Pyomo组件访问语法
    |-- 结果为空或全NaN --> 检查求解是否成功
    |-- 结果不合理（如效率>1或<0）--> 检查模型导向和RTS设定
```

A.3.2 A.3.2 最小可复现示例（MRE）指南

前述决策树提供的是问题定位的思路；而要将故障清晰地呈现给他人（或提交至社区），还需将其凝练为可独立运行的最小片段。在向他人求助或自行调试时，构造**最小可复现示例（Minimum Reproducible Example, MRE）**是最有效的策略。

MRE的构造原则：

1. **最小数据：**用最少的DMU复现错误（通常2-3个DMU、1投入1产出即可）
2. **最小代码：**只保留能触发错误的最少代码行
3. **可独立运行：**他人复制粘贴即可运行

DEA场景的MRE模板：

```
"""
最小可复现示例：DEA模型XX错误的复现
环境：Python 3.11, pyomo 6.6, glpk求解器
"""
from pyomo.environ import *
import pandas as pd

# 最小数据集（2个DMU，1投入，1产出）
```

```
data = pd.DataFrame({
    'dmu': [1, 2],
    'x': [1.0, 2.0],   # 投入
    'y': [1.0, 1.0]    # 产出
})

# 最小DEA模型
def minimal_dea(data, input_col, output_col, dmu0):
    """最小DEA模型(产出导向CRS),一次评价一个DMU,用于调试

    Parameters
    ----------
    data : pd.DataFrame
        含投入列与产出列的数据框。
    input_col, output_col : str
        投入、产出列名(本示例为单投入单产出)。
    dmu0 : int
        被评价DMU在 data 中的位置(0, 1, ...)。

    Returns
    -------
    float
        产出扩张倍数 theta (>=1),技术效率为 1/theta。
    """
    model = ConcreteModel()
    model.I = Set(initialize=range(len(data)))   # 参照集:全部DMU
    model.x = data[input_col].values
    model.y = data[output_col].values

    # 被评价DMU的观测值(标量)
    x0 = model.x[dmu0]
    y0 = model.y[dmu0]

    model.theta = Var(bounds=(0, None))          # 产出扩张倍数,非负
    model.intensity = Var(model.I, bounds=(0, None))

    model.obj = Objective(expr=model.theta, sense=maximize)

    # 投入约束:参照集加权投入不超过被评价DMU的投入(单投入, 1条约束)
```

```
    model.input_con = Constraint(
        expr=sum(model.intensity[i] * model.x[i] for i in model.I) <= x0
    )

    # 产出约束:参照集加权产出不低于被评价DMU产出的theta倍(单产出, 1条约束)
    model.output_con = Constraint(
        expr=sum(model.intensity[i] * model.y[i] for i in model.I)
        >= model.theta * y0
    )

    solver = SolverFactory('glpk')
    result = solver.solve(model)
    return value(model.theta)

# 运行测试:对每个DMU分别求解
try:
    for dmu0 in range(len(data)):
        theta = minimal_dea(data, 'x', 'y', dmu0)
        print(
            f"DMU {data.loc[dmu0, 'dmu']}: theta = {theta:.4f}, te = {1/theta:.4f}")
except Exception as e:
    print(f"错误: {type(e).__name__}: {e}")
```

这个模板的两个要点容易写错。第一，**DEA是"一个DMU一个线性规划"**：约束右端是被评价单元 0 的观测值 x_0、y_0，约束条数等于投入维数加产出维数(这里各为1)，而不是等于DMU个数。若把约束写成对所有 i 成立的 $\sum_j \lambda_j x_j \le x_i$、$\sum_j \lambda_j y_j \ge \theta y_i$，就变成要求同一组 λ 和同一个 θ 同时包络全部单元，返回的单个 `theta` 不是任何一个DMU的效率。评价多个单元要在函数外循环(如上)，或在函数内对 `data.index` 循环。第二，θ **需要下界**：产出导向下 $\theta \ge 0$（最优时 $\theta \ge 1$），写成 `bounds=(None, None)` 在某些约束组合下会让LP无界。

如果遇到"infeasible" 错误，先用上述2个DMU、1投入1产出的最小数据集测试。如果最小数据集可以正常运行，则逐步增加DMU数量和变量数量，直至定位触发错误的数据点。

A.3.3 A.3.3 模型构建阶段错误

(一)案例1：NameError——约束规则中的变量名拼写错误

以下案例展示了DEA编程中因循环变量名不一致导致的 `NameError`，以及如何通过阅读错误追踪信息定位问题。

```
from pyomo.environ import *
import pandas as pd
import numpy as np
```

```
import re



def dea_debug_v1(formula, dataframe, rts, orient, eval_query=None,
    ref_query=None):
    """
    DEA效率计算函数(含调试示例的版本)

    Parameters
    ----------
    formula : str
        公式字符串,如 "Y=K L"
    dataframe : pd.DataFrame
        投入产出数据框
    rts : str
        规模报酬: "crs" 或 "vrs"
    orient : str
        导向: "oo"(产出导向)或 "io"(投入导向)
    eval_query : str or None, default None
        筛选待评价DMU的查询条件
    ref_query : str or None, default None
        筛选参照DMU的查询条件

    Returns
    -------
    pd.DataFrame
        包含theta和技术效率值的结果
    """
    theta_results = {}

    if eval_query is None:
        index_list = dataframe.index
    else:
        index_list = dataframe.query(eval_query).index

    if ref_query is None:
        index_list_ref = dataframe.index
    else:
        index_list_ref = dataframe.query(ref_query).index
```

```
# 解析公式字符串
input_vars = formula.split('=')[1].strip(' ')
xcol = re.compile(' +').sub(' ', input_vars).split(' ')
output_vars = formula.split('=')[0].strip(' ')
ycol = re.compile(' +').sub(' ', output_vars).split(' ')

data = dataframe.loc[index_list, xcol + ycol]
dataref = dataframe.loc[index_list_ref, xcol + ycol]
xref = dataref.loc[:, xcol]
yref = dataref.loc[:, ycol]

for j in data.index:
    x = data.loc[j, xcol]
    y = data.loc[j, ycol]

    model = ConcreteModel()
    model.I = Set(initialize=range(dataref.shape[0]))
    model.K = Set(initialize=range(len(xcol)))
    model.L = Set(initialize=range(len(ycol)))

    model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
    model.intensity = Var(model.I, bounds=(0.0, None),
        doc='intensity variables')

    def obj_rule(model):
        return model.theta
    model.obj = Objective(
        rule=obj_rule,
        sense=(maximize if orient == "oo" else minimize),
        doc='objective function'
    )

    def input_rule(model, k):
        """Return the proper input constraint"""
        if orient == "oo":
            # BUG: 循环变量名为i2,但引用时使用i,导致NameError
            return (
                sum(model.intensity[i] * xref.iloc[i, k]
                      for i2 in model.I)
```

```
                <= x.iloc[k]
            )
        elif orient == "io":
            return (
                sum(model.intensity[i] * xref.iloc[i, k]
                       for i in model.I)
                <= model.theta * x.iloc[k]
            )

    def output_rule(model, l):
        """Return the proper output constraint"""
        if orient == "oo":
            return (
                sum(model.intensity[i] * yref.iloc[i, l]
                       for i in model.I)
                >= model.theta * y.iloc[l]
            )
        elif orient == "io":
            return (
                sum(model.intensity[i] * yref.iloc[i, l]
                       for i in model.I)
                >= y.iloc[l]
            )

    def vrs_rule(model):
        """Return the VRS constraint"""
        return sum(model.intensity[i] for i in model.I) == 1

    model.input = Constraint(model.K, rule=input_rule,
        doc='input constraint')
    model.output = Constraint(model.L, rule=output_rule,
        doc='output constraint')

    if rts == "vrs":
        model.vrs = Constraint(rule=vrs_rule, doc='VRS constraint')

    opt = SolverFactory('mosek')
    solution = opt.solve(model)
```

```
        theta = np.asarray(list(model.theta[:].value))
        theta_results[j] = theta[0]

    thetadf = pd.DataFrame(theta_results, index=["theta"]).T
    thetadf["te"] = (1 / thetadf["theta"] if orient == "oo" else thetadf[
        "theta"])
    return thetadf
```

运行上述函数：

```
import pandas as pd
ex3 = pd.read_stata(r"Ex3.dta")
te = dea_debug_v1("Y=K L", ex3, "vrs", "oo", "dmu==[10,11,2,3,4,5,6,18]",
    "dmu<=20")
```

错误信息分析：

```
ERROR: Rule failed when generating expression for Constraint input with index 0:
    NameError: name 'i' is not defined
```

Python会自动追踪代码的执行路径，并按照由浅入深的代码层次报告错误位置：

1. 最顶层：`----> te = dea_debug_v1(...)` 指向第3行函数调用
2. 第二层：`dea_debug_v1`函数内部的 `model.input = Constraint(...)`指向约束构建行
3. 第三层：`input_rule`函数内部的生成器表达式
4. 最底层：`NameError: name 'i' is not defined`

排查方法：

Pyomo由专业团队开发维护，与其本身相关的报错大概率并非Pyomo的问题，排查时应重点关注自己编写的代码。从上述分析可知，`input_rule`函数中 `for i2 in model.I`使用了循环变量 `i2`，而生成器表达式内部引用的却是 `i`，两者不一致，将 `i2`改为 `i`即可修复：

```
# 修复前(错误)
return (
    sum(model.intensity[i] * xref.iloc[i, k]
           for i2 in model.I)
    <= x.iloc[k]
)

# 修复后(正确)
return (
    sum(model.intensity[i] * xref.iloc[i, k]
           for i in model.I)
    <= x.iloc[k]
)
```

(二)案例2：ValueError——约束规则返回None

以下案例展示了因函数调用参数缺失导致约束规则返回 None的错误。

在 dea_debug_v1的调用代码中，如果遗漏了 orient参数：

```
import pandas as pd
ex3 = pd.read_stata(r"Ex3.dta")
# 错误:遗漏了 orient 参数("oo" 或 "io")
te = dea_debug_v1("Y= K L ", ex3, "vrs", "dmu==[10,11,2,3,4,5,6,18]",
    "dmu<=20")
```

此时，字符串"dmu==[10,11,2,3,4,5,6,18]"被传入 orient参数，eval_query则接收了"dmu<=20"。在 input_rule函数中，if orient == "oo"与 elif orient == "io"两个分支均不满足，函数未执行任何 return语句，因而隐式返回 None。

错误信息：

```
ValueError: Constraint 'input[0]': rule returned None.

Constraint rules must return either a valid expression, a 2- or 3-member
tuple, or one of Constraint.Skip, Constraint.Feasible, or
Constraint.Infeasible. The most common cause of this error is forgetting
to include the "return" statement at the end of your rule.
```

排查方法：

1. 从错误信息定位到约束构建行：model.input = Constraint(...)
2. 检查 input_rule函数的所有分支是否都有 return语句
3. 在 input_rule中添加 else分支和调试 print来定位问题：

```
def input_rule(model, k):
    """Return the proper input constraint（带调试版）"""
    print(f"DEBUG: orient='{orient}', k={k}")   # 添加调试输出
    if orient == "oo":
        return (
            sum(model.intensity[i] * xref.iloc[i, k]
                  for i in model.I)
            <= x.iloc[k]
        )
    elif orient == "io":
        return (
            sum(model.intensity[i] * xref.iloc[i, k]
                  for i in model.I)
            <= model.theta * x.iloc[k]
        )
    else:
        raise ValueError(f"orient必须是'oo' 或'io',当前值为: {orient}")
```

通过 print输出可以发现，orient的值并非预期的"oo"或"io"，由此可追溯至函数调用时参数缺失的

问题。

正确的调用方式：

```
te = dea_debug_v1("Y= K L ", ex3, "vrs", "oo",
                  "dmu==[10,11,2,3,4,5,6,18]", "dmu<=20")
```

（三）案例3：使用pprint()检查约束条件

在线性规划问题编程中，约束条件的正确设置至关重要。当不确定约束是否正确时，可以使用Pyomo模型的 `pprint()`方法打印约束条件的具体表达式，检查系数和不等式方向是否符合预期。

```
def build_single_model(
    formula, dataframe, rts, orient, eval_query=None, ref_query=None
):
    """
    构建单个DEA模型（用于pprint检查，仅循环一次后break）

    Parameters
    ----------
    formula, dataframe, rts, orient, eval_query, ref_query
        同 dea_debug_v1

    Returns
    -------
    ConcreteModel
        构建好的Pyomo模型（仅包含第一个DMU的模型）
    """
    if eval_query is None:
        index_list = dataframe.index
    else:
        index_list = dataframe.query(eval_query).index

    if ref_query is None:
        index_list_ref = dataframe.index
    else:
        index_list_ref = dataframe.query(ref_query).index

    input_vars = formula.split('=')[1].strip(' ')
    xcol = re.compile(' +').sub(' ', input_vars).split(' ')
    output_vars = formula.split('=')[0].strip(' ')
    ycol = re.compile(' +').sub(' ', output_vars).split(' ')

    data = dataframe.loc[index_list, xcol + ycol]
```

```
    dataref = dataframe.loc[index_list_ref, xcol + ycol]
    xref = dataref.loc[:, xcol]
    yref = dataref.loc[:, ycol]

    for j in data.index:
        x = data.loc[j, xcol]
        y = data.loc[j, ycol]

        model = ConcreteModel()
        model.I = Set(initialize=range(dataref.shape[0]))
        model.K = Set(initialize=range(len(xcol)))
        model.L = Set(initialize=range(len(ycol)))

        model.theta = Var(within=Reals, bounds=(None, None), doc='efficiency')
        model.intensity = Var(model.I, bounds=(0.0, None),
            doc='intensity variables')

        def obj_rule(model):
            return model.theta
        model.obj = Objective(
            rule=obj_rule,
            sense=(maximize if orient == "oo" else minimize),
            doc='objective function'
        )

        def input_rule(model, k):
            if orient == "oo":
                return (
                    sum(model.intensity[i] * xref.iloc[i, k]
                           for i in model.I)
                    <= x.iloc[k]
                )
            elif orient == "io":
                return (
                    sum(model.intensity[i] * xref.iloc[i, k]
                           for i in model.I)
                    <= model.theta * x.iloc[k]
                )
```

```
        def output_rule(model, l):
            if orient == "oo":
                return (
                    sum(model.intensity[i] * yref.iloc[i, l]
                           for i in model.I)
                    >= model.theta * y.iloc[l]
                )
            elif orient == "io":
                return (
                    sum(model.intensity[i] * yref.iloc[i, l]
                           for i in model.I)
                    >= y.iloc[l]
                )

        def vrs_rule(model):
            return sum(model.intensity[i] for i in model.I) == 1

        model.input = Constraint(model.K, rule=input_rule,
            doc='input constraint')
        model.output = Constraint(model.L, rule=output_rule,
            doc='output constraint')

        if rts == "vrs":
            model.vrs = Constraint(rule=vrs_rule, doc='VRS constraint')

        break    # 仅构建第一个DMU的模型后即退出

    return model
```

使用 pprint()检查模型：

```
import pandas as pd
ex3 = pd.read_stata(r"Ex3.dta")
model = build_single_model("Y=K L", ex3, "vrs", "oo",
                           "dmu==[10,11,2,3,4,5,6,18]", "dmu<=20")
model.pprint()
```

pprint()的输出包含以下几个关键部分：

1. **Set声明：**检查 I、K、L的维度是否正确
2. **Var声明：**检查 theta和 intensity变量的定义域和边界
3. **Objective声明：**检查目标函数的方向（maximize/minimize）和表达式
4. **Constraint声明：**检查每个约束的Lower/Body/Upper值

重点检查项：

- 投入约束的Upper值应为**正数**（产出导向下，投入约束为<=被评价单元的投入值）
- 产出约束的Lower值应包含 `theta`（产出导向下，产出约束为`>= theta *` 被评价单元的产出值）
- VRS约束的Lower和Upper都应为1.0（`sum(intensity) == 1`）

如果投入约束的Upper值为负数，说明公式中的符号可能写反了（如将`<= x[k]`误写为`<= -x[k]`），这将导致线性规划模型错误。

A.3.4 A.3.4 数据相关错误

（一）零值或负值导致求解失败

DEA模型要求投入产出数据严格为正（部分模型允许非期望产出为0）。数据中存在零值或负值会导致线性规划无可行解或结果异常。

```
def validate_data_for_dea(data, input_cols, output_cols,
    undesirable_cols=None):
    """
    验证DEA输入数据的合法性

    Parameters
    ----------
    data : pd.DataFrame
        投入产出数据
    input_cols : list[str]
        投入变量列名列表
    output_cols : list[str]
        产出变量列名列表
    undesirable_cols : list[str] or None, default None
        非期望产出变量列名列表

    Returns
    -------
    bool
        验证通过返回True，否则抛出AssertionError
    """
    assert isinstance(data, pd.DataFrame), "数据必须是pandas DataFrame"
    assert len(data) >= 2, "DEA分析至少需要2个DMU"

    for col in input_cols + output_cols:
        assert col in data.columns, f"变量 '{col}' 不在数据中"

    assert data[input_cols + output_cols].notna().all().all(), \
```

```
        "数据包含缺失值(NaN),请先处理"

    assert np.isfinite(data[input_cols + output_cols].values).all(), \
        "数据包含无穷大(Inf)值,请先处理"

    # 投入必须严格为正
    assert (data[input_cols] > 0).all().all(), \
        f"投入变量存在零值或负值:\n{data[input_cols][(data[input_cols] <= 0).any(axis=1)]}"

    # 期望产出必须严格为正
    assert (data[output_cols] > 0).all().all(), \
        f"产出变量存在零值或负值:\n{data[output_cols][(data[output_cols] <= 0).any(axis=1)]}"

    if undesirable_cols:
        assert (data[undesirable_cols] >= 0).all().all(), \
            "非期望产出不能为负"

    return True
```

(二)列名不匹配导致KeyError

公式字符串中的变量名必须与DataFrame的列名完全一致(包括大小写、空格):

```
# 错误示例:列名大小写不匹配
ex3.columns  # Index(['dmu', 'K', 'L', 'Y'], dtype='object')
te = dea8("y=k l", ex3, "vrs", "oo")   # KeyError: 'y'(应为'Y')

# 解决方法:
# 1. 打印data.columns确认列名
# 2. 使用 data.columns = data.columns.str.strip().str.upper() 统一格式
```

A.3.5 A.3.5 求解阶段错误

(一)求解器不可用

```
from pyomo.environ import SolverFactory

solver = SolverFactory('mosek')
if not solver.available():
    print("MOSEK不可用,尝试使用备选求解器CBC...")
    solver = SolverFactory('cbc')
    if not solver.available():
```

```
        raise RuntimeError("所有求解器均不可用,请安装求解器(参见A.1.3节) ")
```

(二)求解终止条件异常

```
from pyomo.opt import SolverStatus, TerminationCondition


def solve_with_check(model, solver_name='mosek'):
    """
    带状态检查的求解器调用

    Parameters
    ----------
    model : ConcreteModel
        Pyomo模型
    solver_name : str, default 'mosek'
        求解器名称

    Returns
    -------
    result
        求解结果对象

    Raises
    ------
    RuntimeError
        求解器不可用或求解失败时抛出
    """
    opt = SolverFactory(solver_name)

    if not opt.available():
        raise RuntimeError(
            f"求解器 '{solver_name}' 未安装或不可用。"
            f"请运行: conda install -c conda-forge {solver_name}"
        )

    result = opt.solve(model)

    # 检查求解状态
    if result.solver.status != SolverStatus.ok:
```

```
        raise RuntimeError(f"求解器状态异常: {result.solver.status}")

    # 检查终止条件
    tc = result.solver.termination_condition
    if tc != TerminationCondition.optimal:
        if tc == TerminationCondition.infeasible:
            raise RuntimeError(
                "线性规划无可行解——请检查: \n"
                "  1. 数据是否存在零值或负值\n"
                "  2. 投入产出变量方向是否正确\n"
                "  3. 约束条件不等式方向是否正确"
            )
        elif tc == TerminationCondition.unbounded:
            raise RuntimeError(
                "线性规划无界——请检查是否遗漏了关键约束(如投入上界、VRS约束) "
            )
        else:
            raise RuntimeError(f"求解未达到最优。终止条件: {tc}")

    return result
```

A.3.6 A.3.6 结果相关错误

(一)效率值超出[0, 1]范围投入导向的CCR/BCC模型中,技术效率值应位于[0, 1]范围内;产出导向的效率值(te)为1/theta,同样应位于该范围内。若效率值超出此范围,可能的原因包括:

异常现象	可能原因	解决方案
效率值 > 1	导向参数设置错误;产出导向取了theta而非1/theta	检查 `orient`参数;产出导向使用 `te = 1/theta`
效率值 < 0	数据中存在负值;约束方向错误	检查数据;检查不等式方向(`<=` vs `>=`)
所有DMU效率均为1.0	指标过多而DMU过少(维数灾难),或各单元均具独特投入产出组合,样本内无法区分	增加样本量、合并或删减指标;检查参照集是否过小(见A.6.1的 $n < m + s$ 警告)
效率值全为NaN	求解失败未被捕获	添加 `solve_with_check`检查求解状态

注意" 全体效率为1" 不能靠补上VRS约束来纠正:VRS效率不低于CRS效率,加入 $\sum_i \lambda_i = 1$ 只会让**更多**单元落在前沿上。零值或负值更常导致模型无可行解(见A.3.4),而不是全体有效。

(二)松弛变量计算异常

```
# 计算投入冗余(slack)后验证
assert (slack_input >= 0).all(), "投入冗余不能为负——检查投入约束方向"
assert (slack_output >= 0).all(), "产出不足不能为负——检查产出约束方向"
```

A.4 A.4 Jupyter Notebook特有问题

前述错误排查均假定代码运行于确定的执行环境之中;而本书以Jupyter Notebook作为主要编程环境,其交互式特性本身会引入一类独特问题。在Notebook中,单元格的可乱序执行与内核状态的可丢失性,往往使排查方向偏离真实成因。本节针对DEA编程中常见的Notebook问题予以说明。

A.4.1 A.4.1 单元执行顺序混乱

问题描述:Jupyter Notebook的单元格(cell)可以按任意顺序执行。若来回跳转执行单元格,变量状态可能变得混乱——某个变量保存的可能是旧版本代码的计算结果。

典型表现:

```
# 单元格1:定义函数
def calculate_efficiency(x, y):
    return x / y

# 单元格2:修改函数定义(添加了y=0保护)
def calculate_efficiency(x, y):
    if y == 0:
        return 0
    return x / y

# 单元格3:调用函数(如果先执行单元格2再执行单元格1,
#          则单元格1的旧版本会覆盖新版本)
result = calculate_efficiency(10, 0)   # 可能抛出ZeroDivisionError
```

解决方案:

1. **Kernel → Restart & Run All**:当结果异常时,重启内核并从头到尾重新执行所有单元格
2. **养成顺序执行习惯**:尽量从上到下依次执行单元格,避免来回跳转
3. **使用编号检查**:Notebook单元格左侧的 `In [N]`编号可以反映执行顺序。如果编号不连续(如 `In [1]` 后直接 `In [5]`),说明存在跳转执行

A.4.2 A.4.2 内核重启后变量丢失

问题描述:Jupyter Notebook的所有变量存储在内存中。当内核重启(Kernel → Restart)或内核崩溃后,所有已定义的变量和导入的包都会丢失。

对DEA编程的影响:

```
# 前面单元格已运行:
import pandas as pd
ex3 = pd.read_stata(r"Ex3.dta")

# 内核重启后,以下代码会报错:
te = dea8("Y=K L", ex3, "vrs", "oo")
# NameError: name 'pd' is not defined
# NameError: name 'ex3' is not defined
```

最佳实践:

1. **将所有import语句放在Notebook开头的第一个单元格**,确保内核重启后可以快速恢复
2. **将重要结果保存到文件**,不要仅依赖内存中的变量:

```
# 计算完成后保存结果
te.to_csv('efficiency_results.csv', index=True)

# 后续需要时重新加载
te = pd.read_csv('efficiency_results.csv', index_col=0)
```

3. **使用%who魔术命令查看当前命名空间中的变量:**

```
%who          # 列出所有变量
%whos         # 列出变量及其类型和值
```

A.4.3 A.4.3 大数据集的内存问题

问题描述:处理大规模DEA问题(如>1000个DMU)时，Jupyter Notebook可能因为内存不足而卡顿甚至崩溃。

症状: - 单元格执行时间异常长 - Notebook无响应(单元格左侧显示*长时间不消失) - 浏览器页面崩溃- 出现 `MemoryError`异常

解决方案:

1. **使用.py脚本替代Notebook:**对于大规模计算,将代码保存为.py文件并在命令行运行:

```
python dea_analysis.py
```

2. **及时释放不再需要的变量:**

```
# 计算完成后释放中间变量
del model, xref, yref
import gc
gc.collect()   # 强制垃圾回收
```

3. **分批次处理:**将大规模DEA问题分批求解(详见A.5节)
4. **使用内存分析工具:**

```
# 查看变量内存占用
import sys
for name, obj in sorted(
    locals().items(), key=lambda x: sys.getsizeof(x[1]), reverse=True
)[:10]:
    print(f"{name}: {sys.getsizeof(obj) / 1024**2:.2f} MB")
```

A.4.4 A.4.4 图形不显示

问题描述：使用 matplotlib绘制图形时，图形未在Notebook中内联显示。

解决方案：

```
# 在Notebook开头添加魔术命令（推荐放在import之后）
%matplotlib inline

# 如果需要更高清的图像
%config InlineBackend.figure_format = 'retina'

import matplotlib.pyplot as plt

# 现在绘图会直接在Notebook中显示
plt.hist(te['te'], bins=20, edgecolor='black')
plt.xlabel('Technical Efficiency')
plt.ylabel('Frequency')
plt.title('Distribution of DEA Efficiency Scores')
plt.show()
```

注意事项：- %matplotlib inline必须在任何 plt调用之前执行 - 如果使用了%matplotlib notebook（交互式模式），每次只能有一个图形处于激活状态- 对于静态出版质量图像，始终使用%matplotlib inline

---

A.5 A.5 性能优化与大规模问题处理

当模型能够稳定求解后，研究对象规模往往成为新的瓶颈。DEA的计算量随决策单元数量近似线性累积，因此大规模实证分析对求解效率与内存管理提出了更高要求。本节围绕计算复杂度、求解器选择、并行与分批策略以及内存优化展开讨论，以应对DMU数量庞大的情形。

A.5.1 A.5.1 DEA计算复杂度分析

DEA方法的计算复杂度随决策单元（DMU）数量呈多项式增长，这是DEA编程中最主要的性能瓶颈：

计算量分析：

- 对于 n 个DMU，传统DEA需要求解 n 个独立的线性规划（LP）问题
- 每个LP的变量数约为 n（强度变量 λ_i）加上效率变量数
- 每个LP的约束数约为投入变量数 + 产出变量数 + RTS约束数

DMU数量	需要求解的LP数量	每个LP的变量数	预估总求解时间（GLPK）
100	100	~100	~6秒
500	500	~500	~35秒
1,000	1,000	~1,000	~2分钟
5,000	5,000	~5,000	~25分钟
10,000	10,000	~10,000	~1.5小时

实际影响因素：求解时间还受投入产出变量数量、RTS设定（VRS比CRS多一个约束）、求解器选择等因素影响。上表为产出导向VRS模型（3个投入、1个期望产出）在pyomo + glpk 4.65 环境下的实测结果：每个规模求解若干个LP（n=100为全量100个，更大规模抽样5—50个）取平均后按LP数量外推，含pyomo建模开销；绝对数值随硬件与数据结构而异，量级供参考。

A.5.2 A.5.2 提升求解速度的实用技巧

（一）使用更快的求解器求解器选择对DEA计算速度的影响最大，推荐优先级如下：

1. **MOSEK**（最快，需学术许可）：求解速度约为GLPK的6-8倍
2. **Gurobi**（快，需学术许可）：与MOSEK性能接近
3. **CBC**（免费，较快）：比GLPK稳定，推荐作为免费首选
4. **GLPK**（免费，最慢）：适合小规模问题和教学演示

```
from pyomo.environ import SolverFactory

# 尝试使用最快的可用求解器
for solver_name in ['mosek', 'gurobi', 'cbc', 'glpk']:
    solver = SolverFactory(solver_name)
    if solver.available():
        print(f"使用求解器: {solver_name}")
        break
else:
    raise RuntimeError("没有可用的求解器")
```

（二）并行计算由于每个DMU的DEA模型是独立求解的，可以使用并行计算同时求解多个DMU。以下示例使用 `joblib`库实现并行DEA计算：

```
from joblib import Parallel, delayed
from pyomo.environ import *
import pandas as pd
```

```
import numpy as np



def solve_single_dmu(dmu_idx, data, xcol, ycol, orient='oo', rts='crs'):
    """
    求解单个DMU的DEA模型

    Parameters
    ----------
    dmu_idx : int
        当前评价DMU的索引
    data : pd.DataFrame
        全部DMU的投入产出数据
    xcol : list[str]
        投入变量列名
    ycol : list[str]
        产出变量列名
    orient : str, default 'oo'
        导向类型: "oo"（产出导向)或 "io"（投入导向）
    rts : str, default 'crs'
        规模报酬

    Returns
    -------
    tuple
        (dmu_idx, theta_value)
    """
    x = data.loc[dmu_idx, xcol]
    y = data.loc[dmu_idx, ycol]
    xref = data.loc[:, xcol]
    yref = data.loc[:, ycol]

    if orient not in ("oo", "io"):
        raise ValueError(f"orient必须是'oo' 或'io',当前值为: {orient}")

    model = ConcreteModel()
    model.I = Set(initialize=data.index)
    model.K = Set(initialize=range(len(xcol)))
    model.L = Set(initialize=range(len(ycol)))
```

```
    # theta 非负:产出导向下为产出扩张倍数(>=1),投入导向下为投入收缩比((0,1])
    model.theta = Var(bounds=(0, None))
    model.intensity = Var(model.I, bounds=(0, None))

    model.obj = Objective(expr=model.theta,
                          sense=(maximize if orient == 'oo' else minimize))

    # 投入约束:产出导向下右端为 x_0,投入导向下右端为 theta * x_0
    def input_rule(model, k):
        lhs = sum(model.intensity[i] * xref.loc[i, xcol[k]] for i in model.I)
        if orient == "oo":
            return lhs <= x.loc[xcol[k]]
        else:
            return lhs <= model.theta * x.loc[xcol[k]]
    model.input_con = Constraint(model.K, rule=input_rule)

    # 产出约束:产出导向下右端为 theta * y_0,投入导向下右端为 y_0
    def output_rule(model, l):
        lhs = sum(model.intensity[i] * yref.loc[i, ycol[l]] for i in model.I)
        if orient == "oo":
            return lhs >= model.theta * y.loc[ycol[l]]
        else:
            return lhs >= y.loc[ycol[l]]
    model.output_con = Constraint(model.L, rule=output_rule)

    if rts == 'vrs':
        model.vrs = Constraint(
            expr=sum(model.intensity[i] for i in model.I) == 1)

    solver = SolverFactory('mosek')
    solver.solve(model, tee=False)

    theta = value(model.theta)
    return (dmu_idx, theta)


def parallel_dea(data, xcol, ycol, orient='oo', rts='crs', n_jobs=-1):
    """
    并行计算DEA效率
```

```
    Parameters
    ----------
    data : pd.DataFrame
        投入产出数据
    xcol : list[str]
        投入变量列名
    ycol : list[str]
        产出变量列名
    orient : str, default 'oo'
        导向类型
    rts : str, default 'crs'
        规模报酬
    n_jobs : int, default -1
        并行进程数,-1表示使用所有CPU核心

    Returns
    -------
    pd.DataFrame
        包含theta值和技术效率值的结果
    """
    results = Parallel(n_jobs=n_jobs, backend='loky')(
        delayed(solve_single_dmu)(idx, data, xcol, ycol, orient, rts)
        for idx in data.index
    )

    theta_results = {idx: theta for idx, theta in results}
    thetadf = pd.DataFrame(theta_results, index=["theta"]).T
    thetadf["te"] = (1 / thetadf["theta"] if orient == "oo" else thetadf[
        "theta"])
    return thetadf
```

使用示例：

```
# 使用4个核心并行计算(假设data已定义)
tresults = parallel_dea(data, ['K', 'L'], ['Y'], orient='oo', rts='vrs',
    n_jobs=4)
```

注意事项：当DMU数量较少时，并行计算的开销可能得不偿失，建议在DMU数量大于100时再考虑并行化。`n_jobs`参数建议不超过CPU核心数。

导向切换不只是改 max/min：投入导向的模型是 $\min \theta$、$\sum_i \lambda_i X_{ik} \le \theta x_{0k}$、$\sum_i \lambda_i Y_{il} \ge y_{0l}$，$\theta$ 乘在**投入**一侧；产出导向则是 $\max \theta$，θ 乘在**产出**一侧。若只把目标函数从 `maximize` 改成

`minimize` 而让 θ 留在产出约束上，得到的既不是Farrell投入效率，在 θ 无下界时还会使LP无界（$\theta \to -\infty$）。上面的 `input_rule` / `output_rule` 按 `orient` 切换 θ 所在侧，与A.3.3和第11章的 `dea8` 一致。

（三）减少不必要的约束如果确定使用CRS模型，不要添加VRS的凸性约束，可以减少每个LP的约束行数：

```
# 每个LP的技术约束行数（与A.5.1一致）：
#   CRS模型：投入变量数 P + 产出变量数 Q
#   VRS模型：P + Q + 1（多一条凸性约束 sum(lambda_i) == 1）
# 强度变量的 lambda_i >= 0 是变量边界，不计入约束行。
# 规模差异主要来自可行域：VRS多一条等式，可行域更小但需处理额外的等式约束，
# 因此大规模问题上CRS通常快于VRS。
```

（四）分解大问题对于超大规模DEA问题（如>5000 DMU），可按以下策略分解：

```
def batch_dea(data, xcol, ycol, batch_size=500, orient='oo', rts='crs'):
    """
    分批计算DEA效率，适用于超大规模问题

    Parameters
    ----------
    data : pd.DataFrame
        全部DMU的投入产出数据
    xcol : list[str]
        投入变量列名
    ycol : list[str]
        产出变量列名
    batch_size : int, default 500
        每批处理的DMU数量
    orient, rts : str
        导向和规模报酬设置

    Returns
    -------
    pd.DataFrame
        合并后的效率结果
    """
    all_results = []
    n_batches = (len(data) + batch_size - 1) // batch_size

    for i in range(n_batches):
        start_idx = i * batch_size
```

```
        end_idx = min((i + 1) * batch_size, len(data))
        batch_data = data.iloc[start_idx:end_idx]

        print(f"处理第 {i+1}/{n_batches} 批 (DMU {start_idx}-{end_idx-1})")

        batch_result = parallel_dea(batch_data, xcol, ycol, orient, rts,
            n_jobs=4)
        all_results.append(batch_result)

    return pd.concat(all_results)
```

A.5.3 A.5.3 内存优化建议

（一）使用生成器代替列表

```
# 不推荐:列表推导式一次性存储所有中间结果
all_models = [build_model(data, i) for i in range(n)]  # 占用大量内存

# 推荐:使用生成器逐个产生模型,逐个求解
def model_generator(data):
    for i in data.index:
        yield build_model(data, i)

for model in model_generator(data):
    solve_model(model)
    # 模型在使用后自动释放
```

（二）使用更高效的数据类型

```
# 对于大规模数据,使用float32代替float64可节省一半内存
data_float32 = data.astype('float32')

# 如果数据值范围有限,可使用更紧凑的类型
data_int16 = data.astype('int16')   # 仅适用于整数数据
```

A.5.4 A.5.4 性能优化决策流程

```
DEA程序运行太慢?
|
|-- DMU数量 < 100?
|   |-- 更换求解器（GLPK -> MOSEK/Gurobi）
|   |-- 足够快，无需进一步优化
```

```
|
|-- DMU数量 100-1000?
|   |-- 使用MOSEK/Gurobi求解器
|   |-- 考虑并行计算（joblib）
|
|-- DMU数量 > 1000?
|   |-- 使用MOSEK求解器 + 并行计算
|   |-- 分批处理 + 保存中间结果
|   |-- 考虑使用专门的DEA软件包（如pyStoNED、pyfrontier）
|
|-- 仍然太慢?
    |-- 检查是否可以减少投入产出变量数量
    |-- 检查是否可以使用CRS代替VRS
    |-- 考虑使用参数化/非参数化替代方法
```

A.6 A.6 防御式编程与单元测试

前述各节侧重于错误发生后的排查，而更稳健的取向是在代码编写阶段即主动预防。编写" 能运行" 的代码只是第一步，编写" 能正确运行" 的代码才是目标。在DEA建模中，后一类问题尤为隐蔽：由于模型本身缺乏显式报错机制，参数误置或约束方向颠倒常表现为" 静默错误"——代码正常结束，计算结果却已失真。本节介绍**防御式编程**（Defensive Programming）与**单元测试**（Unit Testing）两项技能，以帮助读者在事前消弭此类错误。

A.6.1 A.6.1 输入验证函数

在DEA函数的开头添加输入验证，可以在问题发生的最早阶段捕获错误，避免将错误数据传播到复杂的模型求解流程中。

```python
import pandas as pd
import numpy as np


def validate_dea_inputs(data, input_cols, output_cols, undesirable_cols=None,
                        orient='oo', rts='crs'):
    """
    验证DEA输入数据的完整性和合法性

    在调用DEA模型之前运行此验证函数，可提前发现数据问题，
    避免在Pyomo建模阶段出现难以理解的错误。

    Parameters
```

```
    ----------
    data : pd.DataFrame
        包含投入产出数据的DataFrame
    input_cols : list[str]
        投入变量列名列表
    output_cols : list[str]
        产出变量列名列表
    undesirable_cols : list[str] or None, default None
        非期望产出变量列名列表
    orient : str, default 'oo'
        导向类型: 'oo'（产出导向)或 'io'（投入导向）
    rts : str, default 'crs'
        规模报酬: 'crs' 或 'vrs'

    Returns
    -------
    bool
        验证通过返回True

    Raises
    ------
    TypeError
        数据类型不正确
    ValueError
        数据内容不合法(缺失值、负值等）
    KeyError
        指定的列名不存在于数据中
    """
    # 类型检查
    if not isinstance(data, pd.DataFrame):
        raise TypeError(f"数据必须是pandas DataFrame,当前类型: {type(data)}")

    if not isinstance(input_cols, (list, tuple)):
        raise TypeError("input_cols必须是列表或元组")

    if not isinstance(output_cols, (list, tuple)):
        raise TypeError("output_cols必须是列表或元组")

    # 参数值检查
```

```
    if orient not in ('oo', 'io'):
        raise ValueError(f"orient必须是'oo'或'io',当前值: {orient}")

    if rts not in ('crs', 'vrs'):
        raise ValueError(f"rts必须是'crs'或'vrs',当前值: {rts}")

    # 列名存在性检查
    all_cols = list(input_cols) + list(output_cols)
    if undesirable_cols:
        all_cols.extend(undesirable_cols)

    missing_cols = [col for col in all_cols if col not in data.columns]
    if missing_cols:
        raise KeyError(
            f"以下列名不存在于数据中: {missing_cols}\n"
            f"数据中的列名: {list(data.columns)}"
        )

    # 样本量检查
    if len(data) < 2:
        raise ValueError(f"DEA分析至少需要2个DMU,当前数据只有{len(data)}个")

    # DMU数量与变量数量的合理性检查
    n_dmu = len(data)
    n_inputs = len(input_cols)
    n_outputs = len(output_cols)
    if n_dmu < n_inputs + n_outputs:
        import warnings
        warnings.warn(
            f"DMU数量({n_dmu})少于投入产出变量总数({n_inputs + n_outputs}), "
            f"可能导致大量DMU位于效率前沿上。",
            UserWarning
        )

    # 缺失值检查
    check_cols = input_cols + output_cols
    if data[check_cols].isna().any().any():
        na_rows = data[check_cols][data[check_cols].isna().any(axis=1)]
        raise ValueError(f"数据包含缺失值(NaN),请处理:\n{na_rows}")
```

```
    # 无穷大值检查
    if not np.isfinite(data[check_cols].values).all():
        raise ValueError("数据包含无穷大(Inf)值,请处理")

    # 零值和负值检查
    invalid_input = data[input_cols] <= 0
    if invalid_input.any().any():
        bad_rows = data[input_cols][invalid_input.any(axis=1)]
        raise ValueError(
            f"投入变量存在零值或负值(DEA要求投入严格为正):\n{bad_rows}"
        )

    invalid_output = data[output_cols] <= 0
    if invalid_output.any().any():
        bad_rows = data[output_cols][invalid_output.any(axis=1)]
        raise ValueError(
            f"期望产出变量存在零值或负值(DEA要求期望产出严格为正):\n{bad_rows}"
        )

    if undesirable_cols:
        if (data[undesirable_cols] < 0).any().any():
            raise ValueError("非期望产出变量不能为负")

    return True
```

使用示例:

```
# 在DEA函数开头调用验证
def my_dea_model(data, input_cols, output_cols, orient='oo', rts='crs'):
    # 防御式编程:先验证输入
    validate_dea_inputs(data, input_cols, output_cols, orient=orient, rts=rts)

    # 继续构建模型...
    model = ConcreteModel()
    # ...
```

A.6.2 A.6.2 断言(Assertion)的使用

断言是验证程序中间状态的利器。在关键步骤插入 assert语句,可以在错误发生时立即停止执行并给出明确提示。

```
def extract_results(model, orient='oo'):
    """提取DEA求解结果并验证合理性"""
    theta = value(model.theta)

    # 断言：theta必须是有限数值
    assert np.isfinite(theta), f"theta值为非有限数：{theta}"

    # 断言：theta的范围应合理（通常theta > 0）
    assert theta > 0, f"theta值应为正，当前：{theta}"

    # 计算技术效率
    te = 1 / theta if orient == 'oo' else theta

    # 断言：技术效率应在(0, 1]范围内
    assert 0 < te <= 1, f"技术效率 {te} 超出合理范围(0, 1]"

    return te
```

注意事项：`assert`语句在Python以优化模式运行（`python -O`）时会被忽略。对于生产环境的关键检查，建议使用 `if ...: raise ValueError(...)`代替 `assert`。

A.6.3 A.6.3 单元测试入门

单元测试是对代码的最小可测试单元（如函数、方法）进行验证的自动化测试。**pytest** 是Python最流行的单元测试框架，使用简单但功能强大。

（一）安装pytest

```
conda install pytest
# 或
pip install pytest
```

（二）DEA函数的单元测试示例

```
"""
test_dea_functions.py
DEA函数的单元测试
运行命令：pytest test_dea_functions.py -v
"""
import pytest
import pandas as pd
import numpy as np
from pyomo.environ import *
```

```python
# ---------- 测试夹具(Fixture):共享的测试数据 ----------

@pytest.fixture
def sample_data():
    """提供标准测试数据集(5个DMU, 2投入, 1产出)"""
    return pd.DataFrame({
        'dmu': ['A', 'B', 'C', 'D', 'E'],
        'x1': [5.0, 5.0, 8.0, 34.0, 9.0],
        'x2': [6.0, 9.0, 6.0, 67.0, 6.0],
        'y': [23.0, 14.0, 45.0, 11.0, 31.0]
    })


@pytest.fixture
def tiny_data():
    """提供最小测试数据集(2个DMU, 1投入, 1产出)"""
    return pd.DataFrame({
        'dmu': [1, 2],
        'x': [1.0, 2.0],
        'y': [1.0, 1.0]
    })


# ---------- 输入验证测试 ----------

class TestInputValidation:
    """测试输入验证功能"""

    def test_empty_dataframe(self):
        """空DataFrame应抛出ValueError"""
        empty_df = pd.DataFrame()
        with pytest.raises((ValueError, KeyError)):
            validate_dea_inputs(empty_df, ['x'], ['y'])

    def test_single_dmu(self):
        """只有一个DMU时应抛出ValueError"""
        single = pd.DataFrame({'x': [1.0], 'y': [1.0]})
        with pytest.raises(ValueError, match="至少"):
            validate_dea_inputs(single, ['x'], ['y'])
```

```
    def test_missing_column(self):
        """列名不存在时应抛出KeyError"""
        data = pd.DataFrame({'x1': [1.0, 2.0], 'y1': [1.0, 1.0]})
        with pytest.raises(KeyError):
            validate_dea_inputs(data, ['x'], ['y'])  # x和y都不存在

    def test_negative_input(self):
        """投入为负时应抛出ValueError"""
        data = pd.DataFrame({'x': [1.0, -2.0], 'y': [1.0, 1.0]})
        with pytest.raises(ValueError, match="零值或负值"):
            validate_dea_inputs(data, ['x'], ['y'])

    def test_zero_input(self):
        """投入为零时应抛出ValueError"""
        data = pd.DataFrame({'x': [1.0, 0.0], 'y': [1.0, 1.0]})
        with pytest.raises(ValueError, match="零值或负值"):
            validate_dea_inputs(data, ['x'], ['y'])

    def test_nan_value(self):
        """数据含NaN时应抛出ValueError"""
        data = pd.DataFrame({'x': [1.0, np.nan], 'y': [1.0, 1.0]})
        with pytest.raises(ValueError, match="缺失值"):
            validate_dea_inputs(data, ['x'], ['y'])

    def test_invalid_orient(self):
        """orient参数无效时应抛出ValueError"""
        data = pd.DataFrame({'x': [1.0, 2.0], 'y': [1.0, 1.0]})
        with pytest.raises(ValueError, match="orient"):
            validate_dea_inputs(data, ['x'], ['y'], orient='invalid')

    def test_valid_data(self, tiny_data):
        """合法数据应返回True"""
        result = validate_dea_inputs(tiny_data, ['x'], ['y'])
        assert result is True


# ---------- DEA模型测试 ----------

class TestDEAModel:
```

```
    """测试DEA模型的计算结果"""

    def test_efficiency_range(self, tiny_data):
        """效率值应在(0, 1]范围内"""
        # 使用最小数据集计算效率
        result = simple_dea_solve(tiny_data, ['x'], ['y'], orient='oo',
            rts='crs')
        efficiencies = result['te'].values

        assert all(0 < e <= 1 for e in efficiencies), \
            f"效率值超出范围: {efficiencies}"

    def test_efficient_dmu(self, tiny_data):
        """在最小数据集中,投入最少的DMU效率应为1"""
        result = simple_dea_solve(tiny_data, ['x'], ['y'], orient='oo',
            rts='crs')
        # DMU 1的投入x=1最少,应为效率前沿
        assert result.loc[0, 'te'] == 1.0, "投入最少的DMU效率应为1"

    def test_crs_vs_vrs(self, sample_data):
        """VRS效率不应低于CRS效率"""
        crs_result = simple_dea_solve(sample_data, ['x1', 'x2'], ['y'],
                                      orient='oo', rts='crs')
        vrs_result = simple_dea_solve(sample_data, ['x1', 'x2'], ['y'],
                                      orient='oo', rts='vrs')

        for idx in sample_data.index:
            assert vrs_result.loc[idx, 'te'] >= crs_result.loc[idx, 'te'], \
                f"DMU {idx}: VRS效率({vrs_result.loc[idx, 'te']}) " \
                f"不应低于CRS效率({crs_result.loc[idx, 'te']})"

    def test_io_vs_oo(self, sample_data):
        """投入导向和产出导向: CRS下两者的技术效率应相等"""
        oo_result = simple_dea_solve(sample_data, ['x1', 'x2'], ['y'],
                                     orient='oo', rts='crs')
        io_result = simple_dea_solve(sample_data, ['x1', 'x2'], ['y'],
                                     orient='io', rts='crs')

        # 两种导向的效率都应在(0, 1]范围内
```

```
        assert all(0 < e <= 1 for e in oo_result['te']), "产出导向效率超出范围"
        assert all(0 < e <= 1 for e in io_result['te']), "投入导向效率超出范围"

        # CRS下径向CCR的投入导向与产出导向效率相等(1/theta_oo = theta_io);
        # 若投入导向的 theta 仍乘在产出约束上,这一断言会失败——它正是用来
        # 检出"只改了max/min、没改theta所在侧"的错误。
        for idx in sample_data.index:
            assert abs(io_result.loc[idx, 'te']
                       - oo_result.loc[idx, 'te']) < 1e-6, \
                f"DMU {idx}: CRS下两种导向的技术效率应相等"


# ---------- 辅助函数:最小DEA求解 ----------

def simple_dea_solve(data, xcol, ycol, orient='oo', rts='crs'):
    """
    最小化的DEA求解函数(仅用于测试)

    Parameters
    ----------
    data : pd.DataFrame
        投入产出数据
    xcol : list[str]
        投入变量列名
    ycol : list[str]
        产出变量列名
    orient : str
        导向
    rts : str
        规模报酬

    Returns
    -------
    pd.DataFrame
        包含te列的结果DataFrame
    """
    validate_dea_inputs(data, xcol, ycol, orient=orient, rts=rts)

    theta_results = {}
```

```
for j in data.index:
    x = data.loc[j, xcol]
    y = data.loc[j, ycol]
    xref = data.loc[:, xcol]
    yref = data.loc[:, ycol]

    model = ConcreteModel()
    model.I = Set(initialize=data.index)
    model.K = Set(initialize=range(len(xcol)))
    model.L = Set(initialize=range(len(ycol)))

    model.theta = Var(bounds=(0, None))
    model.intensity = Var(model.I, bounds=(0, None))

    model.obj = Objective(
        expr=model.theta,
        sense=(maximize if orient == 'oo' else minimize)
    )

    # 投入约束:投入导向时右端乘 theta
    def input_rule(model, k):
        lhs = sum(model.intensity[i] * xref.loc[i, xcol[k]]
                  for i in model.I)
        if orient == "oo":
            return lhs <= x.iloc[k]
        else:
            return lhs <= model.theta * x.iloc[k]
    model.input_con = Constraint(model.K, rule=input_rule)

    # 产出约束:产出导向时右端乘 theta
    def output_rule(model, l):
        lhs = sum(model.intensity[i] * yref.loc[i, ycol[l]]
                  for i in model.I)
        if orient == "oo":
            return lhs >= model.theta * y.iloc[l]
        else:
            return lhs >= y.iloc[l]
    model.output_con = Constraint(model.L, rule=output_rule)
```

```
        if rts == 'vrs':
            model.vrs = Constraint(
                expr=sum(model.intensity[i] for i in model.I) == 1)

        solver = SolverFactory('glpk')
        solver.solve(model)

        theta_results[j] = value(model.theta)

    thetadf = pd.DataFrame(theta_results, index=["theta"]).T
    thetadf["te"] = (1 / thetadf["theta"] if orient == "oo" else thetadf[
        "theta"])
    return thetadf
```

运行测试：

```
# 运行所有测试并显示详细信息
pytest test_dea_functions.py -v

# 只运行特定测试类
pytest test_dea_functions.py::TestInputValidation -v

# 只运行包含特定关键词的测试
pytest test_dea_functions.py -v -k "efficiency"
```

教学建议：读者在修改DEA函数后，应首先运行测试套件确认修改没有破坏现有功能。这是专业软件开发的标准实践，也是确保研究结果可靠性的重要保障。

A.7 A.7 环境可复现性与版本管理

A.7.1 A.7.1 为什么需要版本管理

调试与优化之余，研究结论的可复现性还取决于计算环境的长期一致性。Python包版本更新频繁，API可能在不同版本间发生变化。读者日后运行本书代码时，可能因包版本不兼容而导致代码失效。因此，记录并固定开发环境的版本信息，是保证他人（以及未来的自己）能够重现已得结果的关键环节。

A.7.2 A.7.2 导出环境配置

使用conda导出：

```
# 导出当前环境的精确配置（包含所有包及其版本）
conda env export > environment.yml
```

```
# 他人可使用此文件重建完全相同的环境
conda env create -f environment.yml
```

使用pip导出：

```
# 只导出通过pip安装的包
pip freeze > requirements.txt

# 重建环境
pip install -r requirements.txt
```

A.7.3 A.7.3 本书推荐的environment.yml

以下配置文件包含了运行本书DEA代码所需的所有依赖：

```
# environment.yml
# 本书DEA分析推荐环境配置
# 使用方法: conda env create -f environment.yml

name: dea-book
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - numpy=1.24
  - pandas=2.0
  - scipy=1.10
  - matplotlib=3.7
  - jupyter
  - pytest
  - joblib
  - glpk
  - coincbc
  - pip
  - pip:
    - pyomo==6.6
```

将此文件保存为 `environment.yml`，然后运行：

```
conda env create -f environment.yml
conda activate dea-book
```

A.7.4 A.7.4 在代码中记录版本信息

在研究代码的开头记录关键包的版本，有助于日后排查因版本变化导致的问题：

```
"""
DEA效率分析脚本
运行环境：Python 3.11, pyomo 6.6, pandas 2.0
"""
import sys
import numpy as np
import pandas as pd
import pyomo

print(f"Python: {sys.version}")
print(f"numpy: {np.__version__}")
print(f"pandas: {pd.__version__}")
print(f"pyomo: {pyomo.__version__}")
```

A.8 A.8 获取帮助的渠道

尽管本附录力图覆盖DEA编程中的主要故障类型，仍不免有未尽之处。当遇到本书未涵盖的问题时，借助外部社区与学术资源往往是更高效的解决路径。以下渠道可作为获取帮助的优先选择：

A.8.1 A.8.1 在线社区与资源

下列在线资源按适用场景分类，便于读者快速定位：

资源	网址/关键词	适用问题
Pyomo官方文档	https://pyomo.readthedocs.io/	Pyomo建模语法、API参考
Pyomo论坛	Stack Overflow标签 `[pyomo]`	Pyomo具体编程问题
DEA学术社区	Stack Overflow标签 `[data-envelopment-analysis]`	DEA方法论问题
Pyomo GitHub Issues	https://github.com/Pyomo/pyomo/issues	Pyomo的Bug报告和功能请求
Pandas文档	https://pandas.pydata.org/docs/	数据处理相关问题
Python官方文档	https://docs.python.org/zh-cn/3/	Python语言问题
Google Scholar	scholar.google.com	DEA学术论文和方法查询

A.8.2 A.8.2 提问的最佳实践

向社区提问时，提供清晰、完整的信息可以显著提高获得帮助的概率。一个好的问题应包含以下要素：

1. 最小可复现代码（MRE）

将问题简化为最短的可运行代码（参见A.3.2节）。去除与问题无关的数据和代码。

2. 完整的错误回溯（Traceback）

不要只提供最后一行错误信息，而应提供完整的Traceback：

```
# 好的示例（完整Traceback）
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[5], line 2
      1 model = build_model(data)
----> 2 result = solver.solve(model)
...
ValueError: Constraint 'input[0]': rule returned None.

# 不好的示例（只提供最后一行）
"我的代码报错了：rule returned None"
```

3. 环境信息

在提问时附上环境信息：

```
import sys, platform
import numpy, pandas, pyomo

print(f"操作系统: {platform.system()} {platform.release()}")
print(f"Python: {sys.version}")
print(f"numpy: {numpy.__version__}")
print(f"pandas: {pandas.__version__}")
print(f"pyomo: {pyomo.__version__}")
```

4. 已尝试的解决方案

说明为解决该问题已尝试过哪些方法及其结果，以避免他人重复建议已验证无效的方案。

A.8.3 A.8.3 学术资源

对于DEA方法论层面的问题，以下学术资源可能有所帮助：

- **DEA国际会议**：European Workshop on Efficiency and Productivity Analysis (EWEPA)、Asia-Pacific Productivity Conference (APPC)等
- **主要期刊**：*European Journal of Operational Research*、*Journal of Productivity Analysis*、*Omega*、*Annals of Operations Research*
- **经典教材**：Coelli et al. (2005) *An Introduction to Efficiency and Productivity Analysis*; Cooper et al. (2007) *Data Envelopment Analysis*

A.9 附录末:运行DEA程序前的检查清单

为便于将上述方法付诸实践,以下清单将本附录的关键检查点汇总为可操作的通行步骤。在运行DEA分析之前,逐项核对以下内容,可以避免大部分常见错误:

- ☐ **Python环境已激活**:`conda activate dea`
- ☐ **所需包已安装且可导入**:运行 `import pyomo, pandas, numpy`无错误
- ☐ **求解器可用**:运行A.1.4节的验证脚本,至少有一个求解器显示" 可用"
- ☐ **数据文件路径正确**:使用 `Path.exists()`确认文件存在
- ☐ **数据无缺失值**:使用 `data.isna().sum()`检查
- ☐ **数据无零值/负值**:投入产出数据严格为正
- ☐ **变量名与公式字符串一致**:检查大小写和空格
- ☐ **模型导向(orient)设定正确**:`"oo"`表示产出导向,`"io"`表示投入导向
- ☐ **规模报酬(rts)设定正确**:`"crs"`表示规模报酬不变,`"vrs"`表示规模报酬可变
- ☐ **Jupyter单元按顺序执行**:Kernel → Restart & Run All 无报错
- ☐ **保存了重要结果**:关键结果已导出为CSV或Excel文件

A.10 术语表

为降低阅读门槛,以下汇总本附录及全书DEA编程语境中频繁出现的术语:

英文术语	中文翻译	说明
traceback	回溯/堆栈跟踪	Python错误发生时的调用链信息
debugger	调试器	用于逐步执行代码、检查变量状态的工具
constraint rule	约束规则	Pyomo中定义约束的函数
solver	求解器	求解优化问题的软件(如GLPK、MOSEK)
intensity variable	强度变量	DEA模型中的 λ_i 变量
feasible solution	可行解	满足所有约束条件的解
optimal solution	最优解	使目标函数最优的可行解
termination condition	终止条件	求解器结束求解时的状态
Minimum Reproducible Example (MRE)	最小可复现示例	简化到能复现问题的最少代码和数据

英文术语	中文翻译	说明
defensive programming	防御式编程	在代码中添加验证和异常处理的编程风格
unit test	单元测试	对代码最小单元进行自动化验证
snake_case	蛇形命名法	小写下划线分隔的命名方式（如 `eval_query`）